

Certified Kubernetes Administrator (CKA)

Complete Official KodeKloud Documentation &
Study Guide

Compiled for Mickey (Hailemichael Tehka Meresa)

Section: Introduction

Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

Course Introduction

This article introduces a comprehensive Kubernetes course designed to help learners achieve certification through hands-on labs and real-world exercises.

Kubernetes is one of the most in-demand technologies in cloud computing, and job listings seeking Kubernetes skills have surged over the last year. Whether you're aiming to advance your career or lead production-grade deployments, this course will equip you with the knowledge and confidence to pass the Certified Kubernetes Administrator (CKA) exam.

Hello, I'm Mumshad Mannambeth. With over a decade of production experience and thousands of students trained worldwide, I'll guide you through everything you need to know—step by step.

What You'll Learn

- **Core Kubernetes Concepts:** Pods, Services, Deployments, ConfigMaps, Secrets
- **Cluster Architecture & Setup:** From single-node to highly available clusters
- **Hands-On Labs:** Real Kubernetes clusters you manage directly in your browser
- **Troubleshooting & Best Practices:** Logs, monitoring, scaling, security
- **Exam-Ready Skills:** Focused practice on tasks you'll see in the CKA exam

Course Highlights

- **Interactive Browser Labs:** No local install required—start practicing in minutes.
- **Real-World Scenarios:** Apply concepts to tasks drawn from actual production environments.
- **Community & Support:** Join our Slack workspace for Q\&A, peer learning, and live office hours.
- **Certified Training Partner:** Endorsed by the Cloud Native Computing Foundation (CNCF).
- **Ongoing Updates:** Content refreshed quarterly to mirror the latest Kubernetes releases.
- **Risk-Free Enrollment:** 30-day money-back guarantee—no questions asked.

Ready to become a Kubernetes expert? Enroll now and take the first step toward your CKA certification!

Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

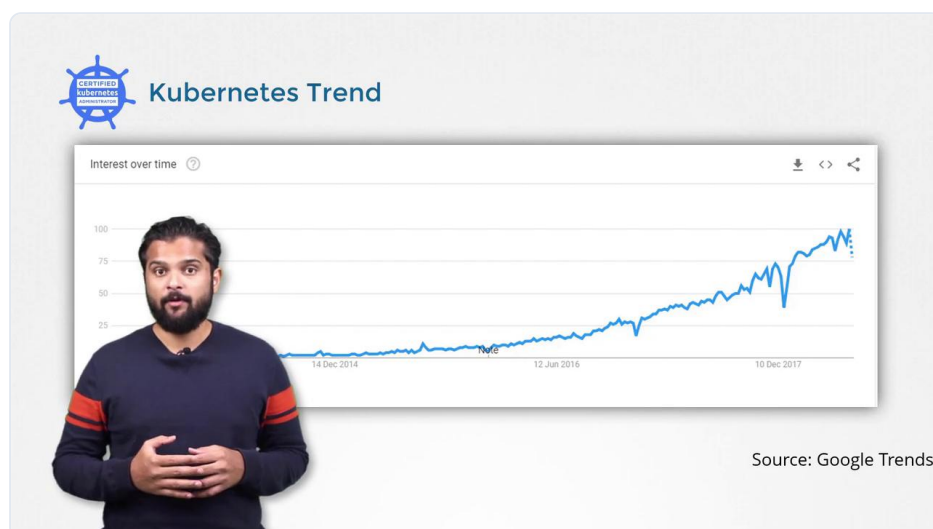
Certification

This article provides a comprehensive guide on the Certified Kubernetes Administrator program, covering its importance, exam details, and preparation tips.

Welcome to this comprehensive guide on the Certified Kubernetes Administrator (CKA) program. In this lesson, we dive into what the CKA certification is, why it is essential in today's tech landscape, and how you can best prepare for the exam.

The Rise of Kubernetes

Kubernetes adoption is surging across industries, and its exponential growth is projected to continue in the coming years. This trend is clearly illustrated by the Google Trends graph below:



According to a recent survey by Indeed, Kubernetes led the job search growth with a remarkable 173% increase year-over-year:



Why Get Certified?

Achieving the CKA certification establishes your credibility as a Kubernetes expert and significantly boosts your value in the job market. Developed by the Cloud Native Computing Foundation in collaboration with the Linux Foundation, this certification distinguishes high-caliber engineers and validates your skills in designing, building, and administering highly available Kubernetes clusters.

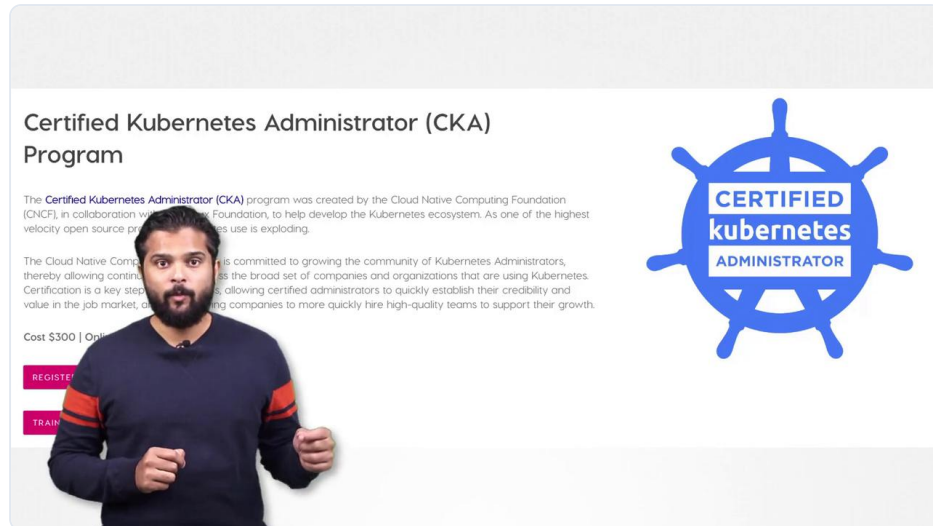


For more details about the certification, visit the [official CKA page](#). The exam currently costs 300 US dollars and includes one free retake. If you do not pass on your initial attempt, you'll have an additional free attempt available within the next 12 months.

The CKA exam is performance-based and requires you to solve real-world problems in a three-hour time window. Unlike multiple-choice exams, you will perform hands-on tasks that mirror real Kubernetes operations. Notably, you will have access to the official Kubernetes documentation throughout the exam.

How the Exam Works

The online delivery format of the exam makes it flexible, allowing you to take it from anywhere at your convenience. During the exam, an online proctor will monitor your session to ensure a secure and fair assessment:



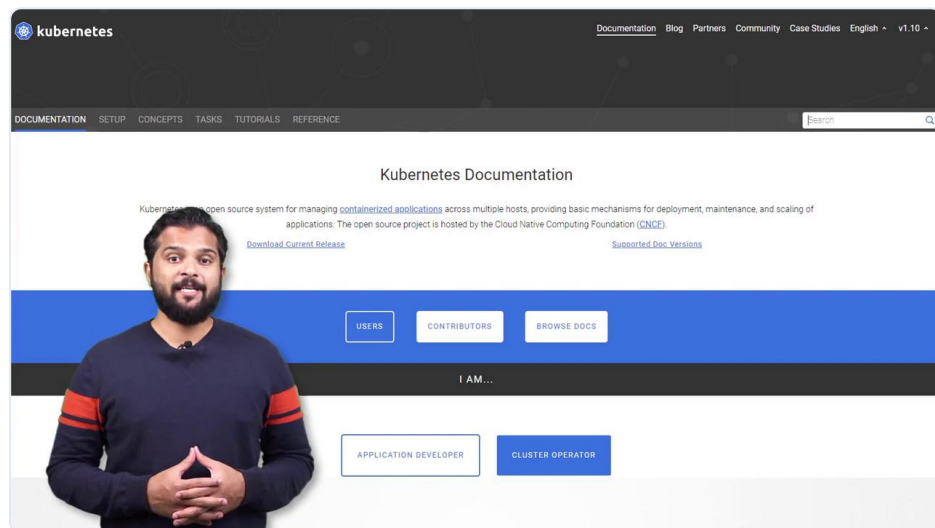
Before you sit for the exam, review all the environmental requirements, such as room setup, system specifications, and network connectivity. These details are clearly outlined in the candidate handbook available on the certification website.

Preparing for Success

A deep understanding of Kubernetes operations, including troubleshooting and cluster management, is crucial for success. During your preparation, focus on honing your hands-on skills and familiarizing yourself with the official Kubernetes documentation. This approach will help you efficiently locate the required information during the exam.

- Ensure your testing environment meets all the technical requirements as detailed in the candidate handbook.
- Practice extensively with real-world Kubernetes scenarios to build confidence.
- Keep calm and manage your time effectively during the exam.

Good luck in your journey towards Kubernetes mastery. With dedication and consistent practice, success is within your reach.



Let us begin.

Section: Core Concepts

Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

Core Concepts Section Introduction

This article introduces essential concepts of modern Cluster Architecture, covering key API primitives for managing containerized applications.

This article introduces the essential concepts underpinning modern Cluster Architecture. We start with a high-level overview to help you understand how various components form a cohesive structure. Following the overview, the discussion advances to key API primitives—including Pods, ReplicaSets, Deployments, and Services—that are integral to managing containerized applications.

If you already have foundational training or have completed certification-level learning for Application Developers, you may opt to bypass some sections and focus solely on the practice tests.

Let's embark on our exploration of Cluster Architecture and uncover how these components interact to deliver a robust deployment environment.

Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

Cluster Architecture

This article provides an overview of Kubernetes cluster architecture, detailing the roles and components of master and worker nodes in managing containerized applications.

Hello and welcome to our comprehensive guide on Kubernetes cluster architecture. In this article, we provide a high-level overview of how Kubernetes organizes and manages containerized applications. You will learn about each component's roles, responsibilities, and configurations, as well as practical insights into examining an existing cluster.

Kubernetes simplifies the deployment, scaling, and management of containerized applications through automation. To help explain this concept, imagine two kinds of ships: cargo ships (worker nodes) that carry containers, and control ships (master nodes) that monitor and manage the cargo ships. In Kubernetes, the cluster consists of nodes—whether physical or virtual, on-premises or cloud-hosted—that host your containerized applications.

Master Node Components

The master node contains several control plane components that manage the entire Kubernetes cluster. It keeps track of all nodes, decides where applications should run, and continuously monitors the cluster. Think of the master node as the central command center coordinating the fleet.



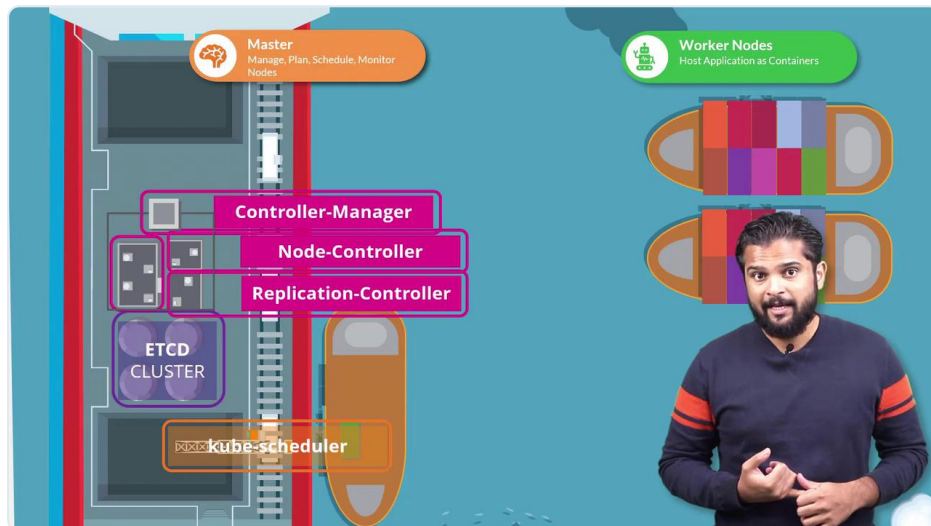
In a busy harbor, many containers are loaded and unloaded daily. Kubernetes maintains detailed information about each container and its corresponding node in a highly available key-value store called etcd. Etcd uses a simple key-value format along with a quorum mechanism, ensuring reliable and consistent data storage across the cluster.

When a new container (or "ship cargo") is ready, the Kubernetes scheduler—similar to port cranes—determines which worker node (or "ship") should host it. The scheduler takes into account current load, resource requirements, and specific constraints like taints, tolerations, or node affinity rules. This scheduling process is vital for efficient cluster operation.

The Kubernetes replication controller and other controllers work like dock office staff, ensuring that the desired number of containers are running and managing node operations.

Other key master node components include:

- **ETCD Cluster:** Stores cluster-wide configuration and state data.
- **Kube Scheduler:** Determines the best node for new container deployments.
- **Controllers:** Manage node lifecycle, container replication, and system stability.
- **Kube API Server:** Acts as the central hub for cluster communication and management.



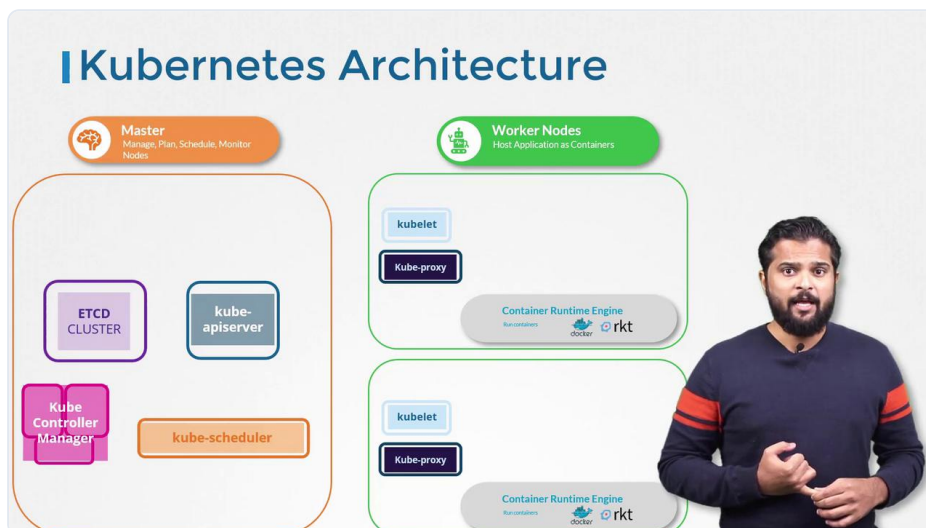
Worker Node Components

Worker nodes, which can be compared to cargo ships, are responsible for running the containerized applications. Each node is managed by the Kubelet, the node's "captain," which ensures that containers are running as instructed.

- **Kubelet:** Manages container lifecycle on an individual node. It receives instructions from the Kube API server to create, update, or delete containers, and regularly reports the node's status.
- **Kube Proxy:** Configures networking rules on worker nodes, thus enabling smooth inter-container communication across nodes. For instance, it allows a web server on one node to interact with a database on another.

The entire control system is containerized. Whether you are using Docker, Containerd, or CRI-O, every node (including master nodes with containerized components) requires a compatible container runtime engine.

The high-level worker node architecture ensures that applications remain available and responsive, even as they communicate across a distributed network.



Summary of Kubernetes Architecture

The Kubernetes cluster architecture is divided into two main segments:

Component Category	Key Components	Description
Master Node	etcd, Kube Scheduler, Controllers, Kube API Server	Centralized control and management of the entire cluster.
Worker Node	Kubelet, Kube Proxy	Responsible for the lifecycle management of containers and ensuring network communication between services.

This clear separation and coordination between master and worker nodes is fundamental to Kubernetes' ability to automate and streamline container orchestration.

We hope this detailed overview of Kubernetes cluster architecture has provided valuable insights. In upcoming articles, we will explore each component in depth, offering practical examples and exercises to further enhance your understanding of Kubernetes systems.

Happy learning and stay tuned for more Kubernetes content!

Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

Docker vs ContainerD

This article explores the relationship between Docker and ContainerD, detailing their differences and the CLI tools for interacting with container runtimes.

In this lesson, we explore the intricate relationship between Docker and ContainerD, as well as the various CLI tools that interact with container runtimes. If you've encountered references to Docker alongside Kubernetes or ContainerD in different articles, this guide will clarify their differences and explain when to use tools like CTR, CRICTL, or NodeController.

The Evolution of Container Runtimes

At the dawn of the container era, Docker reigned supreme thanks to its intuitive interface. Even though alternative tools like Rocket existed, Kubernetes was initially designed to orchestrate Docker-based containers, creating a strong coupling between Docker and Kubernetes. However, as Kubernetes expanded its ecosystem, other container runtimes sought integration.

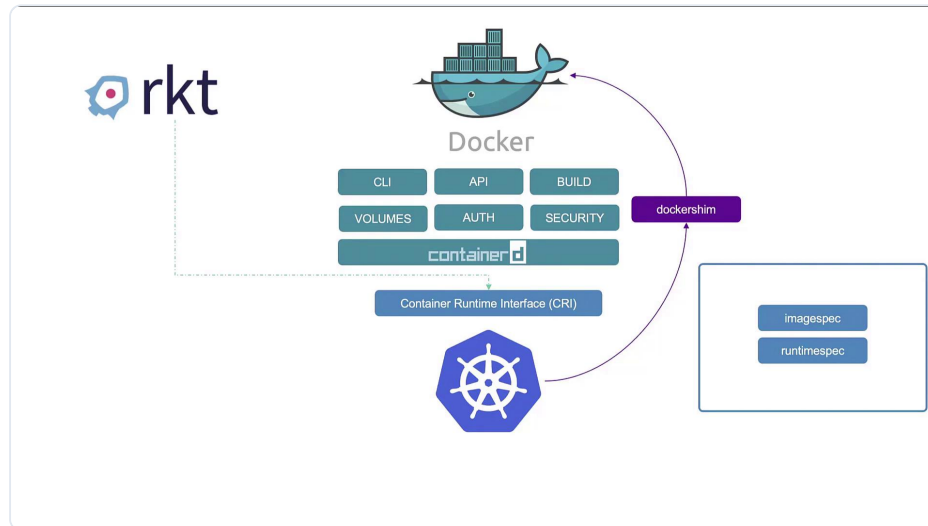
Kubernetes addressed this need by introducing the Container Runtime Interface (CRI). The CRI standardizes container runtimes by ensuring compliance with the Open Container Initiative (OCI) standards—the image specification (for building images) and the runtime specification (for container execution). This standardization paved the way for runtimes like Rocket and others to be supported by Kubernetes.

Since Docker was developed before CRI was established, it wasn't built with CRI compatibility in mind. To allow Docker to function with Kubernetes, a temporary solution known as the Docker Shim was implemented. Over time, native CRI-compatible runtimes such as ContainerD emerged, shifting Docker's primary role within the Kubernetes architecture.

ContainerD, a CRI-compatible runtime, integrates directly with Kubernetes—eliminating the need for the Docker Shim. Originally bundled with Docker, ContainerD has evolved into a standalone project under the Cloud Native

Computing Foundation. This decoupling means you can install and use ContainerD without needing the entire Docker ecosystem if your goal is solely container runtime functionality.

The diagram below illustrates the relationship between Docker, Kubernetes, and alternative container runtimes (like rkt), highlighting components such as containerd, the Docker Shim, and the Container Runtime Interface (CRI).



Within Docker, several components work together to provide its functionality:

- Docker CLI and API
- Image build tools
- Support for volumes, authentication, and security
- The container runtime (runc) managed by containerd

Since ContainerD is CRI-compatible, it works directly with Kubernetes, which eventually led to Kubernetes version 1.24 removing support for Docker as a runtime due to the complexity of maintaining the Docker Shim. However, Docker images remain fully OCI-compliant and continue to work seamlessly with ContainerD.

Diving Deeper into ContainerD

ContainerD began as an internal component of Docker but is now maintained as an independent project. If you do not require Docker's additional features, you can install ContainerD on its own. Traditionally, Docker users ran containers using commands like `docker run`, but ContainerD introduces different CLI tools.

Note: Installation Example To install ContainerD, execute the following command:

```
```bash theme={null} $ tar -C /usr/local -zxvf containerd-1.6.2-linux-  
amd64.tar.gz bin/ bin/ctr bin/containerd ...
```

Once installed, CTR can pull images and execute basic container operations. However, due to

<Frame>

! [The image describes the "ctr" CLI tool, bundled with containerD as not user-friendly

</Frame>

### Using CTR

For example, to pull and run a Redis image using CTR, use the following commands:

```
```bash theme={null}  
$ ctr images pull docker.io/library/redis:alpine  
$ ctr run docker.io/library/redis:alpine redis
```

NerdCTL: A Docker-like CLI for ContainerD

NerdCTL offers a CLI experience similar to Docker's while providing additional features unique to ContainerD. Key features include:

- Encrypted container images
- Lazy pulling of images
- Peer-to-peer image distribution
- Image signing and verification in Kubernetes namespaces

By transitioning from Docker commands to NerdCTL, you maintain familiar syntax. For example, you can replace these Docker commands:

```
```bash theme={null} $ docker run --name redis redis:alpine $ docker run --  
name webserver -p 80:80 -d nginx
```

with the equivalent NerdCTL commands by simply substituting "docker" with "nerdctl" in your

## ## CRI Tools: A Kubernetes Perspective

Another significant CLI utility is CRI CTL (cricctl), designed to interact with any CRI-compliant

<Frame>

! [The image explains "cricctl," a CLI for CRI-compatible container runtimes, used for in

</Frame>

Cricctl is mainly used for tasks such as:

- \* Pulling container images
- \* Listing images and containers
- \* Inspecting logs and executing commands inside containers (using options like `-i` and `-t`)
- \* Listing pods (a functionality not provided by Docker commands)

For example, typical cricctl usage might include:

```
```bash
theme={null}
$ cricctl pull busybox
$ cricctl images
$ cricctl ps -a
```

It is important to note that containers manually created using cricctl may be removed by the Kubelet, as they are not registered as part of a Kubernetes Pod.

A comparison between the Docker CLI and cricctl shows many overlapping commands. Both tools support functions such as attach, exec, images, info, inspect, logs, ps, stats, and version, though there are differences in how they handle container creation and management.

docker vs cricctl

Retrieve debugging information ⓘ

docker cli	cricctl	Description	Unsupported Features
attach	attach	Attach to a running container	<code>--detach-keys</code> , <code>--sig-proxy</code>
exec	exec	Run a command in a running container	<code>--privileged</code> , <code>--user</code> , <code>--detach-keys</code>
images	images	List images	
info	info	Display system-wide information	
inspect	inspect , inspecti	Return low-level information on a container, image or task	
logs	logs	Fetch the logs of a container	<code>--details</code>
ps	ps	List containers	
stats	stats	Display a live stream of container(s) resource usage statistics	Column: NET/BLOCK I/O, PIDs
version	version	Show the runtime (Docker, ContainerD, or others) version information	

<https://kubernetes.io/docs/reference/tools/map-cricctl-dockercli/>

docker vs crictl

Perform Changes

docker cli	crictl	Description	Unsupported Features
create	create	Create a new container	
kill	stop (timeout = 0)	Kill one or more running container	--signal
pull	pull	Pull an image or a repository from a registry	--all-tags, --disable-content-trust
rm	rm	Remove one or more containers	
rmi	rmi	Remove one or more images	
run	run	Run a command in a new container	
start	start	Start one or more stopped containers	--detach-keys
stop	stop	Stop one or more running containers	
update	update	Update configuration of one or more containers	--restart, --blkio-weight and some other resource limit not supported by CRI.

<https://kubernetes.io/docs/reference/tools/map-crictl-dockercli/>

Changes in Kubernetes Runtime Endpoints

Earlier versions of Kubernetes had the kubelet connect to container runtime endpoints in the following default order:

```
```text theme={null} unix:///var/run/dockerhim.sock unix:///run/containerd/
containerd.sock unix:///run/crio/crio.sock unix:///var/run/cri-dockerd.sock
```

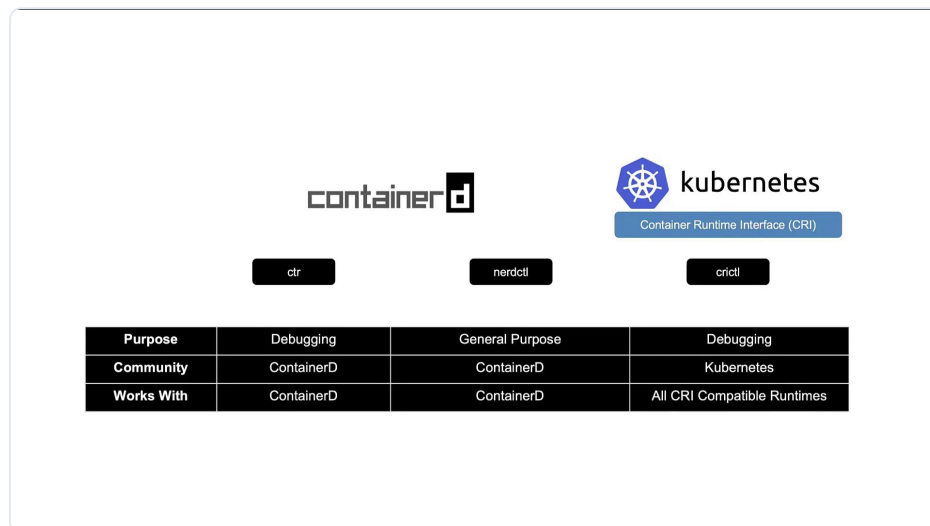
With Kubernetes 1.24, significant changes were introduced. The docker-socket was replaced

```
```bash theme={null}
$ crictl --runtime-endpoint <endpoint>
$ export CONTAINER_RUNTIME_ENDPOINT=<endpoint>
```

For further details, refer to the [Kubernetes CLI Tools repository](#) (see pull request 869 and issue 868).

Summary

- The CTR tool, bundled with ContainerD, is primarily used for debugging.
- NerdCTL offers a Docker-like CLI for general-purpose container operations and advanced ContainerD features.
- Crictl, maintained by the Kubernetes community, is designed for interacting with all CRI-compatible container runtimes and is mainly used for debugging and inspection.



Each tool serves a distinct role within the container ecosystem. While CTR and crictl focus on debugging, NerdCTL is the recommended tool for routine container management tasks, offering a familiar interface for users transitioning from Docker CLI commands.

Thank you for following along in this lesson on Docker versus ContainerD and the accompanying CLI tools. Enjoy exploring container runtimes and enhancing your container management workflows!

Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

A note on Docker Deprecation

The article clarifies Docker's deprecation in Kubernetes while highlighting its continued relevance in development and container management.

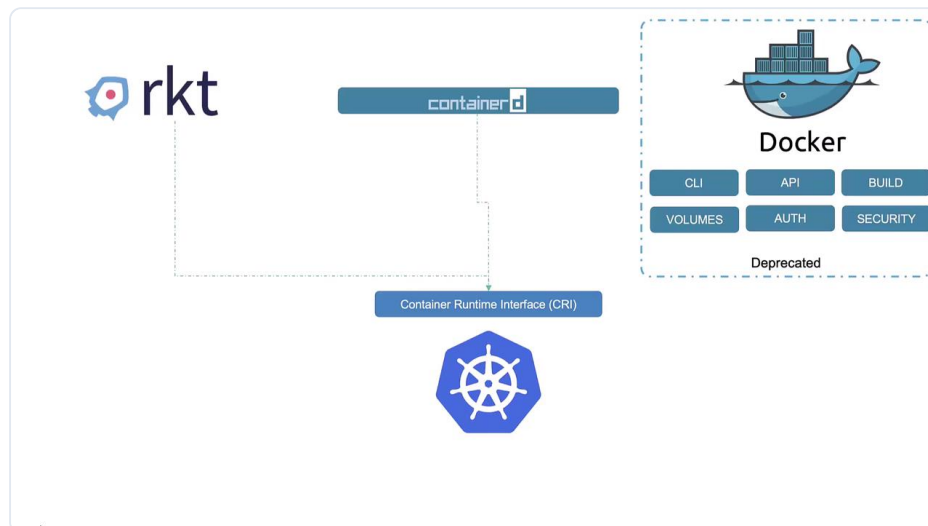
In this lesson, we address common questions regarding Docker's deprecation and clarify why Docker remains relevant despite its deprecated status in Kubernetes. This explanation aims to resolve the confusion many students experience.

Background

Originally, Docker was the sole supported container runtime for Kubernetes. To enable compatibility with other container runtimes, the Container Runtime Interface (CRI) was introduced. Docker, as a complete platform, bundled several tools together, including:

- The Docker CLI
- The API
- Build tools for creating images
- Volume support
- Security configurations
- The container runtime (runc)
- The daemon (containerd) that managed the runtime

Containerd is now CRI-compatible and functions as a standalone component that interacts directly with Kubernetes. Thanks to this change, Kubernetes no longer depends on Docker's additional tools since it manages container operations natively.



Docker's Role in Modern Development

While Kubernetes has officially deprecated Docker as its runtime, Docker remains the most popular container solution for day-to-day development and build processes. Kubernetes simply leverages containerd for orchestration but many development practices still benefit from Docker's extensive toolset.

Even though Kubernetes doesn't require Docker for container orchestration, using Docker to understand container fundamentals remains beneficial. Once these basics are mastered, shifting focus to Kubernetes and containerd becomes more intuitive.

Throughout the course, Docker is used as a primary example to explain container concepts. This strategy allows learners to grasp fundamental ideas before transitioning to Kubernetes-specific orchestration. If Docker is not installed on your system or if you prefer using containerd, you can convert Docker commands to their equivalent [kubectl](#) commands as needed.

This clarification sets the context for the remainder of the course and ensures a smooth transition from understanding container concepts with Docker to applying them within Kubernetes environments.

Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

ETCD for Beginners

This article introduces etcd, a distributed key-value store, covering its operation, installation, and transition from API v2 to v3.

This article provides a concise introduction to etcd—a distributed, reliable key-value store that is both simple and fast. Whether you're new to key-value storage concepts or already familiar with etcd's role in Kubernetes environments, this guide offers valuable insights into its operation and quick start instructions using the etcdctl client tool.

Objectives

- What is ETCD?
- What is a Key-Value Store?
- How to get started quickly?
- How to operate ETCD?

Later:

- What is a distributed system?
- How ETCD Operates
- RAFT Protocol
- Best practices on number of nodes

Later in the article, we will dive deeper into high availability topics. You'll learn about distributed systems, how etcd operates in cluster mode, the Raft consensus protocol, and best practices for configuring a resilient etcd cluster.

What Is a Key-Value Store?

Traditional relational databases, such as SQL databases, store data in tables with rows and columns. For example, a table that contains information about individuals might look like this:

- Each row represents a single person.

- Each column holds a specific detail about that person (e.g., name, age).

If you need to include additional information (like salary data for employed individuals or grades for students), you must expand the table. This means adding columns that may not apply universally to every row.

In contrast, a key-value store organizes data as independent documents or files. Each document contains all relevant information for an individual, allowing flexible and dynamic data structures:

- Working individuals can have documents with salary details.
- Students can have documents with grade details.

key-value store

Key	Value
Name	John Doe
Age	45
Location	New York
Salary	5000

Key	Value
Name	Dave Smith
Age	34
Location	New York
Salary	4000

Key	Value
Name	Aryan Kumar
Age	10
Location	New York
Grade	A

Key	Value
Name	Lauren Rob
Age	13
Location	Bangalore
Grade	C

Key	Value
Name	Lily Oliver
Age	15
Location	Bangalore
Grade	B

For complex data transactions, structured formats like JSON or YAML are used. Here are some examples of key-value pairs stored as JSON documents:

```
```json theme={null} { "name": "John Doe", "age": 45, "location": "New York", "salary": 5000 }
```

```
```json theme={null}
{
  "name": "Dave Smith",
  "age": 34,
  "location": "New York",
  "salary": 4000,
  "organization": "ACME"
}
```

```
```json theme={null} { "name": "Aryan Kumar", "age": 10, "location": "New York", "Grade": "A" }
```

```
```json theme={null}
{
  "name": "Lily Oliver",
  "age": 15,
  "location": "Bangalore",
  "Grade": "B"
}
```

```
```json theme={null} { "name": "Lauren Rob", "age": 13, "location": "Bangalore",
"Grade": "C" }
```

### ## Installing and Getting Started with etcd

Getting started with etcd is straightforward. First, download the correct binary for your system.

For example, you can download etcd using the following command:

```
```bash theme={null}
curl -L https://github.com/etcd-io/etcd/releases/download/v3.3.11/etcd-v3.3.11-linux-amd64.
```

After extracting and starting etcd, the accompanying command-line client, etcdctl, lets you interact with your etcd instance. To store a key-value pair using the etcd API v2, run:

```
```bash theme={null} ./etcdctl set key1 value1
```

And to retrieve it:

```
```bash theme={null}
./etcdctl get key1
```

If you run the etcdctl command without any arguments, it displays a list of available commands similar to this:

```
```bash theme={null} ./etcdctl
```

```
```text theme={null}
NAME:
etcdctl - A simple command line client for etcd.

COMMANDS:
  backup      backup an etcd directory
  cluster-health check the health of the etcd cluster
  mk          make a new key with a given value
  mkdir       make a new directory
  rm          remove a key or a directory
  rmdir       removes the key if it is an empty directory or a key-value pair
  get         retrieve the value of a key
```

Different articles may reference varying versions of etcd. Understanding its version history—starting with version 0.1 in August 2013, introducing the Raft

consensus algorithm in version 2.0 (February 2015), and further optimizations in version 3 (January 2017)—can clarify command differences. The etcd project was incubated in the CNCF in November 2018.



Transitioning from API v2 to v3

One critical change between etcd versions is the API version used by etcdctl. Although etcdctl may be configured to use API v2 by default, newer installations typically default to API v3. You can verify the API version by running:

```
```bash theme={null} ./etcdctl --version
```

This command might output:

```
```text theme={null}
etcdctl version: 3.3.11
API version: 2
```

When you list available commands in API v2 mode:

```
```bash theme={null} ./etcdctl
```

the command list includes options like:

```
```text theme={null}
COMMANDS:
  backup          backup an etcd directory
  cluster-health  check the health of the etcd cluster
  mk              make a new key with a given value
  mkdir           make a new directory
  rm              remove a key or a directory
  rmdir           removes the key if it is an empty directory or a key-value pair
  get             retrieve the value of a key
  ls             retrieve a directory
  set             set the value of a key
  setdir          create a new directory or update an existing directory TTL
  update          update an existing key with a given value
  updatedir       update an existing directory
  watch           watch a key for changes
  exec-watch      watch a key for changes and exec an executable
  member          member add, remove and list subcommands
  user            user add, grant and revoke subcommands
  role            role add, grant and revoke subcommands
```

To switch etcdctl to use API v3, you have two options:

1. Prepend the environment variable to the command:

```
bash theme={null} ETCDCCTL_API=3 ./etcdctl version
```

This produces the output:

```
text theme={null} etcdctl version: 3.3.11 API version: 3
```

1. Or, export the environment variable for your session:

```
bash theme={null} export ETCDCCTL_API=3 ./etcdctl version
```

The output will similarly confirm the switch to API v3.

In API v3, the command for setting a key changes from "set" to "put", while retrieving a key remains the same ("get"). Note that the `version` command is now a subcommand rather than an option.

After setting the environment variable, you can perform key-value operations with API v3 commands. For example:

```
```bash theme={null} export ETCDCCTL_API=3 ./etcdctl version
```

Expected output:

```
```text theme={null}
etcdctl version: 3.3.11
API version: 3
```

Then, set a key-value pair:

```
```bash theme={null} ./etcdctl put key1 value1
```

This should output:

```
```text theme={null}  
OK
```

And retrieve the value:

```
```bash theme={null} ./etcdctl get key1
```

Expected output:

```
```text theme={null}  
key1  
value1
```

Conclusion

In this article, we explored etcd and the fundamentals of key-value storage, contrasting it with traditional relational databases. We demonstrated how to install etcd, use the etcdctl command-line tool, and navigate the transition from API v2 to API v3. In our upcoming article, we will examine how to configure etcd in a high-availability environment and its seamless integration with Kubernetes.

Thank you for reading, and stay tuned for further insights into etcd and distributed systems.

Documentation Index

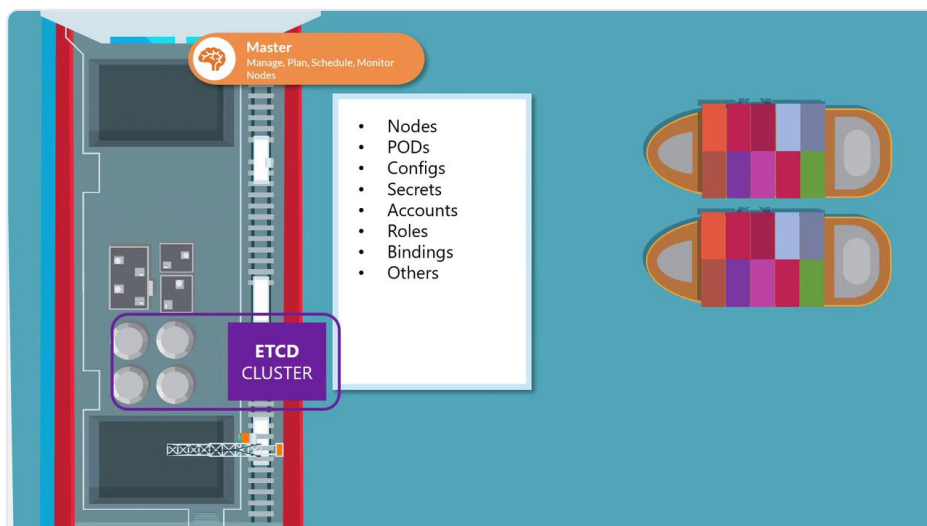
Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

ETCD in Kubernetes

This article explores the role of etcd in Kubernetes, covering deployment methods and high availability considerations.

Welcome to this comprehensive guide on etcd in Kubernetes. In this article, we explore the critical role of etcd in storing cluster state, detail different deployment approaches, and explain high availability considerations. Whether you're setting up a Kubernetes cluster from scratch or using kubeadm, understanding etcd is essential.

etcd is a distributed key-value store that maintains configuration data, state information, and metadata for your Kubernetes cluster. Every object—nodes, pods, configurations, secrets, accounts, roles, and role bindings—is stored within etcd. When you run a command like `kubectl get`, the data is retrieved from this data store.



Any changes you make to the cluster—whether adding nodes, deploying pods, or configuring ReplicaSets—are first recorded in etcd. Only after etcd is updated are these changes considered to be complete.

The etcd server typically listens on port 2379 for client requests. Ensuring that the advertised client URL (via the `--advertise-client-urls` option) is

correctly configured is crucial for proper communication between the Kubernetes API Server and etcd.

Deployment Methods

Depending on your Kubernetes setup, you can deploy etcd in two primary ways: manually from scratch or automatically with kubeadm. Each method has its use cases, with manual setups providing a deeper understanding of etcd configurations and kubeadm streamlining the deployment process.

Deploying etcd from Scratch

When setting up your cluster manually, you'll need to download the etcd binaries, install them, and configure etcd as a service on your master node. Manual deployment gives you more control over configuration options, particularly for setting up TLS certificates.

Below is an example of how you might download the etcd binaries and configure the etcd service:

```
```bash theme={null} wget -q --https-only \ "https://github.com/coreos/etcd/releases/download/v3.3.9/etcd-v3.3.9-linux-amd64.tar.gz"
```

## Example etcd service configuration

---

```
ExecStart=/usr/local/bin/etcd \ --name ${ETCD_NAME} \ --cert-file=/etc/etcd/
kubernetes.pem \ --key-file=/etc/etcd/kubernetes-key.pem \ --peer-cert-file=/
etc/etcd/kubernetes.pem \ --peer-key-file=/etc/etcd/kubernetes-key.pem \ --
trusted-ca-file=/etc/etcd/ca.pem \ --peer-trusted-ca-file=/etc/etcd/ca.pem \ --
peer-client-cert-auth \ --client-cert-auth \ --initial-advertise-peer-urls https://$
${INTERNAL_IP}:2380 \ --listen-peer-urls https://${INTERNAL_IP}:2380 \ --
listen-client-urls https://${INTERNAL_IP}:2379,https://127.0.0.1:2379 \ --
advertise-client-urls https://${INTERNAL_IP}:2379 \ --initial-cluster-token etcd-
cluster-0 \ --initial-cluster controller-0=https://${
${CONTROLLER0_IP}:2380,controller-1=https://${CONTROLLER1_IP}:2380 \
--initial-cluster-state new \ --data-dir=/var/lib/etcd
```



```
<Callout icon="lightbulb" color="#1CB2FE">
```

```
 For detailed information on configuring TLS certificates, refer to the Kubernetes document
</Callout>
```

```

```

## ## High Availability Considerations

In a production Kubernetes environment, high availability (HA) is paramount. By running mul

To enable HA, each etcd instance must know about its peers. This is achieved by configuring

```
```bash theme={null}
```

```
ExecStart=/usr/local/bin/etcd \
```

```
--name ${ETCD_NAME} \
```

```
--cert-file=/etc/etcd/kubernetes.pem \
```

```
--key-file=/etc/etcd/kubernetes-key.pem \
```

```
--peer-cert-file=/etc/etcd/kubernetes.pem \
```

```
--peer-key-file=/etc/etcd/kubernetes-key.pem \
```

```
--trusted-ca-file=/etc/etcd/ca.pem \
```

```
--peer-trusted-ca-file=/etc/etcd/ca.pem \
```

```
--peer-client-cert-auth \
```

```
--client-cert-auth \
```

```
--initial-advertise-peer-urls=https://${INTERNAL_IP}:2380 \
```

```
--listen-peer-urls=https://${INTERNAL_IP}:2380 \
```

```
--advertise-client-urls=https://${INTERNAL_IP}:2379 \
```

```
--initial-cluster-token=etcd-cluster-0 \
```

```
--initial-cluster controller-0=https://${CONTROLLER0_IP}:2380,controller-1=https://${CO
```

```
--initial-cluster-state=new \
```

```
--data-dir=/var/lib/etcd
```

In some deployments, you may use separate certificate files for peer communications (e.g., `/etc/etcd/peer.pem` and `/etc/etcd/peer-key.pem`). Always tailor these settings to match your desired security posture.

Deploying etcd with kubeadm

For many test environments and streamlined deployments, kubeadm automatically configures etcd. When you use kubeadm, the etcd server runs as a pod within the kube-system namespace, abstracting away the manual setup details.

To view all the pods running in the kube-system namespace, including etcd, run:

```
```bash theme={null} kubectl get pods -n kube-system
```

Example output:

```
```text theme={null}
NAMESPACE          NAME                                READY   STATUS    RESTARTS   AGE
kube-system        coredns-78fcd6894-prwl             1/1     Running   0           1h
kube-system        coredns-78fcd6894-vqd9w            1/1     Running   0           1h
kube-system        etcd-master                         1/1     Running   0           1h
kube-system        kube-apiserver-master              1/1     Running   0           1h
kube-system        kube-controller-manager-master     1/1     Running   0           1h
kube-system        kube-proxy-f6k26                   1/1     Running   0           1h
kube-system        kube-proxy-hnzw                    1/1     Running   0           1h
kube-system        kube-scheduler-master              1/1     Running   0           1h
kube-system        weave-net-924k8                    2/2     Running   1           1h
kube-system        weave-net-hzfcz                    2/2     Running   1           1h
```

To examine the keys stored in etcd (organized under the registry directory), use the following command:

```
```bash theme={null} kubectl exec etcd-master -n kube-system -- etcdctl get / --
prefix --keys-only
```

Sample output:

```
```text theme={null}
/registry/apiregistration.k8s.io/apiservices/v1
/registry/apiregistration.k8s.io/apiservices/v1.apps
/registry/apiregistration.k8s.io/apiservices/v1.authentication.k8s.io
/registry/apiregistration.k8s.io/apiservices/v1.authorization.k8s.io
/registry/apiregistration.k8s.io/apiservices/v1.autoscaling
/registry/apiregistration.k8s.io/apiservices/v1.batch
/registry/apiregistration.k8s.io/apiservices/v1.networking.k8s.io
/registry/apiregistration.k8s.io/apiservices/v1.rbac.authorization.k8s.io
/registry/apiregistration.k8s.io/apiservices/v1beta1.admissionregistration.k8s.io
```

The etcd root directory, organized as the registry, contains subdirectories for various Kubernetes components such as nodes, pods, ReplicaSets, and Deployments.

Conclusion

In this article, we examined etcd's essential role in Kubernetes by exploring both manual deployment and automated configuration with kubeadm. We also discussed best practices for configuring high availability in multi-master environments. As you advance, further exploration into HA configurations and enhanced security measures will help you operate a robust and resilient Kubernetes cluster.

For more in-depth resources, check out:

- [Kubernetes Documentation](#)

- [Kubernetes Basics](#)
- [Docker Hub](#)
- [Terraform Registry](#)

Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

Kube API Server

This article provides a comprehensive guide on the Kube API Servers role in managing requests and coordinating components in a Kubernetes cluster.

Welcome to this comprehensive guide on the Kube API Server in Kubernetes. In this article, we explore how the Kube API Server acts as the central management component in a Kubernetes cluster by handling requests from kubectl, validating and authenticating them, interfacing with the etcd datastore, and coordinating with other system components.

When you execute a command like:

```
``bash theme={null} kubectl get nodes
```

the utility sends a request to the API Server. The server processes this request by authenticating the request.

```
``plaintext theme={null}
```

NAME	STATUS	ROLES	AGE	VERSION
master	Ready	master	20m	v1.11.3
node01	Ready	<none>	20m	v1.11.3

API Server Request Lifecycle

When a direct API POST request is made to create a pod, the API Server:

1. Authenticates and validates the request.
2. Constructs a pod object (initially without a node assignment) and updates the etcd store.
3. Notifies the requester that the pod has been created.

For instance, using a curl command:

```
``bash theme={null} curl -X POST /api/v1/namespaces/default/pods ...[other]
Pod created!
```

The scheduler continuously monitors the API Server for pods that need node assignments. Once

<Callout icon="lightbulb" color="#1CB2FE">

At the heart of these operations is the Kube API Server, ensuring secure and validated co

</Callout>

<Frame>

! [The image lists six steps related to the Kube-api Server: Authenticate User, Validate

</Frame>

Deployment and Setup

If your cluster is bootstrapped with a kube admin tool, most of these intricate details are

Typical Service Configuration

The Kube API Server is launched with a variety of parameters to secure communication and ma

```
```bash theme={null}
```

```
wget https://storage.googleapis.com/kubernetes-release/release/v1.18.0/bin/linux/amd64/kube
```

```
kube-apiserver.service
```

```
ExecStart=/usr/local/bin/kube-apiserver \\\
```

```
--advertise-address=${INTERNAL_IP} \\\
```

```
--allow-privileged=true \\\
```

```
--apiserver-count=3 \\\
```

```
--authorization-mode=Node,RBAC \\\
```

```
--bind-address=0.0.0.0 \\\
```

```
--client-ca-file=/var/lib/kubernetes/ca.pem \\\
```

```
--enable-admission-plugins=Initializers,NamespaceLifecycle,NodeRestriction,LimitRanger,Se
```

```
--enable-swagger-ui=true \\\
```

```
--etcd-cafile=/var/lib/kubernetes/ca.pem \\\
```

```
--etcd-certfile=/var/lib/kubernetes/kubernetes.pem \\\
```

```
--etcd-keyfile=/var/lib/kubernetes/kubernetes-key.pem \\\
```

```
--etcd-servers=https://127.0.0.1:2379 \\\
```

```
--event-ttl=1h \\\
```

```
--experimental-encryption-provider-config=/var/lib/kubernetes/encryption-config.yaml \\\
```

```
--kubelet-certificate-authority=/var/lib/kubernetes/ca.pem \\\
```

```
--kubelet-client-certificate=/var/lib/kubernetes/kubernetes.pem \\\
```

```
--kubelet-client-key=/var/lib/kubernetes/kubernetes-key.pem \\\
```

```
--kubelet-https=true \\\
```

```
--runtime-config=api/all \\\
```

```
--service-account-key-file=/var/lib/kubernetes/service-account.pem \\\
```

```
--service-cluster-ip-range=10.32.0.0/24 \\\
```

```
--service-node-port-range=30000-32767 \\\
```

```
--v=2
```

The configuration includes several certificate-related options, securing communication channels between various Kubernetes components. In upcoming sections, we will take a deeper look at SSL/TLS certificates and their role in ensuring secure interactions.

## Verifying the Deployment

For clusters set up with kube-admin tools, the Kube API Server is deployed as a pod in the kube-system namespace. To inspect these pods, run:

```
```bash theme={null} kubectl get pods -n kube-system
```

Expected output may include:

```
```plaintext theme={null}
```

NAMESPACE	NAME	READY	STATUS	RESTARTS	AGE
kube-system	coredns-78fcd6894-hwrq9	1/1	Running	0	16m
kube-system	coredns-78fcd6894-rzhjr	1/1	Running	0	16m
kube-system	etcd-master	1/1	Running	0	15m
kube-system	kube-apiserver-master	1/1	Running	0	15m
kube-system	kube-controller-manager-master	1/1	Running	0	15m
kube-system	kube-proxy-lzt6f	1/1	Running	0	16m
kube-system	kube-proxy-zm5qd	1/1	Running	0	15m
kube-system	kube-scheduler-master	1/1	Running	0	15m
kube-system	weave-net-29z42	2/2	Running	1	16m
kube-system	weave-net-snm1l	2/2	Running	1	16m

For non-kube-admin setups, you can examine the container command options directly within the pod manifest. Here's an excerpt from a pod definition:

```
```yaml theme={null} spec: containers: - command: - kube-apiserver - --
authorization-mode=Node,RBAC - --advertise-address=172.17.0.32 - --allow-
privileged=true - --client-ca-file=/etc/kubernetes/pki/ca.crt - --disable-
admission-plugins=PersistentVolumeLabel - --enable-admission-
plugins=NodeRestriction - --enable-bootstrap-token-auth=true - --etcd-cafile=/
etc/kubernetes/pki/etcd/ca.crt - --etcd-certfile=/etc/kubernetes/pki/apiserver-
etcd-client.crt - --etcd-keyfile=/etc/kubernetes/pki/apiserver-etcd-client.key - --
etcd-servers=https://127.0.0.1:2379 - --insecure-port=0 - --kubelet-client-
certificate=/etc/kubernetes/pki/apiserver-kubelet-client.crt - --kubelet-client-
key=/etc/kubernetes/pki/apiserver-kubelet-client.key - --kubelet-preferred-
address-types=InternalIP,ExternalIP,Hostname - --proxy-client-cert-file=/etc/
kubernetes/pki/front-proxy-client.crt - --proxy-client-key-file=/etc/kubernetes/
pki/front-proxy-client.key - --requestheader-allowed-names=front-proxy-client -
--requestheader-client-ca-file=/etc/kubernetes/pki/front-proxy-ca.crt - --
requestheader-extra-headers-prefix=X-Remote-Extra- - --requestheader-
group-headers=X-Remote-Group - --requestheader-username-headers=X-
Remote-User
```

Another way to review the active API Server configuration is by checking the systemd service

```
```bash theme={null}
cat /etc/systemd/system/kube-apiserver.service
```

An example excerpt from this file might be:

```

bash theme={null} [Service] ExecStart=/usr/local/bin/kube-apiserver
\\ --advertise-address=${INTERNAL_IP} \\ --allow-privileged=true \\
--apiserver-count=3 \\ --audit-log-maxage=30 \\ --audit-log-
maxbackup=3 \\ --audit-log-maxsize=100 \\ --audit-log-path=/var/log/
audit.log \\ --authorization-mode=Node,RBAC \\ --bind-
address=0.0.0.0 \\ --client-ca-file=/var/lib/kubernetes/ca.pem \\ --
enable-admission-
plugins=Initializers,NamespaceLifecycle,NodeRestriction,LimitRanger,ServiceAccount,DefaultStorageClass
\\ --enable-swagger-ui=true \\ --etcd-cafile=/var/lib/kubernetes/
ca.pem \\ --etcd-certfile=/var/lib/kubernetes/kubernetes.pem \\ --
etcd-keyfile=/var/lib/kubernetes/kubernetes-key.pem \\ --etcd-
servers=https://10.240.0.10:2379,https://10.240.0.11:2379,https://
10.240.0.12:2379 \\ --event-ttl=1h \\ --experimental-encryption-
provider-config=/var/lib/kubernetes/encryption-config.yaml \\ --
kubelet-certificate-authority=/var/lib/kubernetes/ca.pem \\ --
kubelet-client-certificate=/var/lib/kubernetes/kubernetes.pem \\ --
kubelet-client-key=/var/lib/kubernetes/kubernetes-key.pem \\ ...

```

## Quick Reference Table

Below is a summary of some key Kubernetes components involved in the Kube API Server workflow:

Component	Role	Command/Action Example
kubectl	CLI tool to send API requests	<code>kubectl get nodes</code>
Kube API Server	Central component for processing, authenticating, and validating requests	Processes API requests and interacts with etcd
Scheduler	Monitors API Server for unassigned pods and assigns them to worker nodes	Automatically assigns node to newly created pods
Kubelet	Runs on worker nodes to manage pod lifecycle and communicate status back to API Server	Interacts with container runtime to deploy images
etcd	Distributed key-value store used for saving cluster configuration	Stores all cluster state data

## Summary

In this article, we provided an overview of the Kube API Server, its interactions with other essential components, and various methods to inspect its configuration—both through pod manifests and systemd service files. In

subsequent sections, we will delve deeper into certificate management, including SSL/TLS configurations, to reinforce secure communications within your Kubernetes cluster.

For a deeper understanding of Kubernetes and its components, explore the [Kubernetes Documentation](#).

This concludes our discussion on the Kube API Server.



## Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

# Kube Controller Manager

This guide covers the Kube Controller Managers role, configuration, and importance in managing controllers within a Kubernetes cluster.

Welcome to this comprehensive guide on the Kube Controller Manager, a vital component in Kubernetes responsible for managing a variety of controllers within your cluster. Understanding its role and configuration is crucial for ensuring a resilient and well-orchestrated Kubernetes environment.

## Overview

In Kubernetes, a controller acts like a department in an organization—each controller is tasked with handling a specific responsibility. For instance, one controller might monitor the health of nodes, while another ensures that the desired number of pods is always running. These controllers constantly observe system changes to drive the cluster toward its intended state.

The Node Controller, for example, checks node statuses every five seconds through the Kube API Server. If a node stops sending heartbeats, it is not immediately marked as unreachable; instead, there is a grace period of 40 seconds followed by an additional five minutes for potential recovery before its pods are rescheduled onto a healthy node.

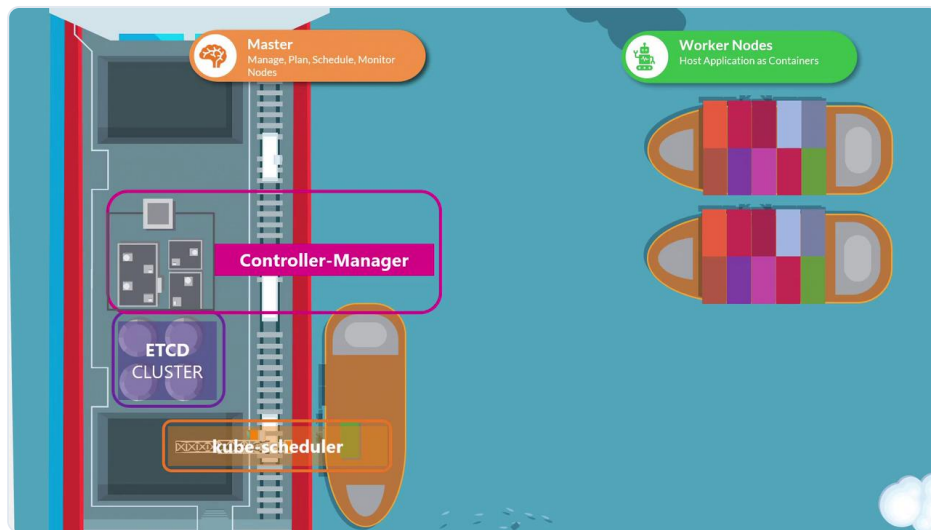
### Example: Checking Node Statuses

```
```bash theme={null} kubectl get nodes NAME STATUS ROLES AGE VERSION
worker-1 Ready 8d v1.13.0 worker-2 Ready 8d v1.13.0
```

In the case where a node fails to recover, the output might look like this:

```
```bash theme={null}
kubectl get nodes
NAME STATUS ROLES AGE VERSION
worker-1 Ready <none> 8d v1.13.0
worker-2 NotReady <none> 8d v1.13.0
```

Another essential controller is the Replication Controller, which ensures that the specified number of pods is maintained by creating new pods when needed. This mechanism reinforces the resilience and reliability of your Kubernetes cluster.



All core Kubernetes constructs—such as Deployments, Services, Namespaces, and Persistent Volumes—rely on these controllers. Essentially, controllers serve as the "brains" behind many operations in a Kubernetes cluster.

## How Controllers Are Packaged

All individual controllers are bundled into a single process known as the Kubernetes Controller Manager. When you deploy the Controller Manager, every associated controller is started together. This unified deployment simplifies management and configuration.

## Installing and Configuring the Kube Controller Manager

To install and view the Kube Controller Manager, follow these steps:

1. Download the Kube Controller Manager from the Kubernetes release page.
2. Extract the binary and run it as a service.
3. Review the configurable options provided, which allow you to tailor its behavior.

### Downloading the Controller Manager

```
```bash theme={null} wget https://storage.googleapis.com/kubernetes-release/release/v1.13.0/bin/linux/amd64/kube-controller-manager
```

Sample Service Configuration

Below is an example of a service file (`kube-controller-manager.service`) used to run the C

```
``bash theme={null}
ExecStart=/usr/local/bin/kube-controller-manager \
  --address=0.0.0.0 \
  --cluster-cidr=10.200.0.0/16 \
  --cluster-name=kubernetes \
  --cluster-signing-cert-file=/var/lib/kubernetes/ca.pem \
  --cluster-signing-key-file=/var/lib/kubernetes/ca-key.pem \
  --kubeconfig=/var/lib/kubernetes/kube-controller-manager.kubeconfig \
  --leader-elect=true \
  --root-ca-file=/var/lib/kubernetes/ca.pem \
  --service-account-private-key-file=/var/lib/kubernetes/service-account-key.pem \
  --service-cluster-ip-range=10.32.0.0/24 \
  --use-service-account-credentials=true \
  --v=2
```

This configuration includes additional options for the Node Controller, such as node monitor period, grace period, and eviction timeout. Additionally, you can control which controllers are enabled through the `--controllers` flag.

By default, all controllers are enabled. You can selectively enable or disable controllers by using the syntax `foo` to enable and `-foo` to disable. For example, `--controllers=*,-tokencleaner` will disable the `tokencleaner` controller.

Example of Specifying Controllers

```
``bash theme={null} --controllers stringSlice Default: [] A list of controllers to
enable. " enables all on-by-default controllers, 'foo' enables the controller
named 'foo', '-foo' disables the controller named 'foo'. All controllers:
attachdetach, bootstrapsigner, clusterrole-aggregation, cronjob, csrapproving,
csrcleaner, csrsigning, daemonset, deployment, disruption, endpoint,
garbagecollector, horizontalpodautoscaling, job, namespace, nodeipam,
nodelifecycle, persistentvolume-binder, persistentvolume-expander, podgc, pv-
protection, pvc-protection, replicaset, replicationcontroller, resourcequota, root-
ca-cert-publisher, route, service, serviceaccount, serviceaccount-token,
statefulset, tokencleaner, ttl, ttl-after-finished Disabled-by-default controllers:
bootstrapsigner, tokencleaner
```

Viewing the Controller Manager in Action

Depending on your cluster setup, the Controller Manager may run as a pod in the `kube-system` namespace.

Service Configuration Example (Non-Kubeadm Environments)

```

`plaintext theme={null}
[Service]
ExecStart=/usr/local/bin/kube-controller-manager \
  --address=0.0.0.0 \
  --cluster-cidr=10.200.0.0/16 \
  --cluster-name=kubernetes \
  --cluster-signing-cert-file=/var/lib/kubernetes/ca.pem \
  --cluster-signing-key-file=/var/lib/kubernetes/ca-key.pem \
  --kubeconfig=/var/lib/kubernetes/kube-controller-manager.kubeconfig \
  --leader-elect=true \
  --root-ca-file=/var/lib/kubernetes/ca.pem \
  --service-account-private-key-file=/var/lib/kubernetes/service-account-key.pem \
  --service-cluster-ip-range=10.32.0.0/24 \
  --use-service-account-credentials=true \
  --v=2
Restart=on-failure
RestartSec=5

```

Checking the Running Process

To verify that the Kube Controller Manager is running and to inspect its active options, execute the following command on the master node:

```

`plaintext theme={null} ps -aux | grep kube-controller-manager

```

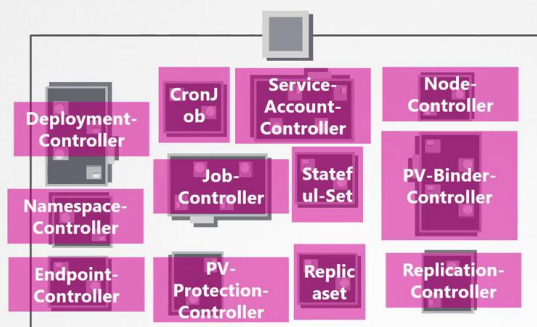
An example output might be:

```

`plaintext theme={null}
root      1994   2.7   5.1 154360 105024 ?        Ssl  06:45   1:25 kube-controller-manager

```

Controller



- Watch Status
- Remediate Situation
- Node Monitor Period = 5s
- Node Monitor Grace Period = 40s
- POD Eviction Timeout = 5m

Conclusion

This guide has provided an in-depth look at the Kube Controller Manager, detailing its critical functions in managing controllers, monitoring system changes, and ensuring the desired state within your Kubernetes cluster. By understanding and properly configuring the Controller Manager, you play a key role in maintaining a robust and scalable environment.

Stay tuned for more lessons to further enhance your Kubernetes expertise!

For additional details, you might find these resources useful:

- [Kubernetes Documentation](#)
- [Kubernetes Basics](#)

Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

Kube Scheduler

This article provides a comprehensive overview of the Kube Scheduler's role in pod placement within Kubernetes.

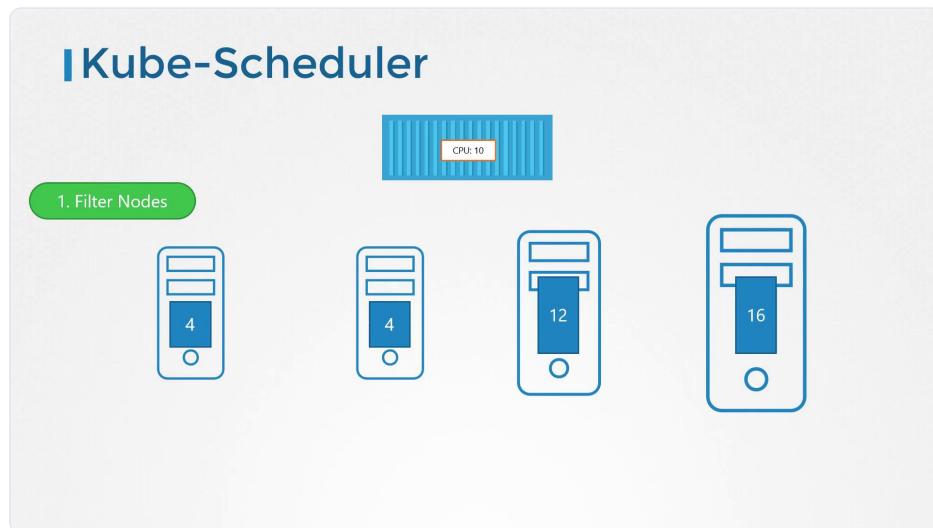
Welcome to this comprehensive lesson on the Kube Scheduler, a core component of Kubernetes. In this guide, we delve into the scheduler's role in determining on which node a pod should be placed. It is important to note that while the scheduler makes the placement decision, the actual creation of the pod on the selected node is carried out by the Kubelet.

Scheduler Process Overview

The primary responsibility of the Kubernetes scheduler is to assign pods to nodes based on a series of criteria. This ensures that the selected node has sufficient resources and meets any specific requirements. For instance, different nodes may be designated for certain applications or come with varied resource capacities. When multiple pods and nodes are involved, the scheduler assesses each pod against the available nodes through a two-phase process: filtering and ranking.

1. Filtering Phase

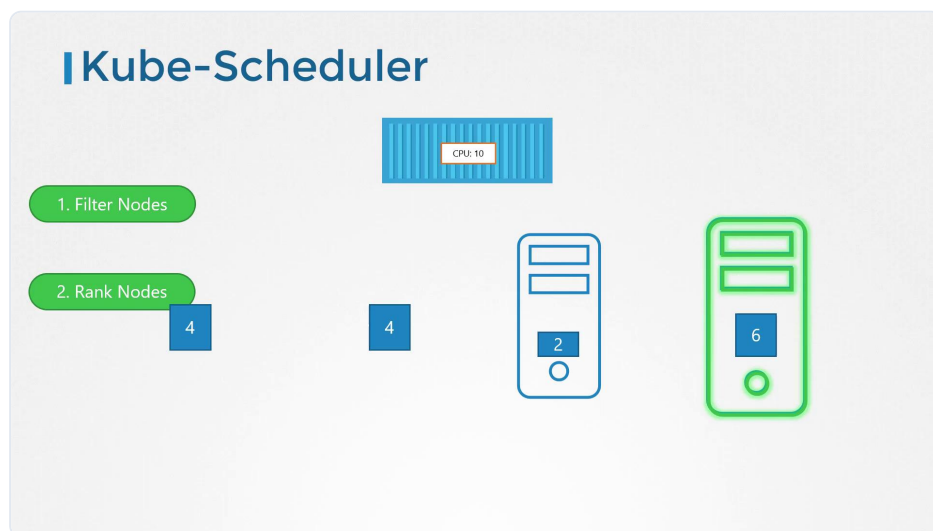
In the filtering phase, the scheduler eliminates nodes that do not meet the pod's resource requirements. For example, nodes that lack sufficient CPU or memory are immediately excluded.



As depicted above, the diagram demonstrates the elimination of nodes with insufficient resources, leaving only the candidate nodes that can accommodate the pod's needs.

2. Ranking Phase

After filtering, the scheduler enters the ranking phase. Here, it uses a priority function to score and compare the remaining nodes on a scale from 0 to 10, ultimately selecting the best match. For instance, if placing a pod on one node would leave six free CPUs (four more than an alternative node), that node is assigned a higher score and is chosen.



This high-level overview outlines how Kubernetes efficiently filters and ranks nodes for optimal pod placement. The scheduler's design is highly customizable, allowing you to develop your own scheduler if the need arises. For more advanced scheduling configurations—such as resource limits, taints and tolerations, node selectors, and affinity rules—refer to the [Kubernetes Documentation](#).

Customizing the scheduling process can help tailor your Kubernetes environment to meet specific workloads and performance requirements.

Installing and Running the Kube Scheduler

To install the kube-scheduler, download the binary from the Kubernetes release page. Once downloaded and extracted, you can run it as a service by specifying the scheduler configuration file. Below is a sample command for downloading the binary and an example systemd service configuration:

```
``bash theme={null} wget https://storage.googleapis.com/kubernetes-release/release/v1.13.0/bin/linux/amd64/kube-scheduler
```

Below is an example of the systemd service configuration for the kube-scheduler:

```
``bash theme={null}
# File: kube-scheduler.service
ExecStart=/usr/local/bin/kube-scheduler \
  --config=/etc/kubernetes/config/kube-scheduler.yaml \
  --v=2
```

If you are using the kubeadm tool to set up your cluster, kubeadm deploys the kube-scheduler as a pod in the `kube-system` namespace on the master node. You can inspect the scheduler configuration by viewing the pod manifest file:

```
``bash theme={null} cat /etc/kubernetes/manifests/kube-scheduler.yaml
```

This manifest file outlines the options used during the scheduler's deployment. To verify t

```
``bash theme={null}
ps -aux | grep kube-scheduler
```

An example output might look similar to:

```
bash theme={null} root 2477 0.8 1.6 48524 34044 ? Ssl 17:31 0:08
kube-scheduler --address=127.0.0.1 --kubeconfig=/etc/kubernetes/
scheduler.conf --leader-elect=true
```

If you need more detailed configuration options or troubleshooting tips for the kube-scheduler, refer to the [Kubernetes Documentation](#).

This concludes our in-depth lesson on the Kube Scheduler. In future modules, we will explore advanced scheduling concepts and configurations to further enhance your Kubernetes deployment strategies.

Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

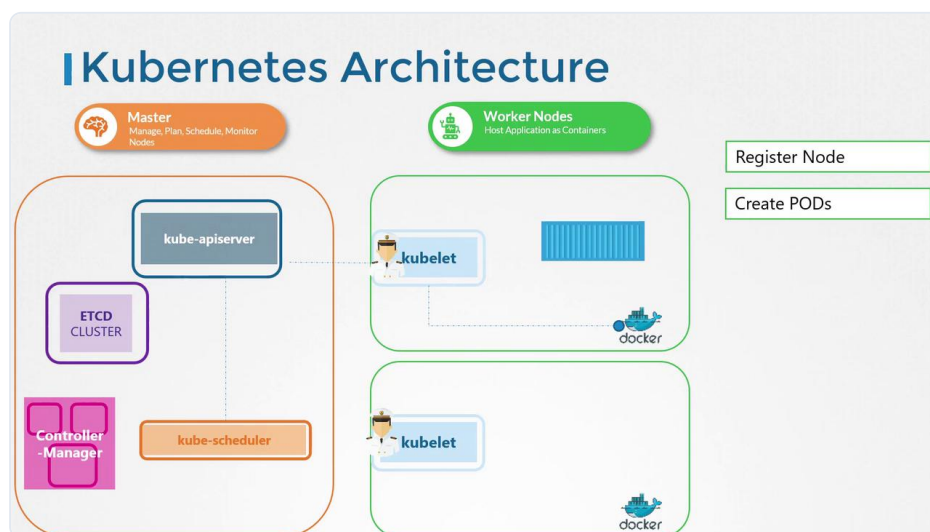
Kubelet

This article covers the Kubelets responsibilities, its role in Kubernetes, and instructions for installing it on worker nodes.

Welcome to this lesson on the Kubelet. In this guide, you will learn about the responsibilities of the Kubelet, its role within the Kubernetes cluster, and how to install it on worker nodes.

The Kubelet is often described as the "captain of the ship." It oversees node activities by managing container operations such as starting and stopping containers based on instructions from the master scheduler. Additionally, the Kubelet registers the node with the Kubernetes cluster and continuously monitors the state of pods and their containers. It regularly reports the status of the node and its workloads to the Kubernetes API server.

When the Kubelet receives instructions to run a container or pod, it communicates with the container runtime (e.g., Docker) to download the required image and initiate the container. It then maintains the health of these containers and ensures they operate as expected.



The Kubelet is essential for node management in Kubernetes, acting as the intermediary between the cluster's control plane and the container runtime.

Installing the Kubelet

Unlike other Kubernetes components, the Kubelet is not automatically deployed when you set up your cluster using tools like kubeadm. It must be installed manually on each worker node. Follow the steps below to install the Kubelet:

Step 1: Download the Kubelet Binary

Execute the following command to download the Kubelet binary:

```
```bash theme={null} wget https://storage.googleapis.com/kubernetes-release/release/v1.13.0/bin/linux/amd64/kubelet
```

#### ### Step 2: Configure and Run the Kubelet as a Service

Set up the Kubelet with the required configuration by running it as a service. Use the command below:

```
```bash theme={null}
ExecStart=/usr/local/bin/kubelet \
  --config=/var/lib/kubelet/kubelet-config.yaml \
  --container-runtime=remote \
  --container-runtime-endpoint=unix:///var/run/containerd/containerd.sock \
  --image-pull-progress-deadline=2m \
  --kubeconfig=/var/lib/kubelet/kubeconfig \
  --network-plugin=cni \
  --register-node=true \
  --v=2
```

Step 3: Verify the Kubelet Process

After installation, verify that the Kubelet is running by checking its process status on the worker node. Run the following command:

```
```bash theme={null} ps -aux | grep kubelet
```

This command will display all active processes containing "kubelet" along with their configuration.

```
```bash theme={null}
root      2095   1.8   2.4 960676 98788 ?        Ssl  02:32   0:36 /usr/bin/kubelet --bootstrap-kub
```

Ensure that all paths and configuration files referenced in your commands exist and are correctly set up on your system.

Next Steps

In the upcoming sections, we will explore additional Kubelet configurations, such as certificate generation and TLS bootstrapping, to further secure and optimize your Kubernetes nodes.

This concludes our lesson on the Kubelet. Continue learning in the next article to deepen your understanding of Kubernetes node management and cluster orchestration.

Happy learning!

Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

Kube Proxy

This article provides an overview of Kube Proxy's role in Kubernetes networking and its function in ensuring reliable communication between pods and Services.

Welcome to this detailed guide on Kube Proxy and its critical role in Kubernetes networking. In this article, you will learn how Kube Proxy ensures reliable communication between pods and how it enables Services to function seamlessly across your cluster.

Pod Networking in Kubernetes

Kubernetes enables every pod within a cluster to communicate with one another by deploying a robust pod networking solution. This creates an internal virtual network that spans all nodes, connecting every pod.

Imagine your web application is running on one node while your database application is on another. Though the web application could connect to the database via its pod IP, these IPs are transient and may change. The recommended solution is to create a Service. By exposing the database through a Service (e.g., using the name "DB"), the web application can maintain a consistent connection without relying on fluctuating pod IPs. Each Service is assigned a stable IP address, and traffic routed to the Service is automatically forwarded to the appropriate backend pod.

A Service in Kubernetes is a virtual entity that doesn't correspond to a container or network interface. Instead, it provides a persistent endpoint in the cluster's memory, allowing stable access to the underlying pods.

How Kube Proxy Works

Kube Proxy is a lightweight process that runs on every node in the Kubernetes cluster. Its key function is to monitor for Service creations and configure

network rules that redirect traffic to the corresponding pods. One common method it uses is by setting up IP tables rules.

For example, if a Service is assigned the IP 10.96.0.12, Kube Proxy configures the IP tables on each node so that any traffic directed to that IP is forwarded to the actual pod IP, such as 10.32.0.15. This redirection mechanism ensures that Services work transparently across the cluster, regardless of which node initiates the request.

Installing Kube Proxy

To get started with Kube Proxy:

1. Download the Kube Proxy binary from the [Kubernetes release page](#).
2. Extract the binary and run it as a service on your nodes.

Typically, when using kubeadm, Kube Proxy is deployed as a DaemonSet. This approach guarantees that a single Kube Proxy pod runs on every node in the cluster.

Using kubeadm, Kube Proxy is automatically managed as a DaemonSet, streamlining the process of ensuring that every node runs a Kube Proxy instance.

Verifying the Kube Proxy Deployment

After installation, you can verify that Kube Proxy is functioning correctly by checking the status of pods and the DaemonSet in the kube-system namespace. Run the following commands:

```
``bash theme={null} kubectl get pods -n kube-system
```

```
``bash theme={null}  
kubectl get daemonset -n kube-system
```

These commands list the running pods and DaemonSets, confirming that Kube Proxy is deployed and operating as expected.

Conclusion

This guide provided a high-level overview of Kube Proxy and its essential role in routing network traffic within Kubernetes. By understanding how Services interact with Kube Proxy, and how the component ensures reliable pod-to-pod communication, you are better prepared to manage and scale your Kubernetes deployments.

Thank you for reading. Stay tuned for more in-depth discussions on Kubernetes networking, Services, and other core components in upcoming modules.

Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

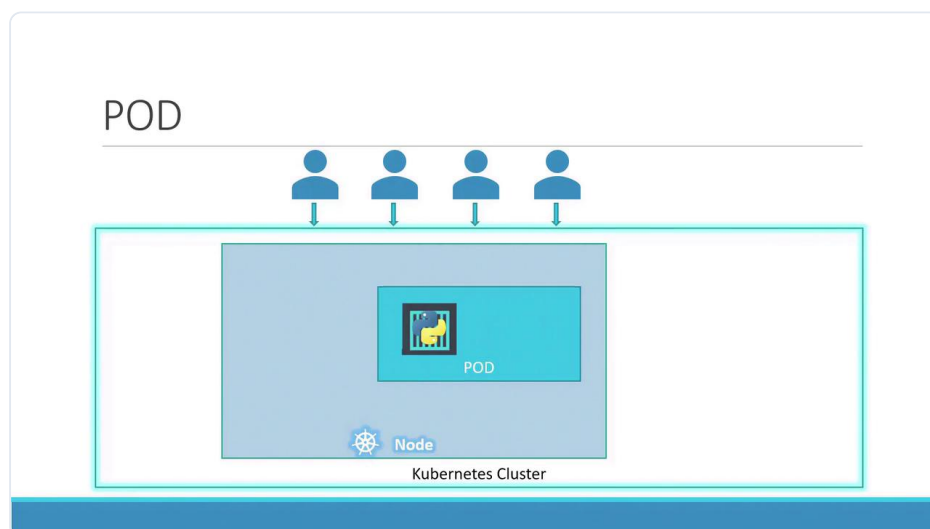
Pods

This article provides an in-depth guide on Kubernetes Pods, covering their deployment, scaling, and management within a Kubernetes cluster.

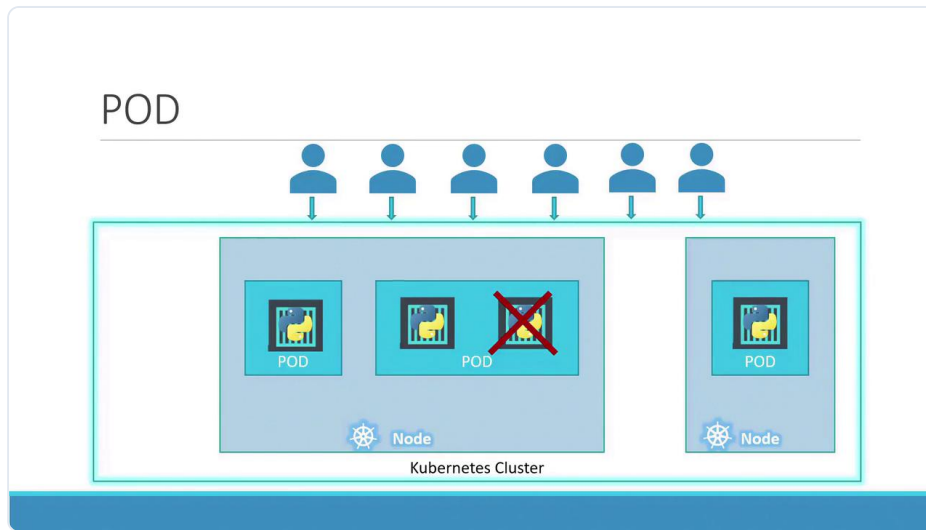
Welcome to this in-depth guide on Kubernetes Pods. In this article, we assume your application is already developed, built into Docker images, and hosted on a Docker repository (such as [Docker Hub](#)). We also assume that your Kubernetes cluster is configured and operational—whether it is a single-node or multi-node cluster. With Kubernetes, the goal is to run containers on worker nodes, but rather than deploying containers directly, Kubernetes encapsulates them within an object called a pod. A pod represents a single instance of an application and is the smallest deployable unit in Kubernetes.

In the simplest scenario, a single-node Kubernetes cluster may run one instance of your application inside a Docker container encapsulated by a pod.

When user load increases, you can scale your application by spinning up additional instances—each running in its own pod. This approach isolates each instance, allowing Kubernetes to distribute the pods across available nodes as needed.

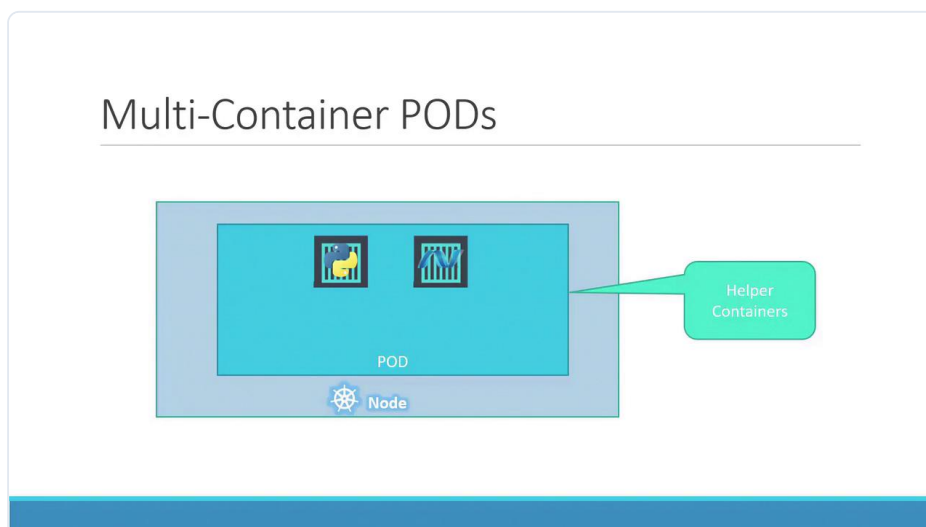


Instead of adding more containers to the same pod, additional pods are created. For instance, running two instances in separate pods allows the load to be shared across the node or even across multiple nodes if the demand escalates and additional cluster capacity is required.



Remember, scaling an application in Kubernetes involves increasing or decreasing the number of pods, not the number of containers within a single pod.

Typically, each pod hosts a single container running your main application. However, a pod can also contain multiple containers, which are usually complementary rather than redundant. For example, you might include a helper container alongside your main application container to support tasks like data processing or file uploads. Both containers in the pod share the same network namespace (allowing direct communication via localhost), storage volumes, and lifecycle events, ensuring they start and stop together.



To better understand the concept, consider a basic Docker example. Suppose you initially deploy your application with a simple command:

```
``bash theme={null} docker run python-app
```


When the load increases, you may launch additional instances manually:

```
```bash theme={null}
docker run python-app
docker run python-app
docker run python-app
docker run python-app
```

Now, if your application needs a helper container that communicates with each instance, managing links, custom networks, and shared volumes manually becomes complex. You'd have to run commands like:

```
```bash theme={null} docker run helper --link app1 docker run helper --link
app2 docker run helper --link app3 docker run helper --link app4
```

With Kubernetes pods, these challenges are resolved automatically. When a pod is defined with

Even if your current application design uses one container per pod, Kubernetes enforces the

Deploying Pods

A common method to deploy pods is using the `kubectl run` command. For example, the following

```
```bash theme={null}
kubectl run nginx --image nginx
```

Once deployed, you can verify the pod's status with the `kubectl get pods` command. Initially, the pod might be in a "ContainerCreating" state, followed by a transition to the "Running" state as the application container becomes active. Below is an example session:

```
```bash theme={null} kubectl get pods
```

Output:

NAME	READY	STATUS	RESTARTS	AGE
------	-------	--------	----------	-----

```
kubectl get pods
```

Output after a few seconds:

NAME	READY	STATUS	RESTARTS	AGE
------	-------	--------	----------	-----

nginx-8586cf59-whssr	1/1	Running	0	8s
----------------------	-----	---------	---	----

...

At this stage, note that external access to the nginx web server has not been configured. The service is accessible only within the node. In a future article, we will explore configuring external access through Kubernetes networking and services.

After mastering pod deployment, advance to networking and service configuration to expose your applications to end users.

That concludes our discussion on Kubernetes Pods. Proceed to the demo section to see these concepts in action, and stay tuned for the next article!

Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

Pods with YAML

This lesson covers creating a Kubernetes Pod using a YAML file, including structure, creation, and status verification.

Welcome to this lesson on creating a Pod in Kubernetes using a YAML configuration file. In this guide, you'll learn how to structure your YAML file, create the Pod with `kubectl`, and verify its status. Kubernetes leverages YAML files to define objects such as Pods, ReplicaSets, Deployments, and Services. These definitions adhere to a consistent structure, with four essential top-level properties: `apiVersion`, `kind`, `metadata`, and `spec`.

Top-Level Fields in a Kubernetes YAML File

Every Kubernetes definition file must include the following four fields:

```
```YAML theme={null} apiVersion: kind: metadata: spec:
```

1. **apiVersion**

This field indicates the version of the Kubernetes API you are using. For a Pod, set `apiVersion: v1`.

2. **kind**

This specifies the type of object being created. In this lesson, since we're creating a Pod, set `kind: Pod`.

3. **metadata**

The metadata section provides details about the object, including its name and labels.

```
```yaml
theme={null}
apiVersion: v1
kind: Pod
metadata:
  name: myapp-pod
  labels:
    app: myapp
spec:
  ```
```

<Callout icon="lightbulb" color="#1CB2FE">

Make sure that the properties under metadata (like name and labels) are indented to the same level. This is crucial for correct YAML parsing.

</Callout>

4. **spec**

The spec section provides specific configuration details for the object. For a Pod, this includes the containers to be run.

Below is the complete YAML configuration for our Pod:

```
```yaml
theme={null}
apiVersion: v1
kind: Pod
metadata:
  name: myapp-pod
  labels:
    app: myapp
    type: front-end
spec:
  containers:
    - name: nginx-container
      image: nginx
  ```
```

## Creating and Verifying the Pod

After you have saved your configuration (for example, as `pod-definition.yaml`), use the following command to create the Pod:

```
```bash
theme={null}
kubectl create -f pod-definition.yaml
```
```

Once the Pod is created, you can verify its status by listing all Pods:

```
```bash
theme={null}
kubectl get pods
```
```

You should see output similar to this:

```
``plaintext theme={null} NAME READY STATUS RESTARTS AGE myapp-pod
1/1 Running 0 20s
```

To view detailed information about the Pod, run:

```
```bash theme={null}
kubectl describe pod myapp-pod
```

This command provides extensive details, including metadata, node assignment, container specifics, and event history such as scheduling, volume mounting, and container start-up. Here is an example output:

```
``plaintext theme={null} Name: myapp-pod Namespace: default Node:
minikube/192.168.99.100 Start Time: Sat, 03 Mar 2018 14:26:14 +0800
Labels: app=myapp Annotations: Status: Running IP: 172.17.0.24 Containers:
nginx: Container ID: docker://
830bb56c8c42a860b4b70e9c1488faelbc38663e49186bc2f5a78e7688b8c9d
Image: nginx Image ID: docker-pullable://
nginx@sha256:4771d09578c7ca65299e110b3ee1c0a2592f5ea2618d32e4ffe7a4cab1c5de
Port: State: Running Started: Sat, 03 Mar 2018 14:26:21 +0800 Ready: True
Restart Count: 0 Environment: Mounts: /var/run/secrets/kubernetes.io/
serviceaccount from default-token-x95w7 (ro)
```

Conditions: Type Status Initialized True Ready True PodScheduled True

```
Events: Type Reason Age From Message ----
-----
Normal Scheduled 27s default-scheduler Successfully assigned myapp-pod to minikube
Normal SuccessfulMountVolume 27s minikube MountVolume.SetUp succeeded for volume "default-token-x95w7"
Normal Pulling 27s minikube pulling image "nginx"
Normal Pulled 27s minikube Successfully pulled image "nginx"
Normal Created 27s minikube Created container
Normal Started 27s minikube Started container ````
```

Using `kubectl describe` helps you gain detailed insights into the internal state of your Pod, which can be invaluable for troubleshooting.

Conclusion

In this lesson, you learned how to structure a Kubernetes YAML file for a Pod, create it using `kubectl`, and verify its status. This hands-on approach equips you to manage and troubleshoot your Kubernetes resources effectively. Happy Kubernetes-ing!

For more information, refer to the following resources:

- [Kubernetes Documentation](#)
- [Kubernetes Basics](#)

Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

Demo Pods with YAML

This article demonstrates how to create a Kubernetes Pod using a YAML definition file for better control over pod specifications.

In this lesson, we will create a Kubernetes Pod using a YAML definition file instead of the "kubectl run" command. This method offers more control by allowing you to define pod specifications explicitly in a file. You can choose any text editor for this task; for instance, Windows users may prefer Notepad++ over Notepad, while Linux users might opt for vim. In future sections, we will explore additional IDEs and tools to streamline YAML editing, but we will stick with the basics for now.

Step 1: Creating the YAML File

Open your terminal and use vim to create a file named pod.yaml:

```
```bash theme={null} vim pod.yaml
```

Inside the file, define the following key elements:

- \* **apiVersion:** Should be set to `v1` for a Pod.
- \* **kind:** Must be `Pod` (case-sensitive).
- \* **metadata:** A dictionary that includes the pod's name and any labels used for grouping.
- \* **spec:** Contains the pod specifications, including a list of containers.

<Callout icon="lightbulb" color="#1CB2FE">

Be sure to follow proper indentation rules. Use two spaces per level (avoid tabs), as misalignment can lead to errors.

</Callout>

Below is a complete example configuration for a single-container Pod using the `nginx` image.

```
```yaml theme={null}
apiVersion: v1
kind: Pod
metadata:
  name: nginx
  labels:
    app: nginx
    tier: frontend
spec:
  containers:
    - name: nginx
      image: nginx
```

To add additional containers, insert another block within the containers list with the appropriate name and image.

Step 2: Saving and Verifying the YAML File

After editing the file, exit vim and save your changes by typing:

```
```bash theme={null} :wq
```

Verify the contents of your YAML file with:

```
```bash theme={null}
cat pod.yaml
```

The output should match the YAML configuration shown above.

Step 3: Creating the Pod in the Cluster

Create the Pod on your Kubernetes cluster using your YAML file. You can use either the `kubectl create` or `kubectl apply` command. Here's an example with `kubectl apply`:

```
```bash theme={null} kubectl apply -f pod.yaml
```



# Output:

## pod/nginx created

To check the status of your Pod, run:

```
```bash theme={null}
kubectl get pods
```

Initially, you might see an output similar to this:

```
```bash theme={null} NAME READY STATUS RESTARTS AGE nginx 0/1
ContainerCreating 0 7s
```

After a short while, re-running the command should show the Pod in a running state:

```
```bash theme={null}
kubectl get pods
```

Example output:

```
```bash theme={null} NAME READY STATUS RESTARTS AGE nginx 1/1
Running 0 9s
```

## Step 4: Inspecting the Pod Details

For a detailed overview of your Pod, use the `kubectl describe` command:

```
```bash theme={null}
kubectl describe pod nginx
```

This command provides comprehensive details about the Pod, including container statuses, event logs, volumes, and node assignments. Below is an example of typical output:

```
bash theme={null} Initialized True Ready True ContainersReady True
PodScheduled True Volumes: default-token-f5ntk: Type: Secret (a
volume populated by a Secret) SecretName: default-token-f5ntk
Optional: false QoS Class: BestEffort Node-Selectors: <none>
Tolerations: node.kubernetes.io/not-ready:NoExecute for 300s
node.kubernetes.io/unreachable:NoExecute for 300s Events: Type
Reason Age From Message ---- -
Scheduled 21s default-scheduler Successfully assigned default/nginx
to minikube Normal Pulling 20s kubelet, minikube Pulling image
"nginx" Normal Pulled 14s kubelet, minikube Successfully pulled
```

```
image "nginx" Normal Created 14s kubelet, minikube Created container  
nginx Normal Started 14s kubelet, minikube Started container nginx
```

Conclusion

This demonstration has guided you through creating a Kubernetes Pod using a YAML configuration file. This approach not only reinforces good configuration practices but also provides enhanced flexibility compared to command-based object creation. In our next lesson, we will cover advanced IDEs and tools to further ease YAML file management.

For additional reading and resources, check out:

- [Kubernetes Documentation](#)
- [Kubernetes Basics](#)

Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

Practice Test Introduction

This article introduces a practice test designed to help users prepare for an exam in a realistic lab environment.

Before you begin the practice test, take a moment to familiarize yourself with the practice test portal. Once the lecture is completed, you will be directed to the lab environment where you'll complete various hands-on exercises.

This practice test is designed to help you prepare for the exam by testing your skills in a realistic lab setting. Please use a laptop or desktop computer with a keyboard for the best experience.

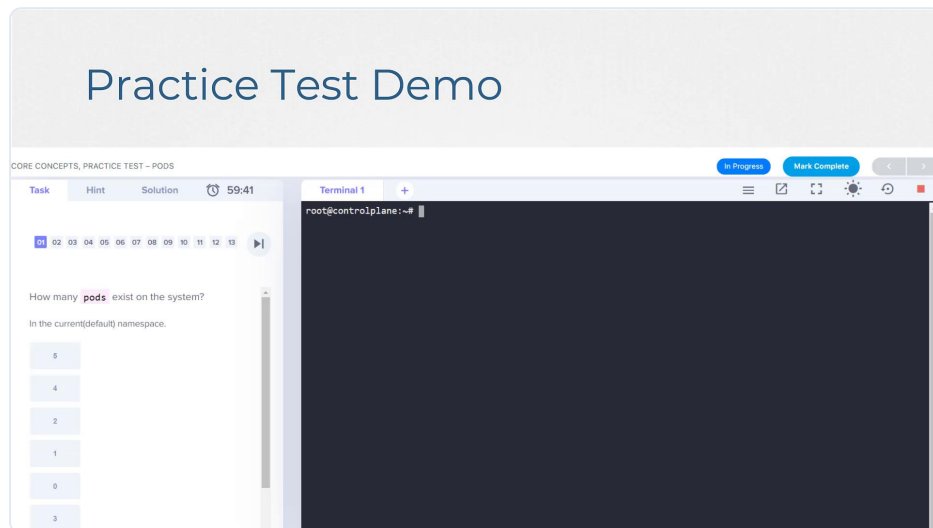
Lab Environment Overview

Click the start button to load the lab environment. Allow a moment for the lab to initialize—typically less than 30 seconds, though occasionally it may take a few minutes.

Once loaded, you will see two main sections:

- **Quiz Portal (Left Side):** Hosts your test questions.
- **Live Terminal (Right Side):** Where you execute commands. This terminal might be a Linux shell for Linux, shell scripting, or Git tasks; a Docker host for Docker tasks; or a Kubernetes control plane for Kubernetes tasks. You can run commands here to interact with and configure the system.

Below is an image illustrating the practice test interface:



Structure of the Practice Test

The lab is divided into two parts:

1. **Terminal:** Use this section on the right to run commands and interact with the system (e.g., view and configure a Kubernetes cluster).
2. **Quiz Portal:** Located on the left, this portal contains practice test questions grouped by topic. Each practice test includes between 5 to 15 questions. The total number of questions is shown at the top, and each question is marked as you progress.

Question Types

The test includes a variety of question types for a comprehensive learning experience:

- **Environment-Based Questions:** Find information within the lab environment to select the correct answer.
- **Configuration Tasks:** Modify the environment by deploying Pods, services, or other components, then verify the changes.
- **Specification Matching:** Given a configuration specification (spec), create a matching configuration using the provided editors (e.g., the vi editor).
- **Multiple-Choice Questions:** Some questions are scenario-based to test your conceptual understanding. Note that these might not appear on the actual certification exam.

If a configuration does not exactly match the given specification, adjust your settings precisely. You can skip questions by clicking the skip button at the top; however, once skipped, these questions cannot be revisited due to the fixed question sequence.

Accessing Web Applications

For certain deployments, you might have a web portal link available above the terminal. This link is provided if the application features a web interface. For instance, when exposing a service, you might run the following command:

```
bash theme={null} root@controlplane:~# kubectl expose pod nginx --port 80 --type NodePort service/nginx exposed root@controlplane:~#
```

If you need an additional terminal window, use the designated button at the top.

Please note that this lab environment is not an exact replica of the actual Kubernetes certification exam environment. It is custom-built to help you practice and prepare. All work will be deleted after one hour (or the specified lab session duration), so do not store any personal credentials or persistent data here.

Retaking the Test and Session Guidelines

You can retake these tests as many times as needed until you feel confident. However, remember:

- The lab environment is temporary and will expire after one hour (or as specified).
- Refreshing the lab window or experiencing temporary connectivity issues should return you to the same environment.
- Extended browser closures or lengthy idle periods may result in the lab being automatically terminated. In such cases, you will need to restart a new lab session from the beginning.

Below is an image that reminds you of the temporary nature of the lab environment:

Please Note

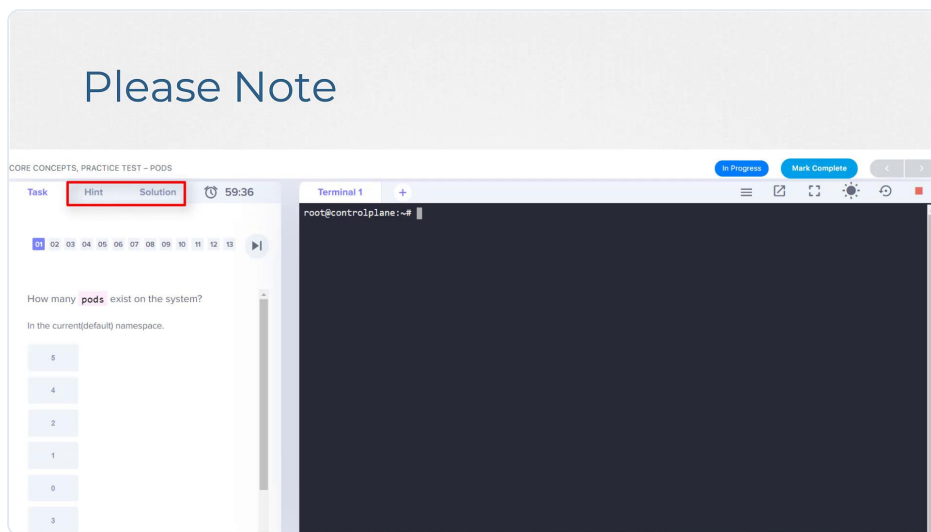
- This is **not** the interface of the actual exam
- As many times as you like
- Temporary Environment – Only available for **1 Hour**
- **No** Personal details

Practice and Learning

These practice tests are designed not only to assess your skills based on the course lecture but also to help you gain essential hands-on experience. Some questions may cover topics beyond the lecture material, so always refer to the documentation and search for solutions if you are stuck. The hints and solutions panel, visible with each question, is provided to guide you.

If any task or question remains unclear, please reach out for further assistance. The goal is to encourage you to read the documentation, find necessary information, and apply it successfully.

Below is an additional image showing a Kubernetes practice test interface:



For further learning, consider exploring:

- [Kubernetes Documentation](#)
- [Kubernetes Basics](#)
- [Docker Hub](#)

Good luck with your practice test and exam preparation!

Documentation Index

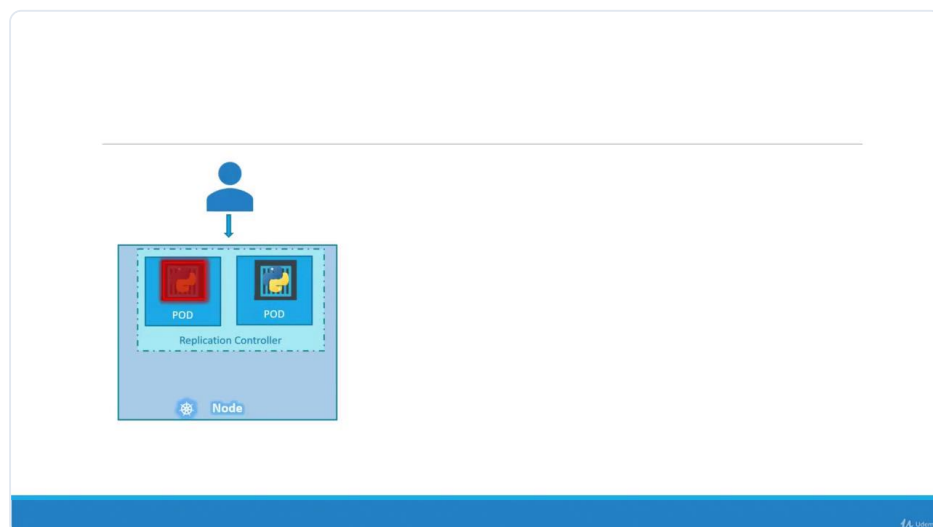
Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

ReplicaSets

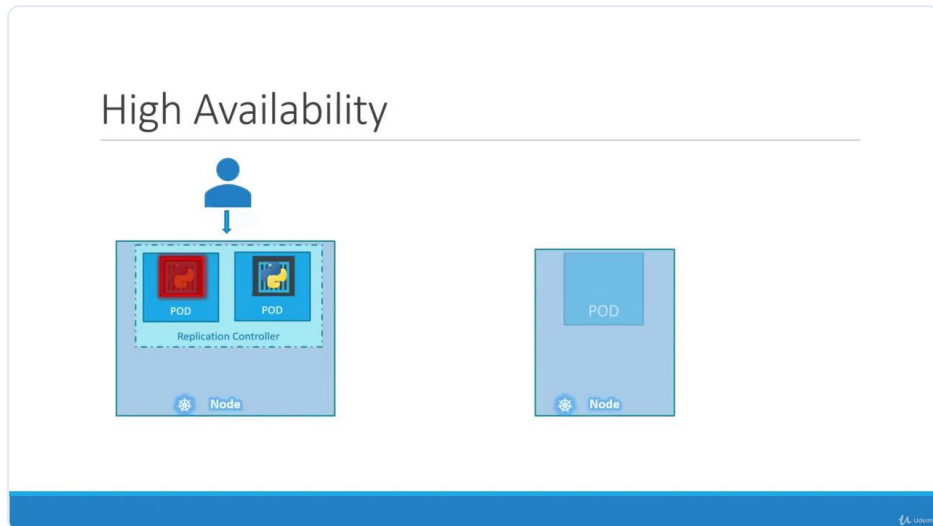
This article explains Kubernetes replication controllers and ReplicaSets, focusing on their roles in maintaining high availability and load balancing in clusters.

Hello, and welcome to this lesson on Kubernetes controllers. I'm Mumshad Mannambeth, and today we'll dive into the essential components that drive Kubernetes operations. Kubernetes controllers continuously monitor objects and take necessary actions, and in this lesson, we focus on the replication controller—an essential building block for maintaining high availability in your cluster.

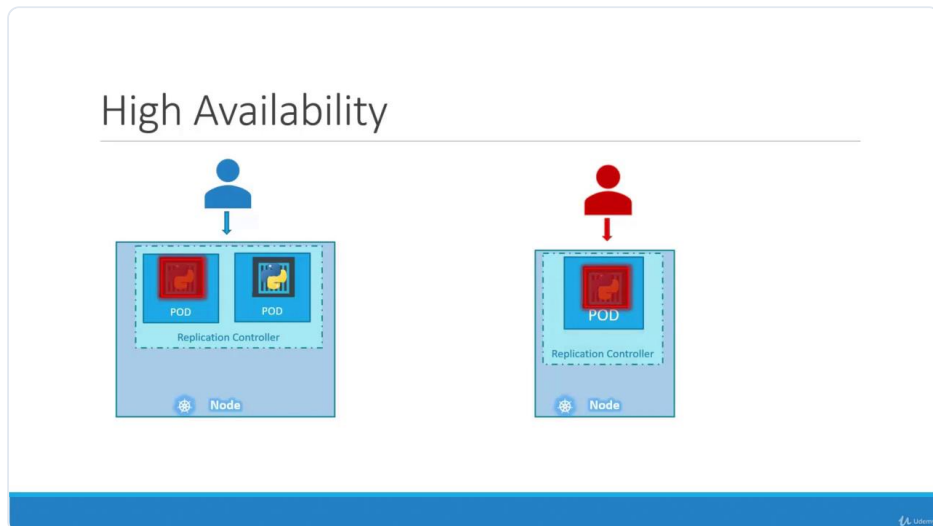
Imagine a scenario where a single pod runs your application. If that pod crashes or fails, users lose access. To prevent this risk, running multiple pod instances is key. A replication controller ensures high availability by creating and maintaining the desired number of pod replicas. Even if you intend to run a single pod, a replication controller adds redundancy by automatically creating a replacement if the pod fails.



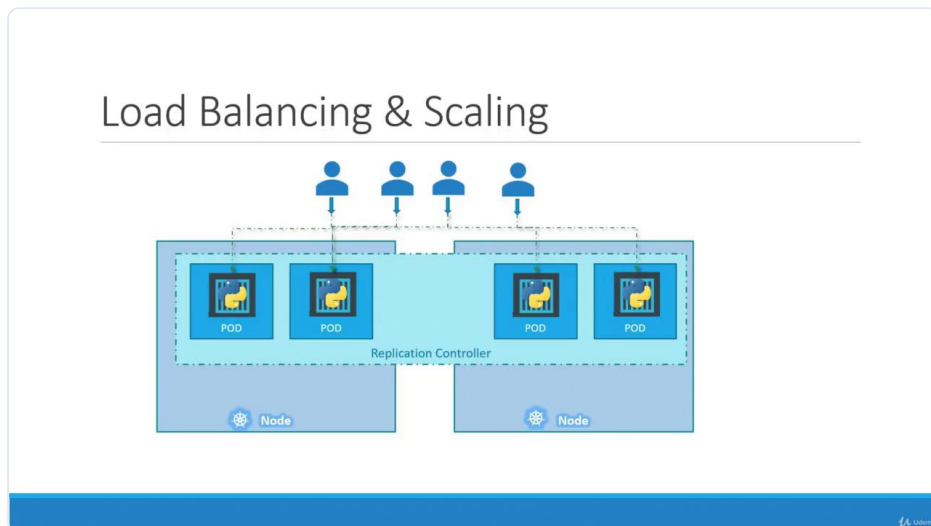
If one pod serving your application crashes, the replication controller immediately deploys a new one to keep the service available.



For example, if you need to maintain a constant service level, the controller ensures the desired number of pods—whether one or one hundred—are always running.



Beyond availability, replication controllers also help distribute load. When user demand increases, additional pods can better balance that load. If resources on a particular node become scarce, new pods can be scheduled across other nodes in your cluster.



While both replication controllers and replica sets serve similar purposes, the replication controller is the older technology being gradually replaced by the replica set. In this lesson, we will focus on replica sets for our demos and implementations.

Creating a Replication Controller

To create a replication controller, start by writing a configuration file (e.g., `rc-definition.yaml`). Like any Kubernetes manifest, the file contains four main sections: `apiVersion`, `kind`, `metadata`, and `spec`.

1. **apiVersion:** For a replication controller, use `v1`.
2. **kind:** Set this to `ReplicationController`.
3. **metadata:** Provide a name (e.g., `myapp-rc`) and include labels such as `app` and `type`.
4. **spec:** This section is crucial. It not only defines the desired number of replicas with the `replicas` key but also includes a `template` section which serves as the blueprint for creating the pods. Ensure that all pod-related entries in the template are indented correctly and aligned with `replicas` as siblings.

Once your YAML file is ready, create the replication controller using the following command:

```
``bash theme={null} kubectl create -f rc-definition.yml
```

Below is a complete example of a replication controller definition:

```
```yaml theme={null}
apiVersion: v1
kind: ReplicationController
metadata:
 name: myapp-rc
 labels:
 app: myapp
 type: front-end
spec:
 replicas: 3
 template:
 metadata:
 name: myapp-pod
 labels:
 app: myapp
 type: front-end
 spec:
 containers:
 - name: nginx-container
 image: nginx
```

When you run the following command, Kubernetes creates three pods according to the provided template:

```
```bash theme={null} kubectl create -f rc-definition.yml
```

Output:

replicationcontroller "myapp-rc" created

To view the replication controller and its pods, run these commands

```
```bash theme={null}
kubectl get replicationcontroller
kubectl get pods
```

A sample output might look like:

```
```bash theme={null}
```

```
kubectl get replicationcontroller NAME DESIRED CURRENT READY
AGE myapp-rc 3 3 3 19s
```

```
kubectl get pods NAME READY STATUS RESTARTS AGE myapp-rc-4lvk9 1/1 Running 0 20s myapp-rc-mc2mf 1/1 Running 0 20s myapp-rc-px9pz 1/1 Running 0 20s
```

Notice that the pods' names include the replication controller's name ("myapp-rc"), indicating

Introducing ReplicaSet

A ReplicaSet is a modern alternative to the replication controller, using an updated API version.

1. **API Version**: Use `apps/v1` for a ReplicaSet.
2. **Selector**: In addition to metadata and specification, a ReplicaSet requires a `selector` field.

Below is an example ReplicaSet definition:

```
```yaml theme={null}
apiVersion: apps/v1
kind: ReplicaSet
metadata:
 name: myapp-replicaset
 labels:
 app: myapp
 type: front-end
spec:
 replicas: 3
 selector:
 matchLabels:
 type: front-end
 template:
 metadata:
 name: myapp-pod
 labels:
 app: myapp
 type: front-end
 spec:
 containers:
 - name: nginx-container
 image: nginx
```

Create the ReplicaSet with:

```
```bash theme={null} kubectl create -f replicaset-definition.yml
```

Then, verify its creation:

```
```bash theme={null}
kubectl get replicaset
```

And view the associated pods:

```
``bash theme={null} kubectl get pods
```

```

```

```
Labels and Selectors
```

Labels in Kubernetes are critical because they enable controllers, such as ReplicaSets, to

```
``yaml theme={null}
```

```
selector:
 matchLabels:
 tier: front-end
```

The pod definition should similarly include the label:

```
yaml theme={null} metadata: name: myapp-pod labels: tier: front-end
```

This label-selector mechanism ensures that the ReplicaSet precisely targets the intended pods and maintains the set number of replicas by replacing any failed pods.

---

## Is the Template Section Required?

Even if three pods with matching labels already exist in your cluster, the template section in the ReplicaSet specification remains essential. It serves as the blueprint for creating new pods if any fail, ensuring the desired state is consistently maintained.

---

## Scaling the ReplicaSet

Scaling a ReplicaSet involves adjusting the number of pod replicas. There are two methods to achieve this:

### 1. Update the Definition File

Modify the `replicas` value in your YAML file (e.g., change from 3 to 6) and update the ReplicaSet with:

```
bash theme={null} kubectl replace -f replicaset-definition.yml
```

### 1. Use the `kubectl scale` Command

Scale directly from the command line:

```
bash theme={null} kubectl scale --replicas=6 -f replicaset-definition.yml
```

Keep in mind that if you scale using the `kubectl scale` command, the YAML file still reflects the original number of replicas. To maintain consistency, it may be necessary to update the YAML file after scaling.

---

## Common Commands Overview

---

Below is a quick reference table summarizing some useful commands when working with replication controllers and ReplicaSets:

Resource Type	Use Case	Example Command
Create Object	Create from a definition file	<code>kubectl create -f &lt;filename&gt;</code>
View ReplicaSets/RC	List replication controllers	<code>kubectl get replicaset</code> or <code>kubectl get replicationcontroller</code>
Delete ReplicaSet/RC	Remove a replication controller	<code>kubectl delete replicaset &lt;replicaset-name&gt;</code>
Update Definition	Replace object using YAML file	<code>kubectl replace -f &lt;filename&gt;</code>
Scale ReplicaSet/RC	Change number of replicas	<code>kubectl scale --replicas=&lt;number&gt; -f &lt;filename&gt;</code>

---

That concludes our lesson on ReplicaSets and replication controllers in Kubernetes. Understanding these concepts is vital for managing high availability and load balancing in your cluster. Happy learning!

For further reading, check out [Kubernetes Documentation](#).

## Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

# Deployments

This guide explores Kubernetes deployments, simplifying application management with features like rolling updates, rollbacks, and high availability.

Hello and welcome! My name is Mumshad Mannambeth. In this guide, we dive into Kubernetes deployments—an abstraction that simplifies managing your applications in a production environment. Rather than interacting directly with pods and ReplicaSets, deployments offer advanced features that enable you to:

- Deploy multiple instances of your application (like a web server) to ensure high availability and load balancing.
- Seamlessly perform rolling updates for Docker images so that instances update gradually, reducing downtime.
- Quickly roll back to a previous version if an upgrade fails unexpectedly.
- Pause and resume deployments, allowing you to implement coordinated changes such as scaling, version updates, or resource modifications.

Previously, we discussed how individual pods encapsulate containers and how ReplicaSets maintain multiple pod copies. A deployment, however, sits at a higher level, automatically managing ReplicaSets and pods while providing enhanced features like rolling updates and rollbacks.

## Creating a Deployment

To create a deployment, start by writing a deployment definition file. This file is similar to a ReplicaSet definition, with the key difference being that the kind is set to Deployment instead of ReplicaSet. Below is an example of a correct deployment definition file:

```
```yaml
theme={null}
apiVersion: apps/v1
kind: Deployment
metadata:
  name: myapp-deployment
  labels:
    app: myapp
    type: front-end
spec:
  replicas: 3
```

```
selector: matchLabels: type: front-end template: metadata: labels: app: myapp
type: front-end spec: containers: - name: nginx-container image: nginx
```

```
Once your deployment definition file (for example, named deployment-definition.yml) is ready,
run the following command to create the deployment:
```

```
```bash theme={null}
kubectl create -f deployment-definition.yml
```

The command output should confirm that the deployment has been created:

```
```console theme={null} deployment "myapp-deployment" created
```

To verify the deployment, run:

```
```bash theme={null}
kubectl get deployments
```

The output will look similar to this:

```
```console theme={null} NAME DESIRED CURRENT UP-TO-DATE
AVAILABLE AGE myapp-deployment 3 3 3 21s
```

```
## Behind the Scenes: How Deployments Work
```

When you create a deployment, Kubernetes automatically creates an associated ReplicaSet. To

```
```bash theme={null}
kubectl get replicaset
```

You'll notice a new ReplicaSet with a name derived from your deployment. This ReplicaSet oversees the creation and management of pods. To view the pods managed by the ReplicaSet, run:

```
```bash theme={null} kubectl get pods
```

While deployments and ReplicaSets work together seamlessly, deployments provide additional

<Callout icon="lightbulb" color="#1CB2FE">

To view all the created Kubernetes objects—deployments, ReplicaSets, pods, and more—use t

```
```bash theme={null}
kubectl get all
```
```

This gives you a comprehensive overview of your deployment's components.

</Callout>

A sample output of the "kubectl get all" command might be:

```
```console theme={null}
NAME DESIRED CURRENT UP-TO-DATE AVAILABLE AGE
deploy/myapp-deployment 3 3 3 3 9h

NAME DESIRED CURRENT READY AGE
rs/myapp-deployment-6795844b58 3 3 3 9h

NAME READY STATUS RESTARTS AGE
po/myapp-deployment-6795844b58-5rbjl 1/1 Running 0 9h
po/myapp-deployment-6795844b58-h4w55 1/1 Running 0 9h
po/myapp-deployment-6795844b58-1fjhv 1/1 Running 0 9h
```

In this output, you can clearly see the deployment, its associated ReplicaSet, and the managed pods.

## Conclusion

This article has covered the fundamentals of creating a deployment in Kubernetes. By leveraging deployments, you gain powerful capabilities like rolling updates and rollbacks that make managing application updates and maintenance in production more efficient. Whether you are scaling your application or rolling out new features, Kubernetes deployments provide a robust solution for modern application management.

Happy deploying!



## Documentation Index

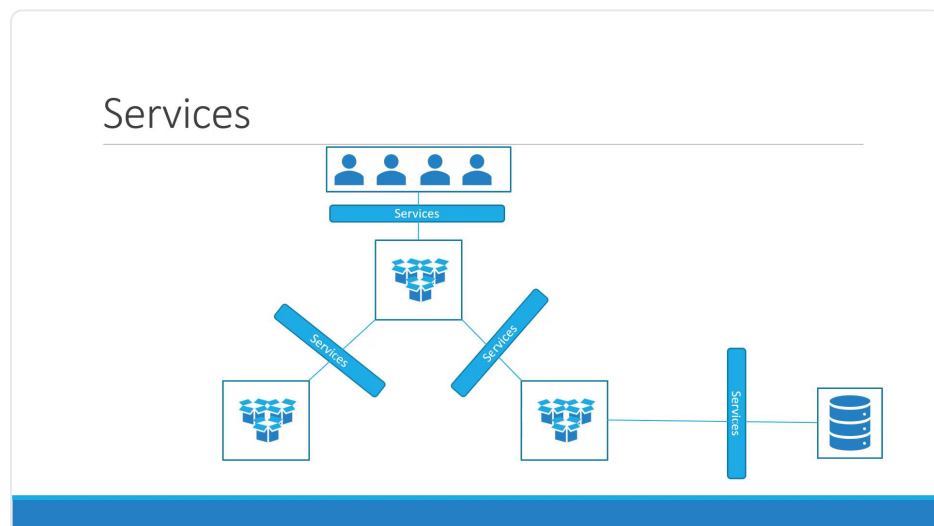
Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

# Services

This guide explores Kubernetes services, focusing on enabling communication between application components and detailing the creation and use of NodePort services.

Hello, and welcome to this in-depth guide on Kubernetes Services for Beginners. My name is Mumshad Mannambeth, and in this article, we'll explore how Kubernetes services enable seamless communication between various application components—both within the cluster and from the outside world.

Kubernetes services allow different sets of Pods to interact with each other. Whether connecting the front end to back-end processes or integrating external data sources, services help to decouple microservices while maintaining reliable communication. For instance, you can expose your front-end to end users and enable back-end components to interact efficiently.



## Use Case: From Internal Networking to External Access

---

So far, we've seen how Pods communicate internally using the Kubernetes network. Consider a scenario where you deploy a Pod running a web application and want an external user to access it. Here's a quick overview of the setup:

- **Kubernetes Node IP:** 192.168.1.2
- **Laptop IP (same network):** 192.168.1.10
- **Internal Pod Network:** 10.244.0.0
- **Pod IP:** 10.244.0.2

Since the Pod is on an isolated internal network, direct access to 10.244.0.2 from your laptop isn't possible. One workaround is to SSH into the Kubernetes node (192.168.1.2) and use `curl` to reach the Pod:

```
``bash theme={null} curl http://10.244.0.2 Hello World!
```

While this method works from the node, the goal is to have external access directly from your laptop.

For example:

```
``bash theme={null}
curl http://192.168.1.2:30008
Hello World!
```

This configuration externally exposes the web server running inside the Pod.

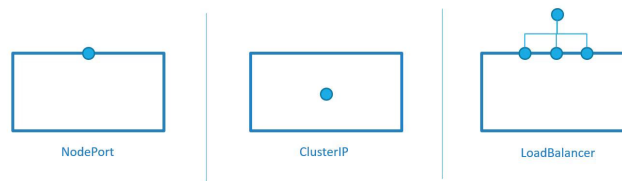
## Types of Kubernetes Services

---

Kubernetes supports several service types, each serving a unique purpose:

- **NodePort:** Maps a port on the node to a port on a Pod.
- **ClusterIP:** Creates a virtual IP for internal communication between services (e.g., connecting front-end to back-end servers).
- **LoadBalancer:** Provisions an external load balancer (supported in cloud environments) to distribute traffic across multiple Pods.

## Services Types



Remember: The NodePort service type maps a specific node port (e.g., 30008) to the target port on your Pod (e.g., 80). This provides external access while keeping internal port targeting intact.

### NodePort Service Breakdown

With a NodePort service, there are three key ports to consider:

1. **Target Port:** The port on the Pod where the application listens (e.g., 80).
2. **Port:** The virtual port on the service within the cluster.
3. **NodePort:** The external port on the Kubernetes node (by default in the range 30000–32767).

## Creating a NodePort Service

The process of creating a NodePort service begins with defining the service in a YAML file. The definition file follows a similar structure to those used for Deployments or ReplicaSets, including API version, kind, metadata, and spec.

Below is an example YAML file that defines a NodePort service:

```
``yaml theme={null} apiVersion: v1 kind: Service metadata: name: myapp-service spec: type: NodePort ports: - targetPort: 80 port: 80 nodePort: 30008
```

In this YAML:

- \* `targetPort` specifies the Pod's application port.
- \* `port` is the port on the service that acts as a virtual server port within the cluster.
- \* `nodePort` maps the external request to the specific port on the node (ensure it's between 30000 and 32767).

Note that if you omit `targetPort`, it defaults to the same value as `port`. Similarly, if you omit `port`, it defaults to the same value as `targetPort`.

However, this YAML definition does not link the service to any Pods. To connect the service to the Pods, you need to define a selector.

```
```yaml theme={null}
apiVersion: v1
kind: Pod
metadata:
  name: myapp-pod
  labels:
    app: myapp
    type: front-end
spec:
  containers:
    - name: nginx-container
      image: nginx
```
```

Now, update the service definition to include a selector that matches these labels:

```
```yaml theme={null} apiVersion: v1 kind: Service metadata: name: myapp-service spec: type: NodePort ports: - targetPort: 80 port: 80 nodePort: 30008 selector: app: myapp type: front-end
```
```

Save the file as `service-definition.yml` and create the service using:

```
```bash theme={null}
kubectl create -f service-definition.yml
```
```

You should see a confirmation message:

```
```bash theme={null} service "myapp-service" created
```
```

Verify the service details with:

```
```bash theme={null}
kubectl get services
```
```

An example output might be:

```
```bash theme={null} NAME TYPE CLUSTER-IP EXTERNAL-IP PORT(S) AGE
kubernetes ClusterIP 10.96.0.1 443/TCP 16d myapp-service NodePort 10.106.127.123 80:30008/TCP 5m
```
```

```
Access the web service externally by pointing your browser or using curl with the node IP

```bash theme={null}
curl http://192.168.1.2:30008
```

A typical response from an Nginx server might be:

```
```html theme={null}
```

... `` ## Kubernetes Services in Production In a production environment, your application is likely spread across multiple Pods for high availability and load balancing. When Pods share matching labels, the service automatically detects and routes traffic to all endpoints. Kubernetes employs a round-robin (or random) algorithm to distribute incoming requests, serving as an integrated load balancer. Furthermore, even if your Pods are spread across multiple nodes, Kubernetes ensures that the target port is mapped on all nodes. This means you can access your web application using the IP of any node along with the designated NodePort, providing reliable external connectivity. [The image illustrates a Kubernetes NodePort service setup, showing nodes with IPs, ports, and pods, enabling external access to services within a cluster.] ([https://kodekloud.com/kk-media/image/upload/v1752869747/notes-assets/images/CKA-Certification-Course-Certified-Kubernetes-Administrator-Services/frame\\_750.jpg](https://kodekloud.com/kk-media/image/upload/v1752869747/notes-assets/images/CKA-Certification-Course-Certified-Kubernetes-Administrator-Services/frame_750.jpg)) Regardless of whether your application runs on a single Pod on one node, multiple Pods on a single node, or Pods spread across several nodes, the service creation process remains consistent. Kubernetes automatically updates the service endpoints when Pods are added or removed, ensuring a flexible and scalable infrastructure. ## Summary This article has provided a comprehensive introduction to Kubernetes NodePort services, covering the following key points: \* The purpose and importance of Kubernetes services in enabling both internal and external communications. \* A detailed explanation of how NodePort services work and the roles of targetPort, service port, and nodePort. \* Step-by-step instructions on creating a NodePort service and linking it to your Pods via selectors. \* An overview of production scenarios where multiple Pods ensure high availability and load balancing. Thank you for reading this guide on Kubernetes NodePort services. Explore the demo, and I look forward to sharing more Kubernetes insights in future articles. For further reading, check out: \* [Kubernetes Documentation] (<https://kubernetes.io/docs/>) \* [Kubernetes Basics] (<https://kubernetes.io/docs/concepts/overview/what-is-kubernetes/>) \* [Docker Hub] (<https://hub.docker.com/>)

## Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

# Services Cluster IP

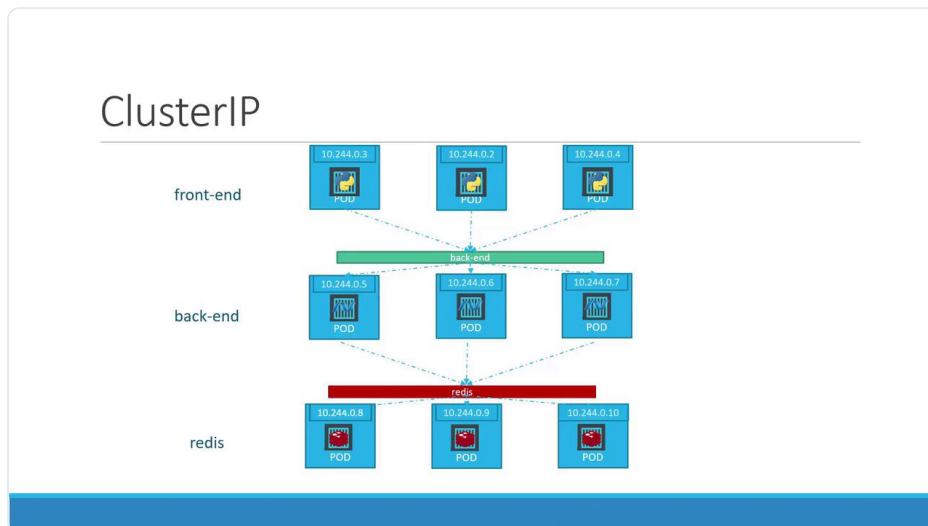
This article explains how Kubernetes Service Cluster IP facilitates stable pod-to-pod communication in microservices-based applications.

Welcome to this lesson on Kubernetes Service Cluster IP. In this guide, we explain how Cluster IP streamlines connectivity within a full-stack web application by providing a stable interface for pod-to-pod communication.

A typical microservices-based application consists of several pods. Some pods host a front-end web server, while others run a back-end server; additional pods manage services like a key-value store using Redis or persistent databases like MySQL. The front-end pods need to communicate with the back-end services, and the back-end servers must interact with databases and caching mechanisms.

Because pods receive dynamic IP addresses that can change when they are recreated, relying on these IPs for internal communication is impractical. Moreover, when a front-end pod (for example, with IP 10.244.0.3) needs to connect to a back-end service, there arises the issue of determining which pod should handle the request. Kubernetes solves this challenge by grouping related pods under a single service. This service provides a fixed Cluster IP or a service name, allowing other pods to access them without worrying about individual IPs. The service automatically load-balances incoming requests among the available pods.

For instance, by creating a service for the back-end pods, you can group them together under one interface. Similarly, services can be set up for Redis or other application tiers, ensuring that each layer can scale independently without disrupting internal connectivity.



Each service in Kubernetes is automatically assigned an IP and DNS name within the cluster. This Cluster IP should be used by other pods when accessing the service, ensuring consistent and reliable connectivity.

## Example: "back-end" Service

Below is a sample YAML configuration for creating a service named "back-end". This service exposes port 80 on the Cluster IP, forwarding requests to the back-end pods that match the specified labels ( `app: myapp` and `type: back-end` ). The `targetPort` is set to 80, matching the port where the back-end container listens:

```
```yaml
theme={null}
apiVersion: v1
kind: Service
metadata:
  name: back-end
spec:
  type: ClusterIP
  ports:
    - port: 80
      targetPort: 80
  selector:
    app: myapp
    type: back-end
```
```

To create the service, run the following command:

```
```bash
theme={null}
kubectl create -f service-definition.yml
```
```

After deploying the service, verify its status with:

```
```bash
theme={null}
kubectl get services
```
```

The output should resemble the following:

```
```plaintext
theme={null}
NAME          TYPE        CLUSTER-IP    EXTERNAL-IP    PORT(S)
kubernetes    ClusterIP   10.96.0.1     <none>         443/TCP
back-end      ClusterIP   10.106.127.123 <none>         80/TCP
```
```

With this setup, components of your application can access the back-end service using either its Cluster IP or its DNS service name, ensuring uninterrupted connectivity even as individual pods scale dynamically.

This concludes the lesson. Thank you for reading, and we look forward to seeing you in the next lesson.



## Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

# Services Loadbalancer

This article explores the LoadBalancer service type in Kubernetes, focusing on simplifying access to front-end applications through cloud-native load balancing.

In this article, we explore another service type: the LoadBalancer. Previously, we examined the NodePort service that exposes an application on a specific port of the worker nodes. Here, we focus on front-end applications such as the voting app and the result app.

Imagine that these pods are distributed across a cluster—say, a four-node cluster. To allow external users to access these applications, services of type NodePort were created. The NodePort service routes incoming traffic from designated ports on the worker nodes to the corresponding pods. With NodePort, you can reach the applications using any node's IP address along with its high port number. For example, if the voting app and the result app are bound to different IP-port combinations, users could access the application using any node's IP address and its specified port, even if the pods are running on only two of the nodes.

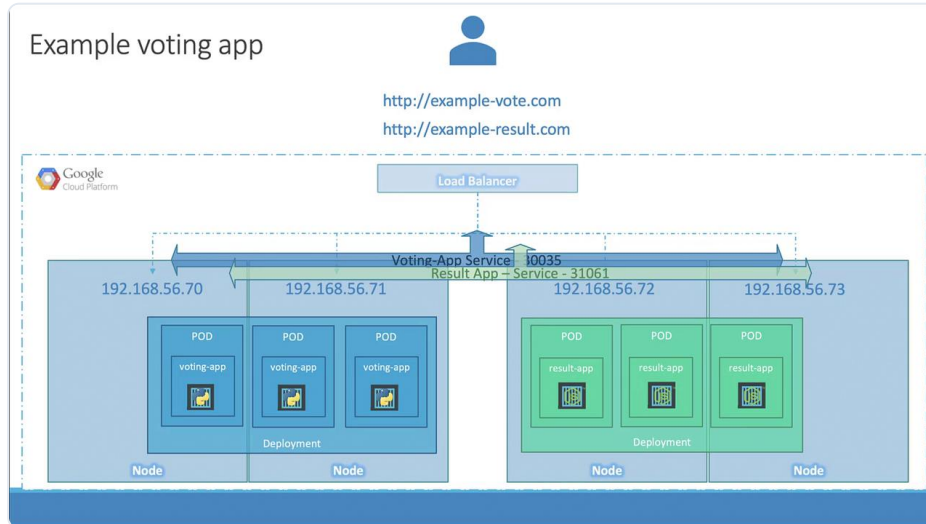
While NodePort works, it forces users to remember multiple IP-port pairs, which can be inconvenient.

End users typically prefer a single URL—for instance, `votingapp.com` or `resultapp.com`—rather than juggling several combinations. One approach to provide a unified entry point is to set up a dedicated virtual machine as a load balancer, using tools like [HAProxy](#) or [Nginx](#) to distribute the traffic across your nodes. However, managing an external load balancer can add complexity and overhead.

For a simpler solution, many cloud platforms such as [Google Cloud Platform \(GCP\)](#), [AWS](#), or [Azure](#) offer integrated load balancing. When running Kubernetes on one of these supported cloud environments, you can simply change the service type from NodePort to LoadBalancer for your front-end services. Kubernetes then automatically provisions and configures a cloud-

native load balancer, providing a single, user-friendly URL to access your application.

Below is a diagram that illustrates a cloud-based voting app architecture on Google Cloud Platform. The image shows nodes, pods, deployments, and a load balancer configured to handle the voting and result services:



Keep in mind that the LoadBalancer service type only functions as intended on supported cloud environments. In unsupported settings—such as VirtualBox—the LoadBalancer type behaves like NodePort by exposing the service on a high port without providing external load balancing.

In upcoming articles, we will demonstrate how to deploy applications on cloud platforms and observe how this configuration works in action. Stay tuned for more detailed guides in our series.

## Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

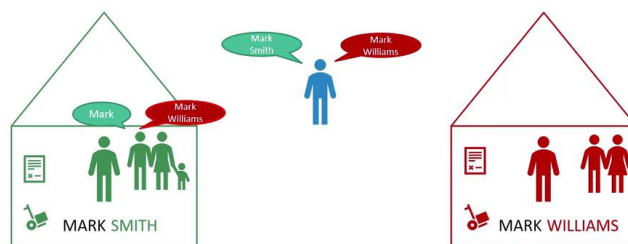
# Namespaces

This article explores Kubernetes namespaces, their role in organizing resources, and how to manage them effectively within a cluster.

In this lesson, we explore the concept of namespaces in Kubernetes and how they help organize and isolate resources within your cluster.

## Understanding Namespaces Through Analogy

Imagine there are two boys named Mark. To differentiate between them, you refer to them by their last names—Smith and Williams. They come from different houses where people often use first names for those familiar with them. However, when addressing someone from another house or an outsider, the full name is used. In Kubernetes, these "houses" represent namespaces. They allow you to group and manage resources differently based on their context and intended use.

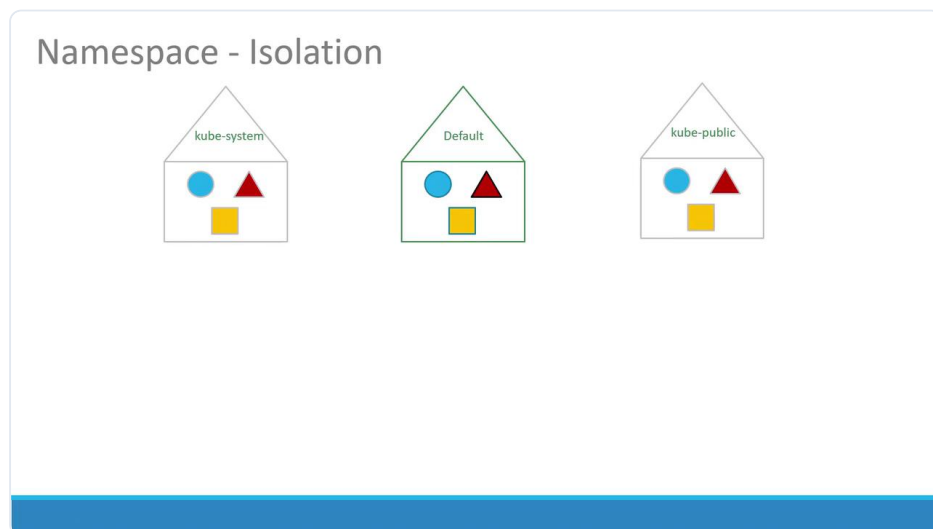


## Default Namespace and System Namespaces

By default, when you create objects such as pods, deployments, and services in your cluster, they are placed within a specific namespace (similar to being "inside a house"). The default namespace is automatically created during the Kubernetes cluster setup. Additionally, several system namespaces are created at startup:

- **kube-system:** Contains core system components like network services and DNS, segregated from user operations to prevent accidental changes.
- **kube-public:** Intended for resources that need to be publicly accessible across users.

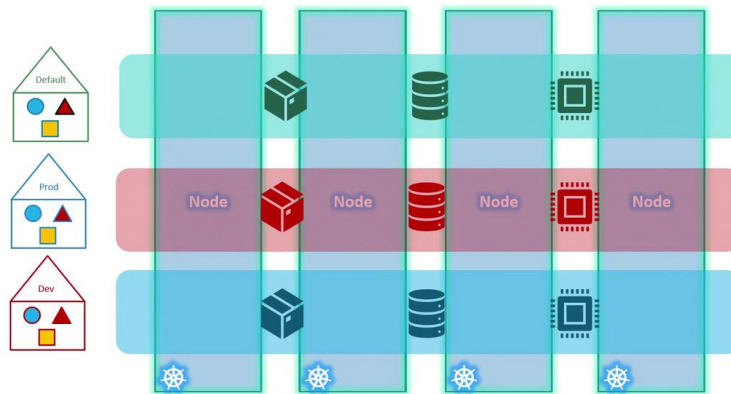
If you're running a small environment or a personal cluster for learning, you might predominantly use the default namespace. In enterprise or production environments, however, namespaces provide essential isolation and resource management by allowing environments like development and production to coexist on the same cluster.



## Isolating Resources with Namespaces

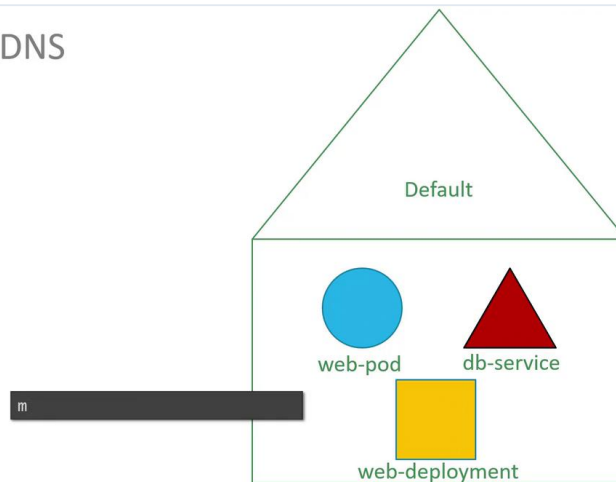
Namespaces allow you to set distinct policies and resource limits for different environments. This isolation prevents one namespace from interfering with another. For instance, you can apply separate resource quotas for CPU, memory, and the total number of pods to ensure fair usage across environments.

## Namespace – Resource Limits



Within a single namespace, resources can refer to each other directly via their simple names. For example, a web application pod in the default namespace can access a database service simply by using its service name:

## DNS



If the web app pod needs to communicate with a service located in a different namespace, you must use its fully qualified DNS name. For example, connecting to a database service named "db-service" in the "dev" namespace follows this format:

```
```SQL theme={null} mysql.connect("db-service.dev.svc.cluster.local")
```

Here, "svc" indicates the service subdomain, followed by the namespace ("dev") and the service name.

Managing Namespaces with kubectl

Using `kubectl`, you can manage resources across different namespaces. Below are some common commands.

Listing Pods in a Namespace

To list all pods in the default namespace:

```
```bash theme={null}
> kubectl get pods
NAME READY STATUS RESTARTS AGE
Pod-1 1/1 Running 0 3d
Pod-2 1/1 Running 0 3d
```

To list pods within the **kube-system** namespace:

```
```bash theme={null}
```

```
kubectl get pods --namespace=kube-system
NAME                                READY   STATUS    RESTARTS   AGE
coredns-78cdf6894-92d52             1/1     Running   7           3d
coredns-78cdf6894-jx25g             1/1     Running   7           3d
etcd-master                         1/1     Running   7           3d
kube-apiserver-master               1/1     Running   7           3d
kube-controller-manager-master     1/1     Running   7           3d
kube-flannel-ds-amd64-hz4cf         1/1     Running   14          3d
kube-proxy-4b8tn                   1/1     Running   7           3d
kube-proxy-98db4                   1/1     Running   7           3d
kube-proxy-jjrb5                   1/1     Running   7           3d
kube-scheduler-master              1/1     Running   7           3d
```

Creating Pods in Specific Namespaces

When creating a pod without specifying the namespace, it is placed in the default namespace.

```
```bash theme={null}
> kubectl create -f pod-definition.yml
pod/myapp-pod created
```

To create the same pod in the "dev" namespace, you can either include the namespace option:

```
```bash theme={null}
```

```
kubectl create -f pod-definition.yml --namespace=dev pod/myapp-pod
created
```

Or define the namespace within the pod definition file:

Without Namespace Specification

```
```yaml theme={null}
apiVersion: v1
kind: Pod
metadata:
 name: myapp-pod
 labels:
 app: myapp
 type: front-end
spec:
 containers:
 - name: nginx-container
 image: nginx
```

### With Namespace Specification

```
```yaml theme={null} apiVersion: v1 kind: Pod metadata: name: myapp-pod
namespace: dev labels: app: myapp type: front-end spec: containers: - name:
nginx-container image: nginx
```

Creating a Namespace

You can create a namespace using a YAML file. For example, create a file named `namespace-dev.yml`.

```
```yaml theme={null}
apiVersion: v1
kind: Namespace
metadata:
 name: dev
```

Then run:

```
```bash theme={null}
```

```
kubectl create -f namespace-dev.yml namespace/dev created
```

Alternatively, create a namespace directly through the command line:

```
```bash theme={null}
> kubectl create namespace dev
```

### Setting the Default Namespace for Your Context

If you're working across multiple namespaces and want to avoid repeatedly specifying the namespace flag, you can set the default namespace for your current context:

```
``bash theme={null} kubectl config set-context $(kubectl config current-context) --namespace=dev
```

After setting this, running:

```
``bash theme={null}
> kubectl get pods
```

will automatically list pods in the "dev" namespace. To list pods from all namespaces, use:

```
``bash theme={null}
```

```
kubectl get pods --all-namespaces
```

<Callout icon="lightbulb" color="#1CB2FE">

Contexts are used to manage multiple clusters and user environments within a single configuration file.

## Controlling Resource Usage with ResourceQuotas

To ensure that no single namespace overconsumes cluster resources, Kubernetes allows you to

```
``yaml theme={null}
apiVersion: v1
kind: ResourceQuota
metadata:
 name: compute-quota
 namespace: dev
spec:
 hard:
 pods: "10"
 requests.cpu: "4"
 requests.memory: 5Gi
 limits.cpu: "10"
 limits.memory: 10Gi
```

Apply this configuration with:

```
``bash theme={null}
```

```
kubectl create -f compute-quota.yaml ``
```

This configuration guarantees that the "dev" namespace does not exceed the specified resource limits.

## Conclusion

Namespaces are a fundamental component in Kubernetes, enabling you to segment and manage resources effectively. Whether you're isolating system



components or separating development and production environments, using namespaces along with appropriate policies and resource quotas leads to a more efficient and organized cluster management.

Practice these concepts and explore additional Kubernetes functionalities to deepen your understanding. Happy clustering!

For further reading, check out these resources:

- [Kubernetes Documentation](#)
- [Kubernetes Basics](#)

## Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

# Solution Namespaces optional

This article provides a lab walkthrough on managing namespaces, pods, and services in Kubernetes.

This article provides a comprehensive lab walkthrough on namespaces in Kubernetes, addressing questions related to namespaces, pods, and services. Follow the steps below to learn how to manage namespaces effectively.

## Identifying the Number of Namespaces

To determine the number of namespaces in your cluster, you can run either of the following commands:

```
``bash theme={null} kubectl get namespaces
```

or its shorthand:

```
``bash theme={null}
kubectl get ns
```

The sample output below shows a list of namespaces:

```
``bash theme={null} controlplane ~ × kubectl get namespaces NAME STATUS
AGE default Active 6m50s kube-system Active 6m49s kube-public Active
6m49s kube-node-lease Active 6m49s finance Active 27s marketing Active 27s
dev Active 27s prod Active 27s manufacturing Active 27s research Active 27s
```

And using the shorthand command:

```
```bash theme={null}
controlplane ~ × kubectl get ns
NAME                STATUS    AGE
default             Active   6m55s
kube-system         Active   6m54s
kube-public         Active   6m54s
kube-node-lease     Active   6m54s
finance             Active   32s
marketing           Active   32s
dev                 Active   32s
prod                Active   32s
manufacturing       Active   32s
research            Active   32s
```

In this example, there are a total of 10 namespaces.

Counting Pods in the Research Namespace

To determine the number of pods running in the research namespace, execute the following command:

```
```bash theme={null} kubectl get pods --namespace=research
```

The output might look like this:

```
```bash theme={null}
controlplane ~ → kubectl get pods --namespace=research
NAME      READY   STATUS    RESTARTS   AGE
dna-2     0/1     CrashLoopBackOff   3 (14s ago)    76s
dna-1     0/1     CrashLoopBackOff   3 (16s ago)    76s
```

Alternatively, you can use the shorthand option:

```
```bash theme={null} kubectl get pods -n research
```

Both commands confirm that there are 2 pods in the research namespace.

\*\*\*

### ## Creating a Pod in the Finance Namespace

Before creating a pod in the finance namespace, ensure that the namespace exists by running

```
```bash theme={null}
kubectl get namespaces
```

You should see output similar to:

```
```bash theme={null} controlplane ~ » kubectl get namespaces NAME STATUS
AGE default Active 6m50s kube-system Active 6m49s kube-public Active
6m49s kube-node-lease Active 6m49s finance Active 32s marketing Active 32s
dev Active 32s prod Active 32s manufacturing Active 32s research Active 32s
```

Next, create a Redis pod named "redis" in the finance namespace with the following command:

```
```bash theme={null}
kubectl run redis --image=redis -n finance
```

After successfully executing this command, you will see a confirmation message:

```
```bash theme={null} pod/redis created
```

You can verify the pod creation by listing the pods in the finance namespace:

```
```bash theme={null}
kubectl get pods -n finance
```

Sample output:

```
```bash theme={null} NAME READY STATUS RESTARTS AGE payroll 1/1
Running 0 2m20s redis 0/1 ContainerCreating 0 8s
```

```

```

### ## Locating the Blue Pod's Namespace

To find out which namespace the blue pod is running in, you have two options:

1. Manually inspect each namespace using:

```
```bash theme={null}
kubectl get pods -n <namespace>
```
```

2. List all pods across every namespace with:

```
```bash theme={null}
kubectl get pods --all-namespaces
```
```

The sample output below illustrates pods across namespaces:

```
```bash theme={null}
```

NAMESPACE	NAME	READY	STATUS
kube-system	local-path-provisioner-6c796847f7-j5sqx	1/1	Running
kube-system	coredns-57889956d-bkj56	1/1	Running
kube-system	helm-install-traefik-crd-d2fg7	0/1	Completed
kube-system	metrics-server-7cd5fcbb67-kdgzb	1/1	Running
kube-system	helm-install-traefik-c987s	2/2	Running
kube-system	svclb-traefik-jptsh	1/1	Running
marketing	redis-db	1/1	Running
dev	redis-db	1/1	Running
manufacturing	red-app	1/1	Running
finance	payroll	1/1	Running
marketing	blue	1/1	CrashLoopBackOff
research	dna-1	0/1	CrashLoopBackOff
finance	redis	1/1	Running

From the output, it is clear that the blue pod is located in the **marketing** namespace.

Accessing the Database Service from the Blue Application

The blue application, a simple web application for connectivity tests, interacts with a database service. Depending on where the DB service is running, the access method varies.

Accessing the DB Service in the Same Namespace (Marketing)

In the marketing namespace, the blue application and its associated DB service coexist. To check the pods in this namespace, run:

```
```bash theme={null} kubectl get pods -n marketing
```

Expected output:

```
```bash theme={null}
NAME      READY   STATUS    RESTARTS   AGE
redis-db   1/1     Running   0           4m14s
blue       1/1     Running   0           4m14s
```

Next, list the services in the marketing namespace:

```
```bash theme={null} kubectl get svc -n marketing
```

This yields an output similar to:

```
```bash theme={null}
NAME            TYPE        CLUSTER-IP      EXTERNAL-IP      PORT(S)          AGE
blue-service    NodePort    10.43.82.162    <none>           8080:30082/TCP   4m27s
db-service      NodePort    10.43.134.33    <none>           6379:30758/TCP   4m27s
```

Since both the blue application and the DB service are in the same namespace, the application can reach the DB service simply by using its DNS name, which is:

db-service

and connecting via port 6379.

Accessing the DB Service in the dev Namespace

Another DB service is running in the dev namespace. To view it, execute:

```
```bash theme={null} kubectl get svc -n dev
```

The output might be as follows:

```
```bash theme={null}
NAME            TYPE        CLUSTER-IP      EXTERNAL-IP      PORT(S)          AGE
db-service      ClusterIP    10.43.252.9     <none>           6379/TCP         5m22s
```

To access the DB service in the dev namespace from the blue application located in the marketing namespace, you must use the fully qualified DNS name. The DNS naming convention is:

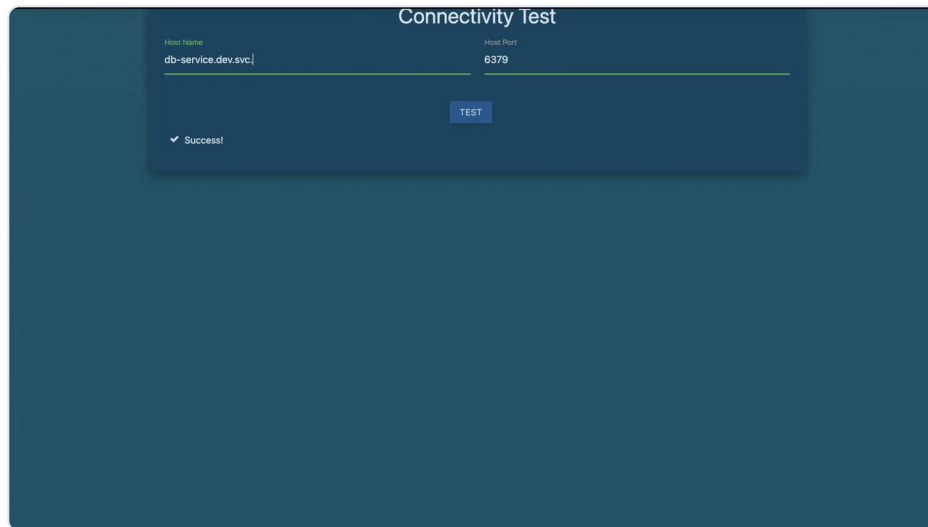
\\.svc.cluster.local

In this case, the DNS name is:

db-service.dev.svc.cluster.local

Using this fully qualified domain name ensures traffic is correctly directed to the DB service in the dev namespace on port 6379.

When performing connectivity tests from the blue application, the console output should verify that the connection to the DB service is successfully established.



This concludes the lab solution for namespaces. Each step demonstrates how to manage and interact with namespaces, pods, and services in a Kubernetes cluster.

For more details on Kubernetes concepts, visit the [Kubernetes Documentation](#).

Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

Imperative vs Declarative

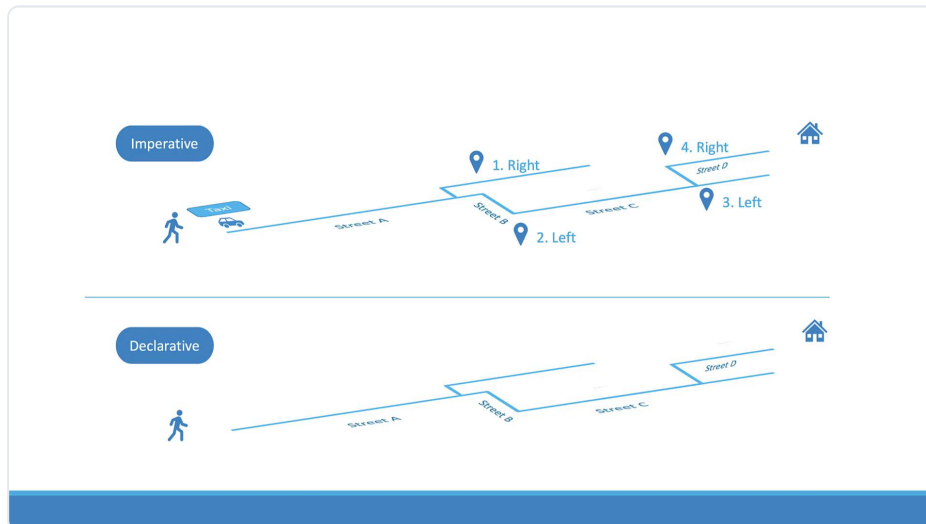
This article explores imperative and declarative approaches to managing Kubernetes objects, highlighting their benefits, drawbacks, and ideal use cases.

In this lesson, we explore the two core approaches used to manage Kubernetes objects: imperative and declarative. By understanding these methods and their respective benefits and drawbacks, you'll be better equipped to manage your clusters and prepare for Kubernetes certification exams.

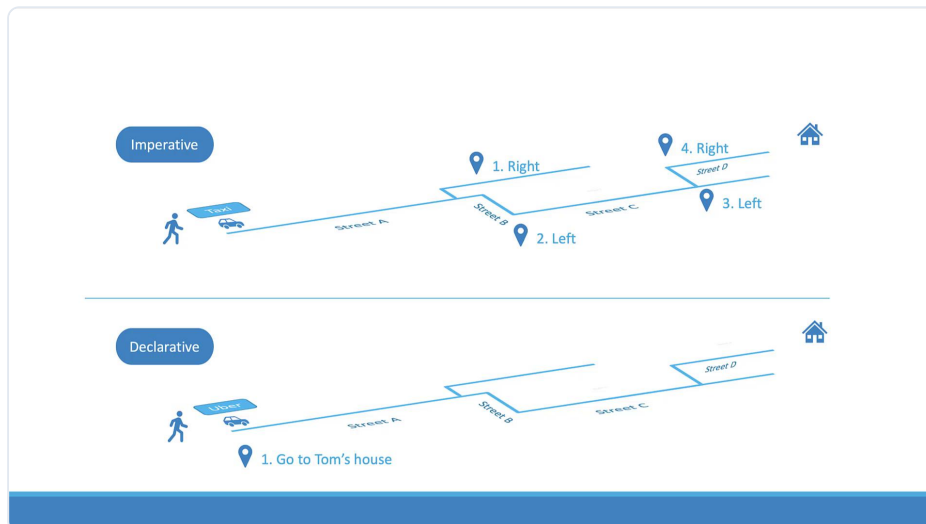
So far, we have covered creating and managing Kubernetes objects either by directly issuing commands or by using configuration files. These approaches fall into two broad categories in the infrastructure-as-code domain: imperative and declarative.

An Analogy

Imagine visiting a friend's house. In the past, you might have taken a taxi and given the driver precise, step-by-step directions—for example, "Take a right onto Street B, then left onto Street C, another left onto Street D, and finally stop at the house." This sequence of detailed instructions illustrates the imperative approach.



Today, using an app like Uber, you simply enter your final destination. This is akin to the declarative approach where you specify the desired outcome —"Drive to Tom's house"—and the system figures out the best route.



Imperative vs Declarative: Kubernetes Perspective

Imperative Approach

The imperative approach in Kubernetes involves executing specific commands to create, update, or delete objects. This method instructs Kubernetes on both what needs to be done and how it should be done. For example, you might run these commands:

```
``bash theme={null} kubectl run --image=nginx nginx kubectl create deployment --image=nginx nginx kubectl expose deployment nginx --port 80 kubectl edit deployment nginx kubectl scale deployment nginx --replicas=5 kubectl set image deployment nginx nginx=nginx:1.18 kubectl create -f nginx.yaml kubectl replace -f nginx.yaml kubectl delete -f nginx.yaml
```

While the imperative approach is effective for quick tasks, it comes with some limitations:

- * If a command partially executes, running it again may require extra checks (e.g., verify)
- * Updating resources—such as changing the image version—demands explicit re-execution with
- * Commands executed interactively are often not persisted, making it challenging for teams

In Kubernetes, imperative commands like ``kubectl run``, ``kubectl create deployment``, ``kubectl`

<Frame>

!The image compares imperative and declarative approaches in Infrastructure as Code, del

</Frame>

Declarative Approach

The declarative approach enables you to specify the desired state of your infrastructure th

```
```yaml theme={null}
nginx.yaml
apiVersion: v1
kind: Pod
metadata:
 name: myapp-pod
 labels:
 app: myapp
 type: front-end
spec:
 containers:
 - name: nginx-container
 image: nginx
```

Applying the configuration is as simple as running:

```
```bash theme={null} kubectl apply -f nginx.yaml
```

Kubernetes will create or update the object automatically to match the state described in y

```
```bash theme={null}
kubectl apply -f nginx.yaml
```

This method ensures that your configuration files remain the single source of truth, which is especially valuable in team environments where version-controlled definitions are critical.

## Imperative vs Declarative Update Dilemma

Sometimes, you might modify a live object using the `kubectl edit` command. This command opens a YAML representation of the current state, including additional fields like status, which are absent from your original configuration file. For instance:

1. Initially, you create the object using your YAML file:

```
bash theme={null} kubectl create -f nginx.yaml
```

1. Later, you edit the deployment:

```
bash theme={null} kubectl edit deployment nginx
```

1. The live object now contains extra status fields. If you later apply the original `nginx.yaml` (perhaps with updates), your live edits might be overwritten.

Always update your local configuration files and use commands like `kubectl replace -f nginx.yaml` to ensure that your changes are consistently tracked and version-controlled.

A typical workflow in a team environment is as follows:

- Create the object:

```
bash theme={null} kubectl create -f nginx.yaml
```

- Modify the local file to implement changes (e.g., update the image version).
- Update the live object with:

```
bash theme={null} kubectl replace -f nginx.yaml
```

This practice reinforces a version-controlled process and promotes collaboration.

## Choosing the Right Approach

Approach	Ideal Use Case	Example Commands
Imperative	Quick, one-off tasks such as creating a pod or deployment.	<pre>kubectl run -- image=nginx nginx kubectl create deployment --image=nginx nginx kubectl expose deployment nginx --port 80</pre>
Declarative	Long-term management with version-controlled infrastructure.	<pre>kubectl apply -f nginx.yaml kubectl apply -f /path/ to/config-files</pre>

- **Imperative Approach:** Use this method for rapid execution when you need to quickly create or modify Kubernetes objects, particularly during certification exams.

- **Declarative Approach:** This approach is recommended for complex, long-term management scenarios. It enables a systematic management of configurations via YAML files, ensuring every change is recorded and version-controlled.

## Exam Tips

---

When preparing for Kubernetes certification exams, consider the following strategies:

- Use imperative commands for speed when creating simple objects like pods or deployments.
- For modifications or more intricate configurations, adopt the declarative approach by updating configuration files and applying changes using `kubectl apply` or `kubectl replace`.
- Always maintain your YAML files in version control to safeguard against unintentional overwrites.

For more detailed guidance on managing a Kubernetes cluster, check the [official Kubernetes documentation](#) and experiment with both approaches in your lab exercises.

Happy learning, and see you in the next lesson!

## Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

# Kubectl Apply Command

This article explores the kubectl apply command, its internal workings, and how it manages Kubernetes object configurations declaratively.

In this article, we explore how the kubectl apply command works and what happens internally during its execution. Using kubectl apply for declarative management of Kubernetes objects is common practice, and here we dive into details such as configuration comparisons and update processing.

## Basic Example

Consider the following local YAML configuration file (nginx.yaml) that defines a Pod:

```
```yaml theme={null} apiVersion: v1 kind: Pod metadata: name: myapp-pod labels: app: myapp type: front-end-service spec: containers: - name: nginx-container image: nginx:1.18
```

Apply this configuration with:

```
```bash theme={null} kubectl apply -f nginx.yaml
```

You can also apply all configuration files within a directory:

```
```bash theme={null} kubectl apply -f /path/to/config-files
```

How kubectl apply Works Internally

When you run the kubectl apply command, it compares three sources:

1. The local configuration file (e.g., nginx.yaml).
2. The live object configuration stored on the Kubernetes cluster.
3. The last applied configuration stored as an annotation on the live object.

If the object does not exist, Kubernetes creates it based on your local configuration. During

<Callout icon="lightbulb" color="#1CB2FE">

When the local configuration is changed (for example, updating the image version), kubectl
</Callout>

For instance, if you update the image version from 1.18 to 1.19 in your local file:

```
```yaml theme={null}
apiVersion: v1
kind: Pod
metadata:
 name: myapp-pod
 labels:
 app: myapp
 type: front-end-service
spec:
 containers:
 - name: nginx-container
 image: nginx:1.19
```

and run:

```
```bash theme={null} kubectl apply -f nginx.yaml
```

kubectl compares the three configurations. If differences are detected—such as the updated

Managing Removed Fields

The last applied configuration annotation is crucial when fields are removed from your local

```
```yaml theme={null}
apiVersion: v1
kind: Pod
metadata:
 name: myapp-pod
 labels:
 app: myapp
spec:
 containers:
 - name: nginx-container
 image: nginx:1.19
```

and run:

```
```bash theme={null} kubectl apply -f nginx.yaml
```

```
kubectl notices that the "type" label, which existed in the last applied configuration, is

## Last Applied Configuration Annotation

When kubectl apply is executed for the first time, the YAML configuration is converted to JSON

```yaml
theme: {null}
apiVersion: v1
kind: Pod
metadata:
 name: myapp-pod
 annotations:
 kubectl.kubernetes.io/last-applied-configuration: '{"apiVersion":"v1","kind":"Pod","metadata":{"name":"myapp-pod"},"spec":{"containers":[{"name":"nginx-container","image":"nginx:1.18"}]}}'
 labels:
 app: myapp
 type: front-end-service
spec:
 containers:
 - name: nginx-container
 image: nginx:1.18
status:
 conditions:
 - lastProbeTime: null
```

This annotation is the key to performing a three-way merge in future apply operations. The process compares:

- The local file.
- The live object configuration.
- The last applied configuration.

This comparison ensures that Kubernetes makes precise updates. It's important to note that mixing imperative commands (like `kubectl create` or `kubectl replace`) with declarative ones can lead to inconsistencies, as only `kubectl apply` stores the last applied configuration.

## Actionable Summary

Step	Description	Example Command
Initial Creation	Creates the object and stores the configuration as an annotation	<code>kubectl apply -f nginx.yaml</code>
Update Configuration	Modifies the object by comparing local, live, and last applied configuration	<code>kubectl apply -f nginx.yaml</code>
Remove a Field	Deletes a field from live configuration when it is removed locally	<code>kubectl apply -f nginx.yaml</code>

Avoid mixing imperative commands with declarative approaches. Imperative actions like `kubectl create` or `kubectl replace` will not record the last applied configuration and may lead to inconsistencies when using `kubectl apply`.

## Conclusion

---

Understanding how `kubectl apply` processes local configurations, the live state of Kubernetes objects, and the last applied configuration annotation is crucial for managing resources declaratively. For more comprehensive information, please refer to the [Kubernetes documentation](#).

Thank you for reading this guide on the `kubectl apply` command. Happy deploying!



## Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

## A Quick Reminder

This article emphasizes the importance of following labs and video lectures for effective learning in the course.

In this article, we emphasize the importance of following the provided labs and video lectures in this course. While it might be tempting to set up your own local environment right away, our structured approach is designed to guide you step-by-step through the core concepts without distractions.

Focus on the course content as provided to maximize your learning efficiency and avoid unnecessary detours.

Following the labs and lectures in the intended order ensures a seamless learning experience, allowing you to build foundational skills before moving on to more advanced topics such as configuring your local environment. This method helps you optimize your study time and develop the necessary skills more effectively.

Below is an example output from the command that lists all Docker images available on your system:

```
```bash theme={null} $ docker images REPOSITORY TAG SIZE redis latest 105MB ubuntu latest 72.7MB mysql latest 556MB nginx latest 22.6MB alpine alpine 5.6MB nginx latest 133MB postgres latest 314MB kodekloud/simple-webapp-mysql latest 96.6MB kodekloud/simple-webapp latest 84.8MB
```

This output showcases the various Docker images installed, which will become increasingly relevant as you progress through the course.

For now, please proceed with the lab that involves running the following command:

```
```bash theme={null}
$ docker run redis
```

This lab is specifically designed to build your foundational Docker skills. Additional resources and detailed guidance—such as instructions on setting up your local environment—will be provided later in the course.

Thank you for choosing our course. We are excited to support you on your journey to mastering Docker and related technologies.

# Section: Scheduling

---

## Documentation Index

---

Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

## Scheduling Section Introduction

---

This article provides a comprehensive lesson on Kubernetes scheduling concepts and techniques for managing pods and workloads effectively.

Hello, my name is Mumshad Mannambeth, and I welcome you to this comprehensive lesson on scheduling in Kubernetes. In this module, you will explore essential scheduling concepts and techniques that empower you to manage pods and workloads effectively. Previously, we briefly introduced the installation and configuration of the Kubernetes scheduler. Now, we will dive deeper into the various customization options that influence scheduler behavior through a series of practical, hands-on labs.

## Overview

---

In this lesson, you will learn about the following topics:

- **Manual Scheduling:** Understand how to manually schedule pods to specific nodes.
- **DaemonSets:** Discover how to run a copy of a pod on each node in your cluster.
- **Labels and Selectors:** Learn how to use labels and selectors to manage and target specific groups of pods.
- **Resource Requests and Limits:** See how resource constraints affect scheduling decisions.
- **Multiple Schedulers:** Explore configuration strategies for running multiple schedulers within a cluster.

- **Viewing Scheduler Events:** Learn how to monitor and troubleshoot scheduler events for optimal performance.

Each section of this lesson is complemented by hands-on labs designed to reinforce your understanding and give you real-world experience with Kubernetes scheduling.

Let's get started with the first topic—Manual Scheduling—and progressively build your expertise in Kubernetes scheduling.



## Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

# Manual Scheduling

This guide explains how to assign pods to nodes without relying on Kubernetes' built-in scheduler for tighter control over pod placement.

Welcome to this lesson on manually scheduling pods in Kubernetes. This guide explains how to assign pods to nodes without relying on Kubernetes' built-in scheduler. Manual scheduling can be useful in niche scenarios where you need tighter control over pod placement. In this article, we review a basic pod manifest, demonstrate how manual scheduling works, and show you how to use binding objects to reassign pods if necessary.

## Understanding the Default Scheduler Behavior

Every pod definition includes a field called `nodeName`, which is left unset by default. The Kubernetes scheduler automatically scans for pods without a `nodeName` and selects an appropriate node by updating this field and creating a binding object. Consider the basic pod manifest below:

```
```yaml
theme={null}
apiVersion: v1
kind: Pod
metadata:
  name: nginx
labels:
  name: nginx
spec:
  containers:
  - name: nginx
    image: nginx
    ports:
    - containerPort: 8080
```

Typically, you do not include the `nodeName` field in your manifest. The scheduler uses the

Manually Setting the Node Name

To manually assign a pod to a specific node during creation, populate the `nodeName` field

```
```yaml
theme: {null}
apiVersion: v1
kind: Pod
metadata:
 name: nginx
 labels:
 name: nginx
spec:
 nodeName: node02
 containers:
 - name: nginx
 image: nginx
 ports:
 - containerPort: 8080
```

After creating the pod with this manifest, check its status with:

```
```bash
theme={null} kubectl get pods
```

Sample output before the scheduler assigns the IP:

```
```bash
theme={null}
NAME READY STATUS RESTARTS AGE
nginx 0/1 Pending 0 3s
```

And after the pod becomes ready:

```
bash theme={null} kubectl get pods
NAME READY STATUS RESTARTS AGE IP nodeName
nginx 1/1 Running 0 9s 10.40.0.4 node02
```

The `nodeName` must be set during pod creation. Once the pod is running, Kubernetes does not permit modifications to the `nodeName` field.

## Reassigning a Running Pod Using a Binding Object

If a pod is already running and you need to change its node assignment, you cannot modify its `nodeName` directly. In this scenario, you can create a binding object that mimics the scheduler's behavior.

1. Create a binding object that specifies the target node ("node02"):

```
yaml theme={null} apiVersion: v1 kind: Binding metadata: name: nginx
target: apiVersion: v1 kind: Node name: node02
```

1. The original pod definition remains unchanged:

```
yaml theme={null} apiVersion: v1 kind: Pod metadata: name: nginx
labels: name: nginx spec: containers: - name: nginx image: nginx
ports: - containerPort: 8080
```

1. Convert the YAML binding to JSON (e.g., save it as `binding.json` ) and send a POST request to the pod's binding API using curl:

```
bash theme={null} curl --header "Content-Type: application/json" --
request POST --data @binding.json http://$SERVER/api/v1/namespaces/
default/pods/nginx/binding
```

This binding instructs Kubernetes to assign the existing pod to the specified node without altering its original manifest.

## Quick Reference Table

Method	Use Case	Example Snippet Reference
Direct Assignment	Set <code>nodeName</code> during pod creation	See manifest with <code>nodeName: node02</code>
Using a Binding Object	Reassign a running pod	See binding object example

## Additional Resources

- [Kubernetes Basics](#)
- [Kubernetes Documentation](#)

That's it for this lesson. Experiment with manually scheduling pods to solidify your understanding of Kubernetes node assignments. Happy scheduling!

## Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

# Labels and Selectors

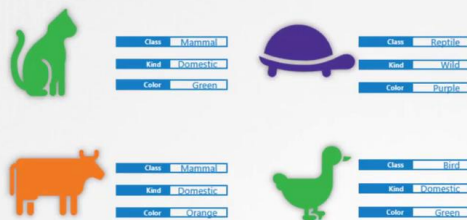
This article explores labels, selectors, and annotations in Kubernetes for effective resource management and organization.

Welcome to this comprehensive guide on labels, selectors, and annotations in Kubernetes. In this article, we will explore how labels and selectors help group and filter items effectively, and we will also discuss how annotations are used to store additional metadata. By the end, you'll understand how these key concepts are applied to manage resources in a Kubernetes environment.

## Introduction to Labels and Selectors

Labels and selectors offer a systematic approach to categorizing items. Think of a scenario where you have various species of animals, and you need to filter them based on attributes such as class, kind, or color. For example, you might want to filter for all green animals or just the green birds. By attaching labels like class, kind, and color to each animal, you can later use selectors to efficiently filter and display the items that meet your criteria.

### Selectors





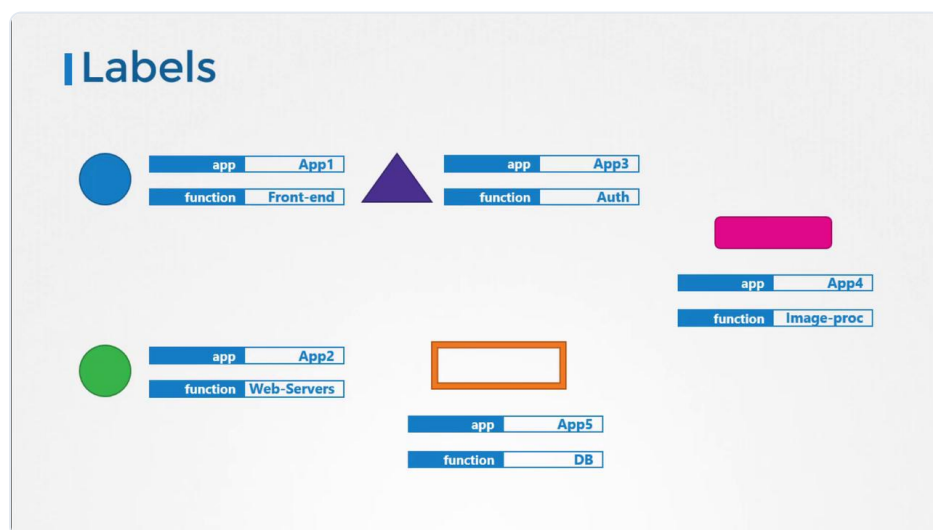
In everyday applications, labels and selectors are ubiquitous. They function similarly to keywords in YouTube videos or blog posts, aiding users in finding relevant content. In online stores, labels empower customers to filter products based on various characteristics.

## Labels and Selectors in Kubernetes

In Kubernetes, labels and selectors are instrumental in managing an array of objects such as Pods, Services, ReplicaSets, and Deployments. As the number of objects in a cluster grows, these tools become essential for grouping and selecting objects by application, functionality, or type.



For instance, you might attach labels like "app" or "function" to your Kubernetes objects and later use selectors to filter objects based on specific conditions (e.g., app equals "App1").



## Specifying Labels in Kubernetes

To apply labels to a Kubernetes object such as a Pod, include a `labels` section under the `metadata` field in its definition file. Consider the following Pod definition example:

```
```yaml theme={null} apiVersion: v1 kind: Pod metadata: name: simple-webapp labels: app: App1 function: Front-end spec: containers: - name: simple-webapp image: simple-webapp ports: - containerPort: 8080
```

After creating the Pod, you can retrieve it using the `kubectl get pods` command with a `selector`

```
```bash theme={null}
kubectl get pods --selector app=App1
NAME READY STATUS RESTARTS AGE
simple-webapp 0/1 Completed 0 1d
```

Using selectors with `kubectl` commands helps you filter and manage resources in large clusters with ease.

## Using Labels and Selectors with ReplicaSets

In Kubernetes, internal mechanisms utilize labels and selectors to connect different objects. When creating a ReplicaSet to manage three Pods, you first label the Pod definitions and then use a selector in the ReplicaSet definition to ensure the correct Pods are grouped together.

A ReplicaSet definition includes labels in two key areas:

1. Within the ReplicaSet's metadata (allowing other objects to reference the ReplicaSet).
2. Within the `template` of the ReplicaSet's specification (applying the labels to the Pods).

By setting the `selector` field in the ReplicaSet specification to match the labels defined on the Pods, you ensure that the ReplicaSet manages the intended Pods. Below is an example configuration:

```
```yaml theme={null} apiVersion: apps/v1 kind: ReplicaSet metadata: name: simple-webapp labels: app: App1 function: Front-end spec: replicas: 3 selector: matchLabels: app: App1 template: metadata: labels: app: App1 function: Front-end spec: containers: - name: simple-webapp image: simple-webapp
```

```
<Callout icon="lightbulb" color="#1CB2FE">
```

```
  If you require more granular control for selecting Pods, you can list multiple labels in
</Callout>
```

Annotations

Annotations differ from labels and selectors in that they are used to store additional metadata

```
```yaml theme={null}
apiVersion: apps/v1
kind: ReplicaSet
metadata:
 name: simple-webapp
 labels:
 app: App1
 function: Front-end
 annotations:
 buildversion: "1.34"
spec:
 replicas: 3
 selector:
 matchLabels:
 app: App1
 template:
 metadata:
 labels:
 app: App1
 function: Front-end
 spec:
 containers:
 - name: simple-webapp
 image: simple-webapp
```

When the ReplicaSet is created, it matches the Pods based on labels, ensuring that only the intended Pods are managed. The same mechanism is used when creating Services, where the Service's selector matches the labels set on the Pods.

## Conclusion

This article has provided an in-depth look at labels, selectors, and annotations in Kubernetes. These concepts are essential for effectively managing and grouping objects within your clusters. For further hands-on practice, navigate to the coding exercises section and start working with labels and selectors today.

## Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

# Taints and Tolerations

This article explores how taints and tolerations control pod scheduling on specific nodes in Kubernetes.

In this article, we explore the relationship between pods and nodes, and demonstrate how taints and tolerations control pod scheduling on specific nodes. Think of it like using a repellent on a person to keep bugs away. In Kubernetes, a taint is applied to a node (the repellent) to repel pods unless they have a matching toleration (an immunity to the repellent). This ensures that only intended pods can run on tainted nodes.



## How Taints and Tolerations Work

Two factors determine whether a pod can run on a node:

1. The taint applied on the node.
2. The pod's toleration for that taint.

In Kubernetes, taints and tolerations are used solely for scheduling control, not security. Consider a simple cluster with three worker nodes (node one, two,

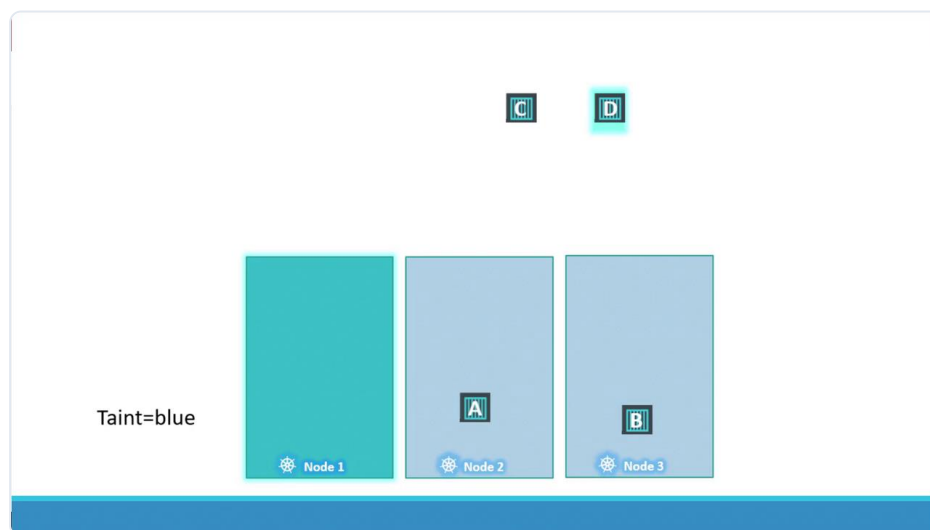
and three) and four pods (A, B, C, and D). By default, without any taints or tolerations, the scheduler distributes the pods evenly across all nodes.

Suppose you want to dedicate node one to a specific application:

- First, taint node one (using, for example, the key-value pair "app=blue") so that no pods are scheduled there by default.
- Then, add an appropriate toleration to the dedicated pod (Pod D) so that it alone can run on node one.

When the scheduler assigns pods:

- Pod A, lacking a toleration, will be scheduled on node two.
- Pod B is repelled from node one and lands on node three.
- Pod C, without a matching toleration, is scheduled on node two or three.
- Pod D, having the correct toleration, is scheduled on node one.



## Tainting a Node

To taint a node, use the following command where you specify the node name along with a taint in the format of a key-value pair and taint effect:

```
``bash theme={null} kubectl taint nodes node-name key=value:taint-effect
```

For example, to taint node one so that it only accepts pods associated with the application

```
``bash theme={null}
kubectl taint nodes node1 app=blue:NoSchedule
```

There are three taint effects:

- **NoSchedule:** Pods that do not tolerate the taint will not be scheduled on the node.
- **PreferNoSchedule:** The scheduler tries to avoid placing non-tolerating pods on the node, but it is not strictly enforced.

- **NoExecute:** New pods without a matching toleration will not be scheduled, and existing pods will be evicted from the node.

## Adding Tolerations to Pods

To allow a specific pod to run on a tainted node, add a toleration to the pod's manifest. For example, to enable Pod D to run on node one (tainted with "app=blue:NoSchedule"), modify the pod definition as follows:

```
```yaml
theme={null}
apiVersion: v1
kind: Pod
metadata:
  name: myapp-pod
spec:
  containers:
    - name: nginx-container
      image: nginx
  tolerations:
    - key: "app"
      operator: "Equal"
      value: "blue"
      effect: "NoSchedule"
```
```

This toleration lets the pod bypass the taint and be scheduled on node one.

### ## Understanding the NoExecute Effect

Using the **NoExecute** taint effect has a more immediate impact. Suppose node one is tainted.

- \* Pods without a matching toleration (e.g., Pod C) will be evicted from node one if already running.
- \* Pods with the correct toleration (e.g., Pod D) continue to run on node one.

<Frame>

! [The image illustrates Kubernetes nodes with a "NoExecute" taint, showing pods A, B, C, and D.]

</Frame>

<Callout icon="lightbulb" color="#1CB2FE">

Taints and tolerations are used to ensure that pods are not accidentally scheduled on nodes.

</Callout>

### ## Master Node Taints

While the discussion so far has focused on worker nodes, master nodes are also capable of running pods.

```
```bash
theme={null}
kubectl describe node kubemaster | grep Taint
```
```

The output will display a taint similar to:

```
text theme={null} Taints: node-role.kubernetes.io/master:NoSchedule
```

This configuration follows best practices by ensuring that only essential components run on the master node.

## Summary

Taints and tolerations provide a powerful mechanism for controlling pod scheduling in Kubernetes clusters:

| Feature                 | Description                                                                                       | Example Command or Manifest Snippet                                     |
|-------------------------|---------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------|
| Tainting a Node         | Prevents pods from being scheduled on a node unless they tolerate the taint.                      | <pre>kubectl taint nodes node1 app=blue:NoSchedule</pre>                |
| Toleration in a Pod     | Allows a pod to run on a tainted node by including a toleration in its manifest.                  | See the YAML snippet above                                              |
| Taint Effect: NoExecute | Evicts pods not tolerating the taint from a node and prevents new non-tolerant pods from joining. | Command similar to the taint command with <code>NoExecute</code> effect |

Now that you understand taints and tolerations, experiment with their configuration in your Kubernetes cluster to better control pod placement. For more details on advanced scheduling techniques, check out [Kubernetes Documentation](#).

Remember that taints and tolerations only influence pod scheduling. If you need to enforce pod placement on specific nodes, use node affinity rules.

## Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

# Node Selectors

This lesson covers how to use node selectors in Kubernetes to ensure specific pods run on designated nodes for better performance and resource management.

Welcome to this lesson on node selectors in Kubernetes. In this guide, you'll learn how to ensure that specific pods run only on designated nodes within your cluster. Node selectors help align your pod deployments with the underlying hardware characteristics of your nodes, enhancing performance and resource management.

Imagine managing a three-node cluster where two nodes have limited hardware resources, and one node is equipped with higher resources. Different workloads run in your cluster, and data processing tasks that demand more computing power should ideally be scheduled on the larger node. Without any scheduling constraints, any pod might land on any node—even on one with insufficient resources—leading to performance bottlenecks.

Node selectors restrict pod placement by matching key-value pairs defined in the pod's specification against the labels on the nodes.

## Configuring Node Selectors

To ensure that a pod is restricted to run on a specific node, you can modify the pod's definition file using node selectors. Below is an example of a pod definition YAML file that deploys a data processing image exclusively on a node labeled as "Large":

```
```yaml
theme={null}
apiVersion: v1
kind: Pod
metadata:
  name: myapp-pod
spec:
  containers:
    - name: data-processor
      image: data-processor
      nodeSelector:
        size: Large
```



```
In this configuration, the Kubernetes scheduler identifies the appropriate node by matching  
  
## Labeling a Node  
  
Before deploying your pod, you must label your node so that it can be recognized by the sel  
  
```bash theme={null}  
kubectl label nodes node-1 size=Large
```

Once the node is labeled, create the pod by applying your configuration:

```
bash theme={null} kubectl create -f pod-definition.yaml
```

Kubernetes will then schedule your pod on the node that matches the selector—in this case, the larger node.

## Limitations and Advanced Scheduling

---

While node selectors are ideal for simple scenarios involving a single label, they come with certain limitations. For instance, if you need to schedule a pod on a node that is either large or medium, or on any node that is not labeled as small, a basic node selector may not suffice. In these cases, consider using node affinity and anti-affinity features, which offer advanced scheduling capabilities to define more complex placement rules.

Make sure that your nodes are pre-labeled with the correct key-value pairs before deploying your pods. Failure to do so may prevent the scheduler from matching pods to the intended nodes.

## Further Resources

---

- [Kubernetes Documentation](#)
- [Kubernetes Basics](#)

Happy learning!

## Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

# Node Affinity

This article explains node affinity in Kubernetes, detailing advanced scheduling rules for pod placement based on node labels.

Welcome to this comprehensive lesson on node affinity in Kubernetes. In this guide, you'll learn how node affinity extends the capabilities of basic node selectors by allowing advanced expressions like In, NotIn, and Exists. This feature enables you to specify detailed rules for pod placement based on node labels.

Previously, you might have used node selectors for basic scheduling. For example, to ensure that a large data processing pod runs on a large node, you could use a configuration like this:

```
```yaml theme={null} apiVersion: v1 kind: Pod metadata: name: myapp-pod spec: containers: - name: data-processor image: data-processor nodeSelector: size: Large
```

While node selectors are simple and intuitive, they lack support for advanced matching operators.

```
```yaml theme={null} apiVersion: v1 kind: Pod metadata: name: myapp-pod spec: containers: - name: data-processor image: data-processor affinity: nodeAffinity: requiredDuringSchedulingIgnoredDuringExecution: nodeSelectorTerms: - matchExpressions: - key: size operator: In values: - Large
```

- The `affinity` key under `spec` introduces the `nodeAffinity` configuration.
  - The `requiredDuringSchedulingIgnoredDuringExecution` field indicates that the scheduler must place the pod on a node meeting the criteria. Once the pod is running, any changes to node labels are ignored.
  - The `nodeSelectorTerms` array contains one or more `matchExpressions`. Each expression specifies a label key, an operator, and a list of values. Here, the `In` operator ensures that the pod is scheduled only on nodes where the label `size` includes 'Large'.

To allow for more flexible scheduling, such as permitting a pod to run on either large or medium nodes, simply add additional values to the list. Alternatively, you can use the `NotIn` operator to explicitly avoid placing a pod on nodes with specific labels. For example, to avoid nodes labeled as small:

```
```yaml
theme={null}
apiVersion: v1
kind: Pod
metadata:
  name: myapp-pod
spec:
  containers:
  - name: data-processor
    image: data-processor
  affinity:
    nodeAffinity:
      requiredDuringSchedulingIgnoredDuringExecution:
        nodeSelectorTerms:
        - matchExpressions:
          - key: size
            operator: NotIn
            values:
            - Small
```
```

In cases where you only need to verify the presence of a label without checking for specific values, you can use the `Exists` operator.

```
```yaml
theme={null}
apiVersion: v1
kind: Pod
metadata:
  name: myapp-pod
spec:
  containers:
  - name: data-processor
    image: data-processor
  affinity:
    nodeAffinity:
      requiredDuringSchedulingIgnoredDuringExecution:
        nodeSelectorTerms:
        - matchExpressions:
          - key: size
            operator: Exists
```
```

Once a pod is scheduled using node affinity rules, these rules are only evaluated during scheduling. Changes to node labels after scheduling will not affect a running pod due to the "ignored during execution" behavior.

There are two primary scheduling behaviors for node affinity:

1. **Required During Scheduling, Ignored During Execution**
2. The pod is scheduled only on nodes that fully satisfy the affinity rules.
3. Once running, changes to node labels do not impact the pod.
4. **Preferred During Scheduling, Ignored During Execution**

5. The scheduler prefers nodes that meet the affinity rules but will place the pod on another node if no matching nodes are available.

## Node Affinity Types

Available:

**requiredDuringSchedulingIgnoredDuringExecution**

**preferredDuringSchedulingIgnoredDuringExecution**

|        | DuringScheduling | DuringExecution |
|--------|------------------|-----------------|
| Type 1 | Required         |                 |
| Type 2 | Preferred        |                 |

Future enhancements may introduce additional affinity types, such as **Required During Execution**. In this model, if a node's labels change after a pod is running and no longer meet the affinity criteria, the pod would be evicted.

## Node Affinity Types

Planned:

**requiredDuringSchedulingRequiredDuringExecution**

**preferredDuringSchedulingRequiredDuringExecution**

|        | DuringScheduling | DuringExecution |
|--------|------------------|-----------------|
| Type 1 | Required         | Ignored         |
| Type 2 | Preferred        | Ignored         |
| Type 3 | Required         | Required        |
| Type 4 | Preferred        | Required        |

## Summary

Node affinity empowers you to define sophisticated scheduling rules for pod placement based on node labels. Key takeaways include:

- Using `nodeSelectorTerms` with `matchExpressions` to specify rules.
- Leveraging operators such as `In`, `NotIn`, and `Exists` for flexible matching.
- Understanding the scheduling phases: during scheduling and after deployment (execution), and how they interact.

This concludes our lesson on node affinity. Practice the provided configurations and explore further by comparing node affinity with taints and tolerations. Happy learning and coding!

## Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

# Taints and Tolerations vs Node Affinity

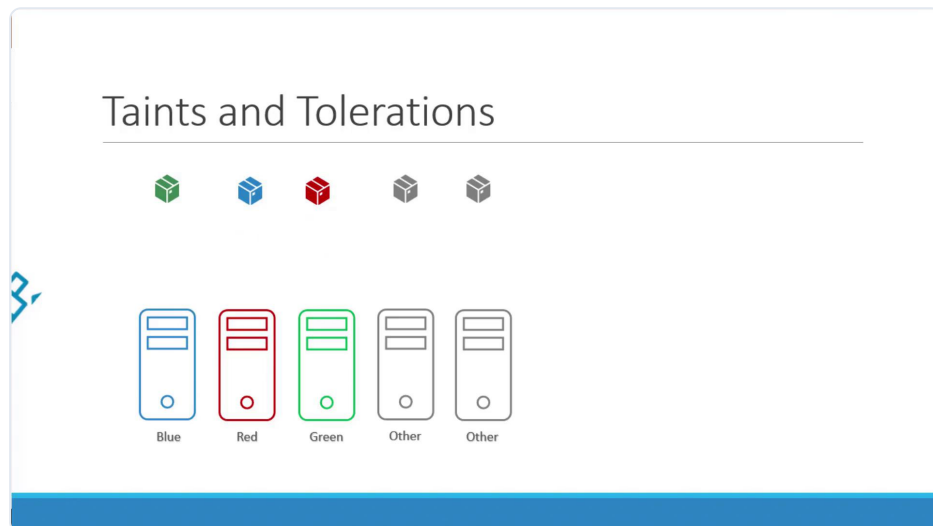
This article explains controlling pod placement in Kubernetes using taints, tolerations, and node affinity for optimal scheduling and exclusive node usage.

Welcome to this article where we explain how to control pod placement in a Kubernetes cluster by combining taints, tolerations, and node affinity. In our example, we have three nodes and three pods, each identified by a distinct color—blue, red, and green. Our objective is to ensure that each pod is scheduled on the node with the corresponding color while preventing unwanted workloads from running on these dedicated nodes.

In Kubernetes, taints and tolerations are primarily used to repel pods from nodes unless they explicitly tolerate the taint, whereas node affinity is used to attract pods to nodes that satisfy specific label criteria.

## Using Taints and Tolerations

To begin, we apply a taint to each node that marks it with its respective color (blue, red, or green). Then, each pod is configured with a corresponding toleration. With this setup, the Kubernetes scheduler places the pods on nodes that accept their tolerations. For instance, the green pod is placed on the green node and the blue pod on the blue node.



However, while taints and tolerations ensure that pods with matching tolerations are admitted by the nodes, they do not guarantee exclusive scheduling. Consequently, a pod (for example, a red pod) might still be scheduled on an untainted node, leading to undesired placements.

## Using Node Affinity

To overcome the limitation of taints and tolerations, we leverage node affinity. This method involves labeling each node with its specific color and then configuring node selectors or advanced affinity rules in the pod specifications. Node affinity ensures a pod lands only on the node with the matching label.

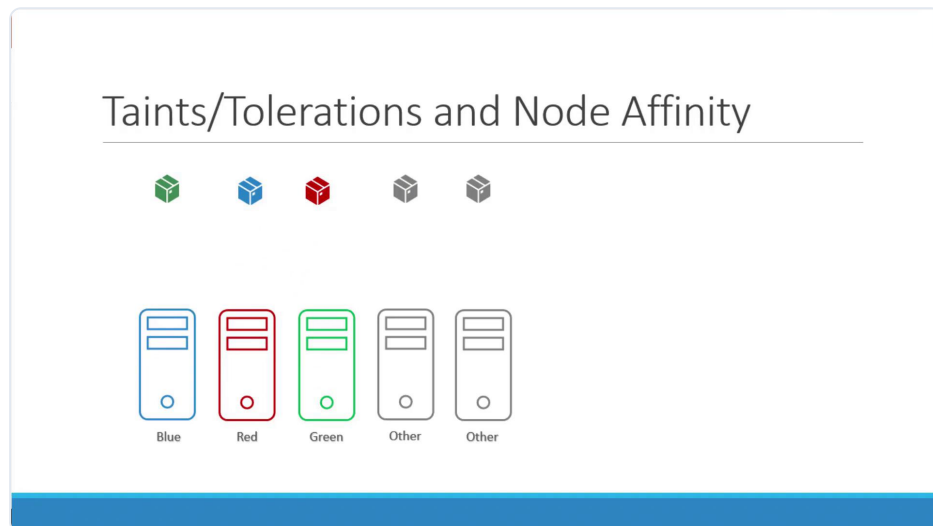
While node affinity directs pods to the correct nodes, it does not restrict other pods from also being scheduled on these nodes. This means that although our desired pods are correctly placed, the nodes might still host pods not meant for them.

## Combining Taints and Tolerations with Node Affinity

For exclusive node usage, combining both strategies is the optimal solution. The integration works as follows:

1. Apply taints on nodes and specify corresponding tolerations in pod configurations to block any pod without the proper toleration.
2. Use node affinity rules to ensure that each pod is only scheduled on a node with a matching label.

This combined approach dedicates the nodes exclusively to the intended pods, assuring correct pod assignments and preventing interference by other workloads.



In summary, leveraging both taints/tolerations and node affinity in Kubernetes ensures precise pod scheduling. This approach is particularly useful in multi-tenant clusters where exclusive node usage is critical.



## Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

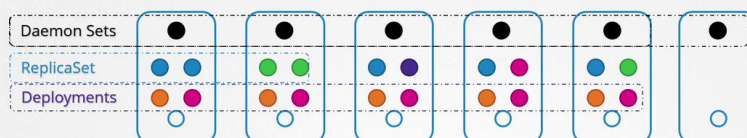
# DaemonSets

This comprehensive guide explores DaemonSets in Kubernetes, their use cases, and provides a step-by-step example for deployment in your cluster.

Welcome to this comprehensive guide on DaemonSets in Kubernetes. In this article, we will dive deep into how DaemonSets work, explore their primary use cases, and provide a step-by-step example to help you deploy one in your cluster.

DemonSets ensure that exactly one copy of a pod runs on every node in your Kubernetes cluster. When you add a new node, the DaemonSet automatically deploys the pod on the new node. Likewise, when a node is removed, the corresponding pod is also removed. This guarantees that a single instance of the pod is consistently available on each node.

## Daemon Sets

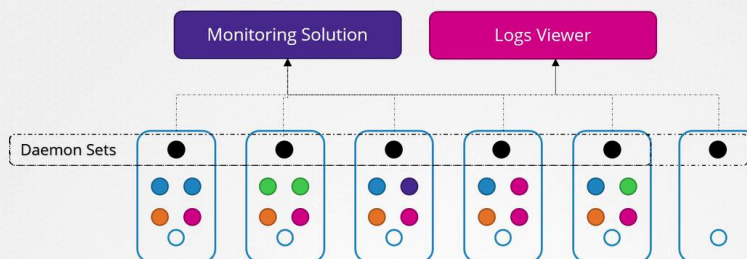


## Use Cases for DaemonSets

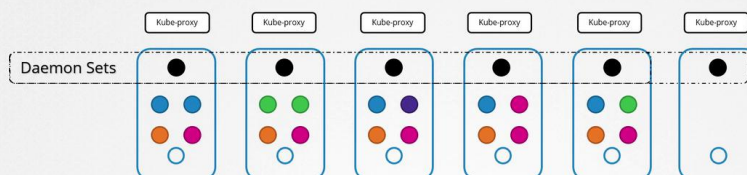
DaemonSets are particularly useful in scenarios where you need to run background services or agents on every node. Some common use cases include:

- **Monitoring agents and log collectors:** Deploy monitoring tools or log collectors across every node to ensure comprehensive cluster-wide visibility without manual intervention.
- **Essential Kubernetes components:** Deploy critical components, such as kube-proxy, which Kubernetes requires on all worker nodes.
- **Networking solutions:** Ensure consistent deployment of networking agents like those used in VNet or weave-net across all nodes.

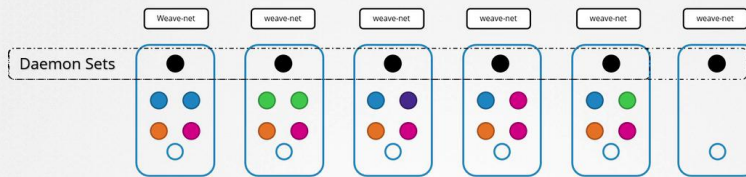
### | Daemon Sets – UseCase



### | Daemon Sets – UseCase – kube-proxy



## Daemon Sets – UseCase – Networking



## Creating a DaemonSet

Creating a DaemonSet is analogous to creating a ReplicaSet. The DaemonSet YAML configuration consists of a pod template under the `template` section and a selector that binds the DaemonSet to its pods. A typical DaemonSet definition includes the API version, kind, metadata, and specifications. Note that the API version is `apps/v1` and the kind is set to `DaemonSet`.

Below is an example DaemonSet definition file that deploys a monitoring agent:

```
```yaml theme={null}
```

daemon-set-definition.yaml

```
apiVersion: apps/v1 kind: DaemonSet metadata: name: monitoring-daemon
spec: selector: matchLabels: app: monitoring-agent template: metadata: labels:
app: monitoring-agent spec: containers: - name: monitoring-agent image:
monitoring-agent
```

```
:::note Important
```

```
Ensure that the labels in the selector match those in the pod template. Consistent labeling
:::
```

```
Once your YAML file is ready, create the DaemonSet using the following command:
```

```
```bash theme={null}
kubectl create -f daemon-set-definition.yaml
```

Verify the DaemonSet's creation by running:

```
```bash theme={null} kubectl get daemonsets
```

This command produces output similar to:

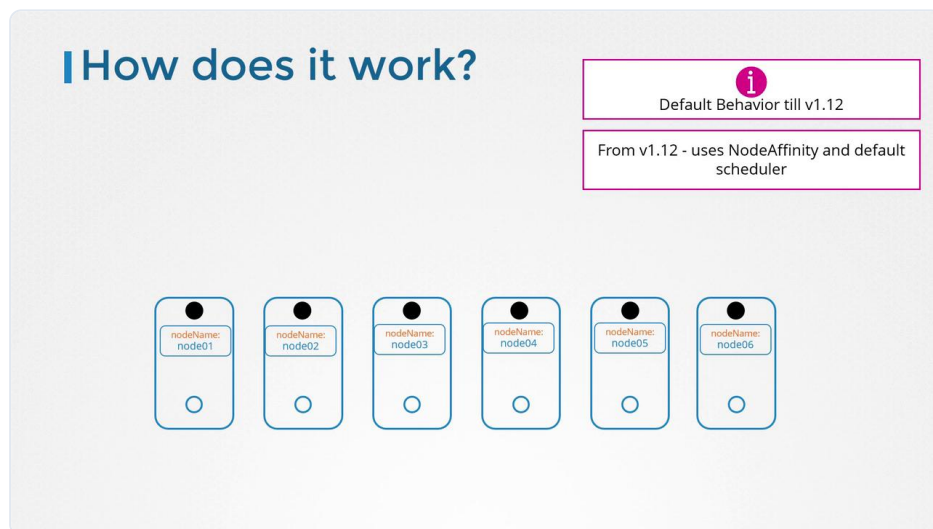
```
```console theme={null}
NAME DESIRED CURRENT READY UP-TO-DATE AVAILABLE AGE
monitoring-daemon 1 1 1 1 1 41
```

For more detailed information on your DaemonSet, use:

```
bash theme={null} kubectl describe daemonset monitoring-daemon
```

## How DaemonSets Schedule Pods

Prior to Kubernetes version 1.12, scheduling a pod on a specific node was often achieved by manually setting the `nodeName` property within the pod specification. However, since version 1.12, DaemonSets leverage the default scheduler in conjunction with node affinity rules. This improvement ensures that a pod is automatically scheduled on every node without manual intervention.



Tip DaemonSets are an ideal solution for deploying services that must run on every node, such as monitoring agents and essential networking components. Leveraging node affinity simplifies management as your cluster scales.

## Conclusion

DaemonSets provide an efficient mechanism to ensure that key services are uniformly deployed across your Kubernetes cluster. Whether you're running log collectors, monitoring agents, or essential network components like kube-proxy and weave-net, DaemonSets help maintain consistency and reliability in dynamic environments.

For further reading, explore these resources:

- [Kubernetes Documentation](#)
- [DaemonSets Concept](#)

Happy clustering!

## Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

# Static Pods

This lesson details static pods in Kubernetes, their configuration, and behavior in standalone and clustered environments.

In this lesson, we explore static pods in Kubernetes and how they operate independently of control plane components. Previously, we reviewed the Kubernetes architecture and explained the role of the kubelet as a primary component. Typically, the kubelet receives instructions from the kube-apiserver about which pods to run on its node. These decisions, made by the kube-scheduler and stored in the etcd data store, are standard in a clustered environment. But what happens when the kube-apiserver, kube-scheduler, and other control plane components are absent? How does the node function on its own?

Imagine being isolated at sea with no master or other nodes nearby. Can the kubelet still guide the node? The answer is yes. When the kubelet and a container runtime (such as Docker) are installed directly on the host without a Kubernetes cluster, the kubelet can independently manage the node. In this scenario, because there is no API server to provide pod details, you must supply the pod definition files directly to the kubelet.

Static pods are created directly by the kubelet without the intervention of the API server or other control plane components.

The kubelet is configured to monitor a designated directory on the host where pod definition files are stored. The kubelet periodically scans this directory, reads available files, and creates the corresponding pods. It also monitors these pods and ensures they remain running. For example, if an application crashes, the kubelet restarts it; if an updated file is detected, the kubelet recreates the pod to apply the changes; and if a file is removed, the corresponding pod is also deleted. These pods, created solely by the kubelet, are known as static pods.

It's important to note that only pod-level resources can be created this way. Higher-level abstractions such as ReplicaSets, Deployments, or Services depend on other control plane components (e.g., the replication and

deployment controllers) and cannot be managed via the static pod definition files.

## Configuring the Static Pods Directory

You can place static pods in any directory on the host. The directory location is provided to the kubelet at startup by using the `--pod-manifest-path` option. Below is an example configuration in the kubelet service file:

```
```bash theme={null} ExecStart=/usr/local/bin/kubelet \ --container-
runtime=remote \ --container-runtime-endpoint=unix:///var/run/containerd/
containerd.sock \ --pod-manifest-path=/etc/kubernetes/manifests \ --
kubeconfig=/var/lib/kubelet/kubeconfig \ --network-plugin=cni \ --register-
node=true \ --v=2
```

Alternatively, you can specify a configuration file that includes the manifest directory path.

```
```bash theme={null}
kubelet.service
ExecStart=/usr/local/bin/kubelet \
 --container-runtime=remote \
 --container-runtime-endpoint=unix:///var/run/containerd/containerd.sock \
 --config=kubeconfig.yaml \
 --kubeconfig=/var/lib/kubelet/kubeconfig \
 --network-plugin=cni \
 --register-node=true \
 --v=2
```

```
```yaml theme={null}
```

kubeconfig.yaml

staticPodPath: /etc/kubernetes/manifests

Many clusters created using kubeadm adopt this approach. When inspecting an existing cluster,

You can verify the status of static pods using container runtime commands. For instance, run

```
```bash theme={null}
docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS
8e5d4c4db7b6	busybox	"sh -c 'echo Hello K...'"	28 seconds ago	Up
f6737e1149cb	k8s.gcr.io/pause:3.1	"/pause"	24 seconds ago	Up

In a standalone scenario, use `docker ps` because the kube-apiserver is not available to process Kubernetes API requests.

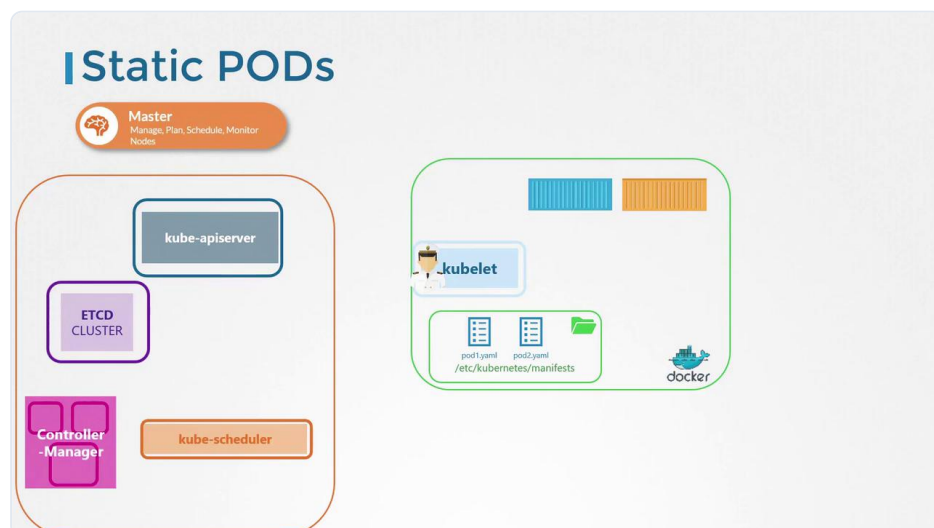
## Behavior When Part of a Cluster

When a node is part of a Kubernetes cluster, the kube-apiserver instructs the kubelet to create pods via its HTTP API endpoint. In this mixed mode, the kubelet handles pod definitions provided both from the static pod directory and from the API server. Whenever the kubelet creates a static pod in this configuration, it also creates a mirror object in the kube-apiserver. This mirror object is read-only and can be viewed with `kubectl get pods`, but you cannot modify or delete it through the API. To update a static pod, modify the file in the node's manifest directory.

For example, running the command below on the master node will display the static pod mirror:

```
bash theme={null} kubectl get pods NAME READY STATUS RESTARTS AGE
static-web-node01 0/1 ContainerCreating 0 29s
```

Notice that the pod name includes the node name (e.g., "node01") to indicate its origin.



## Static Pods vs. DaemonSets

A common question that arises is how static pods differ from DaemonSets. The table below summarizes the key differences between the two:



Feature	Static Pods	DaemonSets
Creation Source	Directly managed by the kubelet	Managed by the DaemonSet controller via the kube-apiserver
Control Plane Involvement	No API server interaction	Requires kube-apiserver communication
Use Case	Typically used for critical control plane components	Ensures a copy of a pod runs on every node
Interaction with Scheduler	Ignored by the kube-scheduler	Ignored by the kube-scheduler

Static pods are especially useful for deploying control plane components themselves. Once the kubelet is installed on all master nodes, you can create pod definition files for essential components like the API server and controller manager. By placing these files in the designated manifest folder, the kubelet ensures they are running as pods and restarts them automatically if they fail. When you check the pods in the kube-system namespace, you'll see these control plane components running as pods—a standard configuration in clusters set up using kubeadm.

## Static PODs vs DaemonSets

Static PODs	DaemonSets
Created by the Kubelet	Created by Kube-API server (DaemonSet Controller)
Deploy Control Plane components as Static Pods	Deploy Monitoring Agents, Logging Agents on nodes
Ignored by the Kube-Scheduler	

This lesson has detailed the concept of static pods, their configuration, and their behavior both in standalone and clustered environments. Using static pods is a simplified yet powerful method to manage critical workloads without relying on the full Kubernetes control plane.

That concludes this detailed discussion on static pods. Be sure to review and practice working with static pods in your upcoming tests to reinforce these concepts.

## Documentation Index

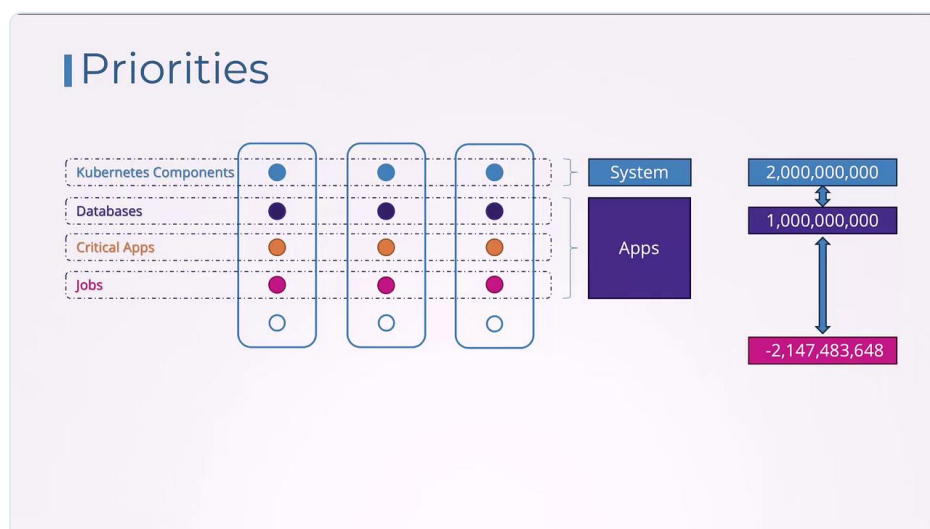
Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

# Priority Classes

Understanding priority classes is essential for managing workload scheduling in Kubernetes by assigning numerical values to Pods based on their importance.

Understanding priority classes is essential for managing workload scheduling in Kubernetes. Kubernetes runs various applications as Pods with different levels of importance. For instance, control plane components run within the cluster as Pods and are vital for its operation. Similarly, production databases and critical applications are high-priority while background jobs generally have lower priority. To ensure that more important workloads are scheduled before less critical ones, Kubernetes uses priority classes.

Priority classes allow you to assign a numerical value to Pods, where a higher number indicates higher priority. For user-deployed applications, the value can range from approximately -2 billion to +1 billion. Additionally, there is a reserved range for internal system-critical Pods (like the Kubernetes control plane) which can have values up to 2 billion.



note To check the current priority classes in your cluster, run the following command:

```
```bash theme={null} kubectl get priorityclass
```

The output may appear as follows:

```
```bash theme={null}
NAME VALUE GLOBAL-DEFAULT AGE PREEMPTIONPOLICY
system-cluster-critical 2000000000 false 7m33s PreemptLowerPriority
system-node-critical 2000010000 false 7m33s PreemptLowerPriority
```

## Creating a New Priority Class

To create a new priority class, define an object with the API version `scheduling.k8s.io/v1`, set the kind to `PriorityClass`, and include metadata with a name, numerical value, and an optional description. For example:

```
```yaml theme={null} apiVersion: scheduling.k8s.io/v1 kind: PriorityClass
metadata: name: high-priority value: 1000000000 description: "Priority class for
mission critical pods"
```

After creating the priority class, you can assign it to a Pod by specifying the `priorityClassName` field in the pod definition.

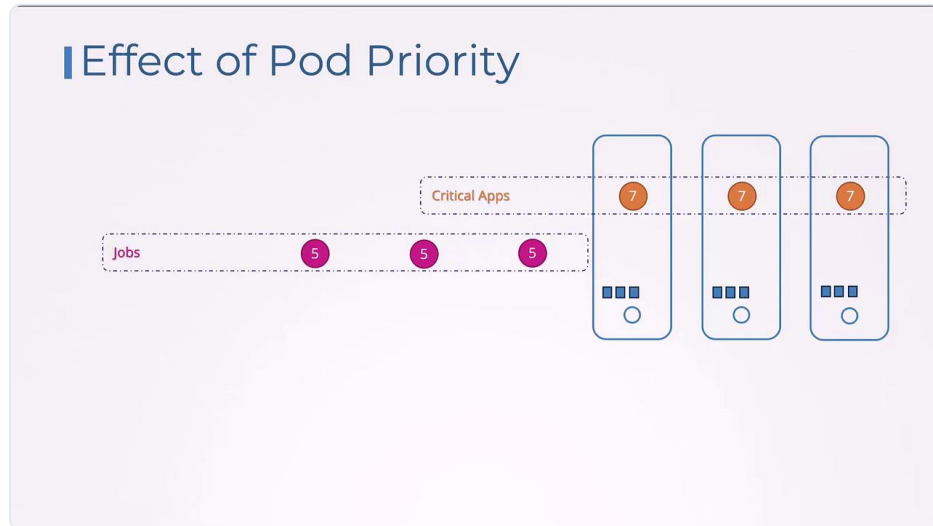
Below is an example that demonstrates both the creation of a priority class and how to use it in a pod definition.

```
```yaml theme={null}
priority-class.yaml
apiVersion: scheduling.k8s.io/v1
kind: PriorityClass
metadata:
 name: high-priority
 value: 1000000000
 description: "Priority class for mission critical pods"
 globalDefault: true

pod-definition.yaml
apiVersion: v1
kind: Pod
metadata:
 name: nginx
 labels:
 app: nginx
spec:
 containers:
 - name: nginx
 image: nginx
 ports:
 - containerPort: 8080
 priorityClassName: high-priority
```

## Pod Priority and Preemption

Consider a scenario where there are two workloads waiting to be scheduled: a critical application with a priority of 7 and a job with a priority of 5. With available resources, the higher priority critical application is scheduled first. If resources remain, the lower priority job is also scheduled.



Now, suppose a new job with a priority of 6 is submitted when no extra resources are available. Whether this new Pod preempts (or evicts) an existing lower priority Pod depends on the preemption policy defined in its priority class. By default, Kubernetes applies the `PreemptLowerPriority` policy, meaning the scheduler will evict lower priority Pods to free up resources for higher priority ones.

The following YAML snippet demonstrates setting the preemption policy to `PreemptLowerPriority`:

```
```yaml
theme={null}
apiVersion: scheduling.k8s.io/v1
kind: PriorityClass
metadata:
  name: high-priority
  value: 1000000000
description: "Priority class for mission critical pods"
preemptionPolicy: PreemptLowerPriority
```

If you prefer that a higher priority Pod waits for resources rather than preempting lower p

```
```yaml
theme={null}
apiVersion: scheduling.k8s.io/v1
kind: PriorityClass
metadata:
 name: high-priority
 value: 1000000000
description: "Priority class for mission critical pods"
preemptionPolicy: Never
```

:::note Additional Information For more details on Kubernetes scheduling and priority, refer to the [Kubernetes documentation](#). :::

This concludes the discussion on priority classes. You can now implement these concepts to better manage workload priorities in your Kubernetes cluster.

## Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

# Multiple Schedulers

This article explains how to deploy and configure multiple schedulers in a Kubernetes cluster alongside the default scheduler.

Welcome to this lesson on deploying multiple schedulers in a Kubernetes cluster. In this guide, you will learn how to deploy custom schedulers alongside the default scheduler, configure them correctly, and validate their operation.

Kubernetes' default scheduler distributes pods across nodes evenly while considering factors such as taints, tolerations, and node affinity. However, certain use cases may require a custom scheduling algorithm. For instance, when an application needs to perform extra verification before placing its components on specific nodes, a custom scheduler becomes essential. By writing your own scheduler, packaging it, and deploying it alongside the default scheduler, you can tailor pod placement to your specific needs.

Ensure that every additional scheduler has a unique name. The default scheduler is conventionally named "default-scheduler," and any custom scheduler must be registered with its own distinct name in the configuration files.

## Configuring Schedulers with YAML

Below are examples of configuration files for both the default and a custom scheduler. Each YAML file uses a profiles list to define the scheduler's name.

```
``yaml theme={null}
```

## my-scheduler-config.yaml

```
apiVersion: kubescheduler.config.k8s.io/v1 kind: KubeSchedulerConfiguration
profiles: - schedulerName: my-scheduler
```

```
```yaml theme={null}
# scheduler-config.yaml
apiVersion: kubescheduler.config.k8s.io/v1
kind: KubeSchedulerConfiguration
profiles:
  - schedulerName: default-scheduler
```

Deploying an Additional Scheduler

You can deploy an additional scheduler using the existing kube-scheduler binary, tailoring its configuration through specific service files.

Step 1: Download the kube-scheduler Binary

Begin by downloading the kube-scheduler binary:

```
```bash theme={null} wget https://storage.googleapis.com/kubernetes-release/
release/v1.12.0/bin/linux/amd64/kube-scheduler
```

#### ### Step 2: Create Service Files

Create separate service files for each scheduler. For example, consider the following definition:

```
```bash theme={null}
# kube-scheduler.service
ExecStart=/usr/local/bin/kube-scheduler --config=/etc/kubernetes/config/kube-scheduler.yaml
```

```
```bash theme={null}
```

## my-scheduler-2.service

```
ExecStart=/usr/local/bin/kube-scheduler --config=/etc/kubernetes/config/my-
scheduler-2-config.yaml
```

#### ### Step 3: Define Scheduler Configuration Files

Reference the scheduler names in the associated configuration files

```
```yaml theme={null}
# my-scheduler-2-config.yaml
apiVersion: kubescheduler.config.k8s.io/v1
kind: KubeSchedulerConfiguration
profiles:
  - schedulerName: my-scheduler-2
```

```
```yaml theme={null}
```

# my-scheduler-config.yaml

apiVersion: kubescheduler.config.k8s.io/v1 kind: KubeSchedulerConfiguration  
 profiles: - schedulerName: my-scheduler

<Callout icon="lightbulb" color="#1CB2FE">

Several code blocks might look similar or repeated. The examples above represent a console

</Callout>

\*\*\*

## Deploying the Custom Scheduler as a Pod

In addition to running the scheduler as a service, you can deploy it as a pod inside the Ku

### Example Pod Definition

```
```yaml theme={null}
apiVersion: v1
kind: Pod
metadata:
  name: my-custom-scheduler
  namespace: kube-system
spec:
  containers:
    - name: kube-scheduler
      image: k8s.gcr.io/kube-scheduler-amd64:v1.11.3
      command:
        - kube-scheduler
        - --address=127.0.0.1
        - --kubeconfig=/etc/kubernetes/scheduler.conf
        - --config=/etc/kubernetes/my-scheduler-config.yaml
```

The corresponding custom scheduler configuration file might look like:

```
```yaml theme={null} apiVersion: kubescheduler.config.k8s.io/v1 kind:
KubeSchedulerConfiguration profiles: - schedulerName: my-scheduler
```



```
<Callout icon="lightbulb" color="#1CB2FE">
 Leader election is an important configuration for high-availability environments. It ensures that only one scheduler is active at a time, preventing conflicts and ensuring consistent scheduling.
</Callout>

Deploying the Custom Scheduler as a Deployment

In many modern Kubernetes setups—especially those using kubeadm—control plane components run on dedicated nodes. This ensures high availability and security for the control plane.

Step 1: Build and Push a Custom Scheduler Image

Create a Dockerfile for your custom scheduler:

```dockerfile
theme={null}
FROM busybox
ADD ./output/local/bin/linux/amd64/kube-scheduler /usr/local/bin/kube-scheduler
```

Build and push the Docker image:

```
```bash
theme={null}
docker build -t gcr.io/my-gcp-project/my-kube-scheduler:1.0 .
gcloud docker -- push gcr.io/my-gcp-project/my-kube-scheduler:1.0
```

### ### Step 2: Create ServiceAccount and RBAC Configurations

Prepare the following YAML to create a service account and set appropriate RBAC permissions

```
```yaml theme={null}
apiVersion: v1
kind: ServiceAccount
metadata:
  name: my-scheduler
  namespace: kube-system
---
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRoleBinding
metadata:
  name: my-scheduler-as-kube-scheduler
subjects:
  - kind: ServiceAccount
    name: my-scheduler
    namespace: kube-system
roleRef:
  kind: ClusterRole
  name: system:kube-scheduler
  apiGroup: rbac.authorization.k8s.io
---
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRoleBinding
metadata:
  name: my-scheduler-as-volume-scheduler
subjects:
  - kind: ServiceAccount
    name: my-scheduler
    namespace: kube-system
roleRef:
  kind: ClusterRole
  name: system:volume-scheduler
  apiGroup: rbac.authorization.k8s.io
```
```

### Step 3: Create a ConfigMap for Scheduler Configuration

Define a ConfigMap that includes your custom scheduler configuration:

```
```yaml theme={null} apiVersion: v1 kind: ConfigMap metadata: name: my-
scheduler-config namespace: kube-system data: my-scheduler-config.yaml: |
apiVersion: kubescheduler.config.k8s.io/v1beta2 kind:
KubeSchedulerConfiguration profiles: - schedulerName: my-scheduler
leaderElection: leaderElect: false
```
```

### ### Step 4: Define the Deployment

Deploy the custom scheduler as a Deployment with the following YAML

```

```yaml theme={null}
apiVersion: apps/v1
kind: Deployment
metadata:
  name: my-scheduler
  namespace: kube-system
  labels:
    component: scheduler
    tier: control-plane
spec:
  replicas: 1
  selector:
    matchLabels:
      component: scheduler
      tier: control-plane
  template:
    metadata:
      labels:
        component: scheduler
        tier: control-plane
        version: second
    spec:
      serviceAccountName: my-scheduler
      containers:
        - name: kube-second-scheduler
          image: gcr.io/my-gcp-project/my-kube-scheduler:1.0
          command:
            - /usr/local/bin/kube-scheduler
            - --config=/etc/kubernetes/my-scheduler/my-scheduler-config.yaml
          livenessProbe:
            httpGet:
              path: /healthz
              port: 10259
              scheme: HTTPS
            initialDelaySeconds: 15
          readinessProbe:
            httpGet:
              path: /healthz
              port: 10259
              scheme: HTTPS
          volumeMounts:
            - name: config-volume
              mountPath: /etc/kubernetes/my-scheduler
      volumes:
        - name: config-volume
          configMap:
            name: my-scheduler-config

```

Also, ensure a proper ClusterRole exists for the scheduler. For example:

```

```yaml theme={null} apiVersion: rbac.authorization.k8s.io/v1 kind: ClusterRole
metadata: name: system:kube-scheduler annotations:

```

```
rbac.authorization.kubernetes.io/autoupdate: "true" labels: kubernetes.io/
bootstrapping: rbac-defaults rules: - apiGroups: - coordination.k8s.io
resources: - leases verbs: - create - apiGroups: - coordination.k8s.io
resourceNames: - kube-scheduler - my-scheduler resources: - leases verbs: -
get - list - watch
```

```

```

### ## Configuring Workloads to Use the Custom Scheduler

To have specific pods or deployments use your custom scheduler, add the "schedulerName" field

```
```yaml theme={null}
apiVersion: v1
kind: Pod
metadata:
  name: nginx
spec:
  containers:
    - name: nginx
      image: nginx
  schedulerName: my-custom-scheduler
```

Deploy this pod with:

```
```bash theme={null} kubectl create -f pod-definition.yaml
```

If the custom scheduler configuration is incorrect, the pod may remain in the Pending state

```

```

### ## Verifying Scheduler Operation

To confirm which scheduler assigned a pod, review the events in your namespace:

```
```bash theme={null}
kubectl get events -o wide
```

A sample output might appear as follows:

```
```text theme={null}
LAST SEEN COUNT NAME KIND TYPE REASON
SOURCE MESSAGE
9s 1 nginx.15 Pod Normal Scheduled my-custom-
scheduler Successfully assigned default/nginx to node01
8s 1 nginx.15 Pod
Normal Pulling kubelet, node01 pulling image "nginx"
2s 1 nginx.15 Pod
Normal Pulled kubelet, node01 Successfully pulled image "nginx"
2s 1
nginx.15 Pod Normal Created kubelet, node01 Created container
2s 1
nginx.15 Pod Normal Started kubelet, node01 Started container
```

```
Notice that the event source is "my-custom-scheduler," confirming that the pod was scheduled by the custom scheduler.

If you encounter issues, view the scheduler logs with:

```bash theme={null}
kubectl logs my-custom-scheduler --namespace=kube-system
```

A sample log output might include messages like:

```
text theme={null} I0204 09:42:25.819338 1 server.go:126] Version:
v1.11.3 W0204 09:42:25.822720 1 authorization.go:47] Authorization
is disabled W0204 09:42:25.822745 1 authentication.go:55]
Authentication is disabled I0204 09:42:25.822801 1
insecure_serving.go:47] Serving healthz insecurely on
127.0.0.1:10251 I0204 09:45:14.725407 1 controller_utils.go:1025]
Waiting for caches to sync for scheduler controller I0204
09:45:14.825634 1 controller_utils.go:1032] Caches are synced for
scheduler controller I0204 09:45:14.825814 1 leaderelection.go:185]
attempting to acquire leader lease kube-system/my-custom-
scheduler... I0204 09:45:14.834953 1 leaderelection.go:194]
successfully acquired lease kube-system/my-custom-scheduler
```

This confirms that the custom scheduler is up and functioning as expected.

Conclusion

By following these techniques, you can run both the default Kubernetes scheduler and one or more custom schedulers concurrently. This flexibility allows you to assign specific workloads to the most appropriate scheduler based on your cluster's requirements.

Happy scheduling!

Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

Configuring Scheduler Profiles

This article explains configuring scheduler profiles in Kubernetes to customize scheduling behavior and manage multiple profiles within a single scheduler binary.

In this lesson, we dive into the concept of scheduler profiles and their configuration in Kubernetes. We will start with a refresher on how the Kubernetes scheduler functions, illustrated by a simple example where a pod is scheduled to one of several available nodes.

How Scheduling Works

When a pod is defined, it enters a scheduling queue along with other pending pods. Consider a pod that requires 10 CPU; it will only be scheduled on nodes with at least 10 available CPUs. Additionally, pods with higher priorities are placed at the beginning of the queue. For instance, the following pod definition uses a high-priority class:

```
```yaml theme={null} apiVersion: v1 kind: Pod metadata: name: simple-webapp-color spec: priorityClassName: high-priority containers: - name: simple-webapp-color image: simple-webapp-color resources: requests: memory: "1Gi" cpu: 10
```

Before using this priority, you must create a priority class with a specific name and a priority value.

## ## Scheduling Phases

After being queued, pods progress through several phases:

1. **Filter Phase:** Nodes that cannot meet the pod's resource requirements (e.g., nodes with insufficient CPU or memory).
2. **Scoring Phase:** Remaining nodes are scored based on resource availability after resource requirements are filtered out.
3. **Binding Phase:** The pod is assigned to the node with the highest score.

## ### Key Scheduler Plugins

Several scheduler plugins play critical roles during these phases:

- \* **Priority Sort Plugin:** Sorts pods in the scheduling queue according to their priority.
- \* **Node Resources Fit Plugin:** Filters out nodes that do not have the needed resources.
- \* **Node Name Plugin:** Checks for a specific node name in the pod specification and filters out nodes that do not match.
- \* **Node Unschedulable Plugin:** Excludes nodes marked as unschedulable. For instance, running `kubectl describe node controlplane` shows:

```

```bash
theme={null}
controlplane ~ → kubectl describe node controlplane
Name:                controlplane
Roles:               control-plane
CreationTimestamp:   Thu, 06 Oct 2022 06:19:57 -0400
Taints:              node.kubernetes.io/unschedulable:NoSchedule
Unschedulable:      true
Lease:
```

```

- \* **Scoring Plugins:** During the scoring phase, plugins (such as the Node Resources Fit and Node Name plugins) calculate a score for each node.
- \* **Default Binder Plugin:** Finalizes the scheduling process by binding the pod to the selected node.

**<Callout icon="lightbulb" color="#1CB2FE">**  
 Kubernetes emphasizes extensibility, allowing you to modify the scheduling process via extensions.  
**</Callout>**

The following image outlines the various extension points of the Kubernetes scheduler, including the Scheduling Queue, Filtering, and Binding phases.

**<Frame>**  
 ![The image outlines Kubernetes scheduler extension points: Scheduling Queue, Filtering, and Binding phases.]  
**</Frame>**

## ## Customizing Scheduler Behavior with Profiles

Rather than running separate scheduler binaries (like default scheduler, MyScheduler, and MyScheduler2), you can use scheduler profiles to customize the scheduler's behavior.

## ### Profile Configuration

Each scheduler profile is defined within the scheduler configuration file and behaves like a scheduler binary.

```

```yaml
theme={null}
# my-scheduler-2-config.yaml
apiVersion: kubescheduler.config.k8s.io/v1

```

```
kind: KubeSchedulerConfiguration
profiles:
  - schedulerName: my-scheduler-2
  - schedulerName: my-scheduler-3
```

```
```yaml theme={null}
```

## my-scheduler-config.yaml

```
apiVersion: kubescheduler.config.k8s.io/v1 kind: KubeSchedulerConfiguration
profiles: - schedulerName: my-scheduler
```

```
```yaml theme={null}
# scheduler-config.yaml
apiVersion: kubescheduler.config.k8s.io/v1
kind: KubeSchedulerConfiguration
profiles:
  - schedulerName: default-scheduler
```

Each profile allows you to customize plugin configurations. For instance, you could disable specific plugins or enable custom ones. Below is an example where the "my-scheduler-2" profile disables the TaintToleration plugin and enables two custom plugins (MyCustomPluginA and MyCustomPluginB). Additionally, the "my-scheduler-3" profile disables all preScore and score plugins:

```
yaml theme={null} apiVersion: kubescheduler.config.k8s.io/v1 kind:
KubeSchedulerConfiguration profiles: - schedulerName: my-scheduler-2
plugins: score: disabled: - name: TaintToleration enabled: - name:
MyCustomPluginA - name: MyCustomPluginB - schedulerName: my-
scheduler-3 plugins: preScore: disabled: - name: '*' score:
disabled: - name: '*' - schedulerName: my-scheduler-4
```

In the plugins section, specify the extension point and then enable or disable plugins by name (or using a wildcard pattern).

This flexible configuration allows you to tailor the scheduling behavior to meet your unique workload requirements by selectively enabling or disabling plugins across different profiles.

Summary

This lesson provided an overview of Kubernetes scheduling and scheduler profiles. We covered:

- The phases of scheduling: queueing, filtering, scoring, and binding.
- The role of various scheduler plugins and extension points.

- How to configure multiple scheduler profiles within a single scheduler binary to customize scheduling behavior.

For further reading, consider exploring the official documentation on [Kubernetes Scheduling](#) and multi-scheduler profiles.

That's all for this lesson. See you in the next one!

Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

2025 Updates Admission Controllers

This article explores admission controllers in Kubernetes, focusing on their role in enhancing security and enforcing policies before object persistence.

In this lesson, we explore admission controllers in Kubernetes and understand how they enhance security and enforce policies before object persistence in etcd. Every operation performed using the kubectl command-line utility—such as creating a pod—is first sent as a request to the API server. The API server then processes the request and stores its information.

When a request reaches the API server, it is first handled by an authentication process. For instance, when using kubectl, the required certificates for authentication are provided in the KubeConfig file:

```
``bash theme={null} cat ~/.kube/config apiVersion: v1 clusters: - cluster:
certificate-authority-data: LS0tLS1CRUdJTiBDRVU...
```

After successful authentication, the request undergoes an authorization process using role-

```
``yaml theme={null}
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: developer
rules:
- apiGroups: [""]
  resources: ["pods"]
  verbs: ["list", "get", "create", "update", "delete"]
```

This configuration permits a user assigned to the developer role to list, get, create, update, and delete pods. RBAC rules can be further refined to target specific resource names. For instance, to restrict a developer so they can only create pods with designated names:

```
```yaml theme={null} apiVersion: rbac.authorization.k8s.io/v1 kind: Role
metadata: name: developer rules: - apiGroups: [""] resources: ["pods"] verbs:
["create"] resourceNames: ["blue", "orange"]
```

However, object-level permissions may not be sufficient in certain scenarios. When a pod is created, the RBAC rules are not applied to the pod's contents.

```
```yaml theme={null}
apiVersion: v1
kind: Pod
metadata:
  name: web-pod
spec:
  containers:
    - name: ubuntu
      image: ubuntu:latest
      command: ["sleep", "3600"]
      securityContext:
        runAsUser: 0
      capabilities:
        add: ["MAC_ADMIN"]
```

Standard RBAC rules operate only at the API level and cannot inspect or modify an object's contents. This limitation is overcome by admission controllers, which validate or even mutate requests prior to persisting objects. Admission controllers can enforce specific policies, such as:

- Changing requests based on internal guidelines.
- Enforcing container image policies.
- Ensuring that certain metadata labels are always applied.

Kubernetes includes a variety of built-in admission controllers. Some common examples include:

- **Always Pull Images:** Forces the pod to pull images from the registry every time.
- **Default Storage Class:** Automatically adds a default storage class to PersistentVolumeClaims (PVCs) when none is provided.
- **Event Rate Limit:** Restricts the API server's request-handling rate.
- **Namespace Exists:** Ensures that requested namespaces exist before proceeding.

Namespace Admission Controllers

The namespace admission controller ensures that pods are created only in existing namespaces. For example, if you run:

```
```bash theme={null} kubectl run nginx --image nginx --namespace blue
```

and if the "blue" namespace does not exist, you will receive an error like:

```
```bash theme={null}
Error from server (NotFound): namespaces "blue" not found
```

In this situation, after authentication and authorization, the namespace admission controller checks for the existence of the "blue" namespace and rejects the request since it does not exist.

Alternatively, Kubernetes offers the namespace auto-provision admission controller, which automatically creates a namespace if it does not exist (this feature is disabled by default). With the auto-provision controller enabled, executing the same command:

```
```bash theme={null} kubectl run nginx --image nginx --namespace blue
```

results in the automatic creation of the "blue" namespace and a successful pod creation.

To view the admission controllers enabled by default, run:

```
```bash theme={null}
kube-apiserver -h | grep enable-admission-plugins
```

This command will list plugins such as NamespaceLifecycle, LimitRanger, ServiceAccount, TaintNodesByCondition, among others. If you are using a kubeadm-based setup, run the command within the kube-apiserver control plane pod using kubectl exec.

Enabling Admission Controllers

To add an admission controller, update the `--enable-admission-plugins` flag on the Kube API server. In a kubeadm-based setup, this involves modifying the Kube API server manifest. For example, update the ExecStart command in the systemd service file as below:

```
```plaintext theme={null} ExecStart=/usr/local/bin/kube-apiserver \ --advertise-
address=${INTERNAL_IP} \ --allow-privileged=true \ --apiserver-count=3 \ --
authorization-mode=Node,RBAC \ --bind-address=0.0.0.0 \ --enable-swagger-
ui=true \ --etcd-servers=https://127.0.0.1:2379 \ --event-ttl=1h \ --runtime-
config=api/all \ --service-cluster-ip-range=10.32.0.0/24 \ --service-node-port-
range=30000-32767 \ --v=2 \ --enable-admission-
plugins=NodeRestriction,NamespaceAutoProvision
```

For kubeadm-based setups where the API server runs as a pod, the manifest might look like this:

```
```yaml
theme: {null}
apiVersion: v1
kind: Pod
metadata:
  creationTimestamp: null
  name: kube-apiserver
  namespace: kube-system
spec:
  containers:
  - command:
    - kube-apiserver
    - --authorization-mode=Node,RBAC
    - --advertise-address=172.17.0.107
    - --allow-privileged=true
    - --enable-bootstrap-token-auth=true
    - --enable-admission-plugins=NodeRestriction,NamespaceAutoProvisioning
    image: k8s.gcr.io/kube-apiserver-amd64:v1.11.3
    name: kube-apiserver
```

To disable specific admission controller plugins, use the `--disable-admission-plugins` flag similarly.

After updating your configuration, running the following command in a non-existent namespace:

```
```bash
theme: {null}
kubectl run nginx --image nginx --namespace blue
```

should output:

```
```bash
theme: {null}
Pod/nginx created!
```

Verifying the available namespaces:

```
```bash
theme: {null}
kubectl get namespaces
```

will display:

```
```bash
theme: {null}
NAME          STATUS    AGE
blue          Active   3m
default       Active  23m
kube-public   Active  24m
kube-system   Active  24m
```

This demonstrates how admission controllers not only reject invalid requests but can also perform backend operations like automatically creating a namespace.

Both the namespace auto-provision and namespace existence admission controllers are deprecated. They have been replaced by the namespace

lifecycle admission controller, which enforces that requests to non-existent namespaces are rejected and protects default namespaces (default, kube-system, and kube-public) from deletion.

That's it for now. Practice using admission controllers in your exercises to reinforce your learnings. See you in the next lesson!

Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

2025 Updates Validating and Mutating Admission Controllers

This article explores Kubernetes Admission Controllers, detailing validating and mutating types, their configurations, and how to implement custom external admission webhooks.

This article provides an in-depth look at Kubernetes Admission Controllers, focusing on both validating and mutating types. Learn how these controllers work, inspect typical API requests, and explore how to configure custom external admission webhooks for advanced validations and mutations.

Validating Admission Controllers

Validating admission controllers verify that an object meets specific criteria before it is persisted in the cluster. For example, the namespace existence or namespace lifecycle admission controller checks if a namespace exists and, if not, rejects the incoming request. Another example is the default storage class admission controller. When you create a PersistentVolumeClaim (PVC) without specifying a storage class, this validating controller intercepts the request and modifies it by adding the default storage class.

Consider the initial PVC creation request:

```
```yaml theme={null} apiVersion: v1 kind: PersistentVolumeClaim metadata:
name: myclaim spec: accessModes: - ReadWriteOnce resources: requests:
storage: 500Mi
```

After the request passes through authentication, authorization, and the admission controllers,

```
``yaml theme={null}
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
 name: myclaim
spec:
 accessModes:
 - ReadWriteOnce
 resources:
 requests:
 storage: 500Mi
 storageClassName: default
```

Since the controller modifies the request, such controllers are also considered mutating admission controllers.

## Mutating vs. Validating Admission Controllers

---

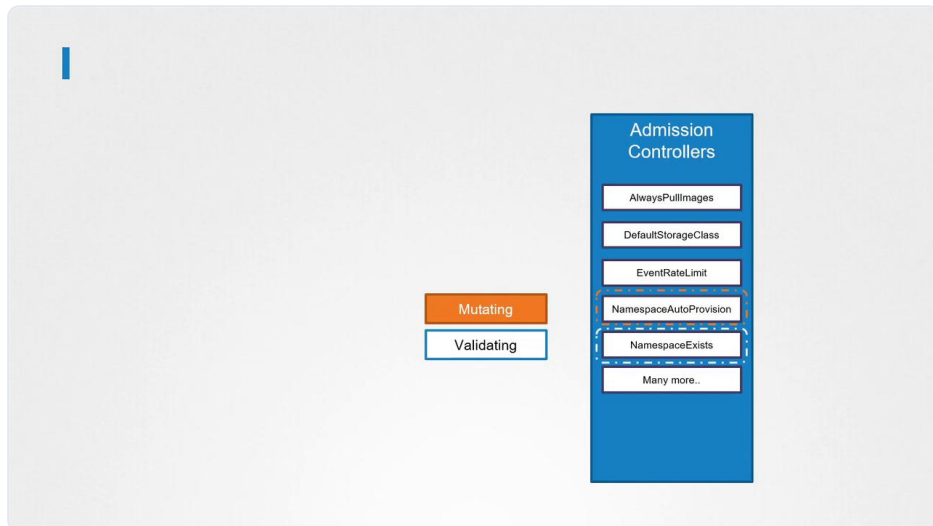
Admission controllers in Kubernetes can be classified into two main types:

- **Mutating Admission Controllers:** Modify (mutate) objects before they are persisted.
- **Validating Admission Controllers:** Validate objects to ensure they meet specific criteria, allowing or denying the request accordingly.

Some controllers perform both mutation and validation. Typically, mutating controllers run first so that subsequent validating controllers can work with the modified object. For instance, if a namespace auto-provisioning mutating admission controller creates a missing namespace before the namespace existence validating controller runs, the request proceeds smoothly. However, if the validating controller executes first, it would reject the request due to the missing namespace.

If any admission controller (mutating or validating) rejects a request, the entire request is denied and an error is returned to the user.





All built-in admission controllers are part of the Kubernetes source code. But for custom validations and mutations, Kubernetes supports external admission controllers using webhook mechanisms:

- Mutating Admission Webhook
- Validating Admission Webhook

## Configuring External Admission Webhooks

External admission webhooks can point to servers either inside or outside the Kubernetes cluster. Once the built-in admission controllers finish processing, the API server sends an AdmissionReview object (in JSON format) containing request details such as user information, operation type, and object metadata to the external webhook.

Below is an example AdmissionReview request sent to a webhook server:

```
```json  theme={null}  {  "apiVersion":  "admission.k8s.io/v1",  "kind":  "AdmissionReview",  "request":  {    "uid":    "705ab415-6393-11e7-b7cc-4201a8000002",    "kind": {"group": "autoscaling", "version": "v1", "kind": "Scale"},    "resource": {"group": "apps", "version": "v1", "resource": "deployments"},    "subresource": "scale",    "requestKind": {"group": "autoscaling", "version": "v1", "kind": "Scale"},    "requestResource": {"group": "apps", "version": "v1", "resource": "deployments"}  }  }
```

The webhook server processes the AdmissionReview request and responds with its own AdmissionReview response.

```
```json theme={null}
{
 "apiVersion": "admission.k8s.io/v1",
 "kind": "AdmissionReview",
 "request": {
 "uid": "705ab41f-6393-11e8-b7cc-4201a8000002",
 "kind": {
 "group": "autoscaling",
 "version": "v1",
 "kind": "Scale"
 },
 "resource": {
 "group": "apps",
 "version": "v1",
 "resource": "deployments"
 },
 "subResource": "scale",
 "requestKind": {
 "group": "autoscaling",
 "version": "v1",
 "kind": "Scale"
 },
 "requestResource": {
 "group": "apps",
 "version": "v1",
 "resource": "deployments"
 }
 },
 "response": {
 "uid": "<value_from request.uid>",
 "allowed": true
 }
}
```

If the "allowed" field is set to false, the API server rejects the request.

## Deploying Your Admission Webhook Server

To implement your custom webhook server, you must deploy it and ensure it can support the required mutation and validation APIs by returning correct JSON responses. The following Go code snippet illustrates a basic webhook server setup:

```
```go theme={null} package main

import ( "encoding/json" "flag" "fmt" "io/ioutil" "net/http" "k8s.io/api/admission/v1beta1" metav1 "k8s.io/apimachinery/pkg/apis/meta/v1" "k8s.io/klog" )

// toAdmissionResponse creates an AdmissionResponse with an error message. func toAdmissionResponse(err error) v1beta1.AdmissionResponse
```

```
{ return v1beta1.AdmissionResponse{ Result: &metav1.Status{ Message:
err.Error(), }, } }
```

// admitFunc defines the signature for validators and mutators. type admitFunc
func(v1beta1.AdmissionReview) v1beta1.AdmissionResponse

// serve processes the HTTP request before calling the admitFunc. func
serve(w http.ResponseWriter, r *http.Request, admit admitFunc) { var data
[]byte if r.Body == nil { return } data, err := ioutil.ReadAll(r.Body) if err != nil
{ return } // Additional processing logic goes here... }

Below is a pseudocode example of a webhook server implemented in Python. This example demon

Python Webhook Server Example

In this Python example, the "/validate" endpoint rejects requests where the object's name m

```

`python theme={null}
from flask import Flask, request, jsonify
import base64

app = Flask(__name__)

@app.route("/validate", methods=["POST"])
def validate():
    object_name = request.json["request"]["object"]["metadata"]["name"]
    user_name = request.json["request"]["userInfo"]["name"]
    status = True
    message = ""
    if object_name == user_name:
        message = "You can't create objects with your own name"
        status = False
    return jsonify(
        {
            "response": {
                "allowed": status,
                "uid": request.json["request"]["uid"],
                "status": {"message": message},
            }
        }
    )

@app.route("/mutate", methods=["POST"])
def mutate():
    user_name = request.json["request"]["userInfo"]["name"]
    patch = [{"op": "add", "path": "/metadata/labels/users", "value": user_name}]
    encoded_patch = base64.b64encode(str(patch).encode()).decode()
    return jsonify(
        {
            "response": {
                "allowed": True,
                "uid": request.json["request"]["uid"],
                "patch": encoded_patch,
                "patchType": "JSONPatch",
            }
        }
    )

if __name__ == "__main__":
    app.run(port=443, debug=True)

```

Even if you may not need to write such code for an exam, understanding the flow and structure of AdmissionReview objects is essential for managing custom admission controllers in Kubernetes.

Hosting the Webhook Server and Configuring Webhooks

After developing your webhook server, deploy it either as a standalone service or containerize it and run it within your Kubernetes cluster. If running inside the cluster, ensure that the server is accessible via a Kubernetes Service.

To instruct the API server to use your webhook for validations or mutations, create a `ValidatingWebhookConfiguration` or a `MutatingWebhookConfiguration` object. Below is an example configuration for a validating webhook that triggers on pod creation:

```
yaml theme={null} apiVersion: admissionregistration.k8s.io/v1 kind:
ValidatingWebhookConfiguration metadata: name: "pod-
policy.example.com" webhooks: - name: "pod-policy.example.com"
clientConfig: service: namespace: "webhook-namespace" name:
"webhook-service" caBundle: "Ci0tLS0tQk.....tLS0K" rules: -
apiGroups: [""] apiVersions: ["v1"] operations: ["CREATE"]
resources: ["pods"] scope: "Namespaced"
```

In the configuration above:

- The **clientConfig** block specifies how the API server connects to your webhook service, including the TLS certificate bundle (`caBundle`).
- The **rules** section defines the operations that trigger the webhook—in this case, whenever a pod is created.

Once this configuration is applied, the Kubernetes API server will call your webhook service for each relevant pod creation event, and the request will be allowed or rejected according to the logic implemented in your webhook.

Conclusion

This guide has provided an overview of Kubernetes admission controllers—both validating and mutating—and demonstrated how to configure external admission webhooks. By applying these concepts, you can enhance the security, flexibility, and management of your Kubernetes clusters.

For further reading, check out these resources:

- [Kubernetes Documentation](#)
- [Kubernetes Basics](#)
- [Terraform Registry](#)

Experiment with these configurations and webhook implementations in a test environment to gain hands-on experience and further solidify your understanding of Kubernetes Admission Controllers.

Section: Logging & Monitoring

Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

Logging and Monitoring Section Introduction

This article covers logging and monitoring techniques for Kubernetes clusters and applications, focusing on performance tracking and log management.

Hello and welcome to this comprehensive lesson on logging and monitoring. In this session, we will explore a variety of options to effectively monitor your Kubernetes cluster components and the applications running on them.

We start by examining key monitoring techniques for Kubernetes clusters. You will learn how to keep track of both the cluster components and the individual applications to ensure optimal performance and rapid issue detection.

Next, we delve into methods for accessing and managing logs from your cluster components as well as the applications. These logs are crucial for troubleshooting issues and understanding system behavior.

Throughout this lesson, you'll engage with practical exercises designed to reinforce these concepts, ensuring you can apply what you learn in real-world scenarios.

Practice regularly using these exercises to become proficient in managing logs and monitoring your Kubernetes environment.

Let's get started on our journey to mastering logging and monitoring!

Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

Managing Application Logs

This article provides a guide on managing application logs in Kubernetes, covering logging mechanisms in Docker and Kubernetes for effective monitoring and troubleshooting.

Welcome to this comprehensive guide on managing application logs in Kubernetes. In this article, we explore various logging mechanisms, starting with Docker and moving on to Kubernetes, to help you monitor and troubleshoot your applications effectively.

Logging in Docker

Docker containers typically log events to the standard output. Consider the "event simulator" container, which generates random events simulating a web server. When you run this container, it writes log entries such as:

```
```bash theme={null} docker run kodekloud/event-simulator 2018-10-06
15:57:15,937 - root - INFO - USER1 logged in 2018-10-06 15:57:16,943 - root
- INFO - USER2 logged out 2018-10-06 15:57:17,944 - root - INFO - USER3 is
viewing page3 2018-10-06 15:57:18,951 - root - INFO - USER4 is viewing
page1 2018-10-06 15:57:19,954 - root - INFO - USER1 logged out 2018-10-06
15:57:21,956 - root - INFO - USER1 logged in 2018-10-06 15:57:22,957 - root
- INFO - USER3 is viewing page2 2018-10-06 15:57:23,959 - root - INFO -
USER1 logged out 2018-10-06 15:57:24,959 - root - INFO - USER2 is viewing
page2 2018-10-06 15:57:25,962 - root - INFO - USER4 is viewing page3
2018-10-06 15:57:26,965 - root - INFO - USER3 is viewing page1 2018-10-06
15:57:27,965 - root - INFO - USER3 logged out 2018-10-06 15:57:29,967 -
root - INFO - USER1 is viewing page2
```

If you run the container in detached mode using the `-d` flag, the logs will not appear on

```
```bash theme={null}
docker run -d kodekloud/event-simulator
docker logs -f <container_id>
```

Logging in Kubernetes

Deploying the same Docker image within a Kubernetes pod leverages Kubernetes' logging capabilities. To get started, create a pod using the following YAML definition:

```
```yaml theme={null} apiVersion: v1 kind: Pod metadata: name: event-simulator-pod spec: containers: - name: event-simulator image: kodekloud/event-simulator
```

Create the pod with this command:

```
```bash theme={null}
kubectl create -f event-simulator.yaml
```

Once the pod is running, view the live logs using:

```
```bash theme={null} kubectl logs -f event-simulator-pod
```

This command outputs logs similar to the Docker example:

```
```bash theme={null}
2018-10-06 15:57:15,937 - root - INFO - USER1 logged in
2018-10-06 15:57:16,943 - root - INFO - USER2 logged out
2018-10-06 15:57:17,944 - root - INFO - USER2 is viewing page2
2018-10-06 15:57:18,951 - root - INFO - USER3 is viewing page3
2018-10-06 15:57:20,095 - root - INFO - USER4 is viewing page1
2018-10-06 15:57:21,956 - root - INFO - USER2 logged out
2018-10-06 15:57:21,956 - root - INFO - USER1 logged in
2018-10-06 15:57:23,093 - root - INFO - USER3 is viewing page2
2018-10-06 15:57:24,959 - root - INFO - USER1 logged out
2018-10-06 15:57:25,961 - root - INFO - USER2 is viewing page2
2018-10-06 15:57:25,961 - root - INFO - USER1 logged in
```

For more effective troubleshooting, use log filtering and analysis tools in combination with Kubernetes logs.

Logging with Multiple Containers in a Pod

Kubernetes supports pods with multiple containers. If you update your pod definition to include an additional container named `image-processor`, the configuration will look like this:

```
```yaml theme={null} apiVersion: v1 kind: Pod metadata: name: event-simulator-pod spec: containers: - name: event-simulator image: kodekloud/event-simulator - name: image-processor image: some-image-processor
```

Create the pod using:

```
```bash theme={null}
kubectl create -f event-simulator.yaml
```

Attempting to view logs without specifying the container when multiple containers are present will result in an error. Instead, specify the container name to view its logs:

```
```bash theme={null} kubectl logs -f event-simulator-pod event-simulator
```

This command displays log output for the `event-simulator` container:

```
```bash theme={null}
2018-10-06 15:57:15,937 - root - INFO - USER1 logged in
2018-10-06 15:57:16,943 - root - INFO - USER2 logged out
2018-10-06 15:57:17,944 - root - INFO - USER2 is viewing page2
2018-10-06 15:57:18,951 - root - INFO - USER3 is viewing page3
2018-10-06 15:57:19,954 - root - INFO - USER4 is viewing page1
2018-10-06 15:57:20,955 - root - INFO - USER2 logged out
```

When working with pods that contain multiple containers, always specify the container name in the `kubectl logs` command to avoid errors.

Additional Resources

For further information on logging and monitoring in Kubernetes, refer to the following resources:

- [Kubernetes Logging Guide](#)
- [Docker Logging Drivers](#)
- [Kubernetes Documentation](#)

By mastering these logging techniques, you ensure efficient monitoring and troubleshooting of your applications in both Docker and Kubernetes environments. Happy logging!

Thank you.

Section: Application Lifecycle Management

Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

Application Lifecycle Management Section Introduction

This article covers essential aspects of application lifecycle management, including rolling updates, rollbacks, configuration methods, scaling strategies, and self-healing design fundamentals.

Hello, my name is Mumshad Mannambeth, and welcome to this lesson on application lifecycle management. In this article, we cover essential aspects such as rolling updates and rollbacks, various application configuration methods, scaling strategies, and the fundamentals of self-healing application design.



Many of these topics are covered in depth throughout the course. If you are already familiar with some of this material, feel free to skip ahead or review the practice tests to reinforce your knowledge.

We will begin by exploring the concepts of rolling updates and rollbacks as part of a comprehensive overview of application lifecycle management. This approach ensures that you grasp both the theoretical aspects and practical implementations.

Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

Rolling Updates and Rollbacks

This guide covers managing updates and rollbacks in Kubernetes deployments, including rollouts, versioning, deployment strategies, and practical commands for minimal downtime.

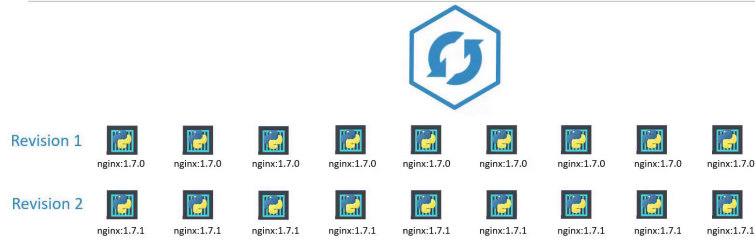
Welcome to this guide on managing updates and rollbacks in Kubernetes deployments. In this article, we explore key concepts such as rollouts, versioning, and various deployment strategies. We also provide practical commands to update your deployments with minimal downtime and to revert changes when necessary.

This article covers the process of monitoring deployment rollouts, updating container images, and performing rollbacks using Kubernetes commands.

Understanding Rollouts and Versioning

When you create a deployment, Kubernetes initiates a rollout that establishes the first deployment revision (revision one). Later, when you update your application—say by changing the container image version—Kubernetes triggers another rollout, creating a new revision (revision two). These revisions help you track changes and enable rollbacks to previous versions if issues arise.

Rollout and Versioning



To monitor and review these rollouts, you can use the following commands:

Check the rollout status:

```
``bash theme={null} kubectl rollout status deployment/myapp-deployment
```

View the history of rollouts:

```
``bash theme={null}
kubectl rollout history deployment/myapp-deployment
```

Deployment Strategies

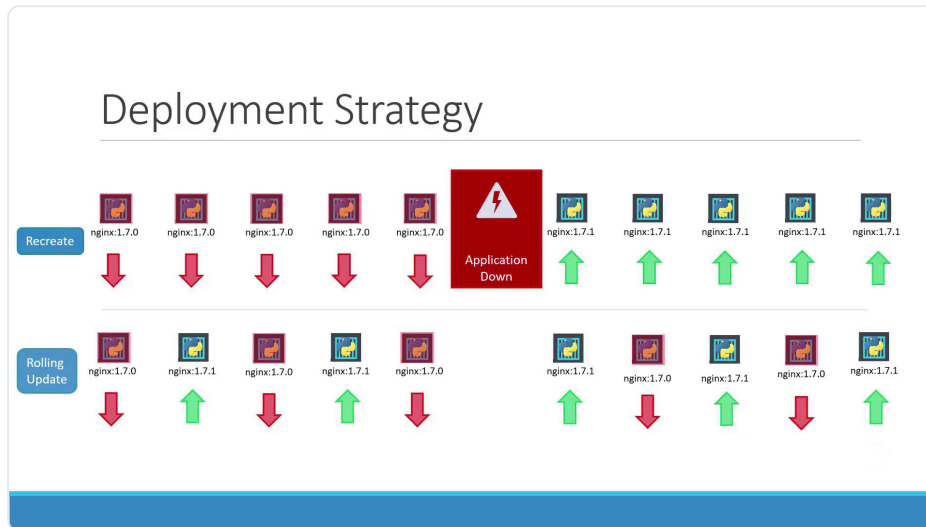
There are different strategies to update your applications. For example, consider a scenario where your web application is running five replicas.

One approach is the "recreate" strategy, which involves shutting down all existing instances before deploying new ones. However, this method results in temporary downtime as the application becomes inaccessible during the update.

Deployment Strategy



A more seamless approach is the "rolling update" strategy. Here, instances are updated one at a time, ensuring continuous application availability throughout the process.



If no strategy is specified when creating a deployment, Kubernetes uses the rolling update strategy by default.

Updating a Deployment

There are several methods to update your deployment, such as adjusting the container image version, modifying labels, or changing the replica count. A common practice is to update your deployment definition file and then apply the changes.

For example, consider the following deployment definition:

```
```yaml
theme={null}
apiVersion: apps/v1
kind: Deployment
metadata:
 name: myapp-deployment
 labels:
 app: myapp
 type: front-end
spec:
 replicas: 3
 selector:
 matchLabels:
 type: front-end
 template:
 metadata:
 name: myapp-pod
 labels:
 app: myapp
 type: front-end
 spec:
 containers:
 - name: nginx-container
 image: nginx:1.7.1
```
```

After updating the file, apply the changes:

```
```bash
theme={null}
kubectl apply -f deployment-definition.yml
```
```

This action triggers a new rollout and creates a new deployment revision.

Alternatively, you can update the container image directly using the following command:

```
```bash
theme={null}
kubectl set image deployment/myapp-deployment nginx-container=nginx:1.9.1
```
```

```
<Callout icon="lightbulb" color="#1CB2FE">
  Remember, using `kubectl set image` updates the running deployment but does not modify yo
</Callout>

## Viewing Deployment Details

To retrieve detailed information about your deployment—including rollout strategy, scaling

```bash theme={null}
kubectl describe deployment myapp-deployment
```

This output shows different details depending on the strategy used:

- **Recreate Strategy:** Events indicate that the old ReplicaSet is scaled down to zero before scaling up the new ReplicaSet.
- **Rolling Update Strategy:** The old ReplicaSet is gradually scaled down while the new ReplicaSet scales up.

For example, a deployment with the recreate strategy might display the following events:

```
```bash theme={null} Name: myapp-deployment Namespace: default
CreationTimestamp: Sat, 03 Mar 2018 17:01:55 +0000 Labels: app=myapp
Annotations: deployment.kubernetes.io/revision=2 kubectl.kubernetes.io/
change-cause=kubectl apply --filename=deployment-definition.yml Selector: 5
desired, 1 updated, 5 total, 5 available, 0 unavailable StrategyType: Recreate
MinReadySeconds: 0 Pod Template: Labels: app=myapp type=front-end
Containers: nginx-container: Image: nginx:1.7.1 Port: Conditions: Type Status
Reason ---- ----- Available True MinimumReplicasAvailable Progressing
True NewReplicaSetAvailable OldReplicaSets: NewReplicaSet: myapp-
deployment-54c7d6ccc (5/5 replicas created) Events:
Type Reason Age From Message ---- ----- ---- ---- ----- Normal
ScalingReplicaSet 11m deployment-controller Scaled up replica set myapp-
deployment-6795844b58 to 5 Normal ScalingReplicaSet 11m deployment-
controller Scaled down replica set myapp-deployment-6795844b58 to 0
Normal ScalingReplicaSet 56s deployment-controller Scaled up replica set
myapp-deployment-54c7d6ccc to 5
```

```
In contrast, a rolling update strategy output would reflect gradual scaling changes:

```bash theme={null}
kubectl describe deployment myapp-deployment
```

```
```bash theme={null} Name: myapp-deployment Namespace: default
CreationTimestamp: Sat, 03 Mar 2018 17:16:53 +0800 Labels: app=myapp
Annotations: deployment.kubernetes.io/revision=2 kubectl.kubernetes.io/
change-cause=kubectl apply --filename=deployment-definition.yml Selector: 6
desired, 5 updated, 6 total, 4 available, 2 unavailable StrategyType:
```


RollingUpdate MinReadySeconds: 0 RollingUpdateStrategy: 25% max unavailable, 25% max surge Pod Template: Labels: app=myapp type=front-end Containers: nginx-container: Image: nginx Port: Conditions: Type Status Reason ---- -

Available True MinimumReplicasAvailable Progressing True ReplicaSetUpdated OldReplicaSets: myapp-deployment-67c749c58c (1/1 replicas created) NewReplicaSet: myapp-deployment-75d7bdbd8d (5/5 replicas created) Events:

Type Reason Age From Message ---- -

Normal ScalingReplicaSet 1m deployment-controller Scaled up replica set myapp-deployment-67c749c58c to 5

Normal ScalingReplicaSet 1m deployment-controller Scaled down replica set myapp-deployment-75d7bdbd8d to 2

Normal ScalingReplicaSet 1m deployment-controller Scaled up replica set myapp-deployment-67c749c58c to 4

Normal ScalingReplicaSet 1m deployment-controller Scaled down replica set myapp-deployment-75d7bdbd8d to 3

Normal ScalingReplicaSet 0s deployment-controller Scaled down replica set myapp-deployment-75d7bdbd8d to 1

Normal ScalingReplicaSet 0s deployment-controller Scaled down replica set myapp-deployment-67c749c58c to 0

Upgrading and Rolling Back

During an upgrade, Kubernetes creates a new ReplicaSet for the updated containers while the

<Frame>

! [The image illustrates a Kubernetes deployment with two replica sets, each containing mu

</Frame>

If an issue is detected after an upgrade, you can revert to the previous version using the

```
```bash theme={null}
kubectl rollout undo deployment/myapp-deployment
```

This command scales down the new ReplicaSet, restoring pods from the older ReplicaSet. Verify the state of ReplicaSets before and after a rollback with:

```
```bash theme={null} kubectl get replicaset
```

For example, before the rollback you might see:

```
```bash theme={null}
NAME DESIRED CURRENT READY AGE
myapp-deployment-67c749c58c 0 0 0 22m
myapp-deployment-7d57dbd8d 5 5 5 20m
```

After executing the rollback command, the old ReplicaSet is restored while the new one is scaled down.

## Summary of Commands

Below is a quick reference table of the key commands discussed in this article:

Command Description	Command
Create the deployment	<code>kubectl create -f deployment-definition.yml</code>
List existing deployments	<code>kubectl get deployments</code>
Update deployment from the YAML definition	<code>kubectl apply -f deployment-definition.yml</code>
Update the container image with <code>kubectl set image</code>	<code>kubectl set image deployment/myapp-deployment nginx-container=nginx:1.9.1</code>
Check the status of the rollout	<code>kubectl rollout status deployment/myapp-deployment</code>
Rollback to a previous deployment revision	<code>kubectl rollout undo deployment/myapp-deployment</code>

With these commands and strategies, you can manage your Kubernetes deployments confidently, ensuring minimal downtime and a reliable process for both updates and rollbacks.

For additional information, see the [Kubernetes Documentation](#).

## Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

# Commands and Arguments in Docker

This article explains Docker commands, arguments, and ENTRYPOINT, focusing on container processes and their relation to Kubernetes pod definitions.

Hello and welcome to this comprehensive lesson on Docker commands, arguments, and entry points. My name is Mumshad Mannambeth, and in this session, we will dive into how containers run processes and how these concepts later translate into pod definitions in Kubernetes. Although these topics are often overlooked in certification curricula, understanding them is essential for mastering containerization.

## Understanding Container Commands

When you run a Docker container using the Ubuntu image, as shown below:

```
```bash theme={null} docker run ubuntu
```

Docker launches a container based on the Ubuntu image, which starts and immediately exits

```
```bash theme={null}
docker ps
```

you will notice that the container is absent because it has already exited. However, viewing all containers—including those that have stopped—with:

```
```bash theme={null} docker ps -a
```

reveals that the container is in an "exited" state:

```
```plaintext theme={null}
CONTAINER ID IMAGE COMMAND CREATED STATUS
45aacca36850 ubuntu "/bin/bash" 43 seconds ago Exited (0)
```

This behavior is different from traditional virtual machines. Containers are optimized to run a single task or process, such as hosting a web server, application server, database, or performing a specific computation. Once the task completes, the container stops because its lifecycle is tied directly to the process running inside it.

## Default Commands in Docker Images

---

Each Docker image contains instructions that define the process to run when a container starts. Many popular images, such as [nginx](#) or [MySQL](#), include a CMD instruction in their Dockerfile that sets the default command. For instance, the nginx image typically has the command `nginx`, and the MySQL image uses `mysqld`.

Consider this Dockerfile snippet for installing and configuring nginx:

```
```dockerfile theme={null}
```

Install Nginx.

```
RUN \ add-apt-repository -y ppa:nginx/stable && \ apt-get update && \ apt-get  
install -y nginx && \ rm -rf /var/lib/apt/lists/* && \ echo "\ndaemon off;" >> /etc/  
nginx/nginx.conf && \ chown -R www-data:www-data /var/lib/nginx
```

Define mountable directories.

```
VOLUME ["/etc/nginx/sites-enabled", "/etc/nginx/certs", "/etc/nginx/conf"]
```

Define working directory.

```
WORKDIR /etc/nginx
```

Define default command.

```
CMD ["nginx"]
```

Now, let's examine the Ubuntu image's Dockerfile. Notice that in this example, the default

```
``dockerfile theme={null}
# Pull base image.
FROM ubuntu:14.04

# Install necessary packages.
RUN \
    sed -i 's/# \(*multiverse$\)/\1/g' /etc/apt/sources.list && \
    apt-get update && \
    apt-get -y upgrade && \
    apt-get install -y build-essential software-properties-common && \
    rm -rf /var/lib/apt/lists/*

# Add configuration files.
ADD root/.bashrc /root/.bashrc
ADD root/.gitconfig /root/.gitconfig
ADD root/.scripts /root/.scripts

# Set environment variables.
ENV HOME /root

# Define working directory.
WORKDIR /root

# Define default command.
CMD ["bash"]
```

Remember: Bash is a shell, not a persistent server process. When the Ubuntu container is launched without an attached terminal, the shell exits immediately.

Overriding the Default Command

To override the default command for a container, you can append a command to the end of the `docker run` command. For example, this command instructs the container to run `sleep 5`:

```
``bash theme={null} docker run ubuntu sleep 5
```

In this scenario, the container executes `sleep 5`, waits for five seconds, and then exits.

If you want to permanently change the behavior of the image so that it always runs `sleep 5`

1. Shell form:


```
```dockerfile
theme={null}
FROM ubuntu
CMD sleep 5
```
```
2. JSON array format:


```
```dockerfile
theme={null}
FROM ubuntu
CMD ["sleep", "5"]
```
```

Build the new image with:

```
```bash
theme={null}
docker build -t ubuntu-sleeper .
```
```

Then run the container using:

```
```bash
theme={null} docker run ubuntu-sleeper
```
```

By default, this container will sleep for five seconds.

Configuring ENTRYPOINT for Runtime Arguments

Sometimes, you may want to specify only runtime arguments without changing the default command.

Consider the following Dockerfile:

```
```dockerfile
theme={null}
FROM ubuntu
ENTRYPOINT ["sleep"]
CMD ["5"]
```
```

After building and running this image:

```
```bash
theme={null} docker build -t ubuntu-sleeper .
docker run ubuntu-sleeper
```
```

the container executes `sleep 5` by default. You can override the sleep duration at runtime

```
```bash
theme={null}
docker run ubuntu-sleeper 10
```
```

This command runs `sleep 10`.

* With CMD alone, runtime arguments replace the default command. * With ENTRYPOINT, runtime arguments are appended to the specified executable, allowing you to override just the parameters.

Overriding ENTRYPOINT at Runtime

At times, you might want to completely override the ENTRYPOINT. For example, if you wish to use a different command (like switching from `sleep` to `sleep2.0`), you can do so using the `--entrypoint` flag in the `docker run` command.

Given the Dockerfile:

```
``dockerfile theme={null} FROM ubuntu ENTRYPOINT ["sleep"] CMD ["5"]
```

Build the image:

```
```bash theme={null}
docker build -t ubuntu-sleeper .
```

Running the container without modifications:

```
```bash theme={null} docker run ubuntu-sleeper
```

will produce an error because `sleep` expects an operand:

```
```bash theme={null}
docker run ubuntu-sleeper
Output:
sleep: missing operand
Try 'sleep --help' for more information.
```

However, running with a new parameter:

```
```bash theme={null} docker run ubuntu-sleeper 10
```

executes `sleep 10`.

To override the ENTRYPOINT with a different executable, run:

```
```bash theme={null}
docker run --entrypoint sleep2.0 ubuntu-sleeper 10
```

This command starts the container with `sleep2.0 10` (provided that `sleep2.0` is a valid command).



## Conclusion

---

This lesson explained the inner workings of Docker commands, arguments, and how the ENTRYPOINT instruction allows dynamic parameter passing at runtime. These concepts not only help in managing Docker containers efficiently but also lay the groundwork for understanding how they translate into pod definitions in Kubernetes in subsequent lessons.

Happy Dockering!

For further reading, check out the following resources:

- [Docker Documentation](#)
- [Kubernetes Basics](#)
- [Nginx Official Website](#)
- [MySQL Official Website](#)



## Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

# Commands and Arguments in Kubernetes

This lesson covers configuring commands and arguments in Kubernetes pods to adjust container behaviors by overriding default settings from the Dockerfile.

Welcome to this lesson on configuring commands and arguments in Kubernetes pods. In this session, we'll learn how to adjust container behaviors by overriding default settings defined in the Dockerfile via the pod definition.

Previously, we built a simple Docker image—named "ubuntu-sleeper"—that executes a sleep command for a specified number of seconds. By default, running a container with this image makes it sleep for five seconds. However, you can easily change this behavior by passing a command-line argument.

## Overriding Default Behavior with Arguments

Suppose we want to create a pod using the "ubuntu-sleeper" image. We begin with a basic pod definition where the pod's name and image are specified. When the pod is created, it starts a container that runs the default sleep command (sleeping for five seconds) and then exits. To modify the sleep duration to 10 seconds, simply append an additional argument in the pod specification. Any argument provided in the Docker run command correlates with the `args` property in the pod definition file (formatted as an array).

When you append an argument to the Docker run command, it overrides the default parameters defined by the CMD instruction in the Dockerfile.

Consider the following examples:

Docker commands:

```
``bash theme={null} docker run --name ubuntu-sleeper ubuntu-sleeper docker run --name ubuntu-sleeper ubuntu-sleeper 10
```

```
Pod definition YAML:

```yaml
theme={null}
apiVersion: v1
kind: Pod
metadata:
  name: ubuntu-sleeper-pod
spec:
  containers:
  - name: ubuntu-sleeper
    image: ubuntu-sleeper
    args: ["10"]
```

Dockerfile Instructions and Their Mappings

The Dockerfile for the "ubuntu-sleeper" image is defined with both an ENTRYPOINT and a CMD:

```
```dockerfile
theme={null} FROM Ubuntu ENTRYPOINT ["sleep"] CMD ["5"]
```

- \* The **ENTRYPOINT** instruction specifies the command to run when the container starts.
- \* The **CMD** instruction provides default parameters for that command.

By specifying the `args` field in the pod definition file, the CMD instruction is overridden.

### ## Overriding the ENTRYPOINT

Now, consider a scenario where you want to override the ENTRYPOINT itself (for example, switch to a different shell).

Docker command example with overridden ENTRYPOINT:

```
```bash
theme={null}
docker run --name ubuntu-sleeper \
  --entrypoint sleep2.0 \
  ubuntu-sleeper 10
```

Pod definition YAML with overridden ENTRYPOINT:

```
```yaml
theme={null}
apiVersion: v1
kind: Pod
metadata:
 name: ubuntu-sleeper-pod
spec:
 containers:
 - name: ubuntu-sleeper
 image: ubuntu-sleeper
 command: ["sleep2.0"]
 args: ["10"]
```

### ## Deployment of the Pod

Once your pod definition is ready, deploy the pod using the following command:

```
```bash
theme={null}
kubectl create -f pod-definition.yml
```

Key Fields Summary

Below is a table summarizing the Kubernetes pod definition fields that map to Dockerfile instructions:

Pod Definition Field	Dockerfile Instruction	Functionality Description
command	ENTRYPOINT	Specifies the command to run when the container starts (completely replacing Dockerfile ENTRYPOINT).
args	CMD	Provides default parameters passed to the command (overriding the Dockerfile CMD).

Remember that specifying the `command` in a pod definition replaces the Dockerfile's ENTRYPOINT entirely, while the `args` field only overrides the default parameters defined by CMD.

This concludes the lesson on managing commands and arguments within your Kubernetes pods. Next, proceed to the coding exercises section to practice these concepts and refine your troubleshooting skills with real-world scenarios.

Happy coding!

Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

Secrets

This article explains how to securely manage sensitive data in Kubernetes using Secrets while avoiding common security pitfalls.

Welcome to this comprehensive guide on managing Secrets in Kubernetes. In this article, we explain how to securely handle sensitive data (such as passwords and keys) in your Kubernetes deployments while avoiding common pitfalls like hardcoding credentials in your application.

Problem with Hardcoding Sensitive Data

Consider a simple Python web application connecting to a MySQL database. When the connection succeeds, the application displays a success message. However, the code includes hardcoded values for hostname, username, and password, which poses a serious security risk.

Previously, configuration data like these values might have been stored in a ConfigMap. For example:

```
```python theme={null} import os from flask import Flask, render_template
import mysql.connector
```

```
app = Flask(name)
```

```
@app.route("/") def main(): mysql.connector.connect(host="mysql",
database="mysql", user="root", password="paswrn") return
render_template('hello.html', color=fetchcolor())
```

```
if name == "main": app.run(host="0.0.0.0", port="8080")
```

```
```yaml theme={null}
apiVersion: v1
kind: ConfigMap
metadata:
  name: app-config
data:
  # Configuration data goes here
```

While storing non-sensitive details like hostnames or usernames in a ConfigMap is acceptable, placing a password in such a resource is not secure. Kubernetes Secrets provide a mechanism to safely store sensitive information by encoding the data (note: this is not encryption by default).

Secrets encode data using Base64. Although it provides obfuscation, it is not a substitute for encryption.

Understanding Kubernetes Secrets

Working with Secrets in Kubernetes involves two main steps:

1. **Create the Secret.**
2. **Inject it into a Pod.**

Below is an illustration of Secret data in their encoded and decoded forms:

Encoded Values

```
```plaintext theme={null} DB_Host: bXlzcWw= DB_User: cm9vdA==
DB_Password: cGFzd3Jk
```

```
Decoded Values

```plaintext theme={null}
DB Host: mysql
DB User: root
DB Password: paswr
```

There are two primary approaches to creating a Secret:

- **Imperative Creation:** Using the command line to create Secrets on the fly.
- **Declarative Creation:** Defining Secrets in YAML files.

Imperative Creation of a Secret

With the imperative method, you can supply key-value pairs directly via the command line. For example, to create a Secret named "app-secret" with the key-value pair `DB_Host=mysql`:

```
```bash theme={null} kubectl create secret generic app-secret --from-
literal=DB_Host=mysql
```

To include multiple key-value pairs, use the `--from-literal` option repeatedly:

```
```bash theme={null}
kubectl create secret generic app-secret \
  --from-literal=DB_Host=mysql \
  --from-literal=DB_User=root \
  --from-literal=DB_Password=paswd
```

Alternatively, create a Secret from a file with the `--from-file` option:

```
```bash theme={null} kubectl create secret generic app-secret --from-
file=app_secret.properties
```

## Declarative Creation of a Secret

For a more manageable approach, define a Secret in a YAML file. This file should include the

```
```yaml theme={null}
apiVersion: v1
kind: Secret
metadata:
  name: app-secret
data:
  DB_Host: bXlzcWw=
  DB_User: cm9vdA==
  DB_Password: cGFzd3Jk
```

Apply the definition with the following command:

```
```bash theme={null} kubectl create -f secret-data.yaml
```

## Converting Plaintext to Base64

On Linux hosts, you can convert plaintext values to Base64-encoded strings using the `echo`

```
```bash theme={null}
echo -n 'mysql' | base64
echo -n 'root' | base64
echo -n 'paswr' | base64
# Output: cGFzd3Jk
```

Viewing and Decoding Secrets

After creating a Secret, you can list and inspect it with the following commands:

- **List Secrets:**

```
bash theme={null} kubectl get secrets
```

Expected output:

```
plaintext theme={null} NAME TYPE DATA AGE app-secret Opaque 3 10m
```

- **Describe a Secret (without showing sensitive data):**

```
bash theme={null} kubectl describe secret app-secret
```

- **View the encoded data in YAML format:**

```
bash theme={null} kubectl get secret app-secret -o yaml
```

If you need to decode an encoded value, use the `base64 --decode` command:

```
``bash theme={null} echo -n 'bXlzcWw=' | base64 --decode echo -n
'cm9vdA==' | base64 --decode echo -n 'cGFzd3Jk' | base64 --decode
```

Output: paswrd

```
## Injecting Secrets into a Pod
```

Once the Secret is created, you can inject it into a Pod using environment variables or by

```
### Injecting as Environment Variables
```

Below is an example Pod definition that injects the Secret as environment variables:

```
``yaml theme={null}
# pod-definition.yaml
apiVersion: v1
kind: Pod
metadata:
  name: simple-webapp-color
  labels:
    name: simple-webapp-color
spec:
  containers:
  - name: simple-webapp-color
    image: simple-webapp-color
    ports:
      - containerPort: 8080
    envFrom:
      - secretRef:
          name: app-secret
```

Mounting Secrets as Files

Alternatively, mount the Secret as files within a volume. Each key in the Secret becomes a separate file:

```
``yaml theme={null} volumes: - name: app-secret-volume secret: secretName:
app-secret
```

After mounting, listing the directory contents should display each key as a file:

```
```bash theme={null}
ls /opt/app-secret-volumes
Output: DB_Host DB_Password DB_User
```

To view the content of a specific file, such as the DB password:

```
```bash theme={null} cat /opt/app-secret-volumes/DB_Password
```

Output: paswrđ

```

## Important Considerations When Using Secrets

---

Remember that Kubernetes Secrets are only encoded in Base64, not encrypted by default. Anyone with sufficient access can decode the data. Always handle secret definition files with care and avoid storing them in public repositories.

Here are some key considerations:

- Secrets offer only Base64 encoding. For enhanced security, consider enabling encryption at rest for etcd.
- Limit access to Secrets using Role-Based Access Control (RBAC). Restrict permissions to only those who require it.
- Avoid storing sensitive secret definition files in source control systems that are publicly accessible.
- For even greater security, explore third-party secret management solutions such as AWS Secrets Manager, Azure Key Vault, GCP Secret Manager, or Vault.

## External Secret Providers

---

External secret providers decouple secret management from etcd and offer advanced encryption, granular access control, and comprehensive auditing capabilities. For further details and best practices, consider exploring courses like the [Certified Kubernetes Security Specialist \(CKS\)](#).



## Note on Secrets

- ✗ Secrets are not Encrypted. Only encoded.
  - ✗ Do not check-in Secret objects to SCM along with code.
- ✗ Secrets are not encrypted in ETCD
  - ✓ Enable encryption at rest
- ✗ Anyone able to create pods/deployments in the same namespace can access the secrets
  - ✓ Configure least-privilege access to Secrets - RBAC
- ✓ Consider third-party secrets store providers  
AWS Provider, Azure Provider, GCP Provider, Vault Provider

## Conclusion

Managing Kubernetes Secrets effectively is crucial for maintaining the security of your applications. By following the best practices outlined above, including using Secrets to handle sensitive data and applying strict RBAC policies, you can mitigate potential security risks associated with managing sensitive configuration data.

Practice these approaches using hands-on labs and ensure your Kubernetes clusters are secure.

For additional resources, consider the following links:

- [Kubernetes Documentation](#)
- [Kubernetes Basics](#)
- [Docker Hub](#)
- [Terraform Registry](#)

## Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

# Demo Encrypting Secret Data at Rest

This guide explains how to secure secret data in Kubernetes by enabling encryption at rest.

In this guide, we explain how to secure secret data in your Kubernetes cluster by enabling encryption at rest. We start by creating secret objects, examine how Kubernetes encodes them in etcd, and then show you how to configure the API server to encrypt these secrets.

## Creating a Secret Object

Begin by launching your Kubernetes playground—a single-node cluster running Kubernetes with ContainerD. Kubernetes makes it easy to create secrets from files, literal values, or environment files. Below are some example commands:

```
```bash theme={null}
```

Create a secret from all files within a directory:

```
kubectl create secret generic my-secret --from-file=path/to/bar
```

Create a secret using specified keys from files:

```
kubectl create secret generic my-secret --from-file=ssh-privatekey=path/to/id_rsa --from-file=ssh-publickey=path/to/id_rsa.pub
```

Create a secret from literal key-value pairs:

```
kubectl create secret generic my-secret --from-literal=key1=supersecret --from-literal=key2=topsecret
```

Create a secret combining a file and a literal:

```
kubectl create secret generic my-secret --from-file=ssh-privatekey=path/to/id_rsa --from-literal=password=topsecret
```

Create a secret from environment files:

```
kubectl create secret generic my-secret --from-env-file=path/to/foo.env --from-env-file=path/to/bar.env
```

Additional options like `--allow-missing-template-keys`, `--append-hash`, and `--dry-run` c

After the command executes, verify the secret:

```
``bash theme={null}
kubectl create secret generic my-secret --from-literal=key1=supersecret
kubectl get secret my-secret
```

Using the `describe` command provides detailed metadata, including the base64-encoded data:

```
``bash theme={null} kubectl describe secret my-secret
```

<Callout icon="lightbulb" color="#1CB2FE">
Secret values are base64-encoded by default; they are not encrypted. Avoid pushing secret
</Callout>

Viewing the Encoded Secret

Kubernetes stores secret values in base64-encoded format. Retrieve the secret as YAML to in

```
``bash theme={null}
kubectl get secret my-secret -o yaml
```

The output might look like:

```
```yaml theme={null} apiVersion: v1 data: key1: c3VwZXJzWmNyZVQ= kind:
Secret metadata: creationTimestamp: "2022-10-24T05:34:13Z" name: my-
secret namespace: default resourceVersion: "2111" uid: dfe97c62-5aa1-46a8-
b71c-ffa0cd4c08ec type: Opaque
```

To decode the secret value:

```
```bash theme={null}
echo "c3VwZXJzWmNyZVQ=" | base64 --decode
```

This reveals that the stored secret is only encoded, not encrypted, making it potentially accessible to anyone with access to the YAML output or an etcd dump.

Inspecting Secret Data in etcd

etcd is the key-value store where Kubernetes persists cluster data. Without encryption at rest, secret values remain only base64-encoded, allowing anyone with access to etcd to decode them. Use the `etcdctl` client (API version 3) to query etcd:

```
```bash theme={null} ETCDCTL_API=3 etcdctl \ --cacert=/etc/kubernetes/pki/
etcd/ca.crt \ --cert=/etc/kubernetes/pki/etcd/server.crt \ --key=/etc/kubernetes/
pki/etcd/server.key \ get /registry/secrets/default/my-secret | hexdump -C
```

Before running the above command, ensure that the `etcdctl` client is installed. On Ubuntu,

```
```bash theme={null}
apt-get install etcd-client
```

Verify the installation:

```
```bash theme={null} etcdctl
```

Also, check that your control plane node can access the necessary certificate files:

```
```bash theme={null}
ls /etc/kubernetes/pki/etcd/ca.crt
```

The hexdump output will display the raw data, illustrating that without encryption, the secret's value is visible within etcd.

Enabling Encryption at Rest

To protect sensitive data stored in etcd, Kubernetes offers an encryption at rest mechanism using an encryption provider configuration. First, verify if encryption is enabled by checking the kube-apiserver process:

```
```bash theme={null} ps -aux | grep kube-api | grep "encryption-provider-config"
```

If no configuration is found, follow these steps:

### 1. Create an Encryption Configuration File

Generate a random 32-byte key (base64-encoded) with:

```
```bash theme={null}
head -c 32 /dev/urandom | base64
```

Next, create a YAML file (e.g., `enc.yaml`) with the following content (replace the sample key with your generated key):

```
```yaml  theme={null}  apiVersion:  apiserver.config.k8s.io/v1  kind:
EncryptionConfiguration resources: - resources: - secrets providers: - aescbc:
keys: - name: key1 secret:
y0xTt+U6xgRdNxe4nDYYSijOGgRDoUYC+wAwOKeNfPs= # Replace with
your generated key - identity: {}
```

Review the file to ensure accuracy:

```
```bash theme={null}
cat enc.yaml
```

2. Mount the Encryption Configuration File into the API Server

Next, incorporate the configuration into the kube-apiserver by performing these steps:

1. Move the encryption configuration file to a secure directory:

```
bash theme={null} mkdir -p /etc/kubernetes/enc mv enc.yaml /etc/
kubernetes/enc/
```

1. Modify the kube-apiserver manifest (found at `/etc/kubernetes/manifests/kube-apiserver.yaml`) by adding the `--encryption-provider-config` flag. Include a new volume and volume mount for the `/etc/kubernetes/enc` directory. For example:

```
yaml theme={null} spec: containers: - command: - kube-apiserver
# ... other flags ... - --encryption-provider-config=/etc/
kubernetes/enc/enc.yaml volumeMounts: # ... other volume mounts ...
- name: enc mountPath: /etc/kubernetes/enc readOnly: true volumes:
```

```
# ... other volumes ... - name: enc hostPath: path: /etc/kubernetes/
enc type: DirectoryOrCreate
```

After saving the changes, the kube-apiserver will restart and begin using the new encryption configuration.

Verifying Encryption

Once encryption is activated, any new secret you create will be encrypted at rest. Create a new secret:

```
```bash theme={null} kubectl create secret generic my-secret-2 --from-
literal=key2=topsecret
```

Verify the secret's creation:

```
```bash theme={null}
kubectl get secret
```

Then inspect etcd to ensure the secret's value is encrypted:

```
```bash theme={null} ETCDCTL_API=3 etcdctl \ --cacert=/etc/kubernetes/pki/
etcd/ca.crt \ --cert=/etc/kubernetes/pki/etcd/server.crt \ --key=/etc/kubernetes/
pki/etcd/server.key \ get /registry/secrets/default/my-secret-2 | hexdump -C
```

You should observe that the secret's data no longer appears in plain text. Remember, secret

```
```bash theme={null}
kubectl get secret --all-namespaces -o json | kubectl replace -f -
```

Then confirm that the secrets in etcd are now encrypted.

Always update your existing secrets after enabling encryption at rest to ensure full protection of sensitive data.

Conclusion

This guide demonstrated how Kubernetes handles secret data by showing that, by default, secrets are only base64-encoded—not encrypted—in etcd. We then detailed the process for enabling encryption at rest: creating an encryption configuration file, mounting it into the API server, and verifying that new secrets store their data securely. By following these steps, you can significantly enhance the security posture of your Kubernetes cluster.

Happy encrypting!

Additional Resources

Resource	Description	Example Command / Link
Kubernetes Secrets	Official documentation on secrets	Kubernetes Secrets
etcd Documentation	Information on etcd and its usage	etcd Documentation
Kubernetes API Server	Details on configuring kube-apiserver flags	Kube-apiserver Docs

For further reading on securing your Kubernetes environment, consider consulting the [Kubernetes Documentation](#).

Documentation Index

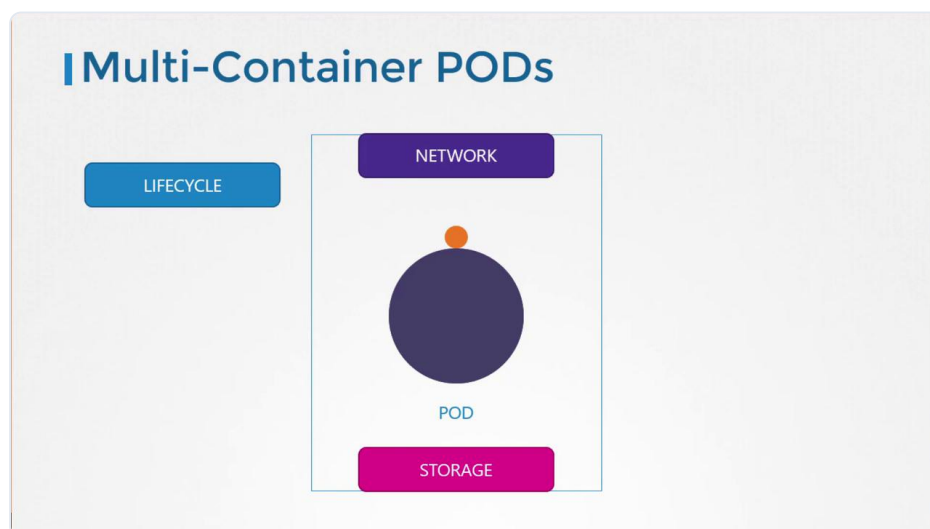
Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

Multi Container Pods

This article explores multi-container pods in Kubernetes, highlighting their benefits for deploying and managing closely linked services together.

In this article, we explore the concept and benefits of multi-container pods in Kubernetes. By breaking down a monolithic application into smaller, independent microservices, you can deploy, manage, and scale each service individually. However, certain scenarios require two closely linked services to run together. For example, a web server might need to be paired with a dedicated logging agent. In this configuration, each web server instance is automatically associated with its own logging service, allowing both services to scale concurrently while keeping their codebases distinct.

Multi-container pods are designed to group containers that share the same lifecycle. This means they are created and terminated together, share a common network namespace (allowing seamless communication via localhost), and have access to shared storage volumes. This design simplifies configurations by eliminating the complexities of volume sharing and networking between separate pods.



To create a multi-container pod, add the configuration for the new container under the `containers` array in your pod definition file. For instance, you can incorporate a container named "log-agent" alongside an existing web application container. The following YAML snippet demonstrates how to configure a pod that contains both a web application and its corresponding logging agent:

```
yaml theme={null} apiVersion: v1 kind: Pod metadata: name: simple-  
webapp labels: name: simple-webapp spec: containers: - name: simple-  
webapp image: simple-webapp ports: - containerPort: 8080 - name:  
log-agent image: log-agent
```

This configuration ensures that both containers share the same lifecycle, network, and storage resources, allowing them to work together seamlessly.

Remember that the `containers` field is an array in the pod specification, which enables you to define and manage multiple containers under a single pod.

That concludes our discussion on multi-container pods. Next, proceed to the coding exercises section to practice configuring multi-container pods and reinforce your understanding of the concepts discussed here. Happy coding, and see you in the next article!

Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

Introduction to Autoscaling 2025 Updates

This article explores autoscaling in Kubernetes, focusing on Horizontal and Vertical Pod Autoscaling for the Certified Kubernetes Administrator exam.

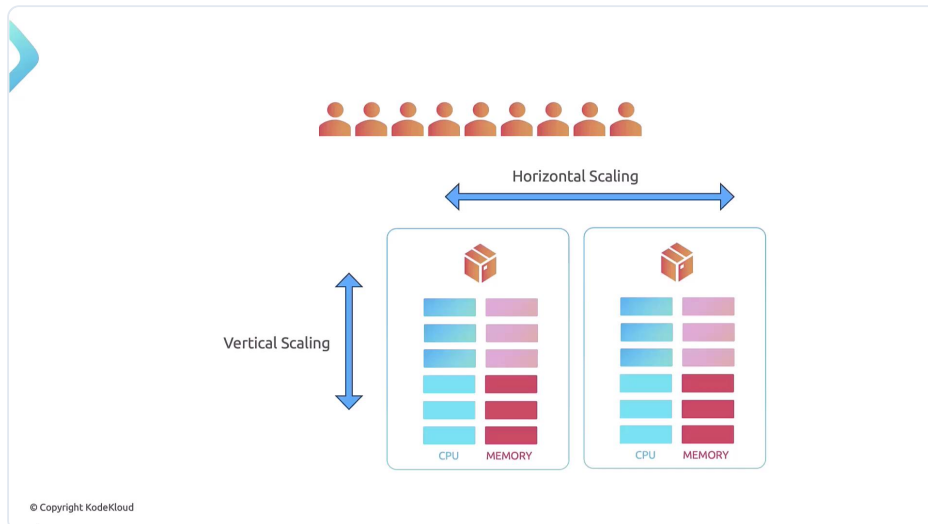
In this lesson series, we explore autoscaling, a key topic for the Certified Kubernetes Administrator (CKA) exam. We will focus on Horizontal Pod Autoscaling (HPA) and Vertical Pod Autoscaling (VPA) to give you the essential knowledge in a concise format. For a comprehensive dive into autoscaling, consider the [Kubernetes Autoscaling course on KodeKloud](#).

Before diving into autoscaling in Kubernetes, let's review the traditional concepts of scaling using physical servers.

Imagine a past scenario where applications were deployed on physical servers with fixed CPU and memory capacities. When the application load exceeded server capacity, you had two options:

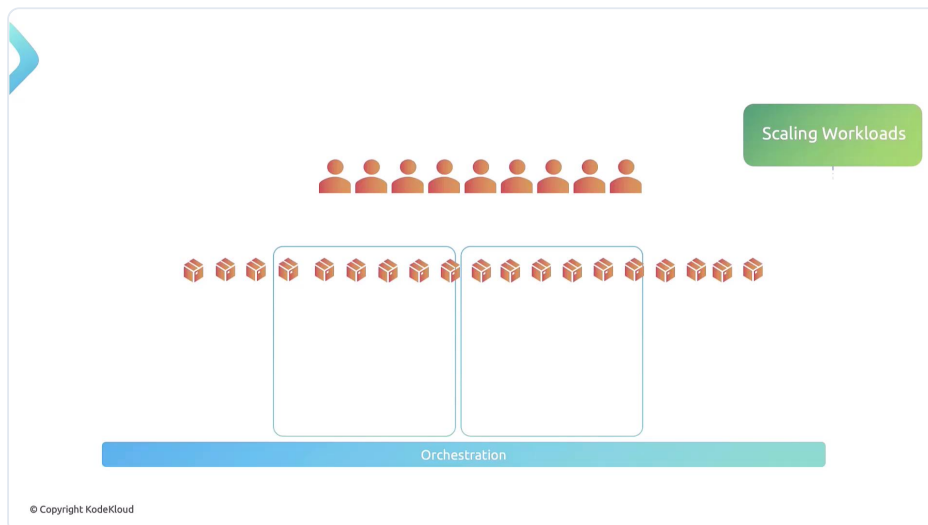
1. Shut down the application and upgrade the existing server by adding more CPU and memory (vertical scaling).
2. If the application could run multiple instances, add another server to distribute the load without downtime (horizontal scaling).

Vertical scaling means enhancing a single server's resources, whereas horizontal scaling means incorporating additional servers to manage increased load.



Now, let's see how these concepts apply to Kubernetes and containerized environments. Kubernetes is designed to dynamically scale containerized applications. Two primary scaling strategies in Kubernetes are:

1. Scaling workloads – adding or removing containers (Pods) in the cluster.
2. Scaling the underlying cluster infrastructure – adding or removing nodes (servers) in the cluster.



To clarify:

- For the cluster infrastructure:
 - **Horizontal scaling:** Add more nodes to the cluster.
 - **Vertical scaling:** Increase resources (CPU, memory) on existing nodes.
- For workloads:
 - **Horizontal scaling:** Create more Pods.
 - **Vertical scaling:** Increase resource limits and requests for existing Pods.

There are two approaches to scaling in Kubernetes: manual and automated.

Manual scaling and automated scaling both have their place. Manual scaling involves direct intervention and command execution, while automated scaling leverages Kubernetes controllers for dynamic adjustments.

Manual Scaling

For manual scaling, use the following methods:

- **Cluster Infrastructure Horizontal Scaling:** Manually provision new nodes and add them to the cluster:

```
bash theme={null} kubeadm join ...
```

- **Workload Horizontal Scaling:** Adjust the number of Pods using:

```
bash theme={null} kubectl scale --replicas=<number> <workload-type>/<workload-name>
```

- **Workload Vertical Scaling:** Edit the deployment, stateful set, or ReplicaSet to change resource limits and requests:

```
bash theme={null} kubectl edit <workload-type>/<workload-name>
```

Vertical scaling of cluster nodes is less common in Kubernetes because it often requires downtime. In virtualized environments, it may be easier to provision a new VM with higher resources, add it to the cluster, and then decommission the older node.

Automated Scaling

Automated scaling in Kubernetes simplifies operations:

- **Cluster Infrastructure:** Managed by the Kubernetes Cluster Autoscaler.
- **Workload Horizontal Scaling:** Managed by the Horizontal Pod Autoscaler (HPA).
- **Workload Vertical Scaling:** Managed by the Vertical Pod Autoscaler (VPA).

The manual commands summarized earlier are:

```
``bash theme={null} kubeadm join ...
```

```
``bash theme={null}
kubectl scale --replicas=<number> <workload-type>/<workload-name>
```

```
bash theme={null} kubectl edit <workload-type>/<workload-name>
```

This lesson provided a high-level overview of scaling concepts—both in traditional environments and in containerized applications managed by

Kubernetes. In upcoming lessons, we will explore these autoscaling methods in greater detail.

See you in the next lesson!

Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

Horizontal Pod Autoscaler HPA 2025 Updates

This article explores the Horizontal Pod Autoscaler in Kubernetes and how it automates workload scaling, improving efficiency over manual scaling methods.

In this article, we explore the Horizontal Pod Autoscaler (HPA) feature in Kubernetes and explain how it automates the scaling of workloads. We'll begin by examining the manual approach to scaling an application and then show how HPA streamlines this process.

Manual Horizontal Scaling

As a Kubernetes administrator, you might manually scale your application to ensure it has enough resources during traffic spikes. Consider the following deployment configuration:

```
```yaml theme={null} apiVersion: apps/v1 kind: Deployment metadata: name: my-app spec: replicas: 1 selector: matchLabels: app: my-app template: metadata: labels: app: my-app spec: containers: - name: my-app image: nginx resources: requests: cpu: "250m" limits: cpu: "500m"
```

In this configuration, each pod requests 250 millicores (mCPU) and is limited to 500 mCPU.

```
```bash theme={null}
$ kubectl top pod my-app-pod
```

The output would be similar to:

```
```bash theme={null} NAME CPU(cores) MEMORY(bytes) my-app-pod 450m
350Mi
```

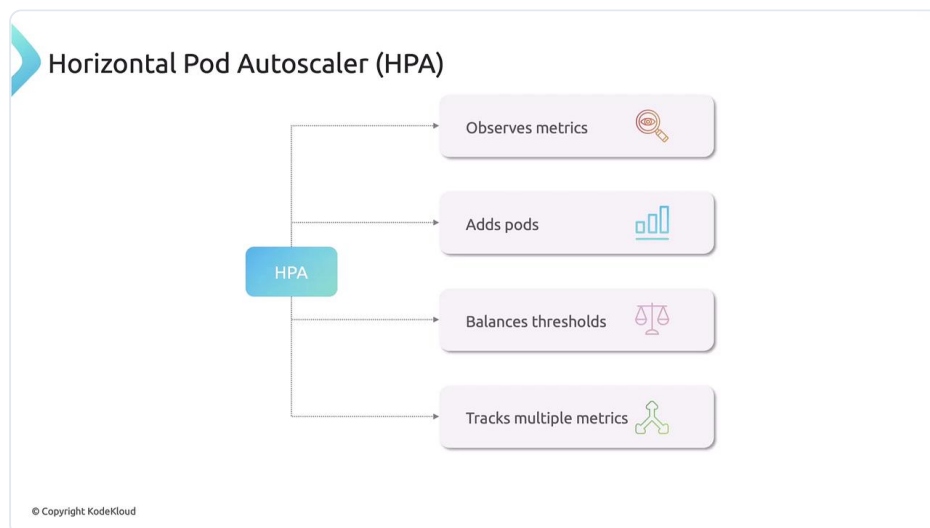
```
Once you observe the pod's CPU usage nearing the threshold (for example, at 450 mCPU), you
```

```
```bash theme={null}
$ kubectl scale deployment my-app --replicas=3
```

Manual scaling requires continuous monitoring and timely intervention, which may not be ideal during unexpected surges in traffic.

Introducing the Horizontal Pod Autoscaler (HPA)

To address the shortcomings of manual scaling, Kubernetes offers the Horizontal Pod Autoscaler (HPA). HPA continuously monitors pod metrics—such as CPU, memory, or custom metrics—using the metrics-server. Based on these metrics, HPA automatically adjusts the number of pod replicas in a deployment, stateful set, or replica set. When resource usage exceeds a preset threshold, HPA increases the pod count; when usage declines, it scales down to conserve resources.



For example, with the nginx deployment above, you can create an HPA by running the command below. This command configures the "my-app" deployment to maintain 50% CPU utilization, scaling the number of pods between 1 and 10:

```
```bash theme={null} $ kubectl autoscale deployment my-app --cpu-percent=50
--min=1 --max=10
```

```
Kubernetes will then create an HPA that monitors the CPU metrics (using the pod's 500 mCPU
```

To review the status of your HPA, use:

```
```bash theme={null}
$ kubectl get hpa
```

This command shows the current CPU usage, threshold set, and the number of replicas—ensuring that pod counts remain within the defined limits. When the HPA is no longer needed, you can remove it with:

```
```bash theme={null} $ kubectl delete hpa my-app
```

```
Declarative Configuration for HPA
```

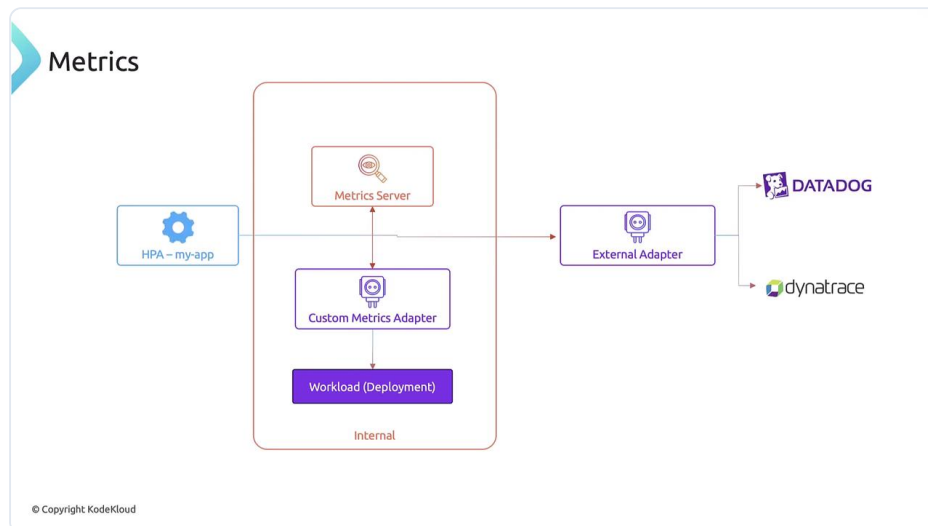
Beyond the imperative approach, you can declare the HPA configuration with a YAML file. Here's an example:

```
```yaml theme={null}
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: my-app-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: my-app
  minReplicas: 1
  maxReplicas: 10
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 50
```

This configuration ensures that the HPA monitors the CPU utilization of the "my-app" deployment, automatically adjusting the replica count as needed. Note that HPA, integrated into Kubernetes since version 1.23, relies on the metrics-server to obtain resource utilization data.

Metrics Sources and External Adapters

Kubernetes supports not only the internal metrics-server for collecting CPU or memory metrics but also custom metrics adapters. These adapters can retrieve metrics from other internal sources or external metrics providers like Datadog or Dynatrace through an external adapter. For further details on advanced configurations, please explore our [Kubernetes Autoscaling](#) course.



Conclusion

This article provided a comprehensive overview of the Horizontal Pod Autoscaler (HPA) in Kubernetes. We discussed the drawbacks of manual scaling and demonstrated how HPA automates scaling based on real-time resource usage. Whether through imperative commands or declarative YAML configurations, HPA ensures that your applications can adapt dynamically to fluctuating workloads.

For additional insights and hands-on experience, consider enrolling in our [Kubernetes Autoscaling](#) course.

Happy scaling!

Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

In place Resize of Pods 2025 Updates

This guide explores in-place resizing of Pod resources to streamline updates and reduce downtime for stateful workloads without recreating the entire Pod.

In this guide, we explore how to perform in-place resizing of Pod resources—a feature that streamlines updates to resource changes without recreating the entire Pod. This innovative approach reduces downtime, especially for stateful workloads, by updating resource requirements directly on the running Pods.

Understanding the Default Behavior

By default, Kubernetes (v1.32 and later) replaces the existing Pod when you modify resource requests or limits in a Deployment. Consider the following deployment configuration:

```
```yaml
theme={null}
apiVersion: apps/v1
kind: Deployment
metadata:
 name: my-app
spec:
 replicas: 1
 selector:
 matchLabels:
 app: my-app
 template:
 metadata:
 labels:
 app: my-app
 spec:
 containers:
 - name: my-app
 image: nginx
 resources:
 requests:
 cpu: "250m"
 memory: "256Mi"
 limits:
 cpu: "500m"
 memory: "512Mi"
```
```

When you update the resource requirements, Kubernetes terminates the existing Pod and creates a new one.

The In-Place Update Mechanism

To address the disruption from full Pod recreation, Kubernetes is developing an in-place update mechanism.

Example of In-Place Update Manifest

Below is an example where the CPU resource is increased from "250m" to "1". Thanks to the in-place update mechanism, the Pod is not recreated.

```
```yaml theme={null}
apiVersion: apps/v1
kind: Deployment
metadata:
 name: my-app
spec:
 replicas: 1
 selector:
 matchLabels:
 app: my-app
 template:
 metadata:
 labels:
 app: my-app
 spec:
 containers:
 - name: my-app
 image: nginx
 resources:
 requests:
 cpu: "1"
 memory: "256Mi"
 limits:
 cpu: "500m"
 memory: "512Mi"
```

To activate this feature, enable the in-place Pod vertical scaling feature flag with the following command:

```
```shell theme={null} $ FEATURE_GATES=InPlacePodVerticalScaling=true
```

Once enabled, you can specify additional resize policy parameters to control restart behavior.

Configuring Resize Policies

The following manifest demonstrates a resize policy for the CPU resource, allowing an update

```
```yaml theme={null}
apiVersion: apps/v1
kind: Deployment
metadata:
 name: my-app
spec:
 replicas: 1
 selector:
 matchLabels:
 app: my-app
 template:
 metadata:
 labels:
 app: my-app
 spec:
 containers:
 - name: my-app
 image: nginx
 resizePolicy:
 - resourceName: cpu
 restartPolicy: NotRequired
 resources:
 requests:
 cpu: "250m"
 memory: "256Mi"
 limits:
 cpu: "500m"
 memory: "512Mi"
```

When the in-place update feature is active, increasing the CPU resource value (for example, from "250m" to "1") updates the running Pod directly without termination, ensuring smooth scalability and minimal service interruption.

The in-place resizing feature currently supports only CPU and memory resources. Familiarize yourself with these constraints before implementation.

## Limitations of In-Place Pod Resizing

It is important to note the following limitations with the current implementation of in-place resizing:

- Only CPU and memory resources can be updated in place.
- Changes to Pod QoS classes and certain other attributes are not supported.
- Init containers and ephemeral containers are not eligible for in-place resizing.

- Resource requests and limits, once assigned to a container, cannot be shifted to another container.
- A container's memory limit cannot be reduced below its current usage; if such a request is made, the resize operation will remain in progress until the new memory limit is achievable.
- Windows Pods are not supported by this feature.

## Looking Ahead: Vertical Pod Autoscaler

---

While this article focused on manually resizing Pod resources, Kubernetes also offers the Vertical Pod Autoscaler (VPA) to automate scaling based on resource usage. VPA continuously monitors your application's resource consumption and adjusts requests and limits as needed, further reducing manual intervention.

For more details on Kubernetes resource management, visit the [Kubernetes Documentation](#) or explore [Kubernetes Basics](#).

That's it for this topic. Enjoy seamless in-place updates and improved resource scaling with Kubernetes!

## Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

# Vertical Pod Autoscaling VPA 2025 Updates

This article explores optimizing Kubernetes workloads by using the Vertical Pod Autoscaler to automatically adjust resource allocations for applications.

In this article, we explore how to optimize Kubernetes workloads by scaling them vertically using the Vertical Pod Autoscaler (VPA). As a Kubernetes administrator, your goal is to ensure that applications always receive optimal resource allocations, such as CPU and memory. Let's start by examining a typical deployment configuration for a pod that specifies a CPU request of 250 millicores and a limit of 500 millicores:

```
```yaml theme={null} apiVersion: apps/v1 kind: Deployment metadata: name: my-app spec: replicas: 1 selector: matchLabels: app: my-app template: metadata: labels: app: my-app spec: containers: - name: my-app image: nginx resources: requests: cpu: "250m" limits: cpu: "500m"
```

In this setup, the pod cannot use more than 500 millicores of CPU. [Click here to monitor its resource](#)

```
```bash theme={null}
$ kubectl top pod my-app-pod
NAME CPU(cores) MEMORY(bytes)
my-app-pod 450m 350Mi
```

If the pod's CPU consumption reaches a predefined threshold, you might need to update its resource specifications manually. For example, you can increase the CPU request to "1" while keeping the limit unchanged:

```
```yaml theme={null} apiVersion: apps/v1 kind: Deployment metadata: name: my-app spec: replicas: 1 selector: matchLabels: app: my-app template: metadata: labels: app: my-app spec: containers: - name: my-app image: nginx resources: requests: cpu: "1" limits: cpu: "500m"
```

To apply this change, run:

```
```bash theme={null}
$ kubectl edit deployment my-app
```

After saving, Kubernetes will terminate the current pod and create a new one with the updated resource configuration.

Manually updating pods can be time-consuming and error-prone. Kubernetes provides the Vertical Pod Autoscaler (VPA) to automate this process.

Kubernetes distinguishes between scaling methods. While the Horizontal Pod Autoscaler (HPA) adds or removes pods based on demand, the VPA continuously monitors metrics and automatically adjusts the CPU and memory allocation of each pod. Since VPA is not enabled by default, you must install it manually. Start by applying the VPA definition file from the autoscaler GitHub repository:

```
```bash theme={null} $ kubectl apply -f https://github.com/kubernetes/autoscaler/releases/latest/download/vertical-pod-autoscaler.yaml
```

Verify that the VPA components are running in the kube-system namespace:

```
```bash theme={null}
$ kubectl get pods -n kube-system | grep vpa
vpa-admission-controller-xxxx Running
vpa-recommender-xxxx Running
vpa-updater-xxxx Running
```

The VPA deployment includes three key components:

1. **VPA Recommender:** Continuously monitors resource usage via the Kubernetes metrics API, analyzes historical and live data, and provides optimized recommendations for CPU and memory.
2. **VPA Updater:** Compares current pod resource settings against recommendations and evicts pods running with suboptimal resources. This eviction triggers the creation of new pods with updated configurations.
3. **VPA Admission Controller:** Intercepts pod creation requests and mutates the pod specification based on the recommender's suggestions, ensuring that new pods start with the ideal resource configuration.

Next, create a VPA resource with a YAML definition. Unlike HPA, the VPA isn't set up through imperative commands. The example below shows a configuration that monitors the "my-app" deployment, enforces minimum and maximum CPU limits, and uses the "Auto" update mode:

```
```yaml theme={null}    apiVersion:    autoscaling.k8s.io/v1    kind:
VerticalPodAutoscaler metadata: name: my-app-vpa spec: targetRef:
apiVersion: apps/v1 kind: Deployment name: my-app updatePolicy:
```

```
updateMode: "Auto" resourcePolicy: containerPolicies: - containerName: "my-app" minAllowed: cpu: "250m" maxAllowed: cpu: "2" controlledResources: ["cpu"]
```

```
In "Auto" mode, the VPA updater behaves similarly to a "recreate" strategy by terminating the pod and creating a new one with the recommended resources.

To inspect the resource recommendations provided by VPA for your deployment, run:

```bash theme={null}
$ kubectl describe vpa my-app-vpa
```

You might see an output similar to this, which indicates a recommended CPU value of 1.5:

```
text theme={null} Recommendations: Target: Cpu: 1.5
```

## Comparing Vertical and Horizontal Pod Autoscaling

Understanding when to use VPA versus HPA is crucial for efficient resource management:

Feature	Vertical Pod Autoscaling (VPA)	Horizontal Pod Autoscaling (HPA)
Scaling Method	Adjusts CPU and memory settings of individual pods (may restart pods for changes).	Increases or decreases the number of pods to distribute load.
Pod Behavior	May cause temporary downtime during pod restarts.	Scales pods seamlessly without interrupting existing ones.
Traffic Handling	Less effective for sudden spikes due to restart delays.	Ideal for handling rapid traffic spikes by adding more pods instantly.
Cost Optimization	Prevents over-provisioning by matching resource allocation with actual usage.	Reduces operational costs by avoiding underutilized pods.
Use Cases	Stateful workloads, databases, JVM-based applications, and AI workloads requiring precise tuning.	Stateless applications, web services, and microservices requiring rapid scaling.



Key Differences		
Feature	VPA (Vertical Scaling)	HPA (Horizontal Scaling)
Scaling Method	Increases CPU & memory of existing Pods	Adds/removes Pods based on load
Pod Behavior	Restarts Pods to apply new resource values	Keeps existing Pods running
Handles Traffic Spikes?	❌ No, because scaling requires a Pod restart	✅ Yes, instantly adds more Pods
Optimizes Costs?	✅ Prevents over-provisioning of CPU/memory	✅ Avoids unnecessary idle Pods
Best For	Stateful workloads, CPU/memory-heavy apps (DBs, ML workloads)	Web apps, microservices, stateless services
Example Use Cases	Databases (MySQL, PostgreSQL), JVM-based apps, AI/ML workloads	Web servers (Nginx, API services), message queues, microservices

© Copyright KodeKloud

VPA focuses on optimizing resource allocation for individual pods, while HPA scales the number of pods to meet demand. The choice depends on your application's workload characteristics and scaling requirements.

In summary, the Vertical Pod Autoscaler (VPA) enhances Kubernetes resource management by dynamically adjusting CPU and memory allocations based on real-time metrics. By applying these VPA techniques, you can ensure that your applications run efficiently without manual intervention.

Now, try deploying VPA in your Kubernetes environment to experience improved resource optimization and operational efficiency.

# Section: Cluster Maintenance

## Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

## Cluster Maintenance Section Introduction

This article explores essential cluster maintenance topics including operating system upgrades, node removal, cluster upgrade processes, and backup methodologies.

Hello, this is Mumshad Mannambeth. In this article, we will explore essential cluster maintenance topics as part of the [Certified Kubernetes Administrators course](#). We'll cover key areas such as operating system upgrades, node removal, and cluster upgrade processes, as well as backup and disaster recovery methodologies.

## Operating System and Node Maintenance

We begin by discussing operating system upgrades and the considerations involved when a node is lost from the cluster, either unintentionally or due to deliberate removal for patching or upgrading purposes. Understanding these procedures is critical to ensuring minimal disruption to your Kubernetes environment.



## Cluster Upgrade Process

Before upgrading your cluster, it's important to have a solid grasp of Kubernetes releases, versioning, and best practices for selecting the appropriate upgrade pathway. Once you understand the upgrade procedure, you'll get hands-on experience performing an end-to-end upgrade on a live cluster running applications.

Before proceeding with an upgrade, always run a pre-flight check using the `kubeadm upgrade plan` command. This command helps you verify the cluster's health and fetch available upgrade versions.

To check the upgrade plan, run the following command:

```
``bash theme={null} kubeadm upgrade plan
```

The expected output will look similar to:

```
```bash theme={null}
[upgrade] Running pre-flight checks:
[upgrade] Making sure the cluster is healthy:
[upgrade] Config: Making sure the configuration is correct:
[upgrade] Fetching available versions to upgrade to:
[upgrade/versions] Cluster version: v1.11.1
[upgrade/versions] Kubernetes version: v1.11.8
[upgrade/versions] Latest stable version: v1.11.3
```

Components that must be upgraded manually after you have upgraded the control plane with ``kubeadm upgrade apply``:

COMPONENT	CURRENT	AVAILABLE
Kubelet	v1.11.1	v1.11.3

Upgrade to the latest stable version:

COMPONENT	CURRENT	AVAILABLE
API Server	v1.11.8	v1.11.3
Controller Manager	v1.11.8	v1.11.3
Scheduler	v1.11.8	v1.11.3
Kube Proxy	v1.11.8	v1.11.3
Etc	v3.1.8	N/A

You can now apply the upgrade by executing the following command:

After upgrading the control plane with `kubeadm upgrade apply`, make sure to manually upgrade components like kubelet, as highlighted in the output above.

Backup and Disaster Recovery

In the final part of this module, we focus on backup and restore methodologies. This section will guide you through a disaster recovery simulation where you back up your Kubernetes cluster, simulate a catastrophic event, and then restore the cluster to its original state. This practical exercise is designed to enhance your ability to manage and recover a Kubernetes cluster effectively.

Let's get started on ensuring your clusters are resilient and well-maintained!

Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

OS Upgrades

This lesson covers handling node upgrades in Kubernetes, focusing on draining, cordoning, and minimizing service interruptions during maintenance or upgrades.

Welcome to this lesson on handling node upgrades in a Kubernetes cluster. In this guide, we will explore various scenarios that require taking a node offline, such as routine maintenance, base software upgrades, or the application of security patches. We also review options to minimize service interruptions during these events.

Imagine a cluster with several nodes where each node runs pods serving your applications. What happens if a node goes down? The pods on that node become inaccessible, and depending on your deployment strategy, this could impact your users. For instance, if you have multiple replicas of a "blue" pod, your users will still be served by the remaining replicas. However, if a "green" pod is the only instance running, its unavailability results in downtime.

Kubernetes offers mechanisms to handle such situations:

- If a node comes back online quickly, the pods are simply restarted.
- If a node remains down for more than five minutes, Kubernetes marks the pods on that node as dead. For pods managed by a ReplicaSet, new pods are automatically created on other nodes. In contrast, pods that are not part of a ReplicaSet will not be restarted, potentially leading to downtime.

When a node is down for a short duration (less than five minutes) and your workload has multiple replicas, a quick upgrade and reboot might be acceptable.

Draining a Node

For situations where the recovery time is uncertain, the safer method is to drain the node. Draining involves gracefully terminating the pods running on that node so they are recreated on other nodes. At the same time, draining

marks the node as unschedulable (cordoned), preventing new pods from being scheduled on it until explicitly allowed.

After the node's workloads have been safely relocated, you can reboot the node. When it comes back online, it remains unschedulable until you uncordon it, allowing new pods to be scheduled. It is important to note that pods moved to other nodes do not automatically revert to the original node after uncordoning. If they were deleted or if new pods have been scheduled across the cluster, Kubernetes maintains the current distribution.

To perform these operations, run the following commands:

```
bash theme={null} kubectl drain node-1 kubectl uncordon node-1
```

Cordon vs. Drain

In addition to draining and uncordoning, Kubernetes provides the cordon command. Unlike drain, cordon only marks a node as unschedulable without terminating or relocating the currently running pods. This ensures that no new pods will be scheduled on the node.

Be cautious when using cordon on a node with critical workloads. Since existing pods remain and new pods cannot be scheduled, your application might become overloaded or experience unexpected behavior.

That concludes this lesson on managing node upgrades in a Kubernetes cluster. You are encouraged to practice these techniques in a hands-on lab or test environment to reinforce your understanding of draining, cordoning, and uncordoning nodes.

Keep building your skills with these essential Kubernetes operations!

Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

Cluster Upgrade Introduction

This article explains the process of upgrading a Kubernetes cluster, focusing on control plane components and strategies for upgrading master and worker nodes.

Welcome to this article on upgrading a Kubernetes cluster. Here, we will explain the upgrade process focusing on the core control plane components, while keeping external dependencies like ETCD and CoreDNS aside. In a previous article, we discussed how Kubernetes manages software releases and how its components can have independent versioning.

It is important to note that not all components are required to run on the same version. Although different components can operate on varying release versions, the Kube API Server remains the primary control plane component that all others communicate with. Consequently, no component should ever run on a version higher than the API Server. For example:

- The controller manager and scheduler may be one version lower than the API Server.
- The Kubelet and Kube Proxy components may be two versions lower than the API Server.

For instance, if the API Server is at version 1.10, then:

- The controller manager and scheduler can run on version 1.10 or 1.9.
- The Kubelet and Kube Proxy can run on version 1.8. Running any component on a version higher than the API Server (e.g., 1.11 when the API Server is 1.10) is not recommended.

The Kube Control utility is an exception and may run on a version that is higher, lower, or the same as the API Server. This flexibility supports live, rolling upgrades where components can be upgraded individually.

When to Upgrade

Suppose you're running Kubernetes 1.10, and new releases 1.11 and 1.12 are available. Kubernetes officially supports up to the three most recent minor versions. With 1.12 as the latest, the supported versions are 1.12, 1.11, and 1.10. When version 1.13 is released, only 1.13, 1.12, and 1.11 will be supported. It is advisable to upgrade your cluster to the next release before support for your current version is dropped.

An effective upgrade strategy is to upgrade one minor version at a time (e.g., upgrade from 1.10 to 1.11, then from 1.11 to 1.12, and finally from 1.12 to 1.13) rather than attempting a large jump between versions. Keep in mind that the upgrade process may vary depending on your cluster setup. Managed Kubernetes services (such as Google Kubernetes Engine) offer a simple upgrade interface, while clusters deployed using tools like kubeadm or manual installation require more hands-on management.



Upgrade Process Overview

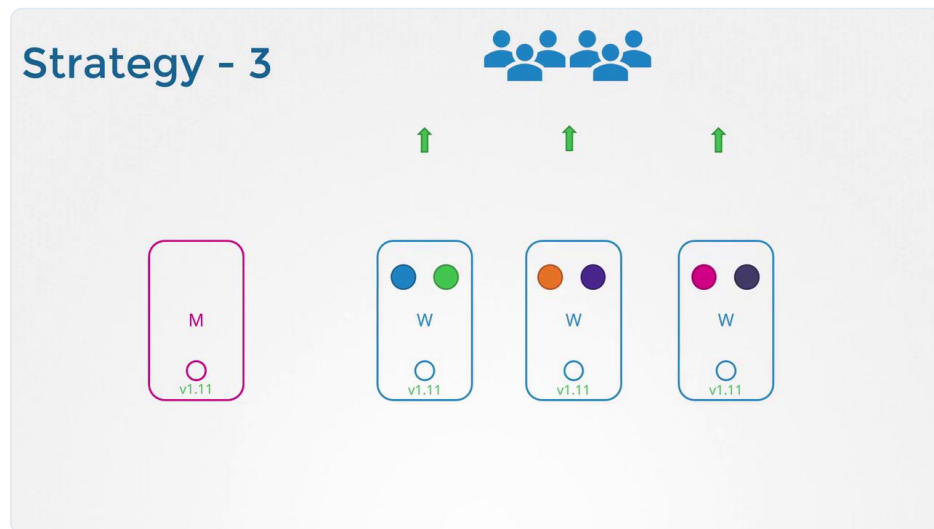
Consider a production cluster with master and worker nodes running version 1.10. The upgrade process generally involves two major steps:

1. Upgrading the master nodes.
2. Upgrading the worker nodes.

During the master node upgrade, control plane components (such as the API server, scheduler, and controller managers) experience a brief interruption. Although management functionality (like `kubectl` commands or scaling deployments) is paused, the worker nodes continue to run and deliver applications. However, keep in mind that if any pods fail during this period, they might not be restarted automatically. Once the master upgrade is complete, normal control plane operations resume.

After the master nodes are upgraded (for example, moving from version 1.10 to 1.11 while the worker nodes are still at 1.10), the next step is to upgrade the worker nodes. There are several strategies for this:

1. Upgrade all worker nodes simultaneously (which may result in downtime).
2. Upgrade one worker node at a time, allowing workloads to be shifted and ensuring continuous service.
3. Add new nodes with the updated software version, migrate workloads to these new nodes, and then decommission the older nodes.



Upgrading with kubeadm

Suppose you want to upgrade your cluster from version 1.11 to 1.13. The kubeadm tool simplifies planning and executing cluster upgrades. To start, run:

```
``bash theme={null} kubeadm upgrade plan
```

This command provides useful information such as:

- * The current cluster version.
- * The version of the kubeadm tool.
- * The latest stable version of Kubernetes.
- * A list of control plane components along with their current versions and the target upgrade version.

Remember that after upgrading the control plane components, you must manually upgrade the worker nodes.

Because upgrades are performed one minor version at a time, if you're on version 1.11 and want to upgrade to 1.12, you need to upgrade to 1.12.0 first.

1. Upgrade the kubeadm tool to version 1.12.
2. Upgrade the cluster using the command provided by the upgrade plan (e.g., "kubeadm upgrade apply v1.12.0").

Below is an example of upgrading the control plane components using kubeadm:

```
```bash theme={null}
apt-get upgrade -y kubeadm=1.12.0-00
kubeadm upgrade apply v1.12.0
Output:
[upgrade/successful] SUCCESS! Your cluster was upgraded to "v1.12.0". Enjoy!
[upgrade/kubelet] Now that your control plane is upgraded, please proceed with upgrading worker nodes.
```

After this step, the control plane components are updated to 1.12. Note that if you run:

```
```bash theme={null} kubectl get nodes
```

```
the master node might still appear as 1.11 because this command displays the version of the kubelet on the node.

```bash theme={null}
kubectl get nodes
```

you should see the master node running 1.12 while the worker nodes continue on 1.11.

## Upgrading Worker Nodes

Upgrading worker nodes should be done one at a time. The recommended approach is to drain one worker node to ensure the applications remain available on other nodes. Use the following command to cordon the node and safely evict pods:

```
```bash theme={null} kubectl drain node-1
```

```
After draining the node, upgrade the kubeadm and kubelet packages on the worker node using:

```bash theme={null}
kubectl uncordon node-1
```

Repeat this procedure for each worker node until the entire cluster is updated. When upgrading from 1.11 to 1.12 and eventually to 1.13, follow this process consistently.

## Conclusion

---

In this article, we outlined the process of upgrading a Kubernetes cluster using kubeadm, focusing on two key scenarios: upgrading the control plane (master nodes) and safely upgrading worker nodes without affecting application availability. For best practices, consider testing the upgrade process in a non-production environment to simulate live upgrades and ensure zero downtime for your users.

Good luck with your Kubernetes cluster upgrade!

For more information on Kubernetes best practices and upgrade strategies, please refer to the [Kubernetes Documentation](#).

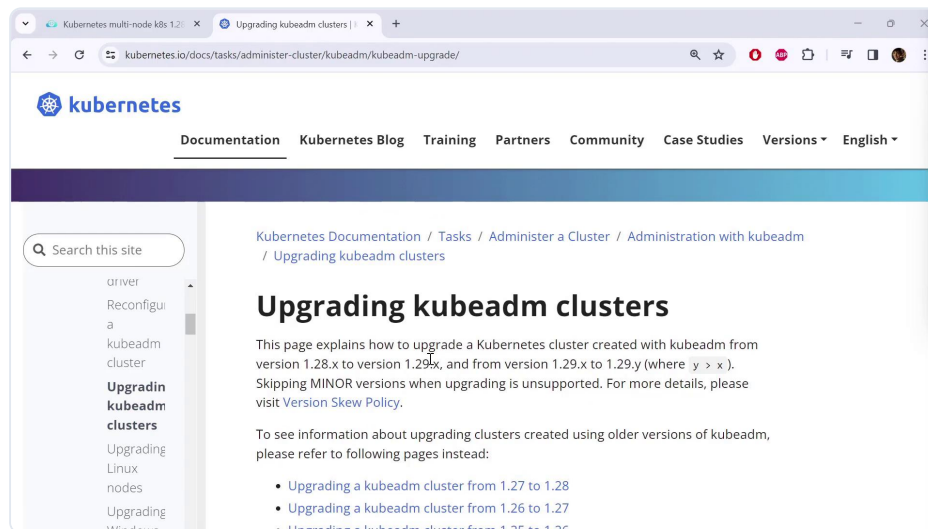
## Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

# Demo Cluster upgrade

This guide demonstrates upgrading a Kubernetes cluster from version 1.28 to 1.29 using kubeadm following official instructions.

In this guide, we demonstrate how to upgrade a Kubernetes cluster from version 1.28 to 1.29 using kubeadm. We follow the official "Upgrading a Kubeadm Cluster" instructions from the Kubernetes documentation. For further details, refer to the "Upgrading a Kubeadm Cluster" section in the [Kubernetes Documentation](#).

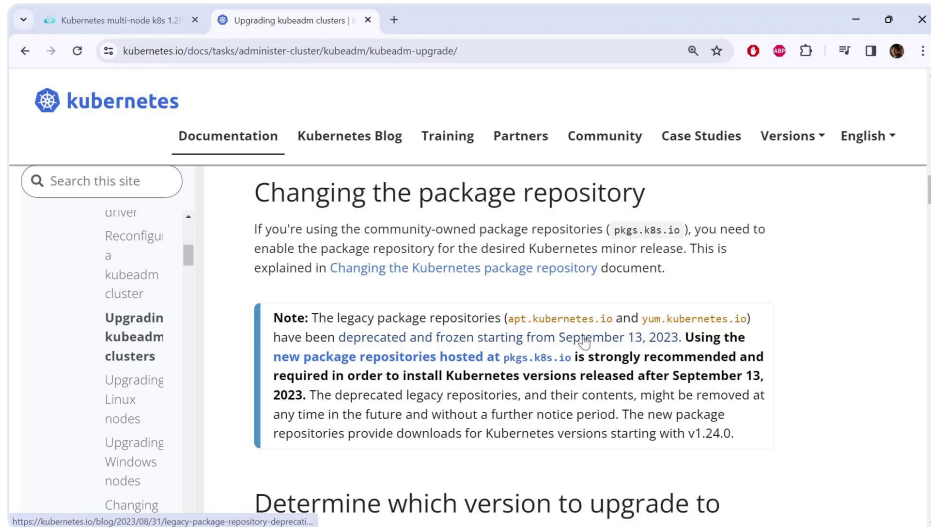


The documentation provides dedicated instructions for each upgrade path. In this case, we are upgrading from 1.28 to 1.29 (the latest release). Similar procedures exist for other upgrade paths (e.g., 1.27 to 1.28, 1.26 to 1.27, etc.). Simply select the correct upgrade path and follow the detailed steps accordingly.

## Updating Package Repositories

Before beginning the upgrade, review the updated package repository information. The legacy repositories ([app.kubernetes.io](http://app.kubernetes.io) and

yum.kubernetes.io) have been deprecated. Moving forward, packages are available at [packages.k8s.io](https://packages.k8s.io). This repository now hosts the latest versions of essential tools such as kubectl and kubeadm.



Before proceeding, confirm that you are using the new package repositories. Check the documentation and click the provided link for the latest instructions.

## Verify Your Node OS and Repository Setup

First, determine your node distribution because the commands differ slightly by operating system. For example, on a two-node cluster, run:

```
``bash theme={null} kubectl get nodes
```

An example output may be:

```
``plaintext theme={null}
NAME STATUS ROLES AGE VERSION
controlplane Ready control-plane 98m v1.28.0
node01 Ready <none> 98m v1.28.0
```

To check your OS distribution, use:

```
``bash theme={null} cat /etc/release
```

Example output for Ubuntu 20.04.6 LTS (Debian-based):

```
``plaintext theme={null}
DISTRIB_ID=Ubuntu
DISTRIB_RELEASE=20.04
DISTRIB_CODENAME=focal
DISTRIB_DESCRIPTION="Ubuntu 20.04.6 LTS"
NAME="Ubuntu"
VERSION="20.04.6 LTS (Focal Fossa)"
ID=ubuntu
ID_LIKE=debian
PRETTY_NAME="Ubuntu 20.04"
VERSION_ID="20.04"
...
```

For Debian/Ubuntu systems, update the repository to the new endpoint. First, switch to the new URL for your version upgrade. For version 1.29, execute:

```
``bash theme={null} echo "deb [signed-by=/etc/apt/keyrings/kubernetes-apt-keyring.gpg] https://pkgs.k8s.io/core/stable/v1.29/deb/ /" | sudo tee /etc/apt/sources.list.d/kubernetes.list
```

Next, download the public signing key:

```
``bash theme={null}
curl -fsSL https://pkgs.k8s.io/core/stable/v1.28/deb/Release.key | sudo gpg --dearmor -o /usr/share/keyrings/kubernetes-apt-keyring.gpg
```

Remember to adjust the version numbers (e.g., change to 1.29) in both commands as needed during the upgrade process.

After making these updates on the control plane and worker nodes, run:

```
``bash theme={null} sudo apt-get update
```

```

Upgrading the Control Plane

Step 1: Upgrade kubeadm

On the control plane node, list the available kubeadm versions:

``bash theme={null}
sudo apt update
sudo apt-cache madison kubeadm
```

A typical output may appear as follows:

```
``plaintext theme={null} kubeadm | 1.29.3-1.1 | https://pkgs.k8s.io/core/stable/v1.29/deb Packages kubeadm | 1.29.2-1.1 | https://pkgs.k8s.io/core/stable/v1.29/deb Packages kubeadm | 1.29.1-1.1 | https://pkgs.k8s.io/core/stable/
```

v1.29/deb Packages kubeadm | 1.29.0-1.1 | <https://pkgs.k8s.io/core/stable/>  
v1.29/deb Packages

Select the latest version (in this example, 1.29.3-1.1) and upgrade kubeadm with:

```
```bash theme={null}
sudo apt-mark unhold kubeadm && \
sudo apt-get update && \
sudo apt-get install -y kubeadm='1.29.3-1.1' && \
sudo apt-mark hold kubeadm
```

Verify that the kubeadm upgrade is successful by checking the version:

```
```bash theme={null} kubeadm version
```

Expected output:

```
```plaintext theme={null}
kubeadm version: &version.Info{Major:"1", Minor:"29", GitVersion:"v1.29.3", ...}
```

Step 2: Run the Upgrade Plan

Before applying the upgrade, conduct a dry run to ensure compatibility:

```
```bash theme={null} sudo kubeadm upgrade plan
```

This command displays the upgrade details, including the components automatically upgraded:

```
```plaintext theme={null}
[upgrade/versions] Cluster version: v1.28.0
[upgrade/versions] kubeadm version: v1.29.3
[upgrade/versions] Target version: v1.29.3
...
Components that must be upgraded manually after you upgrade the control plane:
COMPONENT                CURRENT    TARGET
kubelet                   2 x v1.28.0 v1.28.8
...
```

Kubeadm upgrades most of the control plane components automatically but leaves the kubelet for manual upgrade.

Step 3: Apply the Control Plane Upgrade

Upgrade your control plane with:

```
```bash theme={null} sudo kubeadm upgrade apply v1.29.3
```

Monitor the process as it renews certificates, updates static Pod manifests, and applies co

```
```plaintext theme={null}
[upgrade/successful] SUCCESS! Your cluster was upgraded to "v1.29.3". Enjoy!
[upgrade/kubelet] Now that your control plane is upgraded, please proceed with upgrading yo
```

Note: Until kubelet is upgraded, `kubectl get nodes` will still display version v1.28.0 for the kubelet.

Upgrading kubelet and kubectl on the Control Plane

Step 1: Drain the Control Plane Node

Before updating kubelet and kubectl, drain the control plane node:

```
``bash theme={null} kubectl drain controlplane --ignore-daemonsets
```

```
### Step 2: Upgrade Packages
```

Next, upgrade kubelet and kubectl to version 1.29.3-1.1:

```
``bash theme={null}
sudo apt-mark unhold kubelet kubectl && \
sudo apt-get update && \
sudo apt-get install -y kubelet='1.29.3-1.1' kubectl='1.29.3-1.1' && \
sudo apt-mark hold kubelet kubectl
```

Reload the systemd configuration and restart the kubelet service:

```
``bash theme={null} sudo systemctl daemon-reload sudo systemctl restart
kubelet
```

Finally, uncordon the control plane node to allow pods to be scheduled:

```
``bash theme={null}
kubectl uncordon controlplane
```

Verify the upgrade by checking the node versions:

```
``bash theme={null} kubectl get nodes
```


The control plane should now reflect version v1.29.3.

Upgrading Worker Nodes

For each worker node, perform the following steps:

1. **Upgrade kubeadm:**

```
```bash theme={null}
sudo apt-mark unhold kubeadm && \
sudo apt-get update && \
sudo apt-get install -y kubeadm='1.29.3-1.1' && \
sudo apt-mark hold kubeadm
```
```

2. **Update Node Configuration:**

Run the command on the control plane to update the node configuration:

```
```bash theme={null}
sudo kubeadm upgrade node
```
```

This command refreshes the node configuration without upgrading the kubelet package.

3. **Drain the Worker Node:**

Use the node name (e.g., node01) to drain it:

```
```bash theme={null}
kubectl drain node01 --ignore-daemonsets
```
```

4. **Upgrade kubelet and kubectl:**

Execute the following commands on the worker node:

```
```bash theme={null}
sudo apt-mark unhold kubelet kubectl && \
sudo apt-get update && \
sudo apt-get install -y kubelet='1.29.3-1.1' kubectl='1.29.3-1.1' && \
sudo apt-mark hold kubelet kubectl
```
```

Reload and restart kubelet:

```
```bash theme={null}
sudo systemctl daemon-reload
sudo systemctl restart kubelet
```
```

5. **Uncordon the Worker Node:**

```
```bash theme={null}
kubectl uncordon node01
```
```

```
After upgrading all worker nodes, verify that every node in the cluster is running version
```

```
```bash theme={null}
kubectl get nodes
```

---

## Summary

---

This guide detailed the process of upgrading a Kubernetes cluster using kubeadm by:

1. Updating package repositories to the new [packages.k8s.io](https://packages.k8s.io).
2. Upgrading kubeadm on the control plane and verifying the available versions.
3. Running a dry-run upgrade plan to check compatibility.
4. Applying the control plane upgrade.
5. Upgrading kubelet and kubectl on both the control plane and worker nodes while minimizing downtime by draining and uncordoning nodes.

Following these steps will ensure that your cluster components run on v1.29.3 securely and efficiently.

Happy upgrading!

## Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

# Backup and Restore Methods

This guide covers backup and restore strategies for Kubernetes environments, focusing on securing deployments and critical components like etcd.

Welcome to this guide on backup and restore strategies for Kubernetes environments. In this lesson, you'll learn how to secure your Kubernetes deployments by backing up declarative configurations, imperative resource changes, and critical cluster components such as etcd.

## What to Back Up

For most Kubernetes deployments, consider backing up:

- **Declarative Configuration Files:** Files defining resources like Deployments, Pods, and Services.
- **Cluster State:** Information stored in the etcd cluster.
- **Imperative Objects:** Resources created on the fly (e.g., namespaces, secrets, configMaps) which might not be documented in files.

Using a declarative approach — creating definition files and applying them with `kubectl` — not only documents your configuration but also makes it reusable and shareable. For example, here's a simple Pod definition:

```
```yaml theme={null} apiVersion: v1 kind: Pod metadata: name: myapp-pod labels: app: myapp type: front-end spec: containers: - name: nginx-container image: nginx
```

Apply the Pod definition with:

```
```bash theme={null} kubectl apply -f pod-definition.yml
```

Storing your configuration files in a version-controlled repository (such as GitHub) ensures you can quickly restore and redeploy your applications if needed.

## Imperative vs. Declarative Backup Approaches

While the declarative method is preferred, sometimes resources are created using imperative commands. These changes might not be stored in your version control system, which can lead to gaps in your backups. To capture all configurations, you can query the Kubernetes API server directly.

For instance, back up all resources across every namespace by running:

```
``bash theme={null} kubectl get all --all-namespaces -o yaml > all-deploy-services.yaml
```

```
This command generates a comprehensive YAML snapshot of pods, deployments, services, and ot

Backing Up the etcd Cluster

The etcd cluster is the backbone of your Kubernetes system, storing critical state and conf

Below is an example of how etcd might be configured on a master node:

``bash theme={null}
ExecStart=/usr/local/bin/etcd \
 --name ${ETCD_NAME} \
 --cert-file=/etc/etcd/kubernetes.pem \
 --key-file=/etc/etcd/kubernetes-key.pem \
 --peer-cert-file=/etc/etcd/kubernetes.pem \
 --peer-key-file=/etc/etcd/kubernetes-key.pem \
 --trusted-ca-file=/etc/etcd/ca.pem \
 --peer-trusted-ca-file=/etc/etcd/ca.pem \
 --client-cert-auth \
 --initial-advertise-peer-urls https://${INTERNAL_IP}:2380 \
 --listen-peer-urls https://${INTERNAL_IP}:2380 \
 --advertise-client-urls https://${INTERNAL_IP}:2379 \
 --initial-cluster etcd-cluster-0 \
 --initial-cluster-token etcd-cluster-0 \
 --initial-cluster controller-0=https://${CONTROLLER0_IP}:2379 \
 --initial-cluster-state new \
 --data-dir=/var/lib/etcd
```

etcd offers a built-in snapshot feature via the etcdctl command. To create a snapshot called "snapshot.db", run:

```
``bash theme={null} ETCDCTL_API=3 etcdctl snapshot save snapshot.db
```

After creating the snapshot, you can verify its existence:

```
``bash theme={null}
ls
```

And check the snapshot status:

```
```bash theme={null} ETCDCCTL_API=3 etcdctl snapshot status snapshot.db
```

Restoring from an etcd Backup

In the event of a failure, restoring your cluster from an etcd backup involves several steps:

1. ****Stop the Kubernetes API Server:**** The restore process requires stopping the API server.
2. ****Restore the Snapshot:**** Restore the snapshot to a new data directory (e.g., `/var/lib/etcd`).

```
```bash theme={null}
ETCDCCTL_API=3 etcdctl snapshot restore snapshot.db \
--data-dir /var/lib/etcd-from-backup
```
```

This command initializes a new etcd data directory and reinitializes cluster members.

3. ****Update etcd Configuration:**** Modify your etcd configuration file to point to the new data directory.
4. ****Restart Services:**** Reload the system daemon, restart the etcd service, and finally restart the Kubernetes API server.

<Callout icon="lightbulb" color="#1CB2FE">

Always supply the required certificate files (CA certificate, etcd server certificate, and etcd client certificate).

For an authenticated backup, use:

```
```bash theme={null}
ETCDCCTL_API=3 etcdctl snapshot save snapshot.db \
--endpoints=https://127.0.0.1:2379 \
--cacert=/etc/etcd/ca.crt \
--cert=/etc/etcd/etcd-server.crt \
--key=/etc/etcd/etcd-server.key
```

## Choosing the Right Backup Approach

Depending on your environment, the backup strategy might vary:

Backup Approach	Use Case	Command Example
Declarative File Backup	When configurations are maintained as code	<code>kubectl apply -f pod-definition.yml</code>
API Server Configuration Backup	Capturing all cluster resources imperatively	<code>kubectl get all --all-namespaces -o yaml &gt; all-deploy-services.yaml</code>
etcd Snapshot	Backing up the critical cluster state	<code>ETCDCCTL_API=3 etcdctl snapshot save snapshot.db</code>

For managed Kubernetes environments where you might not have direct etcd access, relying on API queries is often the more practical solution.

Thank you for following this guide. By mastering these backup and restore strategies, you can enhance the resilience of your Kubernetes clusters and ensure business continuity.

# Section: Security

## Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

## Security Section Introduction

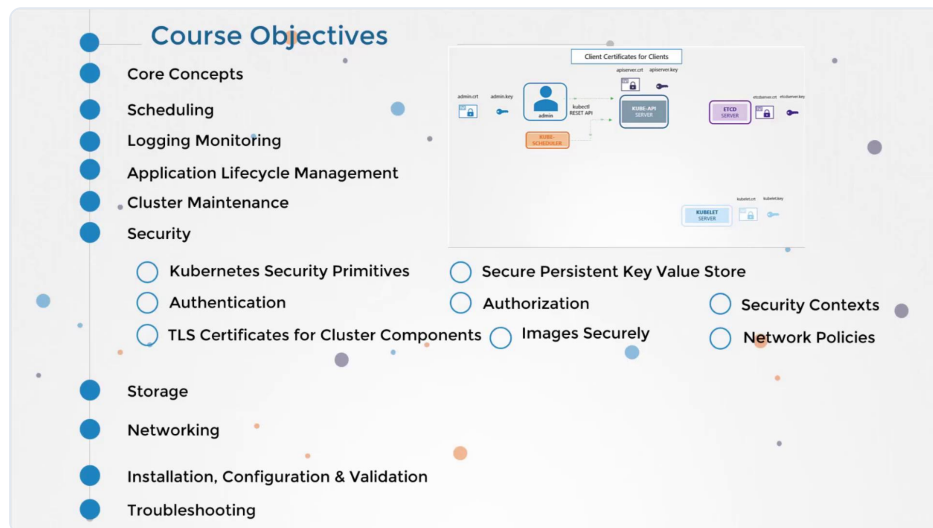
This article provides a comprehensive guide on Kubernetes security, covering access control, authentication mechanisms, TLS certificates, and authorization methods.

Hello, and welcome to this comprehensive article on Kubernetes security. My name is Mumshad Mannambeth, and in this guide, we will begin by exploring the security primitives of Kubernetes. We will explain how access is granted to a Kubernetes cluster, how various actions are controlled, and review the different authentication mechanisms available. Additionally, we will examine the default configurations of a cluster and demonstrate how to view the configurations of an existing cluster.

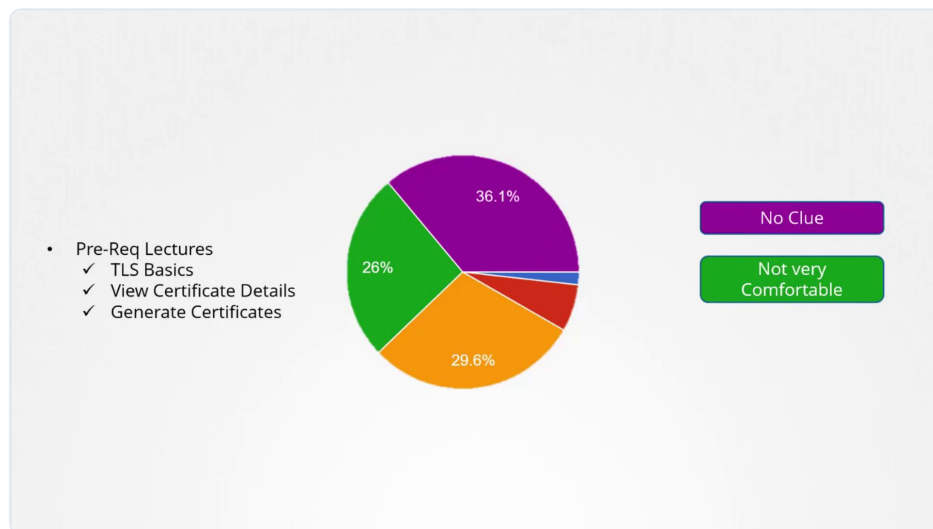
Below is an example CSV file containing user details:

```
csv theme={null} user-details.csv password123,user1,u0001
password123,user2,u0002 password123,user3,u0003
password123,user4,u0004 password123,user5,u0005
```

We then discuss TLS certificates and the role they play in securing components within the cluster. If you are a Kubernetes administrator setting up a cluster on your own, you might encounter challenges related to certificate management. This article covers the basics of certificates from the ground up.



One of the key sections in this article simplifies the core concepts of certificates. For beginners, several prerequisite lectures have been included to build a strong foundation. A poll conducted among the audience revealed that most respondents were new to TLS certificates. This valuable feedback prompted a detailed explanation of how certificates are used within Kubernetes, a topic that often confuses newcomers.



The prerequisite lectures and accompanying practice tests are designed to help you bridge any gaps in your understanding of TLS certificates. Depending on your level of expertise, you might choose to skip some of these sessions.

Later in this article, we will dive into authorization with a specific focus on various mechanisms, including role-based access control (RBAC). We will also cover essential topics such as securing container images, implementing security contexts, and configuring network policies. There is a wealth of lectures and practice tests ahead, so take your time and enjoy the learning process.



## Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

# Kubernetes Security Primitives

This article explores core security features essential for protecting production-grade Kubernetes clusters, including securing hosts, API server access control, and network policies.

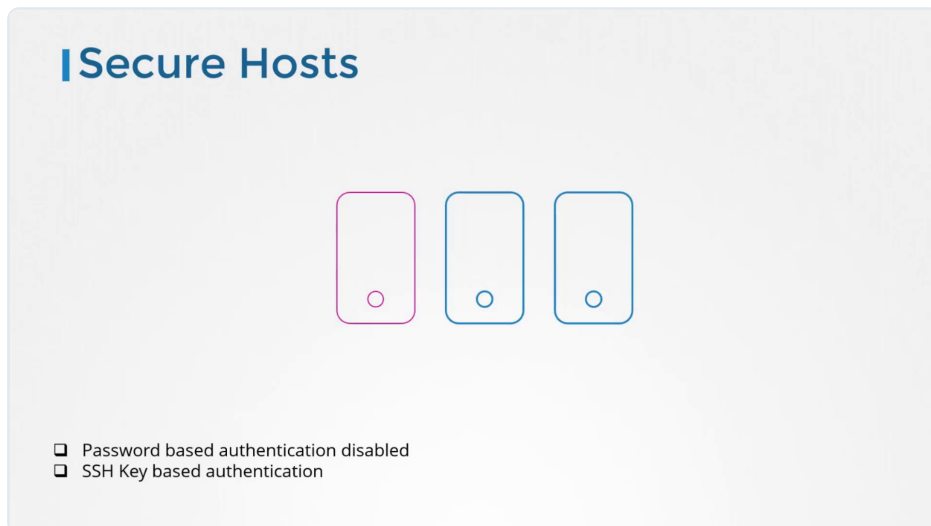
Hello, and welcome to this lesson on Kubernetes security primitives. In this session, we explore the core security features essential for protecting production-grade Kubernetes clusters. This overview introduces key security measures before we delve into detailed configurations and best practices.

## Securing Cluster Hosts

The security of your Kubernetes cluster begins with the hosts themselves. Protect your underlying infrastructure by following these practices:

- Disable root access.
- Turn off password-based authentication.
- Enforce SSH key-based authentication.
- Implement additional measures to secure your physical or virtual systems.

A compromised host can expose your entire cluster, so securing these systems is a critical first step.



## API Server Access Control

The Kube API server is at the heart of Kubernetes operations because all cluster interactions—whether via the `kubectl` command-line tool or directly through API calls—pass through it. Effective access control is essential, focusing on two key questions:

1. Who can access the cluster?
2. What actions are they permitted to perform?

### Authentication

Authentication verifies the identity of a user or service before granting access to the API server. Kubernetes offers various authentication mechanisms to suit different security needs:

- Static user IDs and passwords
- Tokens
- Client certificates
- Integration with external authentication providers (e.g., LDAP)

Additionally, service accounts support non-human processes. Detailed guidance on these methods is available in dedicated sections of our documentation.

## Authentication

### Who can access?

- ☐ Files – Username and Passwords
- ☐ Files – Username and Tokens
- ☐ Certificates
- ☐ External Authentication providers - LDAP
- ☐ Service Accounts

## Authorization

After authentication, authorization determines what actions a user or service is allowed to perform. The default mechanism, Role-Based Access Control (RBAC), associates identities with specific permissions. Kubernetes also supports:

- Attribute-Based Access Control (ABAC)
- Node Authorization
- Webhook-based authorization

These mechanisms enforce granular access control policies, ensuring that authenticated entities can perform only the operations they are permitted to execute.

## Authorization

### What can they do?

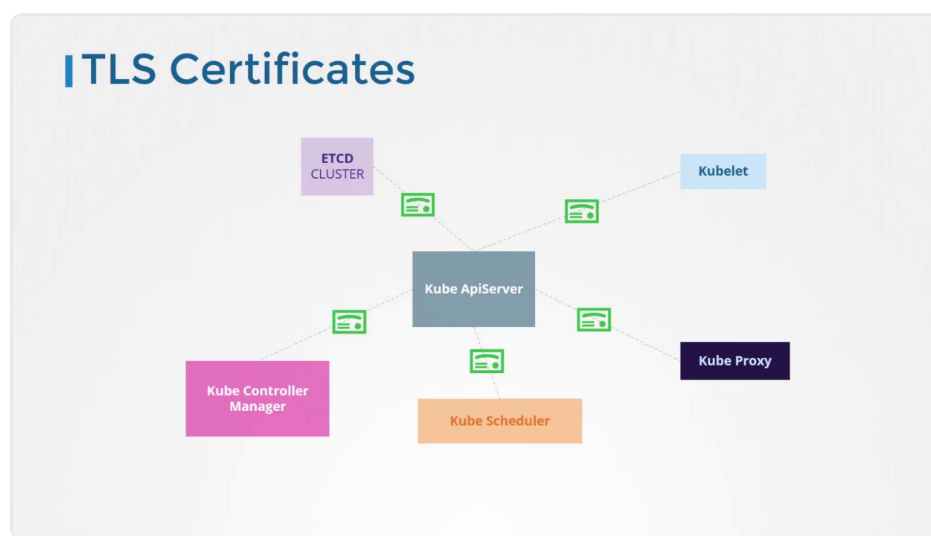
- ☐ RBAC Authorization
- ☐ ABAC Authorization
- ☐ Node Authorization
- ☐ Webhook Mode

## Securing Component Communications

Secure communications between Kubernetes components are enabled via TLS encryption. This ensures that data transmitted between key components remains confidential and tamper-proof. Encryption protects:

- Communication within the etcd cluster
- Interactions between the Kube Controller Manager and Kube Scheduler
- Links between worker node components such as the Kubelet and Kube Proxy

For more detailed instructions on setting up and managing TLS certificates, refer to our comprehensive certificate management guides.



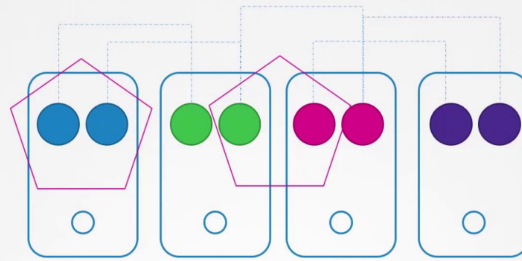
## Network Policies

By default, pods in a Kubernetes cluster communicate freely with one another. To restrict unwanted interactions and enhance security, Kubernetes provides network policies. These policies allow you to:

- Control traffic flow between specific pods
- Enforce security rules at the network level

For an in-depth discussion on implementing network policies with practical examples, please see the related documentation.

## Network Policies



For a secure Kubernetes environment, consider combining these security primitives with additional best practices and tools to counter evolving threats.

This high-level overview has introduced the critical security primitives in Kubernetes. For more detailed procedures and practical examples to further enhance the security of your clusters, please refer to the extended documentation.

Happy securing!

## Documentation Index

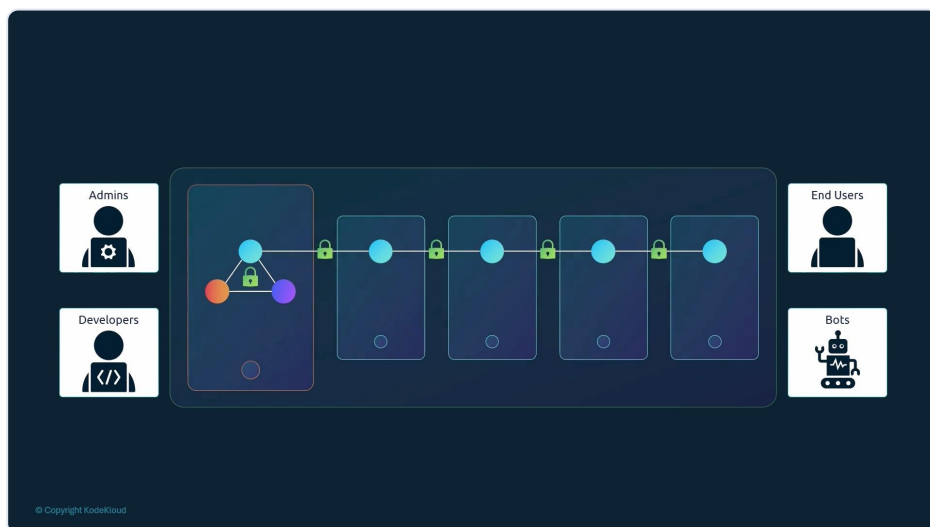
Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

# Authentication

Overview of Kubernetes authentication methods for kube-apiserver, covering legacy static files, TLS client certificates, external identity providers, service accounts, and operational recommendations.

Welcome to this lesson on authentication in a Kubernetes cluster. Kubernetes clusters run on multiple nodes (physical or virtual) and include components that coordinate access to the control plane and workloads. Several types of principals interact with the cluster:

- Administrators who perform cluster-level operations.
- Developers who deploy and iterate on applications.
- End users who access applications (application-level auth is handled by the apps themselves and is out of scope here).
- Robots (processes, controllers, CI systems, and third-party services) that call the Kubernetes API programmatically.



This lesson focuses on securing administrative access to the kube-apiserver — the central API endpoint that authenticates and authorizes all requests to the control plane. That includes access performed by humans (admins, developers) and machines (controllers, CI systems, operators).

Kubernetes does not manage regular user accounts natively: you cannot create or list standard user objects with `kubectl`. User identities are typically introduced to the cluster through external mechanisms, such as:

- static files (legacy),
- TLS client certificates,
- or an external identity provider (OIDC, LDAP, Kerberos, SAML, etc).

Service accounts, on the other hand, are a Kubernetes resource and are created/managed via the API. Example:

```
```bash theme={null}
```

create a service account

```
kubectl create serviceaccount sa1
```

list service accounts in the current namespace

```
kubectl get serviceaccounts
```

All incoming API requests (from kubectl, dashboard, controllers, or direct API calls) are r

Below are the common authentication mechanisms you can configure on kube-apiserver.

<Frame>

</Frame>

Common authentication options

- * Static basic-auth (password) files – legacy.
- * Static token files – legacy.
- * TLS client certificates – recommended for many production setups.
- * External identity providers (OIDC, LDAP, Kerberos) – recommended for large or multi-tenant

Table: Authentication mechanisms at a glance

	Mechanism	Use case
	Static basic-auth file	Small experiments, local demos
	Static token file	Simple automation, throwaway clusters
	TLS client certificates	Secure machine access, admin/user certificates
	External identity providers (OIDC, LDAP)	Enterprise SSO, centralized user management

Legacy mechanisms: static basic-auth and token files

We'll briefly cover these two legacy mechanisms so you understand the underlying concepts.

Static basic-auth file

- * A basic-auth file is a CSV containing password, username, uid, and groups.
- * kube-apiserver reads it when started with the --basic-auth-file flag.
- * Credentials are stored in clear text – insecure and deprecated.

Example basic-auth CSV (legacy format):

```
```csv theme={null}
password,username,uid,groups
KpjCVbI7rCFAHYpKByTIzRb7gu1cUc4B,user10,u0010,group1
rJjncHmvtXHc6MLwQddhtvNyvhgTdxSC,user11,u0011,group1
mjp0FIEiF0KL9toikaRNtt59ePtczZSq,user12,u0012,group2
PG4lIXhs7QjqwWkmBkvgGT9g10yUqZij,user13,u0013,group2
```

Enable this mechanism on kube-apiserver (not recommended):

```
```bash theme={null} --basic-auth-file=/path/to/basic-auth.csv
```

Authenticate with HTTP Basic (example using curl):

```
```bash theme={null}
curl -k -u user10:KpjCVbI7rCFAHYpKByTIzRb7gu1cUc4B https://master-node-ip:6443/api
```

Static token file

- A token file is a CSV mapping bearer tokens to user identities and groups.
- kube-apiserver reads it via the --token-auth-file flag.



- Like the basic file, it stores tokens in plaintext — insecure and deprecated.

Example token CSV (legacy format):

```
``csv theme={null}
```

## token,username,uid,groups

---

```
c29tZXRva2VuMTIz,user21,u0021,group1
```

```
aW5vdGhlcnRva2Vu,user22,u0022,group2
```

Enable on kube-apiserver (not recommended):

```
``bash theme={null}
--token-auth-file=/path/to/token-auth.csv
```

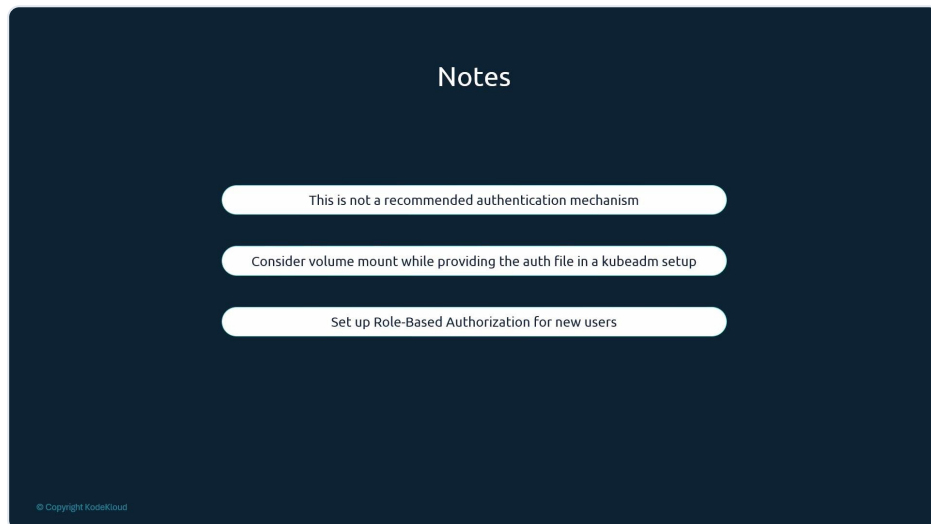
Authenticate with a bearer token (curl example):

```
bash theme={null} curl -k -H "Authorization: Bearer
c29tZXRva2VuMTIz" https://master-node-ip:6443/api
```

### Important operational notes

- kubeadm-managed clusters: kube-apiserver runs as a static pod. To add flags such as `--basic-auth-file` or `--token-auth-file` you must edit the kube-apiserver manifest (commonly `/etc/kubernetes/manifests/kube-apiserver.yaml`) and add `hostPath/volume` mounts so the files are accessible to the pod. The kubelet will detect the manifest change and restart kube-apiserver automatically.
- When you modify kube-apiserver flags or change its static pod manifest, the kube-apiserver process will restart under kubeadm-managed setups; plan for a short control-plane interruption.
- Always combine authentication with proper authorization (RBAC) to control what an authenticated identity can do.

Static basic-auth and token files store credentials in clear text and are considered legacy and unsafe. They are deprecated in modern Kubernetes releases. Prefer certificate-based authentication or external identity providers (OIDC, LDAP, etc.), and always enforce RBAC for authorization.



### Summary and recommendations

- kube-apiserver is the central authentication gateway for Kubernetes; every request to the API is authenticated by it.
- Kubernetes does not create or manage regular user accounts itself — integrate with external identity providers or use client certificates. Service accounts are native Kubernetes objects for in-cluster processes.
- Static basic-auth and token files are easy to understand but insecure and deprecated. Use them only for quick tests or isolated labs.
- For production, prefer TLS client certificates or integrate kube-apiserver with a robust external identity provider (OIDC, LDAP). Always pair authentication with RBAC authorization.

### Further reading and references

- [Kubernetes Authentication Overview](#)
- [kube-apiserver command line reference](#)
- [Using OpenID Connect \(OIDC\) with Kubernetes](#)

## Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

# TLS Introduction

This article provides a guide on understanding and managing TLS certificates in Kubernetes.

Hello, and welcome to this comprehensive guide on TLS certificates in Kubernetes.

Securing your Kubernetes cluster with TLS might seem challenging if you're not already familiar with TLS certificates. In developing this course, we surveyed our audience and discovered that many of you felt uncomfortable or had limited knowledge about TLS certificates. To address this, we've created a series of lessons designed to help you understand and manage certificates within Kubernetes—and beyond.

If you already have a solid understanding of TLS certificates, feel free to skip directly to the Kubernetes-specific sections. By the end of this article, you will be equipped with the skills to efficiently configure, monitor, and troubleshoot certificate-related issues in your Kubernetes environment, while also gaining a broader understanding of how TLS certificates work overall. Mastering these concepts will enhance your ability to secure and maintain robust infrastructures.

## Goals!

- ☐ What are TLS Certificates?
- ☐ How does Kubernetes use Certificates?
- ☐ How to generate them?
- ☐ How to configure them?
- ☐ How to view them?
- ☐ How to troubleshoot issues related to Certificates

We begin with the fundamentals of certificates, covering key concepts such as certificate authorities, issuance, and validation processes. If these basics are already within your grasp, you may proceed to the next section where we delve deeper into managing certificates specifically in Kubernetes.

For best results, it is recommended to review the foundational lessons on certificate management before tackling the Kubernetes-specific content.

## Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

# TLS Basics

This article covers the fundamentals of TLS certificates, their importance for secure communications, and configuration for SSH and web servers.

Welcome to this comprehensive lesson on SSL/TLS certificates. In this guide, we will explore the fundamentals of TLS certificates, discuss why they are essential for secure communications, and demonstrate how to configure them for securing SSH and web servers. A TLS certificate establishes trust during a transaction by ensuring that communications are encrypted and that the server is indeed who it claims to be.

Imagine a scenario where a user accesses their online banking application over an unsecured connection. The credentials entered would be transmitted in plain text, making it easy for a hacker spying on the network to intercept and misuse the sensitive data.

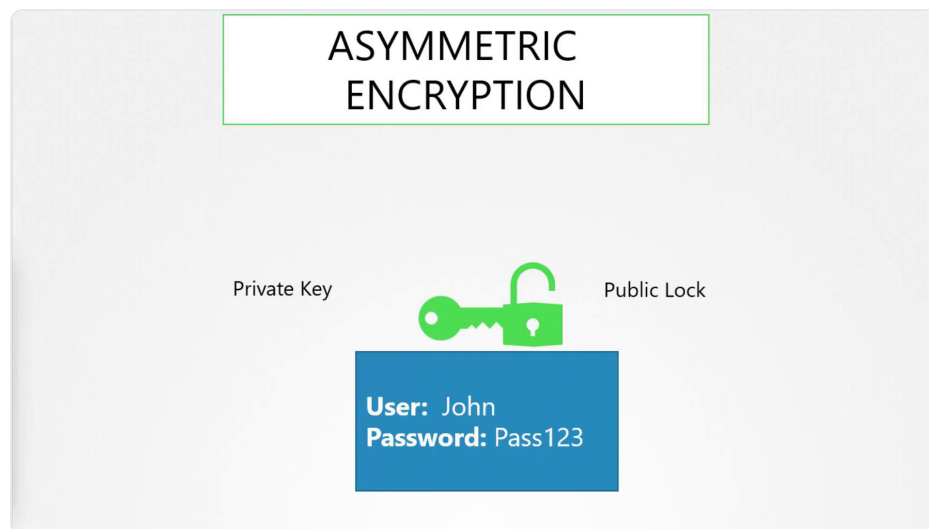


To prevent such risks, data is encrypted using encryption keys—a set of random numbers and characters. Initially, symmetric encryption was used, where the same key is responsible for both encrypting and decrypting data. However, transmitting this key over the network to initiate a secure session

introduces vulnerabilities, as an attacker intercepting the key could decrypt the data.

This is where asymmetric encryption becomes valuable. Asymmetric encryption uses a pair of keys: a private key and a public key. You can think of these as a private key and a public lock. The private key remains securely with the owner, while the public lock can be shared openly. Data encrypted with the public key can only be decrypted using its corresponding private key, ensuring that intercepted data remains secure.

Before delving into the web server example, let's explore a simpler use case: securing SSH access using key pairs.



Imagine you need to access a server but want to avoid the security risks associated with passwords. By using key pairs, you can generate a private key (`id_rsa`) and a public key (`id_rsa.pub`). The private key stays secure on your device, while the public key is added to the server's SSH authorized keys file. When you initiate an SSH connection, you specify your private key to authenticate.

note SSH Key Pair Generation Generate your SSH key pair with the following command:

```
``bash theme={null}
```

## Generate SSH key pair

---

ssh-keygen

## Files generated: id\_rsa (private key) and id\_rsa.pub (public key)

---

You can review the authorized keys on the server as follows:

```
``bash theme={null}
cat ~/.ssh/authorized_keys
```

An example output might appear like:

```
``plaintext theme={null} ssh-rsa
AAAAB3NzaC1yc...KhtUBfoTz1BqRV1NThvO0apzEwRQo1mWx user1
```

To connect securely to the server using your key pair, use:

```
``bash theme={null}
ssh -i id_rsa user1@server1
```

If you have multiple servers, simply copy your public key to each server's authorized keys file so you can authenticate using the same private key on all servers. For additional users who require access, they can generate their own key pairs and have their public keys added to the servers.

Now, let's return to the web server scenario. With symmetric encryption, the key used for encryption must be sent along with the ciphertext, which introduces risk if intercepted. Asymmetric encryption addresses this by securely transferring the symmetric key. Here's how the process works for a web server using HTTPS:

1. The server generates a key pair (private and public keys).
2. Upon a user's initial HTTPS request, the server sends its public key embedded within a certificate.
3. The client's browser encrypts a newly generated symmetric key using the server's public key.
4. The encrypted symmetric key is sent back to the server.
5. The server decrypts the symmetric key using its private key.
6. All subsequent communications are encrypted with this symmetric key.

For example, to generate a key pair with OpenSSL for encrypting the symmetric key, you can use:

```
``bash theme={null}
```

# Generate a private key

```
openssl genrsa -out my-bank.key 1024
```

## Extract the public key

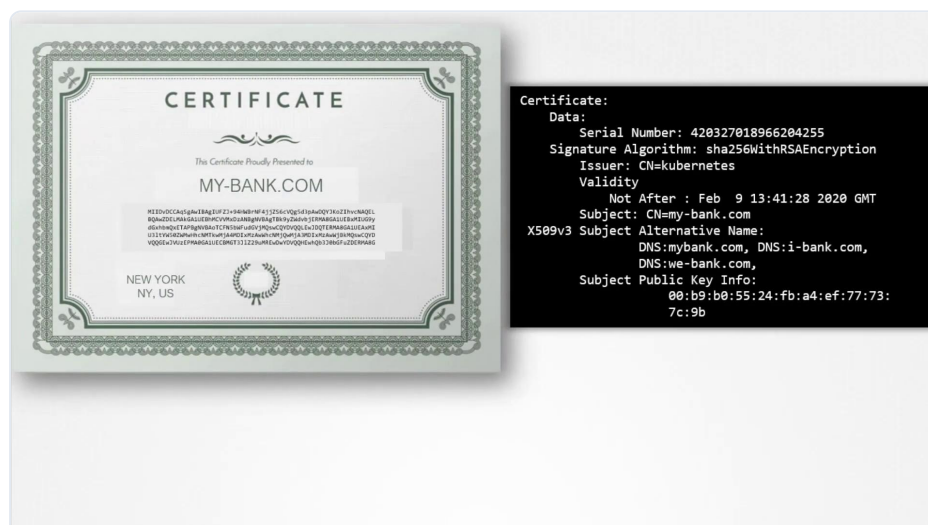
```
openssl rsa -in my-bank.key -pubout > mybank.pem
```

The above commands demonstrate how to create the necessary keys. Although the original content of the certificate is not shown, it is important to understand the structure of a certificate. Imagine a hacker trying to intercept your bank communications by setting up a counterfeit website. A certificate contains essential details that help verify its authenticity:

- \* Identity of the issuing authority
- \* The server's public key
- \* Domain and other related information

Below is an example excerpt from a certificate:

```
```plaintext theme={null}
Certificate:
Data:
  Serial Number: 420327018966204255
  Signature Algorithm: sha256WithRSAEncryption
  Issuer: CN=kubernetes
  Validity
    Not After : Feb  9 13:41:28 2020 GMT
  Subject: CN=my-bank.com
  X509v3 Subject Alternative Name:
    DNS:mybank.com, DNS:i-bank.com,
    DNS:we-bank.com,
  Subject Public Key Info:
    00:b9:b0:55:24:fb:a4:ef:77:73:7c:9b
```



Browsers rely on Certificate Authorities (CAs) to sign and validate certificates. Renowned CAs, such as Symantec, DigiCert, Komodo, and GlobalSign, use their private keys to sign certificate signing requests (CSRs). When you generate a CSR for your web server, it is sent to a CA for signing:

```
bash theme={null} openssl req -new -key my-bank.key -out my-bank.csr  
-subj "/C=US/ST=CA/O=MyOrg, Inc./CN=my-bank.com"
```

Once your details are validated, the CA signs the certificate and sends it back to be installed on your web server. When a user accesses your website, the process is as follows:

1. The server presents the certificate.
2. The browser validates it using pre-installed CA public keys.
3. Upon successful validation, the browser and server establish a secure session using a symmetric key exchanged via asymmetric encryption.

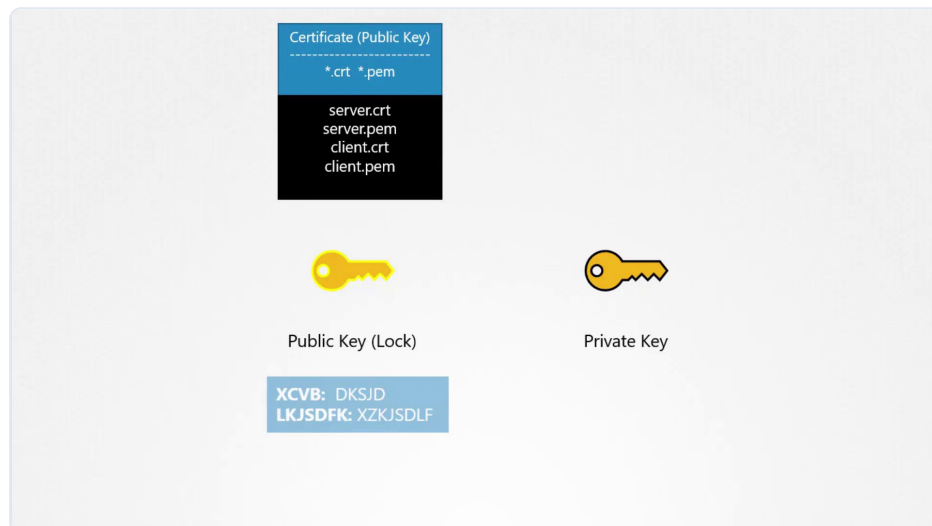
For internal systems, such as corporate payroll applications, organizations may deploy their own private CA and distribute its public key to employee devices.

:::note Key Points Summary

- Asymmetric encryption uses a pair of keys (public and private) to securely exchange symmetric keys.
- SSH access is secured using key pairs.
- Web servers use CA-signed certificates to establish HTTPS connections.
- A Certificate Signing Request (CSR) is generated and sent to a CA for signing.
- Signed certificates, combined with the server's key pair, secure the communication session. :::

It is important to note that although both keys in an asymmetric pair can encrypt data, only the complementary key can decrypt it. For instance, data encrypted with your private key can be decrypted by anyone with your public key; therefore, it's crucial to use the correct key for each operation.

Regarding file naming conventions, certificates containing public keys typically have extensions such as .crt or .pem (e.g., server.crt, server.pem or client.crt, client.pem), and private key files usually include "key" in the filename or extension (e.g., server.key or server-key.pem).



That concludes this lesson on TLS-Basics. Thank you for reading, and I look forward to seeing you in the next lesson!

Additional Resources

- [Kubernetes Documentation](#)
- [SSL/TLS Basics](#)
- [Understanding Cryptography](#)

Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

TLS in Kubernetes

This article provides a guide on securing Kubernetes clusters using TLS certificates, covering key concepts, components, and management practices.

Welcome to this comprehensive guide on securing your Kubernetes cluster using TLS certificates. In this lesson, we cover the fundamentals of public and private keys, the role of certificate authorities (CA), and how both server and client certificates are employed to safeguard connections within your cluster.

Certificate files follow specific naming conventions. Public key certificates typically have a .crt or .pem extension (e.g., server.crt, server.pem, client.crt, client.pem). In contrast, private keys usually include the word "key" in their file name or extension (e.g., server.key or server-key.pem). If a file name lacks "key," it is almost certainly a public key certificate.

TLS Certificates in the Kubernetes Cluster

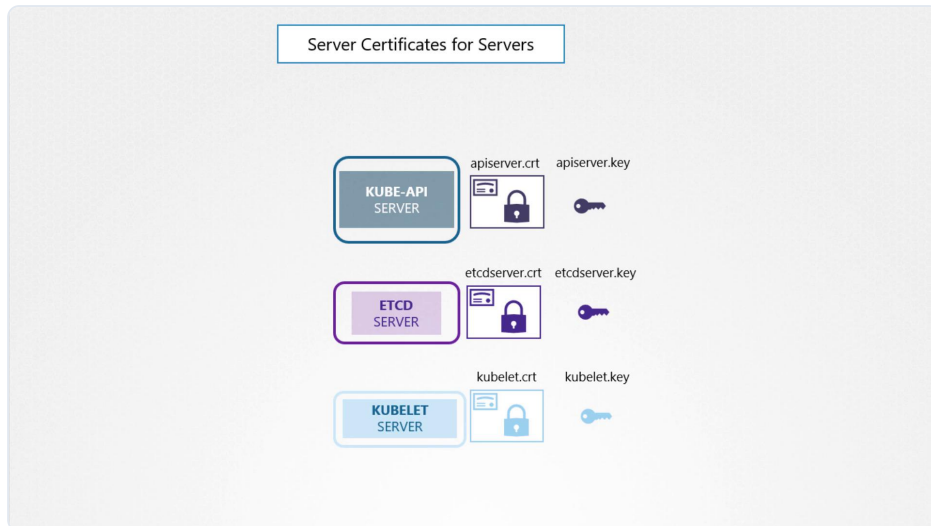
A Kubernetes cluster consists of master and worker nodes that require secure, encrypted communication. Whether the connection is being made by an administrator using the kubectl utility or directly interacting with the Kubernetes API, a secure TLS connection is essential. Additionally, services within the cluster use server certificates to secure their communications, while client certificates authenticate users or other cluster components.

Kubernetes Components and Their Certificates

Below is an overview of the main Kubernetes components and the associated certificate configurations:

1. **Kube API Server** The API server exposes an HTTPS service for managing the cluster. It acts as a server and requires its own certificate and key pair (e.g., `api-server.crt` and `api-server.key`). *Note: Certificate names may vary based on your cluster setup.*

2. **ETCD Server**\ Serving as the datastore for all cluster information, ETCD requires a unique pair of certificates and keys (e.g., `etcd-server.crt` and `etcd-server.key`).
3. **Kubelet on Worker Nodes**\ On each worker node, the kubelet service exposes an HTTPS endpoint that communicates with the API server for monitoring and management. This service uses its own certificate and key pair (typically named `kubelet.crt` and `kubelet.key`).

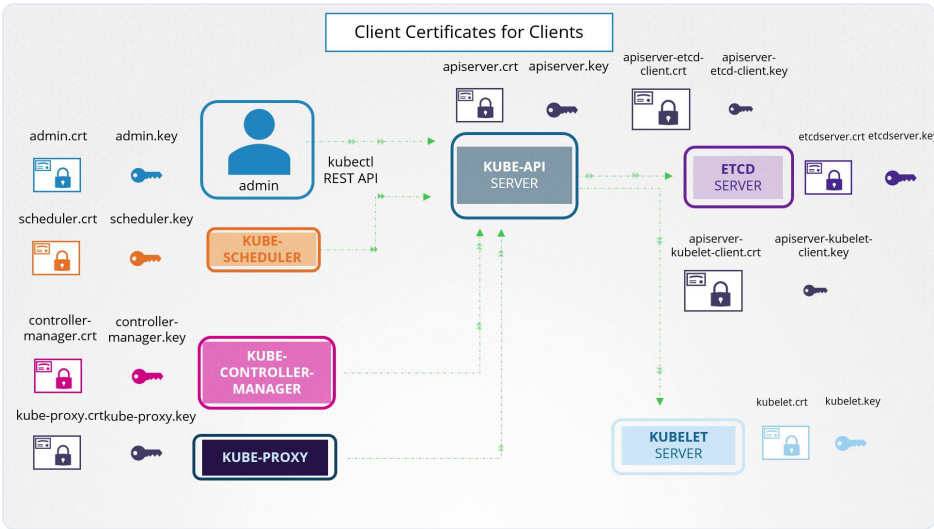


Client Components and Their Certificates

Several components operate as clients and require authentication when connecting to the Kube API Server:

1. **Administrator (kubectl/REST API)**\ The admin user employs a client certificate and key pair (e.g., `admin.crt` and `admin.key`) to establish a secure connection to the API server.
2. **Scheduler**\ Managing pod scheduling, the scheduler requires its own client certificate (e.g., `scheduler.crt` and `scheduler.key`) to interact with the API server.
3. **Kube Controller Manager**\ This component also utilizes a dedicated certificate pair for authenticating its requests to the API server.
4. **Kube Proxy**\ Handling network routing within the cluster, the kube proxy uses a client certificate (generally named `kube-proxy.crt` and `kube-proxy.key`) for secure communications with the API server.

The Kube API Server facilitates secure communications with both the ETCD server and the kubelets. For instance, when connecting to the ETCD server, the API server can either reuse its HTTPS service certificates or use a dedicated certificate pair. Likewise, interactions with kubelets can leverage either the API server's default certificate pair or a separately generated set of certificates.



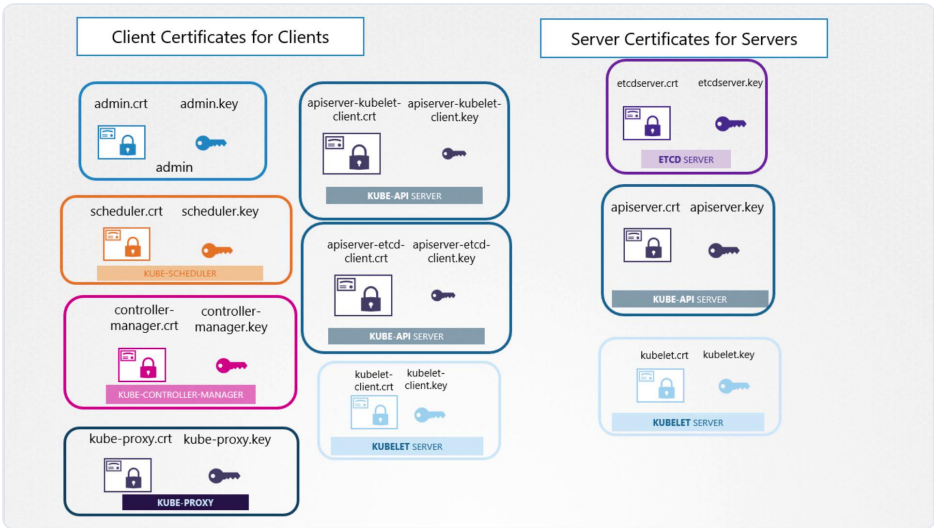
Grouping Certificates for Enhanced Management

For easier certificate management, you can group TLS certificates into two main categories:

Category	Usage
Client Certificates	Used by components (e.g., admin, scheduler, controller manager, kube-proxy) to authenticate with the API server.
Server Certificates	Used by server components (e.g., Kube API Server, ETCD Server, kubelet) to secure their services.

Utilizing a Certificate Authority (CA)

All certificates must be signed by a Certificate Authority. Kubernetes clusters require at least one CA, though multiple CAs may be used for additional segregation (for example, one CA for general cluster services and another exclusively for ETCD). In this guide, we use a single CA for simplicity. The CA itself has a certificate and key pair, typically named `CA.crt` and `CA.key`.



This overview on TLS certificates in Kubernetes sets the foundation for generating and managing these certificates to secure your cluster communications effectively. Happy securing!

Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

TLS in Kubernetes Certificate Creation

This guide explains how to generate TLS certificates for a Kubernetes cluster using OpenSSL, focusing on CA and client/server certificates.

In this guide, you will learn how to generate TLS certificates for a Kubernetes cluster using OpenSSL. While there are other popular tools such as EasyRSA and CFSSL, we will focus on using OpenSSL to create both Certificate Authority (CA) and client/server certificates. This approach secures the communications within your Kubernetes cluster and ensures certified access.

1. Generating CA Certificates

First, create the Certificate Authority (CA) certificates. The process involves generating a private key, creating a Certificate Signing Request (CSR) that includes the CA's common name, and finally signing the CSR with the private key to produce the CA certificate. The completed process provides the CA with its private key (`ca.key`) and root certificate (`ca.crt`), which are essential for subsequently signing other certificates.

```
``bash theme={null} openssl genrsa -out ca.key 2048 openssl req -new -key ca.key -subj "/CN=KUBERNETES-CA" -out ca.csr openssl x509 -req -in ca.csr -signkey ca.key -out ca.crt
```

```
<Callout icon="lightbulb" color="#1CB2FE">
```

```
    The CA certificate is the cornerstone of your certificate infrastructure, so ensure that
```

```
</Callout>
```

```
***
```

2. Creating Client Certificates

2.1 Admin User Certificate

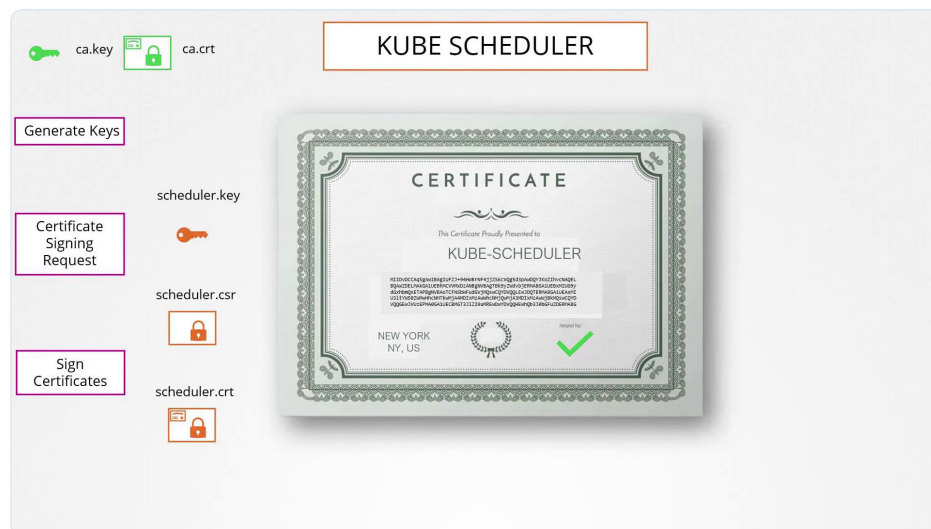
To generate a certificate for the admin user:

1. Create a private key for the admin.
2. Generate a CSR for the admin user specifying the common name (CN) and organizational unit.
3. Sign the admin CSR with the CA certificate to produce the final admin certificate.

```
```bash theme={null}
openssl genrsa -out admin.key 2048
openssl req -new -key admin.key -subj "/CN=kube-admin/O=system:masters" -out admin.csr
openssl x509 -req -in admin.csr -CA ca.crt -CAkey ca.key -out admin.crt
```

The resulting `admin.crt` file functions as a secure credential, akin to a username and password pair, for authenticating the admin user with the Kubernetes API server.

A similar process is followed to generate client certificates for other components such as the scheduler, controller manager, and kube-proxy.



## 3. Using Client Certificates in API Requests

Client certificates eliminate the requirement for using a username and password when making REST API calls to the Kubernetes API server. The admin certificate, for example, can be used to securely communicate with the server by specifying the key, certificate, and CA certificate in your request.

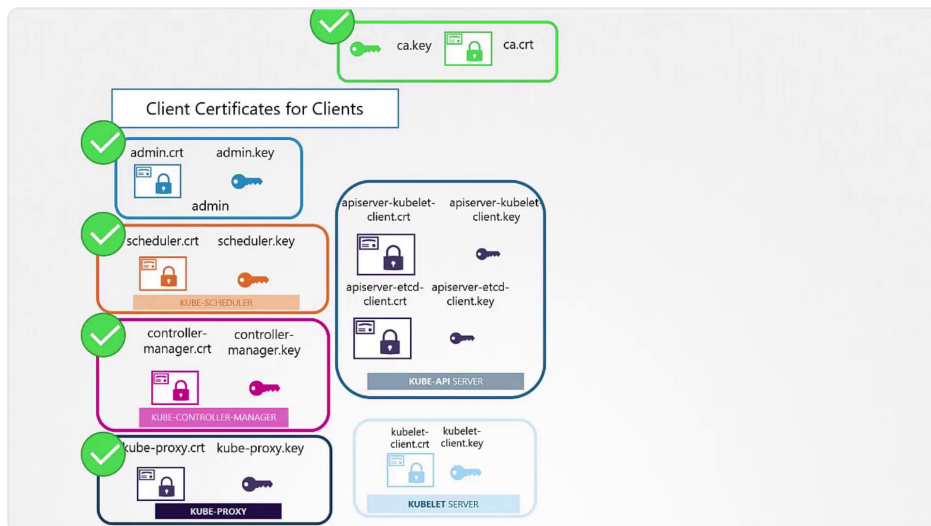


```
```bash theme={null} curl https://kube-apiserver:6443/api/v1/pods \ --key
admin.key --cert admin.crt --cacert ca.crt
```

The API server will respond with a JSON object listing the pods:

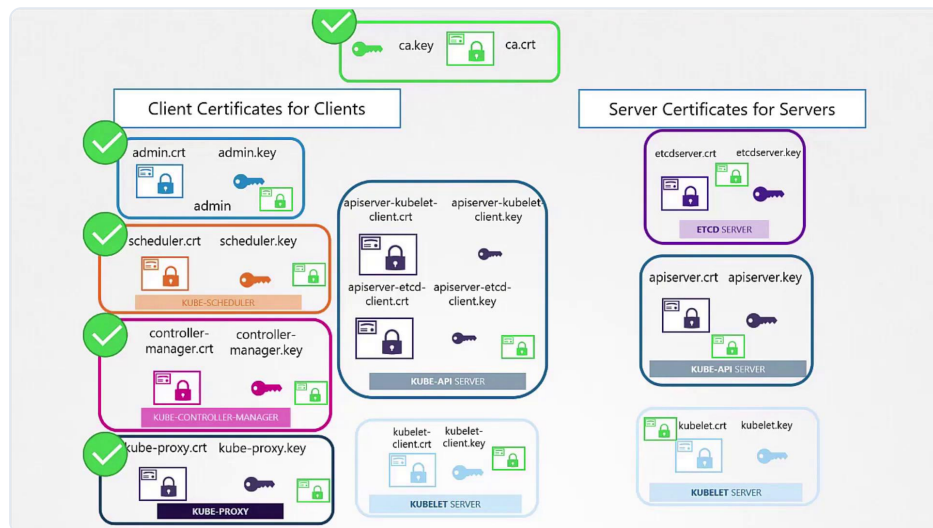
```
```json theme={null}
{
 "kind": "PodList",
 "apiVersion": "v1",
 "metadata": {
 "selfLink": "/api/v1/pods"
 },
 "items": []
}
```

Most Kubernetes clients can load these connection parameters via a kubeconfig file that consolidates the information required to reach the API server.



## 4. Server-Side Certificates

For secure communication, both client and server certificates must trust the same CA root certificate. This certificate is used by both parties to verify the authenticity of the certificate they receive.



## 4.1 Etcd Server Certificate

The etcd server, a critical component in high availability configurations, also requires a certificate. If etcd is running as a cluster, remember to generate peer certificates to secure inter-member communications. Once created, the certificates are referenced in the etcd configuration file (commonly, `etcd.yaml`). See the example below:

```
```bash theme={null} cat etcd.yaml - --advertise-client-urls=https://
127.0.0.1:2379 - --key-file=/path-to-certs/etcdserver.key - --cert-file=/path-to-
certs/etcdserver.crt - --client-cert-auth=true - --data-dir=/var/lib/etcd - --initial-
advertise-peer-urls=https://127.0.0.1:2380 - --initial-cluster=master=https://
127.0.0.1:2380 - --listen-client-urls=https://127.0.0.1:2379 - --listen-peer-
urls=https://127.0.0.1:2380 - --name=master - --peer-cert-file=/path-to-certs/
etcdpeer1.crt - --peer-client-cert-auth=true - --peer-key-file=/etc/kubernetes/pki/
etcd/peer.key - --peer-trusted-ca-file=/etc/kubernetes/pki/etcd/ca.crt - --
snapshot-count=10000 - --trusted-ca-file=/etc/kubernetes/pki/etcd/ca.crt
```

The `--trusted-ca-file` option ensures that etcd client connections are authenticated using

5. Kube API Server Certificates

The Kube API server is the primary access point for the cluster and is known by several al

5.1 Creating the API Server Certificate

Start by generating a CSR for the API server:

```
```bash theme={null}
openssl req -new -key apiserver.key -subj "/CN=kube-apiserver" -out apiserver.csr
```

Then, create an OpenSSL configuration file (e.g., `openssl.cnf`) to include all necessary SANs:

```
``ini theme={null} [req] req_extensions = v3_req distinguished_name =
req_distinguished_name
```

```
[v3_req] basicConstraints = CA:FALSE keyUsage = nonRepudiation,
digitalSignature, keyEncipherment subjectAltName = @alt_names
```

```
[alt_names] DNS.1 = kubernetes DNS.2 = kubernetes.default DNS.3 =
kubernetes.default.svc DNS.4 = kubernetes.default.svc.cluster.local IP.1 =
10.96.0.1 IP.2 = 172.17.0.87
```

After configuring the CSR with the SANs, sign the certificate using your CA certificate and

```
``bash theme={null}
ExecStart=/usr/local/bin/kube-apiserver \
 --advertise-address=${INTERNAL_IP} \
 --allow-privileged=true \
 --apiserver-count=3 \
 --authorization-mode=Node,RBAC \
 --bind-address=0.0.0.0 \
 --enable-swagger-ui=true \
 --etcd-cafile=/var/lib/kubernetes/ca.pem \
 --etcd-certfile=/var/lib/kubernetes/apiserver-etcd-client.crt \
 --etcd-keyfile=/var/lib/kubernetes/apiserver-etcd-client.key \
 --etcd-servers=https://127.0.0.1:2379 \
 --event-ttl=1h \
 --kubelet-certificate-authority=/var/lib/kubernetes/ca.pem \
 --kubelet-client-certificate=/var/lib/kubernetes/apiserver-kubelet-client.crt \
 --kubelet-client-key=/var/lib/kubernetes/apiserver-kubelet-client.key \
 --kubelet-https=true \
 --runtime-config=api/all \
 --service-account-key-file=/var/lib/kubernetes/service-account.pem \
 --service-cluster-ip-range=10.32.0.0/24 \
 --service-node-port-range=30000-32767 \
 --client-ca-file=/var/lib/kubernetes/ca.pem \
 --tls-cert-file=/var/lib/kubernetes/apiserver.crt \
 --tls-private-key-file=/var/lib/kubernetes/apiserver.key \
 --v=2
```

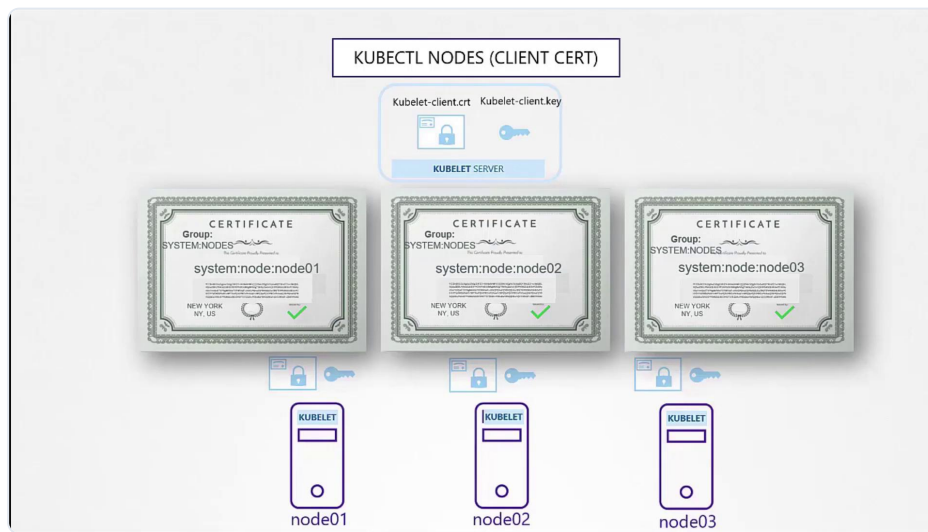
Ensure that all DNS names and IP addresses used by the API server are correctly listed in the SAN section of your OpenSSL configuration.

## 6. Kubelet Certificates

The kubelet is a critical component running on each node, managing node-specific operations and secure communication with the API server. For this purpose, every node needs its own certificate and key pair. When generating these certificates, name them according to the node's identity (e.g., node01, node02, node03).

It is also a best practice to generate a separate certificate for the node when it acts as a client to the API server. This certificate should include an identity

format such as "system:node" to ensure the API server can assign the appropriate group membership (e.g., `system:nodes`).



Once generated, include these certificates in the kubeconfig files for the respective nodes to guarantee secure communication.

---

## Conclusion

This guide has detailed the steps required to generate both client and server certificates using OpenSSL in a Kubernetes environment. You learned how to create a CA certificate, generate and sign client certificates (including for administrative use), and produce server certificates for both etcd and the API server. By including multiple domain names and IP addresses in your certificates, you ensure a robust and flexible security architecture for your cluster.

Future topics will explore how to inspect certificate details and how tools like kubeadm streamline certificate management in Kubernetes clusters.

---

## Additional Resources

- [Kubernetes Documentation](#)
- [Kubernetes Basics](#)
- [Docker Hub](#)
- [Terraform Registry](#)

## Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

# View Certificate Details

Learn to inspect and verify certificates in a Kubernetes cluster, covering both manual setups and automated configurations like kubeadm.

Hello, and welcome to this lesson. In this guide, you will learn how to inspect and verify certificates in an existing Kubernetes cluster. Imagine joining a new team responsible for a Kubernetes environment where you need to perform a health check on cluster certificates due to reported issues. This lesson covers both manually set-up clusters and those configured using automated tools like kubeadm.

## Understanding Your Cluster Setup

Before diving into certificate verification, it's important to understand your cluster's setup. There are several methods for deploying a Kubernetes cluster, and each has its own approach to generating and managing certificates.

- If you deploy a cluster from scratch, you may generate and configure all certificates manually (as explored in a previous lesson).
- If you use an automated provisioning tool like kubeadm, certificate generation and configuration are handled for you. In this case, Kubernetes components are deployed as pods instead of OS services.

### Native Service Deployment

When Kubernetes components are deployed as native services, you can review service files to understand the certificate configuration. For example, inspect the kube-apiserver service file:

```
``bash theme={null} cat /etc/systemd/system/kube-apiserver.service [Service]
ExecStart=/usr/local/bin/kube-apiserver \ --advertise-address=172.17.0.32 \ --
allow-privileged=true \ --apiserver-count=3 \ --authorization-mode=Node,RBAC
\ --bind-address=0.0.0.0 \ --client-ca-file=/var/lib/kubernetes/ca.pem \ --enable-
swagger-ui=true \ --etcd-cafile=/var/lib/kubernetes/ca.pem \ --etcd-certfile=/var/
lib/kubernetes/kubernetes.pem \ --etcd-keyfile=/var/lib/kubernetes/kubernetes-
```

```
key.pem \ --event-ttl=1h \ --kubelet-certificate-authority=/var/lib/kubernetes/
ca.pem \ --kubelet-client-certfile=/var/lib/kubernetes/kubelet-client.crt \ --
kubelet-client-key=/var/lib/kubernetes/kubelet-client.key \ --kubelet-https=true \
--service-node-port-range=30000-32767 \ --tls-cert-file=/var/lib/kubernetes/
kube-apiserver.crt \ --tls-private-key-file=/var/lib/kubernetes/kube-apiserver-
key.pem \ --v=2
```

### ### Deployment Using kubeadm

When using kubeadm, components such as the kube-apiserver are defined as pods in manifest files.

```
```yaml theme={null}
cat /etc/kubernetes/manifests/kube-apiserver.yaml
spec:
  containers:
    - command:
      - kube-apiserver
      - --authorization-mode=Node,RBAC
      - --advertise-address=172.17.0.32
      - --allow-privileged=true
      - --client-ca-file=/etc/kubernetes/pki/ca.crt
      - --disable-admission-plugins=PersistentVolumeLabel
      - --enable-admission-plugins=NodeRestriction
      - --enable-bootstrap-token-auth=true
      - --etcd-cafile=/etc/kubernetes/pki/etcd/ca.crt
      - --etcd-certfile=/etc/kubernetes/pki/apiserver-etcd-client.crt
      - --etcd-keyfile=/etc/kubernetes/pki/apiserver-etcd-client.key
      - --insecure-port=0
      - --kubelet-client-certificate=/etc/kubernetes/pki/apiserver-kubelet-client.crt
      - --kubelet-client-key=/etc/kubernetes/pki/apiserver-kubelet-client.key
      - --proxy-client-certfile=/etc/kubernetes/pki/apiserver-kubelet-client.crt
      - --proxy-client-key=/etc/kubernetes/pki/apiserver-kubelet-client.key
      - --request-timeout=30s
```

Creating a Certificate Inventory

When performing a certificate health check, it's essential to create a checklist—perhaps using a spreadsheet—to record details such as:

- Certificate file paths
- Configured names and alternate names
- Associated organizations
- Certificate owners
- Certificate authorities (issuers)
- Expiration dates

Begin by examining configuration files (such as the kube-apiserver manifest located in `/etc/kubernetes/manifests`) to identify the certificate files in use.

For example, the kube-apiserver manifest might reveal the following options:

```

```yaml
theme={null}
spec:
 containers:
 -
 command:
 - kube-apiserver
 - --authorization-mode=Node,RBAC
 - --advertise-address=172.17.0.32
 - --allow-privileged=true
 - --client-ca-file=/etc/kubernetes/pki/ca.crt
 - --disable-admission-plugins=PersistentVolumeLabel
 - --enable-admission-plugins=NodeRestriction
 - --enable-bootstrap-token-auth=true
 - --etcd-cafile=/etc/kubernetes/pki/etcd/ca.crt
 - --etcd-certfile=/etc/kubernetes/pki/apiserver-etcd-client.crt
 - --etcd-keyfile=/etc/kubernetes/pki/apiserver-etcd-client.key
 - --etcd-servers=https://127.0.0.1:2379
 - --insecure-port=0
 - --kubelet-client-certificate=/etc/kubernetes/pki/apiserver-kubelet-client.crt
 - --kubelet-client-key=/etc/kubernetes/pki/apiserver-kubelet-client.key
 - --kubelet-preferred-address-types=InternalIP,ExternalIP,Hostname
 - --proxy-client-cert-file=/etc/kubernetes/pki/front-proxy-client.crt
 - --proxy-client-key-file=/etc/kubernetes/pki/front-proxy-client.key
 - --secure-port=6443
 - --service-account-key-file=/etc/kubernetes/pki/sa.pub
 - --service-cluster-ip-range=10.96.0.0/12
 - --tls-cert-file=/etc/kubernetes/pki/apiserver.crt
 - --tls-private-key-file=/etc/kubernetes/pki/apiserver.key

```

<Callout icon="lightbulb" color="#1CB2FE">

Make sure to document all certificate details as part of your checklist. This practice si

</Callout>

### ## Inspecting Certificate Details

After identifying certificate files, use OpenSSL to decode them and check their details.

```

```bash
theme={null}
openssl x509 -in /etc/kubernetes/pki/apiserver.crt -text -noout

```

This command displays:

- The subject name and any alternate names
- The validity period (including expiry dates)
- The issuing certificate authority

Repeat this process for all certificates in your Kubernetes cluster. Ensure that:

- Certificate names and alternate names are correctly configured.
- Each certificate is associated with the appropriate organization.
- Certificates are issued by the correct certificate authority (e.g., kubeadm typically uses "Kubernetes" as the CA).
- None of the certificates have expired.

An expired or misconfigured certificate can cause critical disruptions in your cluster's operations. Always validate certificate expiry dates and issuer details.

Troubleshooting with Logs

When certificate issues are suspected, reviewing logs can provide valuable insights.

For Clusters Using Native OS Services

Check service logs using system commands. For example, inspect etcd logs with:

```
``bash theme={null} journalctl -u etcd.service -l
```

Below is an example excerpt from etcd logs:

```
``plaintext theme={null}
2019-02-13 02:53:28.144631 I | etcdmain: etcd Version: 3.2.18
2019-02-13 02:53:28.144680 I | etcdmain: Git SHA: eddf599c6
2019-02-13 02:53:28.144684 I | etcdmain: Go Version: go1.8.7
2019-02-13 02:53:28.144692 I | etcdmain: Go OS/Arch: linux/amd64
2019-02-13 02:53:28.144696 I | etcdmain: setting maximum number of CPUs to 4, total number
2019-02-13 02:53:28.144734 N | etcdmain: the server is already initialized as member before
2019-02-13 02:53:28.146651 I | etcdserver: name = master
...
WARNING: 2019/02/13 02:53:30 Failed to serve client requests on 127.0.0.1:2379
Failed to dial 127.0.0.1:2379: connection error: desc = "transport: authentication handshake"
```

For Clusters Using kubeadm

Since core components are deployed as pods, retrieve logs using:

- Running `kubectl logs <pod-name>` for pod-level logs.
- If the API server or etcd is down and `kubectl` is unresponsive, list all containers with:

```
bash theme={null} docker ps -a
```

Then inspect container logs:

```
bash theme={null} docker logs <container-id>
```

Additional Resources

For further details on certificate requirements and best practices, please refer to the official [Kubernetes Documentation](#) and explore additional resources available on the Kubernetes website.

That's it for this lesson. Head over to your practice environment and gain hands-on experience in viewing and verifying certificates in an existing Kubernetes cluster.

Default CN	Parent CA	O (in Subject)	kind	hosts (SAN)
kube-etcd	etcd-ca		server,client[!etcdbug]	localhost, 127.0.0.1
kube-etcd-peer	etcd-ca		server,client	<hostname>, <Host_IP>, localhost, 127.0.0.1
kube-etcd-healthcheck-client	etcd-ca		client	
kube-apiserver-etcd-client	etcd-ca	system:masters	client	
kube-apiserver	kubernetes-ca		server	<hostname>, <Host_IP>, <advertise_IP>, [1]
kube-apiserver-kubelet-client	kubernetes-ca	system:masters	client	
front-proxy-client	kubernetes-front-proxy-ca		client	

Default CN	recommended key path	recommended cert path	command	key argument	cert argument
etcd-ca		etcd/ca.crt	kube-apiserver		-etcd-cafile
etcd-client	apiserver-etcd-client.key	apiserver-etcd-client.crt	kube-apiserver	-etcd-keyfile	-etcd-certfile
kubernetes-ca		ca.crt	kube-apiserver		-client-ca-file
kube-apiserver	apiserver.key	apiserver.crt	kube-apiserver	-tls-private-key-file	-tls-cert-file
apiserver-kubelet-client		apiserver-kubelet-client.crt	kube-apiserver		-kubelet-client-certificate
front-proxy-ca		front-proxy-ca.crt	kube-apiserver		-requestheader-client-ca-file
front-proxy-client	front-proxy-client.key	front-proxy-client.crt	kube-apiserver	-proxy-client-key-file	-proxy-client-cert-file
etcd-ca		etcd/ca.crt	etcd		-trusted-ca-file, -peer-trusted-ca-file
kube-etcd	etcd/server.key	etcd/server.crt	etcd	-key-file	-cert-file
kube-etcd-peer	etcd/peer.key	etcd/peer.crt	etcd	-peer-key-file	-peer-cert-file
etcd-ca		etcd/ca.crt	etcdctl[2]		-cacert
kube-etcd-healthcheck-client	etcd/healthcheck-client.key	etcd/healthcheck-client.crt	etcdctl[2]	-key	-cert

Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

Certificates API

This article explains managing certificates and the Certificate API in Kubernetes, detailing the lifecycle of certificate signing requests and automation for certificate rotation.

This article explains how to manage certificates and introduces the Certificate API in Kubernetes. In a typical cluster setup, an administrator first configures a Certificate Authority (CA) server and generates certificates for the various components. Once the services are launched with these certificates, the cluster becomes operational. Initially, only one administrator has access via a personal admin certificate and key. However, as new team members join, each must obtain their own certificate and key pair to access the cluster.

The Certificate Lifecycle

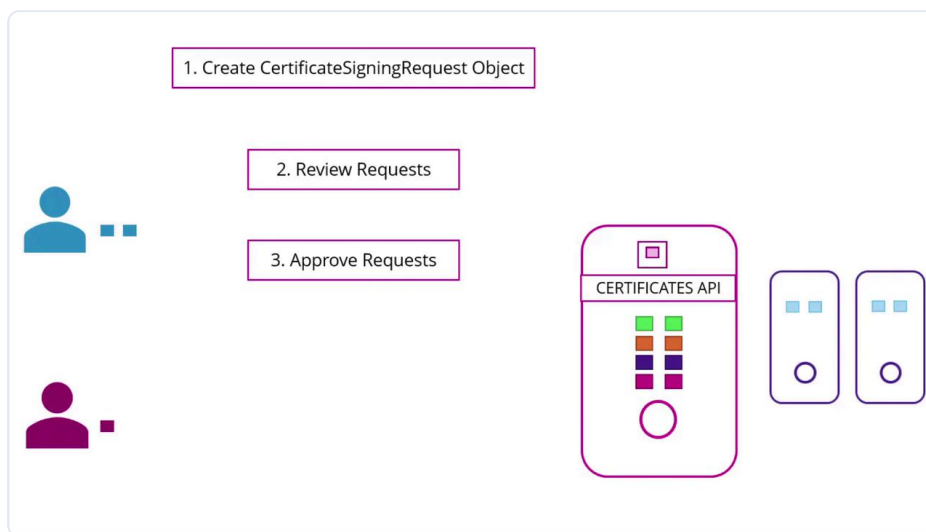
When a new administrator joins, she generates her own private key and creates a certificate signing request (CSR). This CSR is then sent to the existing administrator. As the sole administrator, you review the CSR, sign it using the CA server's private key and root certificate, and return the signed certificate. The new admin now has a valid certificate and key pair to access the cluster. Since certificates have a defined expiration period, the process is repeated when they expire.

The Kubernetes API Server plays a pivotal role in the cluster but is not part of the Certificate Authority. The CA is comprised of only two files—a key and a certificate—generated during initialization. Because these files allow the signing of certificates and thus the creation of users with any privileges, they must be stored securely, typically on a dedicated CA server. In many implementations, the Kubernetes master node also serves as the CA server. For instance, the kubeadm tool creates and stores CA files on the master node.

As the number of users increases, manually signing certificate requests becomes impractical. Kubernetes addresses this challenge with a built-in Certificates API that automates CSR management and certificate rotation.

Managing Certificate Signing Requests (CSRs)

The Kubernetes Certificates API allows users to submit their CSRs via an API call, creating a `CertificateSigningRequest` object. Administrators can then review and approve these requests using `kubectl` commands. Once approved, Kubernetes signs the certificate using the CA's key pair. The signed certificate is then available for extraction and distribution to the requesting user.



Step 1: User Generates Private Key and CSR

A user creates a private key and generates a certificate signing request using the following command:

```
```bash theme={null} openssl genrsa -out jane.key 2048
```

The user then sends the CSR to the administrator.

### ### Step 2: Administrator Creates a CSR Object

The administrator creates a `CertificateSigningRequest` object with a manifest file. In the manifest file, the administrator specifies the following details:

```

```yaml
theme: {null}
apiVersion: certificates.k8s.io/v1
kind: CertificateSigningRequest
metadata:
  name: jane
spec:
  expirationSeconds: 600 # seconds
  usages:
    - digital signature
    - key encipherment
    - server auth
  request: LS0tLS1CRUdJTiBDRVJUSUZJQ0FURSBRSRVFVRVNUUw0tLS0KTU11Q1dE
  ```

```

Administrators can list pending CSRs with the following command:

```

```bash
theme: {null}
kubectl get csr

```

The output may resemble:

```

```plaintext
theme: {null}
NAME AGE SIGNERNAME REQUESTOR REQUESTED
jane 10m kubernetes.io/kube-apiserver-client admin@example.com 10m

```

### Step 3: Approving the CSR

To approve the CSR, run:

```

```bash
theme: {null}
kubectl certificate approve jane

```

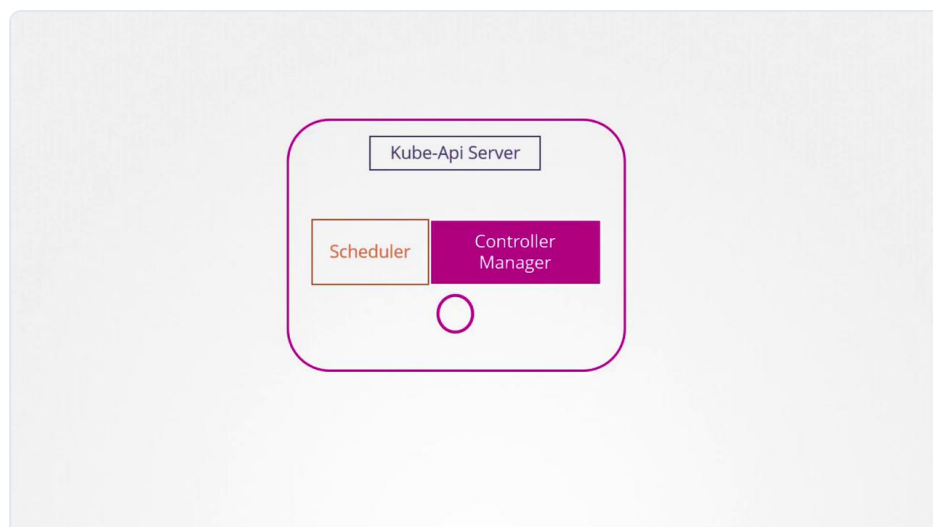
After approval, Kubernetes signs the CSR with the CA key pair, and the certificate is embedded in the pod's kubeletconfig.

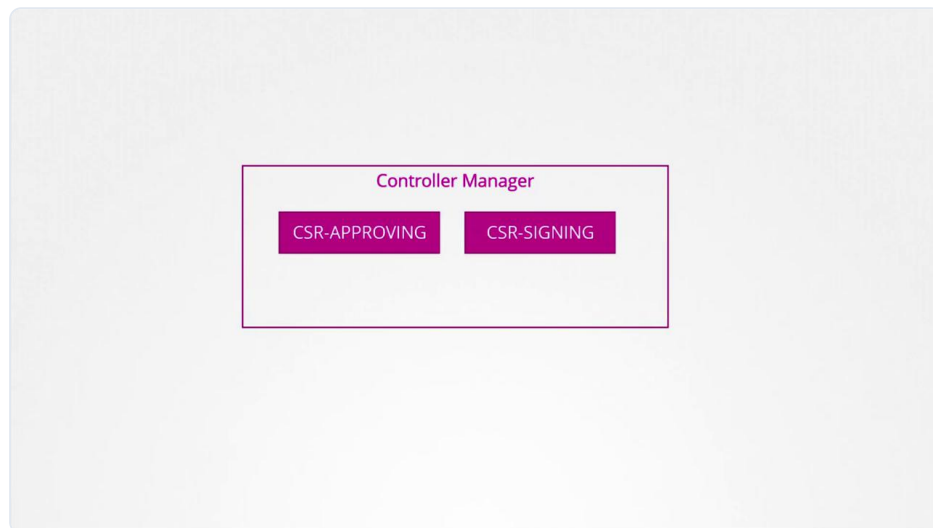
Below is an example output of a CertificateSigningRequest object:

```
```yaml theme={null}
apiVersion: certificates.k8s.io/v1
kind: CertificateSigningRequest
metadata:
 creationTimestamp: 2019-02-13T16:36:43Z
 name: new-user
spec:
 groups:
 - system:masters
 - system:authenticated
 expirationSeconds: 600
 usages:
 - digital signature
 - key encipherment
 - server auth
 username: kubernetes-admin
status:
 certificate: LS0tL1CRUdJTlBDRVJUSUZJQ0FURS9tL0t1SURDakNDQWZLZ0F3SjBZ9lVRmwyQ2wXYXoxa1w5M
 conditions:
 - lastUpdateTime: 2019-02-13T16:37:21Z
 message: This CSR was approved by kubectl certificate approve
 reason: KubectlApprove
 type: Approved
```

## The Role of the Controller Manager

Within the Kubernetes control plane, components such as the API Server, Scheduler, and Controller Manager work together. However, all certificate-related operations—such as CSR approval and signing—are managed by the Controller Manager.





The Controller Manager includes dedicated controllers for CSR approval and CSR signing tasks. Since signing certificates requires access to the CA's root certificate and private key, its configuration specifies the file paths to these credentials. For example, the Controller Manager's configuration file might include settings like the following:

```
yaml theme={null} cat /etc/kubernetes/manifests/kube-controller-manager.yaml spec: containers: - command: - kube-controller-manager - --address=127.0.0.1 - --cluster-signing-cert-file=/etc/kubernetes/pki/ca.crt - --cluster-signing-key-file=/etc/kubernetes/pki/ca.key - --controllers=*,bootstrapsigner,tokencleaner - --kubeconfig=/etc/kubernetes/controller-manager.conf - --leader-elect=true - --root-ca-file=/etc/kubernetes/pki/ca.crt - --service-account-private-key-file=/etc/kubernetes/pki/sa.key - --use-service-account-credentials=true
```

This configuration demonstrates how the Controller Manager accesses the necessary CA certificates for signing new certificate requests.

## Conclusion

In summary, Kubernetes simplifies certificate management through its built-in Certificates API, automating the CSR lifecycle and certificate rotation. This automation is essential for maintaining secure access as team sizes grow and certificates expire over time.

For further exploration, head over to the practice desk to experiment with the Certificates API and enhance your Kubernetes operations.

See you in the next article!

## Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

# KubeConfig

This article explains kubeconfig files in Kubernetes, focusing on certificate-based authentication and access management across multiple clusters.

Welcome to this lesson on kubeconfig files in Kubernetes. In this session, we will dive into certificate-based authentication using both curl and kubectl, and demonstrate how a kubeconfig file simplifies access management across multiple clusters.

## Certificate Authentication with curl and kubectl

Previously, we generated a certificate for a user and utilized the certificate along with a key to query the Kubernetes REST API for a list of pods. For instance, if your cluster is named "my kube playground," you can make a curl request to the API server as follows:

```
```bash theme={null} curl https://my-kube-playground:6443/api/v1/pods \ --key admin.key \ --cert admin.crt \ --cacert ca.crt
```

The API server then returns a response similar to this:

```
```json theme={null}
{
 "kind": "PodList",
 "apiVersion": "v1",
 "metadata": {
 "selfLink": "/api/v1/pods"
 },
 "items": []
}
```

Likewise, when using the kubectl command-line tool, you can supply the same parameters:

```
```bash theme={null} kubectl get pods \ --server https://my-kube-
playground:6443 \ --client-key admin.key \ --client-certificate admin.crt \ --
certificate-authority ca.crt
```

The response in this case might be:

```
```bash theme={null}
No resources found.
```

Instead of typing these options each time, streamline your workflow by moving them into a kubeconfig file.

## Understanding the Kubeconfig File

By default, kubectl searches for a kubeconfig file named "config" in the ~/.kube directory. Once properly set up, you can simply execute:

```
```bash theme={null} kubectl get pods
```

and kubectl will automatically use the configurations defined within the file.

The kubeconfig file is organized into three key sections:

- * **Clusters:** Define the Kubernetes clusters you need access to (e.g., development, production).
- * **Users:** Specify the user accounts and associated credentials (such as admin, dev, or prod).
- * **Contexts:** Link a cluster with a user by specifying which user should access which cluster.

Below is an example of a basic kubeconfig file in YAML format:

```
```yaml theme={null}
apiVersion: v1
kind: Config
clusters:
- name: my-kube-playground # values hidden...
- name: development
- name: production
- name: google
contexts:
- name: my-kube-admin@my-kube-playground
- name: dev-user@google
- name: prod-user@production
users:
- name: my-kube-admin
- name: admin
- name: dev-user
- name: prod-user
```

In this configuration, the server specification for the "my kube playground" cluster is defined in the clusters section, the admin user's credentials are listed in the users section, and the context named `my-kube-admin@my-kube-playground` ties them together. Multiple contexts can be created for different



clusters and users, and you can set a default context using the `current-context` field.

## Viewing and Customizing Your Kubeconfig

To view the current kubeconfig settings, run:

```
``bash theme={null} kubectl config view
```

This command outputs details about clusters, users, contexts, and the active context. An ex

```
``yaml theme={null}
apiVersion: v1
kind: Config
current-context: kubernetes-admin@kubernetes
clusters:
- cluster:
 certificate-authority-data: REDACTED
 server: https://172.17.0.5:6443
 name: kubernetes
contexts:
- context:
 cluster: kubernetes
 user: kubernetes-admin
 name: kubernetes-admin@kubernetes
users:
- name: kubernetes-admin
 user:
 client-certificate-data: REDACTED
 client-key-data: REDACTED
```

If you want to view a custom kubeconfig file, use the `--kubeconfig` option:

```
``bash theme={null} kubectl config view --kubeconfig=my-custom-config
```

A sample custom configuration may appear as follows:

```
``yaml theme={null}
apiVersion: v1
kind: Config
current-context: my-kube-admin@my-kube-playground
clusters:
- name: my-kube-playground
- name: development
- name: production
contexts:
- name: my-kube-admin@my-kube-playground
- name: prod-user@production
users:
- name: my-kube-admin
- name: prod-user
```

To change the active context—for example, switching from the admin user to the production user—execute:

```
``bash theme={null} kubectl config use-context prod-user@production
```

After running this command, the kubeconfig is updated accordingly. *The new configuration*

```
``yaml theme={null}
apiVersion: v1
kind: Config
current-context: prod-user@production
clusters:
- name: my-kube-playground
- name: development
- name: production
contexts:
- name: my-kube-admin@my-kube-playground
- name: prod-user@production
users:
- name: my-kube-admin
- name: prod-user
```

Additional kubectl config commands can be used to update or delete entries as needed.

## Configuring Default Namespaces

Namespaces in Kubernetes help segment clusters into multiple virtual clusters. You can configure a context to automatically use a specific namespace. Consider the following kubeconfig snippet without a default namespace:

```
``yaml theme={null} apiVersion: v1 kind: Config clusters: - name: production
cluster: certificate-authority: ca.crt server: https://172.17.0.51:6443 contexts: -
name: admin@production context: cluster: production user: admin users: -
name: admin user: client-certificate: admin.crt client-key: admin.key
```

To specify a default namespace (for example, "finance"), simply add the "namespace" field:

```
```yaml
theme: {null}
apiVersion: v1
kind: Config
clusters:
- name: production
  cluster:
    certificate-authority: ca.crt
    server: https://172.17.0.51:6443
contexts:
- name: admin@production
  context:
    cluster: production
    user: admin
    namespace: finance
users:
- name: admin
  user:
    client-certificate: admin.crt
    client-key: admin.key
```

When you switch to this context, kubectl will automatically operate within the specified namespace.

Managing Certificates in Kubeconfig Files

For best practices, use full paths for certificate files in your kubeconfig file. Alternatively, you can embed the certificate data directly using the `certificate-authority-data` field.

For instance, specifying a full path looks like this:

```
```yaml
theme: {null}
apiVersion: v1
kind: Config
clusters:
- name: production
 cluster:
 certificate-authority: /etc/kubernetes/pki/ca.crt
```

Alternatively, you may embed the certificate data directly:

```
```yaml
theme: {null}
apiVersion: v1
kind: Config
clusters:
- name: production
  cluster:
    certificate-authority: /etc/kubernetes/pki/ca.crt
    certificate-authority-data: LS0tLS1CRUdJTiBD...
```

To decode base64 encoded certificate data, use the following command:

```
```bash
theme: {null}
echo "LS0tLS1CRUdJTi BD..." | base64 --decode
```

The decoded output will resemble:

```
``text theme={null}
-----BEGIN CERTIFICATE-----
MIICDCCAuCAQAwE...
-----END CERTIFICATE-----
```

## Conclusion

---

This concludes our detailed exploration of kubeconfig files in Kubernetes. Use these best practices and examples to manage your clusters efficiently and troubleshoot any configuration issues you may encounter.

For further learning, explore the following resources:

- [Kubernetes Basics](#)
- [Kubernetes Documentation](#)
- [Docker Hub](#)
- [Terraform Registry](#)

Transcribed by <https://otter.ai>

## Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

# API Groups

This article provides an in-depth look into Kubernetes API groups, their structure, and methods for querying the API server.

Before diving into authorization, it's essential to understand the concept of API groups in Kubernetes and how they integrate with overall cluster operations. This article provides an in-depth look into Kubernetes API groups, their structure, and the methods for querying the API server.

## Understanding the Kubernetes API

The Kubernetes API is the primary interface for interacting with your cluster. Whether using the command-line tool `kubectl` or directly sending HTTP requests via REST, every interaction communicates with the API server. For example, to check your cluster's version, run:

```
```bash theme={null} curl https://kube-master:6443/version
```

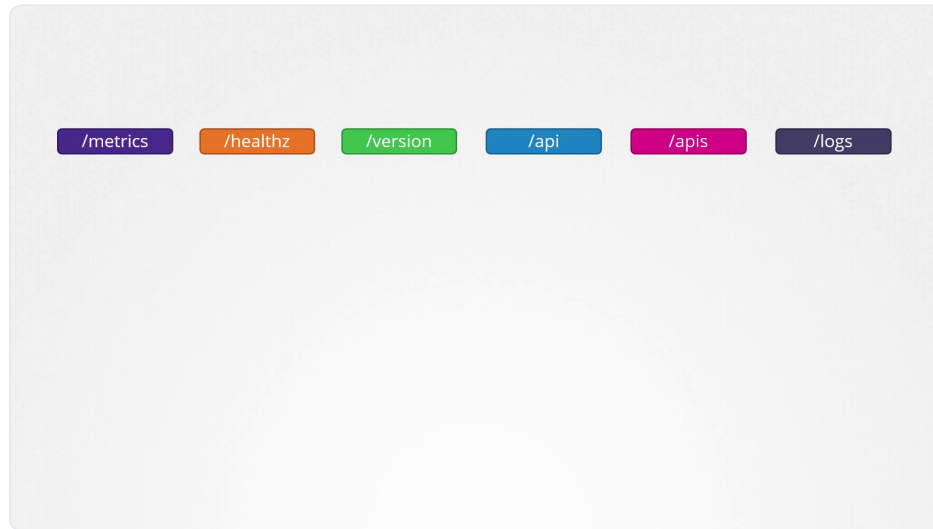
The response may look like:

```
```json theme={null}
{
 "major": "1",
 "minor": "13",
 "gitVersion": "v1.13.0",
 "gitCommit": "ddf47ac13c1a9483ea035a79cd7c1005ff21a6d",
 "gitTreeState": "clean",
 "buildDate": "2018-12-03T20:56:12Z",
 "goVersion": "go1.11.2",
 "compiler": "gc",
 "platform": "linux/amd64"
}
```

Likewise, listing pods in the cluster involves accessing the `/api/v1/pods` endpoint.

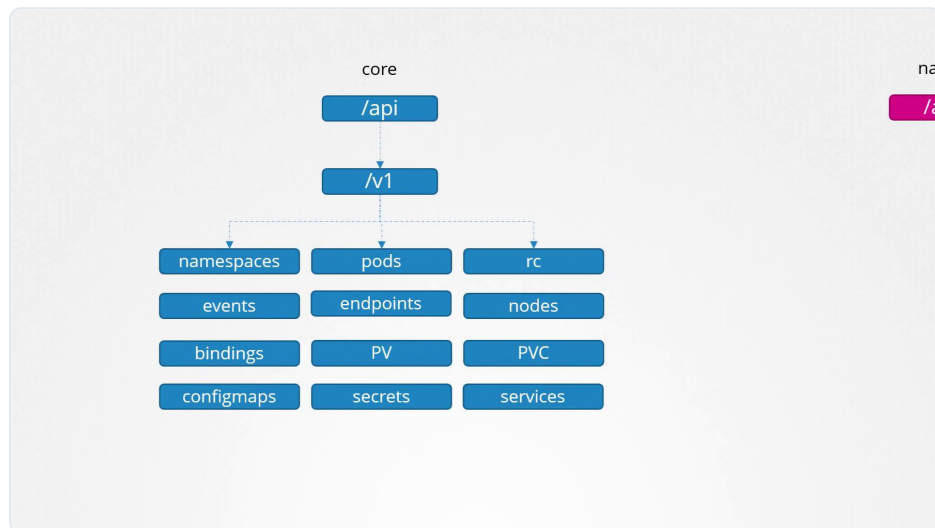
## API Groups and Their Purpose

Kubernetes organizes its API into multiple groups based on specific functionality. These groups help in managing versioning, health metrics, logging, and more. For instance, the `/version` endpoint provides cluster version data, while endpoints like `/metrics` and `/healthz` offer insights into the cluster's performance and health.

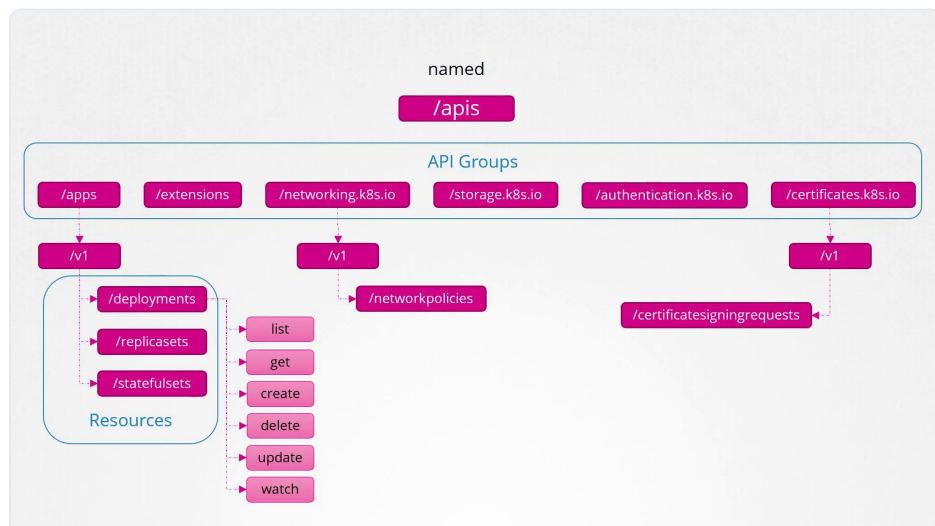


This article focuses on two main API group categories:

1. **Core API Group:** Contains the essential features of Kubernetes such as namespaces, pods, replication controllers, events, endpoints, nodes, bindings, persistent volumes, persistent volume claims, config maps, secrets, and services.
2. **Named API Groups:** Provides an organized structure for newer features. These groups include apps, extensions, networking, storage, authentication, and authorization. For example, under the apps group, you'll find Deployments, ReplicaSets, and StatefulSets, whereas the networking group hosts resources such as Network Policies. Certificate-related resources like Certificate Signing Requests are also grouped under their relevant named groups.



Every API group includes various resources along with associated actions (verbs) such as list, get, create, delete, update, and watch.



For detailed information on the objects in each API group, consult the official Kubernetes API reference documentation.

## Querying the API Server

To retrieve the list of available API groups, access the API server's root endpoint on port 6443:

```
``bash theme={null} curl http://localhost:6443 -k
```

The command returns a JSON response similar to:

```
```json theme={null}
{
  "paths": [
    "/api",
    "/api/v1",
    "/apis",
    "/apis/",
    "/healthz",
    "/logs",
    "/metrics",
    "/openapi/v2",
    "/swagger-2.0.0.json"
  ]
}
```

When using `curl` without proper authentication, only selected endpoints (like `/version`) may be accessible. Unauthenticated requests to protected endpoints will result in a 403 Forbidden error.

For example, an unauthenticated request may yield:

```
```bash theme={null} curl http://localhost:6443 -k
```

```
```json theme={null}
{
  "kind": "Status",
  "apiVersion": "v1",
  "metadata": {},
  "status": "Failure",
  "message": "forbidden: User \"system:anonymous\" cannot get path \"/\"",
  "reason": "Forbidden",
  "details": {},
  "code": 403
}
```

To fully access the API server, use your certificate files with `curl`:

```
```bash theme={null} curl http://localhost:6443 -k \ --key admin.key \ --cert
admin.crt \ --cacert
```

Alternatively, you can use the `kubectl proxy` command, which starts a local HTTP proxy server.

```
```bash theme={null}
kubectl proxy
```

The output confirms the proxy is running:

```
```bash theme={null} kubectl proxy Starting to serve on 127.0.0.1:8001
```



Now, you can access the API server through the proxy:

```
```bash theme={null}
curl http://localhost:8001 -k
```

The typical response should be:

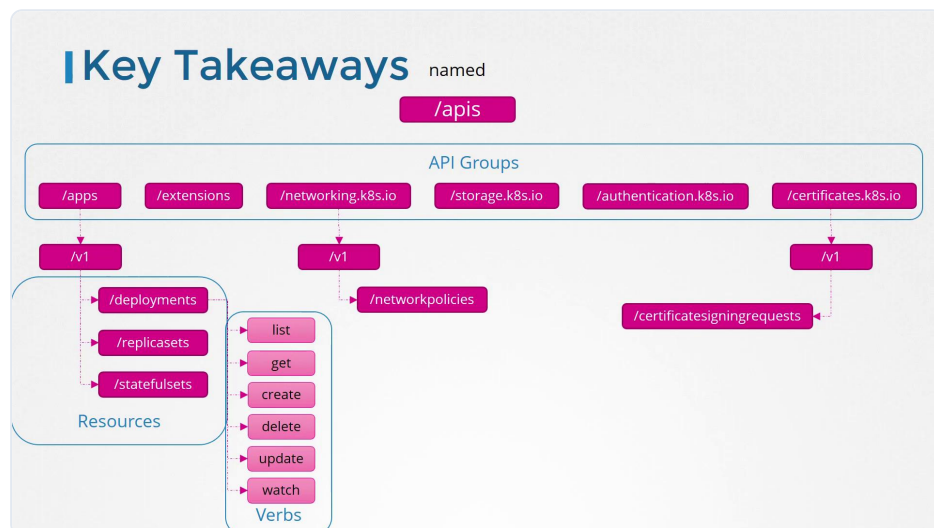
```
json theme={null} { "paths": [ "/api/", "/api/v1", "/apis", "/
apis/", "/healthz", "/logs", "/metrics", "/openapi/v2", "/
swagger-2.0.0.json" ] }
```

Remember that "kube proxy" and "kubectl proxy" serve different purposes. The former facilitates pod-to-pod and service communication within the cluster, while the latter is a local HTTP proxy for accessing the API server.

Kube proxy  Kubectl proxy

Summary

Kubernetes organizes its resources into various API groups. At the highest level, there is the core API group coupled with multiple named API groups, each containing specific resources and actions. The diagram below outlines the hierarchical structure and relationships between API groups, resources, and associated verbs:



In the next section on authorization, we'll explore how these API groups and their associated actions control access to cluster resources.

That's it for this lesson. See you in the next article!

Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

Authorization

This article explores authentication and authorization in Kubernetes, detailing user access, permissions, and various authorization mechanisms like RBAC and external tools.

In this lesson, we explore how authentication allows individuals or machines to gain access to a cluster and how authorization subsequently defines what actions they can perform within that cluster. Once a user gains access, authorization ensures they only have the appropriate permissions for their role. For example, a cluster administrator can view various objects such as Pods, Nodes, and Deployments:

```
```bash theme={null} kubectl get pods NAME READY STATUS RESTARTS AGE
nginx 1/1 Running 0 53s
```

```
kubectl get nodes NAME STATUS ROLES AGE VERSION
worker-1 Ready 5d21h v1.13.0
worker-2 Ready 5d21h v1.13.0
```

kubec

Administrators have full control, allowing them to create or delete objects like Pods or No

Below is an example demonstrating operations executed with limited permissions:

```
```plaintext theme={null}
kubectl get pods
NAME      READY   STATUS    RESTARTS   AGE
nginx     1/1     Running   0           53s

kubectl get nodes
NAME      STATUS   ROLES    AGE    VERSION
worker-1  Ready    <none>   5d21h  v1.13.0
worker-2  Ready    <none>   5d21h  v1.13.0

kubectl delete node worker-2
Node worker-2 Deleted!
```

In contrast, attempting similar operations without sufficient privileges results in the following responses:

```
```bash theme={null} kubectl get pods Error from server (Forbidden): pods is forbidden: User "Bot-1" cannot list "pods"
```

```
kubectl get nodes Error from server (Forbidden): nodes is forbidden: User "Bot-1" cannot get "nodes"
```

```
kubectl delete node worker-2 Error from server (Forbidden): nodes "worker-2" is forbidden: User "developer" cannot delete resource "nodes"
```

When sharing a cluster across different organizations or teams using namespaces, authorization

- \* Node Authorization
- \* Attribute-Based Authorization
- \* Role-Based Access Control (RBAC)
- \* Webhook Authorization

The Kubernetes API Server is the central component accessed by both management users and in

<Frame>

!The image illustrates a Kubernetes node interaction, showing communication between a us

</Frame>

Requests from kubelets—typically using certificates with names prefixed by "system:node" as

<Frame>

!The image illustrates the Node Authorizer process in Kubernetes, showing interactions b

</Frame>

<Callout icon="lightbulb" color="#1CB2FE">

Kubernetes supports several authorization strategies to meet diverse security requirements

</Callout>

## ## Attribute-Based Authorization

Attribute-based authorization associates specific users or groups with a defined set of per

```
```json theme={null}
```

```
{ "kind": "Policy", "spec": { "user": "dev-user", "namespace": "*", "resource": "pods", "api"
```

```
{ "kind": "Policy", "spec": { "user": "dev-user-2", "namespace": "*", "resource": "pods", "api"
```

```
{ "kind": "Policy", "spec": { "group": "dev-users", "namespace": "*", "resource": "pods", "api"
```

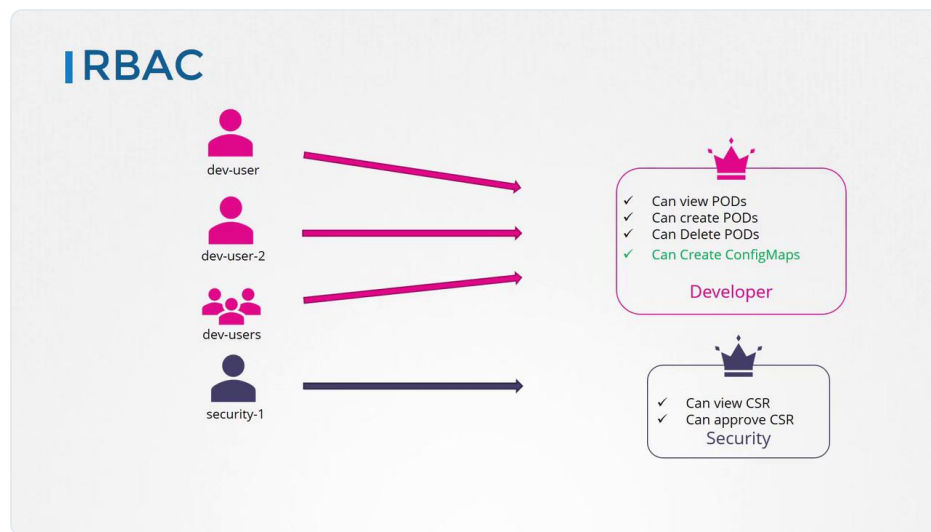
```
{ "kind": "Policy", "spec": { "user": "security-1", "namespace": "*", "resource": "csr", "api"
```

Each time security requirements change, you must manually update this policy file and restart the Kube API Server. This manual process can be tedious and set the stage for more streamlined methods such as Role-Based Access Control (RBAC).

Role-Based Access Control (RBAC)

RBAC simplifies user permission management by defining roles instead of directly associating permissions with individual users. For example, you can create a "developer" role that encompasses only the necessary permissions for application deployment. Developers are then associated with this role, and modifications in user access can be handled by updating the role, affecting all associated users immediately.

RBAC is considered the standard method for managing access within a Kubernetes cluster. The diagram below provides a visual representation of RBAC across different roles:



Further details on RBAC will be discussed in upcoming lessons.

External Authorization Mechanisms

If you prefer managing authorization externally rather than with built-in Kubernetes mechanisms, third-party tools like [Open Policy Agent \(OPA\)](#) are an excellent choice. OPA can handle both admission control and authorization by processing user details and access requirements sent via API calls from Kubernetes. Based on OPA's response, access is either granted or denied.

AlwaysAllow and AlwaysDeny Modes

Kubernetes also supports two basic authorization modes:

- **AlwaysAllow:** Permits all requests without performing any authorization checks.
- **AlwaysDeny:** Denies all requests.

These modes are configured using the `authorization-mode` option on the Kube API Server and are crucial when determining which authorization mechanism is active. In cases where no mode is specified, `AlwaysAllow` is used by default.

Below is an example configuration using `AlwaysAllow`:

```
```bash theme={null} ExecStart=/usr/local/bin/kube-apiserver \
--advertise-address=${INTERNAL_IP} \
--allow-privileged=true \
--apiserver-count=3 \
--authorization-mode=AlwaysAllow \
--bind-address=0.0.0.0 \
--enable-swagger-ui=true \
--etcd-cafile=/var/lib/kubernetes/ca.pem \
--etcd-certfile=/var/lib/kubernetes/apiserver-etcd-client.crt \
--etcd-keyfile=/var/lib/kubernetes/apiserver-etcd-client.key \
--etcd-servers=https://127.0.0.1:2379 \
--event-ttl=1h \
--kubelet-certificate-authority=/var/lib/kubernetes/ca.pem \
--kubelet-client-certificate=/var/lib/kubernetes/apiserver-etcd-client.crt \
--kubelet-client-key=/var/lib/kubernetes/apiserver-etcd-client.key \
--service-node-port-range=30000-32767 \
--client-ca-file=/var/lib/kubernetes/ca.pem \
--tls-cert-file=/var/lib/kubernetes/apiserver.crt \
--tls-private-key-file=/var/lib/kubernetes/apiserver.key \
-v=2
```

You can also specify a comma-separated list of multiple authorization modes. For example, t

```
```bash theme={null}
ExecStart=/usr/local/bin/kube-apiserver \
--advertise-address=${INTERNAL_IP} \
--allow-privileged=true \
--apiserver-count=3 \
--authorization-mode=Node,RBAC,Webhook \
--bind-address=0.0.0.0 \
--enable-swagger-ui=true \
--etcd-cafile=/var/lib/kubernetes/ca.pem \
--etcd-certfile=/var/lib/kubernetes/apiserver-etcd-client.crt \
--etcd-keyfile=/var/lib/kubernetes/apiserver-etcd-client.key \
--etcd-servers=https://127.0.0.1:2379 \
--event-ttl=1h \
--kubelet-certificate-authority=/var/lib/kubernetes/ca.crt \
--tls-cert-file=/var/lib/kubernetes/apiserver.crt \
--tls-private-key-file=/var/lib/kubernetes/apiserver.key \
-v=2
```

When multiple modes are configured, each request is processed sequentially in the order specified. For example, a user's request is first evaluated by the node authorizer. If the request does not pertain to node-specific actions and is consequently denied, it is then passed to the next module, such as RBAC. Once a module approves the request, further checks are bypassed and the user is granted access.

This lesson provided an in-depth look at authorization in Kubernetes—from basic attribute-based policies to advanced RBAC and external mechanisms. Choosing the right authorization methods is essential for securing your cluster while ensuring users have only the permissions they need.

That concludes this lesson on authorization. Stay tuned for further exploration of role-based access controls and other advanced authorization mechanisms.

Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

Role Based Access Controls

This article explains how to implement Role-Based Access Controls in Kubernetes, including creating roles, role bindings, and verifying permissions.

In this lesson, we dive into Kubernetes Role-Based Access Controls (RBAC) to help you manage permissions effectively. You'll learn how to create roles, bind them to users, and verify permissions within a namespace.

Creating a Role

To define a role, create a YAML file that sets the API version to `rbac.authorization.k8s.io/v1` and the kind to `Role`. In this example, we create a role named **developer** to grant developers specific permissions. The role includes a list of rules where each rule specifies the API groups, resources, and allowed verbs. For resources in the core API group, provide an empty string (`"`) for the `apiGroups` field.

For instance, the following YAML definition grants developers permissions on pods (with various actions) and allows them to create ConfigMaps:

```
```yaml theme={null} apiVersion: rbac.authorization.k8s.io/v1 kind: Role
metadata: name: developer rules: - apiGroups: [""] resources: ["pods"] verbs:
["list", "get", "create", "update", "delete"] - apiGroups: [""] resources:
["ConfigMap"] verbs: ["create"]
```

Create the role by running:

```
```bash theme={null}
kubectl create -f developer-role.yaml
```

Both roles and role bindings are namespace-scoped. This example assumes usage within the default namespace. To manage access in a different namespace, update the YAML metadata accordingly.

Creating a Role Binding

After defining a role, you need to bind it to a user. A role binding links a user to a role within a specific namespace. In this example, we create a role binding named **devuser-developer-binding** that grants the user `dev-user` the **developer** role.

Below is the combined YAML definition for both creating the role and its corresponding binding:

```
```yaml theme={null} apiVersion: rbac.authorization.k8s.io/v1 kind: Role
metadata: name: developer rules: - apiGroups: [""] resources: ["pods"] verbs:
["list", "get", "create", "update", "delete"] - apiGroups: [""] resources:
["ConfigMap"] verbs: ["create"]
```

```
apiVersion: rbac.authorization.k8s.io/v1 kind: RoleBinding metadata: name:
devuser-developer-binding subjects: - kind: User name: dev-user apiGroup:
rbac.authorization.k8s.io roleRef: kind: Role name: developer apiGroup:
rbac.authorization.k8s.io
```

Create the role binding using the command:

```
```bash theme={null}
kubectl create -f devuser-developer-binding.yaml
```

Verifying Roles and Role Bindings

After applying your configurations, it's important to verify that the roles and role bindings have been created correctly.

To list all roles in the current namespace, execute:

```
```bash theme={null} kubectl get roles
```

Example output:

```
```bash theme={null}
NAME      AGE
developer 4s
```

Next, list all role bindings:

```
```bash theme={null} kubectl get rolebindings
```

Example output:

```
```bash theme={null}
NAME                                AGE
devuser-developer-binding 24s
```

For detailed information about the **developer** role, run:

```
``bash theme={null} kubectl describe role developer
```

Sample output:

```
``bash theme={null}
Name:          developer
Labels:        <none>
Annotations:   <none>
PolicyRule:
  Resources            Non-Resource URLs   Resource Names       Verbs
  -----
  ConfigMap            []                  []                  [create delete]
  pods                 []                  []                  [get watch list create delete]
```

To view the specifics of the role binding:

```
``bash theme={null} kubectl describe rolebinding devuser-developer-binding
```

Example output:

```
``bash theme={null}
Name:          devuser-developer-binding
Labels:        <none>
Annotations:   <none>
Role:
  Kind:    Role
  Name:    developer
Subjects:
  Kind      Name      Namespace
  ----      -
  User      dev-user
```

Testing Permissions with kubectl auth

You can test whether you have the necessary permissions to perform specific actions by using the `kubectl auth can-i` command. For example, to check if you can create deployments, run:

```
``bash theme={null} kubectl auth can-i create deployments
```

This command might return:

```
``bash theme={null}
yes
```

Similarly, to verify if you can delete nodes:

```
``bash theme={null} kubectl auth can-i delete nodes
```

Expected output:

```
```bash theme={null}
no
```

To test permissions for a specific user without switching user contexts, use the `--as` flag. Although the **developer** role does not permit creating deployments, it does allow creating pods:

```
```bash theme={null} kubectl auth can-i create deployments
```

Output: yes

`kubectl auth can-i delete nodes`

Output: no

`kubectl auth can-i create deployments --as dev-user`

Output: no

`kubectl auth can-i create pods --as dev-user`

Output: yes

You can also specify a namespace in your commands to verify permissions scoped to that part

Limiting Access to Specific Resources

In some scenarios, you may want to restrict user access to a select group of resources. For

Start with a basic role definition without any resource-specific restrictions:

```
```yaml theme={null}
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
 name: developer
rules:
- apiGroups: [""]
 resources: ["pods"]
 verbs: ["get", "create", "update"]
```

Then, update the rule to restrict access solely to the "blue" and "orange" pods:

```
yaml theme={null} apiVersion: rbac.authorization.k8s.io/v1 kind:
Role metadata: name: developer rules: - apiGroups: [""] resources:
```

```
["pods"] verbs: ["get", "create", "update"] resourceNames: ["blue",
"orange"]
```

## Conclusion

---

This lesson provided an in-depth look at implementing Role-Based Access Controls in Kubernetes. You learned how to create roles and role bindings, verify permissions, and restrict access to specific resources. Practicing these exercises will enhance your grasp of RBAC and help you manage Kubernetes security effectively.

For additional details on Kubernetes RBAC, refer to the [Kubernetes Documentation](#) and explore best practices for securing your clusters.

## Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

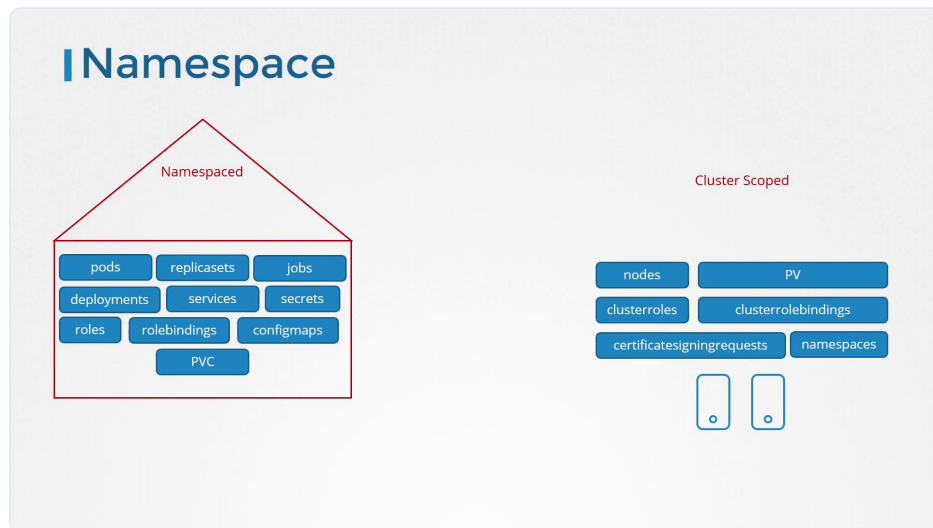
# Cluster Roles

This article introduces cluster roles and bindings for managing permissions across a Kubernetes cluster, distinguishing between namespaced and cluster-scoped resources.

In our previous discussion, we covered roles and role bindings, which are namespace-specific. In this article, we extend that concept by introducing cluster roles and cluster role bindings, allowing you to manage permissions across your entire Kubernetes cluster.

When you create roles and role bindings without specifying a namespace, they are added to the default namespace and only authorize access within that scope. This works well for namespaced resources—such as pods, deployments, and services—but not for cluster-scoped resources. For example, nodes cannot be assigned to a specific namespace (e.g., "node01" cannot belong to the "dev" namespace). Cluster-scoped resources like nodes and persistent volumes are managed at the cluster level, so they require a different approach.

Most resources (such as pods, replica sets, jobs, deployments, services, and secrets) are namespaced. In contrast, cluster-scoped resources, including nodes and persistent volumes, do not belong to any namespace. The following image clearly illustrates the difference between namespaced resources (e.g., pods, services) and cluster-scoped resources (e.g., nodes, cluster roles):



To list namespaced and non-namespaced resources, you can use these commands:

```
``bash theme={null} kubectl api-resources --namespaced=true kubectl api-resources --namespaced=false
```

<Callout icon="lightbulb" color="#1CB2FE">

Remember: Roles and role bindings are ideal for namespace-level access, while cluster roles are ideal for cluster-level access.

## ## Cluster Roles and Cluster Role Bindings

To authorize cluster-scoped resources, such as nodes and persistent volumes, you need to create a cluster role.

For example, you can define a cluster administrator role that grants the ability to list, read, and delete all resources in the cluster.

Below is an example of a cluster role definition file named `cluster-admin-role.yaml`. This file defines a cluster role that grants permissions to all resources in the cluster.

```
``yaml theme={null}
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
 name: cluster-administrator
rules:
- apiGroups: [""]
 resources: ["nodes"]
 verbs: ["list", "get", "create", "delete"]
```

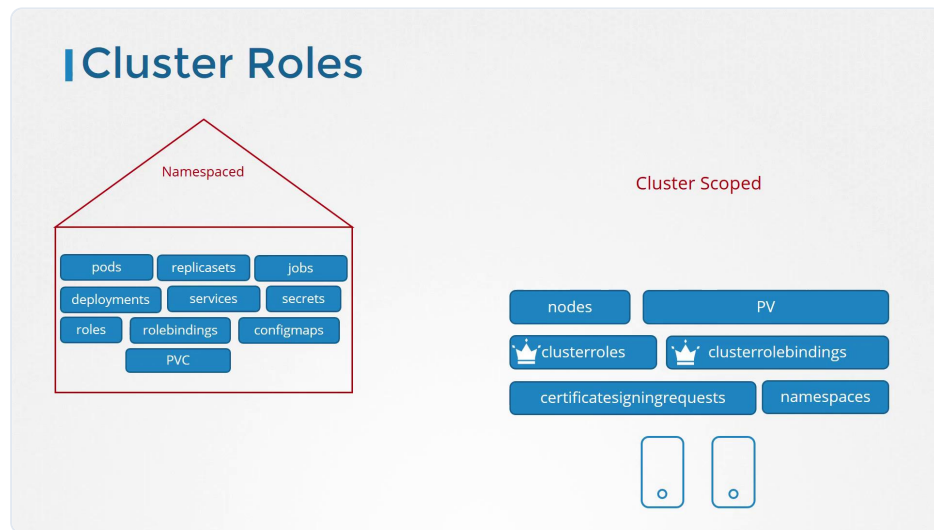
Once the ClusterRole is created, you bind it to a user through a ClusterRoleBinding object. The following example binds the `cluster-administrator` ClusterRole to a user named `cluster-admin`:

```
yaml theme={null} apiVersion: rbac.authorization.k8s.io/v1 kind:
ClusterRoleBinding metadata: name: cluster-admin-role-binding
subjects: - kind: User name: cluster-admin apiGroup:
rbac.authorization.k8s.io roleRef: kind: ClusterRole name: cluster-
administrator apiGroup: rbac.authorization.k8s.io
```

Apply these configurations using the `kubectl create` command.

It's important to note that while cluster roles and role bindings are primarily used for cluster-scoped resources, they can also manage access to namespace-scoped resources. When you bind a cluster role that grants permissions on pods, for instance, the user will have access to pods in every namespace—unlike a namespaced role which restricts access to a single namespace.

The following image further distinguishes how cluster roles work for both namespaced and cluster-scoped resources:



Kubernetes provides several default cluster roles when the cluster is initially set up. Be sure to review these defaults to understand the baseline permissions before creating custom roles.

Good luck with managing your cluster roles and enhancing your Kubernetes RBAC!

## Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

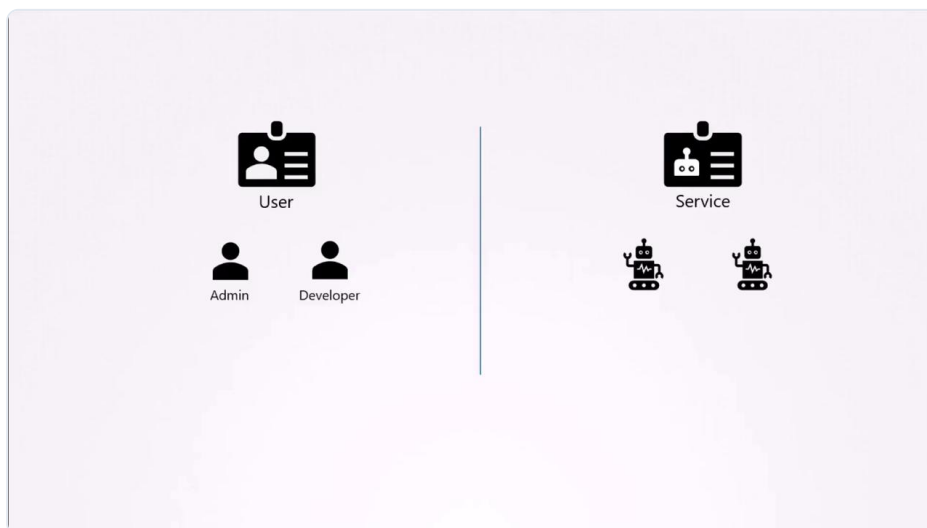
# Service Accounts

This guide explains Kubernetes service accounts, their creation, usage, and security enhancements in recent versions.

Welcome to this comprehensive guide on Kubernetes service accounts. In this lesson, we focus on how service accounts function—primarily from an application developer's perspective. While advanced security topics such as authentication, authorization, and role-based access control (RBAC) are covered extensively in the [Kubernetes Administrators Course](#), this guide is dedicated solely to working with service accounts.

There are two main types of accounts in Kubernetes:

- **User Accounts:** Designed for human users like administrators or developers.
- **Service Accounts:** Intended for machine-to-machine interactions or application-specific tasks. For instance, monitoring tools like Prometheus use a service account to query the Kubernetes API for performance metrics, while Jenkins uses one for deploying applications.





## Example: A Kubernetes Dashboard Application

Consider an example: "my Kubernetes dashboard," a basic dashboard application built with Python. This application retrieves a list of Pods from a Kubernetes cluster by sending API requests and subsequently displays the results on a web page. To authenticate its API requests, the application uses a dedicated service account.



### Creating a Service Account

To create a service account named `dashboard-sa`, run:

```
``bash theme={null} kubectl create serviceaccount dashboard-sa
```

To view all service accounts, use:

```
``bash theme={null}
kubectl get serviceaccount
```

The output will appear similar to:

```
``bash theme={null} NAME SECRETS AGE default 1 218d dashboard-sa 1 4d
```

Upon creation, Kubernetes automatically generates a service account token stored as a Secret.

```
``bash theme={null}
kubectl describe serviceaccount dashboard-sa
```

Expected output:

```
``bash theme={null} Name: dashboard-sa Namespace: default Labels:
Annotations: Image pull secrets: Mountable secrets: dashboard-sa-token-
kbbdm Tokens: dashboard-sa-token-kbbdm Events:
```

To examine the token itself, view the corresponding Secret:

```
```bash theme={null}
kubectl describe secret dashboard-sa-token-kbbdm
```

Sample output:

```
```bash theme={null} Name: dashboard-sa-token-kbbdm Namespace: default
Labels: Type: kubernetes.io/service-account-token Data token:
eyJhbGciOiJIUzI1NiIsImtpZCI6Ij... (truncated for privacy)
```

This token serves as the authentication bearer token for accessing the Kubernetes API. For

```
```bash theme={null}
curl https://192.168.56.70:6443/api -k \
--header "Authorization: Bearer eyJhbG..."
```

In your custom dashboard application, you would typically place the token into the appropriate configuration field to enable API authentication.

Automatic Mounting of Service Account Tokens

When deploying third-party applications (such as a custom dashboard or Prometheus) on a Kubernetes cluster, you can have Kubernetes automatically mount the service account token as a volume into the Pod. This token is typically available at the path: `/var/run/secrets/kubernetes.io/serviceaccount`.



Every namespace includes a default service account that is automatically injected into Pods. For example, consider the following simple Pod manifest using a custom dashboard image:

```
```yaml theme={null} apiVersion: v1 kind: Pod metadata: name: my-kubernetes-
dashboard spec: containers: - name: my-kubernetes-dashboard image: my-
kubernetes-dashboard
```

After creating this Pod, running:

```
```bash theme={null}
kubectl describe pod my-kubernetes-dashboard
```

will reveal a volume mounted from a Secret (usually named something like `default-token-xxxx`). You might see an excerpt similar to:

```
```bash theme={null} Name: my-kubernetes-dashboard Namespace: default
Status: Running IP: 10.244.0.15 Containers: nginx: Image: my-kubernetes-
dashboard Mounts: /var/run/secrets/kubernetes.io/serviceaccount from default-
token-j4hkv (ro) Volumes: default-token-j4hkv: Type: Secret (a volume
populated by a Secret) SecretName: default-token-j4hkv Optional: false
```

Inside the Pod, listing the contents of the service account directory shows files such as t

```
```bash theme={null}
kubectl exec -it my-kubernetes-dashboard -- ls /var/run/secrets/kubernetes.io/serviceaccount
kubectl exec -it my-kubernetes-dashboard -- cat /var/run/secrets/kubernetes.io/serviceaccount
```

Using a Different Service Account

By default, Pods use the `default` service account. To assign a different service account—like the previously created `dashboard-sa`—update your Pod definition to include the `serviceAccountName` field:

```
```yaml theme={null} apiVersion: v1 kind: Pod metadata: name: my-kubernetes-
dashboard spec: serviceAccountName: dashboard-sa containers: - name: my-
kubernetes-dashboard image: my-kubernetes-dashboard
```

<Callout icon="lightbulb" color="#1CB2FE">

Remember that you cannot modify the service account of an existing Pod. To use a new se

</Callout>

After deploying the updated manifest, running:

```
```bash theme={null}
kubectl describe pod my-kubernetes-dashboard
```

will show that the new service account is now in effect, with volume mounting information reflecting the token for `dashboard-sa` (e.g., `dashboard-sa-token-kbbdm`).

If you wish to disable the automatic mounting of the service account token, set `automountServiceAccountToken` to `false` in the Pod specification:

```
```yaml theme={null} apiVersion: v1 kind: Pod metadata: name: my-kubernetes-
dashboard spec: automountServiceAccountToken: false containers: - name:
my-kubernetes-dashboard image: my-kubernetes-dashboard
```

#### ## Changes in Kubernetes Versions 1.22 and 1.24

Prior to Kubernetes v1.22, service account tokens were automatically mounted from Secrets.

Below is an example Pod definition using a projected volume sourced from the TokenRequest API.

```
```yaml theme={null}
apiVersion: v1
kind: Pod
metadata:
  name: nginx
  namespace: default
spec:
  containers:
    - image: nginx
      name: nginx
      volumeMounts:
        - mountPath: /var/run/secrets/kubernetes.io/serviceaccount
          name: kube-api-access-6mtg8
          readOnly: true
  volumes:
    - name: kube-api-access-6mtg8
      projected:
        defaultMode: 420
        sources:
          - serviceAccountToken:
              expirationSeconds: 3607
              path: token
          - configMap:
              name: kube-root-ca.crt
              items:
                - key: ca.crt
                  path: ca.crt
          - downwardAPI:
              items:
                - fieldRef:
                    apiVersion: v1
                    fieldPath: metadata.namespace
```

Starting with Kubernetes v1.24, Kubernetes no longer automatically creates non-expiring service account tokens stored as Secrets. Instead, after creating a new service account, you must generate a token explicitly with:

```
```bash theme={null} kubectl create token dashboard-sa
```

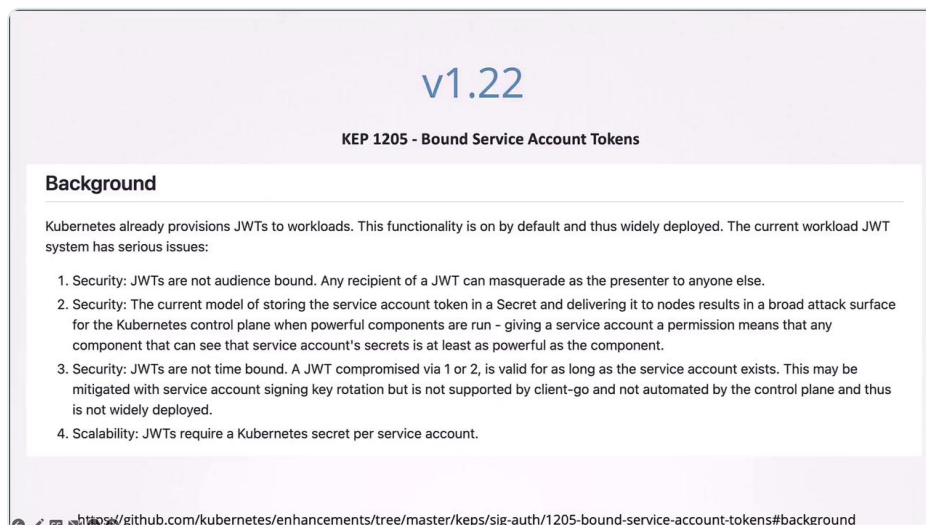
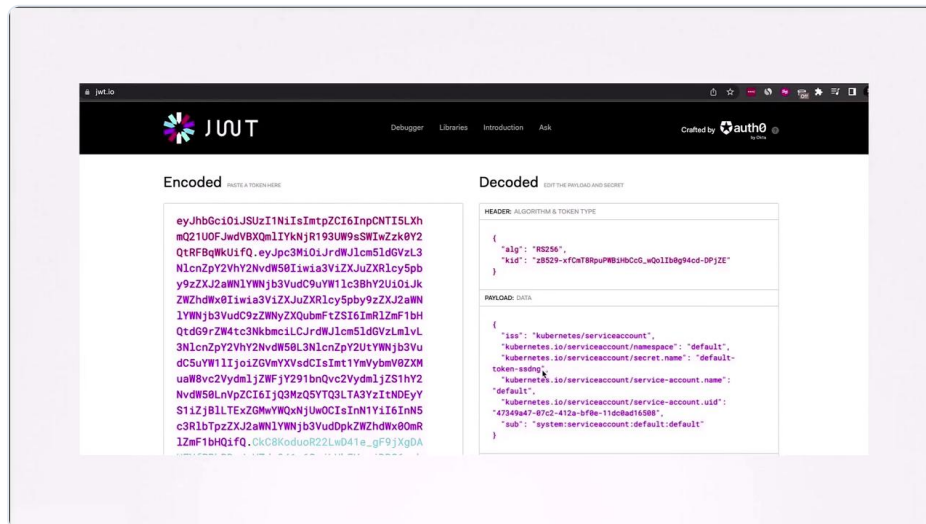
This command produces a token with an expiry (by default, one hour from creation). You can

```
```bash theme={null}
jq -R 'split(".") | select(length > 0) | .[0] | @base64 | fromjson' <<<< <TOKEN>
```

If necessary (though not recommended), you can still create a non-expiring token by manually creating a Secret. Ensure the service account exists first:

```
yaml theme={null} apiVersion: v1 kind: Secret metadata: name:
mysecretname annotations: kubernetes.io/service-account.name:
dashboard-sa type: kubernetes.io/service-account-token
```

It is highly recommended to use the TokenRequest API to generate tokens, as API-generated tokens provide additional security features such as expiry, audience restrictions, and improved manageability.



v1.22

KEP 1205 - Bound Service Account Tokens



TokenRequestAPI

- ✓ Audience Bound
- ✓ Time Bound

<https://github.com/kubernetes/enhancements/tree/master/keps/sig-auth/1205-bound-service-account-tokens#background>

Service account token Secrets

A `kubernetes.io/service-account-token` type of Secret is used to store a token credential that identifies a service account.

Since 1.22, this type of Secret is no longer used to mount credentials into Pods, and obtaining tokens via the `TokenRequest` API is recommended instead of using service account token Secret objects. Tokens obtained from the `TokenRequest` API are more secure than ones stored in Secret objects, because they have a bounded lifetime and are not readable by other API clients. You can use the `kubectl create token` command to obtain a token from the `TokenRequest` API.

You should only create a service account token Secret object if you can't use the `TokenRequest` API to obtain a token, and the security exposure of persisting a non-expiring token credential in a readable API object is acceptable to you.

<https://kubernetes.io/docs/concepts/configuration/secret/#service-account-token-secrets>

Summary

- **Service Accounts vs. User Accounts:** Service accounts are meant for applications (or machines), whereas user accounts are for human users.
- **Token Generation:** Creating a service account automatically generates a token stored in a Secret, which is used for API authentication.
- **Automatic Token Mounting:** Pods can automatically mount the service account token at `/var/run/secrets/kubernetes.io/serviceaccount`, though this behavior can be modified.
- **Enhanced Security:** Since Kubernetes v1.22, tokens are generated using the `TokenRequest` API, making them audience-bound, time-bound, and more secure.
- **Kubernetes v1.24 Changes:** With v1.24, Kubernetes no longer provisions non-expiring tokens automatically via Secrets; use the `kubectl create token` command to generate tokens as needed.

For additional insights, refer to the Kubernetes enhancement proposals and the official documentation on service accounts and tokens.

References

KEP 1205 - Bound Service Account Tokens

<https://github.com/kubernetes/enhancements/tree/master/keps/sig-auth/1205-bound-service-account-tokens>

KEP-2799: Reduction of Secret-based Service Account Tokens

<https://github.com/kubernetes/enhancements/tree/master/keps/sig-auth/2799-reduction-of-secret-based-service-account-token>

<https://kubernetes.io/docs/concepts/configuration/secret/#service-account-token-secrets>
<https://kubernetes.io/docs/reference/access-authn-authz/service-accounts-admin/>

Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

Image Security

This article covers best practices for securing container images during deployment, including naming conventions, private registries, and Kubernetes configuration.

In this lesson, we explore best practices for securing container images throughout the deployment process. You will learn about image naming conventions, securing image repositories, and configuring your pods to pull images from trusted sources. We will illustrate these concepts using several pod examples that deploy various applications such as web apps, databases, and Redis caches.

Understanding Container Image Naming

Let's start by examining a simple pod definition file that deploys an Nginx container:

```
```yaml theme={null} apiVersion: v1 kind: Pod metadata: name: nginx-pod spec: containers: - name: nginx image: nginx
```

Notice the image name "nginx". This follows Docker's image naming convention. When a repository

If you create your own account and build custom images, you should update the image name accordingly.

```
```yaml theme={null} image: your-account/nginx
```

By default, Docker pulls images from Docker Hub (with the DNS name docker.io) if no other registry is specified. The registry is a centralized storage where images are pushed during creation or updates, and subsequently pulled during deployment.

Private Registry Usage

For projects that require enhanced security and privacy, you might opt for private registries. Many popular cloud service providers—such as AWS, Azure, and GCP—offer private registries built into their platforms. Alternatively, tools like [Google Container Registry](https://gcr.io) (gcr.io) are frequently used for Kubernetes-related images and testing purposes.

When referencing an image from a private registry, the full image path should be specified. For example:

```
```yaml theme={null} image: docker.io/library/nginx
```

### ### Authentication for Private Registries

Accessing private repositories requires prior authentication. Start by logging into your private registry.

```
```bash theme={null}
docker login private-registry.io
```

After you provide your credentials, you should see a confirmation similar to this:

```
```plaintext theme={null} Login with your Docker ID to push and pull images
from Docker Hub. If you don't have a Docker ID, head over to https://
hub.docker.com to create one. Username: registry-user Password: WARNING!
Your password will be stored unencrypted in /home/vagrant/.docker/
config.json. Login Succeeded
```

### ## Configuring Kubernetes Pods for Private Registries

To pull an image from a private registry within a pod, specify the full image path in your pod definition.

```
```yaml theme={null}
apiVersion: v1
kind: Pod
metadata:
  name: nginx-pod
spec:
  containers:
    - name: nginx
      image: private-registry.io/apps/internal-app
```

Since Kubernetes worker nodes rely on the Docker runtime for image retrieval, they must be provided with the appropriate credentials. This is achieved by creating a Kubernetes secret of type Docker registry. Execute the following command to create the secret:

```
```bash theme={null} kubectl create secret docker-registry regcred \ --docker-
server=private-registry.io \ --docker-username=registry-user \ --docker-
password=registry-password \ --docker-email=registry-user@org.com
```

Once the secret is created, reference it in your pod specification using the `imagePullSecrets`

```
```yaml theme={null}
apiVersion: v1
kind: Pod
metadata:
  name: nginx-pod
spec:
  containers:
    - name: nginx
      image: private-registry.io/apps/internal-app
  imagePullSecrets:
    - name: regcred
```

When the pod is created, the Kubelet on the worker node will use the credentials stored in the secret to authenticate and pull the image from your private registry.

Summary

This lesson covered key aspects of container image security by demonstrating:

- The importance of proper image naming conventions.
- How to designate public and private repositories.
- Steps for authenticating with private registries.
- Configuring Kubernetes pods with image pull secrets.

By following these practices, you ensure that your applications are deployed using secure and trusted container images. Now, put your understanding into practice and work with secure images in your own projects.

For more information, check out the [Kubernetes Documentation](#) and best practices guides on container security.

Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

Security Contexts

This lesson explains how to secure containers in Kubernetes by configuring security settings at the pod and container levels.

Hello, and welcome to this lesson on Security Contexts in Kubernetes. My name is Mumshad Mannambeth, and in this guide, we'll explain how you can secure your containers by configuring security settings at both the pod and container levels.

When you run a Docker container, you have the flexibility to define various security parameters. For example, you can set a specific user ID or modify Linux capabilities. Consider the following Docker commands:

```
``bash theme={null} docker run --user=1001 ubuntu sleep 3600 docker run --cap-add MAC_ADMIN ubuntu
```

These configurations help manage the security of Docker containers, and similar settings are used in Kubernetes. In Kubernetes, containers are always encapsulated in pods. You can define security settings at the pod level or the container level. Below is an example of a pod definition that configures the security context at the container level.

```
``yaml theme={null}
apiVersion: v1
kind: Pod
metadata:
  name: web-pod
spec:
  containers:
    - name: ubuntu
      image: ubuntu
      command: ["sleep", "3600"]
      securityContext:
        runAsUser: 1000
        capabilities:
          add: ["MAC_ADMIN"]
```

This configuration instructs Kubernetes to run the container as user ID 1000 and adds the `MAC_ADMIN` capability. If your goal is to enforce these security

settings across all containers within a single pod, you can define the security context at the pod level instead.

For more hands-on practice with viewing, configuring, and troubleshooting security contexts in Kubernetes, check out the available lab resources.

By leveraging security contexts, you enhance the security posture of your containerized applications in Kubernetes. For additional guidance, you may find the following resources useful:

- [Kubernetes Documentation](#)
- [Kubernetes Concepts: Security Contexts](#)

Thank you for following along, and happy securing!

Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

Network Policies

This lesson reviews networking and security concepts, focusing on how network policies function within a Kubernetes environment.

Welcome to this lesson on network policies. This guide will review essential networking and security concepts before diving into how network policies function within a Kubernetes environment.

Traffic Flow Example

Consider a simple application configuration consisting of a web server, an API server, and a database server. The traffic flow is as follows:

1. A user sends a request to the web server on port 80.
2. The web server forwards this request to the API server on port 5000.
3. The API server retrieves data from the database server on port 3306, then sends the response back to the user.

There are two main types of network traffic involved:

- **Ingress traffic:** Incoming traffic. For example, user requests arriving at the web server on port 80.
- **Egress traffic:** Outgoing traffic. For example, requests sent from the web server to the API server.

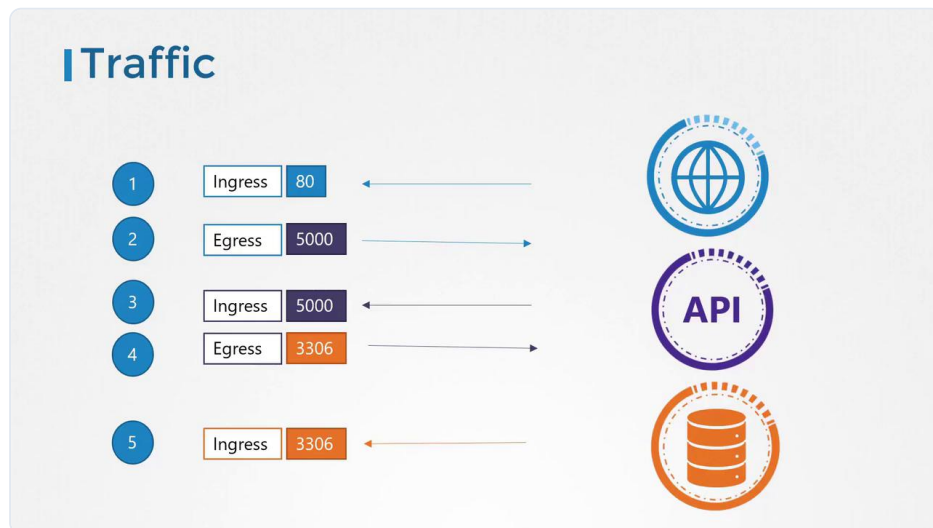
In our diagrams, a solid arrow indicates the direction of the originating traffic (either ingress or egress), while a dotted arrow represents the response flow, which is typically not controlled by network policies.

For clarity:

- The **API server** receives ingress traffic from the web server (port 5000) and sends out egress traffic to the database server (port 3306).
- The **database server** receives ingress traffic on port 3306 from the API server.

To support this traffic flow, the following rules must be established:

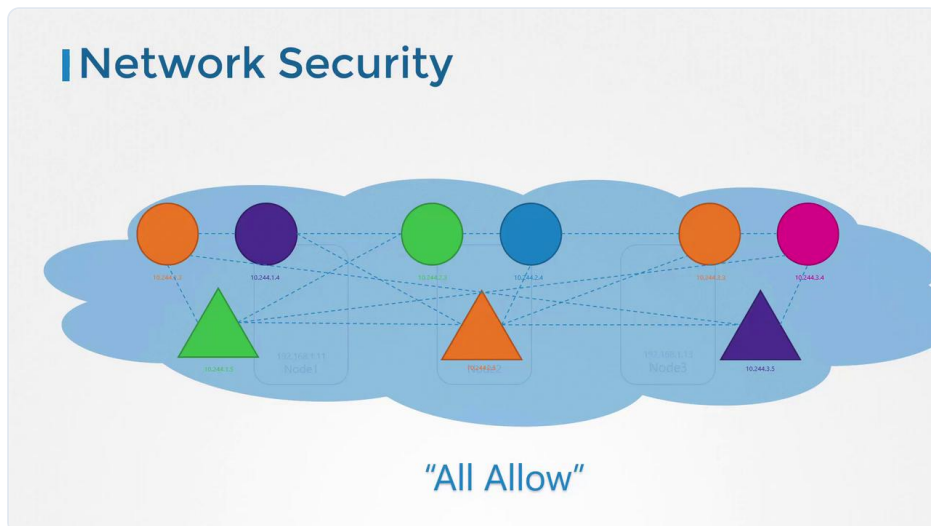
- An ingress rule for the web server to allow HTTP traffic on port 80.
- An egress rule for the web server to permit traffic to port 5000 on the API server.
- An ingress rule for the API server to accept traffic on port 5000.
- An egress rule for the API server to allow traffic to port 3306 on the database server.
- An ingress rule for the database server to allow traffic on port 3306.



Network Security in Kubernetes

In a Kubernetes cluster, nodes host pods and services, each assigned a unique IP address. A crucial capability of Kubernetes is that pods can communicate with one another without extra configuration—such as setting up custom routes. Typically, all pods reside in a virtual private network (VPN) that spans the entire cluster, allowing them to interact using pod IPs, pod names, or configured services.

By default, Kubernetes employs an "all-allow" rule permitting any pod to communicate with every other pod or service within the cluster.

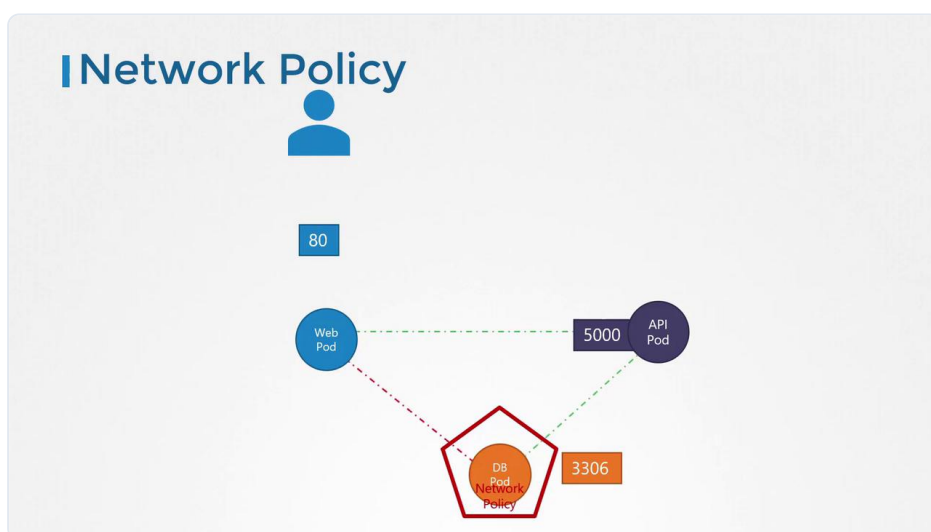


Now, consider the earlier scenario with three pods: one for the front-end web server, one for the API server, and one for the database server. Services facilitate communication between these pods and external users, while the default configuration allows free communication across the cluster.

Restricting Communication with Network Policies

If your security requirements dictate that the front-end web server should not communicate directly with the database server, you can enforce this by implementing a network policy. For example, you might create a policy that only permits the API server to interact with the database server.

A network policy in Kubernetes is defined as an object, which you attach to one or more pods using labels and selectors. In this scenario, the policy would only allow ingress traffic from the API pod on port 3306 while blocking all other sources from accessing the database pod.



Implementing a Network Policy

To apply a network policy, you assign labels to pods and define matching selectors in the network policy object. For example, consider this snippet used to select the database pod:

```
```yaml theme={null} podSelector: matchLabels: role: db
```

This configuration ensures the network policy only applies to pods labeled with `role: db`.

<Frame>

! [The image illustrates a network policy allowing ingress traffic from an API pod to a DB pod.]

</Frame>

Below is the complete network policy configuration:

```
```yaml theme={null}
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: db-policy
spec:
  podSelector:
    matchLabels:
      role: db
  policyTypes:
  - Ingress
  ingress:
  - from:
    - podSelector:
        matchLabels:
          name: api-pod
    ports:
    - protocol: TCP
      port: 3306
```

In this configuration:

- The `podSelector` targets the database pod through its label.
- The `policyTypes` field specifies that only ingress traffic is affected.
- The ingress rule allows traffic specifically from pods with the label `name: api-pod` on TCP port 3306.

Keep in mind that isolation only applies to the traffic explicitly defined under `policyTypes`. Unspecified traffic is automatically allowed by default.

Enabling Network Policies in Kubernetes

To enforce the network policy, execute the following command:

```
bash theme={null} kubectl create -f db-policy.yaml
```


Network policies are enforced by the cluster's networking solution. While solutions like Kube-router, Calico, Romana, and Weave Net support network policies, Flannel does not enforce them. Creating policies with Flannel will not produce an error, but they won't be applied. Always consult your network solution's documentation to verify its support for network policies.

Conclusion

This lesson provided an overview of network policies by demonstrating a basic application setup and illustrating how to restrict pod communication within a Kubernetes cluster. For further practice, be sure to review the Kubernetes documentation and engage in coding challenges to deepen your understanding of network policies.

Additional Resources

- [Kubernetes Documentation](#)
- [Kubernetes Basics](#)
- [Docker Hub](#)
- [Terraform Registry](#)

Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

Developing network policies

This article explores advanced network policy configurations in Kubernetes to secure database pods by controlling ingress and egress traffic.

In this article, we explore advanced network policy configurations within Kubernetes. We will use our familiar web API and database pods to demonstrate how to secure your database pod by allowing only authorized access. Specifically, only the API pod will access the database pod on port 3306, while other pods (such as the web pod) remain unrestricted. By default, Kubernetes permits all traffic between pods, so explicit network policies are necessary to enforce these security measures.

Blocking All Ingress Traffic to the Database Pod

To begin, we block all incoming traffic to the database pod. This is achieved by creating a network policy that targets the database pod using labels and selectors. In our example, the database pod is labeled `role: db`. The following YAML file defines a policy that denies all ingress traffic by default since no specific ingress rules are provided:

```
```yaml theme={null} apiVersion: networking.k8s.io/v1 kind: NetworkPolicy
metadata: name: db-policy spec: podSelector: matchLabels: role: db
policyTypes: - Ingress
```

<Callout icon="lightbulb" color="#1CB2FE">

If no ingress rules are specified, Kubernetes treats the policy as a complete block for incoming traffic.

</Callout>

#### ## Allowing Ingress Traffic from the API Pod

The next step is to allow ingress traffic from the API pod on port 3306. Because responses

For instance, to allow traffic only from pods labeled `name: api-pod` within namespaces lab

```
``yaml theme={null}
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
 name: db-policy
spec:
 podSelector:
 matchLabels:
 role: db
 policyTypes:
 - Ingress
 ingress:
 - from:
 - podSelector:
 matchLabels:
 name: api-pod
 namespaceSelector:
 matchLabels:
 name: prod
 - ipBlock:
 cidr: 192.168.5.10/32
 ports:
 - protocol: TCP
 port: 3306
```

In this configuration, the first entry under the `from` section restricts traffic to API pods in the production namespace (an AND condition). The second entry allows an external backup server by specifying its IP block.

Combining pod selectors with namespace selectors ensures that the rule applies only to the intended pods within the correct namespace.

## Configuring Egress Traffic

In scenarios where the database pod must initiate outbound connections (for example, sending backups to an external server), an egress rule becomes necessary. To support both ingress and egress traffic, include `Egress` in the `policyTypes` and specify an egress rule.

The revised policy below permits the database pod to send traffic to a backup server at IP `192.168.5.10` on port `80` while still restricting all other outbound connections:

```

yaml theme={null} apiVersion: networking.k8s.io/v1 kind:
NetworkPolicy metadata: name: db-policy spec: podSelector:
matchLabels: role: db policyTypes: - Ingress - Egress ingress: -
from: - podSelector: matchLabels: name: api-pod namespaceSelector:
matchLabels: name: prod ports: - protocol: TCP port: 3306 egress: -
to: - ipBlock: cidr: 192.168.5.10/32 ports: - protocol: TCP port:
80

```

Ensure that your egress rules cover all required outbound connections. Missing an egress rule may inadvertently block critical communication between your services.

## Summary of Network Policy Configuration

Configuration Aspect	Description	YAML Reference
Blocking Traffic	Deny all ingress traffic to the database pod by default.	Initial policy with <code>podSelector</code> for <code>role: db</code> .
Allowing Ingress	Permit API pod access on port 3306 by combining pod and namespace selectors. Also allow a specific external IP.	Ingress rule with pod and namespace selectors plus <code>ipBlock</code> .
Configuring Egress	Enable the database pod to send outbound traffic to an external backup server on port 80.	Egress rule addition with <code>ipBlock</code> for backup server.

Experiment in your lab environment with these network policy configurations to gain hands-on experience with Kubernetes security practices. Understanding how selectors and rule ordering interact in your network policies will help you optimize traffic flow and maintain a secure architecture.

For additional resources on Kubernetes networking and security, visit the following links:

- [Kubernetes Documentation](#)
- [Kubernetes Networking Concepts](#)
- [Kubernetes API Reference](#)

Happy networking, and see you in the next article!

## Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

# 2025 Updates Custom Resource Definition CRD

This guide explores Custom Resource Definitions in Kubernetes, detailing standard resources, custom resources like FlightTicket, and the need for custom controllers.

In this guide, we dive into Custom Resource Definitions (CRDs) in Kubernetes, beginning with an overview of standard Kubernetes resources and controllers before extending these principles to custom resources like our FlightTicket example.

## Understanding Standard Kubernetes Resources and Controllers

Kubernetes relies on built-in controllers to manage standard resources. For instance, when you create a Deployment, Kubernetes stores the desired state in its etcd data store and automatically manages related ReplicaSets and Pods. The deployment controller continuously monitors the Deployment and ensures that the cluster state matches the desired configuration. Creating a Deployment with three replicas will result in three Pods being deployed.

Below is an example YAML file defining a Deployment:

```
```yaml
theme={null}
apiVersion: apps/v1
kind: Deployment
metadata:
  name: myapp-deployment
spec:
  replicas: 3
  selector:
    matchLabels:
      type: front-end
  template:
    metadata:
      name: myapp-pod
    labels:
      type: front-end
    spec:
      containers:
      - image: nginx
```
```

```

After saving the above content as `deployment.yml`, run the following commands to create, view, and delete the deployment:

```bash
theme={null}
kubectl create -f deployment.yml
# Output:
kubectl get deployments
# Output:
# NAME                                DESIRED   CURRENT   UP-TO-DATE   AVAILABLE   AGE
kubectl delete -f deployment.yml
# Output:
# deployment "myapp-deployment" deleted

```

The deployment controller (implemented in Go within the Kubernetes source code) handles the creation of a ReplicaSet when a new Deployment is detected. The ReplicaSet then creates the required Pods based on the Deployment's specification. This dynamic process ensures that the actual state of the cluster continuously converges with the desired state.

Custom Resources and Controllers: The Flight Ticket Example

Building on the standard resource management, you can extend Kubernetes by defining custom resources. Imagine a scenario where you want to manage flight ticket bookings directly in Kubernetes. With a custom resource called FlightTicket, you can create objects representing flight ticket bookings, list them, and delete them as needed.

FlightTicket Object Definition

Below is an example YAML file that defines a FlightTicket object:

```

```yaml
theme={null}
apiVersion: flights.com/v1
kind: FlightTicket
metadata:
 name: my-flight-ticket
spec:
 from: Mumbai
 to: London
 number: 2

```

```

To create and manage this custom resource, execute the following commands:

```bash
theme={null}
kubectl create -f flightticket.yml
# Output:
kubectl get flightticket
# Output:
# NAME                                STATUS
kubectl delete -f flightticket.yml
# Output:
# flightticket "my-flight-ticket" deleted

```

At this stage, the FlightTicket object is stored in etcd; however, it does not trigger any actions. To automate operations—such as interfacing with an

external API (e.g., bookflight.com/API) to book or cancel a ticket—you need a custom controller.

Custom Controller for FlightTicket

A custom controller, typically written in Go, monitors FlightTicket objects. When a FlightTicket is created, updated, or deleted, the controller calls an external API to perform actions such as booking or canceling a flight. Below is a streamlined Go snippet to illustrate the controller's logic:

```
```go
package flightticket

import (// Imports omitted for brevity)

var controllerKind = apps.SchemeGroupVersion.WithKind("FlightTicket")

// Run begins watching and syncing FlightTicket resources. func (dc
*FlightTicketController) Run(workers int, stopCh <-chan struct{}) { // Controller
loop implementation here }

// callBookFlightAPI books a flight ticket when a FlightTicket resource is
created. func (dc *FlightTicketController) callBookFlightAPI(obj interface{}) { //
API calling logic here }
```

<Callout icon="lightbulb" color="#1CB2FE">

Without this custom controller, FlightTicket objects remain static data in etcd, and no

### Handling Resource Creation Errors

If you create a FlightTicket object before Kubernetes is aware of its type, you will encounter

```
```bash
theme={null}
kubectl create -f flightticket.yml
# Output:
# no matches for kind "FlightTicket" in version "flights.com/v1"
```

This error appears because Kubernetes does not recognize the FlightTicket resource type. To resolve this, you must first establish a Custom Resource Definition (CRD) for FlightTicket.

Defining a Custom Resource with a CRD

A Custom Resource Definition (CRD) informs Kubernetes about a new resource type, detailing its metadata, scope (namespaced or cluster-scoped), API group, naming conventions (singular, plural, and short names), supported versions, and a validation schema using OpenAPI v3.

Below is a sample CRD for the FlightTicket resource:

```

``yaml  theme={null}  apiVersion:  apiextensions.k8s.io/v1  kind:
CustomResourceDefinition metadata: name: flighttickets.flights.com spec:
group: flights.com scope: Namespaced names: plural: flighttickets singular:
flightticket kind: FlightTicket shortNames: - ft versions: - name: v1 served: true
storage: true schema: openAPIV3Schema: type: object properties: spec: type:
object properties: from: type: string to: type: string number: type: integer
minimum: 1

```

Save this content as `flightticket-custom-definition.yml` and apply it to your Kubernetes cluster.

```

``bash theme={null}
kubectl create -f flightticket-custom-definition.yml
# Output:
# customresourcedefinition "flighttickets.flights.com" created

```

Once the CRD is successfully created, Kubernetes can recognize and store FlightTicket objects. However, remember that without the corresponding custom controller, FlightTicket objects will remain as passive data entries.

Summary

In this guide, we:

- Explored how standard Kubernetes resources, such as Deployments, are managed using built-in controllers.
- Demonstrated the creation, listing, and deletion of a Deployment using a YAML file and kubectl commands.
- Introduced custom resources with a FlightTicket example, emphasizing the need for a Custom Resource Definition (CRD) and a custom controller to automate actions.

Future articles will cover the implementation of a custom controller to automatically process FlightTicket events and integrate with external APIs.

For more details on Kubernetes resources and controllers, check out the [Kubernetes Documentation](#).

Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

2025 Updates Custom Controllers

This lesson covers developing custom controllers for Kubernetes, focusing on monitoring FlightTicket objects and integrating with a flight booking API.

In this lesson, we delve into developing custom controllers for Kubernetes. Building on our previous work, we have already defined a custom resource and created FlightTicket objects with data stored in etcd. The next step is to continuously monitor these objects and perform corresponding actions—such as calling a flight booking API to book, modify, or cancel flight tickets. This process is the core function of a custom controller.

A controller is a process running in a loop that monitors the Kubernetes cluster and reacts to changes in specific objects (in this case, FlightTicket resources). Consider the following FlightTicket definition as a starting point:

```
```yaml theme={null} apiVersion: flights.com/v1 kind: FlightTicket metadata:
name: my-flight-ticket spec: from: Mumbai to: London number: 2
```

You can create the resource using:

```
```bash theme={null}
kubectl create -f flightticket.yml
```

The command output confirms the creation:

```
```bash theme={null} flightticket "my-flight-ticket" created
```

To check the status of your FlightTicket, run:

```
```bash theme={null}
kubectl get flightticket
```

Expected output:

```
```text theme={null} NAME STATUS My-flight-ticket Pending
```

```
<Callout icon="lightbulb" color="#1CB2FE">
```

```
While it is possible to write a controller in a language like Python, challenges such as
</Callout>
```

Developing the controller in Go using the Kubernetes Go client offers a more efficient approach.

## ## Getting Started with the Custom Controller

To begin, clone the SampleController repository from GitHub. Ensure that the Go programming

```
```bash theme={null}
git clone https://github.com/kubernetes/sample-controller.git
```

The terminal should display output similar to:

```
```text theme={null} Cloning into 'sample-controller'... Resolving deltas: 100%
(15787/15787), done.
```

Change into the cloned directory:

```
```bash theme={null}
cd sample-controller
```

Customize the file `controller.go` with your specific business logic. One important function within your controller might involve making a call to the flight booking API after detecting changes in FlightTicket objects.

Once you've incorporated your custom logic, build the controller with the command:

```
```bash theme={null} go build -o sample-controller .
```

During the build process, you may encounter messages such as:

```
```text theme={null}
go: downloading k8s.io/client-go v0.0.0-20211001003700-dbfa30b9d908
go: downloading golang.org/x/text v0.3.6
```

Run the controller by specifying the path to your kubeconfig file:

```
```bash theme={null} ./sample-controller -kubeconfig=$HOME/.kube/config
```

You should see log messages that confirm the event handlers are being set up and that the F

```
```text theme={null}
I1013 02:11:07.489479 40117 controller.go:115] Setting up event handlers
I1013 02:11:07.489701 40117 controller.go:156] Starting FlightTicket controller
```

Inside the Controller Code

Your controller code might include sections like the following:

```
``go theme={null} package flightticket

var controllerKind = apps.SchemeGroupVersion.WithKind("Flightticket")

// Code hidden for brevity

// Run begins watching and syncing. func (dc *FlightTicketController)
Run(workers int, stopCh <-chan struct{}) {}

// Code hidden for brevity

// callBookFlightAPI triggers the flight booking process. func (dc
*FlightTicketController) callBookFlightAPI(obj interface{}) {} ``
```

The controller leverages the specified kubeconfig file to authenticate with the Kubernetes API server. It watches for changes to FlightTicket objects and, upon detecting a creation or modification, it executes your custom logic—such as interfacing with the flight booking API—to reconcile the desired state.

Ensure your API calls are handled efficiently to avoid timeouts and performance issues in production environments.

Deploying Your Controller

Once your custom controller is fully verified and functional, you may want to package it as a Docker image and deploy it within your Kubernetes cluster as a pod or Deployment. This approach streamlines updates by avoiding the need for manual rebuilding and restarting of the controller with every change.

Summary

This lesson provided a comprehensive overview of building a custom controller in Kubernetes. It discussed:

- How to define custom resources for FlightTickets.
- Monitoring and reacting to changes using controllers.
- Building the controller in Go with the built-in queuing and caching support provided by the Kubernetes Go client.
- Deploying your controller in a production environment.

Understanding how Custom Resource Definitions (CRDs) and controllers work together is fundamental, especially since exam questions may cover creating CRDs, managing CRD files, or interacting with existing controllers. Operators, which extend controller functionality further, are another exciting area to explore.

For more details on Kubernetes concepts, check out these resources:

- [Kubernetes Basics](#)

- [Kubernetes Documentation](#)
- [GitHub Repository for SampleController](#)

Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

2025 Updates Operator Framework

This article explores the operator framework, highlighting its role in simplifying Kubernetes resource deployment and management.

In this article, we dive into the operator framework and explore how it simplifies the deployment and management of Kubernetes resources. Previously, we discussed creating a Custom Resource Definition (CRD) and a custom controller to handle resource-specific logic. Traditionally, these components are deployed separately: you first create the CRD and its related resources, and then deploy the controller as a pod or as part of a deployment. With the operator framework, you can package both components into a single deployable entity.

When you deploy the flight operator, it automatically creates the Custom Resource Definition, provisions the required resources, and deploys the custom controller as a Deployment. Consider the following example:

```
```yaml theme={null}
```

## flightticket-custom-definition.yml

```
apiVersion: apiextensions.k8s.io/v1 kind: CustomResourceDefinition metadata:
name: flighttickets.flights.com spec: scope: Namespaced group: flights.com
names: kind: FlightTicket singular: flightticket plural: flighttickets shortnames: -
ft versions: - name: v1 served: true storage: true
```

Below is an example of the custom controller written in Go. This controller monitors and syncs

```
``go theme={null}
package flightticket

import "k8s.io/api/apps/v1"

var controllerKind = v1.SchemeGroupVersion.WithKind("Flightticket")

// Run begins watching and syncing.
func (dc *FlightTicketController) Run(workers int, stopCh <-chan struct{}) {}

// callBookFlightAPI invokes the Book Flight API for a ReplicaSet.
func (dc *FlightTicketController) callBookFlightAPI(obj interface{}) {}
```

To deploy the operator, simply run:

```
``bash theme={null}
```

## Deploy the operator

---

```
kubectl create -f flight-operator.yaml
```

```
<Callout icon="lightbulb" color="#1CB2FE">
```

```
 The operator framework not only streamlines resource deployment but also simplifies ongoing management.
</Callout>
```

One of the most popular examples is the etcd operator. It deploys and manages an etcd cluster.

```
<Frame>
```

```
 ![The image illustrates an Operator Framework with Custom Resource Definitions (CRD) and Operators.]
</Frame>
```

Kubernetes operators handle tasks that would typically require manual intervention by system administrators.

For a comprehensive list of available operators, visit the [Operator Hub](https://operatorhub.io).

## ## How to Deploy an Application Using an Operator

Deploying an application with an operator is an easy process that typically involves:

1. Installing the Operator Lifecycle Manager.
2. Deploying the operator.
3. Enjoying streamlined application management.

The following commands show you how to install the Operator Lifecycle Manager and deploy the etcd operator.

```
```bash theme={null}  
# Install the Operator Lifecycle Manager  
curl -sL https://github.com/operator-framework/operator-lifecycle-manager/releases/download/v0.19.0/operator-lifecycle-manager-0.19.0-linux-amd64.tar.gz  
# Deploy the etcd operator  
kubectl create -f https://operatorhub.io/install/etcd.yaml  
  
# Retrieve the installed Cluster Service Version in the "my-etcd" namespace  
kubectl get csv -n my-etcd
```

This overview provides a high-level understanding of how operators simplify application management. A deep dive into operators will be explored in a dedicated future lesson. For exam preparation, note that most content primarily focuses on CRDs, making this article a valuable supplemental resource.

Thank you for reading, and we'll see you in the next lesson.

Section: Storage

Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/lms.txt> Use this file to discover all available pages before exploring further.

Storage Section Introduction

This guide explores essential Kubernetes storage concepts including persistent volumes, claims, access modes, and configuring applications for persistent storage.

Welcome to the Kubernetes Storage section! In this guide, we will explore essential storage concepts including persistent volumes (PV), persistent volume claims (PVC), and access modes. You'll also learn how to configure your applications for persistent storage in Kubernetes.

Kubernetes provides a versatile range of storage options that can adapt to different environments. This lesson specifically focuses on Kubernetes-native storage mechanisms, ensuring you understand the core concepts needed to integrate any third-party storage solutions with your cluster.

Make sure to review the Kubernetes documentation on storage for additional details and best practices.

Let's dive in and explore how to manage storage effectively in your Kubernetes deployments!

Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

Storage in Docker

This guide explores advanced Docker storage concepts, including storage drivers, data management, and layered architecture for efficient image and container handling.

Welcome to this guide on advanced Docker storage concepts. In this article, we explore how Docker handles storage drivers, manages data on the host file system, and implements a layered architecture to build images and run containers efficiently.

When Docker is installed, it creates a folder structure at `/var/lib/docker` containing subdirectories such as `overlay2`, `containers`, `images`, and `volumes`. These directories store Docker images, container runtime data, and volumes. For instance, files associated with running containers reside in the `containers` folder, image files are stored under `images`, and any created volumes are kept in the `volumes` folder.

Docker Image Layers

Docker images are built using a layered architecture. Each instruction in a Dockerfile generates a new layer, containing only the modifications from the previous layer. Consider this Dockerfile for our first application:

```
``dockerfile theme={null}
```

Dockerfile for Application 1

```
FROM ubuntu
```

```
RUN apt-get update && apt-get -y install python RUN pip install flask flask-mysql
```

```
COPY . /opt/source-code
```

```
ENTRYPOINT FLASK_APP=/opt/source-code/app.py flask run
```

You can build the image using:

```
```bash theme={null}
docker build Dockerfile -t mmumshad/my-custom-app
```

The layers are created in the following order:

1. The base Ubuntu image (approximately 120 MB).
2. A layer installing APT packages (around 300 MB).
3. A layer for Python package dependencies.
4. A layer adding the application source code.
5. A layer that sets the entry point.

Because each layer stores only the changes made in the previous one, Docker caches them for reuse in similar images. For example, a second application with a slight modification might use the following Dockerfile:

```
```dockerfile theme={null}
```

Dockerfile2 for Application 2

```
FROM ubuntu RUN apt-get update && apt-get -y install python RUN pip install
flask flask-mysql COPY app2.py /opt/source-code ENTRYPOINT
FLASK_APP=/opt/source-code/app2.py flask run
```

Build the second image with:

```
```bash theme={null}
docker build Dockerfile2 -t mmumshad/my-custom-app-2
```

Since the first three layers (base image, APT packages, and Python dependencies) are identical, Docker reuses these cached layers and builds only the layers related to the new source code and entry point. This efficient reuse reduces build times and conserves disk space.

When application code changes (for example, modifying `app.py`), Docker leverages the cache for all unchanged layers and rebuilds only the layer with the new code.

## Container Writable Layer and Copy-On-Write

---

Once an image is built, its layers remain immutable (read-only). Running a container from that image with the `docker run` command creates an additional writable layer on top. This layer captures any runtime modifications such as log files, temporary files, or changes to the application. For example:

```
```bash theme={null} docker run mmumshad/my-custom-app
```

If you log into the container and modify a file (say, creating `temp.txt`), Docker employs

<Frame>

! [The image illustrates the "Copy-On-Write" concept, showing container layers with read-w

</Frame>

Persistent Data with Volumes and Bind Mounts

The container's writable layer is ephemeral, meaning any data stored there is lost when the

Volume Mounts

Volumes are managed by Docker and stored under `/var/lib/docker/volumes`. Create and mount

```
```bash theme={null}
docker volume create data_volume
docker run -v data_volume:/var/lib/mysql mysql
```

If you run a container with a volume name that doesn't exist, Docker will automatically create it:

```
```bash theme={null} docker run -v data_volume2:/var/lib/mysql mysql
```

Bind Mounts

Bind mounts allow you to use a specific directory from the Docker host. For example, to use

```
```bash theme={null}
docker run -v /data/mysql:/var/lib/mysql mysql
```

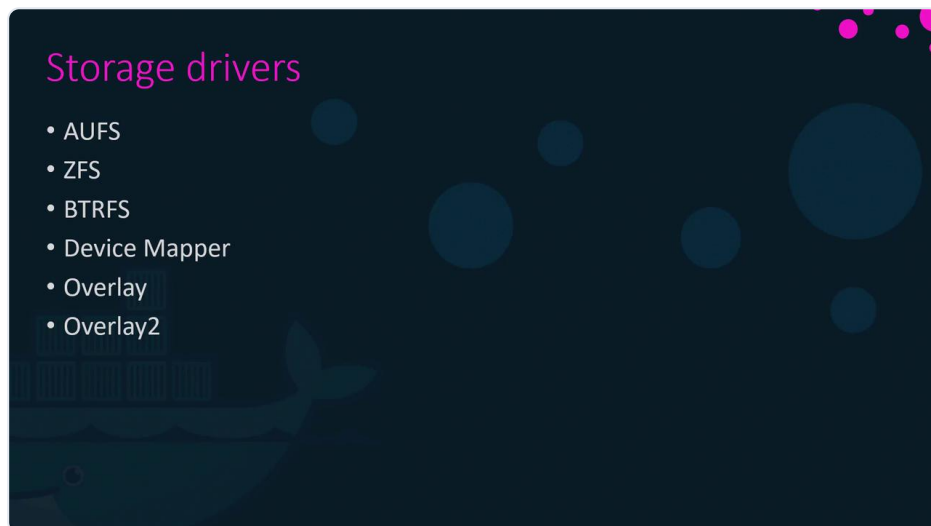
## Using the --mount Option

The `--mount` flag provides a more explicit syntax by requiring all parameters to be specified. The following command is equivalent to the bind mount example above:

```
bash theme={null} docker run \ --mount type=bind,source=/data/
mysql,target=/var/lib/mysql \ mysql
```

## Docker Storage Drivers

Docker's storage drivers manage everything from maintaining image layers to handling writable container layers with copy-on-write. Common storage drivers include AUFS, ZFS, BTRFS, Device Mapper, Overlay, and Overlay2. The selection of a storage driver depends on the host OS. For example, Ubuntu often uses AUFS by default, while Fedora or CentOS might prefer Device Mapper. Docker automatically selects the most appropriate driver for your system based on performance and stability factors.



For more detailed information on these storage drivers, please refer to the [Docker documentation](#).

## Summary

---

Docker's innovative approach to managing storage through image layers, copy-on-write, volumes, and storage drivers enables efficient container builds and resource usage. Understanding these concepts not only improves your workflow but also optimizes container performance and data persistence.

Thank you for reading, and happy containerizing!

## Documentation Index

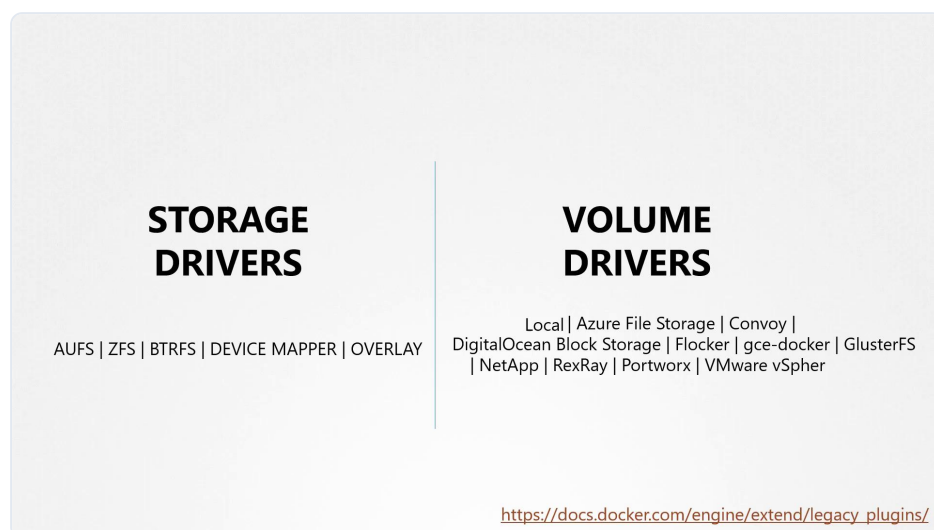
Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

# Volume Driver Plugins in Docker

This article discusses volume driver plugins in Docker for managing persistent storage across various third-party storage solutions.

In previous discussions, we explored storage drivers and their role in managing storage for images and containers. We also examined volumes and emphasized that if you need persistent storage, you must create volumes. It is important to note that volumes are not managed by storage drivers; instead, they are handled by volume driver plugins.

By default, Docker uses the local volume driver plugin. This plugin creates a volume on the Docker host and stores its data under the `/var/lib/docker/volumes` directory. However, many other volume driver plugins are available, enabling you to create volumes on third-party storage solutions such as Azure File Storage, Convoy, DigitalOcean Block Storage, Blocker, Google Compute Persistent Disks, ClusterFS, NetApp, Rex Ray, Portworx, and VMware vSphere Storage.



When selecting a volume driver plugin, consider your storage infrastructure requirements. Plugins such as Rex Ray offer flexibility by supporting various storage providers.

For example, some volume drivers support multiple storage providers. The Rex Ray storage driver, for instance, can provision storage on AWS EBS, S3, EMC storage arrays (like Isilon and ScaleIO), Google Persistent Disk, or OpenStack Cinder. When running a Docker container, you can select a specific volume driver such as Rex Ray EBS to provision a volume from Amazon EBS. This approach ensures that your data remains safe in the cloud even if the container exits.

Below is an example of running a Docker container using a specific volume driver plugin:

```
bash theme={null} docker run -it \ --name mysql \ --volume-driver
rexray/ebs \ --mount src=ebs-vol,target=/var/lib/mysql \ mysql
```

Later in the series, we will further explore how volumes are managed within Kubernetes.

For additional details, refer to the [Docker Documentation](#) and learn more about [persistent storage in containers](#).

## Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

# Container Storage Interface

This article explores the Container Storage Interface (CSI) and its role in integrating storage solutions with container orchestration platforms.

In this article, we explore the Container Storage Interface (CSI) and its critical role in modern container orchestration platforms. CSI provides a standardized way to integrate various storage solutions with container orchestrators, eliminating the need for bespoke, embedded storage code within the core of these systems.

## Evolution in Container Orchestration

Historically, Kubernetes relied exclusively on Docker as the container runtime engine. All code for interfacing with Docker resided within the Kubernetes source. However, with the emergence of alternative container runtimes—such as Rocket and CRI-O—it became necessary to support multiple runtimes without altering Kubernetes' core codebase. This challenge led to the development of the Container Runtime Interface (CRI), a standard that enables orchestration tools like Kubernetes to communicate with different container runtimes seamlessly.

In a similar manner, the Container Networking Interface (CNI) was introduced to facilitate the integration of diverse networking solutions. CNI allows networking vendors to build plugins that adhere to standardized specifications, ensuring smooth operation within Kubernetes environments.

CSI extends these standardization principles to the storage domain, enabling support for multiple storage systems without modifications to container orchestration platforms.

## The Role of CSI

---

The Container Storage Interface (CSI) empowers developers to create custom drivers for various storage systems. Popular examples include:

- Portworx
- Amazon EBS
- Azure Disk
- Dell EMC
- Isilon
- PowerMax
- Unity
- Extreme IO
- NetApp
- Nutanix
- HPE
- Hitachi
- Pure Storage

It is important to emphasize that CSI is not exclusive to Kubernetes. In fact, any container orchestration tool that implements the CSI standard can work with virtually any storage vendor supporting a CSI plugin. Currently, leading platforms such as Kubernetes, Cloud Foundry, and Mesos have adopted CSI.

## How CSI Works

---

When a pod is created that requires persistent storage, the container orchestrator (e.g., Kubernetes) calls the "create volume" Remote Procedure Call (RPC) defined by the CSI standard. This call includes vital details such as the volume name and other parameters. The storage driver then processes the request by provisioning a new volume on the associated storage array and returns the result to the orchestrator.

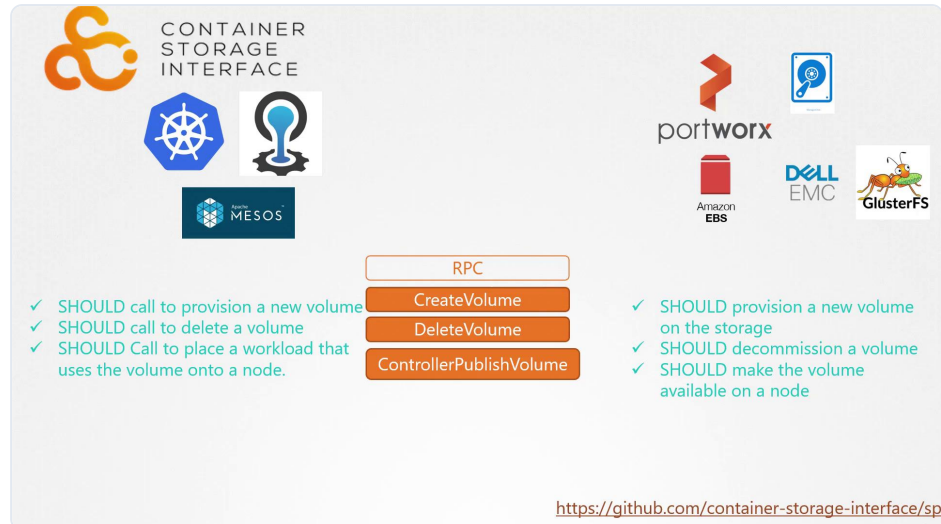
Similarly, when a volume is no longer needed, the orchestrator issues a "delete volume" RPC. The storage driver responds by removing the specified volume from the storage system. The CSI specification outlines the required parameters, expected responses, and error codes for these operations, ensuring consistency and interoperability across different storage solutions.

For those seeking deeper technical insights, the full CSI specification is available on [GitHub](#).



## CSI Architecture Diagram

Below is an illustrative diagram outlining the fundamental components of CSI. The image highlights a series of RPCs defined by the CSI standard, demonstrating how container orchestrators interact with storage drivers:



When integrating CSI, ensure that your storage drivers fully comply with the CSI specification to maintain seamless operation across diverse container orchestrators.

## Conclusion

The Container Storage Interface represents a significant advancement in the way container orchestration platforms interact with storage systems. By standardizing communication through defined RPCs, CSI allows developers to integrate a wide range of storage vendors into their containerized environments with minimal friction.

This article provided an overview of CSI and its functionalities. In the next installment, we will delve into additional aspects of container orchestration, further enhancing your understanding of this dynamic ecosystem.

For more resources on container technologies, please refer to the following links:

- [Kubernetes Documentation](#)
- [Kubernetes Basics](#)
- [Docker Hub](#)
- [Terraform Registry](#)

## Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

# Volumes

This lesson explores persistent volumes in Kubernetes, comparing them to Docker volumes and discussing various storage options with practical examples.

Hello, and welcome to this lesson on persistent volumes in Kubernetes. My name is Mumshad Mannambeth. In this lesson, we will start by exploring volumes in Kubernetes and comparing them to Docker volumes, then move on to a practical example and discuss different storage options.

## Overview of Volumes in Docker

Docker containers are designed to be transient: they are created as needed, process data, and are then destroyed. As a consequence, any data stored within a container is lost when the container is removed. To overcome this limitation, Docker allows you to attach a volume to a container so that the data persists even after the container's lifecycle ends.

## Volumes in Kubernetes

Similarly, Kubernetes pods are ephemeral by design. When a pod processes data and is subsequently deleted, the data inside it is normally lost. To retain this data, a volume is attached to the pod. Any data generated by the pod is stored in the volume and remains available even after the pod is terminated.

## A Simple Example of Using Volumes

Consider a scenario where you have a single-node Kubernetes cluster. A pod generates a random number (between 0 and 100) and writes it to the file `/opt/number.out`. Without a volume, this file would be lost when the pod is deleted. By attaching a volume that uses a directory on the host for storage, the generated number is preserved.

In this example, the volume is configured to use the host's `/data` directory. This implies that any files created within the volume (for instance, the random number written to `/opt/number.out` in the container) are stored in `/data` on the node. The volume is then mounted to the `/opt` directory inside the container.

Below is the YAML configuration for the pod:

```
yaml theme={null} apiVersion: v1 kind: Pod metadata: name: random-number-generator spec: containers: - image: alpine name: alpine command: ["/bin/sh", "-c"] args: ["shuf -i 0-100 -n 1 >> /opt/number.out;"] volumeMounts: - mountPath: /opt name: data-volume volumes: - name: data-volume hostPath: path: /data type: Directory
```

When the pod executes the command, the random number is written to `/opt/number.out` inside the container. Since `/opt` is mounted to the host's `/data` directory, the file persists on the host even after the pod is deleted.

## Volume Storage Options

In the above example, we utilized a `hostPath` volume, which is suitable for a single-node cluster. However, this approach is not ideal for multi-node clusters where each node's `/data` directory may differ. For such environments, Kubernetes supports several external and replicated storage solutions. Some popular storage options include:

- NFS
- GlusterFS
- Flocker
- Fibre Channel
- CephFS
- ScaleIO
- Public cloud storage solutions like AWS EBS, Azure Disk or File, and Google Persistent Disk

For example, to use an AWS Elastic Block Store (EBS) volume, replace the `hostPath` configuration with AWS EBS-specific parameters such as the volume ID and filesystem type. This allows your Kubernetes cluster to leverage AWS EBS for robust, scalable storage.

For a quick comparison, refer to the table below:

Storage Option	Use Case	Example Configuration
hostPath	Single-node clusters	Uses local directory
AWS EBS	Multi-node clusters on AWS	<code>awsElasticBlockStore</code>
Azure Disk	Multi-node clusters on Azure	<code>azureDisk</code>
Google Persistent Disk	Multi-node clusters on Google Cloud	<code>gcePersistentDisk</code>
NFS	Shared storage across nodes	NFS volume configuration

## Conclusion

This lesson provided an overview of volumes in Kubernetes by comparing them with Docker volumes and demonstrating how to use a `hostPath` volume for data persistence. In our next lesson, we will explore persistent volumes in Kubernetes, which offer advanced storage management and additional capabilities tailored for multi-node clusters.

For further reading, check out these resources:

- [Kubernetes Basics](#)
- [Kubernetes Documentation](#)
- [AWS EBS Documentation](#)

## Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

# Persistent Volumes

This lesson explores centralizing storage management in Kubernetes using persistent volumes and how to create and manage them effectively.

Hello and welcome to this lesson on persistent volumes. I'm Mumshad Mannambeth, and in this session, we'll explore how to centralize storage management in Kubernetes using persistent volumes.

Previously, we discussed how volumes are defined within pod manifest files, where the storage settings are directly included in each pod definition. For example, a typical pod configuration with a volume might look like:

```
```yaml theme={null} volumes: - name: data-volume awsElasticBlockStore: volumeID: fsType: ext4
```

In environments where many users deploy multiple pods, duplicating storage configuration in each pod is inefficient. To solve this issue, administrators can create a centralized pool of storage. Users then request storage from this pool. A persistent volume is a cluster-wide storage resource defined and managed by an administrator.

<Frame>
 ![The image illustrates the relationship between Persistent Volume Claims (PVCs) and Persistent Volumes (PVs).]
 </Frame>

Creating a Persistent Volume

In this section, we will create a persistent volume using a base template. First, update the template with the following values:

- * ReadWriteOnce: The volume can be mounted as read-write by a single node.
- * ReadOnlyMany: The volume can be mounted as read-only by multiple nodes.
- * ReadWriteMany: The volume can be mounted as read-write by multiple nodes.

Below is an initial portion of the configuration that defines the persistent volume along with its access modes.

```
```yaml
theme: {null}
apiVersion: v1
kind: PersistentVolume
metadata:
 name: pv-vol1
spec:
 accessModes:
 - ReadWriteOnce
```

Next, specify the storage capacity for this persistent volume. In our example, we set the capacity to 1Gi. After defining the capacity, choose the volume type. Here, we use the hostPath option, which leverages storage from the node's local directory.

The hostPath option is primarily for testing or single-node setups and is not recommended for production environments.

The complete persistent volume manifest appears as follows:

```
```yaml
theme: {null}
apiVersion: v1
kind: PersistentVolume
metadata:
  name: pv-vol1
spec:
  accessModes:
    - ReadWriteOnce
  capacity:
    storage: 1Gi
  hostPath:
    path: /tmp/data
```

To create the persistent volume, execute the following command:

```
```bash
theme: {null}
kubectl create -f pv-d
```

After creating the volume, you can verify its status by running:

```
bash theme={null} kubectl get persistentvolume
```

In a production setup, replace the `hostPath` option with a supported storage solution, such as AWS Elastic Block Store, to ensure data durability and scalability.

This concludes our discussion on persistent volumes. Up next, we will explore persistent volume claims (PVCs) to see how you can request storage from the centralized persistent volume pool.

For further reading, check out these helpful resources:

- [Kubernetes Documentation](#)
- [Kubernetes Basics](#)
- [AWS Elastic Block Store](#)
- [Docker Hub](#)

## Documentation Index

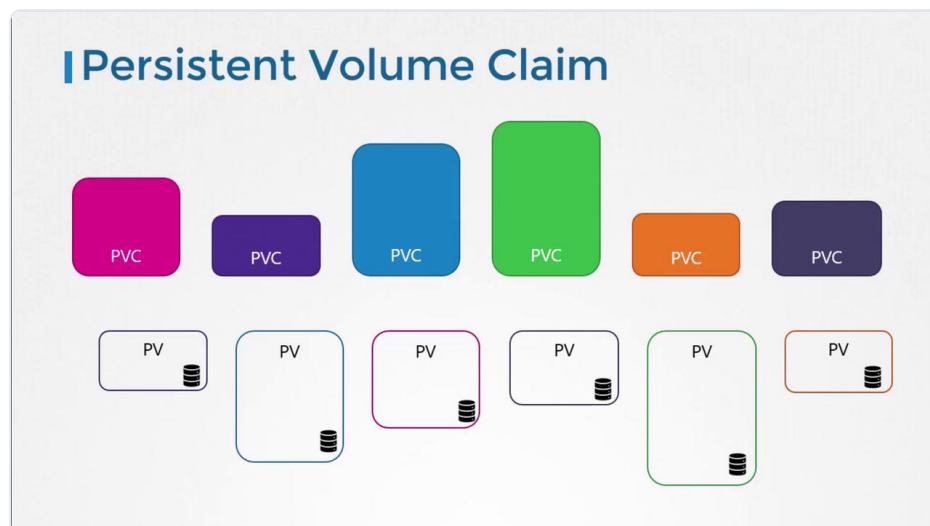
Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

# Persistent Volume Claims

This article explains how to create and manage Persistent Volume Claims in Kubernetes, including binding, deletion, and reclaim policies.

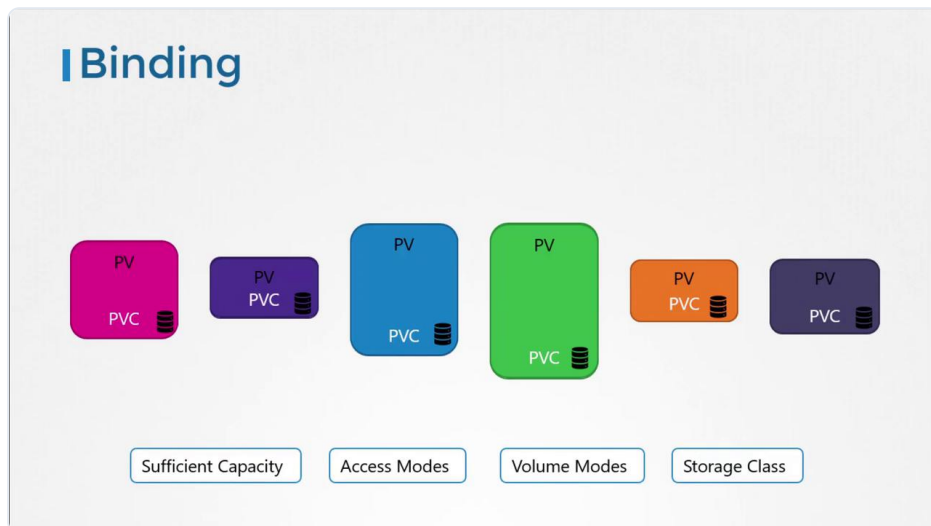
Welcome to this article on Persistent Volume Claims (PVCs) in Kubernetes. In our previous lesson, we created a Persistent Volume (PV). Now, we will explore how to create a Persistent Volume Claim (PVC) to expose that storage to a node.

Persistent volumes and persistent volume claims are two distinct objects in Kubernetes. An administrator is responsible for creating PVs, while users create PVCs to request storage resources. When a PVC is created, Kubernetes automatically binds it to a PV that meets the requested capacity, access modes, volume modes, and storage class.

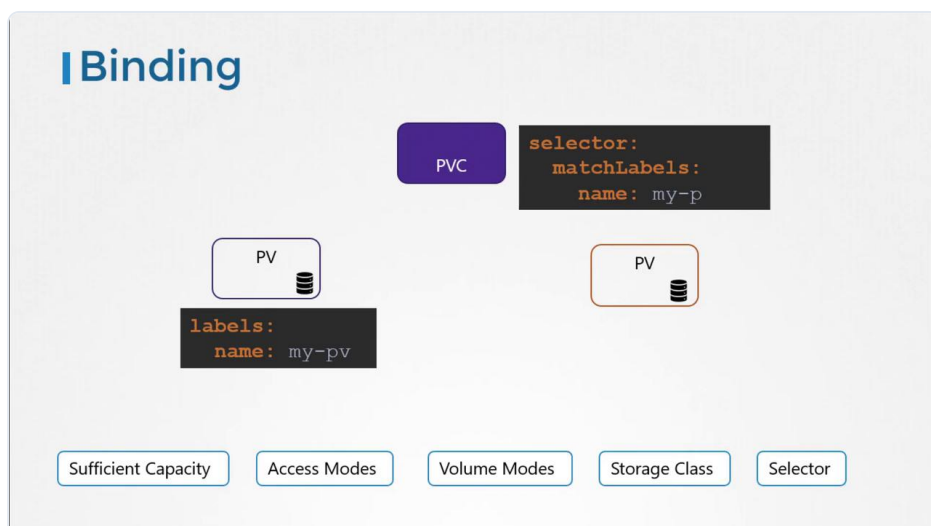


Kubernetes evaluates several factors when binding a PVC to a PV. If multiple PVs can satisfy a claim, you can use labels and selectors to bind the claim to a specific volume.





It is important to note that if a smaller PVC is matched with a larger PV that meets all criteria, the unrequested capacity remains unused by any other PVC. If no PV satisfies the claim's requirements, the PVC will remain in a pending state until a new, suitable PV becomes available.



## Creating a Persistent Volume Claim

Below is an example YAML template for creating a PVC. In this configuration, we set the API version to v1 with kind PersistentVolumeClaim, and name it "myclaim". Under the specification section, the access mode is set to ReadWriteOnce, and 500 MiB of storage is requested.

```
```yaml
theme={null}
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: myclaim
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 500Mi
```
```

To create the PVC:

1. Save the above YAML to a file, for example, `pvc-definition.yaml`
2. Run the command below in your terminal:

```
```bash theme={null}
kubectl create -f pvc-definition.yaml
```
```

You can verify the created PVC by executing:

```
```bash theme={null}
kubectl get persistentvolumeclaim
NAME          STATUS    VOLUME   CAPACITY   ACCESS MODES
myclaim       Pending
```

Kubernetes will inspect the available PV. Suppose, in our example, a PV is configured with 1GiB storage and compatible access modes — if it meets the PVC's criteria, it will automatically bind to the PVC. Here is an example of such a PV definition:

```
```yaml theme={null} apiVersion: v1 kind: PersistentVolume metadata: name:
pv-voll spec: accessModes: - ReadWriteOnce capacity: storage: 1Gi
awsElasticBlockStore: volumeID: fsType: ext4
```

After the binding process, running the `kubectl get persistentvolumeclaim` command will show:

#### ## Deleting a PVC and Persistent Volume Reclaim Policies

To delete a PVC, use the following command:

```
```bash theme={null}
kubectl delete persistentvolumeclaim myclaim
```

When a PVC is deleted, what happens next depends on the underlying persistent volume's reclaim policy. The reclaim policy determines the fate of the PV and can be configured as follows:

Reclaim Policy	Description
Retain	The PV remains in the cluster after the PVC is deleted. An administrator must manually reclaim it.
Delete	The PV is automatically deleted along with the PVC, releasing the storage on the physical device.
Recycle	The PV data is scrubbed before reuse by new claims.

The "Recycle" reclaim policy is deprecated in recent Kubernetes versions and might not be available in your cluster.

For example, to set the reclaim policy to Retain, you would include:

```
yaml theme={null} persistentVolumeReclaimPolicy: Retain
```

Choose the reclaim policy that best fits your storage management strategy.

That concludes our guide on Persistent Volume Claims. We encourage you to practice configuring, binding, and troubleshooting PVs and PVCs in your Kubernetes environment. For further reading, check out the [Kubernetes Documentation](#) and other related resources.

Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

Storage Class

This article explores Kubernetes storage classes and their role in simplifying storage provisioning for applications.

In this lesson, we explore storage classes in Kubernetes and demonstrate how they simplify the process of storage provisioning for applications. Traditionally, administrators manually created PersistentVolumes (PVs) and PersistentVolumeClaims (PVCs) and mounted them to pods. This guide covers both static provisioning (manually creating disks and PVs) and dynamic provisioning using storage classes, making your Kubernetes storage management more efficient.

Static Provisioning

With static provisioning, you manually create the underlying storage (for example, a [Google Cloud persistent disk](#)) and then construct a PV that references that disk. Each time an application requires storage, you must provision a disk on Google Cloud and create the corresponding PV definition.

For example, to create a persistent disk on Google Cloud, you can use the following command:

```
``bash theme={null} gcloud beta compute disks create \ --size 1GB \ --region us-east1 \ pd-disk
```

Then, define your Kubernetes resources as follows:

```
```yaml theme={null}
pv-definition.yaml
apiVersion: v1
kind: PersistentVolume
metadata:
 name: pv-vol1
spec:
 accessModes:
 - ReadWriteOnce
 capacity:
 storage: 500Mi
 gcePersistentDisk:
 pdName: pd-disk
 fsType: ext4
```

```
```yaml theme={null}
```

pvc-definition.yaml

```
apiVersion: v1 kind: PersistentVolumeClaim metadata: name: myclaim spec:
accessModes: - ReadWriteOnce resources: requests: storage: 500Mi
```

```
```yaml theme={null}
pod-definition.yaml
apiVersion: v1
kind: Pod
metadata:
 name: random-number-generator
spec:
 containers:
 - image: alpine
 name: alpine
 command: ["/bin/sh", "-c"]
 args: ["shuf -i 0-100 -n 1 >> /opt/number.out;"]
 volumeMounts:
 - mountPath: /opt
 name: data-volume
 volumes:
 - name: data-volume
 persistentVolumeClaim:
 claimName: myclaim
```

In this setup, the PVC binds to the manually created PV that refers to your existing Google Cloud persistent disk.

---

## Dynamic Provisioning with Storage Classes

Dynamic provisioning removes the need for manual storage pre-provisioning. When you create a PVC, the associated storage class automatically provisions the necessary PV using the defined provisioner.

First, create a storage class object that specifies the provisioner (in this case, Google Cloud's persistent disk):

```
```yaml theme={null}
```

sc-definition.yaml

```
apiVersion: storage.k8s.io/v1 kind: StorageClass metadata: name: google-
storage provisioner: kubernetes.io/gce-pd
```

With the storage class in place, update your PVC to reference it for dynamic provisioning:

```
```yaml theme={null}
pvc-definition.yaml
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
 name: myclaim
spec:
 accessModes:
 - ReadWriteOnce
 storageClassName: google-storage
 resources:
 requests:
 storage: 500Mi
```

The pod definition remains similar:

```
```yaml theme={null}
```

pod-definition.yaml

```
apiVersion: v1 kind: Pod metadata: name: random-number-generator spec:
containers: - image: alpine name: alpine command: ["/bin/sh", "-c"] args: ["shuf
-i 0-100 -n 1 >> /opt/"] volumeMounts: - mountPath: /opt name: data-volume
volumes: - name: data-volume persistentVolumeClaim: claimName: myclaim
```

When you create a PVC with a storage class specified, Kubernetes leverages the defined prov

<Callout icon="lightbulb" color="#1CB2FE">

Using dynamic provisioning simplifies storage management by reducing manual tasks and min

</Callout>

Customizing Storage Classes

Storage classes in Kubernetes support various parameters, allowing you to fine-tune the pro

Below are examples of customized storage class definitions:

```
```yaml theme={null}
silver storage class: standard disk without replication
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
 name: silver
provisioner: kubernetes.io/gce-pd
parameters:
 type: pd-standard
 replication-type: none
```

```
```yaml theme={null}
```

gold storage class: SSD disk without replication

```
apiVersion: storage.k8s.io/v1 kind: StorageClass metadata: name: gold
provisioner: kubernetes.io/gce-pd parameters: type: pd-ssd replication-type:
none
```

```
```yaml theme={null}
platinum storage class: SSD disk with regional replication
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
 name: platinum
provisioner: kubernetes.io/gce-pd
parameters:
 type: pd-ssd
 replication-type: regional-pd
```

By specifying the appropriate storage class in your PVC definitions, you match the storage's performance and reliability to your application's needs.

---

## Summary

---

This lesson demonstrated how Kubernetes storage classes streamline persistent storage provisioning. Whether using static provisioning or dynamic provisioning, storage classes allow for more efficient management of storage resources, reducing manual efforts and enhancing scalability.

For further details on Kubernetes storage and dynamic provisioning, visit the [Kubernetes Documentation](#).



# Section: Networking

---

## Documentation Index

---

Fetch the complete documentation index at: <https://notes.kodekloud.com/lms.txt> Use this file to discover all available pages before exploring further.

## Networking Introduction

---

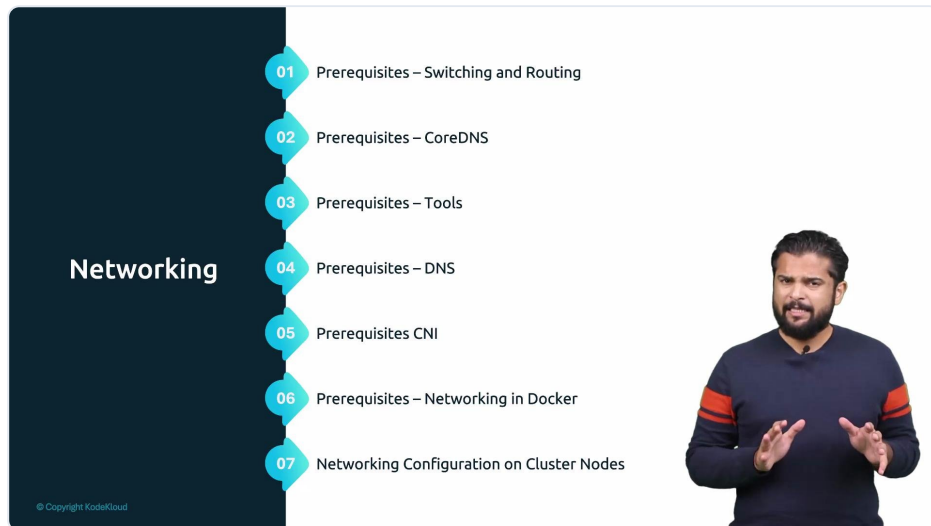
Overview of Kubernetes networking fundamentals including pod IP addressing, CNI plugins, services, Cluster DNS, load balancing, ingress, and troubleshooting.

Hello, and welcome to this section on Kubernetes networking. My name is Mumshad Mannambeth. This lesson covers the core networking concepts you need to design, operate, and troubleshoot Kubernetes clusters.

A solid understanding of a few foundational networking topics will help you get the most from this section. Before diving into cluster-specific details, make sure you are comfortable with:

- Configuring network interfaces and IP addresses
- Gateways and routing basics
- Name resolution and DNS fundamentals
- DNS configuration on Linux systems
- CoreDNS basics
- Network namespaces and how container runtimes (for example Docker) use them

These short prerequisite lectures are optional but recommended. If you already know these basics, skip what you don't need — but review network namespaces and Docker networking if you haven't, since they directly affect how pods and containers are isolated and connected.

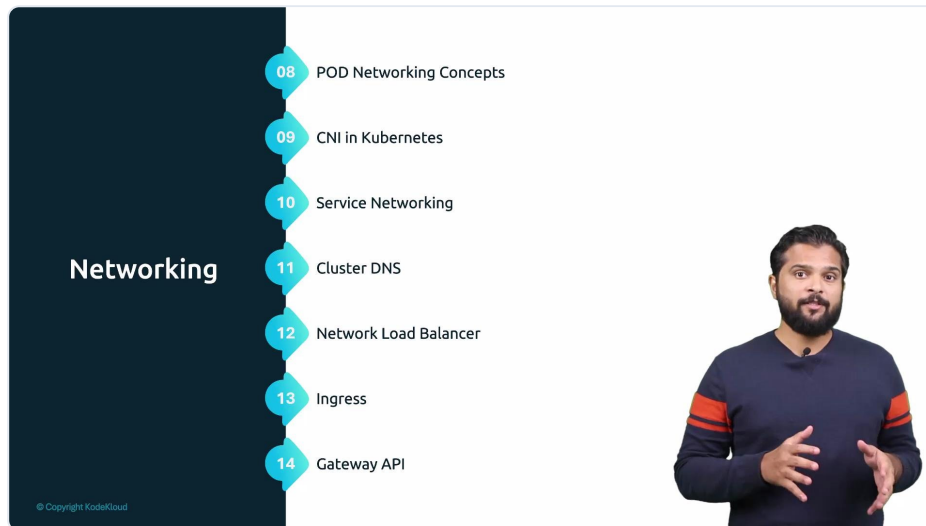


### Why these prerequisites matter

- Network namespaces define per-process network stacks used by container runtimes; understanding them makes pod isolation and container networking much clearer.
- Gateways, routes, and interface configuration are essential when diagnosing connectivity problems between nodes, pods, and external services.
- DNS and CoreDNS are central to Kubernetes service discovery — misconfiguration here is a common source of application failures.

Learning sequence (what we'll cover) To provide a clear learning path, this lesson follows an ordered sequence that builds from cluster-level requirements to higher-level routing and ingress patterns:

1. Cluster networking needs — what Kubernetes expects from the network
2. Pod networking concepts — IP addressing, isolation, and reachability for pods
3. CNI in Kubernetes — how Container Network Interface plugins provide pod networking
4. Service networking — ClusterIP, NodePort, and stable endpoints for applications
5. Cluster DNS — how Kubernetes implements DNS (CoreDNS) for service discovery
6. Network load balancers — external access patterns and load balancing options
7. Ingress and Gateway API — HTTP routing and the newer Gateway API for advanced ingress



What you'll gain from this lesson

- A mental model of how Kubernetes connects pods, services, and external clients.
- Practical knowledge of CNI plugins and how they affect pod IP allocation and routing.
- Familiarity with Service types (ClusterIP, NodePort, LoadBalancer), when to use each, and common troubleshooting steps.
- Understanding of Cluster DNS (CoreDNS) patterns for service discovery and name resolution.
- An overview of ingress patterns, load balancers, and the Gateway API for modern HTTP routing.

Quick reference — concepts and resources

Concept	Purpose	Where to start
Pod networking	IP addressing and connectivity between containers	Pod CIDR, network namespaces
CNI	Plugin model for providing pod network connectivity	CNI plugins (Calico, Flannel, Cilium)
Service types	Stable access to pods: ClusterIP, NodePort, LoadBalancer	<code>kubectl get svc</code> and Service spec
Cluster DNS	Service discovery via CoreDNS	CoreDNS ConfigMap and <code>kube-dns</code>
Ingress / Gateway API	HTTP routing and advanced ingress features	Ingress controllers; Gateway API docs
Network namespaces	Process-level networking isolation	<code>ip netns</code> , container runtime networking

### Further reading and references

- Kubernetes Networking Concepts: <https://kubernetes.io/docs/concepts/cluster-administration/networking/>
- CoreDNS: <https://coredns.io/>
- CNI (Container Network Interface): <https://github.com/containernetworking/cni>
- Kubernetes Services: <https://kubernetes.io/docs/concepts/services-networking/service/>
- Ingress and Gateway API: <https://kubernetes.io/docs/concepts/services-networking/ingress/> and <https://gateway-api.sigs.k8s.io/>

By the end of this lesson you should be able to reason about IP addressing and routing inside a cluster, choose appropriate CNI and Service types for common application topologies, and troubleshoot common DNS and ingress-related networking issues.

## Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

# Prerequisite Switching Routing Gateways CNI in kubernetes

This article explores essential networking concepts for configuring Kubernetes environments, covering switching, routing, gateways, DNS, network namespaces, and Docker networking.

In this article, we explore essential networking concepts from a Linux perspective that are fundamental for configuring Kubernetes environments. We will cover topics such as switching, routing, gateways, DNS, network namespaces, and Docker networking. This guide targets both system administrators and application developers. If you are already confident with Linux networking, feel free to jump ahead to the Kubernetes-focused sections.

## Networking Pre-Requisites

- Switching and Routing
  - Switching
  - Routing
  - Default Gateway
- DNS
  - DNS Configurations on Linux
  - CoreDNS Introduction
- Network Namespaces
- Docker Networking

## Basic Networking Concepts

### Network Interfaces and Switching

Consider two systems—labeled A and B—that could be laptops, desktops, or virtual machines. To enable communication between them, each system must

be connected to a switch with its respective network interface (either physical or virtual). To list available interfaces on a Linux host, execute:

```
``bash theme={null} ip link
```

A sample output might be:

```
``bash theme={null}
eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP mode DEFAULT group
```

Assuming these systems belong to the network 192.168.1.0, you can assign IP addresses using the following command:

```
``bash theme={null} ip addr add 192.168.1.10/24 dev eth0
```

After configuration, test connectivity by pinging another host within the same network:

```
``bash theme={null}
ping 192.168.1.11
```

A successful ping might look like:

```
``bash theme={null} Reply from 192.168.1.11: bytes=32 time=4ms TTL=117
```

### ### Routing Between Subnets

Now, consider a second network, such as 192.168.2.0, with hosts assigned IPs like 192.168.2.1.

A router interconnects two or more networks and holds an IP address in each network—e.g., 192.168.1.1.

<Frame>

! [The image illustrates a network routing diagram with devices A, B, C, and D connected to a central router. Device A is connected to the router's interface 192.168.1.1, and device B is connected to the router's interface 192.168.2.1. Devices C and D are connected to the router's interface 192.168.3.1. The router is configured with routes for each of these networks. The diagram shows the flow of traffic from device A to device B through the router. The router's configuration is shown in a table below.

Each system must be configured with a gateway or specific route entries to ensure that packets are sent to the correct destination.

```
``bash theme={null}
route
```

Initially, communication will be limited to the same subnet. To route traffic destined for 192.168.2.0 via the router (with IP 192.168.1.1), add the following route:

```
``bash theme={null} ip route add 192.168.2.0/24 via 192.168.1.1
```

After adding the route, verifying the routing table should show an entry similar to:

```
``bash theme={null}
route
Kernel IP routing table
Destination Gateway Genmask Flags Metric Ref Use Iface
192.168.2.0 192.168.1.1 255.255.255.0 UG 0 0 0 eth0
```

If a return route is required (for instance, for a host in network 192.168.2.0 to reach a host in 192.168.1.0), add the appropriate route on that system using its corresponding gateway (e.g., 192.168.2.1).

## Configuring Default Routes for Internet Access

To enable internet access (such as reaching external hosts like 172.217.194.0), configure the router as the default gateway. This is done by adding a default route:

```
```bash theme={null} ip route add default via 192.168.2.1
```

Afterward, your routing table might resemble the following:

```
```bash theme={null}
route
Kernel IP routing table
Destination Gateway Genmask Flags Metric Ref Use Iface
192.168.1.0 192.168.2.1 255.255.255.0 UG 0 0 0 eth0
172.217.194.0 192.168.2.1 255.255.255.0 UG 0 0 0 eth0
default 192.168.2.1 0.0.0.0 UG 0 0 0 eth0
```

The "default" or "0.0.0.0" entry indicates that any destination not explicitly listed in the routing table will be directed through the specified gateway.

For scenarios involving multiple routers—such as one handling internet traffic and another managing internal networks—ensure each network has its specific routing entry along with a default route for all other traffic. For example, to route traffic to network 192.168.1.0 via an alternative gateway (192.168.2.2), use:

```
```bash theme={null} ip route add 192.168.1.0/24 via 192.168.2.2
```

The updated routing table should include:

```
```bash theme={null}
route
Kernel IP routing table
Destination Gateway Genmask Flags Metric Ref Use Iface
default 192.168.2.1 0.0.0.0 UG 0 0 0 eth0
192.168.1.0 192.168.2.2 255.255.255.0 UG 0 0 0 eth0
```

If you encounter internet connectivity issues, reviewing the routing table and default gateway configuration is a good troubleshooting practice.

## Configuring a Linux Host as a Router

Consider a scenario with three hosts (A, B, and C) where host B connects to two subnets (192.168.1.x and 192.168.2.x) using two interfaces. For example:

- **Host A:** 192.168.1.5

- **Host B:** 192.168.1.6 and 192.168.2.6
- **Host C:** 192.168.2.5

For host A to communicate with host C, host A must direct traffic aimed at network 192.168.2.0 to host B. On host A, execute:

```
``bash theme={null} ip route add 192.168.2.0/24 via 192.168.1.6
```

```
Similarly, host C needs a route for the 192.168.1.0 network via host B (using 192.168.2.6 as the gateway)

``bash theme={null}
ip route add 192.168.1.0/24 via 192.168.2.6
```

Once these routes are established, the "network unreachable" error should no longer occur when pinging between host A and host C.

## Enabling IP Forwarding on Linux

Even with the correct routing table, Linux does not forward packets between interfaces by default, as a security measure. This setting is controlled by the IP forwarding parameter in `/proc/sys/net/ipv4/ip_forward`.

To check the IP forwarding status, run:

```
``bash theme={null} cat /proc/sys/net/ipv4/ip_forward
```

```
A return value of `0` indicates that packet forwarding is disabled. To enable forwarding to
``bash theme={null}
echo 1 > /proc/sys/net/ipv4/ip_forward
```

Verifying again should now show:

```
``bash theme={null} cat /proc/sys/net/ipv4/ip_forward
```

```
with the output:
```

```
``bash theme={null}
1
```

To ensure this setting persists across reboots, modify `/etc/sysctl.conf` and add or update the following line:

```
text theme={null} net.ipv4.ip_forward = 1
```

Modifying `/etc/sysctl.conf` ensures that IP forwarding remains enabled even after a system restart.

## Summary of Key Commands

Below is a summary table of essential commands covered in this article:



Operation	Command Example
List network interfaces	<code>ip link</code>
View assigned IP addresses	<code>ip addr</code>
Assign an IP address	<code>ip addr add 192.168.1.10/24 dev eth0</code>
View the routing table	<code>route</code>
Add a specific route	<code>ip route add 192.168.1.0/24 via 192.168.2.1</code>
Set a default gateway	<code>ip route add default via 192.168.2.1</code>
Check IP forwarding status	<code>cat /proc/sys/net/ipv4/ip_forward</code>
Enable IP forwarding temporarily	<code>echo 1 &gt; /proc/sys/net/ipv4/ip_forward</code>

Remember, changes made with these commands are temporary and will be reset upon reboot unless they are saved in the appropriate configuration files.

---

That concludes this article. In the next installment, we will dive into DNS configurations and further enhance your understanding of Kubernetes networking.

For additional details and further reading, refer to resources like [Kubernetes Documentation](#) and [Docker Hub](#).

## Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

# Prerequisite DNS

This article introduces DNS for Linux beginners, covering basic concepts, configuration steps, and hands-on labs for understanding local name resolution and centralized DNS servers.

In this article, we introduce DNS for Linux beginners with an SEO-friendly guide that covers basic concepts, configuration steps, and hands-on labs to challenge your understanding of DNS. Learn how local name resolution works and how to transition from simple `/etc/hosts` setups to a full-blown centralized DNS server.

## Understanding Local Name Resolution

Imagine you have two computers on the same network—Computer A with IP address 192.168.1.10 and Computer B with IP address 192.168.1.11. You can easily ping Computer B from Computer A using its IP address:

```
``bash theme={null} ping 192.168.1.11 Reply from 192.168.1.11: bytes=32
time=4ms TTL=117 Reply from 192.168.1.11: bytes=32 time=4ms TTL=117
```

Suppose Computer B offers database services. Instead of remembering its IP address, you'll

```
``bash theme={null}
ping db
ping: unknown host db
```

To make "db" recognizable, add an entry in the `/etc/hosts` file on Computer A. This informs the system that Computer B (192.168.1.11) is known as "db":

```
``bash theme={null} cat >> /etc/hosts 192.168.1.11 db
```

After this change, pings to "db" resolve correctly:

```
```bash theme={null}
ping db
PING db (192.168.1.11) 56(84) bytes of data.
64 bytes from db (192.168.1.11): icmp_seq=1 ttl=64 time=0.052 ms
64 bytes from db (192.168.1.11): icmp_seq=2 ttl=64 time=0.079 ms
```

Once you trust the mappings in `/etc/hosts`, the system does not verify whether the actual hostname (e.g., Computer B's real name) matches the alias you defined.

You can even create multiple aliases for a single IP address. For instance, you might convince Computer A that Computer B is also known as "www.google.com":

```
```bash theme={null} cat >> /etc/hosts 192.168.1.11 db 192.168.1.11
www.google.com
```

```
ping db PING db (192.168.1.11) 56(84) bytes of data. 64 bytes from db
(192.168.1.11): icmp_seq=1 ttl=64 time=0.052 ms 64 bytes from db
(192.168.1.11): icmp_seq=2 ttl=64 time=0.079 ms
```

```
ping www.google.com PING www.google.com (192.168.1.11) 56(84) bytes of
data. 64 bytes from www.google.com (192.168.1.11): icmp_seq=1 ttl=64
time=0.052 ms 64 bytes from www.google.com (192.168.1.11): icmp_seq=2
ttl=64 time=0.079 ms
```

Every time you reference a host by name—whether by ping, SSH, or curl—the system consults t

```
```bash theme={null}
cat >> /etc/hosts
192.168.1.11 db
192.168.1.11 www.google.com

ping db
ssh db
curl http://www.google.com
```

While managing local `/etc/hosts` files works for small networks, it becomes difficult to maintain as the number of systems grows and IP addresses change.

Scaling with a Centralized DNS Server

To overcome the challenges of managing numerous local host mappings, organizations consolidate all mappings on a centralized DNS server. Suppose your centralized DNS server is at IP address 192.168.1.100. You configure each host to use this server by editing the `/etc/resolv.conf` file:

```
```bash theme={null} cat /etc/resolv.conf nameserver 192.168.1.100
```

```
Once configured, any hostname that is not found in `/etc/hosts` is resolved via the DNS server.

```bash theme={null}
cat /etc/nsswitch.conf
...
hosts:          files dns
...
```
```

In this configuration, the system first searches the `/etc/hosts` file for a hostname. If a match is not found, it then queries the DNS server.

Now, if you try pinging a hostname not found in either `/etc/hosts` or the DNS server (e.g., [www.facebook.com](https://www.facebook.com)), the resolution fails:

```
```bash theme={null} cat >> /etc/hosts 192.168.1.115 test
```

```
cat /etc/nsswitch.conf ... hosts: files dns ...
```

```
ping www.facebook.com ping: www.facebook.com: Temporary failure in name resolution
```

To resolve external domains like Facebook, add a public DNS server for example, Google's 8.8.8.8

Domain Names and Structure

Up until now, we have been resolving internal hostnames such as web, db, and nfs. But what about external domains?

- The top-level domain (TLD) appears at the end (e.g., .com, .net, .edu, .org).
- The domain name precedes the TLD (e.g., facebook in [www.facebook.com])(http://www.facebook.com).
- Any segment before the domain name is considered a subdomain (e.g., www).

For instance, consider Google's domain:

- The root is implicit.
- ".com" is the TLD.
- "google" is the main domain.
- "www" is a subdomain.

Subdomains allow organizations to separate services. Examples from Google include [maps.google.com], [mail.google.com], and [www.google.com].

<Frame>

!The image illustrates the structure of domain names, showing root, top-level domain (.com), main domain (google), and subdomain (www).

</Frame>

When your organization attempts to access a domain like apps.google.com, the internal DNS server must resolve the domain name into an IP address.

<Frame>

!The image illustrates the DNS resolution process for "apps.google.com," showing the hierarchy from root to top-level domain (.com), main domain (google), and subdomain (apps).

</Frame>

Similarly, organizations like mycompany.com can structure their domain by using subdomains.

- * www.mycompany.com: External website
- * mail.mycompany.com: Email service
- * drive.mycompany.com: Storage solution
- * payroll.mycompany.com: Payroll systems
- * hr.mycompany.com: Human resources

Using Search Domains for Short Names

Within many organizations, it is often convenient to use short hostnames. To resolve a short hostname, the DNS server must know the search domain.

```
```bash theme={null}
cat >> /etc/resolv.conf
nameserver 192.168.1.100
search mycompany.com
```

```
ping web
PING web (192.168.1.10) 56(84) bytes of data.
64 bytes from web (192.168.1.10): icmp_seq=1 ttl=64 time=0.052 ms
64 bytes from web (192.168.1.10): icmp_seq=2 ttl=64 time=0.079 ms
```

Without the proper search domain, attempts to resolve "web" may fail:

```
```bash theme={null} ping web ping: web: Temporary failure in name resolution
```

```
ping web.mycompany.com PING web.mycompany.com (192.168.1.10) 56(84)
bytes of data. 64 bytes from web.mycompany.com (192.168.1.10): ttl=64
time=0.052 ms
```

```
You can also specify multiple search domains. In the following example, the system will search
for the domain in the following order: mycompany.com, prod.mycompany.com

```bash theme={null}
cat >> /etc/resolv.conf
nameserver 192.168.1.100
search mycompany.com prod.mycompany.com

ping web
PING web.mycompany.com (192.168.1.10) 56(84) bytes of data.
64 bytes from web.mycompany.com (192.168.1.10): icmp_seq=1 ttl=64 time=0.052 ms
64 bytes from web.mycompany.com (192.168.1.10): icmp_seq=2 ttl=64 time=0.079 ms

ping web.mycompany.com
PING web.mycompany.com (192.168.1.10) 56(84) bytes of data.
64 bytes from web.mycompany.com (192.168.1.10): ttl=64 time=0.052 ms

ping web.mvcompany.com
ping: web: Temporary failure in name resolution
```

## Overview of Common DNS Record Types

DNS records map hostnames to IP addresses and serve various other purposes. Here is an overview of some common DNS record types:

Record Type	Hostname	Address/Mapping
A	web-server	Maps hostname to an IPv4 address (e.g., 192.168.1.1)
AAAA	web-server	Maps hostname to an IPv6 address (e.g., 2001:0db8:85a3:0000:0000:8a2e:0370:7334)
CNAME	food.web-server	Aliases one hostname to another (e.g., aliasing to eat.web-server or hungry.web-server)

A records handle IPv4 addresses, AAAA records are for IPv6, and CNAME records allow hostname aliasing.

## Testing DNS Resolution Tools

While ping is the most common tool for verifying basic DNS resolution, utilities like `nslookup` and `dig` provide more detailed insights.

- The `nslookup` command does not consider `/etc/hosts` entries and only queries the configured DNS server.
- The `dig` command offers comprehensive details about DNS queries.

### Example: nslookup

```
``plaintext theme={null}
```

```
nslookup www.google.com Server: 8.8.8.8 Address: 8.8.8.8#53
```

Non-authoritative answer: Name: www.google.com Address: 172.217.0.132

```
Example: dig

``bash theme={null}
dig www.google.com
; <<>> DiG 9.10.3-P4-Ubuntu <<>> www.google.com
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 28065
;; flags: qr rd ra; QUERY: 1, ANSWER: 6, AUTHORITY: 0, ADDITIONAL:

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 512
;; QUESTION SECTION:
;www.google.com. IN A

;; ANSWER SECTION:
www.google.com. 245 IN A 64.233.177.103
www.google.com. 245 IN A 64.233.177.105
www.google.com. 245 IN A 64.233.177.147
www.google.com. 245 IN A 64.233.177.106
www.google.com. 245 IN A 64.233.177.104
www.google.com. 245 IN A 64.233.177.99

;; Query time: 5 msec
;; SERVER: 8.8.8.8#53(8.8.8.8)
;; WHEN: Sun Mar 24 04:34:33 UTC 2019
;; MSG SIZE rcvd: 139
```

## Next Steps

In upcoming practice exercises, you will configure and troubleshoot DNS on actual systems using tools like `dig` and `nslookup`. Later in the course, you will also learn how to set up an actual DNS server using CoreDNS as the DNS solution.

Thank you for reading this article. Enjoy exploring the intricacies of DNS and optimizing your network's name resolution process!

## Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

# Prerequisite Network Namespaces

This article provides a comprehensive guide on network namespaces in Linux, focusing on network isolation and connectivity in containerized environments.

Welcome to this detailed guide on network namespaces in Linux. In this guide, we explain how network namespaces provide network isolation—a critical feature in containerized environments like Docker.

Imagine your host as a house and namespaces as the individual rooms. Each room isolates its occupant (the container), ensuring that processes and network interfaces remain private. While the container only sees the processes within its own namespace, the host maintains oversight over all namespaces and can bridge communication between them when required.

When a container is created, it is placed in its own network namespace. Inside this namespace, the container only sees its own processes. For example, inside a container, running:

```
``plaintext theme={null} ps aux USER PID %CPU %MEM VSZ RSS TTY STAT
START TIME COMMAND root 1 0.0 0.0 4528 828 ? Ss 03:06 0:00 nginx
```

However, listing processes on the host as root shows all processes running on the system.

```
``plaintext theme={null}
ps aux
```

```
``plaintext theme={null} (On the container) USER PID %CPU %MEM VSZ
RSS TTY STAT START TIME COMMAND root 1 0.0 0.0 4528 828 ? Ss 03:06
0:00 nginx
```

```
(On the host) USER PID %CPU %MEM VSZ RSS TTY STAT START TIME
COMMAND project 3720 0.1 0.1 95500 4916 ? R 06:06 0:00 sshd:
project@pts/0 project 3725 0.0 0.1 95196 4132 ? S 06:06 0:00 sshd:
project@notty project 3727 0.2 0.1 21352 5340 pts/0 S 06:06 0:00 -bash root
```



```
3802 0.0 0.0 8924 3616 ? S 06:06 0:00 docker-containerd-shim -namespace m
root 3816 1.0 0.0 4528 828 ? Ss 06:06 0:00 nginx
```

Notice that identical processes appear with different process IDs inside the container comp

## ## Network Isolation

On the networking front, the host maintains its own interfaces, ARP tables, and routing co

For example, running the following command on your host:

```
```bash theme={null}
ip link
```

displays the host's interfaces (such as the loopback and Ethernet interfaces). To examine interfaces within a specific network namespace (for example, the "red" namespace), use:

```
```bash theme={null} ip netns exec red ip link
```

Or with the shorthand using the `-n` option:

```
```bash theme={null}
ip -n red link
```

Inside the namespace, you typically see only a loopback interface, ensuring that host-specific interfaces (e.g., `eth0`) remain hidden. This isolation applies similarly to ARP and routing tables.

Connecting Network Namespaces

By default, a network namespace has no connectivity because it lacks defined interfaces or routes. To connect namespaces, you create virtual Ethernet (veth) pairs that act like cables between two entities.

For example, to connect two namespaces—named "red" and "blue"—first create a veth pair:

```
```bash theme={null} ip link add veth-red type veth peer name veth-blue
```

Then, assign each end to its respective namespace:

```
```bash theme={null}
ip link set veth-red netns red
ip link set veth-blue netns blue
```

Next, assign IP addresses to each namespace's interface—for example, 192.168.15.1 for "red" and 192.168.15.2 for "blue"—and bring the interfaces up:

```
``bash theme={null} ip -n red addr add 192.168.15.1/24 dev veth-red ip -n blue
addr add 192.168.15.2/24 dev veth-blue

ip -n red link set veth-red up ip -n blue link set veth-blue up
```

Test connectivity by pinging from the red namespace to the blue namespace:

```
``bash theme={null}
ip netns exec red ping 192.168.15.2
```

Verify neighbor discovery using ARP from within the red namespace:

```
``bash theme={null} ip netns exec red arp
```

Expected output:

```
``plaintext theme={null}
Address          HWtype  HWaddress          Flags Mask  Iface
192.168.15.2     ether   ba:b0:6d:68:09:e9  C           veth-re
```

Similarly, checking the ARP table in the blue namespace should display an entry for the red namespace. Note that these internal veth interfaces do not appear in the host's ARP table.

Creating a Virtual Switch for Multiple Namespaces

When working with more than two namespaces, linking every pair using veth pairs is impractical. Instead, establish a virtual network switch (or bridge) on the host to interconnect all namespaces. Linux offers tools such as the native Linux bridge or Open vSwitch.

In this example, a Linux bridge is created:

1. Create a new bridge interface (named `v-net-0`):

```
bash theme={null} ip link add v-net-0 type bridge
```

1. Bring the bridge interface up:

```
bash theme={null} ip link set v-net-0 up
```

Namespaces can now be connected to this bridge. Remove the earlier direct veth pair as it is no longer necessary:

```
``bash theme={null} ip -n red link del veth-red
```

Next, create new veth pairs to link each namespace to the bridge. For example, create a pair

```
``bash theme={null}
ip link add veth-red type veth peer name veth-red-br
ip link add veth-blue type veth peer name veth-blue-br
```

Attach the veth interfaces to their corresponding namespaces and the bridge:

```
``bash theme={null} ip link set veth-red netns red ip link set veth-red-br master v-net-0
```

```
ip link set veth-blue netns blue ip link set veth-blue-br master v-net-0
```

Assign IP addresses and activate the interfaces inside each namespace:

```
``bash theme={null}
ip -n red addr add 192.168.15.1/24 dev veth-red
ip -n blue addr add 192.168.15.2/24 dev veth-blue

ip -n red link set veth-red up
ip -n blue link set veth-blue up
```

To allow the host to communicate with the namespaces, assign an IP from the same subnet to the bridge interface:

```
``bash theme={null} ip addr add 192.168.15.5/24 dev v-net-0
```

You can now verify connectivity from the host, for example:

```
``bash theme={null}
ping 192.168.15.1
```

All traffic within this network remains private to the host, ensuring that namespaces are not directly accessible from external networks.

Enabling External Connectivity

The internal network created by namespaces and a bridge is isolated from external networks. Suppose your host's external interface utilizes a LAN subnet (e.g., 192.168.1.0/24) and you wish for namespaces to communicate with external systems. For instance, pinging an external host (192.168.1.3) from the blue namespace would initially result in a "Network is unreachable" error:

```
``bash theme={null} ip netns exec blue ping 192.168.1.3
```

Checking the routing table in the blue namespace using:

```
``bash theme={null}
ip netns exec blue route
```

might display:

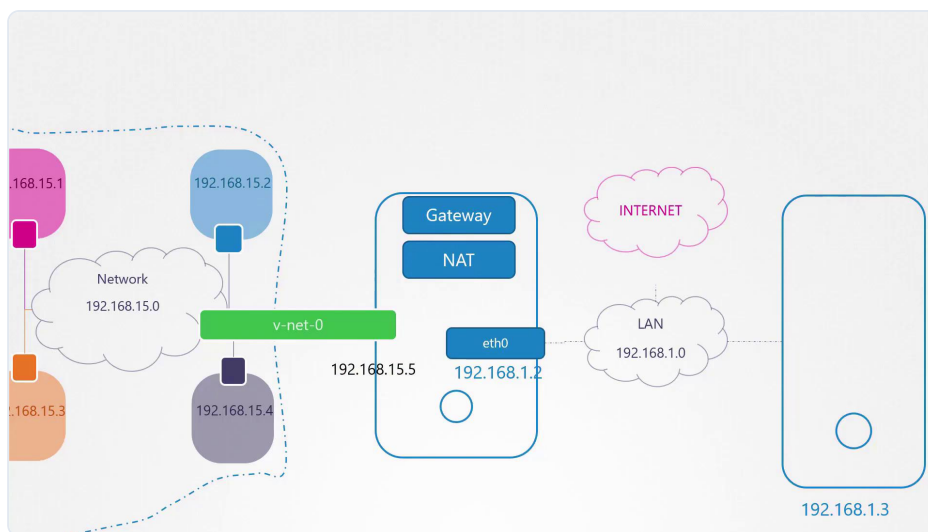
```
``plaintext theme={null} Destination Gateway Genmask Flags Metric Ref Use
Iface 192.168.15.0 0.0.0.0 255.255.255.0 U 0 0 0 veth-blue
```

To enable external connectivity, add a route in the blue namespace that directs traffic des

```
```bash theme={null}
ip netns exec blue ip route add 192.168.1.0/24 via 192.168.15.5
```

After this route is added, the blue namespace can reach external networks. However, external networks will not recognize the private IP addresses. To address this, configure NAT on the host using iptables. This setup masquerades packets from the 192.168.15.0/24 network with the host's external IP, making the traffic routable on the LAN.

To provide namespaces with Internet access (such as pinging 8.8.8.8), add a default route in the namespace pointing to the host; the host must then perform NAT to forward the traffic correctly.



## Enabling Inbound Access

Thus far, namespaces are isolated on an internal network and are not directly accessible from external hosts. For instance, if the blue namespace runs a web application on port 80, external users cannot access it simply by targeting its private IP.

There are two common solutions:

1. Add a static route on the external router or host so that traffic for the 192.168.15.0/24 network is directed through the host's LAN IP (e.g., 192.168.1.2).
2. Use port forwarding via iptables on the host to redirect traffic arriving at a specific port (e.g., port 80) to the corresponding port in the namespace.

The port forwarding method is often preferred, as it eliminates the need to reconfigure external routing.

This concludes our guide on network namespaces. By following these steps, you can effectively isolate, connect, and manage network namespaces for containerized environments. Thank you for reading!

## Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

# Prerequisite Docker Networking

This article provides an overview of Docker networking options, including modes like none, host, and bridge, and explains container connectivity and port mapping.

Welcome to this comprehensive lesson on Docker networking. In this guide, we explore the various networking options available in Docker and how they relate to network namespaces. We'll begin by focusing on a single Docker host — a server with Docker installed. Assume the host has an Ethernet interface (eth0) with the IP address 192.168.1.10 on the local network.

## Docker Networking Modes

When you run a container, Docker provides several networking modes to tailor connectivity to your application's needs:

### None Network

With the "none" network mode, the container is not attached to any network. As a result, it cannot communicate with external systems, nor can external systems reach the container. For example:

```
```bash theme={null} docker run --network none nginx
```

<Callout icon="lightbulb" color="#1CB2FE">

Multiple containers started in the none network mode remain completely isolated from each

</Callout>

Host Network

In the host network mode, the container shares its host's network stack. This means there i

```
```bash theme={null}
docker run --network host nginx
```

## Bridge Network

The default Docker networking mode is the bridge network. When Docker is installed, it automatically creates an internal private network called "bridge" (visible as "docker0" on the host) with a default subnet (usually 172.17.0.0/16). Each container connected to this network receives a unique IP address from this subnet. For example, running two containers:

```
```bash theme={null} docker run nginx docker run nginx
```

These containers communicate with each other over the internal bridge network. To inspect t

```
```bash theme={null}
docker network ls
```

An example output might look like:

```
```text theme={null} NETWORK ID NAME DRIVER SCOPE 2b6008726112
bridge bridge local 0beb4870b093 host host local 99035e02694f none null
local
```

Internally, Docker creates the "docker0" interface on the host, serving as the bridge betwe

```
```bash theme={null}
ip link show docker0
```

By default, the "docker0" interface is assigned the IP address 172.17.0.1. When a container is launched, Docker creates a new network namespace for it (similar to those discussed in earlier lessons). To list the available network namespaces (a minor hack may be required to display Docker-created namespaces):

```
```bash theme={null} ip netns list
```

Typically, the namespace will have a name starting with an identifier like "b3165...". To v

```
```bash theme={null}
docker inspect <container_id>
```

A snippet of the output under "NetworkSettings" might appear as follows:

```
```json theme={null} "NetworkSettings": { "Bridge": "", "SandboxID":
"b3165c10a92b50edc4c8aa5f37273e180907ded31", "SandboxKey": "/var/run/
docker/netns/b3165c10a92b" }
```

Container Attachment to the Bridge Network

Docker attaches each container to the bridge network by creating a pair of virtual interfaces

Inspect the interfaces on the Docker host using:

```
```bash theme={null}
ip link show
```

You might see an output similar to:

```
```text theme={null} 4: docker0: mtu 1500 qdisc noqueue state UP mode
DEFAULT group default link/ether 02:42:9b:5f:d6:21 brd ff:ff:ff:ff:ff:ff 8:
vethbb1c343@if7: mtu 1500 qdisc noqueue master docker0 state UP mode
DEFAULT group default link/ether 9e:71:37:83:9f:50 brd ff:ff:ff:ff:ff:ff link-netnsid
1
```

To inspect the network namespace of a container (for example, with namespace ID b3165c10a92b)

```
```bash theme={null}
ip -n b3165c10a92b link show
```

The output might be similar to:

```
```text theme={null} 7: eth0@if8: mtu 1500 qdisc noqueue state UP mode
DEFAULT group default link/ether 02:42:ac:11:00:03 brd ff:ff:ff:ff:ff:ff link-
netnsid 0
```

To view the IP address assigned to the container's network interface

```
```bash theme={null}
ip -n b3165c10a92b addr show eth0
```

This may display:

```
```text theme={null} 7: eth0@if8: mtu 1500 qdisc noqueue state UP group
default link/ether 02:42:ac:11:00:03 brd ff:ff:ff:ff:ff:ff link-netnsid 0 inet
172.17.0.3/16 brd 172.17.255.255 scope global eth0 valid_lft forever
preferred_lft forever
```


Each time a new container is created, Docker follows these sequential steps:

1. Creates a new network namespace.
2. Establishes a pair of virtual interfaces.
3. Attaches one end to the container's namespace and the other to the "docker0" bridge.
4. Assigns an IP address to the container's interface.

The virtual interface pairs are numbered consistently, with odd and even numbers forming

Port Mapping

Consider a scenario where a container runs an nginx web application that listens on port 80

For example, to map port 8080 on the host to port 80 within the container:

```
```bash theme={null}
docker run -p 8080:80 nginx
```

After setting up port mapping, you can test connectivity as follows:

- Accessing the container directly via its IP on port 80 might result in a failure:

```
bash theme={null} curl http://172.17.0.3:80 # Output: curl: (7)
Failed to connect... No route to host
```

- However, accessing the application using the host's IP and port 8080 should succeed:

```
bash theme={null} curl http://192.168.1.10:8080 # Output: Welcome to
nginx!
```

All incoming traffic to port 8080 on the Docker host is forwarded to port 80 in the container.

## How Port Forwarding Works

Docker employs IP tables to implement port forwarding. This mechanism adds a Network Address Translation (NAT) rule to translate traffic arriving on a specific host port to the corresponding container port. For example, Docker might add a rule similar to the following:

```
```bash theme={null} iptables \ -t nat \ -A PREROUTING \ -j DNAT \ --dport
8080 \ --to-destination 80
```

This rule directs traffic arriving at port 8080 on the host to port 80 on the container. Yo

```
```bash theme={null}
iptables -nVL -t nat
```

An excerpt from the output might resemble:

```
``text theme={null} Chain DOCKER (2 references) target prot opt source
destination RETURN all -- anywhere anywhere DNAT tcp -- anywhere
anywhere tcp dpt:8080 to:172.17.0.2:80
```

After the mapping is configured, testing access to the container using:

```
``bash theme={null}
curl http://172.17.0.3:80
```

should return a response like:

```
text theme={null} Welcome to nginx!
```

## Conclusion

---

In summary, Docker networking offers multiple modes—none, host, and bridge—to manage connectivity for containers. The default bridge network uses a virtual switch (docker0) to connect containers via dedicated network namespaces, while port mapping enables external access by forwarding traffic from a designated host port to the container's port.

For a deeper dive into container networking, consider exploring the [Kubernetes Networking Basics](#) and other resources in the [Docker Documentation](#).

Happy learning!

## Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

## Prerequisite CNI

This article provides a comprehensive guide on the Container Networking Interface (CNI) and its role in simplifying container networking configuration and management.

Welcome to this comprehensive guide on the Container Networking Interface (CNI) and its vital role in container networking. In this lesson, we explore how network namespaces and standardized networking plugins simplify the configuration and management of container networks.

Network namespaces create isolated network environments on a single host. These namespaces are interconnected by a bridge network that establishes virtual interfaces (or virtual cables) for communication between namespaces. This involves assigning IP addresses, activating interfaces, and enabling NAT or IP masquerading for external connectivity. Although Docker configures its bridge networking using similar methods, it employs its own naming conventions. Other container platforms like Rocket, Mesos Containerizer, and Kubernetes address these networking challenges in a comparable way.



To standardize this process and avoid duplicating efforts across multiple platforms, a dedicated program known as "bridge" was developed. This

program automates the tasks required to connect a container to a bridge network. For instance, you can run the program with the container ID and network namespace as shown below:

```
```bash theme={null} bridge add 2e34dcf34 /var/run/netns/2e34dcf34
```

The "bridge" program handles low-level networking configuration, freeing container runtimes from having to implement these details.

<Callout icon="lightbulb" color="#1CB2FE">

By offloading network configuration tasks to a standardized bridge program, container runtimes can focus on managing containers. </Callout>

This brings us to an important question: if you want to develop a similar program for a different container runtime, what standards should you follow?

CNI defines a standard for creating and integrating network plugins with container runtimes.

- * Creating a network namespace for each container.
- * Identifying the networks to which the container should connect.
- * Configuring the network when a container is created (using the "add" command) and cleaning up when it is destroyed.
- * Setting up necessary network details via a JSON configuration file.

On the plugin side, CNI requires support for three command-line arguments: "add", "del", and "check".

<Frame>

! [The image outlines the Container Network Interface (CNI) requirements and processes, including the roles of container runtimes and network plugins.] </Frame>

When both container runtimes and network plugins adhere to CNI standards, seamless interoperability is achieved.

<Callout icon="triangle-alert" color="#FF6B6B">

Docker uses its own networking standard known as the Container Network Model (CNM), which is not compatible with CNI. </Callout>

Consider the following example that demonstrates how Kubernetes handles networking with Docker.

```
```bash theme={null}
docker run --network=none nginx
bridge add 2e34dcf34 /var/run/netns/2e34dcf34
```

In this workflow, Kubernetes first creates a Docker container without any network configuration and then calls the CNI plugin to establish the network. This process highlights how Kubernetes efficiently leverages CNI standards to manage container networks.

For further information, explore these resources:

- [Kubernetes Basics](#)
- [Kubernetes Documentation](#)
- [Docker Hub](#)
- [Terraform Registry](#)

Dive deeper into container networking and harness the power of CNI standards in your deployments. Happy networking!

## Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

# Cluster Networking

This article explores networking configurations for master and worker nodes in a Kubernetes cluster, including required ports and verification commands.

In this article, we explore the networking configurations necessary for both the master and worker nodes within a Kubernetes cluster. Each node must be equipped with at least one network interface configured with an IP address. Additionally, every host should have a unique hostname and MAC address—this is especially important when creating virtual machines (VMs) by cloning existing ones.

## Required Ports for Kubernetes Components

Effective communication among the control plane components and worker nodes relies on specific port configurations. The following table summarizes the key ports that must be open:

Port Range	Component	Description
6443	Kubernetes API Server (master)	Used by worker nodes, the kube-controller-manager, external users, and other control plane components to access the API Server.
10250	Kubelet (master and worker)	Monitors cluster activities and manages nodes.
10259	Kube-scheduler (master)	Required for scheduling operations.
10257	Kube-controller-manager (master)	Needed for managing cluster state and various controllers.
30000–32767	Worker Nodes Services	Exposes services for external access on worker nodes.
2379 & 2380	etcd Server (master)	Port 2379 is used for client communication, and port 2380 is used for communication between etcd servers in multi-master deployments.

For a comprehensive list of required ports and additional configuration details, refer to the [Kubernetes Documentation](#). These details are crucial when configuring firewalls, IP tables, or network security groups across cloud platforms like GCP, Azure, or AWS.

## Verifying Network Configuration

To ensure that your cluster's network environment is set up correctly, it is useful to run several common commands. These commands help you inspect interfaces, IP addresses, hostnames, routing tables, and active services:

```
bash theme={null} ip link ip addr ip addr add 192.168.1.10/24 dev eth0 ip route ip route add 192.168.1.0/24 via 192.168.2.1 cat /proc/sys/net/ipv4/ip_forward arp netstat -plnt
```

These commands are invaluable for gathering information about your network interfaces, IP configurations, and port usage. As you continue to explore your Kubernetes cluster setup, these tools will assist you in troubleshooting and ensuring network connectivity.

In upcoming exercises, we will delve into more complex networking scenarios to further enhance your understanding of Kubernetes networking. For now, take your time to review the current configuration and familiarize yourself with the fundamental network requirements of your cluster.

## Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

# Pod Networking

This article explains pod networking in Kubernetes, covering implementation, inter-pod communication, and the role of CNI in managing network interfaces.

Welcome to this deep dive into pod networking in Kubernetes. In this article, we explain how pod networking is implemented within a Kubernetes cluster, detailing the steps that make inter-pod communication seamless across multiple nodes.

Kubernetes clusters consist of multiple master and worker nodes with pre-configured networking that allows full node-to-node communication. We assume that all Kubernetes control plane components (e.g., kube-apiserver, etcd, and kubelet) are already set up properly. With this foundation in place, the next step is deploying your applications with a focus on robust pod-level networking.

\* Each pod gets a unique IP address. \* Every pod on a node can reach every other pod on that node using its IP address. \* Pods across different nodes can communicate using a consistent addressing scheme.

These requirements ensure that Kubernetes can support both local (node-level) and cross-node communications without relying on NAT rules.

## Understanding the Basics

Before deploying applications, it is crucial to address several questions:

- How are pods assigned IP addresses?
- How do these pods communicate both within a node and across nodes?
- How can services running in these pods be accessed from inside or outside the cluster?

Kubernetes leaves the implementation details for pod networking to the user, as long as the chosen solution meets the basic connectivity requirements.

Many networking solutions are available; here, we demonstrate the core concepts using Linux network namespaces, bridge networks, and IP address management inspired by CNI concepts.

## Setting Up a Simple Bridge Network

Below is an example that sets up a simple bridge network and connects network namespaces:

```
```bash
theme={null}
ip link add v-net-0 type bridge
ip link set dev v-net-0 up
ip addr add 192.168.15.5/24 dev v-net-0
ip link add veth-red type veth
ip link set veth-red netns red
ip -n red addr add 192.168.15.1 dev veth-red
ip -n red link set veth-red up
ip link set veth-red-br master v-net-0
ip netns exec blue ip route add 192.168.1.0/24 via 192.168.15.5
iptables -t nat -A POSTROUTING -s 192.168.15.0/24 -j MASQUERADE
```
```

Imagine a cluster with three nodes where every node runs a combination of management and workload namespaces.

### Cluster Network Planning

Consider the following plan for a three-node cluster:

1. **External Network Configuration**: Each node has an IP in the 10.244.1.0 series (e.g., 10.244.1.1).
2. **Container Network Namespaces**: Kubernetes creates unique network namespaces when containers are created.
3. **Bridge Networks on Each Node**: Create a bridge network on every node. Assign a distinct IP address to each bridge.
  - \* Node one: 10.244.1.0/24
  - \* Node two: 10.244.2.0/24
  - \* Node three: 10.244.3.0/24

When a container is created, a virtual cable (veth pair) connects its network namespace to the host's network namespace.

Below is a sample snippet from a script that connects a container to the network:

```
```bash
theme={null}
# Create veth pair
ip link add <veth-in-host> type veth peer name <veth-in-namespace>
# Attach one end to the container's namespace
ip link set <veth-in-namespace> netns <namespace>
# Attach the other end to the bridge
ip link set <veth-in-host> master <bridge>
# Assign IP address to the container's interface
ip -n <namespace> addr add <IP-address>/24 dev <veth-in-namespace>
# Add default route in the container's namespace
ip -n <namespace> route add default via <bridge-IP>
# Bring up the container's interface
ip -n <namespace> link set <veth-in-namespace> up
```
```

Executing this script on each node ensures that all containers receive an IP address and are connected to their respective internal networks.



## Cross-Node Communication

One of the challenges is enabling communication between pods on different nodes. For example, if a pod with IP 10.244.1.2 on node 1 attempts to ping a pod with IP 10.244.2.2 on node 2, the ping may initially fail due to unknown routes between subnets:

```
```bash theme={null} bluepod$ ping 10.244.2.2
Connect: Network is unreachable
```

To resolve this, add a route on node 1 that directs traffic for 10.244.2.2 via node 2's ext

```
```bash theme={null}
node1$ ip route add 10.244.2.2 via 192.168.1.12
```

After configuring the routing, the ping command should succeed:

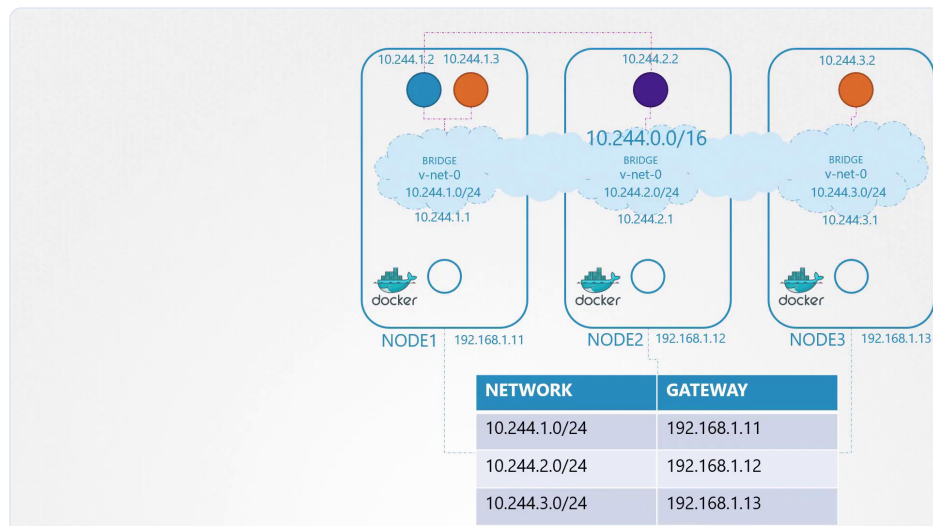
```
```bash theme={null} bluepod$ ping 10.244.2.2
64 bytes from 10.244.2.2: icmp_seq=1 ttl=63 time=0.587 ms
64 bytes from 10.244.2.2: icmp_seq=2 ttl=63 time=0.466 ms
```

Similarly, add these routes on all nodes to cover all pod subnets:

```
```bash theme={null}
node1$ ip route add 10.244.2.2 via 192.168.1.12
node1$ ip route add 10.244.3.2 via 192.168.1.13
node2$ ip route add 10.244.1.2 via 192.168.1.11
node2$ ip route add 10.244.3.2 via 192.168.1.13
node3$ ip route add 10.244.1.2 via 192.168.1.11
node3$ ip route add 10.244.2.2 via 192.168.1.12
```

Manually configuring routes on every host is impractical for large-scale deployments. A more scalable solution involves configuring a centralized router to manage all subnet routes and setting each node's default gateway to this router.

Below is an image that illustrates a Docker network setup with three nodes, each with distinct IP addresses and subnet configurations, connected via a virtual network bridge:



## Integrating Container Network Interface (CNI)

In our lab setup, we executed scripts manually to configure pod networking. However, in a production Kubernetes environment where thousands of pods may be created per minute, this manual approach is not feasible.

This is where the Container Network Interface (CNI) becomes essential. CNI specifies how Kubernetes should invoke a networking script each time a pod is created. To conform with CNI standards, the networking script must have:

- An "add" section to connect the container to the network.
- A "delete" section to disconnect the container, remove interfaces, and free up the IP address.

When the container runtime launches a container, it uses the CNI configuration (provided as a command-line argument) to execute the relevant script with the command "add" and pass the container's name and namespace identifier.

Below is an example snippet that illustrates the CNI execution process:

```
bash theme={null} ip -n <namespace> link set <interface> up ip link del <interface> ./net-script.sh add <container> <namespace>
```

## Conclusion

In this article, we covered the essential concepts behind pod networking in Kubernetes—from manual network namespace and bridge configuration to the role of CNI in automating network interface management. The techniques discussed here lay a solid foundation for understanding and troubleshooting pod networking in a Kubernetes cluster.

Stay tuned for upcoming articles where we integrate detailed CNI configurations into Kubernetes workflows and provide practical tests to reinforce your learning.

For more insight into Kubernetes networking, visit the [Kubernetes Documentation](#).

## Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

# CNI in kubernetes

This lesson explores how Kubernetes uses the Container Network Interface to manage container networking and configure network plugins.

In this lesson, we explore how Kubernetes leverages the Container Network Interface (CNI) to manage container networking. You will gain an understanding of how network plugins are configured and used in a Kubernetes environment.

Earlier, we reviewed:

- The basics of networking and namespaces
- Docker networking fundamentals
- The evolution and rationale behind CNI
- A list of supported CNI plugins

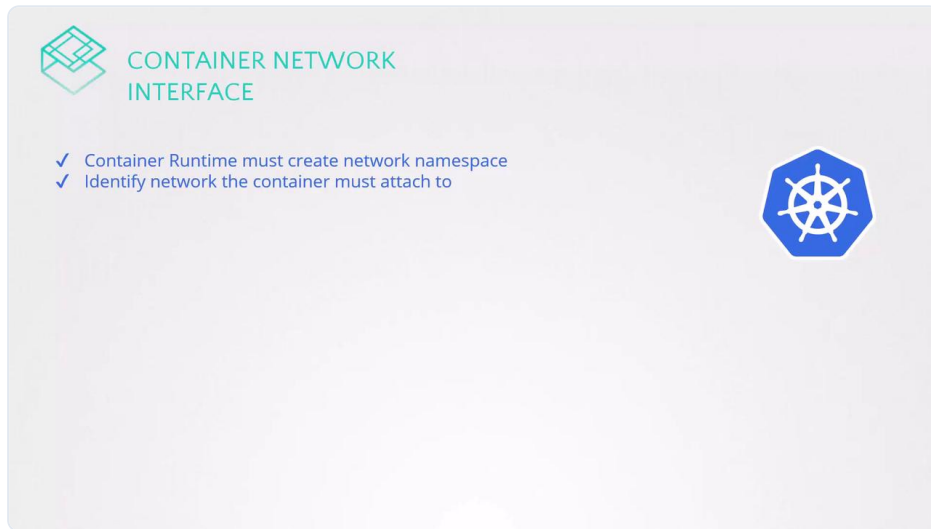
## Pre-Requisites

- ✓ Network Namespaces in Linux
- ✓ Networking in Docker
- ✓ Why and what is Container Network Interface (CNI)?
- ✓ CNI Plugins

Now, we focus on how Kubernetes configures the use of these network plugins.

As discussed in previous lessons, the CNI specifies the responsibilities of the container runtime. In Kubernetes, container runtimes such as Containerd or CRI-O create the container network namespaces and attach them to the

correct network by invoking the appropriate network plugin. Although Docker was initially the primary container runtime, it has largely been replaced by Containerd as an abstraction layer.



## Configuring CNI Plugins in Kubernetes

When a container is created, the container runtime invokes the necessary CNI plugin to attach the container to the network. Two common runtimes that demonstrate how this process works are Containerd and CRI-O.

Container runtimes look for CNI plugin executables in the `/opt/cni/bin` directory, while network configuration files are read from the `/etc/cni/net.d` directory.

### Directory Structure for CNI Plugins and Configuration

The network plugins reside in `/opt/cni/bin`, and the configuration files that dictate which plugin to use are stored in `/etc/cni/net.d`. Typically, the container runtime selects the configuration file that appears first in alphabetical order.

For example, you might see the following directories:

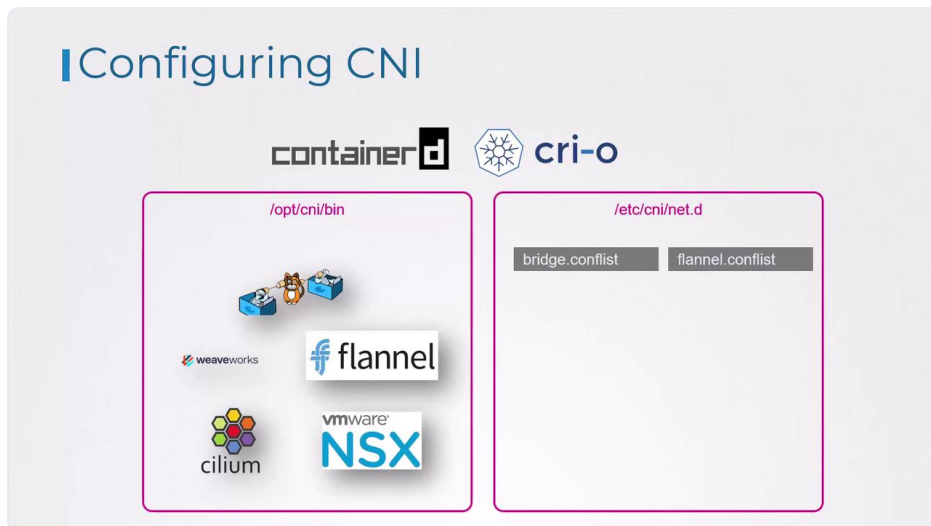
```
```bash theme={null} ls /opt/cni/bin
```

```
```bash theme={null}
bridge dhcp flannel host-local ipvlan loopback macvlan portmap ptp sample tuning
```

```
```bash theme={null} ls /etc/cni/net.d
```

```
```bash theme={null}
10-bridge.conflist
```

In this case, the container runtime chooses the "bridge" configuration file.



## Understanding a CNI Bridge Configuration File

A typical CNI bridge configuration file, adhering to the CNI standard, might look like this:

```
```bash theme={null} cat /etc/cni/net.d/10-bridge.conf
```

```
```json theme={null}
{
 "cniVersion": "0.2.0",
 "name": "mynet",
 "type": "bridge",
 "bridge": "cni0",
 "isGateway": true,
 "ipMasq": true,
 "ipam": {
 "type": "host-local",
 "subnet": "10.22.0.0/16",
 "routes": [
 { "dst": "0.0.0.0/0" }
]
 }
}
```

In this configuration:

- The `"name"` field (e.g., `"mynet"`) represents the network name.
- The `"type"` field set to `"bridge"` indicates the use of a bridge plugin.
- The `"bridge"` field (e.g., `"cni0"`) specifies the network bridge's name.
- The `"isGateway"` flag designates whether the bridge interface should have an IP address to function as a gateway.
- The `"ipMasq"` option enables network address translation (NAT) through IP masquerading.

- The `"ipam"` (IP Address Management) section uses `"host-local"` to allocate IP addresses from the specified subnet ( `"10.22.0.0/16"` ) and defines a default route.

Understanding these configuration fields is crucial for troubleshooting and optimizing Kubernetes networking. The settings in this bridge configuration align with fundamental networking concepts such as bridging, routing, and NAT masquerading.

This concludes our lesson on configuring CNI plugins with Kubernetes. We encourage you to apply these concepts through practical exercises to strengthen your Kubernetes networking skills.

For more information on related topics, consider reviewing:

- [Kubernetes Basics](#)
- [Kubernetes Documentation](#)
- [Docker Hub](#)

Happy networking!

## Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

# Service Networking

This article explains service networking in Kubernetes, covering pod and service communication, service types, and how traffic is managed within a cluster.

Welcome to this comprehensive lesson on service networking. In this discussion, we explore how services function within a Kubernetes cluster and how they differ from pods. We'll also cover the inner workings of pod networking, including bridging, namespace assignments, interface attachments, and IP address management. This guide is designed to clarify how traffic flows through services to access applications running in pods.

## Understanding Pod and Service Networking

Pod networking involves creating bridge networks on each node where pods receive individual namespaces, attach network interfaces, and obtain IP addresses from the node-specific subnet. This setup ensures that pods across different nodes can communicate via dedicated routes or overlay networks. However, rather than configuring pod-to-pod communication directly, you typically set up services. Services allow you to expose and reach applications with an assigned IP address and DNS name, simplifying internal and external access.

### Recap: Types of Services

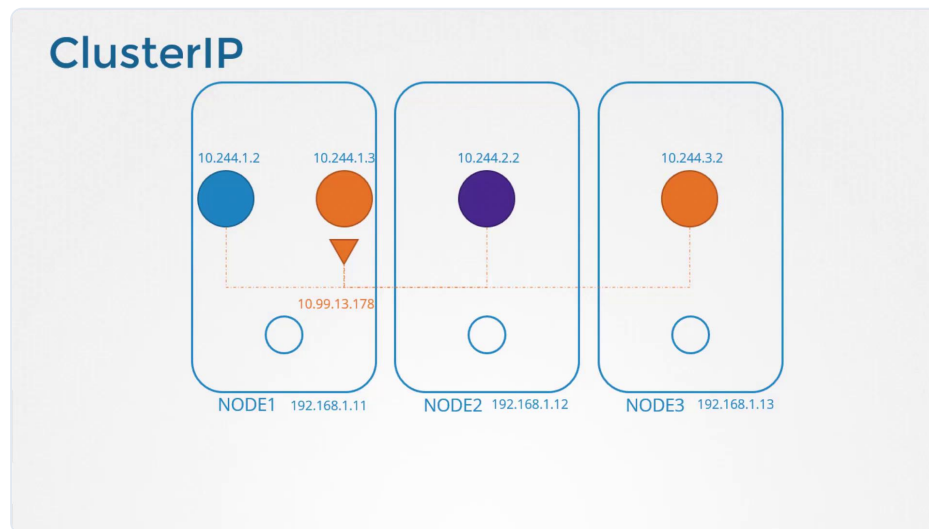
Imagine you have an "orange" pod that must be accessible to a "blue" pod. Instead of connecting the pods directly, you expose the orange pod with a service. This service is provided an IP address and DNS name, which the blue pod can use to establish a connection. When both pods reside on the same node, communication is straightforward; however, the process becomes more intricate when pods are distributed across multiple nodes.



## Service Types: ClusterIP vs. NodePort

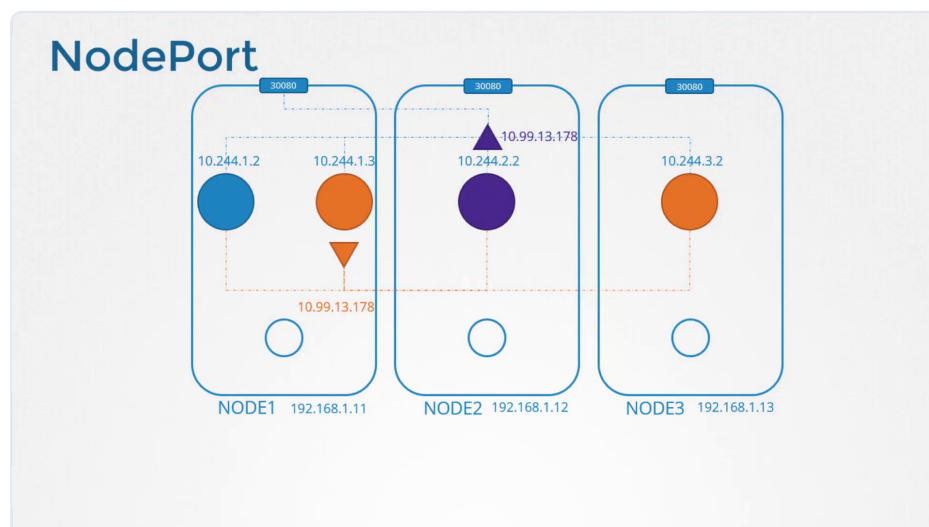
### ClusterIP Services

By default, when a service is created, it is accessible to all pods in the cluster, regardless of the node they run on. This default service type, called ClusterIP, is ideal for applications meant strictly for internal access. For example, if your orange pod hosts a database, a ClusterIP service ensures internal connectivity.



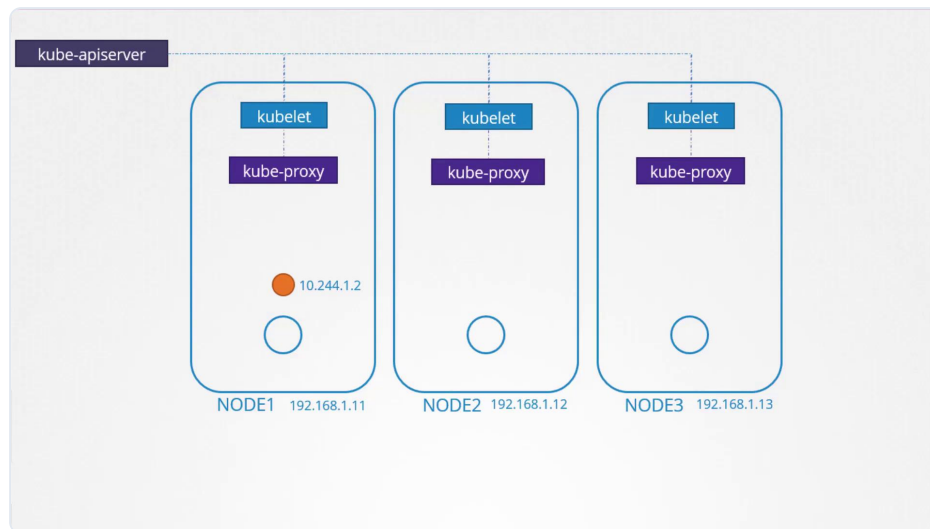
### NodePort Services

When you need a service to be accessible from outside the cluster, such as when a purple pod hosts a web application, you use a NodePort service. This service still receives an internal cluster IP for intra-cluster communication, but it also exposes the application on a specific port on every node. This allows external users or applications to reach the service without direct access to individual pod IPs.

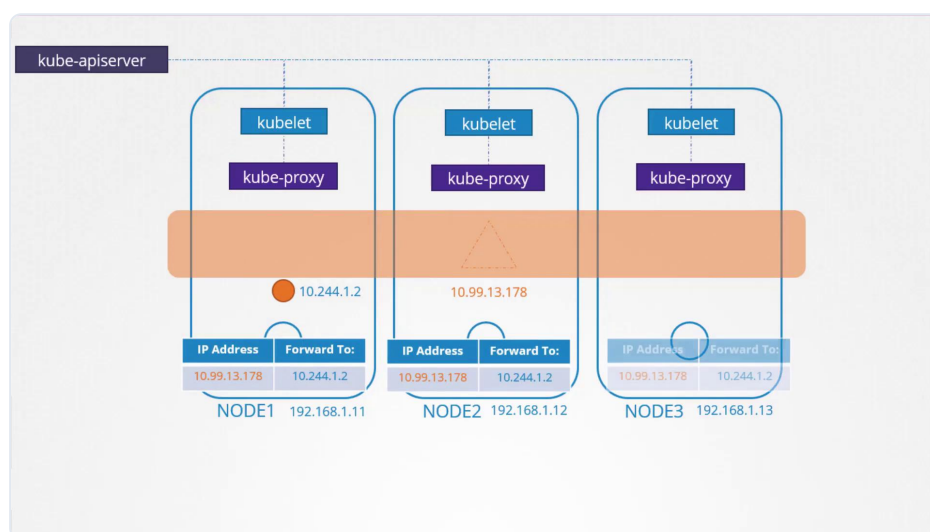


## How Kubernetes Manages Service Networking

Let's start with a clean slate. Picture a three-node Kubernetes cluster that has no pods or services deployed yet. Each node runs a Kubelet process, responsible for pod creation by communicating with the Kube API Server and invoking the CNI plugin to configure networking. In parallel, every node also runs a Kube Proxy, which monitors cluster changes via the API Server and sets up forwarding rules whenever a new service is created.



Unlike pods that have their dedicated network interfaces, services are virtual constructs spanning the entire cluster. When you create a service object, Kubernetes automatically assigns it an IP address from a predefined range (set via the Kube API Server's `--service-cluster-ip-range` option). The kube-proxy on each node then configures the appropriate forwarding rules so that any traffic targeted at the service's IP and port is correctly routed to the backend pod.



Whenever a pod sends traffic to a service IP, the kube-proxy rules step in to forward that traffic to the chosen backend pod. These rules are dynamically updated as services are created or removed. Kube Proxy supports several

proxy modes, including user space, iptables, and IPVS (with iptables being the default unless specified otherwise).

Here's an example command to start the kube-proxy with a specific proxy mode:

```
```bash theme={null} kube-proxy --proxy-mode [userspace | iptables | ipvs] ...
```

<Callout icon="lightbulb" color="#1CB2FE">

By default, kube-proxy uses iptables mode. Ensure your configuration specifies the intended proxy mode.
</Callout>

Practical Example: Service IP Assignment and Traffic Forwarding

Consider a scenario where a pod named "db" is running on node-1 with the IP address 10.244.1.2.

```
```bash theme={null}
```

```
kubectl get pods -o wide
```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE
db	1/1	Running	0	14h	10.244.1.2	node-1

```
kubectl get service
```

NAME	TYPE	CLUSTER-IP	PORT(S)	AGE
db-service	ClusterIP	10.103.132.104	3306/TCP	12h

In some clusters, the default service cluster IP range might be 10.0.0.0/24, but in other setups the API Server may be configured with a different range such as --service-cluster-ip-range=10.96.0.0/12. This means that service IPs can fall anywhere between 10.96.0.0 and 10.111.255.255. Meanwhile, the pod network CIDR might be something like 10.244.0.0/16, ensuring that pod IP addresses range between 10.244.0.0 and 10.244.255.255. It is crucial that these ranges do not overlap.

When the "db-service" is created, kube-proxy establishes corresponding iptables rules so that traffic arriving at the service IP on port 3306 is redirected to the pod's IP on the same port:

```
```bash theme={null} iptables -L -t nat | grep db-service KUBE-SVC-
XA5OGUC7YRHOS3PU tcp -- anywhere 10.103.132.104 / default/db-service:
cluster IP / tcp dpt:3306 DNAT tcp -- anywhere anywhere / default/db-service: /
tcp to:10.244.1.2:3306 KUBE-SEP-JBWCWHHQM57V2WN7 all -- anywhere
anywhere / default/db-service: /
```

```
This DNAT (Destination Network Address Translation) rule ensures that any traffic to the service IP is forwarded to the pod IP.  
Similarly, a NodePort service would have iptables rules that forward traffic arriving on a NodePort to the pod IP.  
  
## Verifying kube-proxy Operation  
  
To check which proxy mode your kube-proxy is using, you can inspect the kube-proxy logs:  
  
```bash theme={null}  
cat /var/log/kube-proxy.log
```

Note that the location of the kube-proxy log file may vary depending on your installation. If you do not see the expected entries, verify that the verbosity level of kube-proxy is set appropriately.

## Conclusion

---

This lesson has provided a detailed overview of service networking in Kubernetes, explaining how services are assigned IP addresses, how kube-proxy forwards traffic, and the differences between ClusterIP and NodePort service types. Further details and related topics will be explored in subsequent content.

Happy learning!

For more information, check out the following resources:

- [Kubernetes Documentation](#)
- [Kubernetes Basics](#)
- [Terraform Registry](#)

## Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

# DNS in kubernetes

This article explains how DNS is managed in Kubernetes, covering service and pod DNS records and their role in pod communication.

Welcome to this comprehensive guide on how DNS is managed within a Kubernetes cluster. In this article, we explore the mechanisms behind both service and pod DNS records, along with practical examples for enabling communication between pods. Before diving in, ensure you are familiar with the basics of DNS. If you're new to DNS concepts, please review the prerequisites below.

## I Pre-Requisite

- ✓ What is DNS?
- ✓ Host/NS Lookup, Dig utility
- ✓ Recorded Types – A, CNAME
- ✓ Domain Name Hierarchy

Previously, we covered the fundamentals of DNS, including common tools such as `host`, `nslookup`, and `dig` alongside various DNS record types (A, CNAME, etc.) and the domain name hierarchy. We even demonstrated how to set up your own DNS server using CoreDNS. Now, we shift our focus to the DNS names assigned to various Kubernetes objects—like services and pods—and the different methods of accessing one pod from another.

## Objectives

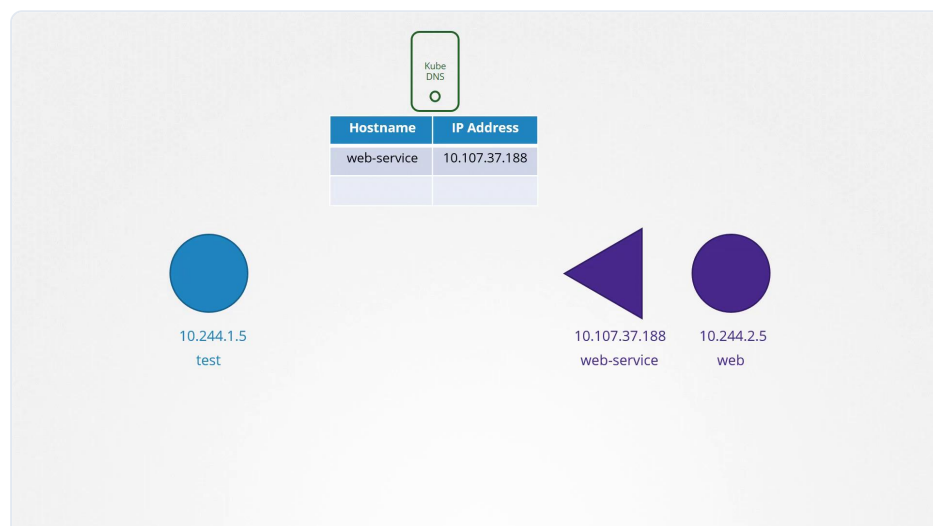
- ☐ What names are assigned to what objects?
- ☐ Service DNS records
- ☐ POD DNS Records

Imagine a three-node Kubernetes cluster with multiple pods and services distributed across them. Each node typically has a unique name and IP address registered in your organization's DNS server. However, our focus here is on the internal DNS resolution among the cluster's pods and services. By default, when you create a cluster, Kubernetes deploys a built-in DNS server (unless manually configured otherwise), which facilitates name resolution for pods and services.

Consider a simple scenario with two pods and a service in your cluster:

- A **test pod** with IP `10.244.1.5`.
- A **web pod** with IP `10.244.2.5`.

Even if these pods reside on different nodes (as indicated by their IP addresses), Kubernetes DNS assumes that all pods and services can be reached via their IP addresses. To allow the test pod to communicate with the web pod, a service named **web-service** is created. This service is assigned its own IP address (e.g., `10.107.37.188`) and automatically gets a DNS record mapping the service name to its IP.



Within the cluster, any pod can resolve and access the web service using its service name. For example, to access the web-service from the test pod, you could use:

```
``bash theme={null} curl http://web-service
```

## Output: Welcome to NGINX!

---

Earlier, we discussed namespaces in Kubernetes. Remember that pods within the same namespace

<Frame>  
! [The image illustrates two namespaces, each containing different individuals named Mark,  
</Frame>

In our scenario, because the test pod, web pod, and web-service are all in the **default**

To illustrate DNS resolution with namespaces, consider the following examples:

```
``bash theme={null}
When the service is in the default namespace
curl http://web-service
When the service resides in the 'apps' namespace
curl http://web-service.apps
Using the fully qualified domain name (FQDN)
curl http://web-service.apps.svc.cluster.local
Output: Welcome to NGINX!
```

Each namespace in Kubernetes gets its own subdomain. All services within that namespace are grouped under a subdomain called "svc." Additionally, the entire cluster is associated with a root domain (by default, `cluster.local`). Thus, the fully qualified domain name for a service in the "apps" namespace is:

`web-service.apps.svc.cluster.local`

Now, let's discuss pod DNS records. By default, DNS records for pods are not created. However, this behavior can be explicitly enabled. When pod DNS records are activated, Kubernetes generates a DNS record for each pod by converting the pod's IP address into a hostname—replacing dots ( `.` ) with dashes ( `-` ). The record includes the pod's namespace, is set to type "pod," and utilizes the cluster's root domain.

For example, if a test pod in the default namespace has the IP `10.244.2.5`, the corresponding DNS record becomes:

`10-244-2-5.apps.pod.cluster.local`

This DNS entry resolves to the pod's IP address. You can test the resolution with the command below:

```
``bash theme={null} curl http://10-244-2-5.apps.pod.cluster.local
```

# Output: Welcome to NGINX!

---

...

For more detailed information on Kubernetes DNS and other concepts, consider reviewing the following resources:

- [Kubernetes Basics](#)
- [Kubernetes Documentation](#)
- [CoreDNS Documentation](#)

By understanding these DNS concepts, you can better manage communication within your Kubernetes cluster and ensure reliable service discovery in your environment.



## Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

# CoreDNS in Kubernetes

This lesson covers how Kubernetes uses CoreDNS for DNS resolution to facilitate communication between pods and services within a cluster.

Welcome to this lesson on CoreDNS in Kubernetes. In this guide, you will learn how Kubernetes implements DNS resolution within a cluster to facilitate seamless communication between pods and services.

In our previous lesson, we explored how to address a service or pod from another pod. Now, we will explain how Kubernetes leverages a centralized DNS server to achieve that functionality.

Imagine you have two pods with different IP addresses. One approach to enable communication between them is to add an entry into each pod's hosts file. For instance, on the first pod, you might map the second pod (named "web") to IP 10.244.2.5, and on the second pod, map the first pod (named "test") to IP 10.244.1.5. However, when dealing with thousands of pods that are frequently created and removed, manually managing these entries becomes impractical.

Instead of manually editing hosts files, Kubernetes deploys a central DNS server. Each pod is pre-configured via its `/etc/resolv.conf` file to use this centralized server (typically at 10.96.0.10), which automatically updates DNS records for new pods and services.

Kubernetes does not create DNS entries for individual pods manually. Instead, it sets up DNS records for services, and for pods, it converts IP addresses into hostnames by replacing dots with dashes.

Before Kubernetes version 1.12, this service was known as Kube-DNS. Starting with version 1.12, however, the recommended DNS server is CoreDNS, which brings enhanced flexibility and performance. Below is a conceptual illustration showing how pods configure their `/etc/resolv.conf` to point to the DNS server:

```
```bash theme={null} cat /etc/resolv.conf nameserver 10.96.0.10
```

CoreDNS Setup in the Cluster

CoreDNS is deployed as a pod within the kube-system namespace. To ensure high availability,

CoreDNS requires a configuration file—commonly named "Corefile" and located at /etc/coredns

```
```bash theme={null}
cat /etc/coredns/Corefile
.:53 {
 errors
 health
 kubernetes cluster.local in-addr.arpa ip6.arpa {
 pods insecure
 upstream
 fallthrough in-addr.arpa ip6.arpa
 }
 prometheus :9153
 proxy . /etc/resolv.conf
 cache 30
 reload
}
```

This configuration performs the following functions:

- Logs and handles errors.
- Provides health check endpoints.
- Integrates with Kubernetes via the Kubernetes plugin, configuring the primary domain to cluster.local and transforming pod IP addresses into a dashed hostname format.
- Exposes Prometheus metrics for monitoring.
- Forwards unresolved DNS queries (such as [www.google.com](http://www.google.com)) to the nameserver specified in the pod's /etc/resolv.conf.
- Caches DNS responses and supports dynamic reloads of the configuration upon changes.

Note that this configuration is stored in a ConfigMap. If adjustments are needed, simply update the ConfigMap:

```
```bash theme={null} kubectl get configmap -n kube-system NAME DATA AGE
coredns 1 168d
```

Once the CoreDNS pod is running with the correct configuration, it continuously monitors the

DNS Service and Pod Configuration

To enable pods to communicate with the CoreDNS server, Kubernetes creates a service (named

```
```bash theme={null}
kubectl get service -n kube-system
NAME TYPE CLUSTER-IP EXTERNAL-IP PORT(S) AGE
kube-dns ClusterIP 10.96.0.10 <none> 53/UDP,53/TCP 1d
```

A typical pod's /etc/resolv.conf file will appear as:

```
```bash theme={null} cat /etc/resolv.conf nameserver 10.96.0.10 search
default.svc.cluster.local svc.cluster.local cluster.local
```

This configuration is automatically managed by the Kubelet. If you inspect the Kubelet con

```
```bash theme={null}
cat /var/lib/kubelet/config.yaml
...
clusterDNS:
- 10.96.0.10
clusterDomain: cluster.local
```

## Resolving Services and Pods

With the correct DNS configuration, pods can resolve services using different domain name formats. For example, if you have a web service deployed in your cluster, you can access it using any of the following names:

- web-service
- web-service.default
- web-service.default.svc
- web-service.default.svc.cluster.local

These examples are demonstrated in the commands below:

```
```bash theme={null} cat /etc/resolv.conf nameserver 10.96.0.10 curl http://web-
service curl http://web-service.default curl http://web-service.default.svc curl
http://web-service.default.svc.cluster.local
```

You can also use tools like nslookup or host to check the fully qualified domain name:

```
```bash theme={null}
host web-service
web-service.default.svc.cluster.local has address 10.107.37.188
```

The search entries in /etc/resolv.conf allow the resolver to append the listed domains when you use partial names. However, individual pod names must always be resolved with their complete fully qualified domain name (FQDN).

For example:

```
```bash theme={null} cat /etc/resolv.conf nameserver 10.96.0.10 search
default.svc.cluster.local svc.cluster.local cluster.local
```

```
host web-service web-service.default.svc.cluster.local has address
10.107.37.188
```

```
host 10-244-2-5 Host 10-244-2-5 not found: 3(NXDOMAIN)
```

```
host 10-244-2-5.default.pod.cluster.local 10-244-2-5.default.pod.cluster.local
has address 10.244.2.5 ``
```

The search entries in the `/etc/resolv.conf` file simplify service resolution by allowing shorter names. However, pod-specific DNS records require their full FQDN for proper resolution.

Conclusion

By deploying CoreDNS and configuring all pods to use the centralized DNS service, Kubernetes enables dynamic and automated DNS resolution for both services and pods. This robust setup eliminates the need for manual host file configuration, thereby streamlining communication across the cluster.

Revisit and practice these concepts to deepen your understanding of managing DNS within a Kubernetes environment. Happy learning!

Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

Ingress

This article provides a comprehensive guide on Kubernetes Ingress, covering its configuration, deployment, and use cases for managing external access to applications.

Welcome to this comprehensive guide on Kubernetes Ingress. In this article, we will explore the differences between Services and Ingress, explain when to use each, and demonstrate how to deploy and configure Ingress resources effectively. We begin by revisiting Services and then build toward understanding Ingress.

Background: Using Services for External Access

Imagine you are deploying an application for a company with an online store at myonlinestore.com. You build your application as a Docker image and deploy it on a Kubernetes cluster as a pod within a Deployment. The application requires a database, so you deploy a MySQL pod and expose it with a ClusterIP Service named MySQL service. Internally, your application functions properly.

To expose the application externally, you create a Service of type NodePort, mapping the application to a high port (e.g., 38080) on your cluster nodes. In this configuration, users access your application using a URL like:

`http://1:38080`

As traffic increases, the Service load-balances the requests among multiple application pods.

In a production environment, however, you likely want users to access your application through a user-friendly domain name instead of a node IP address with a high port number. To achieve this, you would update your DNS configuration to point to your node IPs and deploy a proxy server that forwards requests from standard port 80 (or 443 for HTTPS) on your DNS to the

NodePort defined in your cluster. With this approach, users can simply navigate to myonlinestore.com.

Cloud-Native Approach with LoadBalancer

If your application is hosted on a public cloud platform like [Google Cloud Platform \(GCP\)](#), the process can be simplified further. Instead of creating a NodePort Service, you can deploy a Service of type LoadBalancer. In this setup:

- Kubernetes assigns an internal high port.
- It sends a request to GCP to deploy a network load balancer.
- The cloud load balancer routes traffic to the internal port across all nodes.
- An external IP is provided by the load balancer, which you point your DNS to.

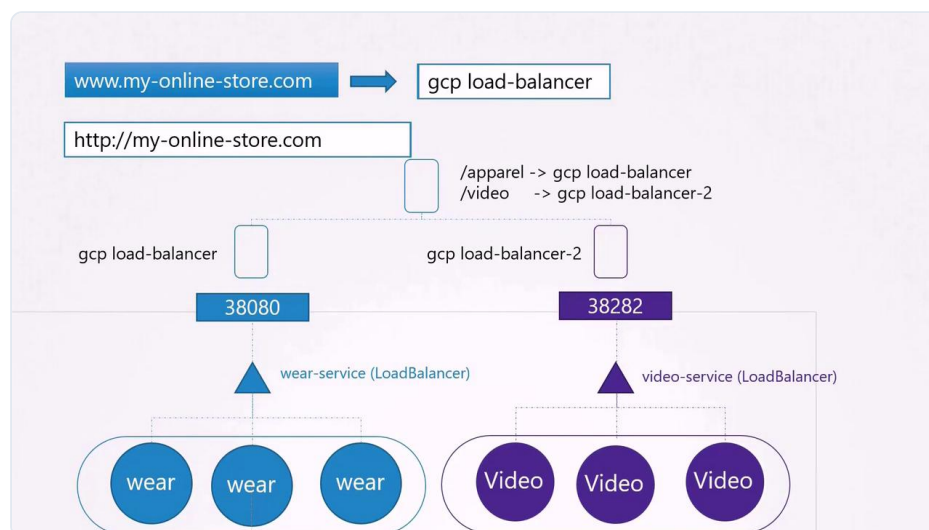
This allows users to access your application directly using myonlinestore.com.

The Need for Ingress

Imagine your company expands and you launch new services. For example, you might offer:

- A video streaming service on myonlinestore.com/watch
- The original application on myonlinestore.com/wear

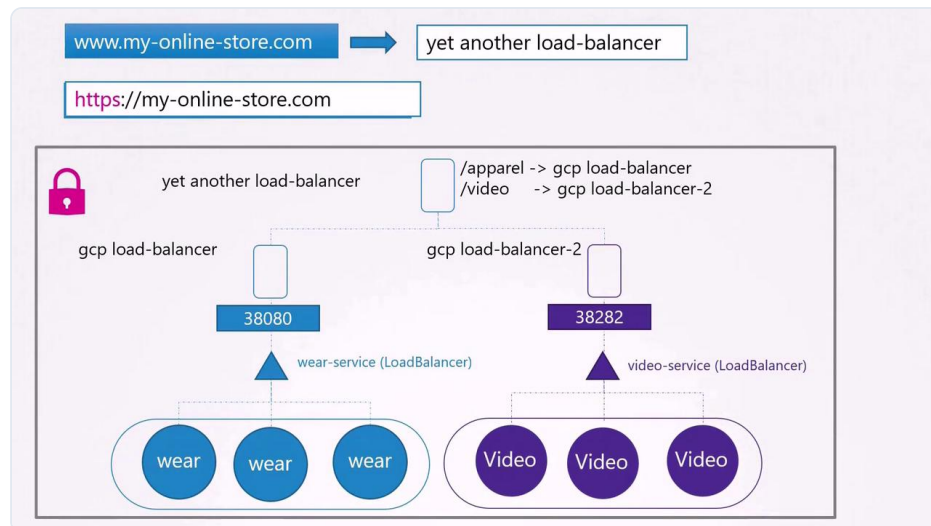
Even if both applications run within the same cluster with separate Deployments and Services (e.g., a LoadBalancer Service for the video service), each service might get its own high port and cloud load balancer. Managing multiple load balancers can increase costs, add complexity, and complicate SSL/TLS (HTTPS) configurations.



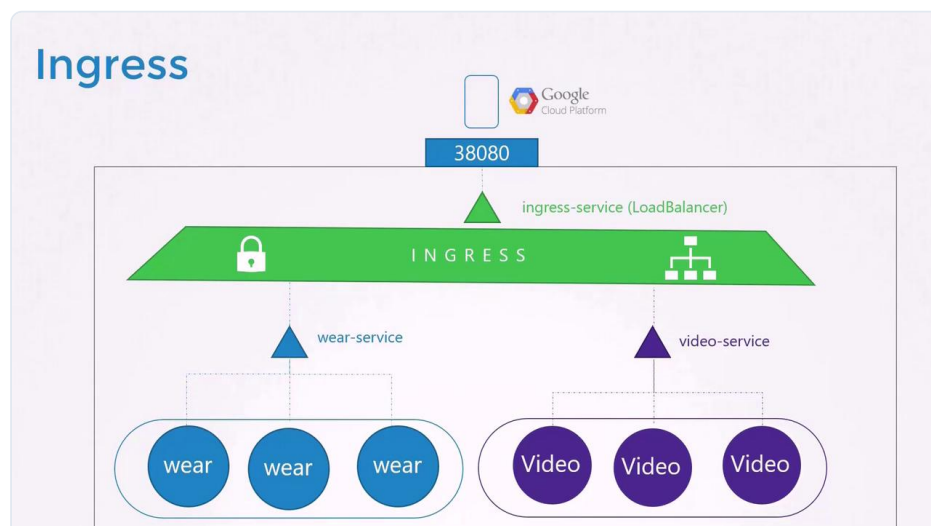
Introducing Ingress

Ingress simplifies external access by providing a single externally accessible IP for your Kubernetes applications. It allows you to configure URL-based routing rules, SSL termination, authentication, and more—acting as a built-in layer 7 load balancer.

Even with Ingress, you still need an initial exposure mechanism (via NodePort or a cloud-native LoadBalancer). However, once this is set up, all further changes are made through the Ingress controller.

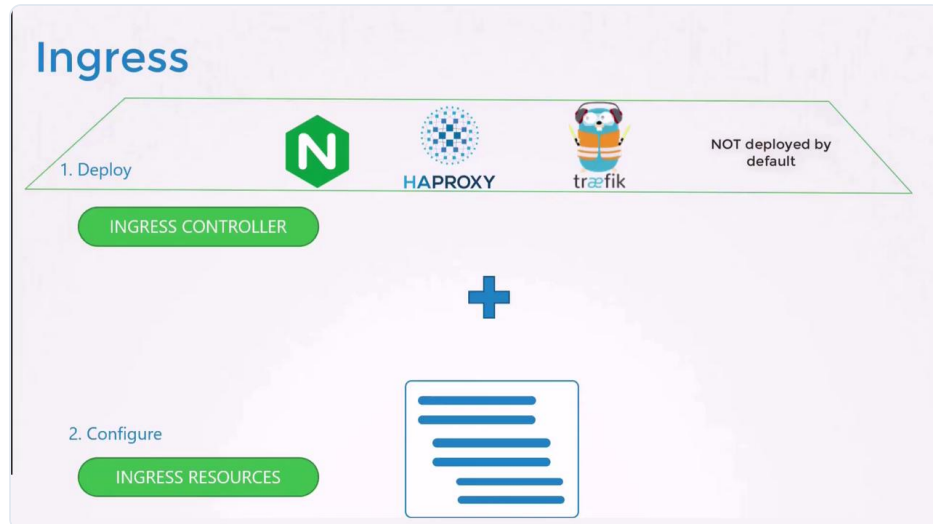


Without Ingress, you would have to manually deploy and configure a reverse proxy or load balancer (such as NGINX, HAProxy, or Traefik) within your cluster to handle URL routing and manage SSL certificates. Ingress builds on these principles to offer an integrated solution.



Deploying an Ingress Controller

To use Ingress, you must first deploy an Ingress controller. The controller continuously monitors the cluster for changes in Ingress resources and reconfigures the underlying load balancing solution accordingly.



A Kubernetes cluster does not include an Ingress controller by default. If you create Ingress resources without deploying an Ingress controller, they will have no effect.

There are several Ingress controllers available, including GCE, NGINX, Contour, HAProxy, Traefik, and Istio. In this guide, we will use NGINX as the example.

Deploying the NGINX Ingress Controller

Below is an example of an NGINX Ingress controller deployment:

```
```yaml
theme={null}
apiVersion: extensions/v1beta1
kind: Deployment
metadata:
 name: nginx-ingress-controller
spec:
 replicas: 1
 selector:
 matchLabels:
 name: nginx-ingress
 template:
 metadata:
 labels:
 name: nginx-ingress
 spec:
 containers:
 - name: nginx-ingress-controller
 image: quay.io/kubernetes-ingress-controller/nginx-ingress-controller:0.21.0
 args:
 - /nginx-ingress-controller
```



This Deployment creates one replica of the NGINX Ingress controller using a Kubernetes-optimized image.

To decouple configuration data from the image, you create a ConfigMap. This allows easy updates to the configuration without needing to rebuild the image.

Below is an example Service to expose the NGINX Ingress controller using NodePort:

```
```yaml
theme={null}
apiVersion: v1
kind: Service
metadata:
  name: nginx-ingress
spec:
  type: NodePort
  ports:
    - port: 80
      targetPort: 80
      protocol: TCP
      name: http
    - port: 443
      targetPort: 443
      protocol: TCP
      name: https
  selector:
    name: nginx-ingress
```
```

For a more comprehensive configuration, you can include environment variables to pass the Pod's name and namespace to the container. This enables the Ingress controller to load its configuration dynamically. Here is a complete configuration that includes the Deployment, Service, ConfigMap, and ServiceAccount:

```
```yaml
theme={null}
apiVersion: extensions/v1beta1
kind: Deployment
metadata:
  name: nginx-ingress-controller
spec:
  replicas: 1
  selector:
    matchLabels:
      name: nginx-ingress
  template:
    metadata:
      labels:
        name: nginx-ingress
    spec:
      containers:
        - name: nginx-ingress-controller
          image: quay.io/kubernetes-ingress-controller/nginx-ingress-controller:0.21.0
          args:
            - /nginx-ingress-controller
            - --configmap=$(POD_NAMESPACE)/nginx-configuration
          env:
            - name: POD_NAME
              valueFrom:
                fieldRef:
                  fieldPath: metadata.name
            - name: POD_NAMESPACE
              valueFrom:
                fieldRef:
                  fieldPath: metadata.namespace
          ports:
            - name: http
              containerPort: 80
            - name: https
              containerPort: 443
```
```

---

```
apiVersion: v1
kind: Service
metadata:
 name: nginx-ingress
spec:
 type: NodePort
 ports:
 - port: 80
 targetPort: 80
 protocol: TCP
 name: http
 - port: 443
 targetPort: 443
 protocol: TCP
 name: https
 selector:
 name: nginx-ingress
```

---

```
kind: ConfigMap
apiVersion: v1
metadata:
 name: nginx-configuration
```

---

```
apiVersion: v1
kind: ServiceAccount
metadata:
 name: nginx-ingress-serviceaccount
```

In addition to the objects above, you must create the necessary Roles, ClusterRoles, and Roles

## ## Creating Ingress Resources

Once the NGINX Ingress controller is deployed, you can begin creating Ingress resources that

### ### Simple Ingress for a Single Service

The following Ingress resource routes all incoming traffic to a single backend Service name

```
```yaml theme={null}
apiVersion: extensions/v1beta1
kind: Ingress
metadata:
  name: ingress-wear
spec:
  backend:
    serviceName: wear-service
    servicePort: 80
```

To create this Ingress resource, run:

```
```bash theme={null} kubectl create -f Ingress-wear.yaml
```

You should see output similar to:

```
```bash theme={null}
ingress.extensions/ingress-wear created
```

Verify its creation with:

```
```bash theme={null} kubectl get ingress
```

Expected output:

```
```bash theme={null}
NAME          HOSTS      ADDRESS      PORTS      AGE
ingress-wear  *         <none>       80         2s
```

This configuration directs all traffic to "wear-service".

Ingress with Multiple URL Paths

For more complex routing—such as directing traffic from different URL paths to different backend Services—use Ingress rules. Suppose you want:

- Traffic to myonlinestore.com/wear to go to "wear-service"
- Traffic to myonlinestore.com/watch to go to "watch-service"

Define an Ingress resource as follows:

```
```yaml theme={null} apiVersion: extensions/v1beta1 kind: Ingress metadata:
name: ingress-wear-watch spec: rules: - http: paths: - path: /wear backend:
```

```
serviceName: wear-service servicePort: 80 - path: /watch backend:
serviceName: watch-service servicePort: 80
```

After creating this resource, you can view its details by running:

```
```bash theme={null}
kubectl describe ingress ingress-wear-watch
```

The output will display the rules and backend configurations similar to:

```
```bash theme={null} Name: ingress-wear-watch Namespace: default Address:
Default backend: default-http-backend:80 () Rules: Host Path Backends ---- ----
----- * /wear wear-service:80 () /watch watch-service:80 () Annotations:
Events: Type Reason Age From Message ---- -
CREATE 14s nginx-ingress-controller Ingress default/ingress-wear-watch
```

If a user visits an undefined URL (e.g., myonlinestore.com/listen), you can configure a default backend to handle the request.

### ### Ingress Based on Host Names

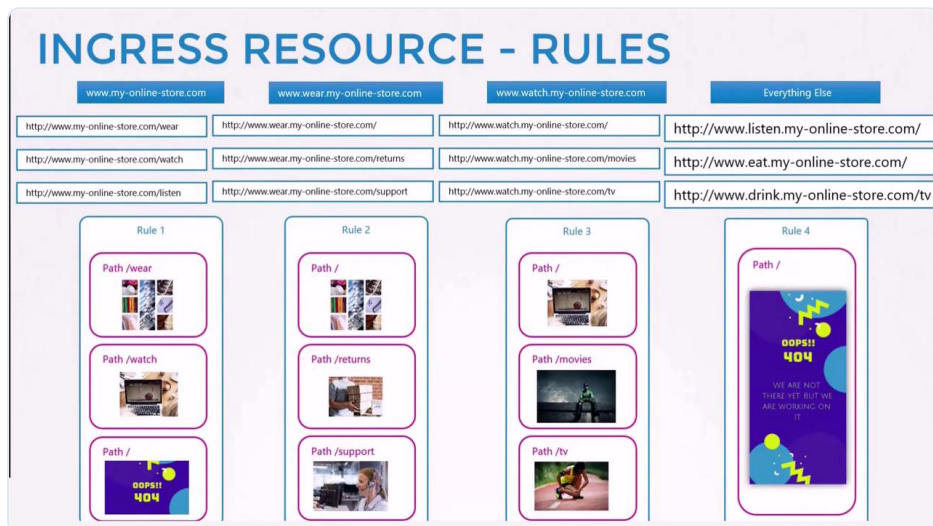
Another common scenario involves routing traffic based on host names. For example, you might want to route traffic for myonlinestore.com to a primary service and traffic for www.myonlinestore.com to a secondary service.

- \* Traffic for myonlinestore.com to go to "primary-service"
- \* Traffic for [www.myonlinestore.com](http://www.myonlinestore.com) to go to "secondary-service"

Here's how you can define the Ingress resource with host-specific rules:

```
```yaml theme={null}
apiVersion: extensions/v1beta1
kind: Ingress
metadata:
  name: ingress-domain-routing
spec:
  rules:
    - host: myonlinestore.com
      http:
        paths:
          - path: /
            backend:
              serviceName: primary-service
              servicePort: 80
    - host: www.myonlinestore.com
      http:
        paths:
          - path: /
            backend:
              serviceName: secondary-service
              servicePort: 80
```

In this example, each rule handles traffic for a specific domain by routing all requests ("/") to the designated backend Service.



Splitting traffic by URL involves one rule with multiple paths, whereas splitting traffic by domain requires multiple rules with specific host fields. If you do not define a host, the Ingress rule will match all incoming traffic, regardless of the domain.

Conclusion

This guide has walked through the process of setting up and configuring Ingress in Kubernetes. We explored a variety of use cases—from simple backend routing to complex rules based on URL paths and host names. The Ingress controller simplifies external access to your cluster, helps manage SSL/TLS termination, and consolidates your load balancing configuration.

In practice tests or production environments, you may encounter labs where the Ingress controller and applications are pre-deployed, or more challenging labs where you need to deploy the Ingress controller and resources from scratch.

Good luck with your Kubernetes projects, and we hope this article has been helpful in advancing your understanding of Kubernetes Ingress.

For additional information, consider reviewing:

- [Kubernetes Documentation](#)
- [Kubernetes Basics](#)
- [NGINX Ingress Controller Repository](#)

Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

Introduction to Gateway API 2025 Updates

This article introduces the Gateway API and its updates, addressing Ingress limitations and enhancing traffic management capabilities in Kubernetes.

In this article, we introduce the Gateway API and explain how it addresses the challenges posed by the traditional Ingress resource. Previously, when using Ingress, multiple teams or organizations sharing a single Ingress resource faced coordination challenges. For instance, if team A manages a web service and team B manages a video service, they would need to coordinate their changes on one Ingress resource. This multi-tenancy issue is a significant limitation of Ingress, which can only be managed by one team at a time.

Limitations of Ingress

Consider the basic Ingress configuration below, which routes traffic based on host names:

```
```yaml theme={null}
```

## ingress-wear-watch.yaml

```
apiVersion: extensions/v1beta1 kind: Ingress metadata: name: ingress-wear-watch spec: rules: - host: wear.my-online-store.com http: paths: - backend: serviceName: wear-service servicePort: 80 - host: watch.my-online-store.com http: paths: - backend: serviceName: watch-service servicePort: 80
```

This configuration supports HTTP-based rules such as host and path matching. However, it does not support HTTPS-based rules.

```

```yaml theme={null}
# ingress-with-annotations.yaml
apiVersion: extensions/v1beta1
kind: Ingress
metadata:
  name: ingress-wear-watch
  annotations:
    nginx.ingress.kubernetes.io/ssl-redirect: "true"
    nginx.ingress.kubernetes.io/force-ssl-redirect: "true"
spec:
  rules:
  - host: wear.my-online-store.com
    http:
      paths:
      - path: /foo
        backend:
          serviceName: wear-service
          servicePort: 80
  - host: watch.my-online-store.com
    http:
      paths:
      - backend:
          serviceName: watch-service
          servicePort: 80

```

Because each controller implements its own set of annotations, these configurations are tightly coupled to specific controllers and cannot be validated by Kubernetes itself.

Similarly, advanced scenarios like configuring Cross-Origin Resource Sharing (CORS) require different annotations depending on the controller. For example, a CORS configuration for NGINX might look like this:

```

```yaml theme={null}

```

## Ingress-cors.yaml (for NGINX)

```

apiVersion: networking.k8s.io/v1 kind: Ingress metadata: name: cors-ingress
annotations:
 nginx.ingress.kubernetes.io/enable-cors: "true"
 nginx.ingress.kubernetes.io/cors-allow-methods: "GET, PUT, POST"
 nginx.ingress.kubernetes.io/cors-allow-origin: "https://allowed-origin.com"
 nginx.ingress.kubernetes.io/cors-allow-credentials: "true"

```

And for a traffic controller like Traefik, a similar configuration could be:

```
```yaml theme={null}
# Ingress-traefik.yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: traefik-ingress
  annotations:
    # CORS Configuration
    traefik.ingress.kubernetes.io/headers.customresponseheaders: |
      Access-Control-Allow-Origin: '*'
      Access-Control-Allow-Methods: GET, POST, PUT, DELETE, OPTIONS
      Access-Control-Allow-Headers: Content-Type, Authorization
      Access-Control-Allow-Credentials: true
      Access-Control-Max-Age: 3600
```

These examples clearly demonstrate that the same use case leads to different configurations based solely on the chosen controller.

Introducing the Gateway API

The Gateway API was created as an official Kubernetes project to overcome the limitations of Ingress. It supports both layer 4 (transport) and layer 7 (application) routing, representing the next generation of load balancing and service mesh APIs. By decoupling responsibilities, the Gateway API introduces three distinct objects:

1. **Gateway Class:** Configured by infrastructure providers to define the underlying network infrastructure (e.g., NGINX, Traefik, or other load balancers).
2. **Gateway:** Managed by cluster operators; these are instances built from a Gateway Class.
3. **HTTPRoute (and other route types):** Managed by application developers; these routes support various protocols such as HTTP, TCP, and gRPC.

Unlike Ingress, the Gateway API offers a unified and declarative configuration that is independent of controller-specific annotations.

Gateway API Configuration Example

The following example illustrates how to define a Gateway Class, a Gateway, and an HTTPRoute using the Gateway API:

```
```yaml theme={null}
```

## gateway-class.yaml

```
apiVersion: gateway.networking.k8s.io/v1 kind: GatewayClass metadata:
name: example-class spec: controllerName: example.com/gateway-controller
```

```
```yaml theme={null}
# gateway.yaml
apiVersion: gateway.networking.k8s.io/v1
kind: Gateway
metadata:
  name: example-gateway
spec:
  gatewayClassName: example-class
  listeners:
    - name: http
      protocol: HTTP
      port: 80
```

```
```yaml theme={null}
```

## http-route.yaml

```
apiVersion: gateway.networking.k8s.io/v1 kind: HTTPRoute metadata: name:
example-httproute spec: parentRefs: - name: example-gateway hostnames: -
"www.example.com" rules: - matches: - path: type: PathPrefix value: /login
backendRefs: - name: example-svc port: 8080
```

```
In this configuration, the HTTPRoute routes requests with a path prefix of "/login" arriving

TLS Configuration with Gateway API

Traditional Ingress configurations handle TLS by using the `spec.tls` section, often accompani

```yaml theme={null}
# ingress-secure-app.yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: secure-app
  annotations:
    nginx.ingress.kubernetes.io/ssl-redirect: "true"
    nginx.ingress.kubernetes.io/force-ssl-redirect: "true"
spec:
  tls:
    - hosts:
        - secure.example.com
      secretName: tls-secret
```

In contrast, the Gateway API allows for a more structured TLS configuration as part of the listener definition:


```
```yaml theme={null}
```

## gateway-secure.yaml

---

```
apiVersion: gateway.networking.k8s.io/v1 kind: Gateway metadata: name:
secure-gateway spec: gatewayClassName: example-gc listeners: - name:
https port: 443 protocol: HTTPS tls: mode: Terminate certificateRefs: - kind:
Secret name: tls-secret allowedRoutes: kinds: - kind: HTTPRoute
```

Here, the HTTPS listener on port 443 is explicitly configured with TLS termination, referen

### ## Traffic Splitting and Canary Deployments

Ingress relies on annotations for complex use cases like canary deployments. For example, y

```
```yaml theme={null}
# canary-ingress.yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: canary-ingress
  annotations:
    nginx.ingress.kubernetes.io/canary: "true"
    nginx.ingress.kubernetes.io/canary-weight: "20"
spec:
  rules:
  - http:
      paths:
      - path: /
        pathType: Prefix
        backend:
          service:
            name: app-v2
            port:
              number: 80
```

The Gateway API simplifies this process by natively defining traffic splitting within an HTTPRoute:

```
```yaml theme={null}
```

## app-gateway.yaml

---

```
apiVersion: gateway.networking.k8s.io/v1 kind: HTTPRoute metadata: name:
split-traffic spec: parentRefs: - name: app-gateway rules: - backendRefs: -
name: app-v1 port: 80 weight: 80 - name: app-v2 port: 80 weight: 20
```

```

In this example, "app-v1" receives 80% of the traffic while "app-v2" handles the remaining

Centralized Advanced Configuration

For features like Cross-Origin Resource Sharing (CORS), the Gateway API centralizes configuration

```yaml theme={null}
# http-route-cors.yaml
apiVersion: gateway.networking.k8s.io/v1
kind: HTTPRoute
metadata:
  name: cors-route
spec:
  parentRefs:
    - name: my-gateway
  rules:
    - matches:
        - path:
            type: PathPrefix
            value: /api
      filters:
        - type: ResponseHeaderModifier
          responseHeaderModifier:
            add:
              - name: Access-Control-Allow-Origin
                value: "*"
              - name: Access-Control-Allow-Methods
                value: "GET, POST, PUT, DELETE, OPTIONS"
              - name: Access-Control-Allow-Headers
                value: "Content-Type,Authorization"
              - name: Access-Control-Allow-Credentials
                value: "true"
              - name: Access-Control-Max-Age
                value: "3600"
    backendRefs:
      - name: api-service

```

This self-contained configuration is consistent and works seamlessly across different Gateway API controllers.

Gateway API Controller Support

A range of major controllers now support, or are actively implementing, the Gateway API. This industry support includes platforms such as Amazon EKS, Azure Application Gateway for Containers, Contour, Envoy, Google Kubernetes Engine, HAProxy, Istio, Kong, Kuma, NGINX, and many others. Such broad adoption underlines the growing momentum and reliability of the Gateway API.

Gateway API

Object	OSI Layer	Routing Discriminator	TLS Support	Purpose
HTTPRoute	Layer 7	Anything in the HTTP Protocol	Terminated only	HTTP and HTTPS Routing
TLSRoute	Somewhere between layer 4 and 7	SNI or other TLS properties	Passthrough or Terminated	Routing of TLS protocols including HTTPS where inspection of the HTTP stream is not required.
TCPRoute	Layer 4	destination port	Passthrough or Terminated	Allows for forwarding of a TCP stream from the Listener to the Backends
UDPRoute	Layer 4	destination port	None	Allows for forwarding of a UDP stream from the Listener to the Backends.
GRPCRoute	Layer 7	Anything in the gRPC Protocol	Terminated only	gRPC Routing over HTTP/2 and HTTP/2 plaintext

Gateway Controller Implementation

Gateway Controller Implementation Status

- Acnodal EPIC
- Airolock Microgateway
- Amazon Elastic Kubernetes Service (GA)
- Apache APISIX (beta)
- Avi Kubernetes Operator (tech preview)
- Azure Application Gateway for Containers (GA)
- Cilium (beta)
- Contour (GA)
- Easegress (GA)
- Emissary-Ingress (Ambassador API Gateway) (alpha)
- Envoy Gateway (GA)
- Flomesh Service Mesh (beta)
- Gloo Gateway (GA)
- Google Kubernetes Engine (GA)

• HAProxy Ingress (alpha)

- HAProxy Kubernetes Ingress Controller (GA)
- HashiCorp Consul
- Istio (GA)
- Kong Ingress Controller (GA)
- Kong Gateway Operator (GA)
- Kuma (GA)
- LiteSpeed Ingress Controller
- LoxiLB (beta)
- NGINX Gateway Fabric (GA)
- ngrok (preview)
- STUNner (beta)
- Traefik Proxy (GA)
- Tyk (work in progress)
- WS02 APK (GA)

Conclusion

The Gateway API overcomes the limitations of Ingress by providing a more declarative, structured, and multi-protocol solution for traffic management. By segregating responsibilities among GatewayClass, Gateway, and HTTPRoute, it simplifies multi-tenant management and advanced routing scenarios such as TLS termination, traffic splitting, and CORS configuration.

For a deeper understanding of these concepts, try practicing with hands-on labs to experiment with different Gateway API configurations.

Section: Troubleshooting

Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/lms.txt> Use this file to discover all available pages before exploring further.

Troubleshooting Section

Introduction

This article explores troubleshooting techniques for diagnosing and resolving common issues in Kubernetes clusters through hands-on labs.

Welcome to our comprehensive Kubernetes troubleshooting guide. In this article, we explore various troubleshooting techniques designed for diagnosing and resolving common issues in Kubernetes clusters. We begin by addressing application failures, then move on to troubleshooting control plane challenges, followed by worker node issues, and finally, network-related problems.

This guide is structured as a series of hands-on labs. You'll work with a simulated broken cluster to practice identifying and resolving issues. This practical approach is intended to enhance your troubleshooting skills and boost your confidence in managing production environments.

Good luck with your troubleshooting efforts and happy learning!

Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

Application Failure

This article provides a guide on troubleshooting application failures in a two-tier architecture, focusing on systematic techniques and practical scenarios.

Welcome to this comprehensive guide on troubleshooting application failures. In this article, we review systematic troubleshooting techniques and provide practical scenarios to help you diagnose and resolve issues in a two-tier application architecture.

Overview

Consider a two-tier application consisting of a web server and a database server. The database pod hosts a database application and serves data to the web servers via a database service. Meanwhile, the web server, running on a separate pod, delivers content to the users through a web service.

Before you begin troubleshooting, document your application's configuration by creating a system map or listing all components and their connections. This documentation can be a valuable reference when determining where to start your investigation.

Troubleshooting the Web Application

When users report issues accessing your application, start by testing the application's front end. For example, if your application is web-based, verify the accessibility of the web server using a tool like curl on the service's IP address and node port:

```
```bash theme={null} curl http://web-service-ip:node-port curl: (7) Failed to connect to web-service-ip port node-port: Connection timed out
```

### ### Validate Service Endpoints

Ensure that the web service has correctly discovered endpoints for the web pod. A common is

View the service details with:

```
```bash theme={null}
bash
kubectl describe service web-service
```

You should see output similar to the following:

```
```bash theme={null} Name: web-service Namespace: default Labels:
Annotations: Selector: name=webapp-mysql Type: NodePort IP: 10.96.0.156
Port: 8080/TCP TargetPort: 8080/TCP NodePort: 31672/TCP Endpoints:
10.32.0.6:8080 Session Affinity: None External Traffic Policy: Cluster Events:
```

### ### Verify Pod Configuration

Review the web application's pod definition to ensure it is correctly configured and running

```
```yaml theme={null}
apiVersion: v1
kind: Pod
metadata:
  name: webapp-mysql
  labels:
    app: example-app
    name: webapp-mysql
spec:
  containers:
    - name: webapp-mysql
      image: simple-webapp-mysql
      ports:
        - containerPort: 8080
```

Next, check the pod status. A stable pod typically shows a running status with minimal restarts:

```
```bash theme={null} bash kubectl get pod
```

Example output:

```
```text theme={null}
NAME    READY    STATUS    RESTARTS    AGE
Web     1/1      Running   5            50m
```

For a more detailed inspection, including event logs, use:

```
```bash theme={null} bash kubectl describe pod web
```

This command may display events like:

```
```text theme={null}
```

```
...
```

Events:

Type	Reason	Age	From	Message
Normal	Scheduled	52m	default-scheduler	Successfully assigned webapp-mysql to worker-1
Normal	Pulling	52m	kubelet, worker-1	pulling image "simple-webapp-mysql"
Normal	Pulled	52m	kubelet, worker-1	Successfully pulled image "simple-webapp-mysql"
Normal	Created	52m	kubelet, worker-1	Created container
Normal	Started	52m	kubelet, worker-1	Started container

Review Application Logs

Examine the application logs with:

```
```bash theme={null} bash kubectl logs web
```

Expected output might look like this:

```
```text theme={null}
```

```
10.32.0.1 - - [01/Apr/2019 12:51:55] "GET / HTTP/1.1" 200 -
10.32.0.1 - - [01/Apr/2019 12:51:55] "GET /static/img/success.jpg HTTP/1.1" 200 -
10.32.0.1 - - [01/Apr/2019 12:51:55] "GET /favicon.ico HTTP/1.1" 404 -
10.32.0.1 - - [01/Apr/2019 12:51:57] "GET / HTTP/1.1" 200 -
10.32.0.1 - - [01/Apr/2019 12:51:58] "GET / HTTP/1.1" 200 -
10.32.0.1 - - [01/Apr/2019 12:51:58] "GET / HTTP/1.1" 200 -
```

If these logs do not shed light on a recent failure, continuously stream the logs to capture real-time activity:

```
```bash theme={null} bash kubectl logs web -f
```

## ## Troubleshooting the Database Service

Apply a similar approach to verify the status of the database (DB) service. Examine the fol

- \* Inspect the DB service details.
- \* Confirm that endpoints are configured correctly.
- \* Check the DB pod's status.
- \* Review the database logs for errors that might signal issues with the database.

<Callout icon="triangle-alert" color="#FF6B6B">

Ensure that both the web and DB services have matching selectors for their respective pod

</Callout>

## ## Visualizing the Dependency Chain

The diagram below illustrates the troubleshooting flow for dependent applications. It outli

<Frame>

! [The image illustrates a flowchart titled "Check Dependent Applications," showing a sequ

</Frame>

## ## Additional Resources

For further troubleshooting guidance and best practices, refer to the following resources:

- \* [Kubernetes Documentation](https://kubernetes.io/docs/tasks/debug-application-cluster/trou
- \* [Kubernetes Basics](https://kubernetes.io/docs/concepts/overview/what-is-kubernetes/)

## ## Final Tip

<Callout icon="lightbulb" color="#1CB2FE">

Use the command below to obtain detailed information about any pod. Replace \\${POD\_NAME}

</Callout>

```
```bash theme={null}
kubectl describe pods ${POD_NAME}
```

By following these steps, you can effectively troubleshoot and resolve application failures in your Kubernetes environment while ensuring a systematic approach that covers both the service and pod components.

Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

Control Plane Failure

This guide explores methods for troubleshooting control plane failures in Kubernetes clusters, including node health checks and component status verification.

In this guide, we explore effective methods for troubleshooting control plane failures in your Kubernetes cluster. The process begins with checking the health of the nodes, followed by verifying the status of control plane components—whether they are deployed as pods or native services—and finally, reviewing detailed logs to identify any issues.

1. Verify Cluster Node Health

Begin by ensuring all cluster nodes are operational. Execute the command below to list the status of each node:

```
``bash theme={null} kubectl get nodes
```

This step helps quickly identify any node-related issues that might impact the control plane.

2. Confirm Control Plane Component Status

Control plane components can either be deployed as pods (common with kubeadm setups) or as

<Callout icon="lightbulb" color="#1CB2FE">

If your control plane components are deployed as pods in the kube-system namespace, ensure

</Callout>

a. Checking the Kubernetes API Server

For systems using native services, inspect the API server status with:

```
```plaintext theme={null}
service kube-apiserver status
• kube-apiserver.service - Kubernetes API Server
 Loaded: loaded (/etc/systemd/system/kube-apiserver.service; enabled; vendor preset: enabled)
 Active: active (running) since Wed 2019-03-20 07:57:25 UTC; 1 weeks 1 days ago
 Docs: https://github.com/kubernetes/kubernetes
 Main PID: 15767 (kube-apiserver)
 Tasks: 13 (limit: 2362)
```

## b. Checking the Controller Manager

Next, verify that the kube-controller-manager is up and running:

```
```plaintext theme={null} service kube-controller-manager status
• kube-controller-manager.service - Kubernetes Controller Manager
  Loaded: loaded (/etc/systemd/system/kube-controller-manager.service; enabled; vendor preset: enabled)
  Active: active (running) since Wed 2019-03-20 07:57:25 UTC; 1 weeks 1 days ago
  Docs: https://github.com/kubernetes/kubernetes
  Main PID: 15771 (kube-controller)
  Tasks: 10 (limit: 2362)
```

c. Checking the Scheduler

Also, verify the status of the kube-scheduler service by running:

```
```bash theme={null}
service kube-scheduler status
```

Ensure that this command returns complete and active service details.

## 3. Review Control Plane Logs

After confirming the service statuses, proceed to review the logs for deeper insights.

### a. Retrieving Pod Logs (When Using kubeadm)

If the control plane components are deployed as pods, fetch the logs for the API server from the kube-system namespace:

```
``bash theme={null} kubectl logs kube-apiserver-master -n kube-system
```

An example excerpt from the logs might appear as follows:

```
``plaintext theme={null}
I0401 13:45:38.190735 1 server.go:703] external host was not specified, using 172.17.0.1
I0401 13:45:38.194290 1 server.go:145] Version: v1.11.3
I0401 13:45:38.819075 1 plugins.go:158] Loaded 8 mutating admission controller(s) successfully in order: NamespaceLifecycle,LimitRanger,ServiceAccount,NodeRestriction,Priority,DefaultTolerationSeconds,DefaultResourceQuota,VolumePlugin
...
W0401 13:45:41.381736 1 genericapiserver.go:319] Skipping API scheduling.k8s.io/v1alpha1
```

### b. Using the Journal for Native Service Logs

For control plane components configured as native services on the master nodes, use the journal control utility to view the logs:

```
bash theme={null} sudo journalctl -u kube-apiserver
```

This command allows you to review the logs specific to the kube-apiserver service, helping in pinpointing issues.

## 4. Additional Resources

For further details and advanced troubleshooting techniques, refer to the official [Kubernetes Documentation](#).

By following these steps, you can systematically diagnose and address control plane issues within your Kubernetes cluster, ensuring a stable and resilient environment.

## Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

# Worker Node Failure

This article explores techniques to troubleshoot worker node failures in a Kubernetes cluster, focusing on node status, kubelet health, and certificate validation.

In this article, we explore various techniques to troubleshoot worker node failures within a Kubernetes cluster. Effective troubleshooting involves checking node status, examining detailed node conditions, and diagnosing issues with the kubelet service and its certificates.

## 1. Check Node Status

Begin by verifying the status of the nodes in your cluster. Use the following command to determine if nodes are reporting as Ready or NotReady:

```
``plaintext theme={null} kubectl get nodes NAME STATUS ROLES AGE
VERSION worker-1 Ready 8d v1.13.0 worker-2 NotReady 8d v1.13.0
```

If a node is listed as NotReady, inspect its details using:

```
``plaintext theme={null}
kubectl describe node worker-1
```

This command produces an output with various conditions, such as OutOfDisk, MemoryPressure, DiskPressure, PIDPressure, and Ready. Each condition will have a status of true or false that helps pinpoint issues. For example, if disk space is insufficient, the OutOfDisk flag will be set to true; if there is low memory, the MemoryPressure flag will reflect that.

Always review the "LastHeartbeatTime" field. It indicates when a node last communicated with the master, which can provide insights if a node has unexpectedly gone down.

## 2. Validate Node Operation and Kubelet Health

After confirming any node issues, verify if the node itself is operational. Check the node's CPU, memory, and disk usage, review the kubelet status, inspect its logs, and ensure that the kubelet certificates are valid and correctly issued by the proper Certificate Authority (CA).

### Check the Kubelet Service Status

Run the following command to check the status of the kubelet service:

```
```bash theme={null} service kubelet status
```

Example output:

```
```plaintext theme={null}
• kubelet.service - Kubernetes Kubelet
 Loaded: loaded (/etc/systemd/system/kubelet.service; enabled; vendor preset: enabled)
 Active: active (running) since Wed 2019-03-20 14:22:06 UTC; 1 weeks 1 days ago
 Docs: https://github.com/kubernetes/kubernetes
 Main PID: 1281 (kubelet)
 Tasks: 24 (limit: 1152)
```

### Inspect Kubelet Logs

For further diagnosis, view the kubelet logs with:

```
```bash theme={null} sudo journalctl -u kubelet
```

An example segment of the logs may look like:

```
```plaintext theme={null}
-- Logs begin at Wed 2019-03-20 05:30:37 UTC, end at Mon 2019-04-01 14:42:42 UTC. --
Mar 20 08:12:59 worker-1 systemd[1]: Started Kubernetes Kubelet.
Mar 20 08:12:59 worker-1 kubelet[18926]: Flag --tls-cert-file has been deprecated, This par
Mar 20 08:12:59 worker-1 kubelet[18926]: Flag --tls-private-key-file has been deprecated, T
Mar 20 08:12:59 worker-1 kubelet[18926]: I0320 08:12:59.915179 18926 flags.go:33] FLAG: -
Mar 20 08:12:59 worker-1 kubelet[18926]: I0320 08:12:59.918149 18926 flags.go:33] FLAG: -
Mar 20 08:12:59 worker-1 kubelet[18926]: I0320 08:12:59.918339 18926 flags.go:33] FLAG: -
Mar 20 08:12:59 worker-1 kubelet[18926]: I0320 08:12:59.918520 18926 flags.go:33] FLAG: -
Mar 20 08:12:59 worker-1 kubelet[18926]: I0320 08:12:59.918621 18926 flags.go:33] FLAG: -
Mar 20 08:12:59 worker-1 kubelet[18926]: I0320 08:12:59.918740 18926 flags.go:33] FLAG: -
Mar 20 08:12:59 worker-1 kubelet[18926]: I0320 08:12:59.919874 18926 flags.go:33] FLAG: -
Mar 20 08:12:59 worker-1 kubelet[18926]: I0320 08:12:59.919929 18926 flags.go:33] FLAG: -
Mar 20 08:12:59 worker-1 kubelet[18926]: I0320 08:12:59.919946 18926 flags.go:33] FLAG: -
Mar 20 08:12:59 worker-1 kubelet[18926]: I0320 08:12:59.919948 18926 flags.go:33] FLAG: -
Mar 20 08:12:59 worker-1 kubelet[18926]: I0320 08:12:59.919950 18926 flags.go:33] FLAG: -
Mar 20 08:12:59 worker-1 kubelet[18926]: I0320 08:12:59.919958 18926 flags.go:33] FLAG: -
Mar 20 08:12:59 worker-1 kubelet[18926]: I0320 08:12:59.920160 18926 flags.go:33] FLAG: -
Mar 20 08:12:59 worker-1 kubelet[18926]: I0320 08:12:59.920172 18926 flags.go:33] FLAG: -
```

### 3. Verify Kubelet Certificates

---

Ensuring that the kubelet certificates are valid and correctly issued is crucial. Use the following command to inspect a kubelet certificate:

```
``bash theme={null} openssl x509 -in /var/lib/kubelet/worker-1.crt -text
```

A valid certificate should display details such as:

```
``plaintext theme={null}
Certificate:
 Data:
 Version: 3 (0x2)
 Serial Number:
 ff:e0:23:9d:fc:78:03:35
 Signature Algorithm: sha256WithRSAEncryption
 Issuer: CN = KUBERNETES-CA
 Validity
 Not Before: Mar 20 08:09:29 2019 GMT
 Not After : Apr 19 08:09:29 2019 GMT
 Subject: CN = system:node:worker-1, O = system:nodes
 Subject Public Key Info:
 Public Key Algorithm: rsaEncryption
 Public-Key: (2048 bit)
 ...
```

Be sure that the certificate is issued by the correct CA and that none of the certificate parameters (e.g., validity period) indicate an impending or current issue.

### Conclusion

---

By following the steps outlined above, you can efficiently troubleshoot worker node failures in your Kubernetes cluster. Regularly monitoring node conditions, validating the health of the kubelet service, and ensuring certificate integrity will help maintain a stable and robust cluster operation.

For further learning, consider exploring additional resources:

- [Kubernetes Documentation](#)
- [Kubernetes Basics](#)
- [Docker Hub](#)

Happy troubleshooting!

# Section: Design and Install a Kubernetes Cluster

---

## Documentation Index

---

Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

## Choosing Kubernetes Infrastructure

---

This guide helps you select the right Kubernetes infrastructure by exploring various hosting options and deployment strategies for local and production environments.

Welcome to this informative guide on selecting the right Kubernetes infrastructure for your needs. In this article, we explore various hosting options for Kubernetes clusters, examine their unique characteristics, and provide insights into both local and production deployments. Whether you're new to Kubernetes or looking to expand your production environment, this guide will help you make an informed decision.

Kubernetes can be deployed on numerous platforms—from your local machine or laptop to both physical and virtual servers hosted on-premises or in the cloud. Your choice will depend on your technical requirements, cloud ecosystem compatibility, and the type of applications you plan to run.

## Deploying Kubernetes on a Local Machine

---

For local development or testing purposes, several setup strategies are available. On a supported Linux system, you can install the Kubernetes binaries manually to configure a local cluster. However, for beginners, this process may be challenging, and a more automated solution is generally preferred to simplify cluster setup.

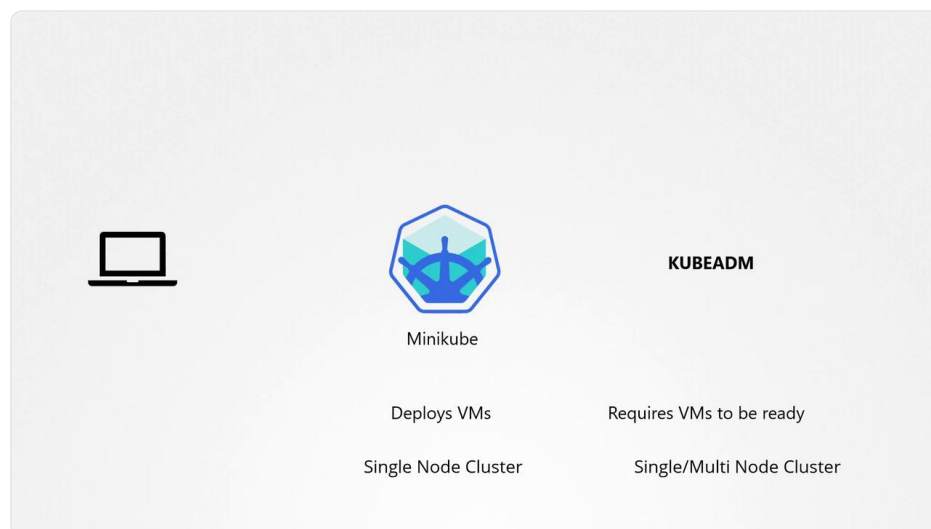
On Windows, since native Kubernetes binaries are not available, you need to rely on virtualization platforms such as Hyper-V, VMware Workstation, or VirtualBox. Windows users typically create Linux virtual machines (VMs) to host Kubernetes. Although Kubernetes components can run as Docker containers on these VMs, they operate on minimal Linux operating systems provided by Hyper-V.

## Local Deployment Solutions

Two popular local deployment methods include:

- **Minikube:** This tool deploys a single-node Kubernetes cluster effortlessly by creating and managing VMs with virtualization software like Oracle VirtualBox. Minikube automatically provisions the VM with the necessary configuration.
- **Kubeadm:** This tool can quickly bootstrap both single-node and multi-node clusters but requires that you have already provisioned the VMs beforehand.

Local Kubernetes deployments on laptops are ideal for learning, development, and testing scenarios.



## Production Deployment Options

In production environments, Kubernetes clusters are typically deployed in private or public clouds. Production solutions fall into two main categories: turnkey solutions and hosted (or managed) solutions.

### Turnkey Solutions

Turnkey solutions enable you to provision and configure Kubernetes clusters with automated tools or scripts. While these solutions automate the deployment process, you remain responsible for maintaining, patching, and



upgrading the underlying VMs. For example, deploying a Kubernetes cluster on AWS using the KOPS tool automates much of the setup.

Turnkey solutions require careful management of the underlying infrastructure, so ensure you monitor and update your VMs regularly.

## Hosted Solutions

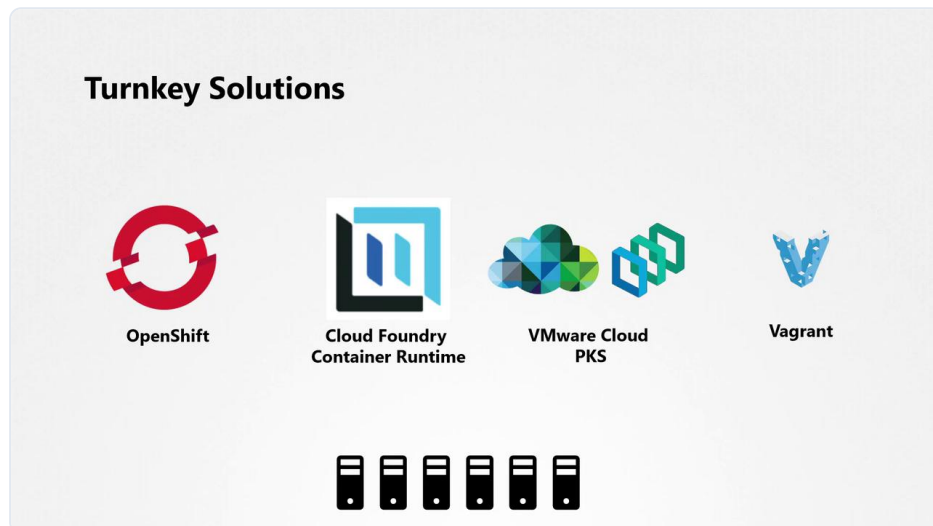
Hosted solutions provide Kubernetes as a service, where the provider manages the entire cluster infrastructure, including VM provisioning, maintenance, and configuration. This approach greatly simplifies the deployment process. For instance, Google Container Engine (GKE) lets you deploy a Kubernetes cluster in minutes with minimal manual intervention.

Turnkey Solutions	Hosted Solutions (Managed Solutions)
• You Provision VMs    • You Configure VMs    • You Use Scripts to Deploy Cluster    • You Maintain VMs yourself    • Eg: Kubernetes on AWS using KOPS	• Kubernetes-As-A-Service    • Provider provisions VMs    • Provider installs Kubernetes    • Provider maintains VMs    • Eg: Google Container Engine (GKE)

## Turnkey Solutions: On-Premises Options

For on-premises deployments, several turnkey solutions are available:

- **OpenShift:** Powered by Red Hat, OpenShift is a robust Kubernetes platform that comes with additional management tools and a graphical user interface for resource management and CI/CD integration. Check out the [OpenShift for Beginners](#) course for an in-depth introduction.
- **Cloud Foundry Container Runtime:** An open-source project that uses the BOSH tool to deploy and manage highly available Kubernetes clusters.
- **VMware Cloud PKS:** An excellent option for leveraging an existing VMware environment to deploy Kubernetes.
- **Vagrant:** Provides scripts to deploy Kubernetes clusters across various cloud service providers.



These turnkey options simplify the setup and management of Kubernetes clusters within your organization, assuming that your virtual machines meet the required specifications. For additional certified solutions and best practices, refer to the [Kubernetes Documentation](#).

## Hosted Solutions: Cloud Offerings

Several cloud providers offer managed Kubernetes services, making it easier to run containerized applications without the overhead of managing the underlying infrastructure. Top hosted offerings include:

- **Google Container Engine (GKE)** on Google Cloud Platform.
- **OpenShift Online** from Red Hat, delivering a fully functional Kubernetes cluster.
- **Azure Kubernetes Service (AKS)** on Microsoft Azure.
- **Amazon Elastic Container Service for Kubernetes (EKS)** on AWS.

These services are designed to reduce operational complexity and support rapid scaling of applications.

## Hosted Solutions



## Our Deployment Choice

For learning and lab environments—especially for those without access to a public cloud account—we opted for a local deployment using VirtualBox. According to our recent poll, the majority of our users prefer VirtualBox for their lab setups.

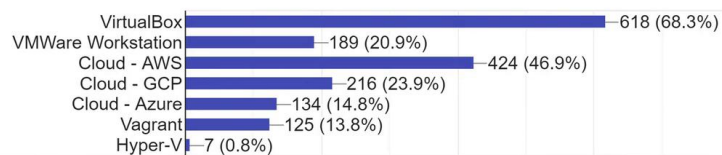
We deployed a local Kubernetes cluster from scratch by creating several VMs on VirtualBox. The design includes three nodes: one master and two worker nodes, all running on a single laptop.

### Our Choice



#### Preferred Virtualization technology for Labs?

905 responses



When setting up your own lab using VirtualBox, ensure that your laptop meets the resource requirements for running multiple VMs efficiently.

This concludes our guide on choosing the right Kubernetes infrastructure. For further details on deployment options and best practices, please refer to additional documentation and trusted sources in the Kubernetes community.

For more in-depth reading:

- [Kubernetes Basics](#)
- [Kubernetes Documentation](#)
- [Docker Hub](#)
- [Terraform Registry](#)

## Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

# ETCD in HA

This lesson covers deploying ETCD in high-availability configurations, including cluster setup, data consistency, and the Raft protocol for Kubernetes resilience.

Welcome to this lesson on deploying ETCD in a high-availability (HA) configuration. In this guide, we will review ETCD fundamentals, explain how to configure a distributed cluster, and detail how data consistency is maintained across nodes using the Raft protocol. This HA setup is a critical requirement for running Kubernetes in a resilient manner.

## What is ETCD?

ETCD is a distributed, reliable key-value store that is both fast and secure. Unlike traditional databases that store data in tables, ETCD organizes data as documents or pages. Each document holds all necessary information about a specific entity and can be formatted in JSON, YAML, or other structures. Changing one document does not affect others, which makes ETCD an excellent choice for modern, scalable architectures.

## Objectives

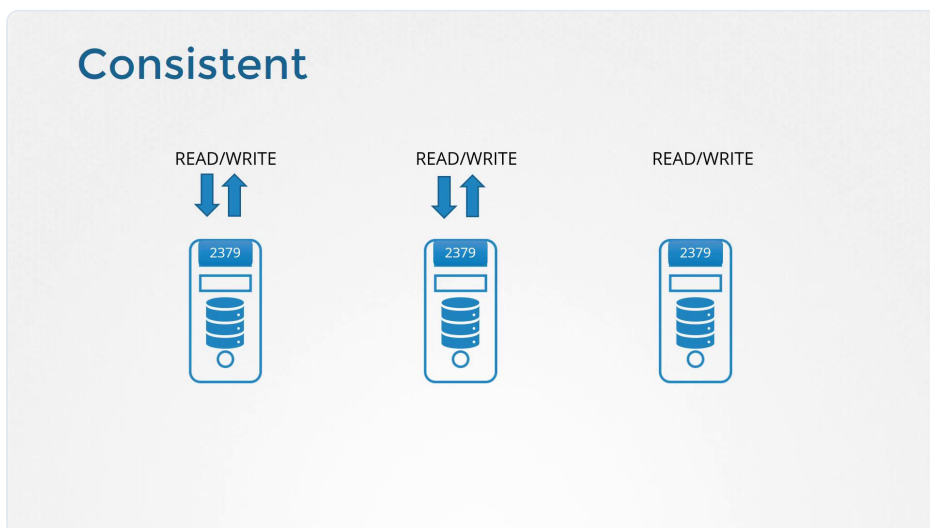
- What is ETCD?
- What is a Key-Value Store?
- How to get started quickly?
- How to operate ETCD?
- What is a distributed system?
- How ETCD Operates
- RAFT Protocol
- Best practices on number of nodes

## Distributed ETCD Clusters

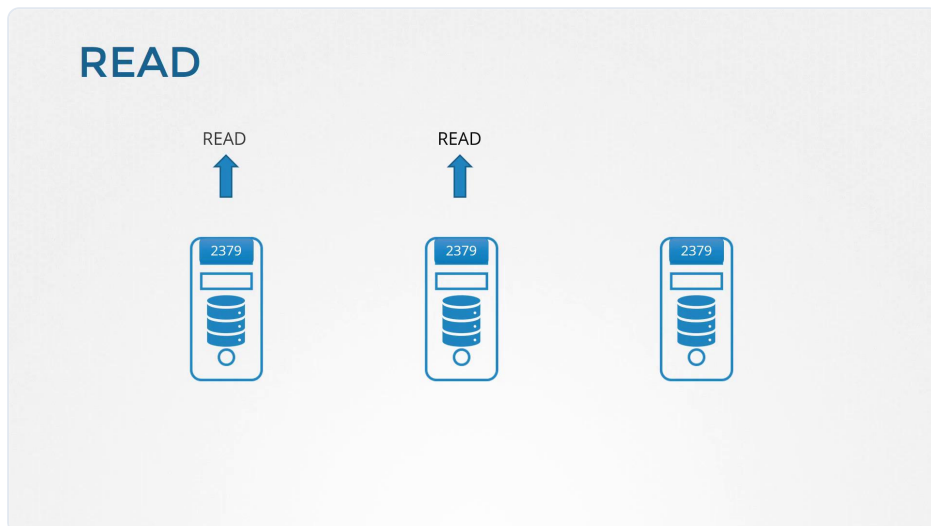
ETCD's design is inherently distributed. Picture three different servers each running an identical instance of ETCD. This redundancy ensures that if one server (or node) fails, the remaining nodes continue to have an accurate copy of the data.



ETCD supports both read and write operations on any node. However, write operations require careful coordination. While clients might send write requests to any node, only the leader node processes these writes. If a write request lands on a follower, it is forwarded to the leader. The write operation is only confirmed after the leader gets an acknowledgement from a majority of the nodes.



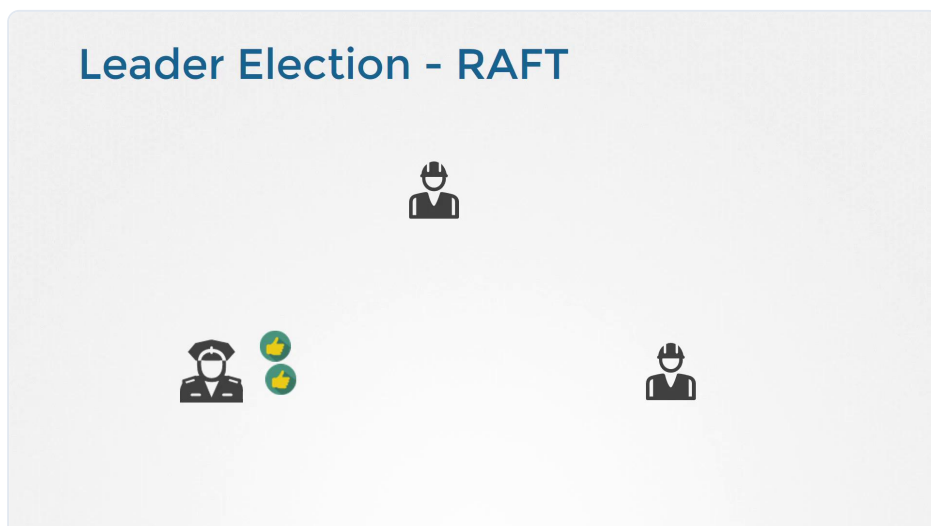
Read operations are simpler since every node contains the same data. Despite this, write operations necessitate a leader election through the Raft consensus protocol.



## The Raft Consensus Protocol

Raft plays a crucial role in ensuring data consistency within an ETCD cluster. When the cluster boots up, no leader is present until one node's randomized timeout expires, triggering an election. During this election, the candidate node requests votes from its peers. Once it obtains the necessary votes, it is crowned the leader and begins sending regular heartbeat messages to assert its control.

If the leader fails or experiences network issues, the remaining nodes automatically trigger a new election to establish a new leader. This robust process guarantees that all write requests are processed correctly and that every node's data remains synchronized.



## Quorum and Fault Tolerance

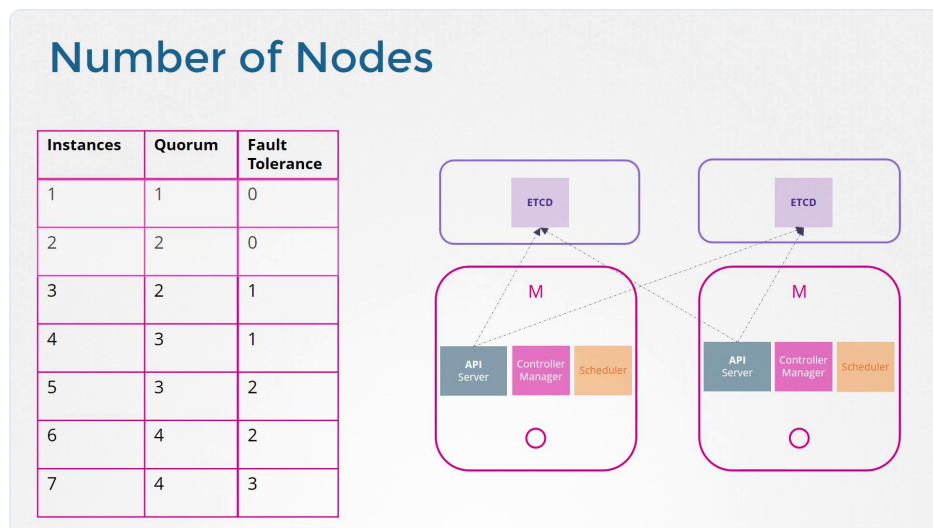
For a write operation to be successful in an ETCD cluster, it must be replicated to a majority of nodes, which is referred to as a quorum. For instance, in a

three-node cluster, the update must reach at least two nodes. The quorum is calculated using the formula:

$$\text{quorum} = (\text{total number of nodes} / 2) + 1$$

This means a three-node cluster requires a quorum of 2, a five-node cluster requires 3, etc. Odd-numbered clusters are preferred because even-numbered ones may split into equal groups during network partitions, preventing either group from achieving quorum and potentially causing cluster failure.

Consider a six-node cluster: if a network partition results in subgroups of four and two, the larger group meets quorum and continues operation. However, if the split creates two groups of three, neither side meets the quorum of four nodes. In contrast, a seven-node cluster might split into groups of four and three, allowing the larger group to maintain functionality. For these reasons, odd node counts (e.g., three, five, seven) are strongly recommended for a robust HA cluster.



## Installing and Configuring ETCD

To install ETCD, follow these steps:

1. Download the latest supported binary.
2. Extract the downloaded archive.
3. Create the required directory structure.
4. Copy the generated certificate files to their designated locations.

For example, to download and extract the ETCD binary, use the following commands:

```
``bash theme={null} wget -q --https-only "https://github.com/coreos/etcd/releases/download/v3.3.9/etcd-v3.3.9-linux-amd64.tar.gz" tar -xvf etcd-v3.3.9-linux-amd64.tar.gz
```



Next, configure the ETCD service. A critical configuration parameter is the "initial cluster"

```
```bash theme={null}
ExecStart=/usr/local/bin/etcd \
  --name ${ETCD_NAME} \
  --cert-file=/etc/etcd/kubernetes.pem \
  --key-file=/etc/etcd/kubernetes-key.pem \
  --peer-cert-file=/etc/etcd/kubernetes.pem \
  --peer-key-file=/etc/etcd/kubernetes-key.pem \
  --trusted-ca-file=/etc/etcd/ca.pem \
  --peer-trusted-ca-file=/etc/etcd/ca.pem \
  --peer-client-cert-auth \
  --client-cert-auth \
  --initial-advertise-peer-urls https://${INTERNAL_IP}:2380 \
  --listen-peer-urls https://${INTERNAL_IP}:2380 \
  --listen-client-urls https://${INTERNAL_IP}:2380,https://127.0.0.1:2379 \
  --advertise-client-urls https://${INTERNAL_IP}:2379 \
  --initial-cluster-token etcd-cluster-0 \
  --initial-cluster peer-1=https://${PEER1_IP}:2380,peer-2=https://${PEER2_IP}:2380 \
  --initial-cluster-state new \
  --data-dir=/var/lib/etcd
```

Ensure your certificate files and network configurations are correctly set up before starting the ETCD service.

Using etcdctl

The command-line tool, etcdctl, is used to interact with the ETCD store by managing key-value pairs. This lesson uses the v3 API of etcdctl. Set the API version using the following command:

```
```bash theme={null} export ETCDCTL_API=3
```

You can add a key-value pair to the ETCD store with:

```
```bash theme={null}
etcdctl put name john
```

Then, retrieve the stored value using:

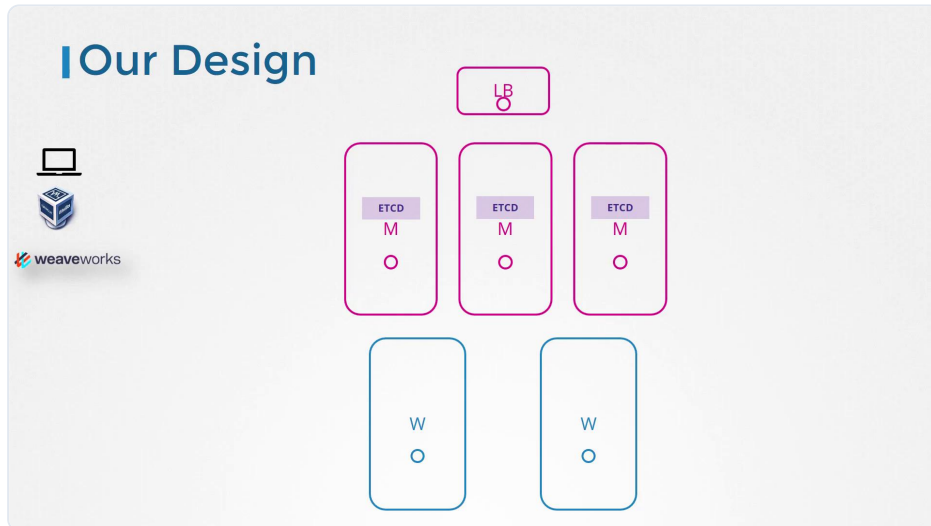
```
```bash theme={null} etcdctl get name
```

If successful, the output will be:

```
```text theme={null}
name
john
```

Choosing the Right Cluster Size

For a highly available environment, a minimum cluster size of three nodes is required. Deploying only one or two nodes is insufficient as it compromises the quorum—if one node fails, the cluster will not have enough nodes to operate. While a three-node cluster provides basic fault tolerance, a five-node cluster typically offers enhanced resilience without unnecessary complexity.



Deploying fewer than three nodes in production may lead to a split-brain scenario during network partitions, severely impacting cluster functionality.

For environments with limited resources, such as on a laptop, a two-node setup might be used for experimentation; however, this does not provide true fault tolerance. In production environments, always aim for a minimum of three nodes to maintain an effective HA configuration.

That concludes our lesson on deploying ETCD in a high-availability setup. Thank you for reading, and we look forward to seeing you in the next lesson.

Section: Helm Basics (2025 Updates)

Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

Installation and configuration

This lesson explains the steps required to install Helm, a package manager for Kubernetes, on Linux systems.

This lesson explains the steps required to install Helm, a package manager for Kubernetes. Before starting, ensure you have a functioning Kubernetes cluster and that `kubectl` is properly configured. Verify that your `kubeconfig` file contains the correct credentials to access your Kubernetes cluster.

Make sure your system has access to a working Kubernetes cluster and that `kubectl` is set up correctly. A valid `kubeconfig` file is essential for Helm to connect to your cluster.

Helm is compatible with Linux, Windows, and macOS. This guide focuses on installing Helm on Linux systems.

Installing Helm on Linux

Using Snap

If your system supports Snap, you can install Helm using the command below. The `--classic` option offers a more relaxed sandbox environment, enabling Helm to easily locate your `kubeconfig` file in your home directory and connect to your Kubernetes cluster:

```
``bash theme={null} sudo snap install helm --classic
```

Using APT (Debian/Ubuntu)

For apt-based systems such as Debian or Ubuntu, follow these steps to add the Helm package

```
```bash theme={null}
curl https://baltocdn.com/helm/signing.asc | sudo apt-key add -
sudo apt-get install apt-transport-https --yes
echo "deb https://baltocdn.com/helm/stable/debian/ all main" | sudo tee /etc/apt/sources
sudo apt-get update
sudo apt-get install helm
```

## Using PKG

On systems that support PKG, you can install Helm with the following command:

```
bash theme={null} pkg install helm
```

For the most current installation procedures, always refer to the official Helm documentation.

## Conclusion

---

With Helm installed, you are now ready to explore its capabilities in your lab environment. This guide provided step-by-step instructions for various Linux distributions to help streamline your Helm installation process.

Happy Helming!

## Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/lms.txt> Use this file to discover all available pages before exploring further.

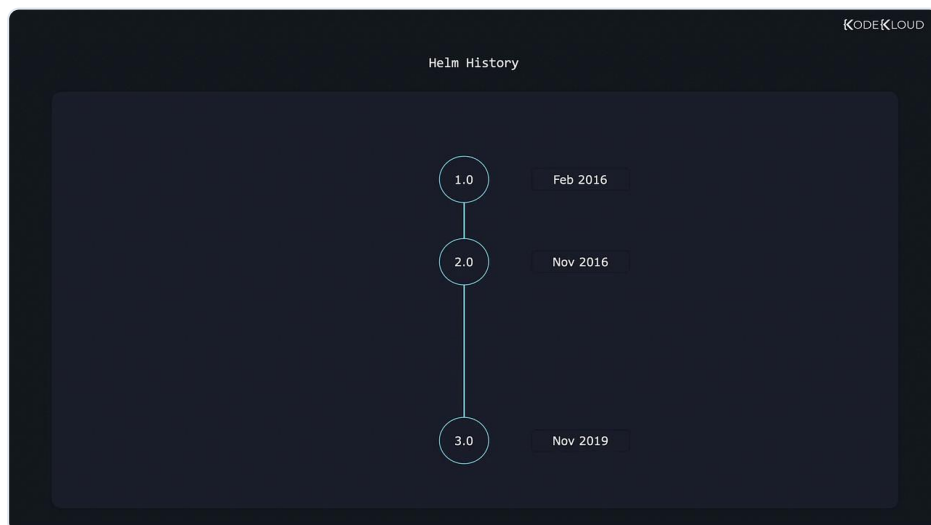
# A quick note about Helm2 vs Helm3

This article explains the differences between Helm 2 and Helm 3, focusing on improvements in usability and security for Kubernetes charts.

Helm has evolved significantly since its inception, and understanding the differences between Helm 2 and Helm 3 is crucial when working with Kubernetes charts. This article provides a brief history of Helm and highlights the key improvements introduced in Helm 3, offering insights into its enhanced usability and security.

## A Brief History of Helm

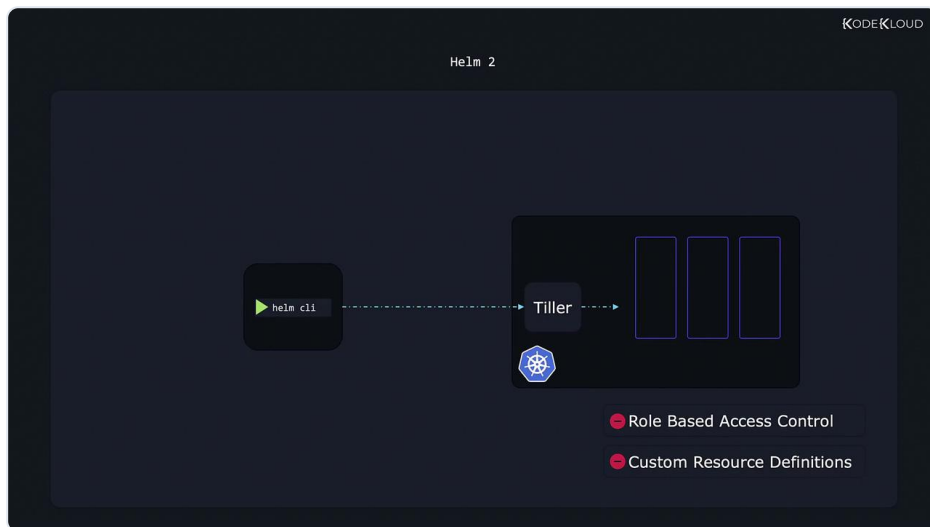
Helm's journey began in February 2016 with Helm 1.0, followed by Helm 2.0 in November 2016. The major milestone was achieved with the release of Helm 3.0 in November 2019. As Kubernetes itself advanced, Helm matured to better leverage Kubernetes functionalities, resulting in a more robust and secure package management experience.



## Helm 2: The Role of Tiller

In the era of Helm 2, Kubernetes limitations such as the absence of role-based access control (RBAC) and custom resource definitions (CRDs) meant that an additional component, Tiller, was required. Tiller acted as an intermediary between the Helm CLI and the Kubernetes cluster, managing the installation and upgrade of charts. However, this approach introduced several challenges:

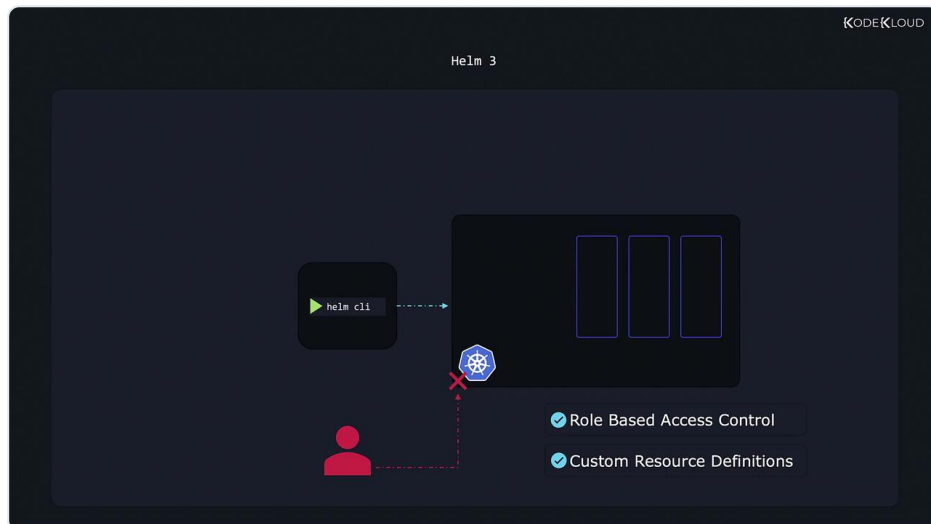
- **Complexity:** The interaction between the CLI and Tiller added an extra layer to the deployment process.
- **Security Risks:** Tiller operated with broad privileges, often referred to as running in "God mode," increasing the risk of potential security issues.



Using Tiller in Helm 2 means that any misconfiguration could lead to elevated security risks due to its extensive privileges.

## Helm 3: Simplified and Secure

Helm 3 brings notable improvements by eliminating the need for Tiller. Instead, the Helm client now communicates directly with Kubernetes, leveraging its native RBAC capabilities. This change simplifies the architecture and enhances security, as every Helm action is subject to the same RBAC permissions applied when using kubectl.



## Key Differences Between Helm 2 and Helm 3

Feature	Helm 2	Helm 3
Intermediary Component	Requires Tiller	No intermediary; direct communication
Security Model	Elevated risk due to broad privileges	Enhanced security leveraging Kubernetes RBAC
Rollback Mechanism	Basic revision comparison	Three-way strategic merge patch

## Improved Rollback and Upgrade Process in Helm 3

One of the significant advancements in Helm 3 is the handling of rollbacks through a three-way strategic merge patch, resembling a snapshot mechanism. Consider the example of deploying a WordPress website using a Helm chart:

1. **Installation:** When you install the chart, Helm creates revision 1.
2. **Upgrade:** After changing configurations, such as upgrading the WordPress image, Helm creates revision 2.
3. **Rollback:** If needed, reverting to revision 1 will restore the previous state.

Below is an example of the process:

```
```bash theme={null} $ helm install wordpress $ helm upgrade wordpress
```

```
### Revision Changes in YAML

```yaml theme={null}
Revision 1: Using an older WordPress image
containers:
 - image: wordpress:4.8-apache
```

```
```yaml theme={null}
```

Revision 2: After upgrading to a newer WordPress image

```
containers: - image: wordpress:5.8-apache
```

To roll back to revision 1:

```
```bash theme={null}
$ helm rollback wordpress
```

With each significant change invoked via Helm commands, a new revision is created. For instance, installation creates revision 1, an upgrade creates revision 2, and a rollback may create revision 3 representing the restored state.

Helm 3's intelligent three-way comparison considers the following:

- The previous chart revision,
- The desired chart state,
- The live state of Kubernetes objects. This approach ensures that discrepancies—such as manual changes using `kubectl`—are correctly reconciled.

## Handling Manual Changes and Upgrades

---

In Helm 2, if a user manually modified Kubernetes objects (e.g., using `kubectl set image`) after deployment, these changes were not recorded in Helm's revision history, meaning Helm might not detect any differences during a rollback. In contrast, Helm 3 compares the live state against both the current and desired revisions. This ensures:

- Manual modifications outside of Helm are preserved during upgrades.
- Overwritten configurations are avoided unless explicitly intended.



## Conclusion

---

The transition from Helm 2 to Helm 3 marks a significant improvement in Kubernetes deployment management. By removing Tiller and implementing a more robust rollback mechanism via a three-way strategic merge patch, Helm 3 ensures enhanced security, simplified architecture, and greater reliability during upgrades.

For more detailed information on Helm and Kubernetes, check out the following resources:

- [Kubernetes Basics](#)
- [Kubernetes Documentation](#)
- [Helm Documentation](#)

This guide should help you understand the evolution of Helm and leverage Helm 3's features for a more secure and streamlined deployment experience.

## Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

# Helm Components

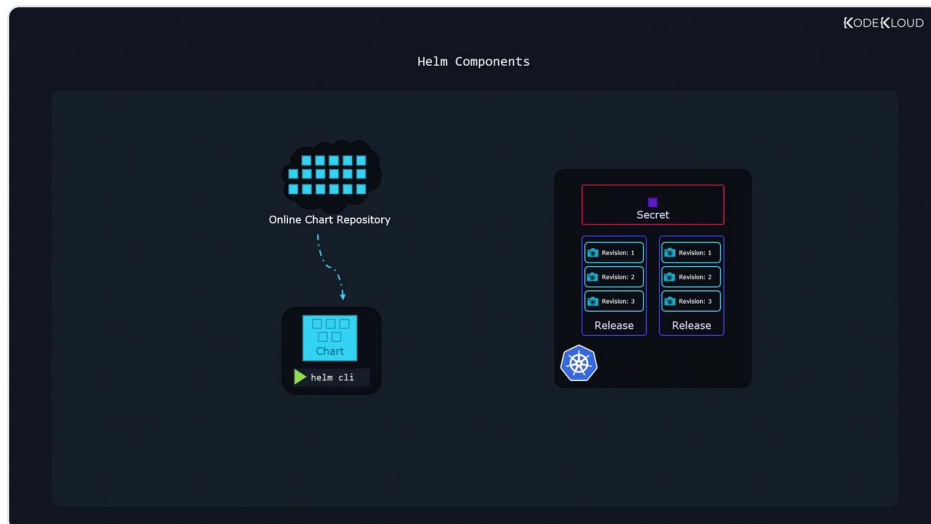
This article provides an in-depth look at Helm components, including its structure, concepts, and key elements for managing Kubernetes applications effectively.

In this article, we dive into the components of Helm, providing an in-depth look at its structure, concepts, and key elements essential for managing Kubernetes applications effectively.

Helm is primarily composed of a command-line tool installed locally, which you can use to install, upgrade, or roll back releases. Charts—collections of files containing instructions for creating Kubernetes objects—are used by Helm to deploy applications. When you deploy a chart to your Kubernetes cluster, Helm creates a release, representing a specific installation of the application. Each release may have multiple revisions, capturing changes like image upgrades, replica adjustments, or configuration updates.

Similar to how [Docker Hub](#) hosts container images or [Vagrant Cloud](#) provides boxes, public repositories host Helm charts. These repositories allow you to quickly download and deploy applications on your cluster.

Helm stores metadata—including information about installed releases, used charts, and revision history—directly into your Kubernetes cluster as secrets. This ensures that metadata is persistent and accessible to all team members, facilitating seamless upgrades and maintenance operations.



Helm maintains a comprehensive record of every action performed within the cluster, enabling precise tracking and management.

## Charts and Templating

Charts are collections of files that contain the instructions for creating Kubernetes objects, making them the backbone of Helm deployments. This article uses two application examples to illustrate various concepts:

1. **HelloWorld Application:** A simple Nginx-based web server with a service for exposure.
2. **WordPress Site:** A more complex application deployment.

The HelloWorld example demonstrates fundamental concepts effectively. In this example, two Kubernetes objects are deployed: a Deployment and a Service. Although similar to standard Kubernetes definitions, you'll notice that parameters like the image name and replica count are templated. These templated values are defined in a separate `values.yaml` file, which allows you to customize your deployment with minimal changes.

Below is an example of a simple HelloWorld chart:

```
``yaml theme={null}
```

## service.yaml

```
apiVersion: v1 kind: Service metadata: name: hello-world spec: type: NodePort
ports: - port: 80 targetPort: http protocol: TCP name: http selector: app: hello-world
```

```
```yaml theme={null}
# deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: hello-world
spec:
  replicas: {{ .Values.replicaCount }}
  selector:
    matchLabels:
      app: hello-world
  template:
    metadata:
      labels:
        app: hello-world
    spec:
      containers:
        - name: nginx
          image: {{ .Values.image.repository }}
          ports:
            - name: http
              containerPort: 80
              protocol: TCP
```

```
```yaml theme={null}
```

## values.yaml

---

replicaCount: 1 image: repository: nginx

Rather than building charts from scratch, you can download pre-built charts from public repositories.

For more complex applications like WordPress, charts can include multiple files and advanced templating.

A more advanced templating snippet within a Deployment might look like this:

```

```yaml
theme: {null}
apiVersion: {{ include "common.capabilities.deployment.apiVersion" }}
kind: Deployment
metadata:
  name: {{ include "common.names.fullname" . }}
  namespace: {{ .Release.Namespace | quote }}
  labels:
    {{- include "common.labels.standard" . | nindent 4 }}
    {{- if .Values.commonLabels }}
    {{- include "common.tplvalues.render" (dict "value" .Values.commonLabels "context" $) }}
    {{- end }}
    {{- if .Values.commonAnnotations }}
  annotations:
    {{- include "common.tplvalues.render" (dict "value" .Values.commonAnnotations "context" $) }}
    {{- end }}
spec:
  selector:
    matchLabels: {{- include "common.labels.matchLabels" . | nindent 4 }}
    {{- if .Values.updateStrategy }}
  strategy: {{- toYaml .Values.updateStrategy | nindent 4 }}
    {{- end }}
    {{- if not .Values.autoscaling.enabled }}
  replicas: {{ .Values.replicaCount }}
    {{- end }}

```

Releases and Multiple Installations

When applying a chart to your cluster, Helm creates a release—a distinct instance of the application. This approach allows you to deploy multiple separate instances of the same chart with unique release names. For example, you can deploy two different WordPress sites using separate releases:

```
```bash
theme={null}
```

## helm install [release-name] [chart]

```
$ helm install my-site bitnami/wordpress
```

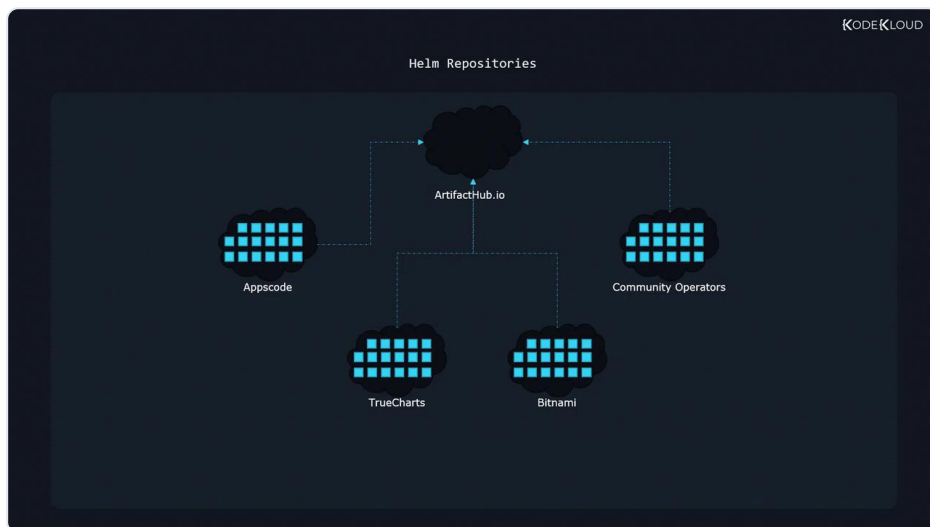
```
$ helm install my-SECOND-site bitnami/wordpress ``
```

Each release is tracked independently, even if they are based on the same chart. This functionality is particularly useful when maintaining different environments, such as having one release for a public-facing website and another for development purposes. Experimentation in the development release can then inform upgrades or changes to the production version.

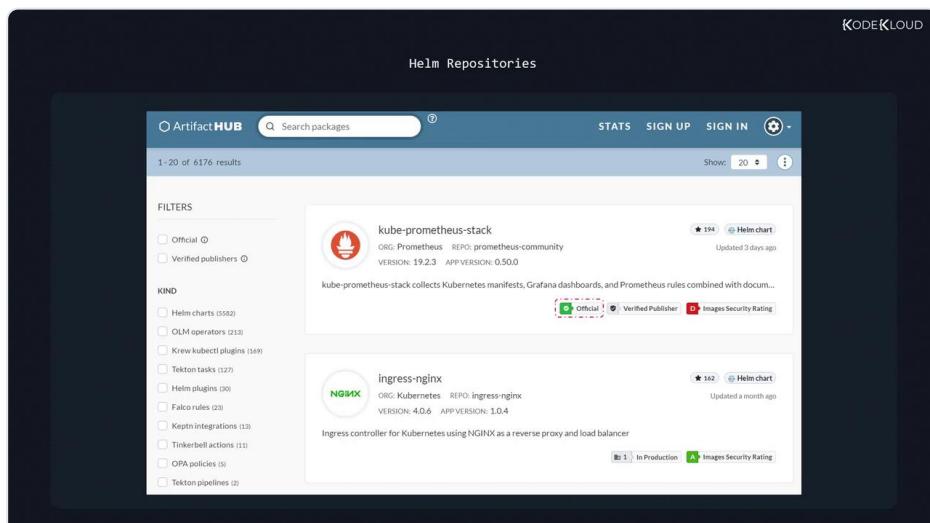
## Helm Repositories and Artifact Hub

Beyond our basic examples, Helm charts are available for a wide range of applications—from Redis to Prometheus—across numerous public repositories. Providers such as Appcode, Community Operators, TrueCharts, and Bitnami host charts in their repositories, making it easy to deploy various applications.

Instead of visiting multiple repositories separately, you can use the centralized [Artifact Hub](#) to search for and manage charts. Artifact Hub currently features over 6,300 packages and highlights charts published by official developers with verified publisher badges for added trustworthiness.



Artifact Hub also provides a searchable interface to help you quickly find the charts you need:



In upcoming lessons, we will explore chart installation and customization in greater detail. Continue following our guide to deepen your knowledge of the practical applications of Helm in Kubernetes.

Happy Helm-ing, and see you in the next lesson!



## Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

# Helm charts

This article explores Helm Charts for managing Kubernetes applications, detailing their structure, usage, and deployment processes.

In this lesson, we explore Helm Charts—a powerful tool for managing Kubernetes applications. Helm simplifies tasks such as installing, upgrading, rolling back, and uninstalling applications by automating the complex steps required to achieve the desired state.

Helm Charts act as comprehensive instruction manuals for your deployments. Each chart is a structured collection of files that define an application's configuration and behavior on Kubernetes. For example, the parameters in the `values.yaml` file enable operators to customize configurations without modifying the underlying templates.

Use Helm's templating syntax (e.g., `{{ .Values.replicaCount }}`) in your manifests to keep configuration flexible and reusable. All dynamic values are defined in the `values.yaml` file.

Below is a simple example of Helm template files that create two Kubernetes objects—a Deployment and a Service. The Deployment manages a set of Pods based on a specified image, and the Service exposes these Pods as a NodePort service:

```
```yaml theme={null} apiVersion: v1 kind: Service metadata: name: hello-world spec: type: NodePort ports: - port: 80 targetPort: http protocol: TCP name: http selector: app: hello-world
```

```
apiVersion: apps/v1 kind: Deployment metadata: name: hello-world spec: replicas: {{ .Values.replicaCount }} selector: matchLabels: app: hello-world template: metadata: labels: app: hello-world spec: containers: - name: hello-world image: "{{ .Values.image.repository }}" ports: - name: http containerPort: 80 protocol: TCP
```


To install this chart, run the following command:

```
```bash theme={null}
$ helm install hello-world
```

Notice that values like the image repository and replica count are not hardcoded. Instead, they utilize Helm's templating syntax, which references configurations defined in the values.yaml file. This approach allows you to easily adjust parameters without directly editing the template files.

## Chart Metadata

Every Helm chart includes a Chart.yaml file that contains essential metadata, such as:

- **API Version:** For Helm 3, set to `v2` (Helm 2 charts use `v1` or omit this field).
- **App Version:** Indicates the version of the application deployed.
- **Chart Version:** Tracks the version of the chart itself, independent of the application version.
- **Name and Description:** Provide identification and a brief summary of the chart.
- **Type:** Specifies whether the chart is for an application (default) or is a library chart.
- **Dependencies:** Declare any external charts that the chart relies on.
- **Additional Fields:** Optional fields like keywords, maintainers, home, and icon help with discovery and branding.

Below is an example that combines Kubernetes manifest templates with chart metadata:

```
```yaml theme={null}
```

Service and Deployment templates

```
apiVersion: v1 kind: Service metadata: name: hello-world spec: type: NodePort
ports: - port: 80 targetPort: http protocol: TCP name: http selector: app: hello-world
```

```
apiVersion: apps/v1 kind: Deployment metadata: name: hello-world spec:
replicas: {{ .Values.replicaCount }} selector: matchLabels: app: hello-world
template: metadata: labels: app: hello-world spec: containers: - name: hello-world
image: "{{ .Values.image.repository }}" ports: - name: http containerPort: 80
protocol: TCP
```

values.yaml snippet

```
replicaCount: 1 image: repository: nginx
```

Chart.yaml snippet

```
apiVersion: v2 appVersion: "1.16.0" name: hello-world description: A web
application type: application
```

Again, the chart can be installed with:

```
```bash theme={null}
$ helm install hello-world
```

## Example: WordPress Chart

For a more complex use case, consider a WordPress chart that depends on additional services like MariaDB. Below is an example of a Chart.yaml file for a WordPress deployment:

```
```yaml theme={null} apiVersion: v2 appVersion: 5.8.1 version: 12.1.27 name:
wordpress description: Web publishing platform for building blogs and
websites. type: application dependencies: - condition: mariadb.enabled name:
mariadb repository: https://charts.bitnami.com/bitnami version: 9.x.x keywords:
- application - blog - wordpress maintainers: - email: containers@bitnami.com
name: Bitnami home: https://github.com/bitnami/charts/tree/master/bitnami/
wordpress icon: https://bitnami.com/assets/stacks/wordpress/img/wordpress-
stack-220x234.png
```

Helm Chart Directory Structure

A typical Helm chart directory includes the following components:

- * **templates/**: Contains all the manifest templates (e.g., Deployment, Service).
- * **values.yaml**: Defines default configuration values.
- * **Chart.yaml**: Holds metadata about the chart.
- * **charts/**: Optionally includes dependent charts (e.g., the MariaDB chart for WordPress).
- * Other optional files such as **LICENSE** or **README** for additional information.

Deploying a Chart from a Repository

To deploy the WordPress chart from the Bitnami repository, execute the following commands:

```
```bash theme={null}
$ helm repo add bitnami https://charts.bitnami.com/bitnami
$ helm install my-release bitnami/wordpress
```

You can verify your installation using similar commands:

```
plaintext theme={null} $ helm repo add bitnami https://
charts.bitnami.com/bitnami $ helm install my-release bitnami/
wordpress
```

This concludes our overview of Helm Charts. In the next lesson, we will delve deeper into chart templating techniques and explore advanced methods to customize your Kubernetes deployments.

## Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

# Working with Helm basics

This article covers basic Helm operations using the command-line interface for managing Kubernetes applications and chart repositories.

This article explains basic Helm operations using the command-line interface. When you run the Helm command without any subcommands, it provides a list of common actions and available commands.

For example, executing:

```
``plaintext theme={null} $ helm --help
```

displays:

```
``plaintext theme={null}
```

The Kubernetes package manager

Common actions for Helm:

- helm search: search for charts
- helm pull: download a chart to your local directory to view
- helm install: upload the chart to Kubernetes
- helm list: list releases of charts

Usage:

```
helm [command]
```

Available Commands:

completion	generate autocompletion scripts for the specified shell
create	create a new chart with the given name
dependency	manage a chart's dependencies
env	helm client environment information
get	download extended information of a named release
help	Help about any command
history	fetch release history

This output serves as a quick reference to the available Helm commands. For instance, if you need to revert a failed upgrade, you can quickly find out by searching within the help details that the correct command is `helm rollback`, not “restore.”

## Managing Chart Repositories

Exploring repository-related commands is straightforward. Running:

```
``plaintext theme={null} $ helm repo --help
```

```
produces:

``plaintext theme={null}
This command consists of multiple subcommands to interact with chart repositories.

It can be used to add, remove, list, and index chart repositories.

Usage:
 helm repo [command]

Available Commands:
 add add a chart repository
 index generate an index file given a directory containing packaged charts
 list list chart repositories
 remove remove one or more chart repositories
 update update information of available charts locally from chart repositories
```

This clearly outlines the commands for managing chart repositories—whether you need to add, list, remove, or update them.

## Deploying an Application on Kubernetes

Imagine you want to launch a WordPress website on Kubernetes using a chart from an online repository, such as [Artifact Hub](https://artifacthub.io/). Charts with an official or verified publisher badge ensure authenticity and reliability. Follow these steps:

1. **Note:** First, add the Bitnami repository:

```
bash theme={null} $ helm repo add bitnami https://charts.bitnami.com/bitnami
```

1. **Note:** Then, install the WordPress chart:

```
bash theme={null} $ helm install my-release bitnami/wordpress
```

After running these commands, you should see an output similar to:

```
``bash theme={null} $ helm repo add bitnami https://charts.bitnami.com/
bitnami "bitnami" has been added to your repositories
```

```
$ helm install my-release bitnami/wordpress NAME: my-release LAST
DEPLOYED: Wed Nov 10 18:03:50 2021 NAMESPACE: default STATUS:
```

deployed REVISION: 1 TEST SUITE: None NOTES: CHART NAME:  
wordpress CHART VERSION: 12.1.27 APP VERSION: 5.8.1

**\*\* Please be patient while the chart is being deployed \*\*** Your WordPress site can be accessed through the following DNS name from within your cluster: my-release-wordpress.default.svc.cluster.local (port 80)

```
This output confirms that your WordPress application is successfully deployed and provides the following information:

Searching for Charts

Helm offers robust search functionality to help you find specific charts. To search for a chart, run:

```bash theme={null}
$ helm search hub wordpress
```

This command returns a list of available WordPress charts with details such as chart version, app version, and a short description. If you prefer to search within your local repositories, simply run:

```
```plaintext theme={null} $ helm search wordpress
```

```
This command provides similar information that helps you identify the correct chart version.

Managing Helm Releases

Once a chart is deployed, it is managed as a release. To view all existing releases, run:

```bash theme={null}
$ helm list
NAME          NAMESPACE    REVISION    UPDATED                               STATUS    CHART
my-release    default       1           2021-11-10 18:03:50.414174217 +0000 UTC deployed    wordpress
```

If you later decide to remove the deployed release (for instance, the WordPress website), you can uninstall it with:

```
```bash theme={null} $ helm uninstall my-release
```

```
This command cleans up all Kubernetes objects that were created during deployment.

Managing Local Helm Repository Data

To view your configured chart repositories, execute:

```plaintext theme={null}
$ helm repo list
NAME    URL
bitnami https://charts.bitnami.com/bitnami
```

Over time, locally cached repository data can become outdated. To refresh this information, run:

```
plaintext theme={null} $ helm repo update Hang tight while we grab  
the latest from your chart repositories... ...Successfully got an  
update from the "bitnami" chart repository Update Complete. 🚀 Happy  
Helming! 🚀
```

This command ensures you have the latest chart data locally.

Wrapping Up: This article provided an overview of basic Helm CLI operations—from accessing help information to deploying and managing applications on Kubernetes. Experiment with these commands in your environment to become more confident with Helm.

Happy Helming!

Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/lms.txt> Use this file to discover all available pages before exploring further.

Customizing chart parameters

Learn to customize chart parameters during a Helm chart installation, including overriding default values for WordPress using command-line options or custom values files.

In this guide, you'll learn how to customize chart parameters during a Helm chart installation. When you deploy WordPress using the Bitnami chart, it uses the default values defined in the chart's values.yaml file. For example, the default blog name is set as "User's Blog!" in the values file. This article explains how this value is configured and outlines the various methods available for overriding it.

Understanding the Default Configuration

The WordPress application is deployed using a Kubernetes Deployment resource. Its configuration is partly derived from the values set in values.yaml. Below is a snippet from the values.yaml file indicating the default settings:

```
``yaml theme={null}
```

values.yaml snippet

```
image: registry: docker.io repository: bitnami/wordpress tag: 5.8.2-debian-10-r0
```

@param wordpressUsername

WordPress username

```
wordpressUsername: user
```

@param wordpressPassword WordPress user password

Defaults to a random 10-character alphanumeric string if not set

wordpressPassword: ""

@param existingSecret

existingSecret: ""

@param wordpressEmail WordPress user email

wordpressEmail: user@example.com

@param wordpressBlogName Blog name

wordpressBlogName: User's Blog!

The corresponding Deployment template uses these values to configure the environment variables:

```

```yaml
theme={null}
Deployment template snippet
apiVersion: {{ include "apiVersion" }}
kind: Deployment
metadata:
 name: {{ include "common.names.fullname" }}
 namespace: {{ .Release.Namespace | quote }}
 labels: {{- include "common.labels.standard" | nindent 4 }}
spec:
 selector:
 matchLabels: {{- include "common.labels.matchLabels" }}
 replicas: {{ .Values.replicaCount }}
 template:
 spec:
 containers:
 - name: wordpress
 image: {{ template "wordpress.images" . }}
 env:
 - name: WORDPRESS_DATABASE_NAME
 value: {{ include "wordpress.databaseName" | quote }}
 - name: WORDPRESS_DATABASE_USER
 value: {{ include "wordpress.databaseUser" | quote }}
 - name: WORDPRESS_USERNAME
 value: {{ .Values.wordpressUsername | quote }}
 - name: WORDPRESS_PASSWORD
 valueFrom:
 secretKeyRef:
 name: {{ include "wordpress.secretName" }}
 key: wordpress-password
 - name: WORDPRESS_BLOG_NAME
 value: {{ .Values.wordpressBlogName | quote }}

```

The `WORDPRESS_BLOG_NAME` environment variable is directly set using the value from `values.yaml`, meaning that the application will deploy with "User's Blog!" as the default blog name unless it's overridden.

## Overriding Default Values with Command-Line Parameters

When you install WordPress with Helm, the chart deploys using the default values from the `values.yaml` file. To override these defaults on the fly, you can use the `--set` option with the `helm install` command. For example, to change the blog name from "User's Blog!" to "Helm Tut", use the following command:

```

bash theme={null} $ helm install --set wordpressBlogName="Helm Tut"
my-release bitnami/wordpress

```

You can override multiple parameters simultaneously. For instance, you may adjust both the WordPress blog name and the user email. Command-line

parameters provided using `--set` will take precedence over the defaults defined in the `values.yaml` file.

## Using a Custom Values File

For numerous parameter overrides, maintaining a custom values file is more efficient than using multiple `--set` options. Follow these steps:

1. Create a file named `custom-values.yaml` with your custom configurations:

```
yaml theme={null} # custom-values.yaml snippet wordpressBlogName:
Helm Tut wordpressEmail: john@example.com
```

1. Deploy the chart using the custom values file by running:

```
bash theme={null} $ helm install my-release bitnami/wordpress --
values custom-values.yaml
```

This instructs Helm to apply the configuration from `custom-values.yaml`, thereby overriding the corresponding default values.

## Modifying the Built-in values.yaml File

If you wish to modify the built-in `values.yaml` within the chart itself, you can do so by following these steps:

1. Use `helm pull` to download the chart in an archived (compressed) form:

```
bash theme={null} $ helm pull bitnami/wordpress
```

1. To automatically uncompress the chart into a directory, use the `--untar` option:

```
bash theme={null} $ helm pull --untar bitnami/wordpress
```

1. List the directory to view the chart files, including `values.yaml`:

```
``bash theme={null} $ ls wordpress
$ ls wordpress ci templates Chart.lock Chart.yaml README.md
values.schema.json values.yaml ``
```

1. Open and edit the `values.yaml` file in your preferred text editor to set your custom configurations.
2. Install the chart locally by referencing the modified chart path:

```
bash theme={null} $ helm install my-release ./wordpress
```

In this command, `./` indicates the current directory, and Helm installs the chart using your modified files.

When modifying the built-in values.yaml file, ensure you maintain the correct file structure to avoid deployment issues.

---

This concludes our discussion on customizing chart parameters. You have learned how to override default values in a Bitnami WordPress Helm chart using command-line options with `--set`, by employing a custom values file, or by directly modifying the chart's source after pulling it locally. For more details on Helm and managing Kubernetes resources, explore the [official Helm documentation](#) and [Kubernetes Basics](#).

## Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

# Lifecycle management with Helm

This article explores managing Kubernetes application lifecycles using Helm, covering releases, upgrades, and rollbacks with practical examples.

In this article, we explore how to effectively manage the lifecycle of Kubernetes applications using Helm. Learn how Helm handles releases, upgrades, and rollbacks through real-world examples that simplify complexity and enhance application management.

## Creating and Managing Releases

When you install a Helm chart, a release is created. Each release is like an application package—a collection of related Kubernetes objects. Since Helm tracks all objects associated with a release, it allows you to upgrade, downgrade, or uninstall a release without affecting other deployments. For instance, even if you deploy the same chart twice, each release remains independent:

```
```bash theme={null} $ helm install my-site bitnami/wordpress $ helm install my-SECOND-site bitnami/wordpress
```

Installing an Older Version

To see Helm in action, let's create a new release by installing an older version of the NGINX

```
```bash theme={null}
$ helm install nginx-release bitnami/nginx --version 7.1.0
```

This command deploys an NGINX release named "nginx-release" using an earlier version of NGINX. After installation, verify the Pod status and details of the image:

```
```bash theme={null} $ kubectl get pods NAME READY STATUS RESTARTS
AGE nginx-release-687cdd5c75-ztn2n 0/1 ContainerCreating 0 13s
```

Once the Pod is running, get detailed information about the image:

```
```bash theme={null}
$ kubectl describe pod nginx-release-687cdd5c75-ztn2n
Containers:
 nginx:
 Container ID: docker://81bb5ad6b5...
 Image: docker.io/bitnami/nginx:1.19.2-debian-10-r28
 Image ID: docker-pullable://bitnami/nginx@sha256:2fcaf0268ac7a...
 Port: 8080/TCP
 Host Port: 0/TCP
 State: Running
```

In this case, the installed NGINX version is 1.19.2, which might become outdated over time.

If you discover security vulnerabilities or need feature updates, Helm makes it easy to upgrade your application along with all associated Kubernetes configurations.

## Upgrading Releases

Helm's upgrade functionality allows seamless transition to new versions. When you upgrade a release, Helm replaces the old Pod with a new one that includes updated settings and images. Here's an example of upgrading the existing NGINX release:

```
```bash theme={null} $ helm upgrade nginx-release bitnami/nginx
Release "nginx-release" has been upgraded. Happy Helming! NAME: nginx-release
LAST DEPLOYED: Mon Nov 15 19:25:55 2021 NAMESPACE: default
STATUS: deployed REVISION: 2 TEST SUITE: None NOTES: CHART NAME:
nginx CHART VERSION: 9.5.13 APP VERSION: 1.21.4
```

After upgrading, verify the updates with the following commands:

```
```bash theme={null}
$ helm list
NAME NAMESPACE REVISION STATUS CHART APP VERSION
nginx-release default 2 deployed nginx-9.5.13 1.21.4
```

Check the revision history to get more insights into the changes:

```
```bash theme={null} $ helm history nginx-release
REVISION    UPDATED
STATUS CHART APP VERSION DESCRIPTION
1 Mon Nov 15 19:20:51 2021
superseded nginx-7.1.0 1.19.2 Install complete
2 Mon Nov 15 19:25:55 2021
deployed nginx-9.5.13 1.21.4 Upgrade complete
```

Rollbacks

If an upgrade leads to unexpected behavior, Helm supports rollbacks

Rolling back reverts t

```

```bash theme={null}
$ helm rollback nginx-release 1
Rollback was a success! Happy Helming!

```

After a rollback, the configuration reverts to the settings in revision 1. However, note that Helm records this as a new revision, providing a complete history for audit and troubleshooting.

Remember that while rollbacks restore Kubernetes manifest configurations, they do not include the actual data stored in persistent volumes or external databases. Always ensure you have a separate backup solution for persistent data.

## Upgrading Complex Charts

When working with more complex applications, such as WordPress, additional parameters might be required during an upgrade. Missing these parameters can result in errors like the one below:

```

```bash theme={null} $ helm upgrade wordpress-release bitnami/wordpress
Error: UPGRADE FAILED: template: wordpress/templates/NOTES.txt:83:4:
executing "wordpress/templates/NOTES.txt" at : error calling include: template:
wordpress/charts/common/templates/_errors.tpl:21:48:      executing
"common.errors.upgrade.passwords.empty" at : error calling fail:
PASSWORDS ERROR: You must provide your current passwords when
upgrading the release. Note that even after reinstallation, old credentials may
be needed as they may be kept in persistent volume claims. Further
information can be obtained at https://docs.bitnami.com/general/how-to/troubleshoot-helm-chart-issues/#credential-errors-while-upgrading-chart-releases

```

'wordpressPassword' must not be empty, please add '--set wordpressPassword=\$WORDPRESS_PASSWORD' to the command. To get the current value: export WORDPRESS_PASSWORD=\$(kubectl get secret --namespace "default" wordpress-release -o jsonpath="{.data.wordpress-password}" | base64 --decode)

'mariadb.auth.rootPassword' must not be empty, please add '--set mariadb.auth.rootPassword=\$MARIADB_ROOT_PASSWORD' to the command. To get the current value: export MARIADB_ROOT_PASSWORD=\$(kubectl get secret --namespace "default" wordpress-release-mariadb -o jsonpath="{.data.mariadb-root-password}" | base64 --decode)

'mariadb.auth.password' must not be empty, please add '--set mariadb.auth.password=\$MARIADB_PASSWORD' to the command. To get the current value: `export MARIADB_PASSWORD=$(kubectl get secret --namespace "default" wordpress-release-mariadb -o jsonpath="{.data.mariadb-password}" | base64 --decode) ```

This error indicates that administrative credentials must be provided for certain components during the upgrade. Always supply the necessary parameters to ensure that all Kubernetes objects and application components are appropriately updated.

Summary

Helm simplifies lifecycle management by:

Action	Description	Command Example
Install	Create a new release from a Helm chart.	<code>helm install my-release bitnami/nginx --version 7.1.0</code>
Upgrade	Update an existing release to a new version with all associated configurations.	<code>helm upgrade my-release bitnami/nginx</code>
Rollback	Revert a release to a previous configuration state in case of issues.	<code>helm rollback my-release 1</code>

By following the practices outlined in this article, you can streamline application management across your Kubernetes clusters. Practice these Helm commands with hands-on exercises to deepen your understanding and improve your deployment workflows.

For more information, consider exploring the [Helm Documentation](#).

Section: Kustomize Basics

(2025 Updates)

Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

Kustomize vs Helm

This guide compares Helm and Kustomize for modifying Kubernetes manifests, highlighting their features, advantages, and trade-offs to help choose the best solution for projects.

Before moving forward, it's important to examine an alternative tool to Kustomize: Helm. This guide provides a high-level overview of how Helm addresses the need to modify Kubernetes manifests on a per-environment basis. By understanding both tools and their trade-offs, you can choose the best solution for your project.

Helm Templating Fundamentals

Helm utilizes Go templating syntax to define variables within your Kubernetes manifests. For example, consider the template snippet below, which outlines a standard Deployment configuration. Notice the use of double curly braces to denote template variables:

```
``yaml theme={null} apiVersion: apps/v1 kind: Deployment metadata: name:
{{ .Values.name }} spec: replicas: {{ .Values.replicaCount }} selector:
matchLabels: app: {{ .Values.name }} template: metadata: labels: app:
{{ .Values.name }} spec: containers: - name: {{ .Chart.Name }} image: "nginx:
{{ .Values.image.tag }}"
```

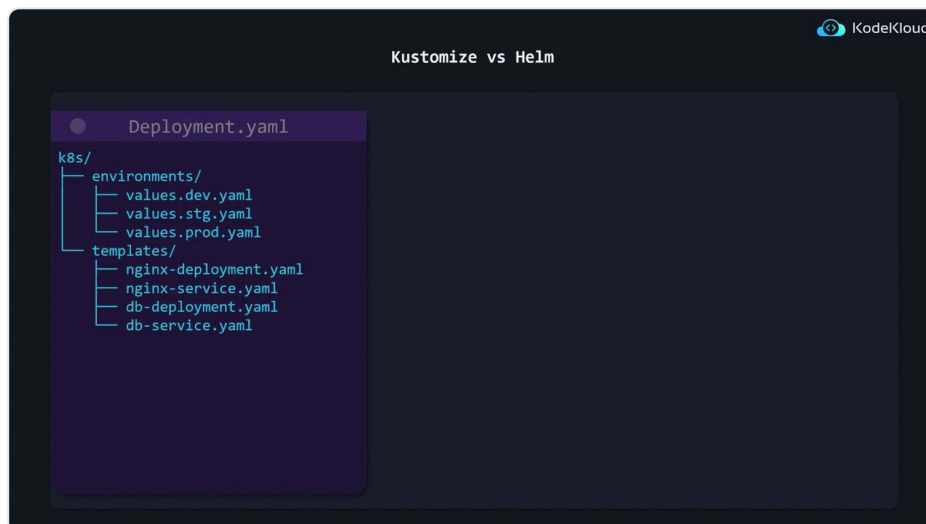
```
In this configuration, the replicas count (`replicaCount`) is not hardcoded. Instead, its value is defined in the values file.

```yaml
theme: {null}
replicaCount: 1
image:
 tag: "2.4.4"
```

When deploying your application, Helm merges the specified values into the template, replacing placeholders with concrete values.

## Project Structure in a Traditional Helm Setup

A conventional Helm project organizes deployment templates in one directory and environment-specific configuration files in another. For instance, you might have a directory structure where the `environments` folder contains files such as `values.dev.yaml`, `values.stg.yaml`, and `values.prod.yaml`, while the `templates` folder stores the Kubernetes manifest files.



Below is an example of how the directory structure may be organized:

```
yaml theme={null} k8s/ environments/ values.dev.yaml values.stg.yaml
values.prod.yaml templates/ nginx-deployment.yaml nginx-service.yaml
db-deployment.yaml db-service.yaml
```

When deploying your application, you select the relevant environment file to inject the proper values into your templates.

## Advantages and Trade-offs of Helm

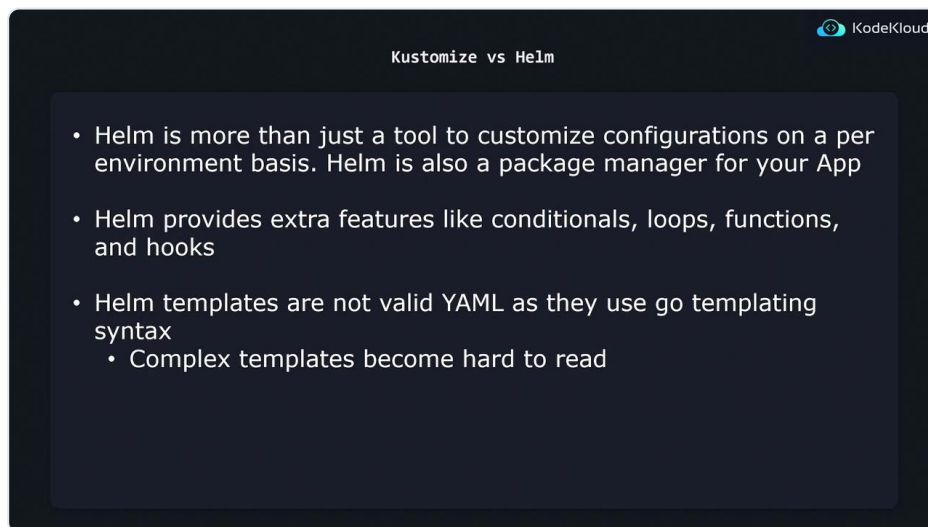
Helm is more than just a templating tool for environment-specific configurations—it serves as a complete package manager for Kubernetes applications.

Similar to package managers like yum or apt in Linux, Helm provides additional features such as:

- Conditionals and loops
- Functions and hooks

While Helm's rich feature set offers significant flexibility, it also introduces complexity. The templates, which include Go templating syntax, are not strictly valid YAML and can be challenging to read and maintain initially.

A comparison between the two tools highlights that while Helm offers advanced capabilities like package management and flexible templating constructs, these benefits come with increased complexity. In contrast, Kustomize relies on plain YAML for base configurations and overlays for environment modifications, which makes it simpler to read and maintain.



## Choosing the Right Tool

When deciding between Helm and Kustomize, consider the specific requirements of your project and your tolerance for complexity:

- Choose **Helm** if you need advanced functionalities like package management, dynamic templating, and additional programming constructs.
- Choose **Kustomize** if you prefer a straightforward, plain YAML approach that improves readability and simplifies maintenance.

Be mindful that Helm templates can become difficult to manage as complexity increases. Always weigh the benefits against the added learning curve and potential maintenance overhead.

## Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

# InstallationSetup

This guide explains how to install and set up Kustomize for customizing Kubernetes resource configurations.

In this guide, you'll learn how to install and set up Kustomize—a powerful tool for customizing Kubernetes resource configurations. Before proceeding, ensure that you have a running Kubernetes cluster and that [kubectl](#) is installed and configured on your local machine. Kustomize supports Linux, Windows, and macOS.

## Installation Steps

The Kustomize team provides a convenient installation script that automatically detects your operating system and installs the appropriate version. To download and run this script, execute the following command in your terminal:

```
```bash theme={null} curl -s "https://raw.githubusercontent.com/kubernetes-sigs/kustomize/master/hack/install_kustomize.sh" | bash
```

After the script completes, verify the installation by checking the version of Kustomize with

```
```bash theme={null}
kustomize version --short
```

You should see an output resembling the following:

```
text theme={null} {kustomize/v4.4.1 2021-11-11T23:36:27Z }
```

If you do not see the expected version output, it may indicate an installation issue or that your current terminal session has not updated the necessary environment variables.

## Troubleshooting

---

If you encounter any issues during installation, try the following steps:

1. Close your current terminal session and reopen it.
2. Rerun the installation script to ensure that all components are set up correctly.

For more detailed troubleshooting and advanced configuration options, refer to the [Kustomize Documentation](#).

With Kustomize properly installed, you are now ready to explore its powerful features for managing Kubernetes configurations. Enjoy customizing your Kubernetes deployments!

## Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

# Kustomize Output

This article explains how to deploy and delete Kubernetes resources using Kustomize in conjunction with the `kubectl` command.

In the previous lesson, we explored the Kustomize build command. Running this command processes all defined resources, applies the necessary transformations, and outputs the final configurations to your console. Remember, these configurations are not automatically deployed to your Kubernetes cluster. When you inspect your cluster using commands like `kubectl get pods`, `kubectl get deployments`, or `kubectl get services`, you'll notice that no resources have been created yet.

Kustomize focuses on building configurations. To deploy these configurations, you must combine it with the `kubectl apply` command.

## Deploying Generated Configurations

To deploy your configurations, you can chain the Kustomize build command with the `kubectl apply` command using the Linux pipe (`|`). The command below demonstrates how to build and immediately apply the configurations from the `k8s/` directory:

```
```bash theme={null} $ kustomize build k8s/ | kubectl apply -f - service/nginx-loadbalancer-service created deployment.apps/nginx-deployment created
```

The Linux pipe utility takes the output from ``kustomize build k8s/`` and supplies it as the

Alternatively, the ``kubectl`` command includes native support for Kustomize. By using the ```

```
```bash theme={null}
$ kubectl apply -k k8s/
service/nginx-loadbalancer-service created
deployment.apps/nginx-deployment created
```

## Deleting Resources Using Kustomize

Removing resources works in a similar way to deploying them. You simply replace the word `apply` with `delete`. This section demonstrates how to delete resources with both methods.

### Using the Pipe Method

The following command uses the pipe utility to delete the previously created resources:

```
```bash theme={null} $ kustomize build k8s/ | kubectl delete -f - service "nginx-loadbalancer-service" deleted deployment.apps "nginx-deployment" deleted
```

```
### Using the Native kubectl Command
```

You can also use the `kubectl` command with the `-k` flag to delete the resources:

```
```bash theme={null}
$ kubectl delete -k k8s/
service "nginx-loadbalancer-service" deleted
deployment.apps "nginx-deployment" deleted
```

This article has demonstrated how to both apply and delete Kubernetes resources using Kustomize in combination with `kubectl`. For more details on using Kubernetes tools and best practices, refer to the [Kubernetes Documentation](#).

## Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

# Managing Directories

This guide explains how to structure Kubernetes YAML files effectively using Kustomize for streamlined management and deployment workflows.

Organizing and managing your Kubernetes manifests across multiple directories can be streamlined with Kustomize. This guide explains how to structure your YAML files effectively, simplify deployment workflows, and maintain a clean configuration hierarchy for your clusters.

## Basic Directory Structure Without Kustomize

Initially, you might store all your Kubernetes YAML files in a single directory (e.g., a directory named "k8s"). In this simple setup, you could have files such as:

- API deployment YAML file
- API service YAML file
- Database deployment YAML file
- Database service YAML file

To deploy these configurations, navigate to your "k8s" directory and run:

```
```bash theme={null} kubectl apply -f .
```

This method works well for a small number of files. However, as your application scales, you

Organizing YAML Files into Subdirectories

A more structured approach is to organize your manifests into subdirectories. For instance,

```
```bash theme={null}
kubectl apply -f k8s/api/
```

```
```bash theme={null} kubectl apply -f k8s/db/
```



```
While this method is functional, it may become cumbersome when dealing with numerous subdirectories.

## Simplifying Deployment with Kustomize

Kustomize simplifies this process by letting you define a single `kustomization.yaml` file.

```yaml
theme={null}
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization

Kubernetes resources to be managed by Kustomize
resources:
 - api/api-depl.yaml
 - api/api-service.yaml
 - db/db-depl.yaml
 - db/db-service.yaml
```

With this configuration, deploy all resources from the root directory using one of the following commands:

```
```bash
theme={null} kustomize build k8s/ | kubectl apply -f -
```

Or leverage kubectl's native Kustomize support:

```
```bash
theme={null}
kubectl apply -k k8s/
```

These commands aggregate the YAML files specified in `kustomization.yaml` and deploy them simultaneously, eliminating the need to apply individual directories manually.

## Scaling with Multiple Directories

As your Kubernetes project grows, you might add more subdirectories for additional services like caches or messaging systems (e.g., Redis or Kafka). For instance, your root `kustomization.yaml` file might expand to include these new resources:

```
```yaml
theme={null}
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization
```

Kubernetes resources across multiple subdirectories

```
resources: - api/api-depl.yaml - api/api-service.yaml - db/db-depl.yaml - db/db-service.yaml
- cache/redis-depl.yaml - cache/redis-service.yaml - cache/redis-config.yaml
- kafka/kafka-depl.yaml - kafka/kafka-service.yaml - kafka/kafka-config.yaml
```

```
<Callout icon="lightbulb" color="#1CB2FE">
```

```
  Consider breaking down the configurations into subdirectories with their own `kustomization.yaml` files.
</Callout>
```

```
### Using Individual kustomization.yaml Files in Subdirectories
```

```
For improved maintainability, create a separate `kustomization.yaml` file within each subdirectory.
```

```
``yaml theme={null}
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization

resources:
  - db-depl.yaml
  - db-service.yaml
```

Then, update the root `kustomization.yaml` file to reference each subdirectory:

```
``yaml theme={null} apiVersion: kustomize.config.k8s.io/v1beta1 kind:
Kustomization

resources: - api/ - db/ - cache/ - kafka/
```

```
Kustomize will recursively process each subdirectory's `kustomization.yaml`, aggregating all manifests.
```

```
## Deploying the Configurations
```

```
After setting up both the root and subdirectory-specific `kustomization.yaml` files, deploy the configurations.
```

```
``bash theme={null}
kustomize build k8s/ | kubectl apply -f -
```

Or, using kubectl's built-in Kustomize support:

```
bash theme={null} kubectl apply -k k8s/
```

Both methods compile all the manifests from your subdirectories and apply them in a single, efficient deployment process.

Conclusion

By organizing your Kubernetes YAML files into dedicated subdirectories and leveraging Kustomize with individual `kustomization.yaml` files, you can manage complex deployments more effectively. This approach not only cleans up your configuration hierarchy but also streamlines CI/CD pipelines by reducing repetitive commands.

For more insights on Kubernetes configuration management, consider exploring the following resources:

- [Kubernetes Basics](#)

- [Kubernetes Documentation](#)
- [Kustomize Overview](#)

Always test your Kustomize configurations in a staging environment before deploying to production. This practice helps ensure that your aggregated manifests work as expected across your Kubernetes clusters.

Documentation Index

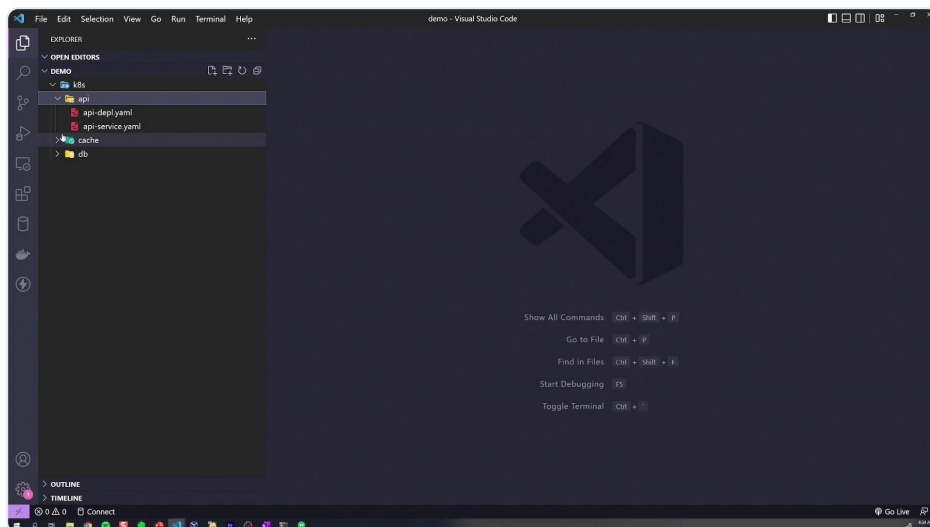
Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

Managing Directories Demo

This article explores managing Kubernetes manifest directories and introduces Kustomize for simplifying resource management and deployment.

In this lesson, we explore how to effectively manage directories containing Kubernetes manifests. The demonstration uses a structured "K8s" directory that holds all Kubernetes configurations organized into three subdirectories: one for the API, one for the cache (acting as a readers' database), and one for the MongoDB database.

When you open the K8s directory, you'll see three distinct folders. Each folder includes configuration files (YAML manifests) tailored for a specific component. For instance, the database folder contains the deployment YAML files for MongoDB, while the API and cache directories contain configurations for services such as ClusterIP or LoadBalancer services along with associated ConfigMaps.



Below is an excerpt showcasing a typical service configuration for the cache component:

```
```yaml theme={null} apiVersion: v1 kind: Service metadata: name: redis-cluster-ip-service spec: type: ClusterIP selector: component: redis ports: - port: 6379 targetPort: 6379
```

Before we introduce Kustomize, let's deploy these resources using the conventional approach

```
```bash theme={null}
kubectl apply -f k8s/api
# Output:
# deployment.apps/api-deployment created
kubectl apply -f k8s/cache
# (Applies cache-related configurations)
```

You can also deploy all configurations in one command by appending multiple `-f` flags:

```
```bash theme={null} kubectl apply -f k8s/api kubectl apply -f k8s/cache kubectl
apply -f k8s/db
```

To delete all resources simultaneously, you can run:

```
```bash theme={null}
kubectl delete -f k8s/db -f k8s/cache -f k8s/api
```

Using multiple `-f` flags simplifies bulk deployment and deletion. However, as your infrastructure grows, managing these commands can become cumbersome.

Simplifying Resource Management with Kustomize

Kustomize makes it easier to manage and customize your application configurations. Begin by creating a `kustomization.yaml` file in the root of your K8s directory. This file specifies the API version and kind required by Kustomize:

```
```yaml theme={null} apiVersion: kustomize.config.k8s.io/v1beta1 kind:
Kustomization
```

```

Next, define the resources you wish to manage by listing the file paths relative to the "k8s/" directory in the following YAML file:

```yaml
theme: {null}
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization
resources:
- api/api-depl.yaml
- api/api-service.yaml
- cache/redis-config.yaml
- cache/redis-depl.yaml
- cache/redis-service.yaml
- db/db-config.yaml
- db/db-depl.yaml
- db/db-service.yaml

```

You can then build the complete set of manifests using the Kustomize CLI:

```
```bash
theme={null} kustomize build k8s/

```

```

This command outputs the final Kubernetes manifests, combining configurations from the API, cache, and database directories.

```bash
theme={null}
kustomize build k8s/ | kubectl apply -f -

```

Alternatively, leverage the built-in support for Kustomize in kubectl with the `-k` flag:

```
```bash
theme={null} kubectl apply -k k8s/

```

```

After applying the configurations, verify that the resources have been successfully created:

```bash
theme={null}
kubectl get pods

```

Expected output:

```

```bash
theme={null}
NAME READY STATUS RESTARTS AGE api-
deployment-64dd567b46-1mw4c 1/1 Running 0 27s db-
deployment-657c8fbd8-vnjs7 1/1 Running 0 26s redis-
deployment-587fd758cf-7pt57 1/1 Running 0 26s

```

```
<Callout icon="lightbulb" color="#1CB2FE">
 For quick troubleshooting, always verify your pods' status with `kubectl get pods` after
</Callout>

Advanced Directory Structuring with Kustomize

While maintaining a single `kustomization.yaml` in the root directory works for simple projects, more complex setups require a hierarchical structure.

API Directory Configuration

In the API folder, create a `kustomization.yaml` that lists the API deployment and service:

```yaml theme={null}
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization
resources:
  - api-depl.yaml
  - api-service.yaml
```

Cache Directory Configuration

In the cache directory, set up the following `kustomization.yaml`:

```
```yaml theme={null} apiVersion: kustomize.config.k8s.io/v1beta1 kind:
Kustomization resources: - redis-config.yaml - redis-depl.yaml - redis-
service.yaml
```

```
Database Directory Configuration

Similarly, for the database directory, create a `kustomization.yaml` file:

```yaml theme={null}
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization
resources:
  - db-config.yaml
  - db-depl.yaml
  - db-service.yaml
```

Root Directory Aggregation

Finally, update the root `kustomization.yaml` to reference these subdirectories. When a directory is specified as a resource, Kustomize automatically searches for a `kustomization.yaml` file inside:

```
```yaml theme={null} apiVersion: kustomize.config.k8s.io/v1beta1 kind:
Kustomization resources: - api/ - cache/ - db/
```

Before re-deploying, delete any previously applied resources:

```
```bash theme={null}
kubectl delete -f k8s/db -f k8s/cache -f k8s/api
```

Then, build the final configuration with Kustomize:

```
```bash theme={null} kustomize build k8s/
```

Review the output to ensure it meets your expectations, then apply the aggregated configuration:

```
```bash theme={null}
kubectl apply -k k8s/
```

The expected output should be similar to:

```
```bash theme={null} configmap/redis-credentials created service/api-service
created service/db-service created service/redis-cluster-ip-service created
deployment.apps/api-deployment created deployment.apps/db-deployment
created deployment.apps/redis-deployment created
```

Verify the pods again:

```
```bash theme={null}
kubectl get pods
```

Expected output:

bash	theme={null}	NAME	READY	STATUS	RESTARTS	AGE	api-
deployment-64dd567b46-1mw4c			1/1	Running	0	27s	db-
deployment-657c8fbd8-vnjs7			1/1	Running	0	26s	redis-
deployment-587fd758cf-7pt57			1/1	Running	0	26s	

This structured approach using Kustomize not only centralizes the management of your Kubernetes configurations but also offers a scalable solution for handling an expanding set of resources within your infrastructure.

Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

Common Transformers

Learn to use Kustomize transformers for modifying Kubernetes configurations, focusing on common transformations for consistent resource management.

In this lesson, you will learn how to use Kustomize transformers to modify Kubernetes configurations. Kustomize supports several built-in transformers, and you can also create custom ones. Here, we focus on a subgroup known as Common Transformers.

Imagine you have multiple YAML files such as `deployment.yaml` and `service.yaml`. You might want to apply a common configuration—for example, adding a label like `org: KodeKloud` or appending `-dev` to resource names—across all these files. Manually updating each file in a production environment isn't scalable or efficient. Kustomize transformers offer a systematic way to make consistent changes across all resources.

Below are the original Kubernetes resource examples:

```
```yaml theme={null} apiVersion: apps/v1 kind: Deployment metadata: name:
api-deployment spec: replicas: 1 selector: matchLabels: component: api
template: metadata: labels: component: api spec: containers: - name: nginx
image: nginx
```

```
```yaml theme={null}
apiVersion: v1
kind: Service
metadata:
  name: db-service
spec:
  selector:
    component: db-depl
  ports:
    - protocol: "TCP"
      port: 27017
      targetPort: 27017
  type: LoadBalancer
```

After applying Kustomize transformations, the resources might look like this:

```
```yaml theme={null} apiVersion: apps/v1 kind: Deployment metadata: name:
api-deployment-dev spec: replicas: 1 selector: matchLabels: component: api
org: KodeKloud template: metadata: labels: component: api org: KodeKloud
spec: containers: - name: nginx image: nginx
```

```
```yaml theme={null}
apiVersion: v1
kind: Service
metadata:
  labels:
    org: KodeKloud
    name: api-service-dev
spec:
  selector:
    component: api
  ports:
    - protocol: "TCP"
      port: 80
      targetPort: 3000
  type: LoadBalancer
```

Kustomize transformers are essential for ensuring that your Kubernetes configurations remain consistent and manageable across various environments.

Common Transformation Methods

Below is an overview of common transformations available in Kustomize for managing Kubernetes resources:

1. Common Label Transformation

This transformer automatically adds the specified labels to all Kubernetes resources. You can define the labels in your `kustomization.yaml` file as shown below:

```
```yaml theme={null} commonLabels: org: KodeKloud
```

For example, a transformed Service resource would appear as:

```
```yaml theme={null}
apiVersion: v1
kind: Service
metadata:
  labels:
    org: KodeKloud
    name: api-service
spec:
  ports:
    - port: 80
      protocol: TCP
      targetPort: 3000
  selector:
    component: api
  type: LoadBalancer
```

2. Namespace Transformation

The namespace transformer assigns all Kubernetes resources to a specified namespace. By specifying the namespace in your `kustomization.yaml`, all resources will be modified accordingly. For example:

```
```yaml theme={null} namespace: lab
```

After this transformation, a Service resource might look like:

```
```yaml theme={null}
apiVersion: v1
kind: Service
metadata:
  annotations:
    branch: master
  labels:
    org: KodeKloud
    name: api-service
    namespace: lab
spec:
  ports:
    - port: 80
      protocol: TCP
      targetPort: 3000
  selector:
    component: api
    org: KodeKloud
  type: LoadBalancer
```

When applying namespace transformations, ensure that the specified namespace exists in your cluster to avoid deployment issues.

3. Name Prefix and Suffix Transformation

This transformer enables you to systematically add a prefix or suffix to resource names. For instance, to prepend "KodeKloud-" and append "-dev" to each resource name, include the following in your `kustomization.yaml`:

```
``yaml theme={null} namePrefix: KodeKloud- nameSuffix: -dev
```

After applying this configuration, a Service resource would be renamed to "KodeKloud-api-svc-dev".

```
``yaml theme={null}
apiVersion: v1
kind: Service
metadata:
  name: KodeKloud-api-service-dev
spec:
  ports:
    - port: 80
      protocol: TCP
      targetPort: 3000
  selector:
    component: api
  type: LoadBalancer
```

4. Common Annotation Transformation

If you need to add specific annotations to all resources, use the common annotations transformer. By setting the annotations in your `kustomization.yaml`, each resource will automatically include them. For example:

```
``yaml theme={null} commonAnnotations: branch: master
```

This transformation results in a Service resource similar to:

```
```yaml theme={null}
apiVersion: v1
kind: Service
metadata:
 annotations:
 branch: master
 labels:
 org: KodeKloud
 name: api-service
 namespace: auth
spec:
 ports:
 - port: 80
 protocol: TCP
 targetPort: 3000
 selector:
 component: api
 org: KodeKloud
 type: LoadBalancer
```

---

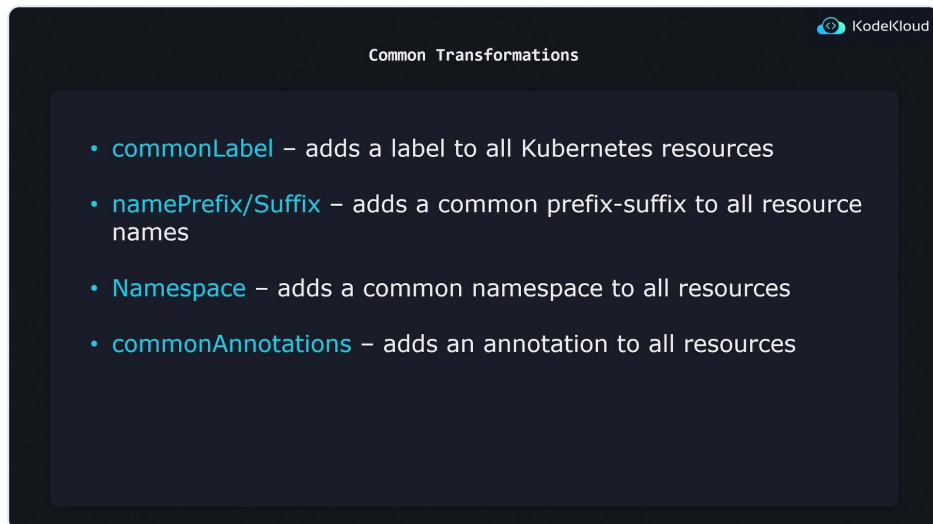
## Summary

---

The common transformations available in Kustomize include:

Transformation Type	Purpose
Common Label Transformation	Adds specified labels (e.g., org: KodeKloud) to all resources
Namespace Transformation	Assigns a specific namespace to all resources
Name Prefix and Suffix	Adds predetermined prefixes and suffixes to resource names
Common Annotation Transformation	Appends specific annotations (e.g., branch: master) to resources

These methods provide a scalable and systematic approach to maintaining consistent configurations across your Kubernetes resources.



In summary, Kustomize transformers offer a robust and error-resistant way to apply common configurations—such as labels, namespaces, name modifications, and annotations—to your Kubernetes resources, ensuring that your deployments remain consistent and manageable across various environments.

## Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

# Image Transformers

Learn to use Kustomize's image transformer for updating Kubernetes deployment images without modifying manifests directly.

In this article, you will learn how to use Kustomize's image transformer to update the images used by your Kubernetes deployments without modifying your deployment manifests directly. This technique allows you to change either the entire image or just update its tag.

## Example Deployment

Consider the following deployment that uses an Nginx container. This configuration is defined in a file named `deployment.yaml`:

```
```yaml theme={null} apiVersion: apps/v1 kind: Deployment metadata: name: web-deployment spec: replicas: 1 selector: matchLabels: component: web template: metadata: labels: component: web spec: containers: - name: web image: nginx
```

Transforming the Image

To transform the image used by the deployment, create a `kustomization.yaml` file that specifies the image transformer.

```
```yaml theme={null} images: - name: nginx newName: haproxy
```

Kustomize will scan all Kubernetes configuration files for containers that use the image named "nginx" and replace them with "haproxy".

The "name" property in the kustomization file is related to the image reference and is separate from the container name (which is "web" in the deployment manifest). This is a common point of confusion.

After applying this transformation, the updated deployment configuration will look like:

```
```yaml theme={null} apiVersion: apps/v1 kind: Deployment metadata: name: web-deployment spec: replicas: 1 selector: matchLabels: component: web template: metadata: labels: component: web spec: containers: - name: web image: haproxy
```

Updating Only the Image Tag

If your goal is to update only the image tag, you can do so by using the `newTag` property.

```
```yaml theme={null}
images:
 - name: nginx
 newTag: 2.4
```

After the transformation, the image reference in your deployment will update to include the new tag:

```
```yaml theme={null} apiVersion: apps/v1 kind: Deployment metadata: name: web-deployment spec: replicas: 1 selector: matchLabels: component: web template: metadata: labels: component: web spec: containers: - name: web image: nginx:2.4
```

Combining Image Name and Tag Changes

You can also combine both the `newName` and `newTag` properties if you wish to replace the

```
```yaml theme={null}
images:
 - name: nginx
 newName: haproxy
 newTag: 2.4
```

This configuration will ensure that your deployment now references the image `haproxy:2.4`.

By leveraging Kustomize's image transformer, you can manage image updates efficiently without directly altering your Kubernetes manifests. This process simplifies deployment updates and ensures consistency across your environments.



## Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

# Transformers Demo

This guide demonstrates using transformation techniques with Kustomize, including managing Kubernetes configurations and applying common labels, namespaces, and image transformations.

In this guide, we demonstrate how to use common transformation techniques with Kustomize, including the image transformer. We also review the directory structure used in this demonstration to help you manage and organize Kubernetes configurations efficiently.

## Directory Structure Overview

Assume you have a directory named "K8s" containing two subdirectories: "API" and "database".

- In the **API** directory, you will find:
  - API deployment YAML file (e.g., `api-depl.yaml`)
  - API service YAML file (e.g., `api-service.yaml`)
  - `kustomization.yaml`
- In the **database** directory, you will find:
  - Database deployment YAML file (e.g., `db-depl.yaml`)
  - Database service YAML file (e.g., `db-service.yaml`)
  - Config YAML file (e.g., `db-config.yaml`)
  - `kustomization.yaml`

Each subdirectory's `kustomization.yaml` is configured to import and manage all resources within that directory. For instance, the **API** directory's `kustomization.yaml` might look like this:

```
``yaml theme={null} apiVersion: kustomize.config.k8s.io/v1beta1 kind: Kustomization resources: - api-depl.yaml - api-service.yaml
```

```
And the database directory's kustomization.yaml imports the database configurations:

```yaml theme={null}
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization
resources:
  - db-config.yaml
  - db-depl.yaml
  - db-service.yaml
```

At the root level, the `kustomization.yaml` aggregates the API and database configurations:

```
```yaml theme={null} apiVersion: kustomize.config.k8s.io/v1beta1 kind:
Kustomization resources: - api/ - db/
```

#### ## Applying a Common Label

To add a common label to all resources, modify the root `kustomization.yaml` file. For example:

1. Open the root `kustomization.yaml`.
2. Add the common label configuration.
3. Save the file and run the Kustomize build command.

When you run:

```
```bash theme={null}
kustomize build k8s/
```

the output will include all the resources with the label `department: engineering`. For example, a ConfigMap might appear as follows:

```
```yaml theme={null} apiVersion: v1 data: password: example username: root
kind: ConfigMap metadata: labels: department: engineering name: db-
credentials
```

---

```
apiVersion: v1 kind: Service metadata: labels: department: engineering name:
api-service spec: ports: - port: 80 protocol: TCP targetPort: 3000
```

```

All resources in both the API and database directories now include the 'department: engineering' label.

Label Application in Subdirectories

When you add a common label within a subdirectory's 'kustomization.yaml' rather than in the root, the label is applied only to the resources listed in that file. This means that the API service will have both the root label (department: engineering) and the subdirectory-specific label (feature: api), while resources in the database directory will only have the global label.

```yaml theme={null}
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization
resources:
  - api-depl.yaml
  - api-service.yaml
commonLabels:
  feature: api

```

the `feature: api` label is applied only to the resources listed in that file. This means that the API service will have both the root label (`department: engineering`) and the subdirectory-specific label (`feature: api`), while resources in the database directory will only have the global label.

A snippet of the resulting output could look like:

```

```yaml theme={null} apiVersion: v1 data: password: example username: root
kind: ConfigMap metadata: labels: department: engineering name: db-credentials

```

---

```

apiVersion: v1 kind: Service metadata: labels: department: engineering feature: api name: api-service spec: ports: - port: 80 protocol: TCP targetPort: 3000

```

```

To differentiate between API and database resources further, you can add common labels specific to each directory.

```yaml theme={null}
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization
resources:
  - db-config.yaml
  - db-depl.yaml
  - db-service.yaml
commonLabels:
  feature: db

```

After building the configuration, database resources will include:

```

```yaml theme={null} metadata: labels: department: engineering feature: db

```

### ## Setting a Specific Namespace

To place all resources into a specific namespace, such as a debugging environment, add the

```
```yaml theme={null}
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization
resources:
  - api/
  - db/
commonLabels:
  department: engineering
namespace: debugging
```

After running the build, all resources from both API and database directories will be assigned the namespace `debugging`.

Adding Name Prefixes and Suffixes

If you want every object name to have a common prefix (e.g., "KodeKloud-") with folder-specific suffixes (like `-web` for API resources and `-storage` for database resources), follow these steps:

Adding a Global Name Prefix

Edit the root `kustomization.yaml` to include the prefix:

```
```yaml theme={null} apiVersion: kustomize.config.k8s.io/v1beta1 kind:
Kustomization resources: - api/ - db/ commonLabels: department: engineering
namespace: debugging namePrefix: KodeKloud-
```

After this change, the API deployment name might appear as ``KodeKloud-api-deployment`` (with

### ### Adding Folder-Specific Name Suffixes

For the API subdirectory, add a name suffix in its `kustomization.yaml`:

```
```yaml theme={null}
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization
resources:
  - api-depl.yaml
  - api-service.yaml
commonLabels:
  feature: api
nameSuffix: -web
```

For the database subdirectory, add a different name suffix:

```
```yaml theme={null} apiVersion: kustomize.config.k8s.io/v1beta1 kind:
Kustomization resources: - db-config.yaml - db-depl.yaml - db-service.yaml
commonLabels: feature: db nameSuffix: -storage
```

```
After rebuilding, you will see that API resource names end with -web while database resou

```yaml theme={null}
name: KodeKloud-api-deployment-web
```

and the database deployment might be:

```
```yaml theme={null} name: KodeKloud-db-deployment-storage
```

```
Adding a Common Annotation

To apply a common annotation across every resource, add the following to your root kustomi

```yaml theme={null}
commonAnnotations:
  logging: verbose
```

After rebuilding, every resource will include the annotation `logging: verbose`.

Image Transformation

Finally, let's look at how to update container images using the image transformer. Consider a scenario where a database deployment is set to use the Mongo image, but you want to replace it with Postgres using a specific tag.

In the database directory's `kustomization.yaml` (or in another appropriate configuration file), add an image transformer configuration like this:

```
```yaml theme={null} images: - name: mongo newName: postgres newTag:
"4.2"
```

```
This configuration instructs Kustomize to search for any container image named "mongo" and

<Callout icon="lightbulb" color="#1CB2FE">
 Ensure you enclose the new tag in quotes to avoid type conversion issues (e.g., newTag

 Error: accumulating resources: ... json: cannot unmarshal number into Go struct field ima
</Callout>

After applying this transformer and running:

```bash theme={null}
kustomize build k8s/
```

the database deployment's container image should update to:

```
yaml theme={null} image: postgres:4.2
```

while all other image configurations remain untouched.

Wrapping Up

This guide demonstrated how to:

- Organize Kubernetes configuration files into modular directories.
- Apply common labels, name prefixes, and suffixes globally or specifically in each subdirectory.
- Configure a namespace and common annotations.
- Use an image transformer to update container images seamlessly.

In the next section, try the lab exercises to gain hands-on experience with customizing Kustomize configurations and transformers.

Happy customizing!

Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

Patches Intro

This lesson explains how patches enable targeted modifications of Kubernetes configurations, allowing precise updates to specific objects rather than applying global changes.

In this lesson, we explain how patches offer a surgical approach for modifying Kubernetes configurations. Unlike common transformers that are efficient for applying global configuration changes—such as adding labels or setting namespaces—patches allow you to target one or a few specific objects. For example, if you need to update the replica count in a particular deployment, a customized patch permits you to precisely match and change the targeted object.

Understanding Patch Components

To create a patch, you need to provide three essential parameters:

1. **Operation Type:** Determines the action performed on a resource. The most commonly used operations are:
 2. **add:** Inserts a new element. For instance, when adding a container to a list in a deployment.
 3. **remove:** Deletes an element, such as removing a container or a label.
 4. **replace:** Swaps an existing value with a new one. For example, changing the replica count from 5 to 10.
5. **Target:** Specifies the criteria to identify the exact Kubernetes object(s) to patch. You can filter objects based on properties like:
 6. kind
 7. version
 8. name
 9. namespace
 10. label selector

11. annotation selector

These criteria can be combined to narrow down your selection.

1. **Value:** Represents the new value to be applied or used for replacement.

Note that for removal operations, no value is provided since the intention is to delete the target property.

When updating configurations, ensure that your patch accurately targets the intended resource to prevent unintended changes.

Example 1: Changing the Deployment Name

Consider the following `deployment.yaml` file with an initial configuration:

```
```yaml theme={null} apiVersion: apps/v1 kind: Deployment metadata: name:
api-deployment spec: replicas: 1 selector: matchLabels: component: api
template: metadata: labels: component: api spec: containers: - name: nginx
image: nginx
```

If you want to update `metadata.name` from `api-deployment` to `web-deployment`, you can use

```
```yaml theme={null}
patches:
  - target:
      kind: Deployment
      name: api-deployment
    patch: |-
      - op: replace
        path: /metadata/name
        value: web-deployment
```

After applying this patch, the updated deployment configuration becomes:

```
```yaml theme={null} apiVersion: apps/v1 kind: Deployment metadata: name:
web-deployment spec: replicas: 1 selector: matchLabels: component: api
template: metadata: labels: component: api spec: containers: - name: nginx
image: nginx
```



### ## Example 2: Updating the Replica Count

Let's review another scenario. Suppose the original deployment has one replica, and you want to update it to 5 replicas.

```
```yaml theme={null}
apiVersion: apps/v1
kind: Deployment
metadata:
  name: api-deployment
spec:
  replicas: 1
  selector:
    matchLabels:
      component: api
  template:
    metadata:
      labels:
        component: api
    spec:
      containers:
        - name: nginx
          image: nginx
```

Using an inline patch in your `kustomization.yaml`, update the replica count as follows:

```
```yaml theme={null} patches: - target: kind: Deployment name: api-deployment
patch: |- - op: replace path: /spec/replicas value: 5
```

After patching, the final `deployment.yaml` is updated to:

```
```yaml theme={null}
apiVersion: apps/v1
kind: Deployment
metadata:
  name: api-deployment
spec:
  replicas: 5
  selector:
    matchLabels:
      component: api
  template:
    metadata:
      labels:
        component: api
    spec:
      containers:
        - name: nginx
          image: nginx
```

Patch Methods in Kustomize

Kustomize supports two primary methods for defining patches:

1. JSON 6902 Patch

This method specifies the target and lists the patch operations. An example using JSON 6902 patch is:

```
```yaml theme={null}
```

# JSON 6902 Patch

---

```
kustomization: patches: - target: kind: Deployment name: api-deployment
patch: - op: replace path: /spec/replicas value: 5
```

### ### 2. Strategic Merge Patch

This approach resembles standard Kubernetes configuration files. You provide only the parts

```
```yaml theme={null}
# Strategic Merge Patch
kustomization:
  patches:
    - patch: |-
        apiVersion: apps/v1
        kind: Deployment
        metadata:
          name: api-deployment
        spec:
          replicas: 5
```

Both methods effectively update Kubernetes resources. The choice between JSON 6902 and strategic merge patches is based on personal preference, with many opting for the readability of strategic merge patches.

Conclusion

This lesson provided an overview of using patches with Kustomize to perform precise modifications on your Kubernetes configurations. By selecting the appropriate patch type, you can update labels, names, replica counts, and other fields with confidence and ease.

For additional details about patching and Kustomize, check out the [Kustomize documentation](#) and explore more in the [Kubernetes Documentation](#).

Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

Different Types of Patches

This guide explains how to define patches using JSON 6902 and strategic merge patch methods with inline and separate file approaches.

In this guide, we explain how to define patches using both JSON 6902 and strategic merge patch methods. There are two primary approaches to defining a patch:

1. Inline—where the patch is embedded directly within the kustomization.yaml file.
2. Separate File—where the patch is stored in an external YAML file, keeping the kustomization.yaml file clean and uncluttered.

Below are detailed examples for both methods.

Inline Patch Definition

In the inline approach, the patch is directly embedded in your kustomization.yaml file. This method is ideal for simple or singular modifications. For example:

```
``yaml theme={null} Kustomization: patches: - target: kind: Deployment name: api-deployment patch: | - op: replace path: /spec/replicas value: 5
```

```

<Callout icon="lightbulb" color="#1CB2FE">
  Use inline patches for quick adjustments when you have only a few changes to manage.
</Callout>

***

## Separate File Patch Definition

Alternatively, you can store your patch in an external YAML file and reference it in your kustomization file.

In your kustomization.yaml file, reference the external patch file:

```yaml
theme={null}
Kustomization:
 patches:
 - path: replica-patch.yaml
 target:
 kind: Deployment
 name: nginx-deployment

```

And here is how the external file named replica-patch.yaml might look:

```
```yaml
theme={null} - op: replace path: /spec/replicas value: 5

```

```

<Callout icon="lightbulb" color="#1CB2FE">
  Using a separate file for patches enhances readability and maintainability when working with patches.
</Callout>

***

## Strategic Merge Patches

You can apply the same methodologies when working with strategic merge patches. Below are examples.

### Inline Approach for Strategic Merge Patch

Embed the strategic merge patch directly into your configuration:

```yaml
theme={null}
patches:
 - target:
 kind: Deployment
 name: api-deployment
 patch: |
 - op: replace
 path: /spec/replicas
 value: 5

```

## Separate File Approach for Strategic Merge Patch

Reference an external patch file in your kustomization.yaml:

```
```yaml
theme={null}
patches:
  - path: replica-patch.yaml
    target:
      kind: Deployment
      name: nginx-deployment

```

And in the external replica-patch.yaml:

```
``yaml theme={null}
- op: replace
  path: /spec/replicas
  value: 5
```

For a strategic merge patch defined as a complete resource patch in a single YAML block, consider this inline approach:

```
``yaml theme={null} patches: | - patch: | apiVersion: apps/v1 kind: Deployment
metadata: name: api-deployment spec: replicas: 5
```

Or reference the complete resource patch from an external file:

```
``yaml theme={null}
patches:
- replica-patch.yaml
```

Always ensure that your patch operations accurately reflect the intended changes to avoid potential misconfigurations.

Conclusion

Choose the inline method for simplicity when only a few patches are needed, or opt for the separate file approach to maintain a cleaner and more manageable kustomization.yaml—especially when dealing with multiple or complex patches.

For further details on patching strategies and best practices, refer to the [Kustomize documentation](#).

Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

Patches Dictionary

Learn to update, add, and remove keys in Kubernetes Deployment configurations using JSON 6902 patches and strategic merge patches with practical examples.

In this article, you'll learn how to update, add, and remove keys in a Kubernetes Deployment configuration using both JSON 6902 patches and strategic merge patches. Each example starts with a Deployment that contains a label with the key "component" set to "api". The goal is to modify or update these labels as needed in each scenario.

Updating a Key in a Dictionary

Using a JSON 6902 Patch

Consider the following Deployment configuration:

```
```yaml theme={null} apiVersion: apps/v1 kind: Deployment metadata: name: api-deployment spec: replicas: 1 selector: matchLabels: component: api template: metadata: labels: component: api spec: containers: - name: nginx image: nginx
```

To update the label from `"component: api"` to `"component: web"` using a JSON 6902 patch,

```
```yaml theme={null} kustomization: patches: - target: kind: Deployment name: api-deployment patch: |- - op: replace path: /spec/template/metadata/labels/component value: web
```

The patch navigates to the "component" key within the labels dictionary using the path `/spec/template/metadata/labels/component` and replaces its value with "web".

The JSON 6902 patch method provides precise control when updating complex configurations. Choose the patch type that best fits your needs.

Using a Strategic Merge Patch

Alternatively, you can update the label using a strategic merge patch stored in a separate file, for example, `label-patch.yaml`. Your main Deployment configuration remains unchanged, and your `kustomization.yaml` is updated as follows:

```
``yaml theme={null} kustomization: patches: - label-patch.yaml
```

Here is an example of the contents for `label-patch.yaml`:

```
``yaml theme={null}
apiVersion: apps/v1
kind: Deployment
metadata:
  name: api-deployment
spec:
  template:
    metadata:
      labels:
        component: web
```

Kustomize will merge this patch with the original Deployment configuration, resulting in an updated "component" label.

Adding a New Key to a Dictionary

Using a JSON 6902 Patch

Suppose you want to add a new label `"org"` with the value `"KodeKloud"` while keeping the original `"component: api"` label. Start with the following Deployment:

```
``yaml theme={null} apiVersion: apps/v1 kind: Deployment metadata: name:
api-deployment spec: replicas: 1 selector: matchLabels: component: api
template: metadata: labels: component: api spec: containers: - name: nginx
image: nginx
```

Then, add the following JSON 6902 patch in your `kustomization.yaml`

```
```yaml theme={null}
kustomization:
 patches:
 - target:
 kind: Deployment
 name: api-deployment
 patch: |-
 - op: add
 path: /spec/template/metadata/labels/org
 value: KodeKloud
```

This patch uses the `add` operation to insert the new key `"org"` with the specified value into the labels dictionary.

## Using a Strategic Merge Patch

For a strategic merge patch, create a separate file (e.g., `label-patch.yaml`) with the following content:

```
```yaml theme={null} apiVersion: apps/v1 kind: Deployment metadata: name:
api-deployment spec: template: metadata: labels: org: KodeKloud
```

Then, reference the patch file in your `kustomization.yaml`:

```
```yaml theme={null}
kustomization:
 patches:
 - label-patch.yaml
```

Kustomize automatically merges the patch with the existing configuration while preserving both labels: `"component: api"` and `"org: KodeKloud"`.

When adding new keys, always verify that the target dictionary exists to avoid runtime errors.

## Removing a Key from a Dictionary

### Using a JSON 6902 Patch

Assume you have a Deployment configuration that includes two labels:

```
```yaml theme={null} apiVersion: apps/v1 kind: Deployment metadata: name:
api-deployment spec: replicas: 1 selector: matchLabels: component: api
template: metadata: labels: org: KodeKloud component: api spec: containers: -
name: nginx image: nginx
```



```
To remove the "org" label using a JSON 6902 patch, modify your `kustomization.yaml` as follows:

```yaml
theme={null}
kustomization:
 patches:
 - target:
 kind: Deployment
 name: api-deployment
 patch: |-
 - op: remove
 path: /spec/template/metadata/labels/org
```

This patch navigates to the "org" key and removes it, leaving only the "component" label in place.

### Using a Strategic Merge Patch

To remove the "org" label via a strategic merge patch, create a patch file (e.g., `label-patch.yaml`) with the following content:

```
```yaml
theme={null}
apiVersion: apps/v1
kind: Deployment
metadata:
  name: api-deployment
spec:
  template:
    metadata:
      labels:
        org: null
```

Reference this patch file in your `kustomization.yaml`:

```
```yaml
theme={null}
kustomization:
 patches:
 - label-patch.yaml
```

Kustomize interprets the `null` value as an instruction to remove the "org" label from the original configuration.

Ensure that you specify the correct path for removal operations to avoid inadvertently deleting other keys in the configuration.

---

With these examples, you now understand how to update, add, and remove keys in a Kubernetes Deployment configuration using both JSON 6902 patches and strategic merge patches. For more detailed information on Kubernetes configurations and patch strategies, visit the [Kubernetes Documentation](#).

## Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

## Patches list

This lesson demonstrates modifying list items in Kubernetes Deployment configurations using JSON 6902 patches and strategic merge patches.

In this lesson, we demonstrate how to modify list items in a Kubernetes Deployment configuration using both JSON 6902 patches and strategic merge patches. You can update, add, or remove containers in the deployment's container list. The examples below illustrate how to work with list indices and container names effectively.

Below is the base deployment configuration defining a single container named "nginx" using the "nginx" image. Notice that the containers section is a list, indicated by the dash (-) before the container definition.

```
```yaml theme={null} apiVersion: apps/v1 kind: Deployment metadata: name: api-deployment spec: replicas: 1 selector: matchLabels: component: api template: metadata: labels: component: api spec: containers: - name: nginx image: nginx
```

Replacing a Container in the List Using a JSON 6902 Patch

To update both the name and image of the container, use a JSON 6902 patch in your `kustomization`

****kustomization.yaml:****

```
```yaml theme={null}
patches:
- target:
 kind: Deployment
 name: api-deployment
 patch: |-
 - op: replace
 path: /spec/template/spec/containers/0
 value:
 name: haproxy
 image: haproxy
```

The path `/spec/template/spec/containers/0` specifies that the first element of the containers list (index 0) will be replaced. Once applied, Kustomize updates the container, switching from "nginx" to "haproxy" for both the name and image.

Remember that list indices in YAML start at zero. Always check the index to target the correct container.

## Replacing a Container Using a Strategic Merge Patch

If you prefer modifying just a part of the container configuration (e.g., only the image), you can use a strategic merge patch. First, reference your patch file in the `kustomization.yaml`:

**kustomization.yaml:**

```
``yaml theme={null} patches: - label-patch.yaml
```

Then, in the `label-patch.yaml` file, specify the container by its name and provide the update:

```
label-patch.yaml:

``yaml theme={null}
apiVersion: apps/v1
kind: Deployment
metadata:
 name: api-deployment
spec:
 template:
 spec:
 containers:
 - name: nginx
 image: haproxy
```

This patch finds the container named "nginx" in the list and updates its image to "haproxy."

## Adding a Container to the List Using a JSON 6902 Patch

If you want to add a second container to your deployment, you can achieve this using a JSON 6902 patch. In the example below, a new container with the name and image "haproxy" is appended to the containers list. The dash (-) at the end of the path indicates that the container should be added at the end of the list.

**kustomization.yaml:**

```
```yaml theme={null} patches: - target: kind: Deployment name: api-deployment
patch: |- - op: add path: /spec/template/spec/containers/- value: name: haproxy
image: haproxy
```

If the order of containers is important, you can specify an exact index; however, appending

After applying this patch, the deployment configuration will include both the original "v"

```
```yaml theme={null}
apiVersion: apps/v1
kind: Deployment
metadata:
 name: api-deployment
spec:
 replicas: 1
 selector:
 matchLabels:
 component: api
 template:
 metadata:
 labels:
 component: api
 spec:
 containers:
 - name: nginx
 image: nginx
 - name: haproxy
 image: haproxy
```

## Adding a Container Using a Strategic Merge Patch

Alternatively, you can add a container using a strategic merge patch. Suppose the original configuration contains a container named "web" using the "nginx" image and you wish to add another container. First, reference the patch file in

```
kustomization.yaml:
```

**kustomization.yaml:**

```
```yaml theme={null} patches: - label-patch.yaml
```

Then, list the additional container in the `label-patch.yaml` file:

```
**label-patch.yaml:**

```yaml
theme: {null}
apiVersion: apps/v1
kind: Deployment
metadata:
 name: api-deployment
spec:
 template:
 spec:
 containers:
 - name: haproxy
 image: haproxy
```

When merged, the resulting configuration will include both the original "web" container and the new "haproxy" container.

---

## Deleting a Container from the List Using a JSON 6902 Patch

Imagine a scenario where your Deployment originally includes two containers: one named "web" with the "nginx" image and another named "database" with the "mongo" image.

### Original Deployment:

```
```yaml
theme: {null}
apiVersion: apps/v1
kind: Deployment
metadata:
  name: api-deployment
spec:
  replicas: 1
  selector:
    matchLabels:
      component: api
  template:
    metadata:
      labels:
        component: api
    spec:
      containers:
        - name: web
          image: nginx
        - name: database
          image: mongo
```

To remove the "database" container (which is at index 1), use the following JSON 6902 patch

```
**kustomization.yaml:**

```yaml
theme: {null}
patches:
 - target:
 kind: Deployment
 name: api-deployment
 patch: |-
 - op: remove
 path: /spec/template/spec/containers/1
```

After applying this patch, the deployment will only contain the "web" container.

Ensure that you use the correct index when attempting to remove a container. An incorrect index could lead to unexpected modifications in your deployment configuration.

---

## Deleting a Container Using a Strategic Merge Patch

---

You can also delete a container using a strategic merge patch with the delete directive. Given the initial configuration that includes both "web" and "database" containers, create a patch that instructs Kustomize to remove the "database" container:

**kustomization.yaml:**

```
``yaml theme={null} patches: - label-patch.yaml
```

```
``label-patch.yaml:``

``yaml theme={null}
apiVersion: apps/v1
kind: Deployment
metadata:
 name: api-deployment
spec:
 template:
 spec:
 containers:
 - $patch: delete
 name: database
```

In this patch file, the "\$patch: delete" directive targets the container named "database" and removes it from the list. Consequently, the final configuration will only include the "web" container.

---

This lesson detailed multiple methods for manipulating list items in Kubernetes Deployments using both JSON 6902 patches and strategic merge patches. For more information on Kubernetes patch strategies, refer to the [Kubernetes Documentation](#).

## Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

# Overlays

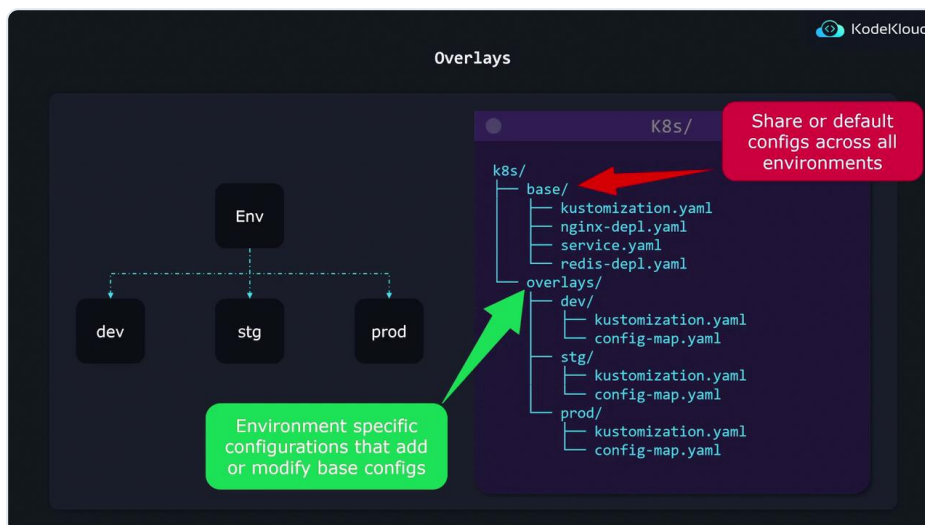
Overlays in Kustomize allow customization of base Kubernetes configurations for different environments like development, staging, and production.

Overlays in Kustomize allow you to customize a base Kubernetes configuration on a per-environment basis. This method is particularly useful for environments such as development, staging, and production, where you need to apply environment-specific adjustments to shared configurations.

Kustomize projects are typically organized into two main sections:

1. **Base Configuration:** Contains all shared and default Kubernetes resource definitions.
2. **Overlay Directories:** Each environment (e.g., dev, stg, prod) has its own overlay folder with patches to modify the base configuration as needed.

Below is a diagram illustrating a common directory structure for managing these configurations:



In this setup, the base folder holds the shared resource files, while each overlay folder contains a `kustomization.yaml` that references the shared

resources in the base along with overlays (patches or additional resources) specific to that environment.

## Base Configuration Example

---

Imagine that you have an `nginx-deployment.yaml` file within your base folder with a replica count set to 1. The corresponding `kustomization.yaml` in the base folder might look like this:

```
``yaml theme={null}
```

## base/kustomization.yaml

---

resources: - nginx-depl.yaml - service.yaml - redis-depl.yaml

And the `nginx-depl.yaml` file is defined as follows:

```
``yaml theme={null}
base/nginx-depl.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
 name: nginx-deployment
spec:
 replicas: 1
```

## Creating Overlays

---

### Development Overlay

To create an overlay for the development environment, you would set up a `kustomization.yaml` file in the `dev` overlay folder. This file references the base configuration and includes a patch to update the replica count:

```
``yaml theme={null}
```

## overlays/dev/kustomization.yaml

---

bases: - ../../base patch: |- - op: replace path: /spec/replicas value: 2



```

In this overlay, the `bases` property points to the shared base resources using the relative path.

Production Overlay

Similarly, to tailor the configuration for production, the overlay can reference the same base resources.

```yaml theme={null}
# overlays/prod/kustomization.yaml
bases:
- ../../base
patch: |-
- op: replace
  path: /spec/replicas
  value: 3

```

This overlay increases the replica count to 3 for production.

Adding New Resources in Overlays

Overlays can also introduce new resources that don't exist in the base configuration. For instance, if you want to add a production-specific Grafana deployment, you can include its YAML file in the production overlay:

```
```yaml theme={null}
```

## overlays/prod/kustomization.yaml

```

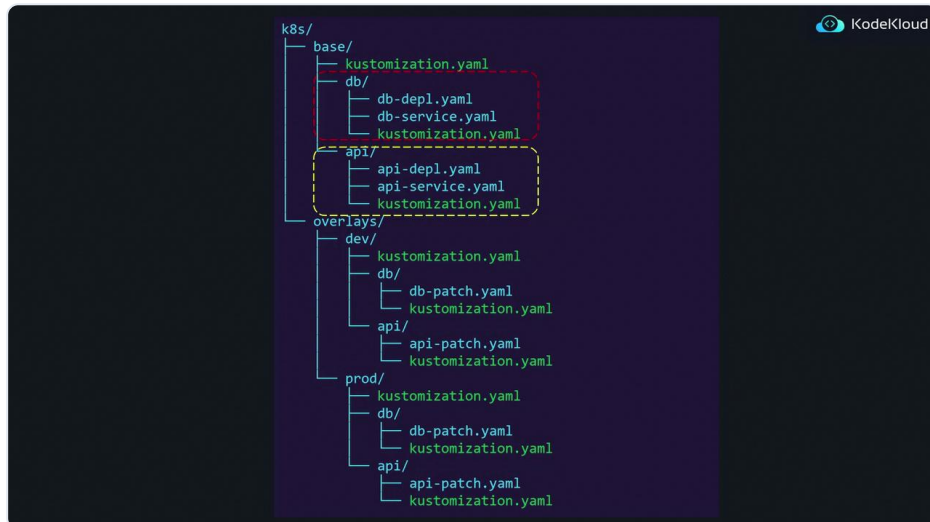
bases: - ../../base resources: - grafana-depl.yaml patch: |-
- op: replace path: /spec/replicas value: 2 ```

```

In this configuration, the overlay imports the base resources, applies a patch to change the replica count for the existing deployment, and adds a new Grafana deployment.

Kustomize is flexible in the way you structure your configurations. While the base can be organized into subdirectories based on features, the overlay directories do not need to mirror that structure. The critical factor is correctly referencing the shared resources in the appropriate `kustomization.yaml` file.

Below is another diagram that provides a more detailed look at the directory structure using Kustomize, showing both the base and overlay directories across different environments:



## Summary

Overlays in Kustomize enable you to:

- Import and reuse a base configuration containing shared resources.
- Apply environment-specific patches to adjust base resources such as replica counts.
- Introduce new resources within an overlay without affecting the base configuration.

This approach helps maintain a clean separation between shared configurations and environment-specific customizations while taking full advantage of Kustomize's powerful features for managing Kubernetes deployments.

For further information on Kubernetes configuration management, consider exploring additional resources in the [Kubernetes Documentation](#) and [Kustomize GitHub repository](#).

## Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

# Components

This article explains Kustomize components for defining reusable configuration logic to minimize duplication and manage optional features across multiple overlays.

In this lesson, you will learn about a powerful Kustomize feature called components. Components allow you to define reusable configuration logic that can be seamlessly integrated into multiple overlays. This approach is especially beneficial for applications that support optional features activated only in selected overlays.

For instance, if you have common configurations that apply to every overlay, you would include them in your Base configuration. However, if specific features need to be enabled only in particular overlays, duplicating the configurations in each overlay quickly becomes unscalable and error-prone. Components address this challenge by letting you define the configuration once and reuse it where needed, minimizing duplication and preventing configuration drift.

## Visual Example

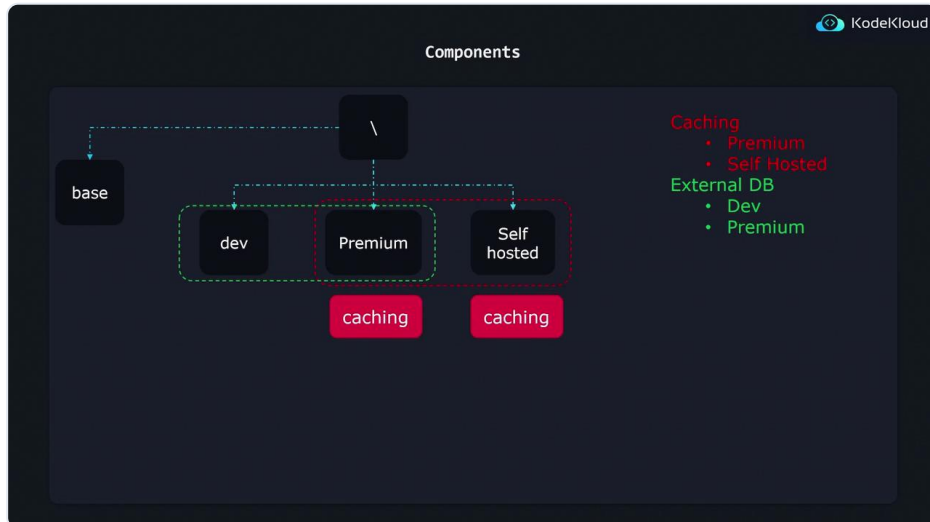
Imagine deploying an application in three variations: Development, Premium (for premium customers), and Self-Hosted (for customers who manage their own hosting). These variations correspond to three overlay folders, which all reference a shared Base configuration folder.

Suppose the application offers two optional features: caching and an external Postgres database. The caching feature, which includes configuration details for a Redis database, should only be applied to the Premium and Self-Hosted overlays. In contrast, the external database is only relevant for the Development and Premium overlays.

Placing the caching configuration in the Base folder would inadvertently activate it across all overlays, while copying it individually into the Premium and Self-Hosted overlays risks inconsistencies during future updates.

Components solve this problem by encapsulating the caching configuration in a reusable block that is imported only by the overlays that require it.

The overall project hierarchy is structured so that the Base configuration is common to all overlays, while each overlay selectively includes components for features like caching or an external database.



## Organizing Your Project Structure

To effectively implement components, consider adding a dedicated folder for components in your project structure. A typical folder layout may resemble the following structure:

```

k8s/
├── base/
│ ├── kustomization.yaml
│ ├── api-depl.yaml
│ ├── components/
│ │ ├── caching/
│ │ │ ├── kustomization.yaml
│ │ │ ├── deployment-patch.yaml
│ │ │ └── redis-depl.yaml
│ │ └── db/
│ │ ├── kustomization.yaml
│ │ ├── deployment-patch.yaml
│ │ └── postgres-depl.yaml
│ └── overlays/
│ ├── dev/
│ │ ├── kustomization.yaml
│ │ └── premium/
│ │ ├── kustomization.yaml
│ │ └── standalone/
│ │ └── kustomization.yaml
├── kustomization.yaml
├── api-depl.yaml
├── components/
├── caching/
├── kustomization.yaml
├── deployment-patch.yaml
├── redis-depl.yaml
├── db/
├── kustomization.yaml
├── deployment-patch.yaml
├── postgres-depl.yaml
├── overlays/
├── dev/
├── kustomization.yaml
├── premium/
├── kustomization.yaml
├── standalone/
└── kustomization.yaml

```

In the structure above, the `components` folder houses dedicated subdirectories for each op

<Callout icon="lightbulb" color="#1CB2FE">

A well-organized project differentiates between the immutable Base configuration and the  
</Callout>

<Frame>

! [The image is a flowchart titled "Components," showing a hierarchical structure with r  
</Frame>

## ## Implementing a Component

Let's focus on the database component. Within the `db` folder, the following three files ar

```
* `kustomization.yaml`
* `deployment-patch.yaml`
* `postgres-depl.yaml`
```

The `postgres-depl.yaml` file defines the deployment for a Postgres database, which is the

```
``yaml theme={null}
apiVersion: apps/v1
kind: Deployment
metadata:
 name: postgres-deployment
spec:
 replicas: 1
 selector:
 matchLabels:
 component: postgres
 template:
 metadata:
 labels:
 component: postgres
 spec:
 containers:
 - name: postgres
 image: postgres
```

## The Component's kustomization.yaml

The `kustomization.yaml` file inside the database component is slightly different from standard configuration files. It indicates that this is a Component:

```
``yaml theme={null} apiVersion: kustomize.config.k8s.io/v1alpha1 kind:
Component resources: - postgres-depl.yaml secretGenerator: - name:
postgres-cred literals: - password=postgres123 patches: - deployment-
patch.yaml
```

This file accomplishes the following:

- \* Imports the Postgres deployment defined in `postgres-depl.yaml`.
- \* Generates a secret named `postgres-cred` containing the database password.
- \* Applies patches from `deployment-patch.yaml` to update the base configuration (for example, to change the image).

### The Deployment Patch

The `deployment-patch.yaml` file constitutes a strategic merge patch that updates the base

```
```yaml
theme: {null}
apiVersion: apps/v1
kind: Deployment
metadata:
  name: api-deployment
spec:
  template:
    spec:
      containers:
        - name: api
          env:
            - name: DB_PASSWORD
              valueFrom:
                secretKeyRef:
                  name: postgres-cred
                  key: password
```

Importing Components in Overlays

To activate a component in an overlay, include its reference in the overlay's `kustomization.yaml` file. For example, the `kustomization.yaml` in the `dev` overlay might be configured as follows:

```
yaml
theme: {null}
bases:
  - ../../base
components:
  - ../../components/db
```

In this configuration:

- The base configuration is imported first.
- The database component is then imported, enabling the external database feature for that overlay.

This same import pattern can be applied to any overlay needing external database functionality or caching, ensuring that optional features are defined once and consistently reused across all environments.

By leveraging components in Kustomize, you can maintain consistency and reduce redundancy by defining optional features only once. This modularity not only promotes reusability but also simplifies managing configurations across multiple overlays.

Section: Mock Exams

Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

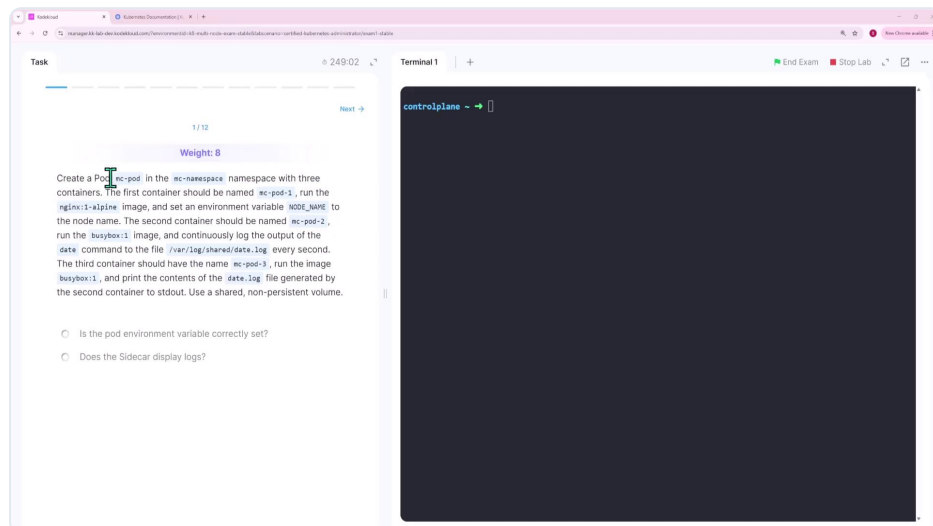
Mock Exam 1 Step by Step Solutions

This article provides step-by-step solutions for tasks in Mock Exam 1 related to Kubernetes configurations and operations.

In this article, we walk through the solutions for Mock Exam 1. Each section corresponds to a specific exam question and includes a technical diagram with its original description. Follow the steps carefully to complete each task.

Question 1 – Create a Pod with Three Containers

You must create a pod named **mc-pod** in the **MC** namespace that includes three containers. You can either consult the documentation for an example pod configuration or use an imperative command with `kubect1 run` and then modify the generated YAML file.



Below is an example of using `kubectl run` with the `--overrides` flag to add container definitions. The `--dry-run=client` flag outputs the YAML for further editing:

```
``bash theme={null} kubectl run mc-pod-1 --image=nginx:1-alpine --env=NODE_NAME=$(POD_NAME) --overrides=' { "apiVersion": "v1", "spec": { "containers": [ { "name": "mc-pod-2", "image": "busybox:1", "command": [ "sh", "-c", "while true; do date >> /var/log/shared/date.log; sleep 1; done" ] }, { "name": "mc-pod-3", "image": "busybox:1", "command": [ "sh", "-c", "tail -f /var/log/shared/date.log" ] } ] } }'
```

Alternatively, you can generate a base YAML file:

```
``bash theme={null}
kubectl run mc-pod --image=nginx:1-alpine --dry-run=client -o yaml > question1.yaml
```

After generating the file, update it as follows:

1. **Rename the pod:** Change its name to **mc-pod** (and remove unnecessary labels and creation timestamps).
2. **Update the first container:** Rename it to **mc-pod-1** and inject an environment variable (`NODE_NAME`) whose value is dynamically set from the `spec.nodeName` field. The YAML snippet becomes:

```
yaml theme={null} apiVersion: v1 kind: Pod metadata: name: mc-pod
spec: containers: - image: nginx:1-alpine name: mc-pod-1 env: -
name: NODE_NAME valueFrom: fieldRef: fieldPath: spec.nodeName
resources: {}
```

1. **Define the second container:** Use the `busybox:1` image with the following command:


```
yaml theme={null} - name: mc-pod-2 image: busybox:1 command: - "sh"
- "-c" - "while true; do date >> /var/log/shared/date.log; sleep 1;
done"
```

1. **Define the third container:** Use the `busybox:1` image to tail the shared log file:

```
yaml theme={null} - name: mc-pod-3 image: busybox:1 command: - "sh"
- "-c" - "tail -f /var/log/shared/date.log"
```

1. **Add a shared volume:** Since containers 2 and 3 need to share the same filesystem, add a non-persistent volume using `emptyDir` and mount it on `/var/log/shared` for both containers. The final YAML is:

```
yaml theme={null} apiVersion: v1 kind: Pod metadata: name: mc-pod
spec: volumes: - name: shared-volume emptyDir: {} containers: -
image: nginx:1-alpine name: mc-pod-1 env: - name: NODE_NAME
valueFrom: fieldRef: fieldPath: spec.nodeName resources: {} - name:
mc-pod-2 image: busybox:1 command: - "sh" - "-c" - "while true; do
date >> /var/log/shared/date.log; sleep 1; done" volumeMounts: -
name: shared-volume mountPath: /var/log/shared - name: mc-pod-3
image: busybox:1 command: - "sh" - "-c" - "tail -f /var/log/shared/
date.log" volumeMounts: - name: shared-volume mountPath: /var/log/
shared dnsPolicy: ClusterFirst restartPolicy: Always
```

After saving and applying this configuration, verify the pod's status and check the logs of container **mc-pod-3**. Its output should display the continuously appended date entries from container **mc-pod-2**.

The screenshot shows a lab interface with a task description on the left and a terminal window on the right.

Task Description:

Create a Pod `mc-pod` in the `mc-namespace` namespace with three containers. The first container should be named `mc-pod-1`, run the `nginx:1-alpine` image, and set an environment variable `NODE_NAME` to the node name. The second container should be named `mc-pod-2`, run the `busybox:1` image, and continuously log the output of the `date` command to the file `/var/log/shared/date.log` every second. The third container should have the name `mc-pod-3`, run the image `busybox:1`, and print the contents of the `date.log` file generated by the second container to stdout. Use a shared, non-persistent volume.

Terminal Output:

```
Thu Apr 24 03:27:13 UTC 2025
Thu Apr 24 03:27:14 UTC 2025
Thu Apr 24 03:27:15 UTC 2025
Thu Apr 24 03:27:16 UTC 2025
Thu Apr 24 03:27:17 UTC 2025
Thu Apr 24 03:27:18 UTC 2025
Thu Apr 24 03:27:19 UTC 2025
Thu Apr 24 03:27:20 UTC 2025
Thu Apr 24 03:27:21 UTC 2025
Thu Apr 24 03:27:22 UTC 2025
Thu Apr 24 03:27:23 UTC 2025
Thu Apr 24 03:27:24 UTC 2025
Thu Apr 24 03:27:25 UTC 2025
Thu Apr 24 03:27:26 UTC 2025
Thu Apr 24 03:27:27 UTC 2025
Thu Apr 24 03:27:28 UTC 2025
Thu Apr 24 03:27:29 UTC 2025
Thu Apr 24 03:27:30 UTC 2025
Thu Apr 24 03:27:31 UTC 2025
Thu Apr 24 03:27:32 UTC 2025
Thu Apr 24 03:27:33 UTC 2025
Thu Apr 24 03:27:34 UTC 2025
Thu Apr 24 03:27:35 UTC 2025
Thu Apr 24 03:27:36 UTC 2025
Thu Apr 24 03:27:37 UTC 2025
Thu Apr 24 03:27:38 UTC 2025
Thu Apr 24 03:27:39 UTC 2025
Thu Apr 24 03:27:40 UTC 2025
Thu Apr 24 03:27:41 UTC 2025
Thu Apr 24 03:27:42 UTC 2025
Thu Apr 24 03:27:43 UTC 2025
Thu Apr 24 03:27:44 UTC 2025
Thu Apr 24 03:27:45 UTC 2025
Thu Apr 24 03:27:46 UTC 2025
Thu Apr 24 03:27:47 UTC 2025
Thu Apr 24 03:27:48 UTC 2025
Thu Apr 24 03:27:49 UTC 2025
```

Question 2 – Prepare Node One for Kubernetes

For this task, you will perform several actions on node one.

1. **SSH into node one** using the provided credentials:

```
bash theme={null} ssh node01 # Accept the host key and provide the
password when prompted.
```

1. **Switch to the root user** (if necessary):

```
bash theme={null} sudo su
```

1. **Navigate to the `/root` directory** and locate the CRI Docker package (e.g., `cri-docker_0.3.16.3-0.debian.deb`).

2. **Install the CRI Docker package:**

```
bash theme={null} dpkg -i ./cri-docker_0.3.16.3-0.debian.deb
```

1. **Start the CRI Docker service:**

```
bash theme={null} systemctl start cri-docker
```

1. **Verify the service status:**

```
bash theme={null} systemctl status cri-docker
```

1. **Enable the service to start on boot:**

```
bash theme={null} systemctl enable cri-docker
```

Ensure that the service is active (running) and enabled.

If you encounter any issues with the service, recheck the installation and confirm your package file's integrity.

Question 3 – Save Vertical Pod Autoscaler CRDs

On the control plane node, list all Custom Resource Definitions (CRDs) related to the Vertical Pod Autoscaler and save their names to `/root/vpa-crds.txt`.

1. **Retrieve all CRDs and filter for “vertical”:**

```
bash theme={null} kubectl get crd | grep -i vertical
```

Expected output:

```
text theme={null}
verticalpodautoscalercheckpoints.autoscaling.k8s.io
2025-04-24T03:13:05Z verticalpodautoscalers.autoscaling.k8s.io
2025-04-24T03:13:05Z
```

1. **Save the CRD names** to a file named `/root/vpa-crds.txt`. The file should include:

```
text theme={null}
verticalpodautoscalercheckpoints.autoscaling.k8s.io
verticalpodautoscalers.autoscaling.k8s.io
```

No further modifications are required.

Question 4 – Create a Service for the Messaging Application

Expose the messaging application by creating a service named **messaging-service** on port **6379** within the cluster.

1. **Verify that the messaging pod is running:**

```
bash theme={null} kubectl get pod
```

1. **Expose the messaging pod:**

```
bash theme={null} kubectl expose pod messaging --port=6379 --name=messaging-service
```

1. **Confirm the service creation** by describing it:

```
bash theme={null} kubectl describe service messaging-service
```

The output should list the service as a ClusterIP service on port **6379** with endpoints matching the messaging pod's IP.

Question 5 – Create a Deployment for the HR Web Application

Create a deployment named **hr-web-app** using the image `kodekloud/webapp-color` with two replicas.

1. **Run the following command:**

```
bash theme={null} kubectl create deployment hr-web-app --image=kodekloud/webapp-color --replicas=2
```

1. **Verify the deployment status:**

```
bash theme={null} kubectl get deployment hr-web-app
```

Ensure that both replicas become available.

Question 6 – Fix the Faulty Init Container in the Orange Pod

The pod named **orange** is failing because its init container is crashing with exit code 127 due to a typo in the command (`sleep` instead of `sleep`).

1. **Inspect the pod logs** to identify the error:

```
bash theme={null} kubectl logs orange -c init-myservice
```

Expected error output:

```
text theme={null} sh: sleep: not found
```

1. **Retrieve the pod configuration** and save it to a file:

```
bash theme={null} kubectl get pod orange -o yaml > question6.yaml
```

1. **Edit the file** `question6.yaml` to correct the command in the init container.

Modify the snippet to:

```
yaml theme={null} initContainers: - name: init-mysvc image: busybox
command: - "sh" - "-c" - "sleep 2"
```

1. **Force update the pod** with the corrected configuration:

```
bash theme={null} kubectl replace -f question6.yaml --force
```

1. **Verify the pod status** to ensure it transitions to a running state and the init container completes successfully.

Question 7 – Expose HR Web App via a NodePort Service

Expose the **hr-web-app** deployment as a service named **hr-web-app-service** to make it accessible on port **30082** from the nodes. Note that the web application listens internally on port **8080**.

1. **Generate a NodePort service configuration with dry-run:**

```
bash theme={null} kubectl expose deployment hr-web-app --
type=NodePort --port=8080 --name=hr-web-app-service --dry-run=client
-o yaml > question7.yaml
```

1. **Edit** `question7.yaml` to include the `nodePort: 30082` under the ports section as shown below:

```
yaml theme={null} apiVersion: v1 kind: Service metadata: name: hr-
web-app-service labels: app: hr-web-app spec: type: NodePort
selector: app: hr-web-app ports: - port: 8080 targetPort: 8080
nodePort: 30082
```

1. **Apply the updated service configuration:**

```
bash theme={null} kubectl apply -f question7.yaml
```

1. **Verify the service details:**

```
bash theme={null} kubectl describe service hr-web-app-service
```

Question 8 – Create a Persistent Volume

Create a persistent volume named **pv-analytics** with the following specifications:

- **Capacity:** 100Mi
- **Access Mode:** ReadWriteMany
- **Type:** hostPath using the directory `/pv/data-analytics`
- **Create a YAML file (e.g., `question8.yaml`)** with the following content:

```
yaml theme={null} apiVersion: v1 kind: PersistentVolume metadata:
name: pv-analytics spec: capacity: storage: 100Mi volumeMode:
Filesystem accessModes: - ReadWriteMany hostPath: path: /pv/data-
analytics
```

1. **Apply the configuration:**

```
bash theme={null} kubectl apply -f question8.yaml
```

1. **Verify that the persistent volume is available:**

```
bash theme={null} kubectl get pv
```

Question 9 – Create a Horizontal Pod Autoscaler (HPA)

Create an HPA for the deployment **kkapp-deploy** in the default namespace with the following configuration:

- **CPU Utilization:** Maintain an average of 50%
- **Replicas:** Scale between 2 and 10 pods
- **Stabilization Window:** 300 seconds when scaling down
- **Prepare a YAML file (e.g., `webapp-hpa.yaml`)** with this content:

```
yaml theme={null} apiVersion: autoscaling/v2 kind:
HorizontalPodAutoscaler metadata: name: webapp-hpa namespace:
default spec: scaleTargetRef: apiVersion: apps/v1 kind: Deployment
name: kkapp-deploy minReplicas: 2 maxReplicas: 10 metrics: - type:
Resource resource: name: cpu target: type: Utilization
averageUtilization: 50 behavior: scaleDown:
stabilizationWindowSeconds: 300
```

1. **Apply the HPA configuration:**

```
bash theme={null} kubectl apply -f webapp-hpa.yaml
```

1. **Check the HPA status:**

```
bash theme={null} kubectl get hpa
```

Question 10 – Create a Vertical Pod Autoscaler (VPA)

Create a Vertical Pod Autoscaler that automatically adjusts CPU and memory requests for the deployment **analytics-deployment** in the default namespace. The VPA should run in auto mode.

1. Create a YAML file (e.g., `question10.yaml`) with the following content:

```
yaml theme={null} apiVersion: autoscaling.k8s.io/v1 kind:
VerticalPodAutoscaler metadata: name: analytics-vpa namespace:
default spec: targetRef: apiVersion: apps/v1 kind: Deployment name:
analytics-deployment updatePolicy: updateMode: "Auto"
```

1. Apply the VPA configuration:

```
bash theme={null} kubectl apply -f question10.yaml
```

1. Verify that the VPA has been created:

```
bash theme={null} kubectl get vpa
```

Question 11 – Create a Kubernetes Gateway Resource

Create a Gateway resource for the web application.

1. Create a YAML file (e.g., `question11.yaml`) with the following content:

```
yaml theme={null} apiVersion: gateway.networking.k8s.io/v1 kind:
Gateway metadata: name: web-gateway namespace: nginx-gateway spec:
gatewayClassName: nginx listeners: - name: http protocol: HTTP port:
80
```

1. Apply the gateway configuration:

```
bash theme={null} kubectl apply -f question11.yaml
```

1. Verify that the Gateway resource is available in the `nginx-gateway` namespace:

```
bash theme={null} kubectl get gateway -n nginx-gateway
```

Question 12 – Update a Helm Chart Deployment

A coworker deployed an NGINX Helm chart called **kk-dash-mock-one** in the **kk-dash-ns** namespace. An update to the chart is available, and you need to update the Helm repository and upgrade the release to version **18.1.15**.

1. **List the current Helm releases** in the concerned namespace:

```
bash theme={null} helm list -n kk-ns
```

1. **Check your chart repositories:**

```
bash theme={null} helm repo list
```

1. **Update all repositories** to fetch the latest chart versions:

```
bash theme={null} helm repo update
```

1. **Search the repository for available versions of the NGINX chart:**

```
bash theme={null} helm search repo nginx --versions
```

1. **Upgrade the release** with the specified chart version:

```
bash theme={null} helm upgrade kk-mock1 kk-mock1/nginx --version  
18.1.15 -n kk-ns
```

1. **Verify the upgrade** by listing the release again:

```
bash theme={null} helm list -n kk-ns
```

By following these detailed steps and verifying each component with the respective `kubectl` and `helm` commands, you will successfully complete all the tasks for Mock Exam 1.

Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

Mock Exam 2 Step by Step Solutions

This article provides step-by-step solutions for Kubernetes cluster configuration and troubleshooting across various scenarios.

This article provides detailed step-by-step solutions for Mock Exam Two. Each question covers a key aspect of Kubernetes cluster configuration—from creating storage classes and deployments to configuring ingress, RBAC, network policies, HPA setups, and troubleshooting common issues. Follow the solutions below to configure your clusters and gain hands-on experience with Kubernetes.

Question 1 – Creating a Default Local StorageClass

In this question, you will create a StorageClass named **local-sc** on Cluster One's control plane. This StorageClass must be set as the default with the following criteria:

1. SSH into your control plane:

```
``plaintext theme={null} ssh cluster1-controlplane Welcome to Ubuntu 22.04.5 LTS (GNU/Linux 5.15.0-1078-gcp x86_64) * Documentation: https://help.ubuntu.com/ * Management: https://landscape.canonical.com * Support: https://ubuntu.com/pro
```

This system has been minimized by removing packages and content that are not required on a system that users do not log into. To restore this content, you can run the 'unminimize' command. ``

1. Referencing the documentation, copy the example configuration and update it as follows:
2. Change the `name` to **local-sc**.

3. Update the annotation `storageclass.kubernetes.io/is-default-class` to `"true"`.
4. Set the `provisioner` to `kubernetes.io/no-provisioner`.
5. Remove any unnecessary fields (such as `reclaimPolicy` and `mountOptions`) not specified.
6. Retain only `allowVolumeExpansion` and `volumeBindingMode: WaitForFirstConsumer`.

The final YAML configuration should look like this:

```
``yaml theme={null} apiVersion: storage.k8s.io/v1 kind: StorageClass
metadata: name: local-sc annotations: storageclass.kubernetes.io/is-default-
class: "true" provisioner: kubernetes.io/no-provisioner allowVolumeExpansion:
true volumeBindingMode: WaitForFirstConsumer
```

After saving the file (e.g., as `question1.yaml`), apply the configuration:

```
``bash theme={null}
kubectl apply -f question1.yaml
```

You should see the following confirmation:

```
``plaintext theme={null} storageclass.storage.k8s.io/local-sc created
```

Question 2 - Deployment with App and Sidecar Containers for Logging

This question guides you through creating a deployment named **logging-deployment** in the

- * **app-container**: Uses the busybox image to continuously append log entries to `/var/log/app.log`
- * **log-agent**: Also based on busybox, this container tails the same log file.

Both containers share an `emptyDir` volume mounted at `/var/log/app`

Below is the complete YAML configuration (save as `question2.yaml`)

```
``yaml theme={null}
apiVersion: apps/v1
kind: Deployment
metadata:
  name: logging-deployment
  namespace: logging-ns
spec:
  replicas: 1
  selector:
    matchLabels:
      app: logger
  template:
    metadata:
      labels:
        app: logger
    spec:
      volumes:
        - name: log-volume
          emptyDir: {}
      containers:
        - name: app-container
          image: busybox
          command:
            - sh
            - -c
            - "while true; do echo 'Log entry' >> /var/log/app/app.log; sleep 5; done"
          volumeMounts:
            - name: log-volume
              mountPath: /var/log/app
        - name: log-agent
          image: busybox
          command:
            - tail
            - -f
            - /var/log/app/app.log
```

Apply the configuration:

```
``bash theme={null} kubectl apply -f question2.yaml
```

```
Next, verify that the pod is running and inspect its logs:
```

```
```bash theme={null}
kubectl get pod -n logging-ns
kubectl logs <pod-name> -c log-agent -n logging-ns
```

You should see repeated "Log entry" outputs confirming that the log file is being written and tailed properly.

---

## Question 3 – Creating an Ingress Resource to Route Traffic

---

Here, you will create an Ingress resource named **webapp-ingress** in the `ingress-ns` namespace. This ingress routes traffic to a service called **webapp-svc**, using the NGINX ingress controller with the following criteria:

- Hostname: `kodekloud-ingress.app`
- Path: `/` (with `PathType: Prefix`)
- Forward traffic to **webapp-svc** on port 80

Below is the YAML configuration (save as `question3.yaml`):

```
```yaml theme={null} apiVersion: networking.k8s.io/v1 kind: Ingress metadata:
name:      webapp-ingress      namespace:      ingress-ns      annotations:
nginx.ingress.kubernetes.io/rewrite-target: / spec: ingressClassName: nginx
rules: - host: kodekloud-ingress.app http: paths: - path: / pathType: Prefix
backend: service: name: webapp-svc port: number: 80
```

```
Apply the configuration:
```

```
```bash theme={null}
kubectl apply -f question3.yaml
```

Test the ingress setup by sending an HTTP request:

```
```bash theme={null} curl -s http://kodekloud-ingress.app/
```

You should see the default NGINX welcome page, confirming correct configuration.

Question 4 - Updating an Nginx Deployment with Rolling Updates

In this question, create a deployment named **nginx-deploy** using **nginx:1.16** and the

1. Generate an initial deployment YAML using a dry run:

```
```bash theme={null}
kubectl create deployment nginx-deploy --image=nginx:1.16 --dry-run=client -o yaml > que
```
```

The generated YAML will be similar to:

```
```yaml theme={null}
apiVersion: apps/v1
kind: Deployment
metadata:
 name: nginx-deploy
 labels:
 app: nginx-deploy
spec:
 replicas: 1
 selector:
 matchLabels:
 app: nginx-deploy
 template:
 metadata:
 labels:
 app: nginx-deploy
 spec:
 containers:
 - name: nginx
 image: nginx:1.16
 resources: {}
```
```

2. Apply the deployment:

```
```bash theme={null}
kubectl apply -f question4.yaml
```
```

3. Update the image to version 1.17 using a rolling update:

```
```bash theme={null}
kubectl set image deployment/nginx-deploy nginx=nginx:1.17
```
```

4. Verify the rollout history:

```
```bash theme={null}
kubectl rollout history deployment nginx-deploy
kubectl rollout history deployment nginx-deploy --revision=2
```
```

The output should indicate that revision 2 is using ****nginx:1.17****.

Question 5 – Creating a User via CSR and Configuring RBAC

This question has two parts:

Part A: Create a CertificateSigningRequest (CSR) for User "john"

1. The private key is located at ``/root/cka/john.key`` and the CSR file at ``/root/cka/john.csr``
2. The CSR object should use the signer ``kubernetes.io/kube-apiserver-client`` and specify the following usages:

Example CSR YAML (save as ``question5.yaml``):

```
```yaml theme={null}
apiVersion: certificates.k8s.io/v1
kind: CertificateSigningRequest
metadata:
 name: john-developer
spec:
 signerName: kubernetes.io/kube-apiserver-client
 request: <BASE64_ENCODED_CONTENT_OF_/root/cka/john.csr>
 usages:
 - digital signature
 - key encipherment
 - client auth
```

To obtain the Base64 encoded CSR, run:

```
```bash theme={null} cat /root/cka/john.csr | base64 | tr -d '\n'
```

Apply the CSR:

```
```bash theme={null}
kubectl apply -f question5.yaml
```

Approve the CSR:

```
```bash theme={null} kubectl certificate approve john-developer
```

Part B: Grant RBAC Permissions with Role and RoleBinding

Create a Role named **developer** in the `development` namespace to allow the following ver

Save the following YAML as `question5-rbac.yaml`:

```
``yaml theme={null}
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: developer
  namespace: development
rules:
- apiGroups: [""]
  resources: ["pods"]
  verbs: ["create", "list", "get", "update", "delete"]

---
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: john-developer-role-binding
  namespace: development
subjects:
- kind: User
  name: john
  apiGroup: rbac.authorization.k8s.io
roleRef:
  kind: Role
  name: developer
  apiGroup: rbac.authorization.k8s.io
```

Apply the RBAC configuration:

```
``bash theme={null} kubectl apply -f question5-rbac.yaml
```

Test the permissions by running:

```
``bash theme={null}
kubectl auth can-i create pods --as=john -n development
```

A response of **yes** confirms that the RBAC configuration is working.

Question 6 – Creating an Nginx Pod with an Internal Service and DNS Testing

In this exercise, you will deploy an Nginx pod named **nginx-resolver** and expose it internally using a ClusterIP service named **nginx-resolver-service**. Then you will verify DNS resolution using a BusyBox pod.

1. Create the Nginx pod:

```
bash theme={null} kubectl run nginx-resolver --image=nginx
```

1. Expose the pod internally:

```
bash theme={null} kubectl expose pod nginx-resolver --name=nginx-resolver-service --port=80 --target-port=80 --type=ClusterIP
```

1. Verify the service endpoints:

```
bash theme={null} kubectl get svc nginx-resolver-service kubectl get pod -o wide
```

1. Run a temporary BusyBox pod to perform an nslookup:

```
bash theme={null} kubectl run test-nslookup --image=busybox:1.28 --rm -it --restart=Never -- nslookup nginx-resolver-service
```

Optionally, redirect the DNS lookup results to a file (e.g., `/root/cka/nginx.svc`) if necessary. Remember that Kubernetes creates DNS entries for both pods and services.

Question 7 – Creating a Static Pod

Create a static pod named **nginx-critical** on Node One. This pod should automatically restart on failure and must reside in the `/etc/kubernetes/manifests` directory.

1. Generate a pod YAML definition via dry run:

```
bash theme={null} kubectl run nginx-critical --image=nginx --restart=Always --dry-run=client -o yaml > static.yaml
```

1. SSH into Node One:

```
bash theme={null} ssh cluster1-node01
```

1. Place the YAML file into `/etc/kubernetes/manifests/static.yaml` with contents similar to:

```
yaml theme={null} apiVersion: v1 kind: Pod metadata: name: nginx-critical labels: run: nginx-critical spec: containers: - name: nginx-critical image: nginx dnsPolicy: ClusterFirst restartPolicy: Always
```

The kubelet will automatically create the static pod. Verify its status:

```
``bash theme={null} kubectl get pod -o wide
```

```
***
```

Question 8 – Creating a Horizontal Pod Autoscaler (HPA)

Create an HPA for a deployment named **backend-deployment** in the **backend** namespace. The

Below is the example HPA YAML (save as `webapp-hpa.yaml`):

```
```yaml theme={null}
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
 name: backend-hpa
 namespace: backend
spec:
 scaleTargetRef:
 apiVersion: apps/v1
 kind: Deployment
 name: backend-deployment
 minReplicas: 3
 maxReplicas: 15
 metrics:
 - type: Resource
 resource:
 name: memory
 target:
 type: Utilization
 averageUtilization: 65
```

Apply the HPA configuration:

```
```bash theme={null} kubectl apply -f webapp-hpa.yaml
```

Verify the HPA details:

```
```bash theme={null}
kubectl describe hpa -n backend
```

---

## Question 9 – Troubleshooting a Non-Responsive API Server

---

If Cluster Two fails to respond to `kubectl` commands (e.g., with the error message "The connection to the server cluster2-controlplane:6443 was refused"), perform the following steps:

1. Run a command such as:

```
bash theme={null} kubectl get nodes
```

which may yield:



```
plaintext theme={null} The connection to the server cluster2-
controlplane:6443 was refused - did you specify the right host or
port?
```

1. Use `crictl pods` to check the running containers and notice that the API server container is missing.
2. Verify the kubelet status:

```
bash theme={null} sudo systemctl status kubelet
```

If it is inactive, start and enable the kubelet:

```
bash theme={null} sudo systemctl start kubelet sudo systemctl enable
kubelet
```

After starting the kubelet, the API server container should be recreated.  
Confirm by running:

```
``bash theme={null} kubectl get nodes
```

```

```

## ## Question 10 - Modifying the Web Gateway for HTTPS

Modify the existing web gateway in the `cka5673` namespace on Cluster One so that it handles HTTPS.

1. Verify that the secret exists:

```
```bash theme={null}
kubectl get secret -n cka5673
```
```

2. Retrieve the current gateway configuration:

```
```bash theme={null}
kubectl get gateway -n cka5673 -o yaml > question10.yaml
```
```

3. Edit the YAML file to update the listener section. Change to the following:

```
```yaml theme={null}
listeners:
- name: https
  port: 443
  protocol: HTTPS
  hostname: kodekloud.com
  tls:
    certificateRefs:
    - name: kodekloud-tls
```
```

4. Apply the updated configuration:

```
```bash theme={null}
kubectl apply -f question10.yaml
```
```

```

```

## ## Question 11 - Uninstalling a Vulnerable Helm Release

Identify and uninstall the Helm release that uses the vulnerable image `kodekloud/webapp-color:v1.0.0`.

1. List all Helm releases across namespaces:

```
```bash theme={null}
helm list -A
```
```

2. Search for the vulnerable image in each release's manifests:

```
```bash theme={null}
helm get manifest <release-name> -n <namespace> | grep -i kodekloud/webapp-color:v1.0.0
```
```

3. Once identified (for example, if the release is named `atlanta-page-app` in the `atlanta` namespace), uninstall the release:

```
```bash theme={null}
helm uninstall atlanta-page-apd -n atlanta-page-04
```
```

This action removes the vulnerable release from your cluster.

\*\*\*

### ## Question 12 – Applying a Network Policy

Implement a network policy that allows traffic from frontend applications (namespace 'frontend')

After reviewing the provided policies, **net-pol-3.yaml** is the correct choice. Its content is as follows:

```
```yaml theme={null}
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: net-policy-3
  namespace: backend
spec:
  podSelector: {}
  ingress:
  - from:
    - namespaceSelector:
        matchLabels:
          name: frontend
  ports:
  - protocol: TCP
    port: 80
```
```

Apply the policy without affecting any other existing policies:

```
bash theme={null} kubectl apply -f net-pol-3.yaml
```

---

## Question 13 – Troubleshooting a Failed Deployment Due to Resource Quota

On Cluster Three, if the **backend-api** deployment fails to scale to three replicas due to resource quota limitations, follow these troubleshooting steps:

1. Describe the deployment to see error events:

```
bash theme={null} kubectl describe deployment backend-api
```

Also check the ReplicaSets for error events indicating that pod creation is forbidden by resource quotas.

1. Describe the resource quota:

```
bash theme={null} kubectl describe resourcequota cpu-mem-quota
```

An example output might be:

```
``plaintext theme={null} Name: cpu-mem-quota Namespace: default Resource
Used Hard
```

---

```
requests.cpu 200m 300m requests.memory 256Mi 300Mi ``
```

If the new pod's request (e.g., 128Mi) exceeds the quota, adjustments are needed.

1. Update the deployment's resource requests to ensure the total for three replicas remains within the limits. For instance, reduce the memory request from 128Mi to 100Mi:

```
yaml theme={null} resources: requests: cpu: 100m memory: 100Mi
limits: cpu: 150m memory: 150Mi
```

1. Update the deployment using editing or applying a modified YAML, and if necessary, delete the problematic ReplicaSet to trigger a new rollout:

```
bash theme={null} kubectl edit deployment backend-api kubectl delete
rs <offending-replicaset>
```

After the changes, all three pods should start and run successfully.

---

## Question 14 – Deploying Calico CNI with a Custom CIDR

---

On Cluster Four, deploy Calico as your CNI provider. Use the official Calico custom-resources YAML from GitHub and modify the CIDR to 172.17.0.0/16.

1. Download the custom-resources file:

```
bash theme={null} curl -O https://raw.githubusercontent.com/
projectcalico/calico/v3.29.2/manifests/custom-resources.yaml
```

1. Edit the downloaded YAML file. Locate the Calico IP pool definition and update the cidr value as shown below:

```
yaml theme={null} apiVersion: operator.tigera.io/v1 kind:
Installation metadata: name: default spec: calicoNetwork: ipPools: -
name: default-ip4-ipool blockSize: 26 cidr: 172.17.0.0/16
encapsulation: VXLANCrossSubnet natOutgoing: Enabled nodeSelector:
all
```

1. Apply the modified configuration:

```
bash theme={null} kubectl apply -f custom-resources.yaml
```

1. To verify that Calico is operating correctly and pod-to-pod communication works, deploy a test pod (such as an Nginx pod) and use a BusyBox or net-tools container for connectivity tests:

```
bash theme={null} kubectl run web-app --image=nginx kubectl run test
--rm -it --image=jrcs/net-tools --restart=Never -- sh -c "curl -s
<web-app-pod-IP>"
```

If the connectivity test is successful, Calico is configured and running correctly.

---

This article provides a comprehensive walkthrough of essential Kubernetes configurations and troubleshooting steps. By following these step-by-step solutions, you can better understand Kubernetes components and improve your operational skills.

Happy learning and good luck on your Kubernetes journey!

## Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/lms.txt> Use this file to discover all available pages before exploring further.

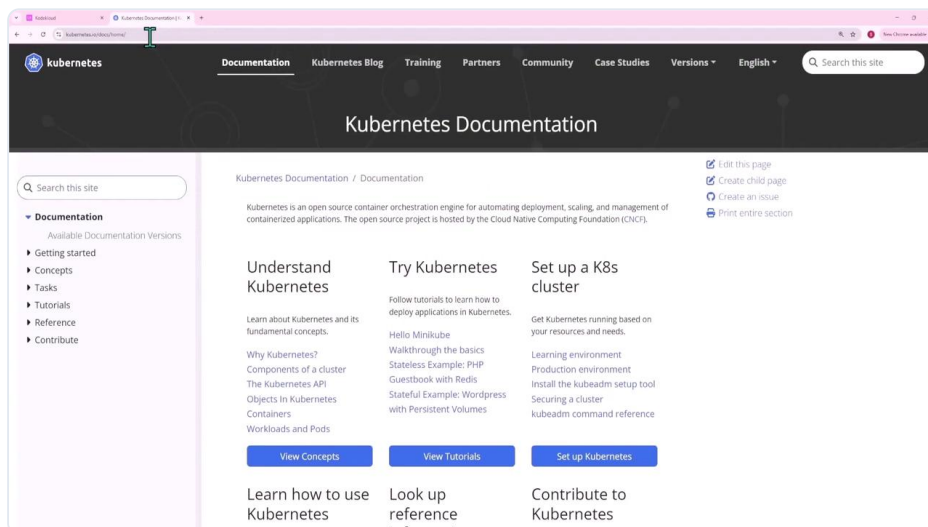
# Mock Exam 3 Step by Step Solutions

This lesson provides detailed solutions for each question in Mock Exam Three, focusing on specific Kubernetes tasks with clear instructions and code examples.

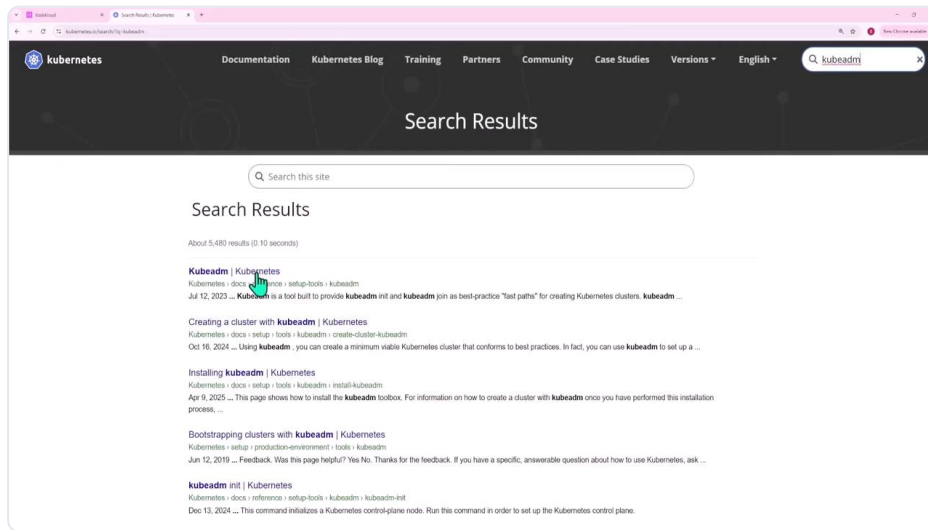
This lesson presents detailed solutions for each question in Mock Exam Three. Each solution focuses on a specific Kubernetes task and provides clear instructions, configuration code blocks, and diagram references. All image links and descriptions remain exactly as provided.

## Question 1 – Adjusting Network Parameters for Kubernetes

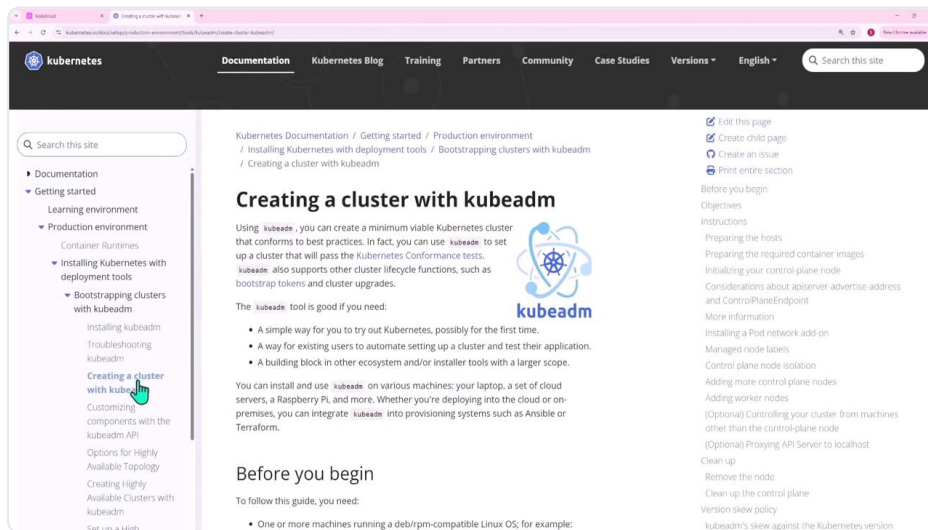
To deploy a Kubernetes cluster using kubeadm, you must enable IPv4 packet forwarding and ensure the settings persist across reboots. Refer to the kubeadm documentation for guidance when provisioning a new cluster.



Searching for “kubeadm” in the docs will help you locate the bootstrapping guide.



Navigate through the following path: Production Environment → Installing Kubernetes Deployment Tools → Bootstrapping a Cluster → Creating a Cluster with kubeadm.



The first step is to set up a container runtime and enable IPv4 packet forwarding using these commands:

```
``bash theme={null} cat <<EOF | sudo tee /etc/sysctl.d/k8s.conf
net.ipv4.ip_forward = 1 EOF
```

```
sudo sysctl --system
```

```
sysctl net.ipv4.ip_forward
```

For additional persistence, use this command if provided:

```
```bash theme={null}
# sysctl params required by setup, params persist across reboots
cat <<EOF | sudo tee /etc/sysctl.d/k8s.conf
net.ipv4.ip.forward = 1
EOF

# Apply sysctl params without reboot
sudo sysctl --system
```

Always copy the exact command names from the exam instructions to avoid errors.

This completes Question 1.

Question 2 – Creating a Service Account and Granting PVC Listing Permissions

In this question you will:

1. Create a service account named **pvviewer**.
2. Create a cluster role (**pvviewer-role**) that grants permission to list persistent volumes.
3. Bind the role to the service account with a cluster role binding (**pvviewer-role-binding**).
4. Launch a pod (**pvviewer**) using the Redis image in the default namespace.

Step 1: Create the Service Account

```
```bash theme={null} kubectl create serviceaccount pvviewer kubectl get sa
```

\*Expected output:\*

```
```plaintext theme={null}
NAME          SECRETS  AGE
default       0        6m55s
pvviewer      0        5s
```

Step 2: Create the Cluster Role

Create the role with the required permission:

```
```bash theme={null} kubectl create clusterrole pvviewer-role --
resource=persistentvolumes --verb=list
```



Verify with:

```
```bash theme={null}
kubectl describe clusterrole pvviewer-role
```

Expected output snippet:

```
```plaintext theme={null} Name: pvviewer-role Labels: Annotations:
PolicyRules: Resource Non-Resource URLs Resource Names Verbs
```

---

persistentvolumes [] [] [list]

### Step 3: Bind the Role to the Service Account

```
```bash theme={null}
kubectl create clusterrolebinding pvviewer-role-binding --clusterrole=pvviewer-role --servi
```

Step 4: Launch the Pod

Create a pod manifest (e.g., `question2.yaml`):

```
```yaml theme={null} apiVersion: v1 kind: Pod metadata: name: pvviewer spec:
serviceAccountName: pvviewer containers: - name: pvviewer image: redis
```

Apply the manifest:

```
```bash theme={null}
kubectl apply -f question2.yaml
```

Verify the pod and its service account:

```
```bash theme={null} kubectl get pod kubectl describe pod pvviewer
```

This completes Question 2.

<Frame>

![[The image shows a Kubernetes task description on the left, instructing to create a service account and a role, and then create a role binding that grants the role to the service account.]]

</Frame>

\*\*\*

### ## Question 3 - Creating a Storage Class

Create a storage class called **rancher-sc** with these settings:

- \* Provisioner: `rancher.io/local-path`
- \* Allow volume expansion: `true`
- \* Volume binding mode: `WaitForFirstConsumer`

Example manifest (`question3.yaml`):

```
```yaml theme={null}
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: rancher-sc
provisioner: rancher.io/local-path
allowVolumeExpansion: true
volumeBindingMode: WaitForFirstConsumer
```
```

Apply the storage class:

```
```bash theme={null} kubectl apply -f question3.yaml
```

This completes Question 3.

<Frame>

![[The image shows a search results page from the Kubernetes website, displaying results for 'storage class'.]]

</Frame>

Question 4 - Configuring a ConfigMap and Updating a Deployment

In the **cm-namespace**, perform these tasks:

1. Create a ConfigMap **app-config** containing key-value pairs such as `ENV=production` and `LOG_LEVEL=info`.
2. Update the existing deployment **cm-web-app** to source environment variables from the ConfigMap.

Step 1: Create the ConfigMap

```
```bash theme={null}
kubectl create configmap app-config -n cm-namespace --from-literal=ENV=production --from-literal=LOG_LEVEL=info
```

Verify with:

```
```bash theme={null} kubectl describe cm app-config -n cm-namespace
```

Step 2: Update the Deployment

Edit the deployment to include the ConfigMap:

```
```bash theme={null}
kubectl edit deployment cm-webapp -n cm-namespace
```

Add the following under the container section:

```
```yaml theme={null} envFrom: - configMapRef: name: app-config
```

After saving, verify that new pods include the environment variable from **app-config**.

This completes Question 4.

<Frame>

! [The image shows a Kubernetes task interface with instructions to create a ConfigMap and

</Frame>

Question 5 - Configuring Priority Classes and Pod Priority

For this task, you need to:

1. Create a PriorityClass **low-priority** with a value of 50,000.
2. Modify the existing pod **lp-pod** (in the **low-priority** namespace) to reference the
3. Recreate the pod so that it picks up the new priority without manually setting a numeric

Step 1: Create the PriorityClass

Create a manifest (e.g., `question5.yaml`):

```
```yaml theme={null}
apiVersion: scheduling.k8s.io/v1
kind: PriorityClass
metadata:
 name: low-priority
value: 50000
globalDefault: false
description: "This is a low priority class"
```

Apply it:

```
```bash theme={null} kubectl apply -f question5.yaml
```

Step 2: Update the Pod Manifest

Create or edit the pod manifest (e.g., `question5-pod.yaml`) to include only the `PriorityClass`

```
``yaml theme={null}
apiVersion: v1
kind: Pod
metadata:
  name: lp-pod
  namespace: low-priority
  labels:
    run: lp-pod
spec:
  priorityClassName: low-priority
  containers:
    - name: lp-pod
      image: nginx
      imagePullPolicy: Always
      resources: {}
      terminationMessagePath: /dev/termination-log
      terminationMessagePolicy: File
  dnsPolicy: ClusterFirst
  restartPolicy: Always
```

Do not include a numeric `priority` field.

Replace the pod if needed:

```
``bash theme={null} kubectl replace -f question5-pod.yaml --force
```

If an error appears about a numeric priority, remove any "priority: 0" specification and apply again:

```
kubectl apply -f question5-pod.yaml
```

Finally, verify the pod:

```
``bash theme={null}
kubectl get pod -n low-priority
```

This completes Question 5.

Question 6 – Fixing Incoming Connection Issues with a Network Policy

A pod (**np-test-1**) and its service (**np-test-service**) are not receiving incoming traffic on port 80. Create a NetworkPolicy named **test-network-policy** to allow TCP traffic on port 80.

First, confirm the pod's labels:

```
``bash theme={null} kubectl get pod --show-labels
```

Then, create a manifest (e.g., `question6.yaml`):

```
``yaml theme={null}
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: test-network-policy
  namespace: default
spec:
  podSelector:
    matchLabels:
      run: np-test-1
  policyTypes:
  - Ingress
  ingress:
  - ports:
    - protocol: TCP
      port: 80
```

Apply the policy:

```
``bash theme={null} kubectl apply -f question6.yaml
```

This policy permits incoming TCP traffic on port 80 for pods labeled `run=np-test-1`.

Question 7 – Tainting a Node and Creating Pods with Tolerations

In this question, you will:

1. Taint a worker node (**node01**) with `env_type=production:NoSchedule`.
2. Create a pod (**dev-redis**) without tolerations so it avoids node01.
3. Create another pod (**prod-redis**) with a toleration to allow scheduling on node01.

Step 1: Taint the Node

```
``bash theme={null}
kubectl taint node node01 env_type=production:NoSchedule
```

Verify the taint:

```
```bash theme={null} kubectl describe node node01 | grep -i taint
```

```
Step 2: Create the Non-Tolerant Pod
```

Using an imperative command:

```
```bash theme={null}
kubectl run dev-redis --image=redis:alpine
```

Step 3: Create the Tolerant Pod

Create a manifest (e.g., `question7.yaml`):

```
```yaml theme={null} apiVersion: v1 kind: Pod metadata: name: prod-redis
spec: containers: - name: prod-redis image: redis:alpine tolerations: - key:
"env_type" operator: "Equal" value: "production" effect: "NoSchedule"
```

Apply it:

```
```bash theme={null}
kubectl apply -f question7.yaml
```

Finally, check that **prod-redis** is scheduled on **node01** while **dev-redis** is not:

```
```bash theme={null} kubectl get pod -o wide
```

```

```

```
Question 8 - Binding a PVC to a PV by Matching Access Modes
```

A PersistentVolumeClaim (\*\*app-pvc\*\*) in the \*\*storage-ns\*\* namespace is not binding with t

After modifying the PVC manifest, remove the old PVC and apply the corrected file:

```
```bash theme={null}
kubectl delete pvc app-pvc -n storage-ns
kubectl apply -f <updated-pvc-manifest.yaml>
```

Verify the binding:

```
```bash theme={null} kubectl get pvc -n storage-ns
```

The PVC should now be **Bound** to the PV.

```

```

```
Question 9 - Troubleshooting a Faulty Kubeconfig File
```

The kubeconfig file **super.kubeconfig** (located at `~/root/CKA/super.kubeconfig`) is retur

```
```text theme={null}
https://controlplane:9999
```

Since the kube-apiserver listens on port **6443**, update the kubeconfig file as follows:

```
``yaml theme={null} clusters: - cluster: certificate-authority-data: server: https://
controlplane:6443 name: kubernetes
```

After saving the changes, test the connection:

```
``bash theme={null}
kubectl get node --kubeconfig=/root/CKA/super.kubeconfig
```

The connection should now work without errors.

Question 10 – Scaling a Deployment

The **nginx-deploy** deployment currently has 1 replica. To scale it to 3 replicas:

1. Check the current status:

```
bash theme={null} kubectl get deployment nginx-deploy
```

1. Scale the deployment:

```
bash theme={null} kubectl scale deployment nginx-deploy --
replicas=3
```

1. Verify the change:

```
bash theme={null} kubectl get deployment nginx-deploy
```

If the deployment still shows one available replica, review the deployment events:

```
``bash theme={null} kubectl describe deployment nginx-deploy
```

Troubleshoot any issues such as ReplicaSet misconfigurations or control plane component errors.

This completes Question 10.

Question 11 – Creating a Horizontal Pod Autoscaler (HPA) with Custom Metric

For the **api-deployment** in the **api** namespace, create an HPA that scales based on a custom metric.

Create a manifest (e.g., `question11.yaml`):

```
```yaml theme={null}
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
 name: api-hpa
 namespace: api
spec:
 scaleTargetRef:
 apiVersion: apps/v1
 kind: Deployment
 name: api-deployment
 minReplicas: 1
 maxReplicas: 20
 metrics:
 - type: Pods
 pods:
 metric:
 name: requests_per_second
 target:
 type: AverageValue
 averageValue: "1000"
```
```

Apply the HPA:

```
```bash theme={null} kubectl apply -f question11.yaml
```

Verify its configuration:

```
```bash theme={null}
kubectl describe hpa -n api
```

This completes Question 11.

Question 12 – Configuring an HTTPRoute to Split Traffic

To distribute incoming web traffic, configure an HTTP route to split between **web-service** (80%) and **web-service-v2** (20%). The associated web gateway and services already exist.

Create an HTTPRoute manifest (e.g., `question12.yaml`):

```
```yaml theme={null} apiVersion: gateway.networking.k8s.io/v1 kind:
HTTPRoute metadata: name: web-route namespace: default spec: parentRefs:
- name: web-gateway namespace: default rules: - matches: - path: type:
PathPrefix value: / backendRefs: - name: web-service port: 80 weight: 80 -
name: web-service-v2 port: 80 weight: 20
```

Apply the route:

```
```bash theme={null}
kubectl apply -f question12.yaml
```

This successfully routes 80% of traffic to **web-service** and 20% to **web-service-v2**.

Question 13 – Upgrading an Application Using Helm

You need to upgrade an application using a Helm chart from the directory `/root/new-version`. Follow these steps:

1. Validate the Chart:

```
bash theme={null} helm lint /root/new-version
```

Expected message:

```
plaintext theme={null} ==> Linting /root/new-version [INFO]
Chart.yaml: icon is recommended 1 chart(s) linted, 0 chart(s)
failed
```

1. Install the Chart:

Use an auto-generated name:

```
bash theme={null} helm install --generate-name /root/new-version
```

List the releases:

```
bash theme={null} helm list
```

1. Uninstall the Old Version:

Replace `<old-release-name>` with the actual release name:

```
bash theme={null} helm uninstall <old-release-name>
```

Verify the installation:

```
```bash theme={null} helm list
```

```
This completes Question 13.
```

```

```

```
Question 14 - Outputting the Pod CIDR Network
```

```
To determine the pod CIDR network of the cluster and save it to `/root/pod-cidr.txt`, ext
```

```
```bash theme={null}
```

```
kubectl get node -o jsonpath='{.items[0].spec.podCIDR}' > /root/pod-cidr.txt
```

Verify the file content:

```
```bash theme={null} cat /root/pod-cidr.txt
```

```
Expected output (example):
```

```
```plaintext theme={null}
```

```
172.17.0.0/24
```

This completes Question 14 and wraps up the mock exam solutions.

End of Lesson.

Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

Whats Next

This course offers comprehensive mock exams for realistic CKA exam practice in a simulated environment.

Hello everyone, and welcome to the Ultimate CKA Mock Exam Series presented by Vijen Palazi from KodeKloud.

Before diving in, please ensure that you have completed all the prerequisite materials, including multiple mock exams and hands-on labs. If your CKA exam is approaching soon, it's essential to review all the background content first.

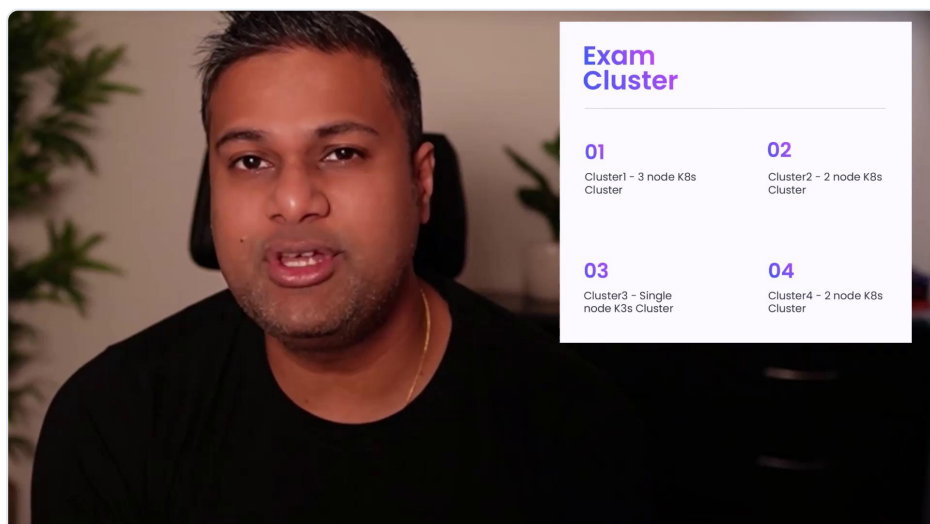
This course is designed as a series of comprehensive and challenging mock exams, offering you realistic, hands-on practice in a simulated exam environment. Unlike our regular labs, each mock exam uniquely mimics the actual test conditions.

The CKA exam assesses your practical knowledge in five key areas:

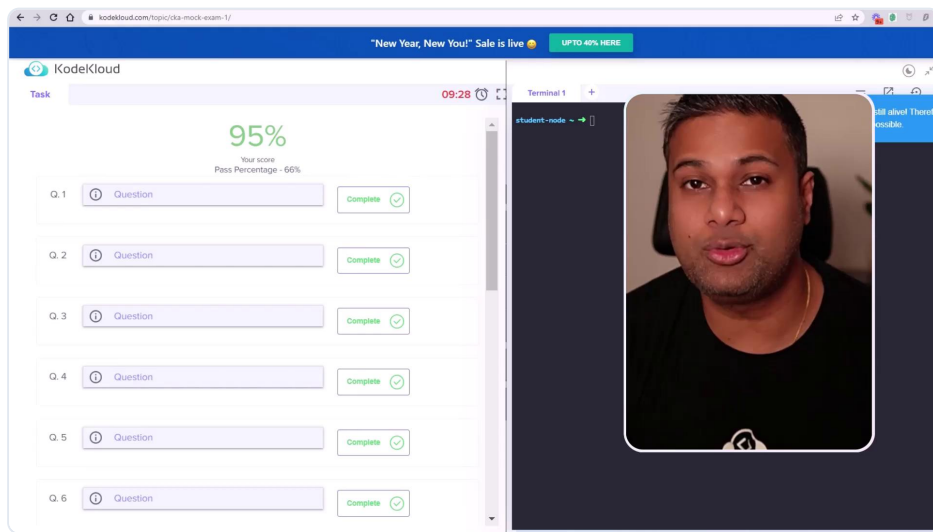
- Architecture, Installation, and Maintenance: 25%
- Workload Scheduling: 15%
- Service Networking: 20%
- Storage: 10%
- Troubleshooting: 30%



Each mock exam respects the weight of these areas, ensuring you receive a realistic testing experience. The series uses four Kubernetes clusters, with some clusters dedicated to specific knowledge areas. By default, you will be logged into the student node, giving you access to all other clusters and allowing SSH access to individual nodes.



This introduction marks the beginning of your journey through the Ultimate CKA Mock Exam Series.



Let's dive into one of these tests.

Starting the Mock Exam

Click on a lab corresponding to one of the mock exams (for example, "CKA Mock Exam One"). The lab should load within 30 seconds—often in just one to two minutes.

Once loaded, you will see that the exam consists of 20 questions. The first question is from the Architecture, Installation, and Maintenance category, representing 25% of the exam. This question requires you to set the context to cluster three.

Before proceeding, verify all the configured clusters. Switch to cluster three with the following command:

```
``bash theme={null} kubectl config use-context cluster3
```

You are now logged into the student node. To view all configured clusters, run:

```
``bash theme={null}
kubectl config get-clusters
```

By default, without an explicitly set context, you connect to cluster one, which contains two worker nodes (node01 and node02). For this task, you must switch to cluster three. On the student node, execute:

```
``bash theme={null} kubectl config use-context cluster3 kubectl config get-clusters kubectl config get-contexts kubectl get nodes
```

```
The student node serves as your client login node. If you open a new terminal session, it will
When your context is correctly set to cluster three, executing:

```bash theme={null}
kubectl get nodes
```

will display a single-node cluster consisting of the control plane (e.g., "cluster3-controlplane") running Kubernetes version 1.24.1. It is highly recommended to set the appropriate context for each question so you interact with the correct cluster.

For example, verify the nodes in cluster three with:

```
```bash theme={null} kubectl config use-context cluster3 kubectl get nodes
```

This command should return:

```
```bash theme={null}
NAME STATUS ROLES AGE VERSION
cluster3-controlplane Ready control-plane,master 18m v1.24.1+k3s1
```

## Working Through a Question

Let's review a sample question. You may need to decode an existing secret, named "beta-sec-CK14-arc", which is created in a separate namespace. First, ensure the namespace exists by listing the secrets in that namespace:

```
```bash theme={null} kubectl get secret -n beta-ns-cka14-arch
```

Then, retrieve the secret in YAML format to inspect the data:

```
```bash theme={null}
kubectl get secrets -n beta-ns-cka14-arch -o yaml
```

The secret data is stored under the "data" section. To decode the secret from a base64-encoded string, run:

```
```bash theme={null} echo 'VGHpncpyB0aG9lUGc2Vjcjmv0IQ=' | base64 -d
```

```
The decoded output should be: "This is the secret." You can redirect the decoded output directly to a file.

```bash theme={null}
kubectl config use-context cluster3
kubectl get secrets -n beta-ns-cka14-arch -o yaml

echo 'VGHpncpyB0aG9lUGc2Vjcjmv0IQ=' | base64 -d > /opt/beta-sec-cka14-arch
```

Scroll through the exam interface to view the remaining questions. The first question is worth eight points from the Architecture, Installation, and

Maintenance section. The second question from the same section can be attempted with:

```
``bash theme={null} kubectl config use-context cluster3 kubectl get secrets -n
beta-ns-cka14-arch kubectl get secrets -n beta-ns-cka14-arch -o yaml echo
'VGhpcyBpc28sIHlvdSBhcmUgY29tbWl0dGVklQ==' | base64 -d echo
'VGhpcyBpc28sIHlvdSBhcmUgY29tbWl0dGVklQ==' | base64 -d > /opt/beta-
sec-cka14-arch
```

Remember, you must compile all solutions before the allotted time expires. If time runs out

<Callout icon="lightbulb" color="#1CB2FE">

Always double-check that you have switched the context appropriately using 'kubectl confi  
</Callout>

You might also encounter additional commands to review logs and secrets, such as:

```
``bash theme={null}
kubectl config use-context cluster1
kubectl logs -f color-app-cka13-arch
kubectl get secrets -n beta-ns-cka14-arch
kubectl get secrets -n beta-ns-cka14-arch -o yaml
echo 'VhgpcyBpyB0aGluZyBQcmI0b3JpdHk=' | base64 -d > /opt/beta-cka14-arch
```

At any time during the exam, you can click the "End Exam" button to check your score. In this demonstration, only the first question is attempted, with all other questions marked incorrect by default.

For example, reviewing the secret on the student node might show output like:

```
``bash theme={null} student-node → kubectl get secrets -n beta-ns-cka14-
arch NAME TYPE DATA AGE beta-sec-cka14-arch Opaque 1 2m19s
```

```
student-node → kubectl get secrets -n beta-ns-cka14-arch -o yaml apiVersion:
v1 items: - apiVersion: v1 data: secret:
VGhpcyBpc8yB0aGluZyBQcmI0b3JpdHk= kind: Secret metadata: annotations:
kubectl.kubernetes.io/last-applied-configuration: | {"apiVersion":"v1","data":
{"secret":"VGhpcyBpc8yB0aGluZyBQcmI0b3JpdHk="},"kind":"Secret","metadata":
{"name":"beta-sec-cka14-arch","namespace":"beta-ns-cka14-
arch"},"type":"Opaque"} creationTimestamp: "2023-02-02T17:08:47Z" name:
beta-sec-cka14-arch namespace: beta-ns-cka14-arch resourceVersion: "846"
uid: ef9c5d98-aec0-4d4d-99be-1cbb989b5e0 type: Opaque kind: List
metadata: resourceVersion: "" selfLink: ""
```

```
student-node → echo VGhpcyBpc8yB0aGluZyBQcmI0b3JpdHk= | base64 -d
This is the secret!
```

Additional exam commands might include:

```
```bash theme={null}
kubectl config use-context cluster1
kubectl get secrets -n beta-ns-cka14-arch -o yaml
curl http://cluster1-node01:30080
echo 'VGhpcyBpcyBhIHNLy3JldCE=' | base64 -d
echo 'VGhpcyBpcyB0aGUgc2VjcmV0IQ==' | base64 -d > /opt/beta-sec-cka14-arch
```

These steps ensure that you are working within the correct context and that your solutions are properly validated.

After completing your attempt, click the "End Exam" button to trigger automatic exam validation.

For reference, a sample terminal session may look like:

```
bash theme={null} student-node ~ → kubectl --context cluster1 create
student-node ~ → kubectl --context cluster1 create student-node ~ →
kubectl --context cluster1 create student-node ~ → kubectl --context
cluster1 auth yes
```

Always verify your solutions and ensure you are working on the correct cluster context to avoid misconfigurations.

I hope you find this article useful. Best of luck with your CKA preparation!

Thank you.

Section: Install "Kubernetes the kubeadm way"

Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

Demo Deployment with Kubeadm

This guide explains how to bootstrap a Kubernetes cluster using kubeadm with a master and two worker nodes.

In this guide, we'll walk through bootstrapping a Kubernetes cluster using kubeadm. The setup involves three virtual machines (VMs): one control plane (master) node and two worker nodes. We will review the VM network configurations, install the container runtime and Kubernetes components, initialize the control plane, deploy a pod network add-on, and finally join the worker nodes to complete the cluster.

1. VM Overview and Network Interfaces

Before you start, ensure that all required VMs are created. The cluster consists of one master and two worker nodes. Verify the network interfaces on each node by executing the `ip add` command.

Master Node Network Configuration

Run the following command on the master node:

```
``bash theme={null} vagrant@kubemaster:~$ ip add 1: lo: mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000 link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00 inet 127.0.0.1/8 scope host lo valid_lft forever preferred_lft forever inet6 ::1/128 scope host valid_lft forever
```

```

preferred_lft forever 2: enp0s3: mtu 1500 qdisc fq_codel state UP group
default qlen 1000 link/ether 02:95:21:8a:38:bd brd ff:ff:ff:ff:ff:ff inet 10.0.2.15/24
metric 100 brd 10.0.2.255 scope global dynamic enp0s3 valid_lft forever
preferred_lft 51sec inet6 fe80::95:21ff:fe8a:38bd/64 scope link valid_lft forever
preferred_lft forever 3: enp0s8: mtu 1500 qdisc fq_codel state UP group
default qlen 1000 link/ether 02:42:fd:69:82:cd brd ff:ff:ff:ff:ff:ff inet
192.168.44.2/24 brd 192.168.44.255 scope global enp0s8 valid_lft forever
preferred_lft forever inet6 fe80::42:fdff:fe69:82cd/64 scope link valid_lft forever
preferred_lft forever

```

vagrant@kubemaster:~\$ ls vagrant@kubemaster:~\$

Worker Node One Network Configuration

On the first worker node, run:

```

```bash theme={null}
vagrant@kubenode01:~$ ip add
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
 link/loopback 00:00:00:00:00:00
 inet 127.0.0.1/8 scope host lo
 valid_lft forever preferred_lft forever
 inet6 ::1/128 scope host
 valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default
 link/ether 02:95:e1:8a:38:bd brd ff:ff:ff:ff:ff:ff
 inet 10.0.2.15/24 metric 100 brd 10.0.2.255 scope global dynamic enp0s3
 valid_lft 83015sec preferred_lft 0sec
 inet6 fe80::a00:27ff:fe47:a2e/64 scope link
 valid_lft forever preferred_lft forever
3: enp0s8: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default
 link/ether 02:95:e1:8a:38:bd brd ff:ff:ff:ff:ff:ff
 inet 192.168.56.101/24 metric 1024 scope global enp0s8
 valid_lft forever preferred_lft forever

vagrant@kubenode01:~$

```

### Worker Node Two Network Configuration

On the second worker node, verify the network configuration:

```

```bash theme={null} vagrant@kubenode02:~$ ip add
1: lo: mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000 link/loopback
00:00:00:00:00:00 inet 127.0.0.1/8 scope host lo valid_lft forever preferred_lft
forever inet6 ::1/128 scope link valid_lft forever preferred_lft forever 2: enp03:
mtu 1500 qdisc fq_codel state UP group default qlen 1000 link/ether
02:95:e1:8a:38:b0 brd ff:ff:ff:ff:ff:ff inet 10.12.0.2/24 brd 10.12.0.255 scope
global dynamic enp0s3 valid_lft 83155sec inet6 fe80::f50:e:38b:d6c5/64 scope
link valid_lft forever preferred_lft forever 3: enp08: mtu 1500 qdisc fq_codel
state UP group default qlen 1000 link/ether 02:80:2c:50:5d:f0 brd ff:ff:ff:ff:ff:ff

```

```
inet6 fe80::27f:f50:5c0:5c0/64 scope link valid_lft forever preferred_lft forever
vagrant@kubenode02:~$
```

```
The output confirms that each node has the correct network interfaces and associated IP addresses.

> ![The image shows a webpage from Kubernetes documentation detailing the installation of Kubernetes.]

***

## 2. Reviewing Prerequisites

Before initializing your Kubernetes cluster, verify the following prerequisites:

* A supported Linux distribution (e.g., Ubuntu).
* A minimum of 2 GB memory and at least two CPUs per node.
* Required kernel modules (BR, netfilter, overlay) are loaded.

<Callout icon="lightbulb" color="#1CB2FE">
  Ensure that system variables are correctly set (to 1) so the network interfaces function properly. For more details, refer to the official [Kubernetes Documentation](https://kubernetes.io/docs/setup/production-environment/container-runtimes/).
</Callout>

***

## 3. Installing the Container Runtime (ContainerD)

A container runtime is essential on every node. In this example, we will use ContainerD.

#### Step 1: Add the Kubernetes Repository and GPG Key

Execute these commands on all nodes:

```bash
theme={null}
sudo apt-get update
sudo apt-get install -y apt-transport-https ca-certificates curl gnupg
Create the keyrings directory if it does not exist
sudo mkdir -p /etc/apt/keyrings
curl -fsSL https://pkgs.k8s.io/core/stable/v1.31/deb/Release.key | sudo gpg --dearmor -o /etc/apt/keyrings/kubernetes.gpg
echo "deb [signed-by=/etc/apt/keyrings/kubernetes.gpg] https://pkgs.k8s.io/core/stable/v1.31/deb/" | sudo tee /etc/apt/sources.list.d/kubernetes.list
sudo apt-get update
sudo apt-get install -y kubelet kubect
```

## Step 2: Install ContainerD

Run these commands on each node to install ContainerD:

```
```bash
theme={null}
sudo apt-get update
sudo apt-get install -y kubelet
kubeadm kubect
```

Now, install ContainerD:

```
```bash theme={null}
sudo apt-get update
sudo apt-get install -y containerd
```

Verify the installation:

```
```bash theme={null} sudo systemctl status containerd
```

Step 3: Configure ContainerD for the systemd Cgroup Driver

It is critical that both ContainerD and kubelet use the same cgroup driver. Since systemd

```
```bash theme={null}
sudo mkdir -p /etc/containerd
containerd config default | sed 's/SystemdCgroup = false/SystemdCgroup = true/' | sudo tee
sudo systemctl restart containerd
```

For further details on cgroup drivers, please review the [container runtime documentation](#).

The screenshot displays the Kubernetes documentation website. The top navigation bar includes links for Documentation, Kubernetes Blog, Training, Partners, Community, Case Studies, Versions, and English. A search bar is located on the right. The left sidebar contains a search bar and a list of navigation links: Documentation, Getting started, Learning environment, Production environment, Container Runtimes, Installing Kubernetes with deployment tools, Turning Cloud Solutions, Best practices, Concepts, Tasks, Tutorials, Reference, and Contribute. The main content area is titled 'Configuring the container runtime cgroup driver'. It includes a 'Note' about kubelet's default cgroup driver, a 'Caution' about changing the cgroup driver, and a section on 'Migrating to the systemd driver in kubeadm managed clusters'. Below this, there is a section on 'CRI version support'. The bottom part of the screenshot shows the 'Before you begin' section, which includes 'Configuring the container runtime cgroup driver' and 'Configuring the kubelet cgroup driver'. A minimal example of configuring the field is provided at the bottom.

```
kubelet.conf.yaml
kind: KubeletConfiguration
apiVersion: kubelet.k8s.io/v1beta2
kubeletxruntime: v1.21.0

kind: KubeletConfiguration
apiVersion: kubelet.k8s.io/v1beta2
cgroupDriver: systemd
```

## 4. Installing kubeadm, kubelet, and kubectl

After configuring the container runtime, install the Kubernetes components. Holding these packages prevents unintentional upgrades:

```
```bash theme={null} sudo apt-get update sudo apt-get install -y kubelet
kubeadm kubectl sudo apt-mark hold kubelet kubeadm kubectl sudo systemctl
enable --now kubelet
```

The components serve the following purposes:

```
* **kubeadm:** Bootstraps and manages the cluster initialization.
* **kubelet:** Manages pods and containers on every node.
* **kubectl:** Provides the command-line interface to interact with the cluster.

***
```

5. Initializing the Kubernetes Cluster

On the master node, initialize the control plane with `kubeadm init`, making sure to specify

```
* `--pod-network-cidr`: Sets the CIDR for pod networking (e.g., "10.244.0.0/16").
* `--apiserver-advertise-address`: Uses the master node's static IP
```

Before proceeding, verify the master node's IP address:

```
```bash theme={null}
ip addr
```

For instance, if your master node's static IP is 192.168.56.11, run:

```
```bash theme={null} sudo kubeadm init --apiserver-advertise-
address=192.168.56.11 --pod-network-cidr="10.244.0.0/16" --upload-certs
```

After initialization, an `admin.conf` file is created. Configure kubectl by copying this fi

```
```bash theme={null}
mkdir -p $HOME/.kube
sudo cp -i /etc/kubernetes/admin.conf $HOME/.kube/config
sudo chown $(id -u):$(id -g) $HOME/.kube/config

export KUBECONFIG=/etc/kubernetes/admin.conf
```

Confirm access to the cluster by checking the pods:

```
```bash theme={null} kubectl get pods -A
```

```

A lack of pods in the default namespace indicates that the cluster is successfully initialized.

***

## 6. Deploying a Pod Network Add-on

To enable inter-pod communication, deploy a pod network add-on. In this demo, we use Weave Net.

```bash theme={null}
kubectl apply -f [podnetwork].yaml

```

Replace `[podnetwork].yaml` with the URL or local file path to the Weave Net configuration file. This command deploys a DaemonSet that ensures the network add-on is applied across the control plane and later propagates to worker nodes.

Verify the network add-on by checking the pods:

```
```bash theme={null} kubectl get pods -A
```

```

Ensure the Weave Net pods are running on the control plane. Adjust network settings if necessary.

***

## 7. Joining Worker Nodes to the Cluster

Once the pod network is deployed, add your worker nodes to the cluster using the "kubeadm join" command.

For example, run the following command on each worker node:

```bash theme={null}
sudo kubeadm join 192.168.56.11:6443 --token ps4rl5.0ns9vwu9exjul8t \
 --discovery-token-ca-cert-hash sha256:fdb5c133b76f41d6d1f9ed72d09b7265de5e53a9156d7d48d

```

After joining, verify the nodes from the master node:

```
```bash theme={null} kubectl get nodes
```

```

All nodes (master and workers) should appear with a "Ready" status.

***

## 8. Verifying the Cluster

To ensure that your Kubernetes cluster is operational, deploy a test pod (for example, an nginx pod).

```bash theme={null}
kubectl run nginx --image=nginx
kubectl get pod

```

Once the test pod is in the Running state, your cluster setup is complete. You may remove the test pod when finished.

## Conclusion

---

This guide covered the following steps to bootstrap your Kubernetes cluster using kubeadm:

1. Reviewed VM network configurations.
2. Installed and configured ContainerD as the container runtime.
3. Installed Kubernetes components (kubeadm, kubelet, and kubect!).
4. Initialized the control plane with `kubeadm init` (including specifying pod network CIDR and the API server advertise address).
5. Deployed a pod network add-on (Weave Net).
6. Joined the worker nodes to the cluster.
7. Verified cluster functionality with a test pod deployment.

You have now successfully set up your Kubernetes cluster from scratch using kubeadm. Happy clustering!